

Layered Network Programming with SNode.C

Building Multi-Protocol Applications in Modern C++

Volker Christian

Contents

Preface	i
Who this book is for	i
What this book assumes	ii
What this book is not	ii
How to Read This Book	iii
Conventions Used in This Book	iv
Author and Framework Note	v
Acknowledgements	vi
I Getting Oriented	1
1 Why SNode.C Exists	2
1.1 A framework that teaches systems, not only APIs	2
1.2 The problem SNode.C addresses	2
1.3 What this book will and will not do	3
1.4 Where SNode.C fits	4
1.5 What this book is, and what it is not	4
1.6 Source version used by this book	4
1.7 Why “layered” matters here	4
1.8 In the spirit of node.js, but not node.js in C++	6
1.9 Why this matters for multi-protocol application development	6
1.10 The central pattern you will meet again and again	7
1.11 Why the first example is an echo pair	7
1.12 Closing perspective	8
2 Preparing Your Environment	9
2.1 Why the environment matters	9
2.2 Working areas, different purposes	9
2.3 What the environment must support	10
2.4 Compiler and CMake expectations	10
2.5 Required and optional packages	11
2.6 Cloning the framework	11
2.7 Use an out-of-tree build	12
2.8 Installing the framework	12
2.9 Build options worth knowing early	13
2.10 Finding built executables	13
2.11 Running server and client programs	13
2.12 Do not hide runtime output too early	14
2.13 Preparing a separate playground project	14
2.14 Components and public headers as installed architecture	14
2.15 Source tree, build tree, install tree	15
2.16 Closing perspective	16
3 Your First Working Program: The Echo Pair	17

3.1	From environment to first program	17
3.2	What the SNode.C repository contains	17
3.3	The four files	18
3.4	The three roles in the first example	18
3.5	The echo context header	19
3.6	Implementing the echo behavior	20
3.7	The server application	21
3.8	The client application	23
3.9	Building the simplified echo pair	24
3.10	Configuring and building the playground	26
3.11	Running the echo pair	26
3.12	Reading the output	26
3.13	Comparing the chapter version with the repository version	27
3.14	What changed compared with ordinary socket programming	27
3.15	Closing perspective	28
4	Reading the Codebase with Confidence	29
4.1	From first program to source-tree orientation	29
4.2	Use the build structure as the first map	29
4.3	Start from applications, then follow inward	30
4.4	Separate example code from framework code	31
4.5	Follow names as architecture	31
4.6	Learn to read type aliases	32
4.7	The recurring path from role to context	33
4.8	Reading core	33
4.9	Reading net	34
4.10	Reading higher protocol directories	34
4.11	Use CMake as a navigation tool	34
4.12	Do not confuse source paths, public includes, and installable components	35
4.13	Use search, but search for roles	36
4.14	Recognize variant-heavy code	36
4.15	Read headers before implementation files	36
4.16	How to read a server or client type	37
4.17	How to read a context class	37
4.18	How to read a factory	37
4.19	Avoid common reading mistakes	38
4.20	A compact reading workflow	38
4.21	Transition to the architecture chapters	39
II	The SNode.C Architecture	40
5	The Mental Model of SNode.C	41
5.1	From source-tree orientation to architectural thinking	41
5.2	The shortest useful description	41
5.3	The runtime model	42
5.4	The normal startup-to-protocol flow	44
5.5	Lifetimes: role, handle, connection, context	46
5.6	The layer model	47
5.7	Names are compressed architecture	48
5.8	The operational model	49
5.9	What stays stable across later chapters	50
5.10	A compact model to carry forward	51
6	Core Runtime and Event Processing	53
6.1	Why this chapter matters	53
6.2	The runtime picture in one diagram	53
6.3	A short source-level reading	55
6.4	The public runtime surface: core::SNodeC	55

6.5	The orchestrator behind the facade: <code>core::EventLoop</code>	56
6.6	Runtime state is coarse on purpose	57
6.7	Tick-by-tick execution and <code>TickStatus</code>	57
6.8	From handles to runtime-visible instances	58
6.9	From runtime intent to event delivery	58
6.10	<code>EventReceiver</code> and <code>Event</code> : the small but important abstraction pair	59
6.11	The multiplexer is the coordination backbone	60
6.12	Observed event sources	61
6.13	Descriptor event publishers: managing observed descriptor populations	61
6.14	Descriptor event receivers: behavior attached to observed descriptors	62
6.15	Timers are not bolted on	62
6.16	One tick, conceptually	63
6.17	What application authors should learn from this	64
6.18	Stable concepts versus implementation details	65
6.19	Closing perspective	65
7	Layers in Practice: Network, Transport, Connection, Application	67
7.1	Why this chapter comes here	67
7.2	The communication stack in four layers	67
7.3	Names are maps of architecture	68
7.4	The network layer: endpoint identity	70
7.5	The transport layer: communication form	71
7.6	The connection layer: managing the peer relationship	72
7.7	The application layer: protocol behavior	73
7.8	The build system as confirmation	75
7.9	One protocol, many lower carriers	76
7.10	Layers are real, but not walls	76
7.11	What belongs where?	77
7.12	Closing perspective	78
III	Networking Foundations in SNode.C	79
8	Socket Addresses and Address Semantics	80
8.1	From layers to endpoint identity	80
8.2	The shared <code>SocketAddress</code> pattern	80
8.3	Local and remote endpoint roles	81
8.4	Endpoint identity by family	81
8.5	Patterns that repeat across families	84
8.6	Address semantics and the layer model	86
8.7	Why IP families have richer resolution behavior	86
8.8	What belongs where?	86
8.9	Closing perspective	87
9	Servers, Clients, and Connections	88
9.1	Why these three belong in one chapter	88
9.2	From registered instance to concrete connection	88
9.3	Server and client instances as runtime-visible roles	90
9.4	Server instances	91
9.5	Client instances	92
9.6	Server and client symmetry, and where it ends	93
9.7	<code>SocketConnection</code> : the concrete peer relationship	94
9.8	Callback layers	97
9.9	Where the context factory fits	99
9.10	Putting the pieces together	100
9.11	What belongs where?	101
9.12	Closing perspective	101
10	IPv4 and IPv6 as the First Concrete Network Families	103

10.1	Why IPv4 and IPv6 come first	103
10.2	What stays shared across IPv4 and IPv6	103
10.3	What remains stable	106
10.4	What changes	107
10.5	From IPv4 to IPv6	109
10.6	Stable model, different family semantics	110
10.7	Public surface of the primary families	111
10.8	Closing perspective	112
11	Unix Domain Sockets	113
11.1	From host-plus-port to local path identity	113
11.2	Same SNode.C model, different endpoint identity	113
11.3	Path identity in <code>net::un::SocketAddress</code>	114
11.4	Server and client use with path-based endpoints	116
11.5	What remains stable	117
11.6	What changes operationally	119
11.7	Stream focus and a datagram note	120
11.8	Preparing the Bluetooth shift	120
11.9	Public surface of Unix-domain stream roles	120
11.10	Closing perspective	121
12	Bluetooth in SNode.C: RFCOMM and L2CAP	122
12.1	From path identity to Bluetooth endpoint identity	122
12.2	Two Bluetooth families, two service selectors	122
12.3	Bluetooth address classes	124
12.4	Server and client use with Bluetooth endpoints	125
12.5	Local and remote Bluetooth identity	127
12.6	Pairing before SNode.C communication	128
12.7	What remains stable	128
12.8	What changes operationally	129
12.9	Public surface of Bluetooth stream roles	130
12.10	Closing perspective	131
IV	From Raw Connections to Application Protocols	132
13	Writing SocketContext Classes Well	133
13.1	From lower-family choice to protocol behavior	133
13.2	What a <code>SocketContext</code> is	133
13.3	The context surface seen by protocol code	135
13.4	Lifecycle, input, signals, and errors	136
13.5	A compact worked context	137
13.6	Design habits for good context code	139
13.7	What not to put into a context	142
13.8	Logging in a context	142
13.9	The echo context as a minimal pattern	143
13.10	Keeping context code mentally testable	143
13.11	The factory as the next bridge	144
13.12	Closing perspective	144
14	Writing SocketContextFactory Classes Well	145
14.1	From protocol endpoint to construction boundary	145
14.2	What a <code>SocketContextFactory</code> is	145
14.3	The construction path from connection to context	147
14.4	One fresh context per creation	147
14.5	Construction responsibilities of a factory	148
14.6	Responsibilities that should stay out of the factory	150
14.7	Ownership and lifecycle	151
14.8	Passing shared application state	152

14.9	Factory design shapes	153
14.10	The strongest factory tests	155
14.11	The factory as the Chapter 15 bridge	155
14.12	Closing perspective	155
15	Building the Same Protocol over Different Lower Layers	157
15.1	From context and factory separation to lower-family transfer	157
15.2	The lower-family transfer model	157
15.3	What can remain stable	158
15.4	What changes with the lower family	160
15.5	Echo as the smallest transfer microscope	161
15.6	One protocol, several carriers	162
15.7	Designing for lower-family transfer	162
15.8	Preconfigured factories and endpoint roles	164
15.9	Same protocol shape does not mean same deployment	165
15.10	When reuse should stop	165
15.11	What remains stable and what changes	166
15.12	Configuration becomes visible here	166
15.13	The public surface changes with the lower layer	167
15.14	Closing perspective	167
V	Configuration and Operational Behavior	168
16	Configuration Philosophy in SNode.C	169
16.1	From lower-family transfer to configured communication roles	169
16.2	Configuration as part of the architecture	169
16.3	Three input paths, one configuration model	171
16.4	Precedence and the startup boundary	172
16.5	Named instances as configuration addresses	173
16.6	Sections as structural scopes	175
16.7	Configuration as discovery, persistence, and inspection	176
16.8	Parameterless <code>listen()</code> and <code>connect()</code>	177
16.9	CLI11 as implementation foundation	178
16.10	Configuration as architectural leverage	178
16.11	Closing perspective	179
17	Application and Instance Configuration in Detail	180
17.1	From configuration philosophy to configuration anatomy	180
17.2	The anatomy of the configuration hierarchy	180
17.3	Application configuration: the operational envelope	183
17.4	Instance configuration: the configured communication role	184
17.5	Section configuration: scoped responsibilities	185
17.6	Three views of the same model	188
17.7	Required values and progressive disclosure	190
17.8	Persistent and nonpersistent values	191
17.9	Generated and shown configuration	192
17.10	Configuration files as operational artifacts	192
17.11	Designing configuration for real applications	192
17.12	What remains stable	193
17.13	Closing perspective	194
18	Logging, Diagnostics, and Runtime Introspection	195
18.1	From configured roles to visible runtime behavior	195
18.2	Runtime visibility as a diagnostic map	195
18.3	The logging surface: ordinary logs, system-error logs, and verbose depth	197
18.4	Severity and diagnostic depth	199
18.5	Output modes in real deployments	199
18.6	Logging should follow responsibility boundaries	200

18.7	Visibility at lifecycle boundaries	202
18.8	Configuration as a diagnostic source	204
18.9	Connection metrics and identity	205
18.10	Runtime introspection is broader than logging	206
18.11	Too much logging is a failure mode	206
18.12	A good log line answers a question	206
18.13	A practical diagnostic workflow	207
18.14	Closing perspective	207
VI Secure and Robust Communication		209
19	TLS Across the Framework	210
19.1	From runtime visibility to secure connection handling	210
19.2	TLS as a connection-layer specialization	210
19.3	Legacy and TLS streams as neighboring connection variants	211
19.4	The TLS wrapper shape in code	212
19.5	What TLS adds to the connection layer	213
19.6	What can remain stable above TLS	214
19.7	The TLS connection object as the layer boundary	214
19.8	TLS configuration and the <code>tls</code> section	215
19.9	TLS adds work between connection creation and readiness	216
19.10	TLS also affects shutdown	217
19.11	TLS does not erase the lower family	217
19.12	Protocol contexts should stay TLS-independent when possible	217
19.13	TLS diagnostics through the Chapter 18 lens	218
19.14	A useful teaching path: legacy first, TLS second	219
19.15	A rule of thumb for TLS-capable applications	219
19.16	TLS capability and deployment responsibility	219
19.17	Closing perspective	220
20	Timeouts, Retries, and Failure Modes	221
20.1	Communication over time	221
20.2	The time-and-failure map	221
20.3	Where waiting is bounded	223
20.4	Retry and reconnect are not the same thing	225
20.5	Server and client symmetry and difference	228
20.6	Retry timing policy	228
20.7	<code>NO_RETRY</code> as retry-control information	230
20.8	Failure states carry runtime meaning	230
20.9	Flow controllers as role-level owners	231
20.10	Output pressure and bounded write-buffer policy	232
20.11	Protocol-level timeout use	232
20.12	Failure visibility	233
20.13	The architectural boundary for failure behavior	233
20.14	Closing perspective	234
VII Web Protocols and Web Applications		235
21	The HTTP Layer	236
21.1	From robust streams to HTTP messages	236
21.2	HTTP in the layered <code>SNode.C</code> model	236
21.3	Plain streams and HTTP messages side by side	237
21.4	Server-side HTTP: from bytes to ready requests	237
21.5	Client-side HTTP: from connection to requests and responses	239
21.6	What HTTP adds above the stream layer	240
21.7	Lower families and connection handling still matter	241
21.8	HTTP as a bridge to higher web protocols	242

21.9	What remains from the lower architecture	244
21.10	From HTTP support to web application structure	244
21.11	HTTP public surface: role headers and components	245
21.12	Closing perspective	245
22	The Express-Like Framework	246
22.1	From HTTP messages to application structure	246
22.2	The Express-like layer in the layered SNode.C model	246
22.3	HTTP messages and Express-like application flow side by side	247
22.4	What “Express-like” means here	247
22.5	WebApp, WebAppT, and the HTTP server below	248
22.6	Router as the application composition unit	249
22.7	Application callbacks and middleware callbacks	250
22.8	Dispatchers behind the API	251
22.9	Request and Response as web-application facades	252
22.10	Built-in middleware as reusable application behavior	253
22.11	Lower layers still matter	254
22.12	From Express-like routing to Server-Sent Events	254
22.13	Express public surface: WebApp header and carrier component	255
22.14	Closing perspective	255
23	Server-Sent Events and Real-Time HTTP	256
23.1	From request / response to event streams	256
23.2	SSE in the layered SNode.C web stack	257
23.3	Ordinary HTTP, SSE, and WebSocket side by side	257
23.4	SSE as long-lived HTTP	257
23.5	Client-side EventSource and server-side streaming endpoints	258
23.6	A compact server-side SSE endpoint	259
23.7	A compact EventSource client	260
23.8	The EventSource client abstraction	261
23.9	SSE starts as an HTTP request	263
23.10	Parsing the event stream	263
23.11	Retry and continuity	264
23.12	Express-like routes and SSE endpoints	265
23.13	Lower layers and diagnostics still matter	265
23.14	Real-time-style HTTP	266
23.15	EventSource public surface	266
23.16	Closing perspective	266
24	WebSocket and Protocol Upgrade	267
24.1	From event streams to upgraded bidirectional communication	267
24.2	WebSocket in the layered SNode.C web stack	267
24.3	HTTP, SSE, and WebSocket side by side	268
24.4	WebSocket after the HTTP upgrade boundary	269
24.5	WebSocket as the concrete upgraded protocol	269
24.6	SocketContextUpgrade as the boundary object	270
24.7	WebSocket messages and frames	271
24.8	Server-side and client-side upgrade responsibilities	272
24.9	The WebSocket module structure	273
24.10	Subprotocols as the next semantic layer	273
24.11	Lower layers and diagnostics still matter	278
24.12	From WebSocket carriers to MQTT semantics	278
24.13	WebSocket public surface	279
24.14	Closing perspective	279
VIII	IoT and Message-Oriented Systems	280
25	MQTT Support in SNode.C	281

25.1	From web protocols to message-oriented communication	281
25.2	MQTT in the layered SNode.C model	281
25.3	Native MQTT and MQTT over WebSocket side by side	282
25.4	MQTT as a protocol family	282
25.5	Mqtt as the protocol object	284
25.6	Native MQTT over stream connections	285
25.7	Server and client specialization	286
25.8	A compact MQTT client role	287
25.9	MQTT as a WebSocket subprotocol	288
25.10	MQTT public surface: protocol headers and components	288
25.11	Build/component note: JSON dependency is not MQTT identity	289
25.12	MQTT and IoT-facing systems	289
25.13	Closing perspective	290
26	MQTT over WebSocket	291
26.1	MQTT carried by the WebSocket upgrade stack	291
26.2	The composition in one model	292
26.3	Native MQTT and MQTT-over-WebSocket side by side	292
26.4	Why this carrier composition matters	293
26.5	The MQTT-over-WebSocket subprotocol type	293
26.6	Server-side and client-side aliases	296
26.7	Each layer keeps its responsibility	296
26.8	Native MQTT and MQTT-over-WebSocket as sibling compositions	297
26.9	Build artifacts mirror the composition	297
26.10	MQTT-over-WebSocket as component selection	297
26.11	Diagnostics across the composed stack	298
26.12	From one composed stack to multi-protocol systems	299
26.13	Native MQTT and MQTT-over-WebSocket public surfaces	300
26.14	Closing perspective	300
27	Designing IoT Systems with Multiple Protocols	301
27.1	From protocol composition to system boundaries	301
27.2	IoT systems are boundary systems	301
27.3	Recurring boundary roles in IoT systems	302
27.4	Choosing protocol families from boundary needs	303
27.5	A constellation of protocol stacks	304
27.6	Telemetry, control, observation, and administration are different conversations	305
27.7	MQTT as an integration spine, not the whole system	305
27.8	HTTP and Express as management and operator-facing surfaces	306
27.9	SSE and WebSocket as live observation and interaction surfaces	306
27.10	MQTT-over-WebSocket as a bridge boundary	307
27.11	Bluetooth at the device edge	307
27.12	Unix domain sockets for local control	308
27.13	One process or several cooperating applications	308
27.14	Configuration as a map of roles	308
27.15	Observability across role boundaries	309
27.16	Failure policy by role	309
27.17	Reusing meaning across different surfaces	310
27.18	Practical design recipe	310
27.19	Protocol diversity is not architectural chaos	311
27.20	Closing perspective	311
IX	Persistence and Full Systems	313
28	Database Support and Application State	314
28.1	From protocol boundaries to persistence boundaries	314
28.2	Persistence as an application-state boundary	315
28.3	What SNode.C currently supports	316

28.4	Runtime state and database state	317
28.5	<code>MariaDBClient</code> as the application-facing database object	318
28.6	A minimal MariaDB client example	319
28.7	Database work inside the event-driven runtime	321
28.8	Database work as commands and command sequences	322
28.9	Transactions as sequenced database work	325
28.10	Timers and database work	326
28.11	Persistence service design	326
28.12	Persistence, failure, and backpressure	327
28.13	Database state in multi-protocol IoT systems	328
28.14	Closing perspective	328
29	Learning from the Applications in <code>src/apps</code>	330
29.1	From framework pieces to application structure	330
29.2	<code>src/apps</code> as study material	330
29.3	Build targets as the first architectural reading layer	331
29.4	Read include blocks beside link lines	336
29.5	Entry points as assembly points	336
29.6	Application families and focused examples	338
29.7	Reading applications in <code>src/apps</code>	341
29.8	Closing perspective	344
30	From Applications to Systems	345
30.1	Why this chapter comes after applications	345
30.2	From applications to role constellations	346
30.3	Boundaries define the system	348
30.4	System operation	350
30.5	Stable protocol cores and domain code	352
30.6	Reading a <code>SNode.C</code> system	353
30.7	Closing perspective	354
31	MQTTSuite as a Reference Ecosystem	355
31.1	From system vocabulary to a concrete ecosystem	355
31.2	Ecosystem shape	355
31.3	Shared infrastructure where it belongs	358
31.4	MQTTBroker: broker role plus web administration	359
31.5	Integration and topology: MQTTIntegrator and MQTTBridge	360
31.6	Operational and persistence roles	362
31.7	System surfaces	363
31.8	Reading MQTTSuite architecturally	364
31.9	Closing perspective	365
X	Building, Porting, and Maintaining	366
32	CMake Components, Public Headers, and Linking Strategy	367
32.1	Why CMake starts Part X	367
32.2	The build structure as architecture	367
32.3	Component targets, public headers, and dependency surfaces	370
32.4	Core, network, and transport composition	377
32.5	Protocol and application-layer components	378
32.6	Optional features and generated configuration	380
32.7	Exported package targets and external consumers	381
32.8	Reading a <code>SNode.C</code> CMake target	384
32.9	Closing perspective	384
33	Deployment on Linux and OpenWrt	385
33.1	Why deployment belongs after CMake	385
33.2	Deployment as installed architecture	386

33.3	Packages and installed component surfaces	387
33.4	Runtime state and service operation	389
33.5	Deployment-specific resources	392
33.6	Embedded Linux and OpenWrt	393
33.7	Reading a deployment	395
33.8	Closing perspective	396
34	Testing, Debugging, and Benchmarking	397
34.1	Why testing follows deployment	397
34.2	Testing follows framework boundaries	397
34.3	Build-time confidence	399
34.4	Protocol-boundary confidence	402
34.5	Runtime confidence	405
34.6	Deployment confidence	406
34.7	Debugging and diagnostics	407
34.8	Benchmarking	409
34.9	Current practice and future strategy	410
34.10	Closing perspective	411
XI	Designing with SNode.C	412
35	Architectural Judgment: Choosing the Right Layer and Boundary	413
35.1	Why this chapter matters now	413
35.2	Good judgment begins by refusing category mistakes	414
35.3	Choosing the communication family	414
35.4	Choosing the protocol surface	414
35.5	Choosing native or composed protocol form	415
35.6	Choosing role boundaries	416
35.7	Choosing process and deployment boundaries	416
35.8	Choosing the implementation layer	416
35.9	Choosing code or configuration	417
35.10	Asking what the endpoint should know	417
35.11	Separating protocol meaning from carrier mechanics	417
35.12	Choosing the extensibility point	418
35.13	Placing failure policy	418
35.14	Choosing what must be operator-visible	419
35.15	Simplicity usually beats cleverness	419
35.16	The deeper rule: preserve meaning as long as possible	419
35.17	Common bad instincts this chapter tries to cure	420
35.18	Common misunderstandings	420
35.19	The architectural principle	421
35.20	Closing perspective	421
36	Extending the Framework Safely	422
36.1	Why this chapter follows architectural judgment	422
36.2	What safe extension means	422
36.3	Start application-local unless the boundary is reusable	423
36.4	Three extension levels	423
36.5	Extending with a new <code>SocketContext</code>	424
36.6	Extending with a new <code>SocketContextFactory</code>	424
36.7	Extending with middleware or routers	425
36.8	Extending with a <code>WebSocket</code> subprotocol	425
36.9	Extending MQTT behavior	426
36.10	Extending configuration	426
36.11	Extending the build and component surface	427
36.12	Extending diagnostics	428
36.13	Extending failure policy	428
36.14	Extending tests with the same boundary vocabulary	429

36.15	Avoiding framework pollution	429
36.16	Avoiding abstraction too early	430
36.17	Review questions for a proposed extension	430
36.18	From safe extension to a guided project	430
XII	Building a MiniGateway Application	431
37	Building MiniGateway	432
37.1	Why this chapter exists	432
37.2	What the application does	432
37.3	How to use the base version	433
37.4	The shape of the base application	434
37.5	Stage 1: the build target	434
37.6	Stage 2: the domain fact	435
37.7	Stage 3: the current state	436
37.8	Stage 4: the internal measurement bus	437
37.9	Stage 5: MiniGateway-specific MQTT configuration	438
37.10	Stage 6: the MQTT protocol object	440
37.11	Stage 7: constructing MQTT contexts	442
37.12	Stage 8: assembling the runtime roles	443
37.13	Building the base source package	447
38	Extending, Testing, and Deploying MiniGateway	448
38.1	Why this chapter follows the base build	448
38.2	What changes in the extended version	448
38.3	Extension overview	449
38.4	The build target after extension	450
38.5	The Unix-domain measurement context	451
38.6	The Unix-domain measurement factory	454
38.7	Runtime assembly after extension	455
38.8	Build and usage note for the extended version	458
38.9	What the extension teaches	459
38.10	Extending beyond this gateway input	460
38.11	Testing MiniGateway by boundaries	460
38.12	Manual checks while developing	461
38.13	Debugging by role	461
38.14	Deployment shape	462
38.15	OpenWrt and constrained systems	462
38.16	When to split the application	462
38.17	Extending safely in general	463
	Epilogue — What to Take Away from SNode.C	464
	Layers are not decoration	465
	Boundaries are design decisions	466
	Roles and instances need precise language	467
	Protocols carry meaning, not only bytes	468
	Build and deployment are part of architecture	469
	The guided project is not the end of the idea	470
	A philosophical review of the book	471
	Where to go next	472

Closing perspective	473
Further Reading	474
C++ and build systems	474
POSIX, Linux, and network programming	474
TLS and security protocols	474
HTTP, Server-Sent Events, and WebSocket	474
MQTT and messaging systems	474
Deployment and embedded Linux	475
Databases and persistence	475
SNode.C project material	475
Index	477

Preface

This book is an architecture-first guide to SNode.C. It teaches layered, event-driven network programming through one concrete C++ framework instead of treating the framework as a collection of isolated APIs.

The subject is practical, but the purpose is not only practical. The examples show how to start servers, connect clients, expose HTTP routes, stream events, upgrade to WebSocket, publish and consume MQTT traffic, persist application state, and deploy applications on Linux-oriented systems. The larger purpose is to make the boundaries visible while doing that work: lower communication family, transport form, connection handling, protocol meaning, configuration, diagnostics, deployment, and application role.

SNode.C is the worked system throughout the book. That is deliberate. A general survey can compare many libraries, but it often cannot follow one design deeply enough to show how the pieces hold together. This book makes the opposite choice. It follows one framework across many layers so that the same vocabulary can be recognized in small examples, protocol-specific chapters, and the MiniGateway capstone.

Who this book is for

The book is centered on technically engaged readers who need to understand how C++ components, protocols, configuration, diagnostics, deployment, and long-running data flows become maintainable systems. Many readers will arrive from professional C++ development; others will arrive because a course, a prototype, an installation, or a measurement system has grown large enough that its communication boundaries can no longer remain implicit.

Experienced C++ developers and technical system builders are the closest readers. They already understand classes, templates, ownership, build systems, and long-lived software, and they want more than a recipe. They want to see where the boundaries are, which pieces can be reused, which pieces should remain separate, and how new behavior can be added without turning an application into an accidental collection of callbacks.

Teachers and lecturers are another important audience. For them, the value is not only in the individual examples, but in the path through the material: a small echo pair, a visible runtime model, network families, protocol contexts, configuration, diagnostics, deployment, and finally an integrating MiniGateway project. The book can serve as a course spine for event-driven programming, network protocols, IoT architecture, and systems thinking.

Advanced learners and students may already know the broad idea of a protocol stack: perhaps Ethernet, IP, TCP, TLS, HTTP, WebSocket, MQTT, local IPC, or Bluetooth. Such readers may understand the layers in theory, but still have difficulty recognizing those layers in real C++ code. The book gives them a concrete source tree in which abstract networking concepts become roles, types, callbacks, configuration sections, components, and deployable applications.

Makers and prototypers may come from microcontrollers, home labs, sensor systems, Linux boards, routers, dashboards, artistic installations, or experimental technical setups. They may not want a purely academic treatment, but they do need growing systems to remain understandable. For these readers, SNode.C is useful because it lets practical work keep an architectural shape while devices, protocols, and services accumulate.

The book can also support technically involved scientists, domain researchers, and interdisciplinary teams who specify, adapt, or supervise data-collection and integration systems together with developers. Envi-

ronmental monitoring, biological observation, lab instrumentation, field sensing, distributed measurement stations, and similar systems all face the same architectural question: how sensing, transport, storage, observation, control, and management interfaces fit together without becoming an opaque tangle.

The book remains a technical book. It does not hide C++, protocols, build systems, or operational detail. Its aim is to make those details readable as parts of a system.

What this book assumes

The book assumes that you can read C++ source code, build CMake projects, work in a POSIX-like shell, and recognize common networking terms such as socket, address, connection, TLS, HTTP, WebSocket, and MQTT. It does not assume prior knowledge of SNode.C.

The examples are written for SNode.C release tag `v1.0.2`. The exact source commit used for this edition is shown below. Later versions of the framework may remain compatible, but the book's examples and source-derived explanations are tied to that baseline unless the manuscript is deliberately updated.

`6e475262084ae2dab2daef8781ab9e4adb82d18e`

What this book is not

This is not a beginner's introduction to C++, a complete textbook on networking, or an independent comparison of C++ networking frameworks. It is also not a complete reference manual for every SNode.C symbol.

The book explains SNode.C by showing why its abstractions exist, how they are used, and how they compose. When a chapter discusses TLS, HTTP, Server-Sent Events, WebSocket, MQTT, databases, OpenWrt, or testing, the purpose is to show how those subjects meet the framework's architecture. Each of those topics has deeper specialist literature; this book stays focused on the framework and the design vocabulary needed to use it responsibly.

How to Read This Book

The book is organized as a gradual widening of scope.

Parts I and II establish the basic vocabulary: runtime, role, connection, layer, context, factory, configuration, and event processing. These chapters are best read in order. Later chapters rely on their terminology and usually apply it rather than redefining it.

Parts III and IV move from raw communication families to protocol-local behavior. They show how the same design vocabulary appears with IPv4, IPv6, Unix domain sockets, Bluetooth, custom `SocketContext` classes, and context factories.

Parts V and VI explain operational behavior: configuration, diagnostics, TLS, timeouts, retry, reconnect, and failure handling. These chapters are especially useful when an example has to become a service rather than a demonstration.

Parts VII and VIII apply the lower model to web and messaging protocols: HTTP, the Express-like layer, Server-Sent Events, WebSocket, MQTT, and MQTT over WebSocket.

Parts IX and X widen the view to persistence, larger applications, MQTTSuite, components, linking, deployment, and testing. Chapter 32 is the central place for component, public-header, and linking information; protocol chapters only keep the local details that matter for understanding the chapter itself.

Parts XI and XII bring the material together. The design chapters discuss judgment and extension boundaries. The MiniGateway chapters then use the earlier concepts in a guided application that is larger than the introductory examples.

The book can be read linearly, but it is also useful with the source tree open. When a chapter names a component, public header, example, or source-derived excerpt, treat the source tree as part of the reading experience. The text explains the architecture; the source tree shows where that architecture becomes code.

Different readers may emphasize different routes through the same material. C++ developers and technical system builders can read linearly, or they can first concentrate on Chapters 3–20 to understand the core model before moving into the web, MQTT, persistence, deployment, and MiniGateway chapters.

Teachers and lecturers can treat the book as a course spine: introductory examples, architecture chapters, protocol chapters, system-design chapters, and MiniGateway as an integrating project.

Advanced students benefit from the full progression: echo pair, architecture, network families, contexts and factories, configuration, protocols, systems, and MiniGateway. Makers and prototypers may prefer a practical route through Chapters 1, 3, 5, 7, 16–20, 23–28, 31, 33, and the MiniGateway chapters.

Scientists, domain researchers, and interdisciplinary technical teams may first read the architectural and system-design chapters to understand how sensing, transport, storage, observation, control, and management surfaces fit together, then return to implementation details as needed.

Conventions Used in This Book

Code identifiers, paths, component names, command names, and configuration keys are written in monospace, for example `SocketContext`, `net-in-stream-legacy`, `CMakeLists.txt`, and `--help`.

Shell examples assume a POSIX-like shell unless stated otherwise. Commands are intended to show the shape of the workflow. Local installation prefixes, compiler choices, package-manager paths, and service names may differ between systems.

C++ listings are teaching excerpts. Some are complete files from the companion examples; others are shortened to keep attention on the relevant boundary. When a listing is described as abridged or source-derived, it should be read as an explanatory slice, not as a replacement for the full file.

The companion example trees under `companion/examples/` are the checked source of truth for the larger examples. Printed excerpts are selected to explain the relevant boundary; when a complete file matters, read the companion source tree beside the chapter.

The word *role* is used deliberately. It usually means the part an object or configured instance plays in the communication architecture: server role, client role, protocol role, context role, or deployment role. It does not imply a separate operating-system process unless the text says so.

The word *layer* is also used deliberately. A layer is not decoration in the diagrams. It marks a boundary where one kind of responsibility should not silently collapse into another.

Author and Framework Note

This is a first-party book. The author is the creator and maintainer of SNode.C and writes from inside the design of the framework.

That position has advantages and limits. The advantage is direct architectural knowledge: the book can explain why the framework separates runtime roles, connections, contexts, factories, protocol layers, configuration, diagnostics, and deployment concerns in the way it does. It can also connect public examples to implementation choices that are not obvious from an API reference alone.

The limit is that this book is not an independent review of SNode.C against every alternative C++ networking library. It does not try to rank frameworks or claim that one design is best for every project. It explains the design SNode.C uses, the kinds of systems that design supports, and the tradeoffs a reader should understand when adopting or extending it.

The book therefore asks to be judged on two levels: as a practical guide to SNode.C, and as a worked study of layered, event-driven network architecture in modern C++.

Acknowledgements

SNode.C grew out of practical networking work, teaching, experiments, and repeated attempts to keep communication software understandable as it becomes larger.

I am grateful to the students, colleagues, users, and reviewers whose questions forced the design vocabulary to become sharper. Questions about why a boundary exists are often more valuable than requests for another shortcut; they expose whether an architecture can still explain itself.

This book also depends on the wider open-source ecosystem around C++, CMake, Linux, TLS, HTTP, WebSocket, MQTT, databases, documentation tools, and typesetting. SNode.C is one framework, but it exists inside that larger technical culture.

Part I

Getting Oriented

This part gives the reader the first stable map of the book. It explains why `SNode.C` exists, prepares the development environment, builds the first echo pair, and shows how to read the repository without getting lost in details too early.

The goal is not to teach every API immediately. The goal is to establish the first vocabulary: runtime, role, connection, layer, context, and application shape.

Chapter 1

Why SNode.C Exists

1.1 A framework that teaches systems, not only APIs

Many networking frameworks are optimized for speed of first use. That is not a criticism. In practice, speed matters. A developer often wants to start a web server, connect a client to a broker, or move data from one endpoint to another without first studying the whole internal architecture.

But convenience has a cost when it hides too much.

If the framework hides the deeper structure of the system, the reader may learn how to *call* it without learning how the communication model is organized. That can be enough for a small example, but it becomes fragile when the application grows: one endpoint becomes several, one protocol becomes many, TLS is added, configuration becomes operationally important, and deployment suddenly matters.

SNode.C is interesting because it takes a different path. It is a layered, event-driven C++ framework that keeps the structure of networked applications visible without reducing networking to socket helpers, HTTP handlers, or MQTT utilities.

That visibility gives the framework a specific teaching value. SNode.C is useful where software stops being a single endpoint and becomes a small ecosystem: a service and its clients, devices and dashboards, a lab setup, an artistic or experimental installation, a course project, or a measurement system with observation, control, storage, and a management surface.

In such systems, the central question is not only how to open a socket. It is how to organize roles, protocols, configuration, diagnostics, persistence, deployment, and operational surfaces so the result can still be understood and maintained.

SNode.C lets the reader build real network applications, but it also exposes the architectural boundaries those applications depend on.

The promise of this book is therefore practical and architectural at the same time:

By the end of the book, you should be able to build servers and clients with SNode.C, understand why its architecture has the shape it has, and extend that architecture without breaking its conceptual boundaries.

The goal is not simple copying. The goal is that you can explain why the example has its shape.

1.2 The problem SNode.C addresses

To understand why SNode.C is worth studying, it helps to look at what often goes wrong in networking code.

A first problem is **collapse of layers**. A small program begins with a socket. Then protocol parsing is added directly into the same class. Then configuration logic appears nearby. Then retries, logging, TLS, HTTP behavior, or MQTT integration are bolted on. Before long, the program works, but nobody can explain its structure cleanly.

A second problem is **one-protocol tunnel vision**. Some developers learn networking through HTTP and begin to see every system as a collection of routes. Others learn raw sockets and treat higher protocols as strings over a connection. Others learn MQTT or WebSocket first and skip the lower model entirely. Each entry point is useful, but none of them alone gives a durable architecture.

A third problem is **lack of transfer**. A developer learns how to build one TCP/IPv4 server, but cannot immediately answer what changes for IPv6, Unix domain sockets, Bluetooth RFCOMM, Bluetooth L2CAP, TLS, HTTP, WebSocket, MQTT, or database-backed state.

A fourth problem is **framework opacity**. Some frameworks are pleasant as long as the application stays close to the examples. The moment the developer wants to combine several protocols, reason about connection lifetime, control configuration, or diagnose a failure after deployment, the internal model becomes difficult to see.

SNode.C addresses these problems by making structure explicit.

It distinguishes among lower communication family, transport form, connection handling, and application protocol. It repeatedly uses roles and boundaries such as application-side handles, configured communication roles, registered runtime-visible instances, connections, contexts, factories, configuration, and runtime. It supports several lower communication families within one conceptual scheme. It can add TLS without forcing the application model to be reinvented. It then builds HTTP, an Express-like web layer, Server-Sent Events, WebSocket, MQTT, MQTT over WebSocket, and database-facing applications on top of the same layered discipline.

This gives the reader transfer.

A concept learned in one place can be recognized again in another. A server over IPv4, a server over IPv6, a local service over Unix domain sockets, and a Bluetooth-oriented endpoint are not unrelated worlds. They have different endpoint identities and deployment assumptions, but the architectural questions remain comparable.

That is the first reason this book exists. It teaches SNode.C, but it also uses SNode.C to teach how networked systems can be organized.

1.3 What this book will and will not do

This book is not a replacement for formal networking theory. It assumes the reader already knows the broad idea of network layers and common protocols.

It is also not merely an API reference. A reference answers questions such as “what is the signature of this function?” or “which overloads exist?” Those questions matter, but a teaching book has a different job. Its job is to build durable understanding.

This book therefore treats SNode.C abstractions in four linked ways. It explains **why** a particular abstraction exists, shows **how** that abstraction is used in small but realistic examples, connects the mechanism back to the **layered model** of the framework, and repeatedly asks the transfer question: what remains the same, and what changes, if the lower communication family changes, TLS is inserted, or a higher protocol layer is added?

The book will touch many technologies, but it will not become a specialist book about all of them. It is not a complete TLS book, Bluetooth book, HTTP book, MQTT book, database book, or OpenWrt deployment manual.

Instead, it uses these technologies to explain how SNode.C organizes networked software. That distinction is important.

When Bluetooth RFCOMM and Bluetooth L2CAP appear, they are not treated as strange appendices. They are lower communication families that exercise the same architectural model in a device-near setting.

When HTTP, the Express-like layer, Server-Sent Events, WebSocket, MQTT, and MQTT over WebSocket appear, they are not presented as isolated feature islands. They are higher protocol structures built on the lower communication model.

When persistence and MQTTSuite appear, they show how SNode.C concepts become larger systems.

The focus remains the same throughout the book:

```
understand the boundary
-> understand the role
    -> understand the code
        -> understand what can change safely
```

1.4 Where SNode.C fits

SNode.C is not trying to replace every C++ networking approach. It is most useful when an application needs explicit communication roles, layered protocol structure, runtime-visible configuration, diagnostics, and several transport or protocol surfaces within one architectural model.

If an application only needs a small one-off TCP client, direct POSIX sockets, Boost.Asio, standalone Asio, or another focused library may be the simpler choice. If the application wants a narrowly optimized HTTP service, a specialized web framework may be more direct. SNode.C is useful when the structure of the communication system itself matters.

This book therefore does not position SNode.C as a universal answer to every networking problem. It positions SNode.C as a coherent framework for readers who want to build multi-protocol applications while keeping the communication model visible.

1.5 What this book is, and what it is not

This book is:

- an architecture-first guide to SNode.C
- a layered network-programming book
- a framework book for mature C++ readers
- a practical design guide for multi-protocol applications

This book is not:

- a general C++ networking survey
- a Boost.Asio replacement tutorial
- a beginner C++ book
- a pure IoT recipe book
- a complete TLS, HTTP, MQTT, Bluetooth, or database reference

1.6 Source version used by this book

SNode.C is an active framework. This book describes the public architecture, component names, public include paths, examples, and package layout as they exist in the SNode.C_{v1.0.2} baseline used for this edition. When reading a newer repository checkout, some implementation details, component inventories, or example applications may have changed.

1.7 Why “layered” matters here

In technical writing, the word *layered* is sometimes used lazily. It can mean almost anything from “we have modules” to “there are several abstraction levels.” In SNode.C, however, layering is not decorative. It is operational.

Figure 1.1 is the first compact view of that structure: each row names a design question that becomes more concrete in later chapters, from endpoint family up to application role.

At the bottom is the lower communication family. It answers:

```
Which communication family carries the endpoint?
IPv4, IPv6, Unix domain sockets, Bluetooth RFCOMM, or Bluetooth L2CAP?
```

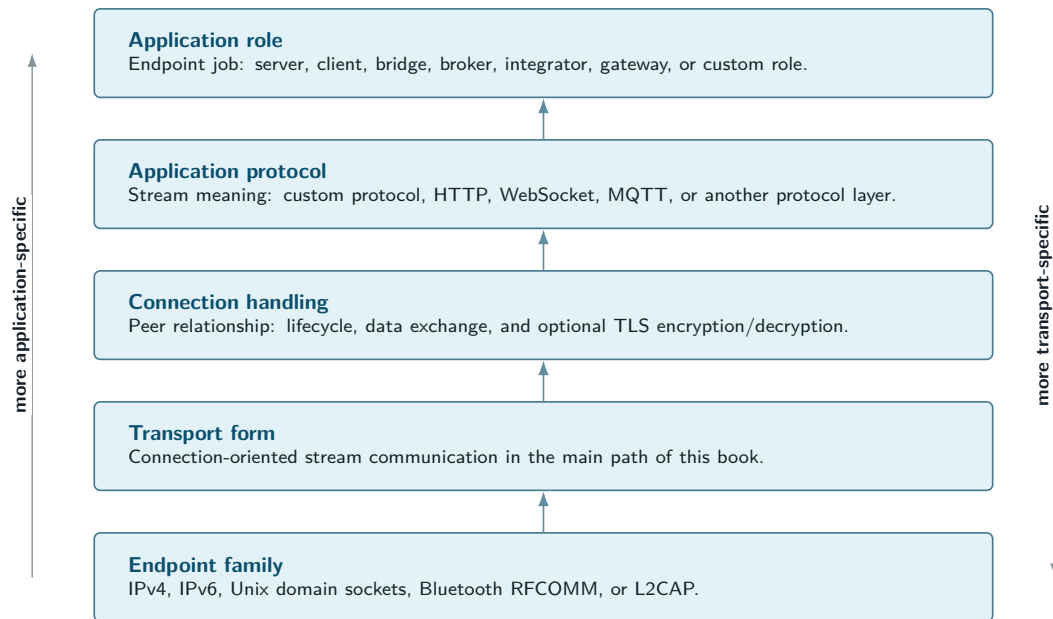



Figure 1.1: The basic SNode.C layer model used throughout the book.

Above that is the transport form. It answers:

What form does communication take?

In the main parts of this book, connection-oriented stream communication.

Above that is connection handling. It answers:

How is a concrete peer relationship advanced?

Connection lifecycle, data exchange, and, in TLS variants, encryption and decryption.

Above that is the application protocol. It answers:

What meaning does the byte stream have?

A custom protocol, HTTP, WebSocket, MQTT, or another protocol-specific layer.

At the top is the application role. It answers:

What job does this endpoint perform in the application?

Server role, client role, bridge, broker, integrator, gateway, or another application-specific role.

The consequence is simple but powerful:

Not everything changes at once.

A protocol implemented in a SNode.C context can often remain recognizable when the lower family changes. That is true for small teaching protocols, but it is not limited to them. Larger protocols also keep their application-level structure when the lower communication family changes, even though their internal behavior may be richer. The endpoint identity and deployment environment may change, but the protocol does not have to be redesigned merely because the communication is carried differently.

This does not mean lower layers are irrelevant. They are very relevant. IPv4, IPv6, Unix domain sockets, RFCOMM, and L2CAP have different addressing models, operating-system assumptions, and deployment consequences.

The point is different:

Separation rule

SNode.C lets lower-layer choices and application-layer behavior be reasoned about separately without pretending that they are unrelated.

That is the architectural lens the reader should keep from the beginning.

1.8 In the spirit of node.js, but not node.js in C++

A reader encountering SNode.C for the first time may notice a relationship to node.js-style thinking. That relationship is real, but it should be understood carefully.

The similarity is not that SNode.C imitates JavaScript.

The similarity is deeper: both environments place event-driven flow and network communication near the center of the programming model.

But SNode.C is not node.js in C++.

It is a C++ framework with a C++ way of expressing structure. Types matter. Namespaces matter. Public include paths matter. Construction matters. Ownership matters. Build components matter. Configuration and deployment surfaces matter. The architecture is not hidden behind a dynamic runtime; it is expressed through classes, templates, specializations, modules, and explicit roles. The include tree and the component tree are two public views of the same layered design: one is the C++ source-facing view, the other is the build/link-facing view.

That gives SNode.C its particular teaching value.

In a dynamic environment, a developer can sometimes get surprisingly far without understanding the whole internal shape. In modern C++, that approach is less satisfying. Long-term confidence usually comes from understanding the structure properly.

SNode.C rewards that effort.

1.9 Why this matters for multi-protocol application development

The phrase *multi-protocol application* is used deliberately here. This book is not limited to exposing an HTTP route, opening a TCP socket, or publishing an MQTT message. It is about recognizing how several communication surfaces can belong to one application while still keeping their responsibilities separate.

A useful SNode.C application may need to reason across communication layers and protocol boundaries.

A realistic networked system may involve:

- a local control channel over Unix domain sockets,
- a remote endpoint over IPv4 or IPv6,
- a Bluetooth endpoint for device-near communication,
- TLS for secure transport,
- HTTP for service exposure,
- Server-Sent Events or WebSocket for live communication,
- MQTT for message-oriented integration,
- and MariaDB-backed persistence for selected application state.

These are not separate worlds. In a real system, they meet.

SNode.C is especially interesting in the context of IoT and machine-to-machine communication because such systems rarely live at only one level. They often touch devices, local transports, network transports, secure channels, web interfaces, broker-oriented messaging, persistence, deployment, and diagnostics at the same time.

MQTTSuite is the reference ecosystem that makes this concrete. It builds focused MQTT applications on top of SNode.C, including broker, integrator, bridge, command-line, and store roles. MQTTSuite appears later, after the framework model has been established. That order is deliberate: the reader first needs to understand the recurring SNode.C roles and boundaries before seeing them at application and system scale.

1.10 The central pattern you will meet again and again

Although the details come later, the recurring SNode.C application shape can already be previewed.

Many SNode.C applications involve:

```
runtime
  -> server or client instance
      -> connection
          -> context factory
              -> per-connection context
```

At this introductory point, read *instance* as the server-side or client-side communication role of an application. It is not a concrete peer connection. Chapter 3 connects that role to the visible C++ objects used in the first program, and Chapter 5 sharpens the term: the visible object is the application-side handle, while the instance is the configured role registered with the framework and advanced by the runtime.

The connection represents a concrete peer relationship. The factory creates the context. The context contains the application protocol behavior for one connection.

The runtime coordinates event-driven progress.

At this point, this is only a preview. Chapter 3 shows the first working program, and Chapter 5 names the model more formally. The important point is that SNode.C is regular: the same few roles reappear in different protocol settings, and that regularity is one of the reasons the framework is teachable.

The first concrete program will therefore be modest: an echo server and client.

1.11 Why the first example is an echo pair

Some readers see an echo example and assume it is too simple to matter. That reaction is understandable, but mistaken. A good echo application is a microscope.

It exposes server and client creation, context construction, callback flow, data input and output, runtime startup, and the boundary between framework logic and application logic with very little distraction.

The first echo pair is not meant to impress with protocol sophistication. It is meant to show the shape of the framework before higher layers add vocabulary of their own.

Chapter 3 gives the first full example.

Here it is enough to understand why the book starts small: the small example gives the first clear view of the structure.

What to remember

- SNode.C is introduced in this book as the concrete framework for layered network programming, not merely as a collection of APIs.
- Its value comes from recurring roles and boundaries: runtime, configured communication role, registered runtime-visible instance, concrete connection, factory, and per-connection context.
- The framework separates lower communication choices from application protocol behavior, so the same mental model can survive changes in network family, transport, or protocol layer.
- The first echo example is small because it exposes the shape of the framework without hiding it behind protocol complexity.
- MQTTSuite becomes important later as a reference ecosystem, but the book first builds the framework model that makes such systems understandable.

1.12 Closing perspective

A good framework should provide features and help its users think clearly.

This chapter has explained why SNode.C is worth studying as a framework and as a teaching object. The next step is practical: before writing the echo pair, the reader needs a working environment. Architecture becomes meaningful only when examples can be built, run, inspected, and modified.

SNode.C is worth studying because it offers a coherent way to think about networked applications in modern C++. It connects lower communication families such as IPv4, IPv6, Unix domain sockets, Bluetooth RFCOMM, and Bluetooth L2CAP with higher protocol systems such as HTTP, WebSocket, and MQTT. It supports practical concerns such as configuration, TLS, deployment, diagnostics, and persistence. Most importantly, it does so without erasing the architecture.

The goal of this book is not simply that you can write code *with* SNode.C, but that, after some chapters, you begin to see networked systems *through* the architectural lens that SNode.C makes available.

Once that happens, the framework becomes a way of organizing thought with practical tooling behind it. And that is the right place to begin.

Chapter 2

Preparing Your Environment

2.1 Why the environment matters

Chapter 1 introduced SNode.C as a framework whose teaching value comes from visible structure: event-driven runtime behavior, explicit layers, named roles, and reusable application boundaries.

Before we can study that structure in code, the development environment must make it visible.

That may sound like a purely practical concern. It is not. For this book, the environment is part of the learning model. SNode.C is not a single-file demonstration library. It is a real C++ framework with a core runtime, network-family components, stream transports, legacy and TLS connection variants, higher protocol layers, example applications, generated CMake targets, installable package components, and optional support for technologies such as Bluetooth and MariaDB.

If the source tree, build tree, install prefix, and external playground are mixed together, the architecture becomes harder to see. If they are kept separate, the structure of the framework becomes much easier to inspect.

The goal of this chapter is therefore simple:

Prepare a development setup that lets you build SNode.C, run examples, inspect the framework source tree, and later compile small applications against installed SNode.C public headers and installed SNode.C components.

This is not a general Linux installation guide. It does not try to cover every distribution, every package manager, every editor, or every cross-compilation target. The concrete commands assume a normal Linux development machine, with Debian-style package names where package installation is shown. The ideas transfer to other systems, but package names and installation commands may need adjustment.

2.2 Working areas, different purposes

During the book it helps to keep several locations mentally separate. The first is the **SNode.C source tree**.

That is where the framework implementation lives. It contains the C++ source code, the CMake build system, the core runtime, network layers, protocol layers, database support, and example applications.

The second is the **build tree**.

That is where CMake writes generated build files, intermediate object files, and compiled binaries. It should not be mixed into the source tree.

The third is the **install prefix**.

That is where an installed SNode.C package places headers, libraries, CMake package files, and possibly installed executables. A local install prefix is especially useful for learning because it keeps the experiment self-contained.

The fourth is a small **playground project**.

That is where you can later write small external applications that consume an installed SNode.C package without modifying the framework source tree itself.

A useful layout is:

```
projects/  
  snode.c/  
  snode.c-build/  
  snodec-playground/
```

A local install prefix can live outside these source and build directories:

```
~/local/snodec/
```

This separation is not just tidiness. It protects the reader from confusing source files with generated files, framework examples with local experiments, and build artifacts with installed package files.

The framework source tree remains the source of truth for the implementation. The build tree is disposable. The install prefix is what external projects consume. The playground is where you can test your understanding.

2.3 What the environment must support

For the first part of the book, the environment must support five activities. First, it must allow us to **clone and build SNode.C**. Second, it must allow us to **run small server and client examples**.

Third, it must allow us to **inspect the framework source tree and build tree separately**. Fourth, it must allow us to **install the framework into a known prefix**. Fifth, it must allow us to **compile small external programs against installed SNode.C public headers and installed SNode.C components**.

Optional technologies should also be visible when available. Bluetooth RFCOMM and Bluetooth L2CAP require the appropriate BlueZ development files. MariaDB support requires MariaDB development files. These optional areas do not have to be used immediately, but the environment should not hide them accidentally.

2.4 Compiler and CMake expectations

SNode.C is a modern C++ framework.

The build system expects CMake and a C++20-capable compiler. The top-level build requires CMake 3.18 or newer, and the `src` build configures the project as C++20.

The most important compiler expectations are:

- GCC 12.2 or newer,
- or Clang 13.0 or newer.

This is not an arbitrary preference. SNode.C uses modern C++ internally. Building it with an older compiler is not a good way to learn the framework: the reader will spend time fighting toolchain problems instead of understanding architecture.

A quick check is useful:

```
cmake --version  
g++ --version  
clang++ --version
```

It is fine if only one of GCC or Clang is used, as long as it satisfies the required version.

2.5 Required and optional packages

Package names differ across distributions. On a Debian-style development system, the baseline tools are usually:

```
sudo apt update
sudo apt install git cmake make ninja-build g++ clang pkg-config
```

The core framework path also needs development libraries. A useful baseline is:

```
sudo apt install libssl-dev nlohmann-json3-dev
```

Optional libraries unlock additional framework areas:

```
sudo apt install libbluetooth-dev libmagic-dev libmariadb-dev
```

`libbluetooth-dev` is relevant for Bluetooth RFCOMM and Bluetooth L2CAP. `libmagic-dev` is relevant where file-type or content-type detection is used.

`libmariadb-dev` is relevant for MariaDB support.

Additional developer tools are useful later, especially during maintenance and review:

```
sudo apt install iwyu clang-format cmake-format doxygen
```

These tools are not required for the first echo example. They are mentioned here because SNode.C is a framework, and framework work eventually benefits from include checking, formatting, and documentation generation.

2.6 Cloning the framework

Choose a directory where you keep source repositories:

```
mkdir -p ~/projects
cd ~/projects
git clone https://github.com/SNodeC/snode.c.git
cd snode.c
git checkout v1.0.2
```

This creates and selects the public SNode.C release tag used by this book:

```
~/projects/snode.c/
```

The book baseline is SNode.C_{v1.0.2}. The tag `v1.0.2` is the reader-facing checkout target. For exact reproducibility, compare your checkout with the source pin stated once in the Preface.

The SNode.C repository uses `master` as its moving development line. Do not build the examples against an arbitrary newer `master` checkout unless you deliberately want to check the book against a newer framework state. If you already have a clone, update the tag information and select the book release explicitly:

```
cd ~/projects/snode.c
git fetch --tags
git checkout v1.0.2
```

For normal reading, do not edit the framework repository immediately. First learn where examples are located, how the build is organized, and which parts of the framework are source code, generated build output, or installation artifacts.

2.7 Use an out-of-tree build

A build directory should be separate from the source directory. This keeps generated files away from the code you want to read.

A simple layout is:

```
~/projects/  
  snode.c/  
    snode.c-build/
```

Configure the build:

```
cd ~/projects  
cmake -S snode.c -B snode.c-build
```

Then build:

```
cmake --build snode.c-build -j$(nproc)
```

Using `cmake --build` is deliberate. It does not force the reader to care immediately whether the generator is Make, Ninja, or something else.

If you prefer Ninja explicitly, configure with:

```
cmake -S snode.c -B snode.c-build -G Ninja  
cmake --build snode.c-build -j$(nproc)
```

The build can take some time. That is normal. The framework uses many templates and builds many components and examples.

2.8 Installing the framework

For experimenting inside the framework build tree, installation is not always necessary.

For compiling separate applications against SNode.C with `find_package(snodec ...)`, installation is useful. It turns the framework from “source tree plus build tree” into a package that another CMake project can consume.

A local user installation avoids touching system directories:

```
cmake -S snode.c -B snode.c-build \  
  -DCMAKE_INSTALL_PREFIX="$HOME/.local/snodec"  
  
cmake --build snode.c-build -j$(nproc)  
cmake --install snode.c-build
```

When using a custom prefix, later projects may need to know where to find the installed package:

```
export CMAKE_PREFIX_PATH="$HOME/.local/snodec:$CMAKE_PREFIX_PATH"
```

A system-wide installation is also possible:

```
sudo cmake --install snode.c-build  
sudo ldconfig
```

Use a system-wide install only when you are comfortable installing development libraries into the system prefix.

For the book, a local prefix is often the safer teaching setup. It makes it clear which installation belongs to the experiments and avoids accidental interaction with other system packages.

2.9 Build options worth knowing early

The first chapters do not require many CMake options.

Still, two logging-related options are useful to recognize early:

```
SNODEC_DISABLE_LOGLEVEL_LOGGING
SNODEC_DISABLE_VERBOSE_LOGGING
```

These options can disable logging categories at compile time.

Do not disable logging while working through the first examples unless you have a specific reason. Early in the book, visible runtime output is useful. It helps connect the code you write with the runtime behavior you observe.

Later chapters will treat logging and configuration more carefully. At this stage, the important rule is simple:

Early diagnostic warning

Do not make the first examples silent before you understand what they are doing.

2.10 Finding built executables

Build-tree layouts differ depending on generator, configuration, and installation choices. Rather than memorizing one exact executable path, learn how to inspect the build tree.

From the build directory, search for echo-related executables:

```
cd ~/projects/snode.c-build
find . -type f -executable -name '*echo*
```

Or search more generally for application executables:

```
find . -type f -executable | grep apps
```

This is often more robust than assuming one fixed path.

When later chapters refer to example applications, the important distinction is:

```
source file location
-> where the example is implemented

build-tree executable location
-> where the configured build placed the binary

installed executable location
-> where installation placed the binary, if it was installed
```

Do not confuse these three. Many build problems become easier to diagnose once this distinction is clear.

2.11 Running server and client programs

The basic pattern for server/client examples is simple. Open one terminal for the server. Open another terminal for the client.

Start the server first. Then start the client.

The echo pair in Chapter 3 uses this pattern. The server listens. The client connects and initiates the first message. Once both sides reflect received data, the program intentionally creates visible ping-pong behavior.

That behavior is useful for teaching. It proves that the runtime is active, the connection has been established, callbacks are executed, data is read, and data is written back.

To stop the example, interrupt one side with:

```
Ctrl-C
```

The first time you run such an example, pay attention to the output. You are not simply checking that the program works. You are learning how the framework makes runtime behavior visible.

2.12 Do not hide runtime output too early

At the beginning, visible runtime output is your friend.

If an example logs that it is listening, connecting, connected, reading, writing, or disconnecting, that output is part of the learning process.

Later, logging can be configured more carefully. In the first chapters, however, do not silence the framework too aggressively.

A quiet program is not automatically a clean program. Sometimes it is only a program that has hidden the evidence you need for understanding.

2.13 Preparing a separate playground project

Besides building the framework source tree itself, it is useful to have a small external playground project.

For example:

```
~/projects/  
snode.c/  
snode.c-build/  
snodec-playground/
```

The playground is where you can later put small experiments without modifying the framework source tree.

A minimal playground for the first echo-style experiments might eventually contain:

```
snodec-playground/  
CMakeLists.txt  
EchoSocketContext.h  
EchoSocketContext.cpp  
echoserver.cpp  
echoclient.cpp
```

Do not worry yet about writing all of this. Chapter 3 introduces the first concrete program. The point here is only to prepare a place where an external application can live.

2.14 Components and public headers as installed architecture

When an external application consumes SNode.C, it asks CMake for package components. The same application also includes public SNode.C headers. These two selections belong together: the component selection describes the binary/link surface, while the include selection describes the C++ source surface.

For the first IPv4 legacy stream example, the relevant component is:

```
find_package(snodec COMPONENTS net-in-stream-legacy)
```

The executable then links against the corresponding imported target:

```
target_link_libraries(echoserver PRIVATE snodec::net-in-stream-legacy)
```

The matching public include path has the same architectural shape, expressed with directories rather than CMake component dashes:

```
#include <net/in/stream/legacy/SocketServer.h>
#include <net/in/stream/legacy/SocketClient.h>
```

The application does not include the lower core socket headers merely because the concrete server and client are built from lower socket machinery. It includes the highest public header for the abstraction it directly names. Likewise, the build links the highest component that represents the direct framework surface used by the target.

The actual echo pair has separate server and client targets, but both follow the same public-header/component pairing.

The component name is not arbitrary. It encodes a path through the architecture:

```
net-in-stream-legacy
```

can be read as:

```
layer area:      net
network family: in      (IPv4)
transport:      stream
connection:     legacy  (non-TLS stream connection variant)
```

The term `legacy` is important. In this naming context it denotes the non-TLS stream connection variant. It does not, by itself, mean that the component is obsolete or deprecated.

Later chapters will introduce additional component names for IPv6, Unix domain sockets, Bluetooth RFCOMM, Bluetooth L2CAP, TLS variants, HTTP, WebSocket, MQTT, and database support.

Do not try to memorize all names at this stage. Learn the shape instead. SNode.C component names and public include paths are compact forms of architectural information. The dashes in a component name and the slashes in an include path often describe the same stack from different technical viewpoints.

2.15 Source tree, build tree, install tree

Before moving on, keep three locations distinct:

```
source tree
  the files you read and edit

build tree
  generated files and compiled binaries

install tree
  headers, libraries, CMake package files, and installed executables
```

For example:

```
~/projects/snode.c/
~/projects/snode.c-build/
~/.local/snodec/
```

This distinction will appear again throughout the book. It matters for CMake, for debugging, for deployment, and for understanding why an external playground project is different from an in-tree example.

Environment checklist

Before moving on, you should be able to answer these questions.

- Can I clone the SNode.C repository?
- Can I configure an out-of-tree build?
- Can I compile the framework?
- Can I find built example executables?
- Can I run a server and a client in separate terminals?
- Do I understand the difference between the source directory, build directory, and install prefix?
- Do I know whether optional Bluetooth development files are installed?
- Do I know whether optional MariaDB development files are installed?
- Do I know whether I am using a local install prefix or a system-wide install?
- Do I have a separate playground directory for later external experiments?

If the answer to these questions is yes, the environment is ready for the first program.

What to remember

- Keep the source tree, build tree, install tree, and playground project separate; each has a different purpose.
- Use an out-of-tree build so generated files and compiled binaries do not obscure the source tree.
- A local install prefix makes external examples predictable because headers, libraries, and CMake package files live in one known place.
- Component names such as **net-in-stream-legacy** encode architectural position; public include paths such as `<net/in/stream/legacy/SocketServer.h>` encode the corresponding C++ source-facing entry point; **legacy** means the non-TLS stream connection variant.
- Runtime output is part of the learning process. Do not silence diagnostics before the first examples are understood.
- The environment is ready when the framework builds, example programs can be found and run, and an external playground project can consume the installed package.

2.16 Closing perspective

A good development environment keeps the compiler happy and shapes how the reader thinks.

The example will be small. But it will already expose the recurring shape of SNode.C applications.

Chapter 3

Your First Working Program: The Echo Pair

3.1 From environment to first program

Chapter 2 prepared the practical ground: source tree, build tree, install tree, and a separate playground project. It uses that environment for the first complete program.

The goal is not to build a useful echo service. The goal is to make the recurring shape of SNode.C visible in real code. A good first example should be small enough that the reader can hold the whole program in mind, but complete enough that it is not pseudocode.

An echo pair is ideal for that purpose.

It contains one server and one client. The client sends the first message. The server reflects the received bytes. The client receives the reflected bytes and sends them again. The result is a visible ping-pong.

That behavior is simple, but the structure is already the structure of many later SNode.C programs:

```
runtime
-> server or client instance
    -> socket context factory
        -> per-connection socket context
            -> application protocol behavior
```

The rest of the book will add other network families, TLS, HTTP, WebSocket, MQTT, configuration, deployment, and persistence. The basic shape appears here first.

3.2 What the SNode.C repository contains

The SNode.C repository contains a full echo application under `src/apps/echo`.

The repository example is more general than the first teaching version in this chapter. It uses a shared echo model and builds several variants by combining network-family and stream-mode choices. The echo context is implemented in `src/apps/echo/model/EchoSocketContext.h` and `EchoSocketContext.cpp`; the server and client entry points are `src/apps/echo/echoserver.cpp` and `echoclient.cpp`.

The repository design is important because it demonstrates that the echo application is not tied to one hard-coded socket kind. The same application model can be combined with different lower layers.

For a first chapter, however, that full matrix would hide the essential pattern. We therefore use a reduced version:

```
network family: IPv4 / in
transport:      stream
connection:     legacy
```

```
role:           one server, one client
```

Here `legacy` has the same meaning introduced in Chapter 2: it denotes the non-TLS stream connection variant. It does not mean that the component is obsolete.

The reduced chapter example is not meant to replace the repository version. It is the smallest readable form of the same architectural pattern.

3.3 The four files

The teaching version consists of four source files:

```
EchoSocketContext.h
EchoSocketContext.cpp
echoserver.cpp
echoclient.cpp
```

The first two files define the application behavior. The last two files define the server and client entry points.

That separation is already meaningful.

The echo protocol itself should not be mixed into `main()`. The server and client applications should create handles, configure roles, and register those roles with the framework. The per-connection protocol behavior belongs in a `SocketContext`.

3.4 The three roles in the first example

Before writing code, it helps to name the three roles.

The instance

In everyday discussion, it is natural to call the visible `SocketServer` or `SocketClient` object an instance. That is acceptable as long as the basic idea is clear. In SNode.C's stricter architectural vocabulary, however, the visible C++ object is the application-side handle. Through that handle, the application configures and registers a server-side or client-side communication role. After `listen(...)` or `connect(...)`, that configured role is the instance the framework can advance through the runtime and flow-controller machinery. It is not yet a concrete peer connection; connections appear later.

For this chapter, the visible handle types are:

```
net::in::stream::legacy::SocketServer<...>
net::in::stream::legacy::SocketClient<...>
```

The `net::in` part means IPv4. The `stream` part means connection-oriented stream communication. The `legacy` part means the non-TLS stream connection variant.

The server-side role listens. The client-side role connects. Neither one contains the echo protocol directly.

The factory

A `SocketContextFactory` creates a new context object for each established connection.

That is an important design choice. Connection-specific protocol state should not be stored globally and should not be constructed manually in `main()` whenever a peer appears. The framework asks the factory for a context when a connection needs one.

The context

A `SocketContext` contains the application protocol behavior for one connection.

In the echo example, that behavior is small:

```
onConnected()
    client sends the first message

onReceivedFromPeer()
    read bytes from the peer
    send the same bytes back

onDisconnected()
    log that the connection closed
```

This is the first place where the event-driven nature of `SNode.C` becomes visible. The program does not write its own blocking read loop. It implements callback methods that the framework calls when the connection lifecycle or input state changes.

3.5 The echo context header

We first define the context and the two factories.

EchoSocketContext.h

```
#ifndef ECHO_SOCKET_CONTEXT_H
#define ECHO_SOCKET_CONTEXT_H

#include <core/socket/stream/SocketContext.h>
#include <core/socket/stream/SocketContextFactory.h>

#include <cstdint>

namespace core::socket::stream {
    class SocketConnection;
}

class EchoSocketContext : public core::socket::stream::SocketContext {
public:
    enum class Role {
        SERVER,
        CLIENT
    };

    explicit EchoSocketContext(core::socket::stream::SocketConnection* socketConnection,
                              Role role);

private:
    void onConnected() override;
    void onDisconnected() override;
    bool onSignal(int signum) override;
    std::size_t onReceivedFromPeer() override;

    Role role;
};

class EchoServerSocketContextFactory
    : public core::socket::stream::SocketContextFactory {
private:
    core::socket::stream::SocketContext*
    create(core::socket::stream::SocketConnection* socketConnection) override;
```

```
};

class EchoClientSocketContextFactory
: public core::socket::stream::SocketContextFactory {
private:
    core::socket::stream::SocketContext*
    create(core::socket::stream::SocketConnection* socketConnection) override;
};

#endif
```

There are a few details worth noticing.

The `EchoSocketContext` derives from `core::socket::stream::SocketContext`. That says where the application protocol sits: above a stream connection.

The constructor receives a `core::socket::stream::SocketConnection*`. The connection object is managed by the framework. The context uses it indirectly through methods such as `readFromPeer(...)` and `sendToPeer(...)`.

The factories implement `create(...)`. They are small because their job is small: create the correct context object for a new connection.

The `Role` enum lets one context class serve both sides. The server and client share the same echo behavior, but the client must send the first message. The role tells the context whether it should initiate the ping-pong when the connection becomes active.

The `onSignal(...)` override is included because it belongs to the context interface used by the repository echo example. Signal handling is not the conceptual focus of this chapter.

3.6 Implementing the echo behavior

The implementation is short.

`EchoSocketContext.cpp`

```
#include "EchoSocketContext.h"

#include <log/Logger.h>

#include <string>

EchoSocketContext::EchoSocketContext(
    core::socket::stream::SocketConnection* socketConnection,
    Role role)
: core::socket::stream::SocketContext(socketConnection)
, role(role) {
}

void EchoSocketContext::onConnected() {
    VLOG(1) << "Echo connected";

    if (role == Role::CLIENT) {
        sendToPeer("Hello peer! Nice to see you!!!");
    }
}

void EchoSocketContext::onDisconnected() {
    VLOG(1) << "Echo disconnected";
}

bool EchoSocketContext::onSignal([[maybe_unused]] int signum) {
    return true;
}
```



```

}

std::size_t EchoSocketContext::onReceivedFromPeer() {
    char chunk[4096];

    const std::size_t chunkLen = readFromPeer(chunk, sizeof(chunk));

    if (chunkLen > 0) {
        VLOG(1) << "Data to reflect: " << std::string(chunk, chunkLen);
        sendToPeer(chunk, chunkLen);
    }

    return chunkLen;
}

core::socket::stream::SocketContext*
EchoServerSocketContextFactory::create(
    core::socket::stream::SocketConnection* socketConnection) {
    return new EchoSocketContext(socketConnection,
                                EchoSocketContext::Role::SERVER);
}

core::socket::stream::SocketContext*
EchoClientSocketContextFactory::create(
    core::socket::stream::SocketConnection* socketConnection) {
    return new EchoSocketContext(socketConnection,
                                EchoSocketContext::Role::CLIENT);
}

```

This file contains the whole protocol behavior.

`onConnected()` is called when the connection has reached the connected state. Only the client sends an initial message. If the server also sent immediately, the example would no longer show the client-initiated communication pattern clearly.

`onReceivedFromPeer()` reads available bytes into a local buffer. If bytes were read, the context logs them and sends the same bytes back. The return value tells the framework how many bytes were consumed.

The factories allocate the concrete context. The framework owns the surrounding connection machinery; the user supplies the protocol object that belongs to a connection.

This is the first important boundary in code:

```

framework-managed connection
-> user-created SocketContext
    -> application protocol behavior

```

3.7 The server application

The server entry point is small because the protocol behavior already lives in the context.

`echoserver.cpp`

```

#include "EchoSocketContext.h"

#include <core/SNodeC.h>
#include <log/Logger.h>
#include <net/in/stream/legacy/SocketServer.h>

int main(int argc, char* argv[]) {
    core::SNodeC::init(argc, argv);
}

```

```

using EchoServer =
    net::in::stream::legacy::SocketServer<EchoServerSocketContextFactory>;

EchoServer server("echoserver");

server.listen(
    8080,
    5,
    [instanceName = server.getConfig()->getInstanceName()]
    (const EchoServer::SocketAddress& socketAddress, const core::socket::State& state) {
        switch (state) {
            case core::socket::State::OK:
                VLOG(1) << instanceName << ": listening on '"
                    << socketAddress.toString() << "'";
                break;
            case core::socket::State::DISABLED:
                VLOG(1) << instanceName << ": disabled";
                break;
            case core::socket::State::ERROR:
                LOG(ERROR) << instanceName << ": "
                    << socketAddress.toString()
                    << ": " << state.what();
                break;
            case core::socket::State::FATAL:
                LOG(FATAL) << instanceName << ": "
                    << socketAddress.toString()
                    << ": " << state.what();
                break;
        }
    }
);

return core::SNodeC::start();
}

```

The SNode.C include is the public front-door header for the concrete server role used here: IPv4, stream transport, non-TLS `legacy` connection handling, and server-side behavior. The application does not manually include every lower `core/socket/...` header that participates in that composition. It includes the highest public header for the abstraction it directly names.

The first important line is:

```
core::SNodeC::init(argc, argv);
```

This initializes the SNode.C runtime environment before the server handle is used to configure and register the server-side instance.

The next important line is the type alias:

```
using EchoServer =
    net::in::stream::legacy::SocketServer<EchoServerSocketContextFactory>;
```

This is the complete lower-layer choice for this server. It says:

```
IPv4
-> stream
    -> non-TLS connection
        -> context factory for echo
```

Then the server handle is created with a name:

```
EchoServer server("echoserver");
```

The name already matters. Named instances become important when configuration and diagnostics enter the book. Even in the first program, the name gives log output and callbacks a clear identity.

Finally, the server handle registers the listening role:

```
server.listen(8080, 5, callback);
```

For a first reading, it is tempting to say that this line “starts the server.” That is close, but not precise enough for SNode.C.

A better mental model is:

Runtime registration rule

`listen(...)` configures and registers the server-side communication role. The runtime and flow-controller machinery advance the actual event-driven flow after `core::SNodeC::start()` is called.

This distinction will matter later for configuration, retries, and runtime behavior.

3.8 The client application

The client mirrors the server.

echoclient.cpp

```
#include "EchoSocketContext.h"

#include <core/SNodeC.h>
#include <log/Logger.h>
#include <net/in/stream/legacy/SocketClient.h>

int main(int argc, char* argv[]) {
    core::SNodeC::init(argc, argv);

    using EchoClient =
        net::in::stream::legacy::SocketClient<EchoClientSocketContextFactory>;

    EchoClient client("echoclient");

    client.connect(
        "localhost",
        8080,
        [instanceName = client.getConfig()->getInstanceName()]
        (const EchoClient::SocketAddress& socketAddress, const core::socket::State& state) {
            switch (state) {
                case core::socket::State::OK:
                    VLOG(1) << instanceName << ": connected to '"
                        << socketAddress.toString() << "'";
                    break;
                case core::socket::State::DISABLED:
                    VLOG(1) << instanceName << ": disabled";
                    break;
                case core::socket::State::ERROR:
                    LOG(ERROR) << instanceName << ": "
                        << socketAddress.toString()
                        << ": " << state.what();
                    break;
                case core::socket::State::FATAL:
                    LOG(FATAL) << instanceName << ": "
                        << socketAddress.toString()
                        << ": " << state.what();
                    break;
            }
        });
}
```

```

    }
    }
};

return core::SNodeC::start();
}

```

The client uses the matching public front-door header for the client role. Server and client are different public roles even though they share the same lower family, transport form, and connection variant. The include path therefore changes only at the final role file: `SocketServer.h` versus `SocketClient.h`.

The type alias is the client-side counterpart to the server type:

```

using EchoClient =
    net::in::stream::legacy::SocketClient<EchoClientSocketContextFactory>;

```

The client handle also has a name:

```

EchoClient client("echoclient");

```

The client handle registers the connecting role in the code:

```

client.connect("localhost", 8080, callback);

```

Again, this should not be read as a blocking call that performs the entire communication on the caller's stack. It registers the client-side communication role. The runtime and flow-controller machinery then perform the actual event-driven work.

The symmetry between server and client is intentional:

```

server
  init
  create named server handle
  register listen
  start runtime

client
  init
  create named client handle
  register connect
  start runtime

```

This symmetry is one reason the echo pair is a useful first example.

3.9 Building the simplified echo pair

Assume the four source files are in the playground directory prepared in Chapter 2:

```

snodec-playground/
  CMakeLists.txt
  EchoSocketContext.h
  EchoSocketContext.cpp
  echoserver.cpp
  echoclient.cpp

```

A minimal `CMakeLists.txt` for this example is:

```

cmake_minimum_required(VERSION 3.18)

project(echo-pair LANGUAGES CXX)

```

```

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

find_package(snodec REQUIRED COMPONENTS net-in-stream-legacy)

add_library(echosocketcontext STATIC
    EchoSocketContext.cpp
    EchoSocketContext.h
)

target_include_directories(echosocketcontext
    PUBLIC
    ${CMAKE_CURRENT_SOURCE_DIR}
)

target_link_libraries(echosocketcontext
    PUBLIC
    snodec::net-in-stream-legacy
)

add_executable(echoserver
    echoserver.cpp
)

target_link_libraries(echoserver
    PRIVATE
    echosocketcontext
)

add_executable(echoclient
    echoclient.cpp
)

target_link_libraries(echoclient
    PRIVATE
    echosocketcontext
)

```

This build file imports the installed SNode.C package and requests the component introduced in Chapter 2:

```

find_package(snodec REQUIRED COMPONENTS net-in-stream-legacy)

```

The context library links publicly to `snodec::net-in-stream-legacy` because its public header derives from SNode.C stream-context types. The two executables then link to the context library. The source-side and build-side selections now agree: the application includes `<net/in/stream/legacy/SocketServer.h>` or `<net/in/stream/legacy/SocketClient.h>` for the concrete C++ role, and the CMake target links `snodec::net-in-stream-legacy` for the corresponding binary surface.

That means the build structure follows the code structure:

```

SNode.C component
-> echo context library
    -> echoserver
    -> echoclient

```

The example is smaller than the repository build. The repository echo application uses generated variants and shared model headers. It uses one explicit IPv4 legacy variant so that the first build remains readable.

3.10 Configuring and building the playground

If SNode.C was installed into the local prefix from Chapter 2, configure the playground like this:

```
cd ~/projects
cmake -S snodec-playground -B snodec-playground-build \
  -DCMAKE_PREFIX_PATH="$HOME/.local/snodec"

cmake --build snodec-playground-build -j$(nproc)
```

If CMAKE_PREFIX_PATH is already exported in the shell, the explicit -DCMAKE_PREFIX_PATH=... argument may not be necessary. Keeping it visible here makes the first external build easier to understand.

After a successful build, the build directory should contain two executables:

```
echoserver
echoclient
```

The exact path depends on the CMake generator and build layout, but both executables belong to the playground build tree, not to the SNode.C framework build tree.

3.11 Running the echo pair

Open two terminals.

In the first terminal, start the server:

```
cd ~/projects/snodec-playground-build
./echoserver
```

In the second terminal, start the client:

```
cd ~/projects/snodec-playground-build
./echoclient
```

The client connects to localhost on port 8080 and sends the first message. The server receives it and reflects it. The client receives the reflection and reflects it again.

That produces intentional ping-pong behavior.

This is not a production protocol. It is a visibility tool. It shows that:

```
the server role was registered
the client role was registered
the runtime started
a connection was established
a context was created
onConnected() was called
onReceivedFromPeer() was called
bytes moved in both directions
```

Stop the example with Ctrl-C.

3.12 Reading the output

Do not treat the output as noise.

The first example is meant to connect source code to runtime behavior. When you see a log line from onConnected(), it corresponds to the lifecycle callback. When you see the reflected message, it corresponds

to `onReceivedFromPeer()`. When the callback passed to `listen(...)` or `connect(...)` logs a state, it tells you whether the configured communication role was registered successfully.

That is why Chapter 2 warned against silencing runtime output too early. The output is part of the teaching instrument.

3.13 Comparing the chapter version with the repository version

The repository version and the chapter version are deliberately not identical.

The repository version uses helper headers such as `servers.h` and `clients.h` to select the concrete server or client type from build-time choices. It can build several variants from the same model. That does not change the rule used in this simplified version: a source file should include the public header that owns the `SNode.C` abstraction it directly names.

The chapter version writes the concrete type directly:

```
net::in::stream::legacy::SocketServer<EchoServerSocketContextFactory>
net::in::stream::legacy::SocketClient<EchoClientSocketContextFactory>
```

That is less flexible, but much clearer for the first reading.

The repository version often uses parameterless `listen(callback)` and `connect(callback)` after instance configuration has been established elsewhere. The chapter version uses direct overloads with host and port visible in the call.

Again, that is a teaching choice.

The important common structure is the same:

```
application-side handle
-> registered server/client instance
    -> context factory
        -> per-connection context
            -> event-driven protocol behavior
                -> runtime start
```

3.14 What changed compared with ordinary socket programming

If you have written direct POSIX socket code before, this example may look unusual. There is no explicit `accept()` loop in `main()`. There is no blocking `recv()` loop in the application entry point.

There is no direct manual construction of per-peer protocol state in the server loop.

Instead, the framework owns the event-driven connection machinery, and the application supplies three things:

```
the communication role
the factory
the context behavior
```

That is the central tradeoff.

You give up the illusion that `main()` controls every socket operation directly. In return, you get a structure that can be transferred across lower layers and reused by higher protocol systems.

This does not make the low-level details disappear. It organizes where they belong.

What to remember

- The first working example is small, but it already contains the core `SNode.C` application pattern.

- The visible server or client object is the application-side handle; the instance is the configured communication role registered through `listen(...)` or `connect(...)`.
- A `SocketContextFactory` creates one context for each established connection.
- A `SocketContext` contains the application protocol behavior for that connection.
- `listen(...)` and `connect(...)` register communication roles; the runtime and flow-controller machinery advance the actual event-driven work.
- The same structure used for IPv4 legacy streams can later be recognized again when the lower family, stream mode, or application protocol changes.

3.15 Closing perspective

This chapter turned the prepared environment into a working program.

That is the next step from “it works” to “I understand why it works.”

Chapter 4

Reading the Codebase with Confidence

4.1 From first program to source-tree orientation

Chapter 3 built the first working echo pair.

That example was small. It used one lower communication family, one transport form, one connection mode, one server, one client, one context class, and two factories. The point was not to cover the whole framework. The point was to make the recurring SNode.C application shape visible:

```
runtime
  -> server or client instance
      -> socket context factory
          -> per-connection socket context
              -> application protocol behavior
```

Here, read *instance* in the refined sense used throughout the book: the configured server-side or client-side communication role, not an already established peer connection.

After that first program, the next difficulty is orientation.

The SNode.C source tree is much larger than the first echo pair. It contains runtime code, logging support, utilities, lower communication families, transport specializations, HTTP and web-protocol layers, an Express-like framework, database support, IoT-oriented pieces, examples, applications, build logic, and packaging support.

A reader who opens such a tree without a map can easily get the wrong impression. The project may look like a large collection of unrelated files. It is not. The source tree is best read as an expression of the same layered model introduced by the first example.

The rule for this chapter is therefore simple:

Reading rule

Do not read SNode.C as a pile of files. Read it as a set of layers, roles, and recurring boundaries.

This chapter does not try to explain every file. It teaches a reading strategy. Later chapters will move into the runtime, the lower communication families, application protocols, configuration, TLS, HTTP, WebSocket, MQTT, persistence, deployment, and testing in detail. Here the goal is more basic: learn how to open the repository and stay oriented.

4.2 Use the build structure as the first map

The first useful map is not a class diagram. It is the build structure.

The top-level `CMakeLists.txt` defines the project, prepares project-level helper modules, delegates into `src`, and then includes packaging support. the top-level file is mostly a gateway into the framework source tree rather than the place where the framework structure itself is expressed.

```
CMakeLists.txt
-> src/
-> packaging support
```

The `src/CMakeLists.txt` file is more informative for architectural orientation. It defines compiler requirements and options, configures logging switches, and adds the major source subdirectories:

```
log/
utils/
core/
net/
web/
express/
database/
iot/
apps/
```

That list is the first practical source-tree map.

It says that `SNode.C` is not arranged around one monolithic executable. It is arranged around framework regions. When reading the codebase, begin with those regions before jumping into individual implementation files.

A first mental map is:

```
src/
  log/      logging support
  utils/    common utilities and support code
  core/     runtime, event processing, timers, socket base abstractions
  net/      lower communication families and transport specializations
  web/      HTTP, WebSocket, and related protocol infrastructure
  express/  Express-like server framework
  database/ database-facing framework support
  iot/      IoT-oriented protocol and integration pieces
  apps/     example and reference applications
```

This map is not an exhaustive inventory. It is an orientation tool. It answers the first question a reader should ask about any file:

Which architectural region am I reading?

That question is more useful than immediately searching for every occurrence of a type name.

4.3 Start from applications, then follow inward

There are two common ways to read a framework.

One way is to start at the lowest-level runtime code and work upward. That can be useful after the architecture is already familiar, but it is a hard way to begin. The reader sees many abstractions before seeing why they matter.

The better first strategy for `SNode.C` is the reverse:

```
application
-> concrete server/client type
  -> factory
    -> context
      -> lower communication type aliases
        -> core abstractions
```

```
-> runtime machinery
```

Therefore, the echo pair is valuable. It gives the reader a concrete entry point. A good first reading path is:

```
src/apps/echo/
-> model/
    -> EchoSocketContext
    -> EchoServerSocketContextFactory
    -> EchoClientSocketContextFactory
-> echoserver.cpp
-> echoclient.cpp
-> lower-layer variants built around the same model
```

From that path, follow the type names into **net**. From **net**, follow the generic base abstractions into **core**. This order prevents the reader from drowning in implementation depth before understanding the application role that caused the abstraction to exist.

The question is not:

How many templates are involved?

The better question is:

Which architectural decision does this type fix, and which decision does it leave to the application?

That question turns a complex C++ type into a readable design artifact.

4.4 Separate example code from framework code

The **apps** directory is a practical guide to how **SNode.C** is used, but it is not the framework core.

That distinction is important.

If you are reading **src/apps/echo**, you are reading an application built with **SNode.C**. If you are reading **src/core**, you are reading runtime and foundational abstractions. If you are reading **src/net**, you are reading lower communication families and transport specializations. If you are reading **src/web**, **src/express**, **src/iot**, or **src/database**, you are reading higher-level support built on top of the lower runtime and communication model.

Confusing those regions leads to wrong conclusions.

An example application may choose a shape because it is readable, demonstrative, or useful for testing a combination of layers. A framework layer must support more than one application shape. When reading code under **apps**, therefore, separate three things:

```
framework pattern
-> application-specific decision
    -> build or variant mechanism
```

For the echo application, the framework pattern is the server/client/factory/context structure. The application-specific decision is echo behavior. The variant mechanism is the way the application model is combined with several lower-layer choices.

Those are related, but they are not the same.

4.5 Follow names as architecture

SNode.C names are intentionally descriptive. They are sometimes long because they encode architectural position.

Consider this name:

```
net::in::stream::legacy::SocketServer
```

It can be read as a path:

```
net      network-facing layer
in       IPv4 family
stream   connection-oriented stream transport
legacy   non-TLS stream connection variant
server   server-side role
```

The same style appears in related names:

```
in6      IPv6
un       Unix domain sockets
rc       Bluetooth RFCOMM
l2       Bluetooth L2CAP
tls      TLS-secured stream connection variant
```

The term `legacy` is especially important. In `SNode.C` naming, it denotes the non-TLS stream connection variant. It should not be read as “obsolete” merely because the word normally has that association.

When reading a long `SNode.C` name, do not treat the length as noise. Treat it as information. A name often tells you:

- which architectural region the type belongs to,
- which lower communication family it targets,
- whether it is stream-oriented,
- whether the connection mode is non-TLS or TLS,
- and whether the role is server-side or client-side.

This is one of the main reasons the source tree is readable once the naming scheme is understood.

4.6 Learn to read type aliases

`SNode.C` uses type aliases and templates to turn generic machinery into concrete, usable types.

An application may use a convenient concrete type for IPv4, stream-oriented, non-TLS communication. That type may be an alias or specialization of a more general server or client template. This is normal in a modern C++ framework.

The useful question is:

What does this alias fix?

Usually, it fixes lower-layer choices such as:

```
network family
transport form
connection mode
physical socket type
configuration type
```

The application still supplies protocol behavior, usually through a `SocketContextFactory`.

That is the key boundary:

```
framework fixes lower communication machinery
application supplies per-connection protocol behavior
```

This boundary is why the echo application can stay conceptually small while the framework around it supports many communication variants.

4.7 The recurring path from role to context

Most `SNode.C` applications can be read through the same path.

For a server, the path is:

```
SocketServer
-> configuration
-> listen(...)
-> registered server instance / listen flow machinery
-> SocketConnection
-> SocketContextFactory
-> SocketContext
```

For a client, the path is:

```
SocketClient
-> configuration
-> connect(...)
-> registered client instance / connect flow machinery
-> SocketConnection
-> SocketContextFactory
-> SocketContext
```

This path is more important than any single file name. When a file feels difficult, ask where it sits in the path.

Is it part of the application-side server or client handle, the configured instance, its configuration, the lower-layer socket machinery, a connection object, a factory, the per-connection context, or an application protocol implementation?

That question restores orientation because it reconnects the file to a role.

4.8 Reading core

The `core` directory contains foundational runtime machinery. It is not the best place to begin the book, but it is the place where many later explanations eventually arrive.

Read `core` in conceptual groups rather than linearly.

Runtime control

Start with the public runtime surface. `core::SNodeC` is the runtime facade most applications touch. Around it are the event loop, event handling, multiplexing, timers, and runtime-level coordination mechanisms.

This group explains how work moves forward after an application has registered a listening or connection intention.

Socket abstractions

After runtime control, read the socket abstractions. The stream `SocketContext`, `SocketContextFactory`, and connection abstractions are especially important because application code derives from or interacts with them.

They form the bridge between framework-managed connection machinery and user-defined protocol behavior.

Flow control

The flow-controller path explains why `listen(...)` and `connect(...)` should not be read as blocking calls that perform all communication immediately on the caller's stack.

They register communication intent. The runtime-visible flow is then carried by configuration, connection, and flow-controller state. This distinction matters later for retries, timeouts, configuration, diagnostics, and shutdown behavior.

4.9 Reading net

The `net` directory is where lower communication families become concrete.

The same framework shape is specialized for families such as:

```
in    IPv4
in6   IPv6
un    Unix domain sockets
rc    Bluetooth RFCOMM
l2    Bluetooth L2CAP
```

When reading `net`, ask three questions in order:

```
Which lower communication family is this?
Which transport form is this?
Which connection mode is this?
```

For example:

```
net/in/stream/legacy
```

means:

```
IPv4
-> stream
    -> non-TLS connection variant
```

The same reading strategy applies to IPv6, Unix domain sockets, Bluetooth RFCOMM, Bluetooth L2CAP, and TLS variants where those combinations are supported by the build and platform.

This is the transfer value of `SNode.C`. The lower details differ, but the reading strategy remains stable.

4.10 Reading higher protocol directories

The higher protocol directories should be read after the lower communication model is understood.

`web` contains web-oriented protocol infrastructure. `express` contains the Express-like framework layer. `iot` contains IoT-oriented protocol and integration pieces. `database` contains database-facing support.

These directories are important, but they are not isolated frameworks in a separate universe. They build on the lower runtime and communication model.

The guiding question is:

Which lower-layer and runtime mechanisms does this higher-level feature depend on?

That question prevents HTTP, WebSocket, MQTT, Express-like routing, and database-backed application state from looking like unrelated feature islands. They are higher-level structures attached to the same event-driven and layered foundation.

4.11 Use CMake as a navigation tool

CMake files are build instructions and maps at the same time.

A `CMakeLists.txt` file can tell you:

- which subdirectories matter,
- which targets are built,
- which libraries are linked,
- which optional dependencies enable additional pieces,
- which public headers are installed,
- which components are installed,
- and how applications relate to framework libraries.

The top-level build delegates into `src`. The `src` build adds the major framework regions. Application subdirectories then define executables and link against the required framework components.

This gives a practical reading habit:

Build-reading habit

When you do not know where an executable, library, or component comes from, read its nearest `CMakeLists.txt` before searching the whole tree.

Build files reveal boundaries that may be less obvious from implementation files alone.

4.12 Do not confuse source paths, public includes, and installable components

The source tree is organized as directories. The installed framework is exposed through public headers and CMake package components. These views are related, but they are not identical.

A source path such as:

```
src/net/in/stream/legacy
```

helps a reader navigate the implementation.

A component name such as:

```
net-in-stream-legacy
```

helps an external project request and link the required installed framework part.

The same architectural position can therefore appear in several forms:

```
source path:
src/net/in/stream/legacy

C++ namespace / type path:
net::in::stream::legacy

public include path:
<net/in/stream/legacy/SocketServer.h>
<net/in/stream/legacy/SocketClient.h>

CMake component name:
net-in-stream-legacy
```

Read these as related views of the same decision stack:

```
network family: IPv4 / in
transport form: stream
connection mode: legacy / non-TLS
```

This relationship is not accidental. It is one of the ways `SNode.C` makes layer choices visible across source layout, C++ names, and build consumption.

4.13 Use search, but search for roles

Text search is useful, but search without an architectural question can mislead.

A weak search question is:

Where is `SocketServer`?

A better search habit asks for the role that a name plays:

Search by role

- Which `SocketServer` is the stream server template?
- Which `SocketServer` name selects IPv4 legacy stream communication?
- Which overload registers the port and backlog?
- Which code path starts the flow controller?
- Which factory creates the per-connection context?

A good search query combines a name with a role:

```
grep -R "class SocketContextFactory" src/core src/net
grep -R "startFlow" src/core
grep -R "net::in::stream::legacy" src
grep -R "EchoSocketContext" src/apps/echo
```

The exact command is less important than the habit. Search for roles, not only for words.

4.14 Recognize variant-heavy code

`SNode.C` supports many combinations of lower communication family, transport form, connection mode, server/client role, and higher protocol layer.

That is a strength of the framework, but it can make the source tree look larger than the underlying idea.

When you see similar code shapes across IPv4, IPv6, Unix domain sockets, RFCOMM, L2CAP, legacy, and TLS, do not assume that every variant must be learned from zero.

Ask first:

What is structurally the same?

Then ask:

What changes because the lower layer or connection mode changes?

That is the right way to read variant-heavy framework code. It also prevents Bluetooth support from looking like an unrelated exception. RFCOMM and L2CAP are lower communication families that exercise the same reading strategy in a device-near setting, subject to platform and build availability.

4.15 Read headers before implementation files

For orientation, read headers first. In `SNode.C`, some headers are ordinary local declarations, but many public headers are also architectural front doors. A header such as `net/in/stream/legacy/SocketServer.h` selects a concrete server role and exports or composes the lower public pieces needed by that role.

A header often tells you:

- the public role of a class,
- which base class it uses,
- what dependencies matter,
- which methods define the boundary,
- which names are aliases,

- and which types are intended for application code.

Implementation files explain behavior. Headers explain shape. Public front-door headers explain source-facing stack selection. When learning `SNode.C`, shape should come first.

A useful order is:

1. Read the header to understand the role and its public include boundary.
2. Read the nearby `CMake` file to understand the target or component.
3. Read the implementation to understand behavior.
4. Return to the header to confirm the public boundary.

This order prevents implementation details from hiding the architecture.

4.16 How to read a server or client type

When you encounter a server or client type, use this checklist:

```
Which lower communication family does it belong to?
Is it stream-oriented?
Is it legacy or TLS?
What configuration type is involved?
What physical socket type does it wrap?
What factory type does the application provide?
Which listen(...) or connect(...) overload is used?
Where does the flow enter the runtime?
Which context is created when a connection is established?
```

These questions turn a complex type into a structured reading exercise. They also keep the focus on architecture. The exact spelling of a template instantiation matters, but it matters because it encodes a boundary decision.

4.17 How to read a context class

When you encounter a `SocketContext`-derived class, use a different checklist:

```
Which base class does it derive from?
Which constructor receives the SocketConnection pointer?
Which lifecycle methods are overridden?
Does onConnected() initiate behavior?
Does onReceivedFromPeer() read and process data?
Does it call sendToPeer(...)?
Does it set timeouts or close the connection?
Does it distinguish server and client roles?
```

In Chapter 3, the echo context used a role enum so one context class could serve both server and client behavior. Other applications may use separate context classes. Both designs can be valid. The important point is whether the per-connection protocol state is located in the context rather than being spread through unrelated code.

A context class is often the best place to understand application behavior, because it is where incoming data becomes protocol meaning.

4.18 How to read a factory

A `SocketContextFactory` looks small, but it marks an important boundary.

It answers this question:

When the framework has a connection, which application context should be attached to it?

That is why factories are so useful for orientation. They connect framework-managed connection lifetime with user-defined protocol behavior.

When reading a factory, ask:

```
Which context type does it create?
Does it pass role information?
Does it pass configuration or shared application state?
Is one factory used for one role, or does it abstract over several roles?
```

A factory marks the handoff point between the framework and the application.

4.19 Avoid common reading mistakes

Several mistakes are easy to make when first reading SNode.C.

One mistake is to start in the deepest runtime file and try to understand everything before seeing a working application. That makes the framework look more abstract than it is.

A second mistake is to treat long names as accidental verbosity. In SNode.C, long names often encode layer position.

A third mistake is to confuse source-tree directories with public include paths or installable package components. They are related views, not the same object. The source path shows where the framework implementation lives. The public include path shows which C++ front door an application should include. The component name shows which installed binary/link surface an application should select.

A fourth mistake is to treat lower communication families as unrelated worlds. IPv4, IPv6, Unix domain sockets, RFCOMM, and L2CAP differ in addressing and deployment assumptions, but the SNode.C reading strategy remains comparable.

A fifth mistake is to read an example as if every application must copy its exact shape. Examples teach patterns; they do not eliminate design judgment.

The remedy is always the same: return to layer, role, boundary, and ownership.

4.20 A compact reading workflow

A practical reading workflow for SNode.C is:

1. Start from an application or testable example.
2. Identify the server or client type.
3. Decode the public include path, namespace, type path, and component name.
4. Find the factory.
5. Read the context class.
6. Follow the lower-layer alias into net.
7. Follow the generic abstraction into core.
8. Use CMake to confirm the target and component boundary.
9. Return to the application and re-read it with the framework roles visible.

This workflow is not only useful for beginners. It also matters when extending the framework. A new feature should have a clear place in the same map.

What to remember

- Read SNode.C as a set of layers and recurring roles, not as a flat pile of source files.
- CMake files are navigation aids: they reveal subdirectories, targets, dependencies, components, public header installation, and install boundaries.
- Long public include paths, namespace paths, and component names usually encode architecture: layer area, network family, transport form, connection mode, and role.
- Example applications show framework usage, but they are not the framework core; separate

application decisions from reusable framework patterns.

- A reliable reading path starts from an application, finds the application-side handle, configured instance, factory, and context, then follows aliases into **net** and abstractions into **core**.
- Variant-heavy code becomes easier to read once you ask what stayed structurally the same and what changed because a lower layer changed.

4.21 Transition to the architecture chapters

Part I began with motivation, prepared the build environment, built the first echo pair, and now established a way to read the source tree. The next part can therefore become more architectural. Chapter 5 names the mental model more formally. It will discuss SNode.C as an event-driven, layered framework built from recurring roles: runtime, instance, connection, factory, context, configuration, and protocol behavior.

The important result of this chapter is not a memorized directory list. The important result is confidence:

```
when you open a file,  
ask which layer it belongs to,  
ask which role it plays,  
ask which boundary it expresses,  
and only then read the implementation details.
```

That habit is the difference between browsing the repository and understanding it.

Part II

The SNode.C Architecture

This part introduces the architectural core of SNode.C. It explains the mental model, the event-driven runtime, and the layered shape that will be reused throughout the rest of the book.

The reader should leave this part with a clear distinction between lower communication family, transport form, connection handling, protocol context, and application role.

Chapter 5

The Mental Model of SNode.C

5.1 From source-tree orientation to architectural thinking

Chapter 4 showed how to read the SNode.C source tree without getting lost in individual files.

The most important rule was to read the source tree as a set of layers, roles, and recurring boundaries. That rule is useful when navigating the codebase. It is also the starting point for understanding the framework itself.

This chapter turns that reading strategy into a compact mental model.

A framework like SNode.C can be approached in two ways. One way is to memorize names: `SocketServer`, `SocketClient`, `SocketContext`, `SocketContextFactory`, `listen(...)`, `connect(...)`, `core::SNodeC::start()`, `net::in`, `net::in6`, `net::un`, `legacy`, `tls`, `HTTP`, `WebSocket`, `MQTT`, Express-like routing, and so on.

That approach can work for the first small example, but it does not scale. The moment an application becomes less standard, the reader needs structural answers:

Why is protocol behavior placed in a context?
Why does a factory sit between the instance and the context?
What does `listen(...)` or `connect(...)` actually register?
Why can the same application shape move across IPv4, IPv6, Unix domain sockets, Bluetooth, or TLS?
Why do configuration and retry behavior belong to the framework model, not merely to command-line decoration?

Those are not isolated API questions. They are architectural questions.

A good mental model does not list every class. It tells you what kind of thing each part is, what job it has, where the boundary lies, and how control and data move through the system.

Once that is clear, SNode.C becomes much easier to predict.

5.2 The shortest useful description

The shortest useful description of SNode.C is this:

SNode.C is an event-driven, layer-based framework in which configured server and client instances register communication intent, the runtime advances that intent through event processing, and per-connection contexts express application protocol behavior on top of selectable lower communication layers.

That sentence is dense, but it contains the core of the book.

event-driven means that application code does not normally write its own blocking read/write loop. It registers communication roles and implements callbacks and context methods that are invoked as the runtime observes events.

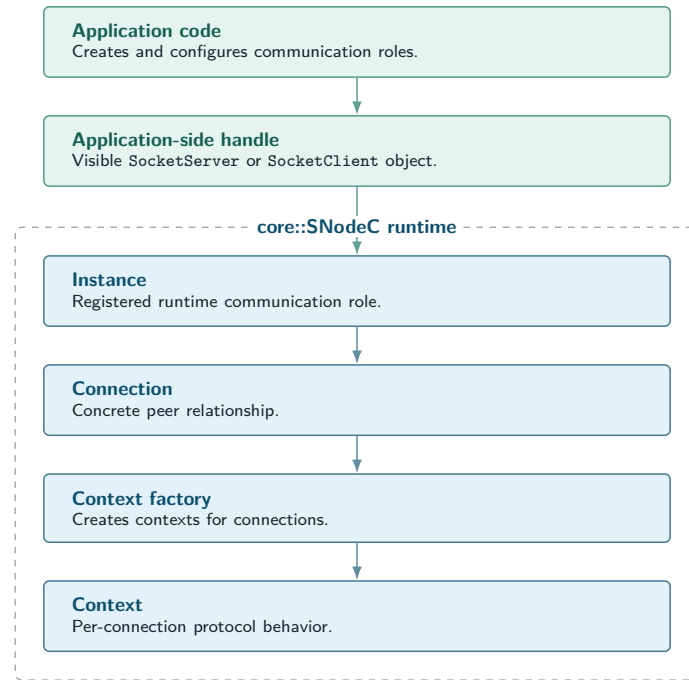


Figure 5.1: The runtime view of application-side handles, registered instances, connections, factories, and contexts.

layer-based means that network family, transport form, connection handling, and application protocol are separated enough to be reasoned about independently.

configured server and client instances are communication roles registered with and managed by the framework’s runtime and flow-controller machinery. In user code, they are configured and registered through visible server or client objects.

communication intent is the fact that an application wants to listen or connect. Calling `listen(...)` or `connect(...)` is best read as registering that intent with the framework, not as performing all later communication immediately on the caller’s stack.

per-connection contexts are the protocol endpoints attached to concrete peer connections.

selectable lower communication layers are what make the same application shape usable over IPv4, IPv6, Unix domain sockets, Bluetooth RFCOMM, Bluetooth L2CAP, and TLS-capable stream variants where the corresponding modules support that combination.

This chapter unpacks that model in three views:

```

runtime model
layer model
operational model
  
```

They are not separate frameworks. They are three views of the same design.

5.3 The runtime model

The runtime model answers the question:

How does a SNode.C application come alive?

The answer has four recurring roles and one coordinating runtime. As shown in Figure 5.1, application code uses a visible handle to register a communication role; the runtime advances the registered instance; a concrete connection appears; the factory creates a context; and the context expresses protocol behavior for that peer.

The figure is deliberately small. Its purpose is to keep the handle, instance, connection, factory, and context roles distinct when moving from the echo pair to larger applications: a handle is visible in application code, an instance is registered in the runtime, connections appear under that role, and contexts carry per-connection protocol behavior.

The runtime

At the top of the runtime picture is `core::SNodeC`.

`core::SNodeC` is not instantiated by application code. It exposes static lifecycle functions such as `init(...)`, `start(...)`, `stop()`, `tick(...)`, `free()`, and `state()`. That public surface is already enough to reveal an important design decision: initialization, event-loop progress, shutdown, and state inspection are explicit runtime concepts.

A typical application therefore begins by initializing the framework:

```
core::SNodeC::init(argc, argv);
```

and later enters runtime processing:

```
return core::SNodeC::start();
```

Between those two lines, the application creates communication roles and registers what they should do.

This is the reason `core::SNodeC::start()` is not a ceremonial final line. It is the point where registered communication work can be advanced by the framework runtime.

The instance

An instance is a configured communication role managed by the framework after registration. It participates in the `SNode.C` runtime and is advanced through the flow-controller machinery.

In everyday discussion, the visible `SocketServer` or `SocketClient` object in user code may also be called an instance. That is natural and often harmless. In the stricter architectural vocabulary used here, however, the visible C++ object is the application-side handle. Through `listen(...)` or `connect(...)`, that handle registers a configured communication role with the framework. That registered role is the instance. It is carried by framework-owned runtime and flow-controller state, and it must not be confused with the concrete peer connection that may appear later.

For example, an IPv4 stream legacy server object in `main()` is the handle through which the application names the role, adjusts configuration, attaches callbacks, and finally registers the listening role. The exact type name encodes lower-layer choices, but the architectural sequence is stable: handle, registered instance, runtime flow, connection, context.

The instance is not the application protocol.

It carries the application protocol into the runtime.

This distinction prevents a common beginner mistake. It is tempting to put all behavior into the server or client object because that object is visible in `main()`. In `SNode.C`, the server or client instance should usually describe the communication role. Protocol behavior belongs one step deeper, in the context attached to an actual peer connection.

Named instances deserve special attention.

A name such as `echoserver` or `echoclient` already carries architectural weight. Instance names become natural anchors for configuration, diagnostics, callbacks, and operational behavior. Later chapters will use that fact more heavily.

The connection

A connection is the concrete communication relationship to a peer.

In the stream-oriented parts of SNode.C, that role is represented by `SocketConnection` and its specializations. A connection exposes peer-oriented operations and observable state: local and remote addresses, send and read operations, shutdown and close operations, timeout handling, byte counters, and online timing.

That tells us something important about the framework.

SNode.C treats a connection as a visible runtime object with lifecycle and measurable behavior, not as an implementation detail hidden behind the protocol code.

This is also why connection-level callbacks and context-level callbacks must not be confused. Connection callbacks observe or adapt connection-level events. Context methods implement application protocol behavior for the connection.

The context

A context is the application protocol endpoint attached to one connection.

For stream communication, a user context derives from `core::socket::stream::SocketContext`. The base class gives the derived context operations such as `sendToPeer(...)`, `readFromPeer(...)`, timeout control, shutdown, close, and access to connection metrics.

The derived class supplies protocol behavior through lifecycle methods such as `onConnected()` and `onDisconnected()` and through protocol-specific receive handling.

This gives one of the most important rules in the book:

Instance/context boundary

The instance is the runtime-facing communication role; the context expresses protocol behavior for a concrete connection.

The echo pair used that rule in a very small form: the server and client handles registered roles, the framework advanced those roles through runtime flow, and the context performed the echo behavior once a connection existed. Larger applications follow the same boundary even when the protocol is HTTP, WebSocket, MQTT, or a custom stream protocol.

The factory

The factory creates contexts.

A `SocketContextFactory` exists because a context belongs to a connection, and connections appear dynamically. A server may accept many peers over time. Each peer needs its own protocol endpoint object. The framework therefore asks the factory to create a context when a connection needs one.

This is not accidental indirection.

It is the mechanism that connects a longer-lived communication role to shorter-lived per-connection protocol state.

Read the factory as the boundary between these two lifetimes:

```
configured instance
-> flow-controller machinery advances listen/connect behavior
    -> connections appear over time
        -> factory creates a fresh context per connection
```

Once this boundary is understood, the factory becomes one of the most natural parts of the design.

5.4 The normal startup-to-protocol flow

A SNode.C stream application can be read as a sequence of phases.

The exact classes vary by network family and connection mode, but the shape remains stable.

Phase 1: initialize the framework

The application prepares the framework runtime:

```
core::SNodeC::init(argc, argv);
```

This prepares the runtime environment and the surrounding framework machinery that later chapters will examine in more detail.

Phase 2: create one or more handles

The application creates server or client handles.

For a simple program, that may be one server and one client. For a real system, there may be several configured instances: perhaps an HTTP server, an MQTT client, a WebSocket bridge, or several independent communication endpoints.

The important point is that each instance represents a configured communication role, while the visible object is the handle used to configure and register that role.

Phase 3: register communication intent

A server calls `listen(...)`.

A client calls `connect(...)`.

These calls should be read as registration of intent. They set addresses, callbacks, backlog or peer information where appropriate, and enter the flow-controller path. The actual progress of the communication role belongs to the runtime and event system.

This distinction is central to SNode.C.

The application does not say:

Perform the whole network interaction here, synchronously, on this stack.

It says:

This communication role should listen or connect. Let the runtime advance that role.

Phase 4: start runtime processing

The application enters the runtime:

```
core::SNodeC::start();
```

From this point, registered roles can be advanced, event receivers can react to descriptor events and timers, connections can be established, data can arrive, callbacks can run, and contexts can perform protocol behavior.

Phase 5: connections appear

For a server, a connection appears when a peer is accepted.

For a client, a connection appears when the connection attempt succeeds.

This is where the abstract communication role becomes a concrete relationship to a peer.

Phase 6: the factory creates a context

The connection needs an application protocol endpoint.

The framework uses the configured factory to create the corresponding context. The context is then attached to the connection.

Phase 7: protocol behavior reacts to events

The context receives lifecycle calls and data-related callbacks. It can read from the peer, send to the peer, close the connection, stream data, set timeouts, or inspect metrics.

At this stage, the application protocol is alive.

The full flow can be summarized as:

```
init runtime
  -> create application-side handle
      -> configure and register instance
          -> start runtime
              -> establish connection
                  -> create context through factory
                      -> react to lifecycle and data events
```

That flow is one of the best compact mental models for SNode.C.

5.5 Lifetimes: role, handle, connection, context

A large part of SNode.C becomes clearer when lifetimes are separated carefully.

There are at least four different things that beginners may accidentally merge into one idea.

The local C++ handle

This is the local server or client object in user code.

It is the object the application uses to configure and register the role. Keeping it in scope is clear and often useful, but the local variable is not the whole runtime story.

The instance as configured communication role

This section fixes the strict meaning of the term. An instance is the role registered with the framework: listen here, connect there, use this configuration, use this factory, use these callbacks. After registration, it is no longer just an intention expressed by a local object; it is runtime-visible framework state.

From this point on, the book can usually say simply *instance*. The longer phrase remains useful when the definition itself is being emphasized, but repeating it everywhere would make the prose heavier without adding precision.

The instance is the long-lived entity in the framework model. The local server or client handle is the object through which the application configures and registers it, but the registered instance is what the runtime and flow-controller machinery keep advancing. Connections are shorter-lived concrete peer relationships produced by the instance; contexts are shorter-lived protocol endpoints attached to those connections.

The connection

This is the concrete relationship to one peer.

A server instance can produce many connections over time. A client instance may create a connection, lose it, and reconnect depending on configuration. A connection therefore has a different lifetime from the instance that produced it.

The context

This is the protocol object attached to one connection.

It should not be used as a global protocol singleton. It exists to hold per-connection protocol behavior and per-connection protocol state.

The lifetime rule is:

Lifetime rule

```
instance lifetime >= connection lifetime >= context lifetime
```

That rule is conceptual rather than a claim about exact ownership mechanics in every implementation detail. It is the mental ordering that matters. An instance can outlive one connection. A connection carries one context at a time. The context is meaningful only in relation to the connection it serves.

5.6 The layer model

The layer model answers the question:

What can vary without changing the whole application shape?

For practical work, it is useful to think in four levels.

```
network family
  -> transport form
      -> connection handling
          -> application protocol
```

Network family

The network family answers:

What kind of endpoint identity and addressing are used?

Examples include:

```
in    IPv4
in6   IPv6
un    Unix domain sockets
rc    Bluetooth RFCOMM
l2    Bluetooth L2CAP
```

IPv4 and IPv6 use IP addresses and ports.

Unix domain sockets use local socket identities.

Bluetooth RFCOMM and L2CAP use Bluetooth-specific endpoint concepts.

The details differ, but the application shape can remain familiar when the selected SNode.C layer exposes a compatible stream abstraction.

Transport form

The main teaching path in the early chapters is stream communication.

Stream communication gives the application a connection-oriented byte flow. That is why a context can think in terms of reading from and sending to a peer.

Other communication forms may appear in the framework, but the mental model in these chapters starts from stream-oriented communication first because it is the path that makes server, client, connection, factory, and context easiest to see.

Connection handling

Connection handling describes how a concrete peer relationship is managed.

This is where the `legacy` versus `tls` distinction appears in the stream hierarchy.

In this book, `legacy` means the non-TLS stream variant. It does not mean obsolete.

TLS changes important security properties: certificates, handshakes, validation, encrypted data flow, and additional failure modes. But TLS does not require the application writer to abandon the core instance/factory/context model.

That is why TLS often feels like a layer inserted below the application protocol rather than a completely different application architecture.

Application protocol

The application protocol is the behavior above the connection.

It may be tiny and custom, like the echo protocol. It may be HTTP. It may be WebSocket. It may be MQTT. It may be an Express-like application built on routing and middleware. It may be a bridge or integrator in a larger system.

The important point is that protocol behavior belongs above the lower communication layers.

That separation is what gives the framework its transfer value.

When a lower layer changes, the application writer asks:

What changed because the endpoint family changed?
 What stayed the same because the protocol shape stayed the same?

That question is more useful than treating every type name as an unrelated API.

5.7 Names are compressed architecture

The source-tree reading strategy from Chapter 4 continues here.

A name such as:

```
net::in::stream::legacy::SocketServer
```

is long because it compresses architecture.

Read it from left to right:

net	lower communication layer
in	IPv4 family
stream	stream-oriented transport
legacy	non-TLS connection mode
SocketServer	server-side role

The same logic applies to installed component names:

```
net-in-stream-legacy
net-in6-stream-tls
net-un-stream-legacy
```

Those names are not decorative. They encode layer decisions.

The same logic also applies to public include paths. A source file that directly names an IPv4 legacy stream server normally includes the public front door for that role:

```
#include <net/in/stream/legacy/SocketServer.h>
```

That path encodes the same stack as the namespace and the component name, but in the syntax of the C++ include tree.

This gives a useful reading habit:

Reading habit

When a `SNode.C` name feels long, do not shorten it mentally too early. First ask which decisions it records.

Often the long name tells you exactly where the type sits in the framework.

5.8 The operational model

The operational model answers the question:

How does the framework behave as a running system?

This is where configuration, callbacks, retry behavior, flow control, diagnostics, logging, and metrics become part of the mental model.

They are not secondary details.

A `SNode.C` instance is managed by the framework through runtime and flow-controller state; treating it as a plain C++ object that happens to own a socket misses the runtime model.

That has several consequences.

Configuration shapes behavior

Configuration is not an external side channel.

Server and client templates consult configuration while advancing the communication lifecycle. Addresses, backlog, retry behavior, retry limits, retry-on-fatal behavior, jitter, reconnect behavior, instance requirements, timeouts, and similar settings shape what the instance actually does.

This is why the book treats named instances and configuration as part of architecture, not merely as command-line convenience.

The better mental model is not:

The program has some flags.

The better mental model is:

Each named instance is a runtime-managed role with its own configuration and operational behavior.

That idea will become much more important in the configuration chapters.

Flow control is runtime behavior

The flow-controller path is what gives `listen(...)` and `connect(...)` their runtime semantics beyond the immediate system call.

A flow can be started, observed, retried, terminated, and associated with runtime-visible state. This is what allows a server or client role to behave operationally rather than just perform one procedural action.

For the reader, the important mental rule is:

Runtime-flow rule

`listen(...)` and `connect(...)` register instances; the runtime and flow-controller machinery advance them.

That rule is more precise than saying that these calls “start the server” or “open the connection,” although those informal phrases may be acceptable in casual discussion.

Connection callbacks and context callbacks are different layers

SNode.C exposes both connection-level callbacks and context-level lifecycle methods.

They are related, but they do different jobs.

Connection-level callbacks are associated with the server or client machinery. They are useful for observing, logging, measuring, supervising, or adapting connection-level behavior.

Context lifecycle methods belong to the protocol endpoint. They are where application behavior reacts to connection lifecycle and incoming data.

A useful rule is:

```
connection callback: observe or supervise the connection
context method:      implement the protocol behavior for that connection
```

This separation is especially important in larger systems.

If all protocol behavior is placed in instance-level callbacks, the code tends to lose the clean per-connection shape. If all operational supervision is hidden inside protocol contexts, the instance-level behavior becomes hard to observe and configure.

Metrics belong to the model

The connection and context abstractions expose quantities such as total sent, total queued, total read, total processed, online-since time, and online duration. Those values support diagnostics, runtime inspection, and operational reasoning.

They reflect the fact that SNode.C treats communication as observable runtime behavior. This becomes important for logging, diagnostics, flow control, backpressure discussions, deployment, and system-level behavior.

A framework that exposes these quantities is inviting the application writer to think operationally.

5.9 What stays stable across later chapters

The rest of the book will introduce many additional topics:

```
runtime internals
network families
socket addresses
server and client variants
contexts and factories in detail
configuration
logging
TLS
HTTP
Express-like routing
Server-Sent Events
WebSocket
MQTT
MQTTSuite
database support
deployment
testing
```

The details will change, but the mental model should remain recognizable.

When you meet a new subsystem, ask the same questions:

```
What is the runtime role?
What is the registered configured instance?
What lower layer does it use?
```

Where does the connection live?
 Where is the context or protocol endpoint?
 What creates it?
 What configuration shapes the flow?
 Which callbacks observe the connection, and which callbacks implement the protocol?

These questions are more valuable than memorizing a table of classes. They also help distinguish implemented behavior from design intention.

If the code shows a specific callback, ownership pattern, retry path, or component boundary, read that as implemented behavior. If the book recommends where to place protocol logic or how to separate supervision from protocol code, read that as architectural guidance based on the implemented framework shape.

5.10 A compact model to carry forward

The mental model can be summarized in one diagram:

```
core::SNodeC runtime
  -> application-side handle registers a server/client instance
    -> flow controller owns and advances listen/connect flow
      -> concrete peer connection
        -> SocketContextFactory
          -> per-connection SocketContext
            -> application protocol behavior
```

And the layer stack can be summarized as:

```
network family
  -> transport form
    -> connection handling
      -> application protocol
```

These two pictures belong together.

The first picture explains how the application becomes active at runtime. The second picture explains why the same application shape can be carried by different lower communication layers.

The operational model then adds the third dimension:

```
configuration
callbacks
retry behavior
flow control
diagnostics
metrics
```

Together they form the working mental model of SNode.C.

What to remember

- SNode.C is best understood as an event-driven, layer-based framework built from recurring roles.
- `core::SNodeC` owns the visible runtime lifecycle; `listen(...)` and `connect(...)` register instances that the runtime and flow-controller machinery advance.
- A server or client instance is the long-lived runtime-managed role; the visible `SocketServer` or `SocketClient` object is the application-side handle used to configure and register it.
- A connection is a concrete peer relationship, not the same thing as an instance.
- A `SocketContextFactory` creates per-connection contexts, and a `SocketContext` is where protocol behavior belongs.

- The practical layer stack is network family, transport form, connection handling, and application protocol.
- Configuration, callbacks, retry behavior, flow control, diagnostics, and metrics are part of the operational model, not decoration around it.

The next chapters will open the model from the inside. Chapter 6 looks more closely at the core runtime and event processing. Chapter 7 then turns the layer model into practical reading and design guidance for network, transport, connection, and application boundaries.

Chapter 6

Core Runtime and Event Processing

6.1 Why this chapter matters

The first working echo pair introduced the runtime shape, and the mental model chapter named its recurring roles: handles, instances, connections, contexts, factories, layers, and operational concerns. It also made the central distinction precise: the visible `SocketServer` or `SocketClient` object is the application-side handle, while the instance is the registered communication role advanced by the framework.

That model is necessary, but it still leaves an important question open.

How does the framework actually keep everything moving?

A server can register an intention to listen. A client can register an intention to connect. A context can define reactions such as `onConnected()` and `onReceivedFromPeer()`. A timer can be armed. A descriptor can become readable. A timeout can expire. Something has to coordinate all of that.

That “something” is the runtime core. This chapter therefore moves from application shape to runtime machinery. It stays close to the `SNode.C` source structure, but it describes that structure in a teaching-oriented way: as a small number of cooperating runtime concepts.

The key insight is this:

`SNode.C` does not treat event processing as a hidden implementation detail. It gives it a clear architecture.

That architecture has several visible pieces:

- `core::SNodeC` as the public runtime control surface,
- `core::EventLoop` as the singleton orchestrator,
- `core::EventMultiplexer` as the waiting and dispatch backbone,
- descriptor event publishers and receivers for file-descriptor activity,
- timer publishers and timer handles for scheduled activity,
- and an event queue for deferred execution.

Once those pieces are placed correctly in the reader’s mind, the framework’s behavior stops feeling magical. Registered instances are not advanced by scattered blocking calls. They are advanced by a coordinated event-processing core.

6.2 The runtime picture in one diagram

Before looking at the individual parts, it helps to place the event-processing core in one visual model. As shown in Figure 6.1, the public runtime surface leads into the event loop and multiplexer, which coordinate descriptor readiness, timers, queued work, and dispatch.

The figure is an orientation map. It places the major runtime ideas in relation to each other while leaving implementation order, call paths, and lower-level scheduling details to the source code.

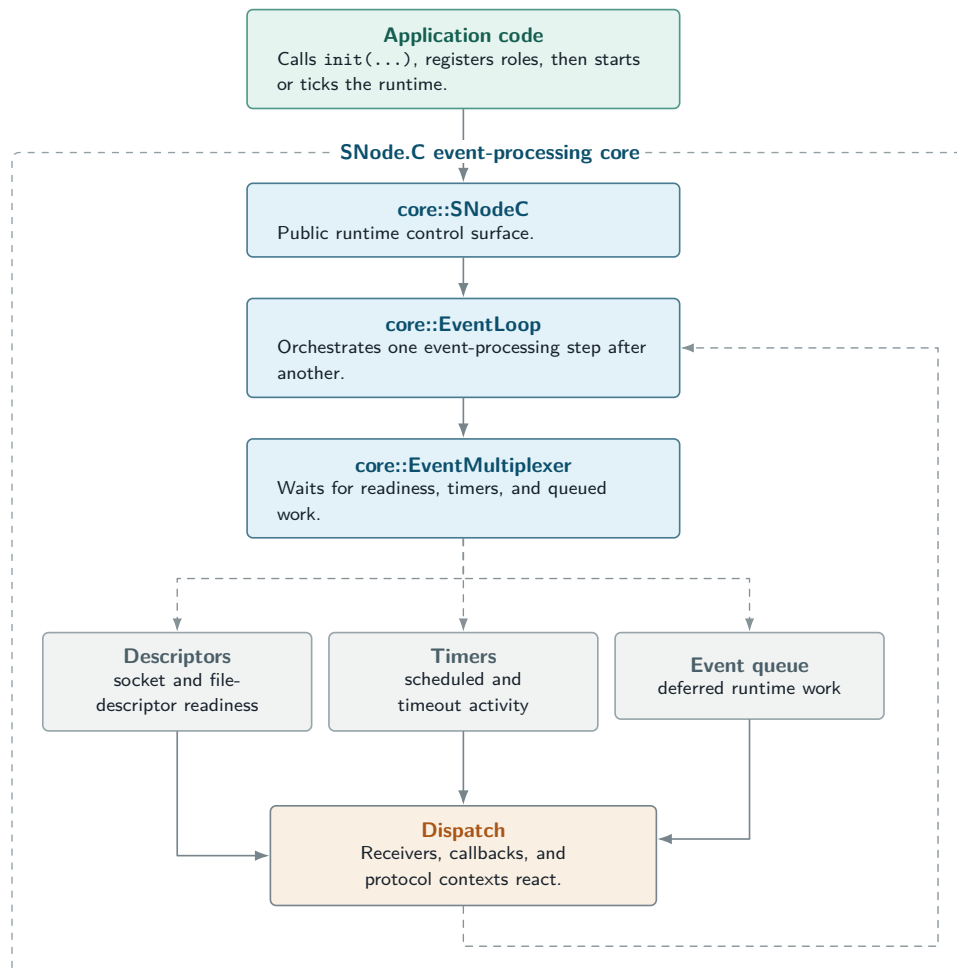


Figure 6.1: The SNode.C event-processing core connects runtime control, descriptor readiness, timers, queued work, and protocol dispatch.

The application normally talks to `core::SNodeC`. `core::SNodeC` forwards runtime control to the event loop. The event loop delegates low-level waiting and dispatch coordination to the multiplexer. The multiplexer brings together descriptor readiness, timers, queued events, timeout processing, signals, and cleanup.

That is the backbone of the chapter.

6.3 A short source-level reading

The implementation follows the same shape. The excerpts below are abridged from the pinned `SNode.C v1.0.2` source in `src/core/SNodeC.cpp`, `src/core/EventLoop.cpp`, and `src/core/EventMultiplexer.cpp`. They are not a second model of the runtime; they are source anchors for the model used in this chapter.

First, the public facade really is a facade. `core::SNodeC` forwards runtime control to `core::EventLoop`:

```
int SNodeC::start(const utils::Timeval& timeOut) {
    return EventLoop::start(timeOut);
}

TickStatus SNodeC::tick(const utils::Timeval& timeOut) {
    return EventLoop::tick(timeOut);
}

State SNodeC::state() {
    return EventLoop::getEventLoopState();
}
```

Second, `EventLoop::start(...)` bootstraps configuration, enters the running state, and then advances the runtime through repeated calls to `_tick(...)` while the loop remains active:

```
if (utils::Config::bootstrap()) {
    eventLoopState = State::RUNNING;

    do {
        tickStatus = EventLoop::instance()._tick(timeOut);
    } while ((tickStatus == TickStatus::SUCCESS ||
             tickStatus == TickStatus::INTERRUPTED) &&
             eventLoopState == State::RUNNING);
}
```

Third, a multiplexer tick has the expected coordination shape: wait, publish active work, execute queued work, check timeouts, and release expired resources:

```
const TickStatus tickStatus = waitForEvents(...);

if (tickStatus == TickStatus::SUCCESS) {
    spanActiveEvents(...);
    executeEventQueue(...);
    checkTimedOutEvents(...);
    releaseExpiredResources(...);
}
```

Those excerpts are deliberately small. They show that the diagram's main path is not only a teaching abstraction: facade, event loop, multiplexer, descriptor readiness, timers, queued work, timeout handling, and cleanup are visible as source-level responsibilities.

6.4 The public runtime surface: `core::SNodeC`

The first runtime-facing type most users encounter is `core::SNodeC`.

Its public surface is small:

- `init(int argc, char* argv[])`
- `start(const utils::Timeval& timeout = {LONG_MAX, 0})`
- `stop()`
- `tick(const utils::Timeval& timeout = 0)`
- `free()`
- `state()`

This compact interface already teaches several important things.

The runtime has explicit phases

The existence of `init`, `start`, `stop`, `free`, and `state` tells us that the framework runtime has an explicit lifecycle instead of being a side effect of object construction.

That is a design strength. It reduces ambiguity for applications and gives the reader a stable mental timeline. A typical application does not simply create a server and hope that background machinery appears. It initializes the framework, registers communication roles, and then starts the runtime.

Event-loop control is centralized

The existence of `start()` and `tick()` tells us that `SNode.C` treats the event loop as a central framework capability rather than as behavior buried inside one particular server or client type.

That means sockets, timers, retries, reconnects, and protocol contexts participate in one runtime. A `SocketServer` is not an independent miniature event loop. A `SocketClient` is not an independent miniature event loop. Both register work that is advanced by shared runtime machinery.

`start()` is the normal ownership mode

For most applications, `core::SNodeC::start()` is the normal mode. It means:

Let the `SNode.C` runtime run the event loop until it is stopped or until the configured timeout condition ends the run.

That is the style already used in the echo pair. It is the natural mode when `SNode.C` is the central network runtime of the process.

`tick()` is the controlled stepping mode

The presence of `tick(...)` shows that `SNode.C` is not limited to one usage pattern. It can run in the classical “start the event loop and let it own the thread” mode, but it also exposes a per-iteration stepping model.

A framework that offers `tick()` is telling you:

The event loop is real, structured, and externally stepable.

That matters for testing, embedding, integration with another outer loop, or simply understanding what the runtime does one iteration at a time.

6.5 The orchestrator behind the facade: `core::EventLoop`

Although most application code touches `core::SNodeC`, the actual loop orchestrator is `core::EventLoop`.

Its interface exposes several key facts:

- it is a singleton through `instance()`,
- it owns or exposes an `EventMultiplexer`,
- it tracks a tick counter,
- it tracks runtime state,
- and it is the actual home of `init`, `tick`, `start`, `stop`, and `free`, with `SNodeC` acting as the public facade.

This is a useful separation in a systems framework. The application sees a compact facade. The runtime internals still have a concrete orchestrator object.

That separation keeps everyday use simple without flattening the underlying architecture.

Why a singleton event loop makes sense here

The singleton event loop is a tradeoff.

On the positive side, it gives the framework one primary event domain per process. That simplifies ownership, coordination, and runtime reasoning. For the applications emphasized in this book, that is helpful: the reader does not need to imagine several unrelated event loops competing inside the same process. There is one central runtime story.

The tradeoff is equally important. The framework is not centered around multiple independent reactor domains inside one process. That is a real architectural choice, and it is better to understand it early. For the kind of layered network applications SNode.C is built to express, one central event domain is a clarifying design.

6.6 Runtime state is coarse on purpose

The `core::State` enum is small:

- `LOADED`
- `INITIALIZED`
- `RUNNING`
- `STOPPING`

At first glance, this may seem too simple. For operational reasoning, it is useful precisely because it is coarse. It gives the framework a phase language without turning ordinary users into internal state-machine archaeologists.

A reader can think in a clean timeline:

1. the framework is loaded,
2. it is initialized,
3. it is running,
4. it is stopping.

That is enough to reason about many important behaviors. Server and client flows are not isolated library calls. They integrate with runtime state. A listen or connect flow can only move forward meaningfully when the runtime is in a phase where such work can be processed.

This sharpens what Chapters 3 and 5 introduced:

Runtime-state rule

Runtime state is not background decoration. It is part of the control logic.

6.7 Tick-by-tick execution and `TickStatus`

If `State` describes coarse lifecycle phases, `TickStatus` describes the result of a single event-loop iteration.

`TickStatus` defines:

- `SUCCESS`
- `INTERRUPTED`
- `NOOBSERVER`
- `TRACE`

This enum is more informative than it may first appear.

`SUCCESS` means the iteration completed normally. In a tick-driven or embedded integration scenario, that allows the caller to distinguish ordinary progress from exceptional control outcomes.

INTERRUPTED matters because an event loop can be interrupted by signals or other control conditions. Exposing this as a separate status keeps interruption explicit instead of silently collapsing it into generic success or failure.

NOOBSERVER is architecturally revealing. It says that the event-loop iteration can recognize the absence of observed runtime participants. Descriptor receivers, timers, and queued work form a population of things the runtime can observe or advance. The runtime is therefore not looping blindly. It is managing a set of event participants.

TRACE is part of the per-tick vocabulary, but it should not be mistaken for a full tracing subsystem. The important point for this chapter is narrower: per-tick outcomes are explicit enough to be named and propagated.

That is a useful rule for reading the core runtime:

Header names reveal design vocabulary, but the architectural meaning should be taken from how the pieces cooperate.

6.8 From handles to runtime-visible instances

Chapter 5 separated the application-side handle from the runtime-visible instance.

That distinction becomes concrete in the stream server and client code. The local **SocketServer** or **SocketClient** object used in application code is the handle through which the application names, configures, and registers a communication role. After **listen(...)** or **connect(...)**, the runtime-visible instance is carried by shared configuration, shared context, and flow-controller state.

The application begins with local expressions such as:

```
server.listen(...);
client.connect(...);
```

But the runtime story does not stop on that stack frame. The public call enters the real **listen/connect** path, that path uses the flow controller, and the flow controller schedules the actual work for an event-loop turn.

For stream servers and clients, the practical path is:

```
listen(...) / connect(...)
  -> realListen(...) / realConnect(...)
      -> flowController.startFlow(...)
          -> EventReceiver::atNextTick(...)
```

This source-level path is the runtime version of the mental model introduced earlier:

```
application-side handle
  -> register instance
      -> runtime and flow-controller machinery advance the instance
          -> concrete connections appear later
```

That distinction is not cosmetic. The server and client templates keep the runtime-relevant state in shared context objects, and the scheduled flow captures that shared state. The event loop therefore does not depend on the caller's stack frame to advance the registered instance. The visible local object is the registration handle; the framework advances the instance through shared state, flow-controller machinery, descriptor receivers, timers, the multiplexer, and queued event work.

This prevents the wrong intuition that **SNode.C** is a blocking socket wrapper tied to one caller stack.

6.9 From runtime intent to event delivery

To understand event processing, we need to distinguish between two different meanings of “event” in **SNode.C**:

- a **runtime scheduling event**,
- and a **descriptor or timer readiness event**.

These are related, but not identical.

Scheduling events

The `core::EventReceiver` API makes one important mechanism directly visible:

```
core::EventReceiver::atNextTick(...);
```

This is the framework’s way of saying:

Deferred-work warning

Do not do this immediately on the caller’s stack. Queue it for an event-loop turn.

The implementation creates a temporary event receiver, spans it into the event system, invokes the stored callback from `onEvent(...)`, and destroys the temporary receiver after execution. The exact caller path depends on the surrounding flow, but the architectural meaning is stable: operational work can be moved into the runtime domain instead of being executed immediately by arbitrary user call stacks.

That produces cleaner phase boundaries and more predictable sequencing.

Readiness events

A descriptor may become readable, writable, exceptional, or timed out. A timer may expire. A signal may arrive. These are not “next-tick callbacks” in the same sense. They are external or scheduled conditions that the runtime notices and then dispatches.

So the mental model should be:

- `atNextTick(...)` is how framework or user code can enqueue work into the loop;
- descriptor and timer mechanisms are how the loop notices the outside world and scheduled time.

Both meet in the same runtime, but they enter it through different doors.

6.10 EventReceiver and Event: the small but important abstraction pair

The `EventReceiver` and `Event` headers are compact, but together they reveal a lot about the runtime’s shape.

An `EventReceiver`:

- has a name,
- exposes `onEvent(const utils::Timeval& currentTime)`,
- can `span()` and `relax()`,
- and supports `atNextTick(...)`.

An `Event`:

- is associated with an `EventReceiver`,
- has a name,
- can `span()` and `relax()`,
- can be `dispatch(...)`,
- and is connected to the `EventMultiplexer`.

The cleanest way to think about this pair is:

- the **receiver** is the runtime-facing behavior endpoint;
- the **event** is the thing that becomes queued and dispatched for that receiver.

That is a useful split. It means `SNode.C` can talk about *who handles work* and *what gets published into the loop* as related but distinct concepts.

What `span()` and `relax()` suggest

Both `EventReceiver` and `Event` expose `span()` and `relax()`-style operations. The names may feel unusual at first, but conceptually they fit the framework.

At the level of the runtime vocabulary, these names suggest a lifecycle-oriented model. Runtime work can be published into, withdrawn from, and dispatched by the event system. The reader does not need to memorize every internal method here. The important point is the shape of the runtime: work is represented, published, dispatched, and eventually released.

6.11 The multiplexer is the coordination backbone

At the center of event processing sits `core::EventMultiplexer`. This is the most important internal runtime concept in this chapter.

A good teaching summary is:

One tick is the multiplexer coordinating descriptor readiness, timers, queued events, timeout processing, signals, and cleanup.

The multiplexer is the runtime hub that combines several kinds of progress into one coordinated event-loop iteration, rather than a thin wrapper around `epoll`, `poll`, or `select`.

Its structure shows that it:

- owns access to three descriptor event publisher channels,
- owns a timer event publisher,
- maintains an internal event queue,
- supports spanning and relaxing events,
- can receive signals and termination requests,
- computes timeouts,
- waits for events,
- spans active events,
- executes the event queue,
- checks timed-out events,
- and releases expired resources.

That is a richer mental picture than “the multiplexer waits for file descriptors.”

Why there are three descriptor channels

The multiplexer distinguishes three descriptor categories:

- RD
- WR
- EX

These correspond to readable, writable, and exceptional descriptor activity. The runtime does not treat all descriptor activity as one undifferentiated stream. It keeps distinct observation channels for the major categories of interest.

That matters for network systems because read readiness, write readiness, and exceptional conditions often mean different things. A readable socket may allow protocol input processing. A writable socket may allow queued output to drain. An exceptional condition may require error handling or shutdown.

Keeping these channels distinct makes the runtime more expressive.

The event queue inside the multiplexer matters

The embedded `EventQueue` inside `EventMultiplexer` is especially revealing. It has operations for insertion, removal, execution, and clearing. It also uses separate publish and execute queues.

The architecture tells us that queued event work is a first-class part of a loop tick, not an ad-hoc side mechanism. This helps explain why `atNextTick(...)` can be such a clean primitive: there is a defined place in the loop structure for queued runtime work.

Multiplexer implementations are replaceable at the lower level

The conceptual role of the multiplexer is stable, but the concrete waiting mechanism can vary. The source tree separates lower-level multiplexer backends for mechanisms such as `epoll`, `poll`, and `select`.

For the reader, the important point is not to memorize a backend first. The important point is the architectural contract:

The multiplexer is the place where observed descriptors, timers, queued work, timeout processing, signals, and cleanup meet.

That contract is what later socket and protocol layers rely on.

6.12 Observed event sources

Once the multiplexer is understood as the coordination backbone, the next question is:

What exactly does it observe?

For this chapter, the two most important groups are descriptor event sources and timer event sources.

Descriptor event sources connect the runtime to file descriptors. Timer event sources connect the runtime to scheduled time. Both are managed as runtime participants, not as unrelated utilities.

6.13 Descriptor event publishers: managing observed descriptor populations

A descriptor event publisher manages a population of observed descriptor receivers.

It can:

- `enable(...)`
- `disable(...)`
- `suspend(...)`
- `resume(...)`
- `spanActiveEvents()`
- `checkTimedOutEvents(...)`
- `releaseDisabledEvents(...)`
- `signal(...)`
- disable the whole publisher

It also maintains maps of observed receiver lists keyed by descriptor.

This is a strong clue about the internal runtime philosophy. `SNode.C` does not think of descriptor handling merely as:

The loop calls my callback when `fd` is ready.

Instead, it thinks in terms of **managed observed populations**.

Receivers can become observed, temporarily suspended, resumed, timed out, disabled, or released. This is a more operationally mature model than a naive reactor sketch.

Why suspend/resume is different from enable/disable

Even without walking through every implementation detail, the distinction between suspend/resume and enable/disable is conceptually valuable.

- **enable/disable** suggests entering or leaving the observed population;
- **suspend/resume** suggests remaining conceptually present while temporarily inactive.

That distinction becomes important in real systems where backpressure, staged activity, retry delays, or temporary quiescence must be represented cleanly. A receiver may still exist as part of the runtime model even when it is not meant to produce events for a while.

6.14 Descriptor event receivers: behavior attached to observed descriptors

If publishers manage descriptor-facing populations, receivers express the behavior of one observed descriptor participant.

A descriptor receiver:

- derives from `EventReceiver`,
- knows whether it is enabled or suspended,
- can be enabled on a file descriptor,
- can be disabled, suspended, and resumed,
- can have timeout behavior,
- and must implement runtime reactions such as `dispatchEvent()`, `timeoutEvent()`, and `signalEvent(int)`.

This tells us how the descriptor side of the runtime fits together.

A descriptor receiver is not “just a callback object.” It is a runtime participant with observation state, timeout state, signal behavior, and dispatch behavior. That is why the framework can support a wide range of connection and socket workflows while still feeling regular. The same basic runtime pattern is reused with different specialized receivers.

Publisher and receiver side by side

A small table helps to keep the distinction clear.

Runtime object	Main responsibility	Simple mental rule
<code>DescriptorEventPublisher</code>	Manages the observed population for a descriptor channel	Decides <i>who is being observed</i>
<code>DescriptorEventReceiver</code>	Defines behavior for one observed descriptor participant	Decides <i>what happens when observation produces work</i>

This distinction will matter again when socket acceptors, connectors, readers, and writers enter the picture. They are specialized runtime participants managed by the event system, not anonymous callbacks attached to file descriptors.

6.15 Timers are not bolted on

A good runtime design treats timers as first-class event sources. `SNode.C` does exactly that.

The `TimerEventPublisher` maintains a timer set, supports insertion and removal, computes the next timeout, spans active events, and can stop the timer publisher itself.

The `core::Timer` base class is a small ownership-oriented handle around a timer event receiver. It offers operations such as `cancel()` and `restart()`. On top of that, `core::timer::Timer` provides concrete creation helpers such as:

- `intervalTimer(...)`
- `singleshotTimer(...)`

This means the timer story fits naturally into the broader runtime story. A timer is not a foreign utility living outside the event loop. It is part of the loop’s own scheduled event fabric.

Timers share the same central loop worldview

A timeout does not require a second runtime model. It is simply another kind of event source participating in the same tick progression.

The multiplexer computes timeout information together with descriptor observation and queued event work. That is why time-based behavior can be integrated without inventing a second scheduler beside the event loop.

Retry and reconnect features fit naturally

Chapter 5 introduced retry and reconnect behavior as operational concerns of configured instances. In the stream flow-controller path, that now becomes concrete. Retry and reconnect need time, and the runtime already has first-class timer support.

Therefore retries and reconnects do not need to be improvised as sleeps, blocking loops, or external threads. They can be represented as scheduled runtime work that belongs to the same event-loop world as descriptor activity and callbacks.

This is the architectural reason why retry, reconnect, and timeout behavior feel like part of `SNode.C` rather than add-ons.

6.16 One tick, conceptually

With the main pieces now placed, we can describe one conceptual event-loop iteration. This is not a line-by-line implementation trace. It is the teaching model that best fits the implementation structure.

The central sentence is:

One tick is the multiplexer coordinating descriptor readiness, timers, queued events, timeout processing, signals, and cleanup.

The loop begins a tick

The event loop begins one iteration and has a timeout budget. If the application called `start()`, this happens as part of the continuous runtime loop. If the application called `tick()`, this happens as one explicit step controlled by the caller.

The multiplexer determines what to wait for

The multiplexer considers descriptor activity and timer deadlines. It computes the next relevant timeout and prepares to monitor the registered descriptor sources. This is where descriptor observation and timer observation meet.

External readiness or timer expiration occurs

Descriptors may become readable, writable, exceptional, or timed out. Timers may become due. Signals may arrive. The runtime now has work to publish or dispatch.

Active events are spanned

The multiplexer spans active events and publishes them toward the relevant receivers. The concept is stable: observed runtime participants that are ready for work are brought into the event-processing path.

The event queue is executed

Queued work, including next-tick callbacks and other internal runtime events, is executed. This is where scheduled framework work can run outside the original caller's stack.

It is also the reason `listen(...)` and `connect(...)` can register intent rather than doing the whole operation immediately.

Timeouts and cleanup are checked

The loop checks for timed-out observed entities and releases expired or disabled resources. Event-driven systems must not only react to positive readiness. They must also notice inactivity, timeout, cancellation, disablement, and cleanup.

The tick returns a status

The loop iteration returns a `TickStatus`.

For normal `start()` usage, this status is usually part of the runtime's internal loop control. For explicit `tick()` usage, it becomes visible to the caller and can guide embedding, testing, or controlled integration.

6.17 What application authors should learn from this

This chapter has looked under the visible application API. The purpose is not to turn every application writer into a maintainer of the runtime core. The purpose is to develop the right instincts.

Do not block inside callbacks

Callbacks run inside the runtime's event-processing world. If a callback blocks for a long time, it prevents the runtime from processing other descriptors, timers, queued events, retries, reconnects, timeout checks, and cleanup work.

A callback should do the immediate protocol work it is responsible for, preserve its invariants, and then return control to the event loop. Long-running work must be designed carefully.

Treat callbacks as runtime-owned execution points

A callback is a runtime-owned execution point, not an ordinary helper function called by your own code at a predictable place on your own stack.

That means callback code should be clear about ownership, reentrancy assumptions, and the amount of work it performs before returning.

Use timers instead of manual sleeps

If you need to delay work, retry later, or run periodic activity, the runtime timer model is the natural place.

A manual sleep inside a callback is almost always the wrong instinct. It stops the event loop from doing the work it exists to do.

Remember that `listen(...)` and `connect(...)` register intent

The public call configures and registers an instance. The flow-controller/shared-context path and the event loop advance the actual work.

This wording marks the difference between reading `SNode.C` as a blocking socket wrapper and reading it as an event-driven runtime.

Understand where descriptor activity, timers, and queued work meet

Descriptor readiness, timer expiration, and next-tick scheduling are different sources of activity. But they meet in the same event-loop architecture.

The `EventMultiplexer` is the central place where they are coordinated. That is the core runtime picture to carry forward.

Use `tick()` deliberately

Most applications should simply call `core::SNodeC::start()`.

Use `tick()` when you deliberately want controlled stepping: for embedding, testing, integration, or inspection. The existence of `tick()` is valuable, but it is not a reason to overcomplicate a normal application.

6.18 Stable concepts versus implementation details

This chapter has intentionally stayed close to the `SNode.C` headers. That is useful because `SNode.C` is a systems framework, and systems frameworks should not be taught as vague abstractions only.

Still, a book should distinguish stable concepts from implementation details. Function bodies may change. Additional event receiver types may appear. Multiplexer backends may vary. But the architectural concepts are durable:

- public runtime control through `core::SNodeC`,
- event-loop orchestration through `core::EventLoop`,
- multiplexing through `core::EventMultiplexer`,
- descriptor event publishers and receivers,
- timer event publishing,
- queued runtime work,
- timeout processing,
- cleanup,
- and per-tick progress.

Two stable distinctions matter especially for the rest of the book.

The first is the distinction between a handle and an instance. The visible server or client object is the application-side handle. The registered instance is advanced by shared runtime and flow-controller machinery.

The second is the distinction between an instance and a connection. The instance is the long-lived communication role. A connection is a concrete peer relationship that appears through runtime progress.

Those are the ideas the reader should carry forward.

What to remember

- `core::SNodeC` is the public runtime facade; `core::EventLoop` is the central event-loop orchestrator behind it.
- `core::EventMultiplexer` coordinates descriptor readiness, timers, queued work, timeout checks, signals, and cleanup.
- `listen(...)` and `connect(...)` do not perform the whole operation on the caller's stack; they enter the flow-controller path and schedule runtime work.
- The visible server or client object is an application-side handle; the registered instance is advanced through shared state and flow-controller machinery.
- Descriptor receivers and timer receivers are managed runtime participants with enable/disable, suspend/resume, timeout, and cleanup behavior.
- Application callbacks should return control to the event loop rather than blocking the runtime.

6.19 Closing perspective

The runtime core is the reason the higher layers of `SNode.C` can remain regular.

A server can register a listening instance because the runtime can later advance that instance. A client can register a connecting instance because the runtime can later progress the connection attempt. A context can express protocol behavior because the runtime knows when to call it.

Retries and reconnects can exist as part of the model because timers are first-class event sources. Descriptor readiness can become protocol progress because descriptor receivers participate in a managed event system.

This is why the event loop is not a footnote. It is the machinery that turns the framework's architecture into motion.

The reader does not need to memorize every internal method in this chapter. But the reader should leave with a durable picture:

```
SNodeC facade
-> EventLoop
  -> EventMultiplexer
    -> descriptor readiness
    -> timer progression
    -> queued runtime work
    -> timeout handling
    -> cleanup
```

That picture will support the next chapters. When we later discuss socket layers, connection objects, contexts, HTTP, WebSocket, MQTT, and system-level applications, the same runtime story will still be underneath.

SNode.C applications are not driven by scattered blocking calls.

They are driven by a coordinated event-processing core.

Chapter 7

Layers in Practice: Network, Transport, Connection, Application

7.1 Why this chapter comes here

Chapter 5 introduced the mental model of SNode.C.

Chapter 6 then explained how the runtime advances registered instances. Descriptor readiness, timers, queued work, callbacks, and flow-controller progress are not scattered side effects. They are coordinated by the runtime core.

This chapter turns to a different question:

What kind of communication structure is the runtime advancing?

The answer is the communication layer stack.

SNode.C organizes communication concerns into a practical sequence:

```
network family
  -> transport form
    -> connection handling
      -> application protocol
```

This is not a decorative diagram. It explains why long SNode.C names are readable and why components can be selected systematically. It also explains why the same application shape can move from IPv4 to IPv6, from a Unix domain socket to Bluetooth RFCOMM or L2CAP, or from the non-TLS stream variant to TLS without becoming unrelated code. Higher protocols such as HTTP, WebSocket, MQTT, and MQTT over WebSocket add their own structure, but they do not make the lower stack disappear.

Chapter 5 introduced this stack as part of the general mental model. Chapter 6 showed how registered instances make progress inside the runtime. It explains how the communication choices behind those instances are named, built, and composed.

The goal is not to repeat a textbook theory of layering. The goal is more practical: after this chapter, a reader should be able to look at a SNode.C type name, component name, or source-tree path and understand what it says about the communication stack.

7.2 The communication stack in four layers

For practical work, SNode.C can be read through four communication layers.

Layer	Main question	Examples
Network	Which endpoint family are we using?	IPv4, IPv6, Unix domain sockets, Bluetooth RFCOMM, Bluetooth L2CAP
Transport	What communication form is assumed?	stream
Connection	How is the concrete peer relationship handled?	legacy , tls
Application	What protocol behavior runs above the connection?	custom context, HTTP, WebSocket, Express-like routing, MQTT

This four-layer view is not a replacement for the runtime model from Chapter 6.

The runtime model explains how work is scheduled and dispatched. It talks about the event loop, descriptor readiness, timers, queued work, timeout processing, signals, and cleanup.

The communication layer model explains what kind of communication structure that runtime activity belongs to. It answers different questions: Which endpoint family? Which transport form? Which connection machinery? Which application protocol?

That distinction is important because both models meet in every real application.

A registered instance is advanced by the runtime, but it is not a purely abstract runtime label. It is carried by some concrete stack: perhaps an IPv4 stream legacy client, an IPv6 TLS server, a Unix-domain HTTP endpoint, a Bluetooth RFCOMM stream endpoint, a WebSocket endpoint, an MQTT participant, or another combination. The runtime advances the instance; the layer stack tells us what kind of communication that instance represents.

A compact separation is:

Concern	Where it belongs
Event loop, descriptor readiness, timers, queued work	Runtime core
IPv4 versus IPv6 versus Unix versus Bluetooth	Network layer
Stream-oriented peer communication	Transport layer
Non-TLS versus TLS connection handling	Connection layer
HTTP, WebSocket, MQTT, or custom protocol behavior	Application layer
Factory and context classes	Application-facing protocol endpoint machinery

This prevents a common confusion. The runtime tells us *how progress happens*. The communication layer stack tells us *what kind of communication structure is making progress*. Each row in the table becomes visible later in type names, component names, configuration choices, and source-tree paths. In SNode.C that stack appears not only in type names and component names, but also in public include paths.

7.3 Names are maps of architecture

The fastest practical way to understand SNode.C layering is to read names from left to right.

A long SNode.C name is usually not long by accident. In many cases, it is a compact description of the communication stack. The name identifies a class, but it also records a set of architectural decisions. Reading such names structurally is one of the quickest ways to stop seeing them as noise.

Consider this type:

```
net::in::stream::legacy::SocketClient<MyFactory>
```


Read it as a sentence:

```
network-facing code
-> IPv4 family
    -> stream transport
        -> non-TLS stream connection variant
            -> client-side handle type
                -> factory for per-connection contexts
```

The visible `SocketClient` object in application code is still the handle, in the sense established in Chapters 5 and 6. It is the object through which the application names, configures, and registers the client-side instance. The name of the type tells us which lower-layer choices that instance will use when the runtime advances it.

A similar server type can be read the same way:

```
net::rc::stream::tls::SocketServer<MyFactory>
```

This says:

```
network-facing code
-> Bluetooth RFCOMM family
    -> stream transport
        -> TLS connection handling
            -> server-side handle type
                -> factory for per-connection contexts
```

Once this habit is learned, long names become useful instead of intimidating. They are not random C++ namespace growth. They are compact layer descriptions.

A useful reading rule is:

Layer-reading rule

Read a `SNode.C` communication type as a stack description before reading it as an isolated API name.

Component names and public include paths tell the same story

`SNode.C`'s installable CMake components encode the same architectural discipline. Public include paths encode the C++ source-side version of that same discipline.

A component such as:

```
net-in-stream-legacy
```

can be read as:

```
net      network-facing component family
in       IPv4
stream   stream transport
legacy   non-TLS stream connection variant
```

The corresponding public include path uses slashes instead of dashes and ends in the concrete public role header:

```
#include <net/in/stream/legacy/SocketServer.h>
#include <net/in/stream/legacy/SocketClient.h>
```

Likewise:

```
net-in6-stream-tls
```

means:

```
IPv6
  -> stream
    -> TLS connection handling
```

The source-tree view, public include view, namespace/type view, and component view are not identical. They serve different technical purposes, but they express the same idea.

A public include path such as `<net/in/stream/legacy/SocketServer.h>` tells the C++ preprocessor and reader which public front door is being used. A namespace such as `net::in::stream::legacy` helps the C++ reader locate a type in the layer stack. A component such as `net-in-stream-legacy` helps the CMake user select the corresponding build product.

These names are examples of the naming discipline, not a vocabulary test. The reader does not need to memorize every component in one sitting. The useful habit is to ask:

Which layer decisions are encoded in this name?

That question turns long names into maps.

7.4 The network layer: endpoint identity

The network layer answers the first concrete communication question:

Which kind of endpoint identity are we using?

This is where `SNode.C` chooses among lower communication families.

Family	Namespace fragment	Typical endpoint identity
IPv4	<code>net::in</code>	IPv4 address and port
IPv6	<code>net::in6</code>	IPv6 address and port
Unix domain sockets	<code>net::un</code>	local socket path or local socket identity
Bluetooth RFCOMM	<code>net::rc</code>	Bluetooth device address and RFCOMM channel
Bluetooth L2CAP	<code>net::l2</code>	Bluetooth device address and L2CAP PSM

This side-by-side treatment is one of the important architectural features of `SNode.C`.

It does not mean these families are identical. They are not. IPv4 and IPv6 use network addresses and ports. Unix domain sockets live in a local operating-system namespace. Bluetooth RFCOMM and L2CAP use Bluetooth-specific addressing and deployment assumptions. Binding, connecting, permissions, diagnostics, and operational behavior can differ significantly.

The design point is subtler:

Endpoint-semantics note

`SNode.C` reuses the application shape without pretending that endpoint semantics are the same.

That is exactly the balance a layered framework should aim for. It lets the reader carry a mental model from one family to another, while still respecting family-specific facts.

Address semantics belong here

The network layer becomes concrete through address types and address semantics.

An IPv4 endpoint is not the same thing as an IPv6 endpoint. A Unix domain socket path is not the same thing as a Bluetooth device address plus channel or PSM. SNode.C does not erase those differences behind one vague endpoint abstraction.

That is good design.

A framework that hides all endpoint identity behind a single general address type may look simpler at first, but the simplification can become misleading. Real systems need to know whether they are binding a TCP port, opening a local socket path, or addressing a Bluetooth service.

SNode.C keeps the family-specific meaning visible while still preserving the higher-level pattern.

Once the endpoint family is chosen, the next question is what kind of communication relationship that family will carry. Before going there, however, the family-specific address meaning deserves its own treatment. Therefore, Chapter 8 follows naturally from this chapter. Once the reader understands that the network layer chooses an endpoint family, the next question is unavoidable:

What exactly does an address mean in each family?

That is the topic of socket addresses and address semantics.

7.5 The transport layer: communication form

Above the network family sits the transport form.

For the early and central parts of this book, the important transport form is:

```
stream
```

A stream transport gives the framework a stable model for connection-oriented byte communication. It lets SNode.C speak about establishing a peer relationship, reading and writing byte sequences, observing connection lifetime, shutting down, applying timeouts, and inserting TLS below the application protocol.

This does not mean every communication form is the same. It means that the stream abstraction gives many SNode.C examples a common conceptual base.

The network layer decides what kind of endpoint identity is used. The transport layer decides what kind of communication relationship is assumed.

When these two decisions are combined, the result appears directly in namespaces such as:

```
net::in::stream
net::in6::stream
net::un::stream
net::rc::stream
net::l2::stream
```

This is the first place where the layer model becomes visible in code.

A reader can ask:

```
Am I looking at IPv4 stream communication?
IPv6 stream communication?
Unix-domain stream communication?
Bluetooth RFCOMM stream communication?
Bluetooth L2CAP stream communication?
```

That question is more useful than asking only which socket class is involved. It places the code in the communication stack. Once the endpoint family and transport form are fixed, the next question is how the concrete peer relationship is managed while it exists.

7.6 The connection layer: managing the peer relationship

The network layer chooses the endpoint family. The transport layer chooses the communication form.

The connection layer answers a different question:

How is the concrete peer relationship handled while it exists?

In the stream-oriented SNode.C model, the central connection-layer distinction is:

```
legacy
tls
```

`legacy` means the non-TLS stream connection variant.

`tls` means TLS-secured connection handling.

The word `legacy` is important to read correctly. In this book it does not mean obsolete. It is the established SNode.C name for the stream variant that does not add TLS at the connection layer.

This distinction is not the same as the transport distinction. The transport layer says that communication is stream-oriented. The connection layer says how the active peer relationship is managed, including whether TLS is part of that management.

TLS changes real behavior:

- handshake sequence,
- certificate material,
- peer validation,
- SNI or hostname-related concerns,
- timing,
- failure modes,
- logging and diagnostics.

Yet TLS should not force the application writer to abandon the whole application model.

That is the useful architectural point. Moving from `legacy` to `tls` is a connection-layer change, not a complete application rewrite.

Often, the application keeps:

- the same server/client handle shape,
- the same registered-instance model,
- the same factory/context pattern,
- the same broad protocol behavior,
- the same runtime model,
- and the same lower family choice.

What changes is the connection machinery below the application protocol.

That is exactly what a good layer model should achieve. It isolates the change without pretending that TLS is trivial. TLS has operational and security consequences, but architecturally it belongs in the connection layer.

Practical connection-layer combinations

The component structure reflects this separation.

Core stream connection components include:

```
core-socket-stream-legacy
core-socket-stream-tls
```

Concrete network stream components combine network family, stream transport, and connection handling. Examples include:

```
net-in-stream-legacy
net-in-stream-tls
net-in6-stream-legacy
net-in6-stream-tls
net-un-stream-legacy
net-un-stream-tls
```

The matching public include paths follow the same shape, ending in the role header selected by the application:

```
#include <net/in/stream/legacy/SocketServer.h>
#include <net/in/stream/tls/SocketServer.h>
#include <net/in6/stream/legacy/SocketServer.h>
#include <net/un/stream/legacy/SocketServer.h>
```

The application includes the role it names. It does not include the whole lower core/socket stack manually just because that role is implemented from lower pieces.

Bluetooth-related stream components follow the same conceptual pattern where Bluetooth support and the corresponding build options are available:

```
net-rc-stream-legacy
net-rc-stream-tls
net-l2-stream-legacy
net-l2-stream-tls
```

The exact availability of Bluetooth components depends on the build environment and enabled options. The architectural reading strategy remains the same:

```
network family
-> stream transport
    -> non-TLS or TLS connection handling
```

7.7 The application layer: protocol behavior

Above the connection layer sits the application layer. This is where communication receives meaning.

For a small custom protocol, the application layer may be a `SocketContext` derived class written by the user. The context reacts to lifecycle and data events for one concrete connection. That is what the echo example did in Chapter 3.

For higher-level framework support, the application layer may involve:

- HTTP,
- WebSocket,
- Express-like routing,
- MQTT,
- or MQTT over WebSocket.

Later chapters also show how application protocols can be connected to persistent application state. That persistence belongs to the larger application architecture; it is not itself another communication layer in this stack.

The architectural lesson is that higher protocols do not erase the lower layers. A web server is web behavior carried by some lower communication stack.

An MQTT application is MQTT behavior carried by a connection, which is carried by a transport, which is carried by a network family. MQTT over WebSocket adds another important lesson: an application protocol can itself be carried through another application-layer protocol structure while still relying on the lower stack below it.

This is the practical value of the layer model. Instead of treating higher protocols as feature islands, it lets the reader ask precise questions:

```
Which lower family carries this service?
Is TLS below the protocol?
Is MQTT native over a stream connection, or carried through WebSocket?
Where does the context sit?
Which parts would change if the lower family changed?
```

Those questions keep the application layer connected to the rest of the stack without collapsing all layers into one undifferentiated design.

Application-layer components

The component list reflects higher-level protocol support. The following names are representative examples rather than a vocabulary test:

```
http
http-server
http-client
http-server-express
websocket-server
websocket-client
mqtt
mqtt-server
mqtt-client
mqtt-server-websocket
mqtt-client-websocket
```

Express-based HTTP server components can also encode lower choices, for example:

```
http-server-express-legacy-in
http-server-express-legacy-in6
http-server-express-legacy-rc
http-server-express-legacy-un
http-server-express-tls-in
http-server-express-tls-in6
http-server-express-tls-rc
http-server-express-tls-un
```

The point is not to memorize every name at once. The point is to read component names as architectural statements. A component name tells the reader which higher framework behavior is selected and, in many cases, which lower communication stack carries it.

The corresponding include hierarchy is equally important. A source file that names an Express IPv4 legacy WebApp includes a public Express header, not the lower socket headers directly:

```
#include <express/legacy/in/WebApp.h>
```

That public header represents an Express WebApp over an HTTP server over the selected lower carrier. If a different source file directly names a lower socket client, that source file includes the matching lower public header as well.

The application layer is still connected to sockets

A common beginner mistake is to imagine a sharp psychological break:

```
below: network code
above: web code or MQTT code
```

SNode.C helps the reader avoid that mistake.

HTTP, WebSocket, Express-like APIs, and MQTT are application-layer behaviors carried by lower layers. They may introduce their own vocabulary, abstractions, and configuration surfaces, but they do not float above the network stack as independent magic.

A reader who understands the lower layers can ask better questions:

- Which lower family is this web server using?
- Is this MQTT application native over a stream connection or carried through WebSocket?
- Where does TLS sit relative to HTTP, WebSocket, or MQTT?
- What changes if a service moves from IPv4 to a Unix domain socket locally?
- What changes if a device-near endpoint uses Bluetooth RFCOMM instead of IPv4?

Those questions are exactly the kind of thinking SNode.C is meant to support.

7.8 The build system as confirmation

A useful way to test an architectural description is to ask whether the build system reflects it. In SNode.C, it does.

The `src` build adds major framework regions such as `core`, `net`, `web`, `express`, `database`, `iot`, and `apps`. The supported components include core stream components, concrete network stream components, HTTP, Express, WebSocket, MQTT, and MQTT-over-WebSocket components.

A small example makes the relationship visible:

```
public include path:
<net/in/stream/legacy/SocketServer.h>
<net/in/stream/legacy/SocketClient.h>

C++ namespace / type path:
net::in::stream::legacy

CMake component name:
net-in-stream-legacy

Meaning:
network family: IPv4 / in
transport form: stream
connection handling: non-TLS stream connection variant
```

The same pattern appears in the TLS variant:

```
public include path:
<net/in/stream/tls/SocketServer.h>
<net/in/stream/tls/SocketClient.h>

C++ namespace / type path:
net::in::stream::tls

CMake component name:
net-in-stream-tls

Meaning:
network family: IPv4 / in
transport form: stream
connection handling: TLS
```

This matters because the layer model affects how code is organized, how components are built, and how external projects link against the framework.

A reader who learns the layer names is also learning how to navigate the build.

7.9 One protocol, many lower carriers

The layer model becomes most useful when the reader asks a transfer question. Suppose you write one simple application protocol using a `SocketContext`.

What changes if the same protocol is carried over:

- IPv4,
- IPv6,
- Unix domain sockets,
- Bluetooth RFCOMM,
- Bluetooth L2CAP?

Some things change:

- address syntax,
- bind and connect semantics,
- required environment,
- permissions or operating-system behavior,
- deployment pattern,
- configuration details,
- diagnostics and failure modes.

But other things remain stable:

- the event-driven runtime,
- the registered instance as the communication role advanced by the runtime,
- the factory/context pattern,
- the broad connection lifecycle,
- the idea that TLS belongs to the connection layer,
- the idea that higher protocols can sit above the connection.

That stable remainder is the value of the layer model.

It is not that lower carriers are interchangeable in every practical detail. They are not. The value is that the reader can separate what changed from what did not.

That is transfer.

Where Bluetooth fits

RFCOMM and L2CAP appear here for the same reason IPv4, IPv6, and Unix domain sockets appear: they are endpoint families in the lower communication model. They are not exceptions to the stack; they exercise it in a device-near setting.

This is especially important for IoT, embedded, and machine-to-machine systems. Such systems often combine local device communication, network transport, web interfaces, message-oriented integration, and deployment constraints. Treating Bluetooth as a strange appendix would weaken the architectural lesson. Treating it as a lower communication family makes the model stronger.

Once the reader understands that RFCOMM and L2CAP belong to the same architectural model, later chapters can discuss their concrete address and API details without repeatedly defending their presence.

7.10 Layers are real, but not walls

A good systems book should avoid two opposite mistakes. The first mistake is to collapse everything into one big communication soup. The second mistake is to imagine layers as perfectly sealed walls that never influence one another.

`SNode.C` avoids the first mistake structurally. As readers, we should avoid the second. Layers are real and useful, but they still influence each other.

For example:

- a network family affects address representation and deployment;

- TLS affects connection timing, failure modes, and configuration;
- an application protocol may affect buffering and backpressure expectations;
- a local Unix domain socket deployment may change security assumptions;
- Bluetooth may introduce discovery, pairing, permissions, or hardware availability concerns.

So the correct mental model is not:

Layers never interact.

The correct model is:

Layers isolate concerns well enough that changes can be reasoned about locally, while still allowing real cross-layer consequences to be understood.

That is a clearer and more useful systems view.

The layer model should therefore not be used as an excuse to ignore operational reality. It should be used to ask better questions. If TLS is added, ask which connection-layer behavior changes. If IPv4 is replaced by a Unix domain socket, ask which endpoint and deployment assumptions change. If MQTT is carried through WebSocket, ask where each protocol sits and which layer is responsible for which behavior.

This is how the model becomes practical instead of merely tidy.

7.11 What belongs where?

The following table is a compact orientation aid. It is simple: its job is not to solve every design decision, but to help the reader ask the right first question.

Concern	Layer or model
IPv4, IPv6, Unix domain sockets, RFCOMM, L2CAP	Network layer
Endpoint identity and address structure	Network layer
Stream-oriented connection form	Transport layer
Non-TLS versus TLS-secured communication	Connection layer
Handshake, encryption, certificate validation	Connection layer
<code>SocketContext</code> protocol behavior	Application layer
HTTP, WebSocket, MQTT, Express-like routing	Application layer
Timers, descriptor readiness, event queue	Runtime model
Retry/reconnect scheduling	Operational/runtime behavior
Public include selection	C++ source expression of the layer stack
CMake component selection	Build/link expression of the layer stack

The question to keep asking is:

Which layer am I reasoning about?

Sometimes the answer is one layer. Often, it is a primary layer plus consequences in another layer. That is normal. Real systems are layered, but they are still systems.

What to remember

- The communication stack is not the runtime core: the runtime advances instances, while the stack describes what kind of communication those instances represent.
- The network layer chooses the endpoint family, such as IPv4, IPv6, Unix domain sockets, RFCOMM, or L2CAP.
- The transport layer gives the communication form; in the early teaching path, that form is **stream**.
- The connection layer manages the concrete peer relationship; **legacy** names the non-TLS stream connection variant, while **tls** adds TLS connection handling.
- The application layer contains protocol behavior, whether that behavior is a custom **SocketContext**, HTTP, WebSocket, Express-like routing, MQTT, or MQTT over WebSocket.
- `SNode.C` public include paths, type names, and component names are compressed architectural

statements, and layers are real boundaries without being sealed walls.

7.12 Closing perspective

Chapter 5 gave the mental model. Chapter 6 explained how registered instances are advanced by the runtime machinery. This chapter has shown how the communication layer stack appears in SNode.C names, components, and transfer questions.

The layer model tells us that address semantics belong to the network layer. Chapter 8 therefore moves from the general stack to the first concrete network-layer question:

What does an endpoint identity mean for each supported family?

Part III

Networking Foundations in SNode.C

This part walks through the lower communication families that make SNode.C concrete. IPv4, IPv6, Unix domain sockets, Bluetooth RFCOMM, and Bluetooth L2CAP are not treated as interchangeable address spellings. They are different boundary choices with different deployment assumptions.

The purpose is to make lower-family selection visible before higher protocol layers are added.

Chapter 8

Socket Addresses and Address Semantics

8.1 From layers to endpoint identity

Chapter 7 showed how SNode.C communication choices are encoded in the layer stack.

The first of those choices is the network family. An application may use IPv4, IPv6, Unix domain sockets, Bluetooth RFCOMM, or Bluetooth L2CAP. That choice changes what an endpoint *is*; the namespace or build component is only the visible consequence.

It therefore asks a narrower but important question:

What does endpoint identity mean for each supported family?

When people first learn network programming, they often treat addresses as simple input values. An IP address is a string. A port is a number. A Unix domain socket is a path. A Bluetooth endpoint is a device address plus a service selector.

That view is understandable, but it is too shallow for a framework like SNode.C.

In SNode.C, addresses are part of the network layer. They are not incidental parameters passed into `listen(...)` and `connect(...)` and then forgotten. The address type tells the framework and the reader what kind of endpoint identity is being described.

That affects practical questions:

- how a server binds,
- how a client selects a peer,
- what a wildcard endpoint means,
- what local and remote identity mean,
- how diagnostics should render the endpoint,
- and, in the IP families, how a name can resolve to candidate addresses.

Therefore, SNode.C does not collapse all endpoints into one vague address class. It uses a shared pattern, but preserves the family-specific meaning.

8.2 The shared SocketAddress pattern

The common base is:

```
net::SocketAddress<SockAddrT>
```

That template already tells us something useful.

A `SNode.C` address is backed by a concrete socket-address structure and length, and it participates in the framework's broader socket-address abstraction. The template parameter keeps the operating-system address representation visible enough for the family-specific class to remain honest.

The shared base gives the framework a common shape. The derived family classes preserve the differences. That is the right compromise for this part of the design:

Endpoint identity note

One address pattern, but no false claim that all endpoint identities mean the same thing.

This mirrors the layer model from Chapter 7. The application shape may stay recognizable, but the endpoint family still matters.

8.3 Local and remote endpoint roles

Before looking at the concrete families, it helps to separate two questions that are often mixed together.

The first question is:

What kind of endpoint identity does this family use?

The second question is:

Is this address being used as a local endpoint or as a remote peer endpoint?

A server usually starts with a local bind identity. It may bind to a specific interface, a wildcard address, a Unix-domain path, a Bluetooth address, or a service selector.

A client usually starts with a remote peer identity. It may connect to a host and port, a Unix-domain path, a Bluetooth device and channel, or a Bluetooth device and PSM.

But the distinction is not absolute. A client may also have a local-side address. A server may later observe the remote addresses of connected peers. The same address class can therefore appear in different roles.

This is one reason address semantics deserve careful treatment. A value such as 0, "", or 00:00:00:00:00:00 is not meaningful by itself. It becomes meaningful in a family and in a role.

8.4 Endpoint identity by family

The concrete address families are:

Family	Class	Endpoint identity
IPv4	<code>net::in::SocketAddress</code>	host plus port
IPv6	<code>net::in6::SocketAddress</code>	host plus port
Unix domain sockets	<code>net::un::SocketAddress</code>	local socket path
Bluetooth RFCOMM	<code>net::rc::SocketAddress</code>	Bluetooth address plus channel
Bluetooth L2CAP	<code>net::l2::SocketAddress</code>	Bluetooth address plus PSM

This table is the concrete continuation of Chapter 7. The network layer chooses the endpoint family; the address class expresses that family in C++.

The useful reading habit is:

Address-reading habit

Do not ask only what values an address object stores. Ask what kind of endpoint identity those values describe.

IPv4: host plus port

The IPv4 address class is:

```
net::in::SocketAddress
```

Its identity shape is host plus port.

Here, *host* may mean a numeric IPv4 address or a name that has to be resolved. The port is the IP service selector. Together, they describe an endpoint in the IPv4 family.

A concrete remote endpoint may look like this:

```
net::in::SocketAddress remote("127.0.0.1", 8080);
```

A server-side address that binds to a port on the IPv4 wildcard address may be expressed as:

```
net::in::SocketAddress local(8080);
```

The default IPv4 address is wildcard-oriented:

```
0.0.0.0
```

with port:

```
0
```

For a server, 0.0.0.0 naturally means that the server is not restricted to one specific local IPv4 interface. Port 0 means that the concrete service selector has not yet been fixed and may be selected later by the operating system if used in that way.

The IPv4 class also contains the resolution-oriented pieces that belong to IP-style addressing: `Hints`, canonical-name access, and `useNext()`. That is not accidental API growth. A host name may resolve to more than one candidate endpoint. The address abstraction therefore needs room for resolution and iteration.

IPv6: similar shape, different family

The IPv6 address class is:

```
net::in6::SocketAddress
```

Its surface deliberately resembles the IPv4 address class. It has the same host-plus-port teaching shape, the same need for name resolution, and the same idea that a host may be a numeric address or a resolvable name.

A concrete loopback endpoint may look like this:

```
net::in6::SocketAddress remote("::1", 8080);
```

The default IPv6 host is:

```
::
```

with port:

```
0
```

This gives IPv6 the same broad wildcard-oriented starting point as IPv4, but in IPv6 vocabulary.

The similarity is useful, but it should not hide the family boundary. IPv6 has its own address syntax, operational behavior, and deployment consequences. SNode.C gives IPv4 and IPv6 a similar address interface where that helps, while preserving the fact that they are different endpoint families.

Unix domain sockets: local endpoint identity

The Unix domain socket address class is:

```
net::un::SocketAddress
```

Its endpoint identity is not host plus port.

It is a local socket identity, normally represented by a path-like value:

```
net::un::SocketAddress local("/tmp/snodec.sock");
```

This is conceptually different from IP addressing. There is no remote internet host and no port number. The endpoint belongs to the local operating-system namespace.

Default construction uses the empty string:

```
""
```

In the SNode.C address model, that is the wildcard or not-yet-specific Unix-domain endpoint identity.

Unix domain sockets are pedagogically useful because they break a common habit. They remind the reader that not every endpoint is internet-shaped. The same server/client/context architecture can still appear, but the network-layer identity is different.

RFCOMM: Bluetooth address plus channel

The RFCOMM address class is:

```
net::rc::SocketAddress
```

An RFCOMM endpoint is:

```
Bluetooth device address  
+ RFCOMM channel
```

A concrete remote endpoint may look like this:

```
net::rc::SocketAddress remote("00:11:22:33:44:55", 3);
```

The Bluetooth address identifies the device. The RFCOMM channel selects the service within the RFCOMM family. The channel should not just be translated into “a Bluetooth port number.” That wording would pull the reader back into IP vocabulary. A channel is the service selector used by RFCOMM.

Default construction is wildcard-oriented in the SNode.C model:

```
00:00:00:00:00:00
```

with channel:

```
0
```

That means the address can express a broad or deferred Bluetooth endpoint identity until the application or configuration makes it more specific.

L2CAP: Bluetooth address plus PSM

The L2CAP address class is:

```
net::l2::SocketAddress
```

It belongs to the same Bluetooth world as RFCOMM, but it uses a different service selector:

```
Bluetooth device address
+ PSM
```

A concrete remote endpoint may look like this:

```
net::l2::SocketAddress remote("00:11:22:33:44:55", 0x1001);
```

The Bluetooth address again identifies the device. The PSM identifies the L2CAP service endpoint.

Default construction follows the same wildcard-oriented Bluetooth pattern:

```
00:00:00:00:00:00
```

with PSM:

```
0
```

RFCOMM and L2CAP therefore share the idea of Bluetooth device identity, but they do not share the same service selector vocabulary. RFCOMM uses a channel. L2CAP uses a PSM.

That is why SNode.C gives them distinct address classes. Similar Bluetooth vocabulary does not mean identical endpoint semantics.

8.5 Patterns that repeat across families

The five families differ, but several patterns repeat.

The point of this section is not to list every constructor or every member function. The useful question is:

What does the API shape teach about endpoint identity?

Default construction and wildcard meaning

Default construction is meaningful. It is not uninitialized data.

Across the supported families, the default address expresses a wildcard-like, broad, or deferred endpoint identity:

Family	Default / wildcard shape
IPv4	0.0.0.0, port 0
IPv6	::, port 0
Unix domain sockets	empty string
RFCOMM	00:00:00:00:00:00, channel 0
L2CAP	00:00:00:00:00:00, PSM 0

This matters because socket programming often begins with partial endpoint descriptions.

A server may specify only the service selector it wants to listen on. A client may specify a remote endpoint while leaving its local side broad. A wildcard address is not an error. It is often the correct expression of “this endpoint is broad until bind or connect makes it concrete.”

Constructors teach identity shape

Constructors reveal what each family considers natural.

Port-only construction makes sense for IPv4 and IPv6 because a server often listens on a service port without caring which local interface receives the connection.

Path construction makes sense for Unix domain sockets because the path is the local endpoint identity.

Channel construction makes sense for RFCOMM because an RFCOMM service can be described by a channel while the Bluetooth address remains wildcarded.

PSM construction makes sense for L2CAP for the same reason, but in L2CAP vocabulary.

Full constructors express a fully specified peer or an explicitly specified local endpoint:

```
net::in::SocketAddress ip4("127.0.0.1", 8080);
net::in6::SocketAddress ip6("::1", 8080);
net::un::SocketAddress unixAddress("/tmp/snodec.sock");
net::rc::SocketAddress rfcommAddress("00:11:22:33:44:55", 3);
net::l2::SocketAddress l2capAddress("00:11:22:33:44:55", 0x1001);
```

These examples are not application programs. They are anchors for the address model.

Setters and getters use family language

Each address class names its fields in the language of its family. That is an important design choice.

The framework does not force all address data through vague names such as `value1` and `value2`, or through a single generic key-value map. It uses names such as:

```
setHost(...)
setPort(...)
setSunPath(...)
setBtAddress(...)
setChannel(...)
setPsm(...)
```

The names teach the endpoint model.

A host is not a Unix path. A Unix path is not a Bluetooth address. An RFCOMM channel is not an L2CAP PSM.

The API vocabulary preserves those differences.

String rendering is part of runtime literacy

All concrete address classes provide string rendering.

That may look secondary, but it is part of operational readability. A framework that cannot report endpoint identity clearly is harder to teach, harder to debug, and harder to operate.

Address rendering appears in places such as:

- startup logging,
- listen and connect callbacks,
- error messages,
- status output,
- diagnostic traces,
- and teaching examples.

A good address string is often the first evidence a reader sees that a server has bound the expected endpoint or that a client tried to connect to the expected peer.

8.6 Address semantics and the layer model

This chapter is the first concrete network-layer chapter after the general layer model.

The runtime may still advance registered instances in the same broad way. The application may still use a factory and a context. The connection layer may still distinguish non-TLS stream handling from TLS stream handling.

But the network layer changes the meaning of the endpoint being bound or connected. That is why `SocketAddress` is such a useful bridge between architecture and code.

It shows:

- what counts as endpoint identity in each family,
- how wildcarding is expressed in that family,
- which values naturally belong together,
- how local and remote endpoint roles are described,
- and how higher layers can stay similar while lower identity rules differ.

This is the practical value of separating layers. The application writer does not have to relearn the whole framework for every family, but neither does the framework pretend that every family has the same endpoint semantics.

8.7 Why IP families have richer resolution behavior

IPv4 and IPv6 differ from Unix domain sockets, RFCOMM, and L2CAP in one especially important way.

Their address classes expose resolution-oriented API pieces:

- `Hints`,
- `useNext()`,
- canonical-name access.

This reflects the fact that internet-style host names may resolve to more than one candidate endpoint.

A name such as:

```
example.org
```

is not necessarily one concrete address. It may produce several candidates. The framework's address abstraction therefore leaves room for iterating through them.

Therefore, the IP-family address classes are richer in this specific area. The richness is not accidental; it reflects the semantics of the family.

That distinction also prevents a false generalization. Unix domain sockets, RFCOMM, and L2CAP do not need to mimic IP name resolution merely to fit a common abstract shape. They keep the address model appropriate to their own family.

8.8 What belongs where?

The following table summarizes the address-level meaning of common terms.

Concept	Address-layer meaning
Address family	The endpoint universe the address belongs to
Host	IP-family peer or local-interface identity
Port	IP-family service selector
Unix path	Local operating-system endpoint identity
Bluetooth address	Bluetooth device identity
RFCOMM channel	RFCOMM service selector
L2CAP PSM	L2CAP service selector
Wildcard/default address	Broad or deferred endpoint identity

Concept	Address-layer meaning
<code>toString(...)</code>	Runtime-readable endpoint representation

The table is compact. Its job is not to replace the API reference. Its job is to keep the central question visible:

What kind of endpoint identity am I describing?

Once that question becomes natural, the address classes become much easier to read.

What to remember

- Socket addresses belong to the network layer: they describe endpoint identity for a selected family.
- `SNode.C` uses a shared address pattern, but each supported family keeps its own concrete `SocketAddress` type.
- IPv4 and IPv6 use host-plus-port identity and add resolution-oriented behavior such as hints, canonical names, and candidate iteration.
- Unix domain sockets use local socket path identity.
- RFCOMM and L2CAP use Bluetooth device identity with different service selectors: channel for RFCOMM, PSM for L2CAP.
- Default construction is meaningful and wildcard-oriented in family-specific ways.

8.9 Closing perspective

Chapter 7 explained the communication stack. This chapter made the first layer of that stack concrete by studying endpoint identity.

Chapter 9 can therefore ask the next architectural question with sharper vocabulary: how do server instances, client instances, and concrete connections use those endpoint identities while the runtime advances the communication flow?

Chapter 9

Servers, Clients, and Connections

9.1 Why these three belong in one chapter

Chapter 8 made the network layer concrete. It showed that an address in `SNode.C` is a family-specific description of endpoint identity, not an interchangeable string, number, or path. IPv4 and IPv6 use host-plus-port identity. Unix domain sockets use local socket identity. RFCOMM uses Bluetooth address plus channel. L2CAP uses Bluetooth address plus PSM.

This chapter asks the next question:

Who uses those endpoint identities, and what appears when the operation succeeds?

The answer brings together three concepts that should not be learned as unrelated types.

A server-side instance uses a local endpoint identity to listen and accept peers. A client-side instance uses a remote endpoint identity, and optionally a local one, to initiate a connection. A connection is the concrete peer relationship that appears when the listen or connect flow succeeds.

Figure 9.1 turns that sequence into a single path. It starts with the application-side handle, passes through the registration call and the runtime-visible server or client role, and then follows the accept/connect machinery to the concrete connection, factory, context, and protocol behavior.

Read Figure 9.1 as the bridge between the previous chapters, not as a full implementation diagram. Chapter 3 showed the first echo pair. Chapter 5 distinguished handles, instances, connections, factories, and contexts. Chapter 6 explained the runtime and flow-controller machinery that advances registered instances. Chapter 7 placed communication choices into a layer stack. Chapter 8 explained endpoint identity.

Chapter 9 now brings those ideas together around the most important practical boundary in the stream layer:

Instance/connection warning

A server or client instance is not a connection. It is the runtime-visible role under which concrete connections appear.

That distinction is simple, but it carries a large part of the framework's architecture. If it is missed, everything tends to collapse into one vague object: the thing that listens, connects, owns the socket, handles data, stores state, performs retries, and implements the protocol. `SNode.C` does not use that collapsed model. It separates those responsibilities deliberately.

9.2 From registered instance to concrete connection

The useful first distinction is not a class hierarchy. It is a responsibility map.

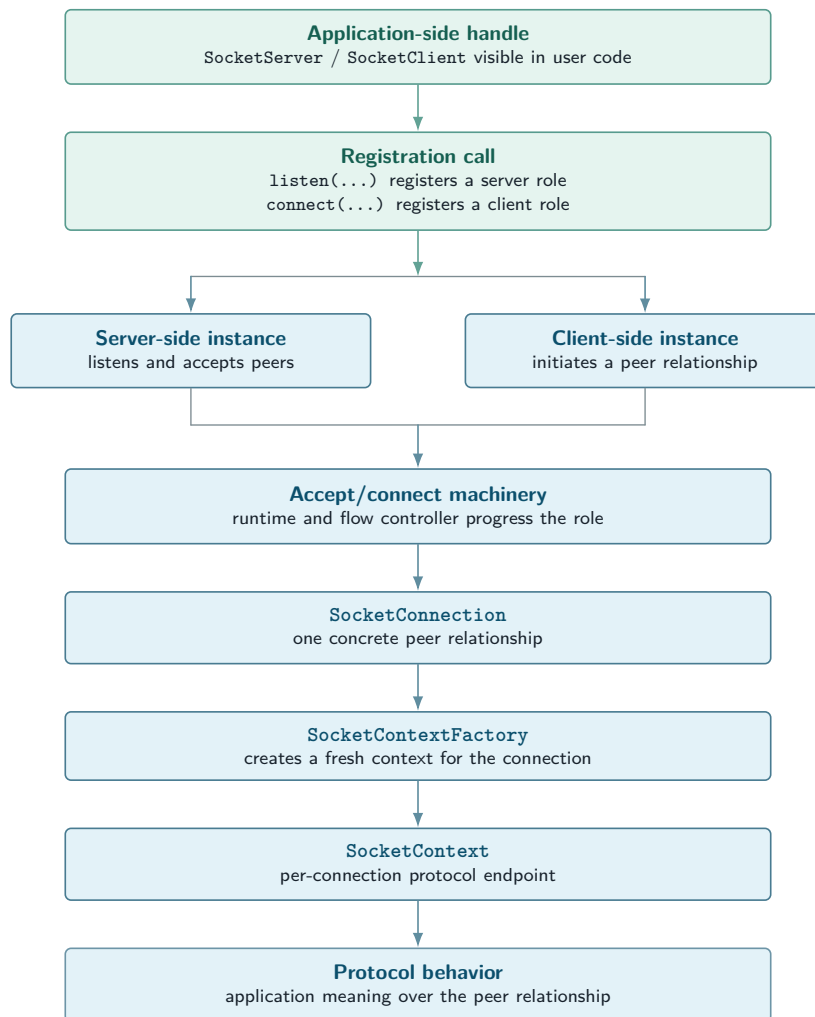


Figure 9.1: The server/client role path from application-side handle to protocol behavior.

Concept	Typical C++ representation	Responsibility
Application-side handle	<code>SocketServer</code> / <code>SocketClient</code> object visible in user code	configure and register the role
Registered instance	server-side or client-side runtime-visible role	participate in runtime and flow-controller progress
Connection	<code>SocketConnection</code>	represent one concrete peer relationship
Factory	<code>SocketContextFactory</code>	create a per-connection context
Context	<code>SocketContext</code>	implement protocol behavior for one connection

The table is deliberately phrased in concepts first and class names second.

The visible `SocketServer` or `SocketClient` object is the application-side handle through which application code configures and registers a role. After `listen(...)` or `connect(...)`, the registered instance is represented by framework-owned shared state and flow-controller machinery. That instance may then create or accept one or more concrete connections over time.

The connection is the peer relationship. It has addresses, a descriptor, data flow, shutdown behavior, timeouts, timing information, counters, and names.

The context is the application protocol endpoint attached to that connection. It is where protocol code reacts to lifecycle and input events.

These boundaries matter because they prevent a common collapse:

```
server object = connection = protocol
```

That collapse is wrong for `SNode.C`.

The better picture is:

```
handle
-> registered instance
    -> connection
        -> context
```

The handle registers the role. The instance is advanced by the runtime. The connection is a concrete peer relationship. The context implements protocol behavior over that relationship.

The distinction has direct consequences for how an application is structured.

Configuration belongs naturally to the instance, because configuration describes how the communication role should behave over time. Addresses appear at registration and later on concrete connections, because they describe endpoint identity. Retry and reconnect behavior belongs to the instance and its flow-controller machinery, because it concerns how the role should keep trying or resume later. Protocol behavior belongs to the context, because it is relative to one concrete peer relationship.

Once that separation is clear, the rest of the chapter becomes much easier to read.

9.3 Server and client instances as runtime-visible roles

At the application-facing level, `SNode.C` exposes stream server and stream client templates. Concrete user-facing handle types are formed by combining the lower communication family, the transport form, and the connection handling variant.

Examples include:

```
net::in::stream::legacy::SocketServer<MyFactory>
net::in::stream::legacy::SocketClient<MyFactory>
net::rc::stream::tls::SocketServer<MyFactory>
net::l2::stream::legacy::SocketClient<MyFactory>
```

These names are long, but they are not arbitrary. They state the lower communication family, the transport form, the connection handling variant, and the server/client direction. In application code, objects of these types are the visible handles. Through those handles, the application configures and registers a server-side or client-side instance.

The important conceptual point is:

A server or client instance is not the protocol endpoint.

It is the configured communication role that participates in the runtime, creates or accepts concrete connections, and arranges for contexts to be attached to them.

The protocol endpoint lives in the context.

The concrete peer relationship lives in the connection.

The runtime-visible instance is the durable role that ties the two together over time.

Local handle and runtime-visible instance

This chapter uses the local-handle distinction introduced earlier.

A local `SocketServer` or `SocketClient` object is the handle used to configure and register the role. After `listen(...)` or `connect(...)`, the active runtime story continues through shared configuration, flow-controller state, accept/connect machinery, and connection objects.

That distinction explains why the outer object should not be mentally reduced to one connection.

The instance is the durable role.

The connection is the concrete peer relationship.

The local handle is the application-side entry point through which the role is configured and registered.

This is especially important for examples. A small example may keep the handle visible in `main()` until `core::SNodeC::start()` returns. That is a clear and readable style. But the architectural model is not “the local variable is the whole runtime entity.” The registered instance is carried by framework-owned state and advanced by the runtime.

9.4 Server instances

A server-side instance is often introduced with a simple phrase such as “the server listens on a port.” That phrase is useful, but it is incomplete.

A more accurate description is:

A server-side instance is a configured runtime-visible role that registers listening intent, starts accept machinery through the runtime, accepts peers through the selected lower layer, creates connection objects, and attaches contexts through a factory.

That sounds longer because it names the responsibilities that a real server role has.

Listening as registered intent

Calling `listen(...)` should be understood as registering listening intent.

The call does not mean that all listening and accepting work happens immediately on the caller’s stack. It enters the flow-controller path and lets the runtime advance the work.

This matters because retry logic, status reporting, accept-event observation, and context creation all belong to the managed runtime story.

The application says, in effect:

This server-side role should listen here.

The runtime and flow-controller machinery then move that intention forward.

That is different from a small blocking wrapper around `bind(...)`, `listen(...)`, and `accept(...)`. `SNode.C` uses the same operating-system concepts underneath, but the framework-level design is event-driven and role-based.

Accepting peers

When the server-side instance succeeds in listening, peers can be accepted.

Each accepted peer becomes a concrete `SocketConnection`.

That connection is not the server itself. It is one peer relationship under the server-side role.

A server may therefore create many connection lifecycles over its own lifetime. Some may be short. Some may be long. Some may close normally. Some may fail. Each one is still a concrete connection under the same server-side instance.

This is one of the reasons the framework keeps the outer role and the concrete connection separate. A server-side instance can survive many peer relationships. A connection belongs to one peer relationship. A context belongs to that connection.

Server lifecycle callbacks

The server-side role can expose lifecycle callbacks such as:

```
onConnect
onConnected
onDisconnect
```

These callbacks receive a `SocketConnection*`.

That detail is important. They are hooks into the lifecycle of a concrete connection under the server-side instance, not status messages about the server as a whole.

Typical uses include:

- logging local and remote addresses,
- inspecting connection properties,
- observing timing or metrics,
- adding operational diagnostics,
- applying role-level setup that should not live in the application protocol context.

These callbacks should not be confused with context callbacks. A server lifecycle callback observes or supervises a connection. The context implements application protocol behavior over that connection.

A useful rule is:

```
role callback
-> observes or adapts the connection lifecycle

context callback
-> implements protocol behavior on the connection
```

That distinction becomes more valuable as applications grow. In a tiny echo example, it may feel like everything could live in one place. In a real application, separating supervision from protocol behavior is what keeps the code understandable.

9.5 Client instances

A client-side instance is the client counterpart of the server-side role. It is often introduced as “the object that connects somewhere,” but that is again only the beginning.

A more accurate description is:

A client-side instance is a configured runtime-visible role that registers connection intent, starts connect machinery through the runtime, establishes peer relationships through the selected lower layer, attaches contexts through a factory, and can coordinate retry and reconnect behavior over time.

That is why a `SNode.C` client is more substantial than a blocking `connect()` call wrapped in a class. It is a communication role with lifecycle.

Connecting as registered intent

Calling `connect(...)` registers connection intent.

The remote endpoint comes from the address semantics described in Chapter 8. The runtime then advances the actual connection attempt through the selected lower layer.

This is the client-side counterpart of the server's listening story.

The application says, in effect:

This client-side role should connect there.

The runtime and flow-controller machinery then move that intention forward.

A client-side instance may produce one connection, no connection, or several connection episodes over time if retry or reconnect behavior is configured. That is why the instance must not be confused with a single successful connection.

Client lifecycle callbacks

Like the server-side role, the client-side role can expose lifecycle callbacks:

```
onConnect
onConnected
onDisconnect
```

These also receive a concrete `SocketConnection*`.

The callbacks are structurally parallel to the server side, but the operational meaning differs because the client initiates the connection and may later reconnect.

This symmetry helps the reader transfer understanding from server code to client code. Once the callback layers are understood on one side, the other side feels familiar. But the direction of setup remains different: the server accepts, the client initiates.

9.6 Server and client symmetry, and where it ends

Server and client instances are conceptually parallel.

Both:

- are configured instances,
- participate in the runtime,
- carry a socket-context factory,
- expose connection lifecycle callbacks,
- lead to concrete `SocketConnection` objects,
- attach application behavior through contexts.

The differences are also real:

Aspect	Server-side instance	Client-side instance
Outer intention	listen	connect
Peer creation	accepts peers	initiates connection attempts

Aspect	Server-side instance	Client-side instance
Common long-term shape	one role, many accepted connections	one role, potentially many attempts or reconnect episodes
Retry focus	listening retry	connect retry and reconnect
Primary address concern	local bind address	remote peer address, optional local bind address

The reader should keep both ideas at once.

The structure is shared.

The direction of communication setup is different.

That is why the same mental model can carry both sides without pretending that server and client behavior are identical.

Retry, reconnect, and role lifetime

Server and client instances both involve flow control, but the client side makes reconnect especially visible.

A server-side flow can retry listening when configured.

A client-side flow can retry connection attempts when the initial attempt fails, and it can reconnect later after a connection has ended when reconnect behavior is configured.

The distinction is:

Role	Retry/reconnect focus
Server-side instance	retry listening when configured
Client-side instance	retry connection attempts and optionally reconnect after disconnect

This belongs in the instance discussion, not in the application protocol context.

The context should not have to become a reconnect manager.

That rule is important enough to state directly:

Retry/reconnect ownership

Retry and reconnect are behavior of the configured instance and its flow-controller machinery; they are not responsibilities of the per-connection protocol context.

The context may react to a connection while it exists. It may send application data, parse incoming data, close the connection, or keep protocol-side state. But it should not be responsible for recreating the whole communication role after a connection ends. That responsibility belongs to the role and its runtime machinery.

9.7 SocketConnection: the concrete peer relationship

If the server or client instance is the outer role, the `SocketConnection` is the concrete peer relationship.

This is where communication becomes tangible.

The connection object carries several groups of responsibility:

Category	Examples
Descriptor identity	<code>getFd()</code>
Context attachment	<code>setSocketContext(...)</code> , <code>getSocketContext()</code>

Category	Examples
Addresses	<code>getBindAddress()</code> , <code>getLocalAddress()</code> , <code>getRemoteAddress()</code>
Data flow	<code>sendToPeer(...)</code> , <code>readFromPeer(...)</code> , <code>streamToPeer(...)</code> , <code>streamEof()</code>
Shutdown and closing	<code>shutdownRead()</code> , <code>shutdownWrite()</code> , <code>close()</code>
Timeouts	<code>setTimeout(...)</code> , <code>setReadTimeout(...)</code> , <code>setWriteTimeout(...)</code>
Metrics	<code>getTotalSent()</code> , <code>getTotalQueued()</code> , <code>getTotalRead()</code> , <code>getTotalProcessed()</code>
Time information	<code>getOnlineSince()</code> , <code>getOnlineDuration()</code>
Naming	<code>getInstanceName()</code> , <code>getConnectionName()</code>

This table is not meant to replace API documentation. Its job is to show why the connection is a runtime object with identity, data flow, time, diagnostics, and protocol attachment.

The connection is therefore not a hidden pipe. It is a visible runtime object with identity, timing, addresses, data flow, protocol attachment, and operational metrics.

Connection versus context

A connection object represents the managed communication relationship.

A context object represents the application protocol endpoint attached to that relationship.

The correct mental model is:

```

SocketConnection
  -> concrete peer relationship
  -> addresses, timeouts, metrics, data path

SocketContext
  -> application protocol behavior
  -> reacts to lifecycle and input events through the connection

```

This distinction should be kept clear.

The context works through the connection, but it is not the connection.

The connection carries the peer relationship, but it is not the application protocol.

This separation is one of the reasons SNode.C applications can remain readable as they grow. A protocol context can stay focused on protocol behavior, while the connection object remains the place where peer relationship, address visibility, timeouts, counters, and shutdown behavior are represented.

Addresses on the connection

Chapter 8 explained what address objects mean.

Here we see where they appear during connection lifetime.

A `SocketConnection` exposes three address views:

```

getBindAddress()
getLocalAddress()
getRemoteAddress()

```

These are not redundant.

The **bind address** describes the address requested or used for binding.

The **local address** describes the actual local endpoint of the connection.

The **remote address** describes the peer endpoint.

Keeping these separate is useful for wildcard binding, client-side automatic local endpoint selection, diagnostics, and understanding what the operating system ultimately chose.

For example, a server may be configured with a wildcard bind address. A concrete accepted connection will still have an actual local endpoint and a remote peer endpoint. A client may specify a remote peer address while leaving the local side broad enough for the operating system to choose. Those distinctions should remain visible.

The address model from Chapter 8 therefore does not end at startup. It remains visible on the live connection.

Data flow

The connection is where data flow belongs.

The core operations include:

```
sendToPeer(...)
readFromPeer(...)
streamToPeer(...)
streamEof()
shutdownRead()
shutdownWrite()
close()
```

User code often calls these through the context. That is exactly the right shape: the context expresses protocol behavior, while the connection provides the communication relationship through which that behavior acts.

This keeps protocol code from owning transport machinery directly.

It also keeps a useful conceptual boundary in place. Sending bytes, reading bytes, streaming a source, signalling end of stream, shutting down one side, and closing the connection are operations on the peer relationship. Deciding *when* and *why* to do those things belongs to the application protocol context.

Timeouts

Timeouts belong naturally on the connection.

A server-side instance may live for a long time. A specific peer connection may be idle, slow, stalled, or temporarily write-blocked.

Those are different runtime concerns.

Connection-level timeout operations let the framework express that difference:

```
setTimeout(...)
setReadTimeout(...)
setWriteTimeout(...)
```

This also connects back to Chapter 6: timers and event processing make timeout behavior part of the runtime, not a manual sleep loop in protocol code.

A timeout is part of how the runtime supervises a concrete peer relationship, rather than a loose application-level preference.

Metrics and duration

A connection is also measurable.

Useful connection-level quantities include:

```

getTotalSent()
getTotalQueued()
getTotalRead()
getTotalProcessed()
getOnlineSince()
getOnlineDuration()

```

These answer operational questions:

- How long was this peer connected?
- How much data was queued?
- How much data was actually sent?
- How much data was read?
- How much data was processed?

By keeping these notions on the connection, `SNode.C` treats them as natural properties of a peer relationship rather than as external accounting hacks.

This is particularly helpful for diagnostics. A disconnection message that can include addresses, duration, queued bytes, sent bytes, read bytes, and processed bytes is much more useful than a generic “peer disconnected” line.

Naming

A connection also exposes both:

```

getInstanceName()
getConnectionName()

```

The instance name identifies the configured role.

The connection name identifies a concrete peer relationship under that role.

This distinction helps logs and diagnostics, especially for servers that accept many peers over time.

A server-side instance may have one name that remains stable across the process lifetime, while many connection names appear and disappear as peers connect and disconnect. That is exactly the distinction the logging output should preserve.

9.8 Callback layers

Chapter 9 needs one especially clear distinction: not all callbacks report the same kind of event.

`SNode.C` has several callback layers.

Callback type	Receives	Meaning
listen/connect status callback	<code>SocketAddress, State</code>	outer role status
connection lifecycle callback	<code>SocketConnection*</code>	lifecycle of one connection
context callback	context method call	protocol behavior on that connection

This table is one of the most important practical tools in the chapter.

It prevents three different ideas from being collapsed into one vague “callback” concept.

Status callbacks

The status callbacks used with `listen(...)` and `connect(...)` report the outcome or state of the outer communication attempt.

They receive:

```
SocketAddress
core::socket::State
```

The address identifies the relevant endpoint.

The state describes the role-level outcome.

This is different from a connection lifecycle callback. A status callback may tell us whether listening or connecting succeeded, failed, was disabled, or should not be retried. It does not by itself represent protocol behavior.

This is the right level for messages such as:

```
this server-side instance is listening
this client-side instance connected
this connection attempt failed
this role is disabled
this failure is fatal
```

The status callback belongs to the outer role, not to a per-connection protocol context.

`core::socket::State`

`core::socket::State` is richer than a Boolean.

Its principal values include:

```
OK
DISABLED
ERROR
FATAL
NO_RETRY
```

It also carries explanatory information through functions such as `what()` and `where()`.

For Chapter 9, the important point is not to turn `State` into its own topic. The important point is:

Status callbacks report role-level outcomes using a richer state object, not just true or false.

That is enough to understand why the status callback API has the shape it has.

`NO_RETRY` is especially revealing. It shows that the state object can carry more than “success” or “failure.” It can also express control information that affects retry behavior. That fits the broader theme of the chapter: retry is role-level operational behavior, not protocol-context behavior.

Connection lifecycle callbacks

Instance-level lifecycle callbacks such as:

```
onConnect
onConnected
onDisconnect
```

receive a `SocketConnection*`.

They observe the lifecycle of a concrete peer relationship under the server-side or client-side instance.

They are useful for connection inspection, address logging, operational diagnostics, and role-level setup.

They are not the same as status callbacks.

They are also not the same as context callbacks.

The distinction is practical. If the question is “what happened to the outer listen/connect attempt?”, the status callback is involved. If the question is “what can we observe about this concrete peer relationship?”,

the connection lifecycle callback is involved. If the question is “how does the application protocol react?”, the context callback is involved.

onConnect versus onConnected

The framework exposes both `onConnect` and `onConnected` as separate lifecycle hooks.

This means the reader should not collapse them into one undifferentiated event.

The names mark distinct lifecycle hooks in the framework. Later TLS and setup paths make the distinction more visible; for now, the important rule is not to treat them as interchangeable just because they may appear close together in simple examples.

A simple legacy echo example may not make the distinction feel dramatic. A more complex connection path can. That is why the names should be respected from the beginning.

onDisconnect

`onDisconnect` is not a destructor-like cleanup moment.

It is often the place where a completed connection lifecycle becomes interpretable.

At disconnect time, useful information such as addresses, online duration, queued bytes, sent bytes, read bytes, and processed bytes can be logged or inspected.

That makes disconnect a meaningful lifecycle point, not just the end of an object.

This also explains why disconnect callbacks receive a `SocketConnection*`: the connection still carries the information needed to understand what happened.

Context callbacks

Context callbacks are different again.

They are protocol-facing methods on the per-connection context.

Typical examples include lifecycle and input handling methods such as:

```
onConnected()  
onDisconnected()  
onReceivedFromPeer()
```

The exact set depends on the context abstraction being used, but the boundary is stable:

Lifecycle responsibility rule

Instance callbacks observe connection lifecycle; context callbacks implement protocol behavior.

This is the same distinction introduced earlier, but Chapter 9 is where it becomes operationally concrete.

In the echo example, the context decides what to do when data arrives. It reads from the peer and sends data back. That is protocol behavior. It belongs in the context, not in the server-side or client-side instance callback.

9.9 Where the context factory fits

Even though this chapter focuses on servers, clients, and connections, the factory is still part of the story.

The factory is the bridge from long-lived instance to per-connection protocol endpoint.

The server-side or client-side instance carries the factory.

When a concrete connection is created, the factory creates a context for that connection.

The context then expresses the application protocol behavior.

The sequence is:

```
registered instance
  -> SocketConnection
      -> SocketContextFactory
          -> SocketContext
```

This is why the factory belongs conceptually between instance and context.

Without the factory, a server accepting multiple peers would have no clean general mechanism for producing a fresh protocol endpoint for each connection. The application entry point would have to know too much about connection creation timing. The role would begin to absorb protocol construction. The structure would become less reusable.

With the factory, the division is clean:

```
instance
  -> long-lived configured communication role

connection
  -> concrete peer relationship

factory
  -> creates a protocol endpoint for that relationship

context
  -> protocol endpoint attached to the relationship
```

That is the same model from Chapter 5, now placed directly into the server/client/connection flow.

9.10 Putting the pieces together

The full stream communication path can now be read without collapsing responsibilities:

```
application creates a SocketServer / SocketClient handle
  -> application configures the handle
      -> listen(...) / connect(...) registers the instance
          -> runtime and flow-controller machinery advance the role
              -> accept/connect machinery creates or establishes a peer relationship
                  -> SocketConnection represents that relationship
                      -> SocketContextFactory creates a SocketContext
                          -> SocketContext implements protocol behavior
```

Every line in this picture matters.

The handle is not the whole instance.

The instance is not the connection.

The connection is not the context.

The context is not the factory.

The factory is not protocol behavior.

These distinctions may feel careful, but they are what make the framework scalable as a mental model.

A single echo pair can hide the need for this structure because it has only one small behavior. A real system cannot. Once there are retries, reconnects, TLS setup, diagnostics, metrics, multiple accepted peers, protocol upgrades, or different lower-layer families, the boundaries become essential.

9.11 What belongs where?

The following table summarizes the responsibility boundaries.

Concern	Belongs primarily to
naming and configuration of the role	instance / handle configuration
listen or connect intent	registered instance
retry and reconnect policy	instance and flow-controller machinery
concrete peer relationship	SocketConnection
local, bind, and remote endpoint views	SocketConnection
data movement to and from the peer	SocketConnection , usually used through context methods
protocol behavior	SocketContext
creation of per-connection protocol endpoint	SocketContextFactory
role-level status	listen/connect status callback
connection lifecycle observation	instance lifecycle callback
protocol lifecycle reaction	context callback

The table is not a substitute for reading the code. Its purpose is to give the reader a stable classification habit.

When reading a new `SNode.C` type or callback, the first question should be:

Which level of the communication story am I looking at?

That question usually prevents the most common misunderstandings.

What to remember

- The visible **SocketServer** or **SocketClient** object is the application-side handle used to configure and register a server-side or client-side instance.
- The registered instance is the long-lived runtime-visible role; a **SocketConnection** is one concrete peer relationship under that role.
- **listen(...)** and **connect(...)** register intent and enter runtime/flow-controller machinery; they do not make the local handle itself become a peer connection.
- Server and client roles share the same broad pattern, but differ in setup direction: servers accept peers, clients initiate connection attempts and may reconnect.
- Connection objects carry addresses, data flow, shutdown, timeouts, metrics, duration, and naming for one peer relationship.
- Factories create per-connection contexts; contexts implement protocol behavior over the connection.
- Status callbacks, connection lifecycle callbacks, and context callbacks belong to different layers of the model.

9.12 Closing perspective

Chapter 8 explained endpoint identity. This chapter explained the runtime roles that use those identities and the concrete connection objects that appear when communication succeeds.

We can now separate three questions that are often mixed together:

Which endpoint identity is being used?
 Which instance is registering listen or connect intent?
 Which concrete connection exists after success?

That separation prepares the next step.

Chapter 10 uses IPv4 and IPv6 as the primary teaching path. With the distinctions from this chapter in place, we can read those examples more precisely. A server-side instance binds to a local endpoint

identity. A client-side instance connects to a remote endpoint identity. A `SocketConnection` then exposes the actual local and remote endpoints of the concrete peer relationship.

The next chapter therefore does not need to reintroduce the whole model. It can use IPv4 and IPv6 to show the model in its most familiar network-family setting.

Chapter 10

IPv4 and IPv6 as the First Concrete Network Families

10.1 Why IPv4 and IPv6 come first

Chapter 9 separated the application-side server or client handle, the registered runtime-visible instance, the concrete `SocketConnection`, the context factory, and the per-connection context.

This chapter keeps that model intact and changes only the first concrete lower-family choice:

IPv4 and IPv6

IPv4 and IPv6 form the first concrete comparison pair in this book because they are familiar enough to keep the application shape recognizable, but different enough to make the lower-family boundary visible.

Most readers already have some intuition for hosts, addresses, ports, listening servers, and remote peers. That makes IPv4 and IPv6 a good place to begin the concrete family discussion. At the same time, IPv4 and IPv6 are not the same family. Their namespaces, address classes, textual address forms, wildcard forms, and deployment questions differ. `SNode.C` makes that difference explicit instead of hiding it behind one overloaded network type.

IPv4 and IPv6 are therefore not chosen because Unix domain sockets or Bluetooth are secondary. They are chosen because they provide a controlled first variation:

```
same handle / instance / connection model
same factory / context / runtime model
different lower-family semantics
```

That controlled variation is the main teaching value of this chapter. The reader can see what changes when the lower family changes, and just as importantly, what does not change.

10.2 What stays shared across IPv4 and IPv6

For both IPv4 and IPv6, the outer application shape remains recognizable.

A typical stream application still involves:

- an application-side `SocketServer` or `SocketClient` handle,
- a registered server-side or client-side instance after `listen(...)` or `connect(...)`,
- a `SocketContextFactory`,
- a per-connection `SocketContext`,
- a status callback for `listen(...)` or `connect(...)`,
- connection lifecycle callbacks if the application uses them,
- and a runtime started through `core::SNodeC::start()`.

Nothing about moving from IPv4 to IPv6 changes that architectural shape.

The first major observation is:

Family-independent pattern

The handle / instance / connection / factory / context / runtime pattern is more fundamental than the specific IP family.

IPv4 and IPv6 show this without changing endpoint identity as radically as Unix domain sockets or Bluetooth do. Both IP families still use the familiar host-plus-port idea. That makes the comparison focused: the lower family changes, but the application architecture does not collapse into a different model.

Same server/client/context shape

The server/client distinction from Chapter 9 remains stable.

An IPv4 server handle and an IPv6 server handle are both application-side handles used to configure and register server-side instances.

An IPv4 client handle and an IPv6 client handle are both application-side handles used to configure and register client-side instances.

After registration, the instance remains the runtime-visible role. The connection remains the concrete peer relationship. The context remains the application protocol endpoint attached to that connection.

So the first teaching comparison is not:

IPv4 code versus completely different IPv6 code

It is:

```
one SNode.C communication model
-> specialized through IPv4
-> specialized through IPv6
```

That is the model to keep in mind while reading the examples. The lower namespace changes, the address family changes, and deployment assumptions may change. The server/client/connection/context pattern remains structurally the same.

Parallel type names

The type names show the parallelism immediately.

For IPv4, a minimal legacy stream pair often begins like this:

```
using EchoServer4 = net::in::stream::legacy::SocketServer<MyFactory>;
using EchoClient4 = net::in::stream::legacy::SocketClient<MyFactory>;
```

For IPv6, the corresponding pair is:

```
using EchoServer6 = net::in6::stream::legacy::SocketServer<MyFactory>;
using EchoClient6 = net::in6::stream::legacy::SocketClient<MyFactory>;
```

The first visible difference is the network-family namespace:

```
net::in    -> IPv4
net::in6   -> IPv6
```

The rest of the layered name remains recognizably stable:

```
stream
  -> legacy
    -> SocketServer / SocketClient
      -> MyFactory
```

This is the kind of layered reading introduced in Chapter 7. A long `SNode.C` type name says which lower family is used, which transport form is used, which connection variant is selected, which application-side handle type is visible, and which factory creates the per-connection protocol contexts.

The IPv4/IPv6 comparison is therefore visible before a single socket operation is called. The type name already tells the reader which lower family has been selected.

Parallel address classes

Chapter 8 introduced address semantics. That model now applies directly to the two IP families.

Family	Address class	Basic endpoint shape
IPv4	<code>net::in::SocketAddress</code>	host plus port
IPv6	<code>net::in6::SocketAddress</code>	host plus port

The class names differ because the endpoint families differ. The basic shape is similar because both families use a host-plus-port model.

That is the right balance. One address pattern transfers across both IP families, but the similarity should not be exaggerated. IPv4 and IPv6 have different textual forms, different wildcard forms, and different deployment questions.

The useful lesson is:

Parallel-shape note

The API shape is intentionally parallel where that helps, but the endpoint families remain distinct.

Here “host” should be read broadly. It may be a numeric address such as `127.0.0.1` or `::1`, or it may be a name that has to be resolved. The convenience API makes the common case readable, but Chapter 8’s address model still matters underneath.

Parallel convenience overloads

The stream wrappers make the shared model especially visible.

The IPv4 and IPv6 server wrappers provide parallel `listen(...)` convenience overloads. These overloads set local configuration such as host, port, and backlog, then delegate to the general `listen(onStatus)` path.

Likewise, the IPv4 and IPv6 client wrappers provide parallel `connect(...)` convenience overloads. These overloads set the remote host and port, optionally set local bind information, and then delegate to the general `connect(onStatus)` path.

So the convenience API is not a separate communication model. It configures the application-side handle and then enters the same registration path that Chapter 9 described.

The pattern can be summarized like this:

Surface call expresses	Configuration effect
<code>listen(port, ...)</code>	local port
<code>listen(port, backlog, ...)</code>	local port + backlog
<code>listen(host, port, ...)</code>	local host + local port
<code>listen(host, port, backlog, ...)</code>	local host + local port + backlog

Surface call expresses	Configuration effect
<code>connect(host, port, ...)</code>	remote host + remote port
<code>connect(host, port, bindHost, ...)</code>	remote endpoint + local host
<code>connect(host, port, bindPort, ...)</code>	remote endpoint + local port
<code>connect(host, port, bindHost, bindPort, ...)</code>	remote endpoint + full local bind endpoint

The exact overload count is less important than the pattern.

The architectural idea is:

Easy-looking calls set configuration and then use the same underlying registration and flow machinery.

This is important for understanding `SNode.C` code. A short `listen(8080, ...)` call is not a different runtime concept from constructing a `SocketAddress` and calling the more general path. It is a convenience surface over the same role configuration and registration model.

10.3 What remains stable

Moving from IPv4 to IPv6 should not feel like changing the application architecture. The stable core remains the same.

Aspect	IPv4	IPv6	Architectural point
Application-side handle	<code>SocketServer</code> / <code>SocketClient</code>	<code>SocketServer</code> / <code>SocketClient</code>	handle shape stays
Registered instance	server-side or client-side role	server-side or client-side role	role model stays
Connection object	<code>SocketConnection</code>	<code>SocketConnection</code>	concrete peer relationship stays
Context factory	same pattern	same pattern	context creation transfers
Protocol context	same class often possible	same class often possible	protocol logic can transfer
Runtime	same event-driven model	same event-driven model	runtime understanding transfers
Status callback	<code>SocketAddress, State</code>	<code>SocketAddress, State</code>	outer role status model stays
Lifecycle callbacks	connection callbacks	connection callbacks	connection lifecycle model stays

This is the main transfer point.

If the application protocol is written in a `SocketContext`, it does not automatically become an “IPv4 protocol” or an “IPv6 protocol.” It is application behavior carried over a lower communication family. The same context class can often be reused when the protocol behavior does not inspect or depend on family-specific address details.

What changes is the lower family and therefore the endpoint representation, bind/connect configuration, and deployment environment.

What often remains stable is the protocol class, the factory structure, the connection lifecycle, the runtime, and the overall server/client model.

Runtime and instance model

The runtime is not replaced when the network family changes.

An IPv4 server and an IPv6 server still participate in the same kind of event-driven runtime. They still register communication intent through `listen(...)`. They still produce connection objects. They still attach contexts through factories.

The same applies on the client side. An IPv4 client and an IPv6 client both register connection intent through `connect(...)`, enter the runtime and flow-controller machinery, and produce concrete connection episodes when the connection succeeds.

IPv4 and IPv6 therefore show that the runtime and instance model are not tied to one address family. The event loop does not become an “IPv4 event loop” or an “IPv6 event loop.” The lower family affects the physical and address layer. The runtime architecture remains the same.

Factory, context, and protocol logic

The factory/context pattern also transfers.

A simple echo protocol implemented as a `SocketContext` can often be used with both IPv4 and IPv6 roles. The context does not need to care whether the peer endpoint was represented by an IPv4 or IPv6 address unless the protocol itself inspects, prints, filters, or interprets that address.

This separation keeps protocol logic from being unnecessarily tied to one lower family.

The factory carries the rule for creating the context. The context carries the protocol behavior. The selected IP family supplies the endpoint family and the lower communication path. Keeping those responsibilities separate is what makes the IPv4/IPv6 comparison useful instead of duplicating the whole application.

Status and lifecycle callbacks

Chapter 9 distinguished status callbacks from connection lifecycle callbacks. That distinction still applies here.

A `listen(...)` or `connect(...)` status callback reports outer role status. It sees the relevant address object and a `core::socket::State` value.

Connection lifecycle callbacks observe concrete connections. They receive a `SocketConnection*` and can inspect local and remote addresses, timing, metrics, and connection names.

Context callbacks implement protocol behavior.

Moving from IPv4 to IPv6 does not collapse those callback layers. The lower-family namespace changes, but the callback model remains the same.

10.4 What changes

The stable structure does not mean IPv4 and IPv6 are identical. They are not.

The goal is not to turn this into a full IPv6 chapter, but to show where `SNode.C` keeps the architecture stable and where the lower family still matters.

Namespace and address family

The most visible code change is the namespace:

```
net::in
  -> IPv4

net::in6
  -> IPv6
```

This is not cosmetic. It tells the reader that the lower communication family has changed.

The address class changes too:

```
net::in::SocketAddress
net::in6::SocketAddress
```

Both are host-plus-port address classes, but they belong to different endpoint families. That distinction becomes important in examples, diagnostics, wildcard binding, and deployment.

The namespace is therefore a compact architectural statement:

```
same stream model
same legacy/TLS layer choice
same server/client handle pattern
different internet family
```

Textual address form and wildcard form

The textual address form changes.

IPv4 examples often use addresses such as:

```
127.0.0.1
0.0.0.0
```

IPv6 examples often use addresses such as:

```
::1
::
```

The wildcard form also changes:

Family	Wildcard host	Wildcard port
IPv4	0.0.0.0	0
IPv6	::	0

Chapter 8 already explained address semantics in detail. Here the important point is simply that the wildcard idea transfers, but its concrete family representation changes.

The reader should not think:

```
IPv6 is just IPv4 with longer text.
```

IPv6 has its own address family and its own operational questions, even though the SNode.C API is intentionally parallel.

Local and remote configuration

The convenience overloads also make local and remote roles visible.

For a server, the most important early configuration is local:

```
local host
local port
backlog
```

For a client, the most important early configuration is remote:

```
remote host
remote port
```


and, optionally:

```
local bind host
local bind port
```

This directly continues Chapter 8 and Chapter 9.

Chapter 8 explained what endpoint identity means. Chapter 9 explained how servers and clients use endpoint identity. Chapter 10 shows that IPv4 and IPv6 use the same local/remote role distinction with different address-family semantics.

IPv6-only, IPv4-mapped, and dual-stack thinking

IPv6 introduces one additional technical point: deployment can involve IPv6-only behavior, IPv4-mapped IPv6 addresses, or platform-specific dual-stack behavior.

This topic should not overwhelm the first example. It should also not be hidden.

At this stage, the important point is not to master every platform switch. The important point is to notice that IPv6 is a separate endpoint family with deployment semantics of its own.

An IPv6 endpoint is not only a different address string. It may also raise configuration and platform questions about whether IPv4 traffic is included, excluded, or represented through IPv4-mapped IPv6 addresses.

That is one reason SNode.C keeps IPv6 as its own family instead of treating it as an overloaded IPv4 mode.

10.5 From IPv4 to IPv6

The move from IPv4 to IPv6 should make transfer visible without overwhelming the reader.

The useful reading path is simple. First, look at a minimal IPv4 legacy pair. Then look at the corresponding IPv6 legacy pair. Then compare what changed in code and what stayed stable in architecture. Later chapters can repeat the same comparison under TLS.

This keeps the lower-family comparison sharp. It also prepares the next variations: after IPv4 and IPv6, Unix domain sockets and Bluetooth can be introduced as more different endpoint families while the server/client/connection model remains familiar.

Start with IPv4

IPv4 remains the easiest first concrete example for many readers. That is not because it is more important. It is because its notation and everyday expectations are familiar.

A minimal IPv4 server may look like this in outline:

```
using EchoServer4 = net::in::stream::legacy::SocketServer<MyFactory>;

EchoServer4 server("echo4");
server.listen(8080, 5, onStatus);
```

The matching client may look like this:

```
using EchoClient4 = net::in::stream::legacy::SocketClient<MyFactory>;

EchoClient4 client("echo4");
client.connect("127.0.0.1", 8080, onStatus);
```

The numeric loopback address keeps the family comparison explicit. A host name may also be used where name resolution is desired, but 127.0.0.1 makes it clear that this first example is deliberately IPv4.

This path shows host plus port, server plus client, status callback, and runtime integration before IPv6 notation or dual-stack questions enter the picture.

Move immediately to IPv6

IPv6 should follow soon after the first stable IPv4 picture.

If IPv6 appears only much later, IPv4 can start to look like the normal case and IPv6 like an advanced appendix. That would be the wrong lesson for SNode.C.

The IPv6 version should feel deliberately close:

```
using EchoServer6 = net::in6::stream::legacy::SocketServer<MyFactory>;

EchoServer6 server("echo6");
server.listen(8080, 5, onStatus);
```

and:

```
using EchoClient6 = net::in6::stream::legacy::SocketClient<MyFactory>;

EchoClient6 client("echo6");
client.connect("::1", 8080, onStatus);
```

The important comparison is:

```
net::in    -> net::in6
127.0.0.1 -> ::1
```

The outer structure remains stable. The lower family and address notation change.

Compare what changed and what did not

Once the IPv4 and IPv6 examples are side by side, the comparison becomes clear. What changed?

- the network-family namespace,
- the address class,
- address text,
- wildcard representation,
- possible IPv6-specific deployment questions.

What did not change?

- application-side server and client handle shape,
- registered instance model,
- connection model,
- context factory,
- context class shape,
- runtime startup,
- status callback model,
- connection lifecycle model,
- layered reading of the type name.

This comparison is the core result. The point is controlled variation across lower communication families, with IPv4 and IPv6 as the teaching path.

10.6 Stable model, different family semantics

The following table is the transfer map for the chapter. It shows which ideas stay stable and which ones move with the lower family.

Aspect	IPv4	IPv6	Teaching point
Namespace	<code>net::in</code>	<code>net::in6</code>	lower family changes

Aspect	IPv4	IPv6	Teaching point
Address class	<code>net::in::SocketAddress</code>	<code>net::in6::SocketAddress</code>	family-specific endpoint identity
Address shape	host + port	host + port	interface is parallel
Wildcard host	<code>0.0.0.0</code>	<code>::</code>	representation differs
Application-side handle	<code>SocketServer / SocketClient</code>	<code>SocketServer / SocketClient</code>	handle shape stays
Registered instance	server/client role	server/client role	role model stays
Connection	<code>SocketConnection</code>	<code>SocketConnection</code>	peer relationship model stays
Protocol context	same class often possible	same class often possible	protocol logic can transfer
Convenience API	sets config, delegates	sets config, delegates	easy API and configured API are one model
Extra concern	simpler first case	IPv6-only / IPv4-mapped / dual-stack questions	deployment semantics differ

This table connects directly to Chapters 8 and 9. Chapter 8 explains the address rows. Chapter 9 explains the handle, instance, connection, factory, context, and callback rows.

This chapter shows IPv4 and IPv6 as the first concrete comparison pair.

What to remember

- IPv4 and IPv6 are the first concrete lower-family comparison because they are familiar, parallel, and still meaningfully different.
- `net::in` selects IPv4; `net::in6` selects IPv6.
- Both families use host-plus-port endpoint identity, but their textual forms, wildcard forms, and deployment questions differ.
- The application-side server/client handle, registered instance, connection, factory, context, and runtime model stay structurally the same.
- Convenience `listen(...)` and `connect(...)` overloads set configuration and then delegate to the general registration path.
- Protocol contexts can often be reused across IPv4 and IPv6 when the protocol itself does not depend on address-family details.

10.7 Public surface of the primary families

IPv4 and IPv6 are the simplest place to see the source/build pairing. A file that directly names an IPv4 legacy stream role uses the IPv4 public role headers:

```
#include <net/in/stream/legacy/SocketServer.h>
#include <net/in/stream/legacy/SocketClient.h>
```

with the matching component:

```
net-in-stream-legacy
```

The IPv6 variant changes only the family fragment:

```
#include <net/in6/stream/legacy/SocketServer.h>
#include <net/in6/stream/legacy/SocketClient.h>
```

with the corresponding component:

```
net-in6-stream-legacy
```

The local point is the family selection. Chapter 32 gives the consolidated include/component matrix.

10.8 Closing perspective

IPv4 and IPv6 keep the familiar host-plus-port view while introducing the lower-family distinction that later chapters vary. Unix-domain sockets are the next controlled change: the role model remains stable, but endpoint identity becomes local and path-based.

Chapter 11

Unix Domain Sockets

11.1 From host-plus-port to local path identity

Chapter 10 used IPv4 and IPv6 as the first concrete lower-family comparison. Both families still used a host-plus-port endpoint identity.

This chapter keeps the same SNode.C runtime model, but changes endpoint identity more strongly:

```
host + port
-> local Unix-domain path
```

That is the next controlled variation in Part III.

With IPv4 and IPv6, an endpoint is described by an address and a port. With Unix domain sockets, an endpoint is described primarily by a local path. The endpoint no longer looks like an internet endpoint at all. It is a local interprocess-communication identity.

The larger SNode.C model remains recognizable:

- an application-side `SocketServer` or `SocketClient` handle,
- a registered server-side or client-side instance,
- a concrete `SocketConnection`,
- a socket-context factory,
- a per-connection socket context,
- runtime integration,
- status callbacks,
- connection lifecycle callbacks,
- and the same broad stream-based server/client/connection model.

This shows that SNode.C's role model is not tied to host-plus-port addressing. The point of the chapter is therefore not only to introduce another address class. It is to test whether the mental model from the previous chapters survives when the most familiar shape of a socket endpoint disappears.

11.2 Same SNode.C model, different endpoint identity

The transition from IPv4/IPv6 to Unix domain sockets can be summarized as follows.

Aspect	IPv4 / IPv6	Unix domain sockets
Namespace	<code>net::in</code> , <code>net::in6</code>	<code>net::un</code>
Endpoint identity	host + port	local path
Server bind value	local host + port	local socket path
Client target	remote host + port	remote socket path
Optional local bind	local host and/or port	local socket path
Runtime model	stable	stable

Aspect	IPv4 / IPv6	Unix domain sockets
Server/client role model	stable	stable
Context/factory model	stable	stable

The key point is the controlled change. The lower communication family changes. The endpoint identity changes.

The application architecture does not need to be reinvented.

The table should therefore be read as a transfer map, not as a feature comparison. It shows which ideas move unchanged into the Unix-domain family and which words must be reinterpreted once endpoint identity becomes path-based.

This is the same teaching pattern as in Chapter 10, but the shift is stronger. IPv4 and IPv6 were different internet families with a similar host-plus-port shape. Unix domain sockets change the addressing category itself.

11.3 Path identity in `net::un::SocketAddress`

The Unix-domain address class is:

```
net::un::SocketAddress
```

Its surface is smaller than the IPv4 and IPv6 address classes because the endpoint identity is smaller. There is no host name to resolve, no internet port number, and no candidate list of remote network addresses. The address object represents a Unix-domain path and the socket-address structure built from it.

Its conceptual surface includes:

- default construction,
- construction from a `sunPath`,
- construction from an existing Unix-domain `sockaddr`,
- initialization,
- `setSunPath(...)` and `getSunPath()`,
- and string rendering.

This surface reflects the family.

A Unix-domain endpoint is not a host with a strange name. It is a local IPC endpoint identified primarily by a path.

Path as endpoint identity

When using Unix domain sockets, the path should be understood as endpoint identity in the local IPC space.

A typical endpoint may look like:

```
/tmp/my-service.sock
```

In application code this may appear as:

```
net::un::SocketAddress address("/tmp/my-service.sock");
```

The path is the name through which local processes identify the communication endpoint. In practice, it acts as a rendezvous name inside the local operating-system environment: one side creates or binds that endpoint identity, and the other side uses the same identity to reach the service.

This avoids two common wrong instincts:

- treating Unix domain sockets as IP sockets without a network,
- or treating the socket path as an ordinary file with no communication semantics.

The better view is:

```
path
-> local endpoint identity
    -> stream communication endpoint
```

That is why Unix domain sockets belong in this part of the book. They are not a new application protocol. They are a different lower-family endpoint model under the same `SNode.C` stream architecture.

Default construction and empty path

Default construction is meaningful in the Unix-domain address model.

In the `SNode.C` address vocabulary used here, an empty Unix-domain path acts as the wildcard or deferred endpoint indicator for this family. It gives `SNode.C` a way to represent an address object whose concrete path has not yet been made specific.

The point is not to turn this into a long operating-system detour. The important point is consistency across families:

Family	Default / wildcard shape
IPv4	0.0.0.0, port 0
IPv6	::, port 0
Unix domain sockets	empty path

The wildcard idea transfers.

The concrete representation changes. This is the useful reading habit for all lower families in `SNode.C`: the abstract role of a default address may be comparable, but the actual meaning is always expressed in the vocabulary of the selected family.

Locality as the defining idea

A Unix domain socket is local interprocess communication. It is not used to reach a peer on another machine over an IP network. This changes the deployment questions.

With IPv4 or IPv6 one often asks:

- Which interface should listen?
- Which port should be exposed?
- Which remote host should be contacted?
- Which firewall or routing rule might be involved?

With Unix domain sockets one more often asks:

- Which local path represents this service?
- Which local processes are allowed to connect?
- Where does this endpoint belong in the local service layout?
- Who owns the lifecycle of that path?
- What happens if an old path is left behind?

Unix domain sockets are therefore especially useful for:

- local service boundaries,
- helper daemons,
- internal process-to-process APIs,
- development setups where network exposure is unnecessary,
- appliance-like or embedded systems where components communicate locally.

The lower-family choice is not decorative. It changes the operational shape of the system. The code can keep the same server/client outline, but the questions a developer asks while deploying and diagnosing the application become local-service questions rather than network-reachability questions.

11.4 Server and client use with path-based endpoints

The stream Unix-domain wrappers follow the same SNode.C pattern as the IPv4 and IPv6 wrappers.

The convenience calls set family-specific configuration and then delegate to the general `listen(onStatus)` or `connect(onStatus)` path.

The difference is that the configured value is now a Unix-domain path. That keeps the application code pleasantly familiar while still making the lower-family decision explicit. A reader should see both sides at once: the call shape transfers, but the endpoint value has a different meaning.

Server-side `listen(...)`

A Unix-domain stream server typically looks like this in outline:

```
using LocalServer = net::un::stream::legacy::SocketServer<MyFactory>;

LocalServer server("local-service");
server.listen("/tmp/my-service.sock", 5, onStatus);
```

The visible `LocalServer` object is the application-side handle. The `listen(...)` call configures the server-side path and registers the server-side instance through the usual runtime path.

The server is still a server role.

The call is still a `listen(...)` call.

The status callback still reports outer role status.

The difference is the endpoint identity:

```
IPv4 / IPv6:
local host + local port

Unix domain:
local socket path
```

The server-side convenience overloads are path-centered:

Surface call	Configuration effect
<code>listen(sunPath, ...)</code>	<code>Local::setSunPath(sunPath)</code>
<code>listen(sunPath, backlog, ...)</code>	<code>Local::setSunPath(sunPath) + backlog</code>

This mirrors the Chapter 10 pattern. A readable convenience call configures the application-side handle and then enters the same registration path.

Client-side `connect(...)`

A Unix-domain stream client has the same overall shape:

```
using LocalClient = net::un::stream::legacy::SocketClient<MyFactory>;

LocalClient client("local-client");
client.connect("/tmp/my-service.sock", onStatus);
```


Here the visible `LocalClient` object is again the application-side handle. The `connect(...)` call configures the remote Unix-domain service path and registers the client-side instance through the usual runtime path.

If the client also needs an explicit local bind path, the call can express that too:

```
client.connect("/tmp/my-service.sock", "/tmp/my-client.sock", onStatus);
```

The client-side convenience overloads are also path-centered:

Surface call	Configuration effect
<code>connect(sunPath, ...)</code>	<code>Remote::setSunPath(sunPath)</code>
<code>connect(sunPath, bindSunPath, ...)</code>	<code>Remote::setSunPath(sunPath) +</code> <code>Local::setSunPath(bindSunPath)</code>

This keeps the local/remote distinction visible. The remote path is the service endpoint the client wants to reach.

The optional local path describes the client's own local endpoint identity. Most simple clients do not need to spell this out, but the overload exists because the model still distinguishes the peer being contacted from the local endpoint used for the connection.

Local and remote paths

Path-based endpoint identity does not remove the local/remote distinction. It only changes what the endpoint value looks like.

For a Unix-domain server:

```
local path
-> path on which the server listens
```

For a Unix-domain client:

```
remote path
-> service path to connect to

optional local path
-> explicit local endpoint for the client side
```

This continues Chapter 9's connection model.

A connection can still have bind, local, and remote address views. The address family has changed, but directional endpoint thinking remains useful. This is one of the main reasons to keep the address family visible in the type system: it prevents the local/remote distinction from being flattened into an unhelpful generic string.

11.5 What remains stable

Unix domain sockets change endpoint identity, but they do not require a different application architecture.

Server/client/connection/context model

The same core model remains:

Concept	Meaning with Unix domain sockets
Application-side <code>SocketServer</code> handle	configures and registers a server-side instance
Application-side <code>SocketClient</code> handle	configures and registers a client-side instance

Concept	Meaning with Unix domain sockets
Registered instance	runtime-visible server/client role using <code>net::un</code> endpoint semantics
<code>SocketConnection</code>	one concrete peer relationship
<code>SocketContextFactory</code>	creates a context for a connection
<code>SocketContext</code>	implements application protocol behavior

The server-side instance is still the listening role. The client-side instance is still the connecting role. The connection is still the concrete peer relationship.

The context is still the protocol endpoint attached to that connection.

Therefore, moving from IPv4 to Unix domain sockets should not feel like moving to a different framework. The lower family has changed, not the architecture. For application developers this is an important payoff: once the instance, connection, factory, and context boundaries are understood, a new endpoint family does not force the whole program shape to be relearned.

Context and protocol logic

A `SocketContext` implementing a small request/response or streaming protocol does not need to become fundamentally Unix-domain-specific just because it runs over:

```
net::un::stream::legacy
```

What changes is the carrier.

What often remains stable is:

- how the protocol reacts to connection establishment,
- how it reads data,
- how it sends data,
- how it handles disconnection,
- how it thinks in terms of one connection and one context.

The same context class can often be reused when the protocol behavior does not depend on family-specific address details.

The protocol context may inspect the address if it wants to log or display endpoint information. But the protocol logic itself can often remain the same. This is the separation the book is building toward: lower layers decide how peers are reached, while the context concentrates on what the application protocol does once a connection exists.

Legacy and TLS

Chapter 7 introduced `legacy` and `tls` as connection-layer variants. Unix domain sockets fit into that same layer story.

A Unix-domain stream component can participate in the pattern:

```
net::un
-> stream
    -> legacy or TLS
```

Here `legacy` is the non-TLS stream connection variant. `tls` adds TLS connection handling.

This matters because Unix domain sockets are not a special branch outside the framework. In `SNode.C`'s terminology, they are another lower communication family in the network-layer part of the stack, while stream transport and connection-layer handling remain recognizable.

11.6 What changes operationally

Unix domain sockets are local, path-based endpoints. That changes deployment habits.

Deployment habits

With IP sockets, operational thinking often starts with:

```
address
port
interface
network reachability
```

With Unix domain sockets, operational thinking starts with:

```
local path
local process boundary
local access expectations
path ownership
path cleanup
```

That is a real design difference.

Unix domain sockets are often the right choice when the communication should stay inside one machine. They are not the right choice when the peer must be reached over a network.

SNode.C makes that family choice explicit instead of hiding it behind one generic endpoint abstraction. That explicitness is useful during design reviews as well as during debugging: the namespace and address type already tell the reader that this is local IPC, not a network-facing endpoint.

Path ownership and cleanup thinking

When an endpoint is identified by a path, responsible design includes path lifecycle thinking.

An application should be clear about:

- where the socket path is placed,
- which process creates it,
- which process is allowed to use it,
- what permissions or ownership expectations belong to the path,
- what happens when the service stops,
- what happens if an old path is left behind.

This does not mean Unix domain sockets are complicated.

It means the family expresses locality through path identity, and application design should respect that. A path under a temporary directory, a runtime directory, or a service-specific directory communicates different operational expectations. The book does not need to prescribe one layout here, but the chapter should make clear that the path is part of the service design, not an incidental string literal.

Unix domain sockets are not a replacement for IP

Unix domain sockets are not better or worse IP sockets. They answer a different design question.

They are excellent when communication is local to one machine. They are not suitable when a process must communicate with a peer on another machine over a network.

The right question is therefore not:

```
Are Unix domain sockets better than IP?
```

The better question is:

Is this communication local enough that path-based IPC is the right lower family?

This keeps the comparison architectural rather than emotional. The lower family should match the communication boundary.

11.7 Stream focus and a datagram note

It focuses on stream Unix domain sockets. That is intentional for architectural continuity. Stream Unix-domain sockets preserve the server/client/connection/context model used throughout this part of the book.

The SNode.C build also contains a `net-un-dgram` component. That is useful to know, but it is not the focus here. Datagram communication introduces a different communication shape and should not distract from the stream-based role model being developed in Chapters 8–12.

The practical lesson is simple: Unix domain sockets are not limited to one possible socket style, but this chapter follows the stream path because the book is still building the layered server/client/connection model.

11.8 Preparing the Bluetooth shift

Unix domain sockets are also a useful bridge to Bluetooth. IPv4 and IPv6 showed host-plus-port endpoint identity. Unix domain sockets showed path-based local endpoint identity.

Bluetooth will introduce endpoint identities that are different again:

```
Bluetooth address + RFCOMM channel
Bluetooth address + L2CAP PSM
```

After Chapter 11, the reader has seen two kinds of endpoint shift: internet host-plus-port identity and local path identity. Chapter 12 adds device address plus service selector.

The server/client/connection/context model remains available, while the lower-family address semantics change again. That prepares the next chapter: Bluetooth should not feel like an exception to the architecture, but like another controlled variation in endpoint identity.

What to remember

- Unix domain sockets replace host-plus-port endpoint identity with local path identity.
- `net::un` selects the Unix-domain family, and `net::un::SocketAddress` represents the path-based endpoint.
- The application-side handle, registered instance, connection, factory, context, callbacks, and runtime model remain structurally familiar.
- `listen(sunPath, ...)` configures the server's local Unix-domain path; `connect(sunPath, ...)` configures the client's remote Unix-domain service path.
- `connect(sunPath, bindSunPath, ...)` keeps the local/remote distinction visible even though both endpoints are path-based.
- The empty Unix-domain path is the family's wildcard or deferred endpoint representation in the SNode.C address model.

11.9 Public surface of Unix-domain stream roles

Unix-domain legacy stream roles use the `un` family fragment on both public surfaces. Source files that directly name those roles include:

```
#include <net/un/stream/legacy/SocketServer.h>
#include <net/un/stream/legacy/SocketClient.h>
```

and link the matching component:

```
net-un-stream-legacy
```

The same include/component rule from the primary families applies; only the family fragment changes. The consolidated mapping is collected in Chapter 32.

11.10 Closing perspective

Unix-domain sockets replace host-plus-port identity with path identity while preserving the same application shape. Bluetooth is the next variation: endpoint identity becomes device-oriented and family-specific service selectors enter the address model.

Chapter 12

Bluetooth in SNode.C: RFCOMM and L2CAP

12.1 From path identity to Bluetooth endpoint identity

Chapter 11 replaced host-plus-port endpoint identity with Unix-domain path identity. This chapter changes endpoint identity again.

Bluetooth RFCOMM and L2CAP are neither host-plus-port families nor path-based local IPC families. They use Bluetooth device identity together with a family-specific service selector:

RFCOMM:
Bluetooth address + channel

L2CAP:
Bluetooth address + PSM

After IPv4, IPv6, and Unix domain sockets, Bluetooth is the final lower-family variation in this part of the book before the focus moves upward to application protocol contexts.

The larger SNode.C model remains recognizable:

- an application-side `SocketServer` or `SocketClient` handle,
- a registered server-side or client-side instance,
- a concrete `SocketConnection`,
- a socket-context factory,
- a per-connection socket context,
- runtime integration,
- status callbacks,
- connection lifecycle callbacks,
- and the same broad stream-based server/client/connection model.

The lower communication family changes. The endpoint identity changes. The application architecture does not need to be reinvented.

12.2 Two Bluetooth families, two service selectors

SNode.C represents Bluetooth through two distinct lower communication families:

Aspect	RFCOMM	L2CAP
Namespace	<code>net::rc</code>	<code>net::l2</code>
Address class	<code>net::rc::SocketAddress</code>	<code>net::l2::SocketAddress</code>
Endpoint identity	Bluetooth address + channel	Bluetooth address + PSM

Aspect	RFCOMM	L2CAP
Service selector	channel	PSM
Server role	<code>SocketServer</code>	<code>SocketServer</code>
Client role	<code>SocketClient</code>	<code>SocketClient</code>
Context/factory model	stable	stable
Runtime model	stable	stable

This table contains the main distinction of the chapter.

RFCOMM and L2CAP share the idea of Bluetooth device identity. They differ in the service selector that identifies the communication endpoint within that Bluetooth family.

That difference is not cosmetic. An RFCOMM channel is not an L2CAP PSM, and a PSM is not an RFCOMM channel.

RFCOMM: Bluetooth address plus channel

RFCOMM endpoint identity is built from:

```
Bluetooth device address
+ RFCOMM channel
```

A typical address-like pair may therefore look like:

```
10:3D:1C:AC:BA:9C
channel 16
```

The channel is the RFCOMM service selector. It should not be treated as a generic “Bluetooth port” in the IP sense.

In `SNode.C` this distinction appears directly in the API:

```
setBtAddress(...)
setChannel(...)
```

That naming is important because it keeps RFCOMM vocabulary visible in code.

L2CAP: Bluetooth address plus PSM

L2CAP endpoint identity is built from:

```
Bluetooth device address
+ L2CAP PSM
```

A typical address-like pair may therefore look like:

```
10:3D:1C:AC:BA:9C
PSM 0x1001
```

The PSM is the L2CAP service selector. It is not the same concept as an RFCOMM channel.

In `SNode.C` this distinction appears directly in the API:

```
setBtAddress(...)
setPsm(...)
```

The shared Bluetooth-address part makes RFCOMM and L2CAP look related. The service-selector part keeps them distinct.

Channel and PSM are not interchangeable

The most important conceptual mistake would be to collapse RFCOMM and L2CAP into one vague “Bluetooth socket” idea. They belong in the same chapter because they are both Bluetooth-related lower communication families.

They must still remain conceptually separate:

```
RFCOMM channel != L2CAP PSM
```

SNode.C preserves this separation in three places:

- separate namespaces: `net::rc` and `net::l2`,
- separate address classes,
- separate family-specific setters and getters.

This prevents the convenience of a shared framework pattern from hiding the real endpoint semantics.

12.3 Bluetooth address classes

The address classes make the RFCOMM/L2CAP distinction concrete.

Their surface is not arbitrary. The setters, getters, constructors, and string rendering expose the two pieces of endpoint identity for each family.

`net::rc::SocketAddress`

The RFCOMM address class is:

```
net::rc::SocketAddress
```

Its conceptual surface includes:

- default construction,
- construction from a Bluetooth address,
- construction from a channel,
- construction from Bluetooth address plus channel,
- construction from an existing RFCOMM socket address,
- initialization,
- `setBtAddress(...)` and `getBtAddress()`,
- `setChannel(...)` and `getChannel()`,
- and string rendering.

This mirrors the endpoint identity:

```
Bluetooth address + channel
```

A compact code anchor is:

```
net::rc::SocketAddress address("10:3D:1C:AC:BA:9C", 16);
```

`net::l2::SocketAddress`

The L2CAP address class is:

```
net::l2::SocketAddress
```

Its conceptual surface includes:

- default construction,
- construction from a Bluetooth address,

- construction from a PSM,
- construction from Bluetooth address plus PSM,
- construction from an existing L2CAP socket address,
- initialization,
- `setBtAddress(...)` and `getBtAddress()`,
- `setPsm(...)` and `getPsm()`,
- and string rendering.

This mirrors the endpoint identity:

```
Bluetooth address + PSM
```

A compact code anchor is:

```
net::l2::SocketAddress address("10:3D:1C:AC:BA:9C", 0x1001);
```

Default construction and wildcard Bluetooth address

Default construction follows the same general address-model idea introduced in Chapter 8.

In the SNode.C address model used here, default Bluetooth construction belongs to the same wildcard or deferred-endpoint pattern as the earlier families: wildcard Bluetooth address plus a zero service selector.

For Bluetooth address families, the wildcard Bluetooth address is represented as:

```
00:00:00:00:00:00
```

The service selector starts at zero:

Family	Default / wildcard shape
RFCOMM	00:00:00:00:00:00, channel 0
L2CAP	00:00:00:00:00:00, PSM 0

This does not make RFCOMM and L2CAP the same family.

It means the broad default/wildcard idea exists in both Bluetooth address models, while the service selector remains family-specific.

12.4 Server and client use with Bluetooth endpoints

The stream Bluetooth wrappers follow the SNode.C pattern already seen for IPv4, IPv6, and Unix domain sockets. The convenience calls configure the application-side handle and then enter the usual registration path.

The difference is the configured endpoint identity:

```
RFCOMM:
Bluetooth address + channel
```

```
L2CAP:
Bluetooth address + PSM
```

RFCOMM server-side `listen(...)`

A simple RFCOMM server may look like this:

```
using RcServer = net::rc::stream::legacy::SocketServer<MyFactory>;

RcServer server("echo-rc");
server.listen(16, 5, onStatus);
```

or with an explicit local Bluetooth address:

```
server.listen("10:3D:1C:AC:BA:9C", 16, 5, onStatus);
```

The server-side convenience overloads are channel-centered:

Surface call	Configuration effect
<code>listen(channel, ...)</code>	<code>Local::setChannel(channel)</code>
<code>listen(channel, backlog, ...)</code>	<code>Local::setChannel(channel) + backlog</code>
<code>listen(btAddress, channel, ...)</code>	<code>Local::setBtAddress(btAddress)->setChannel(channel)</code>
<code>listen(btAddress, channel, backlog, ...)</code>	<code>Local::setBtAddress(btAddress)->setChannel(channel) + backlog</code>

The server still registers a listening instance. The endpoint identity is now RFCOMM-specific.

RFCOMM client-side connect(...)

A simple RFCOMM client may look like this:

```
using RcClient = net::rc::stream::legacy::SocketClient<MyFactory>;

RcClient client("echo-rc-client");
client.connect("10:3D:1C:AC:BA:9C", 16, onStatus);
```

The client-side convenience overloads are also channel-centered:

Surface call	Configuration effect
<code>connect(btAddress, channel, ...)</code>	<code>Remote::setBtAddress(btAddress)->setChannel(channel)</code>
<code>connect(btAddress, channel, bindBtAddress, ...)</code>	remote address/channel + <code>Local::setBtAddress(bindBtAddress)</code>
<code>connect(btAddress, channel, bindChannel, ...)</code>	remote address/channel + <code>Local::setChannel(bindChannel)</code>
<code>connect(btAddress, channel, bindBtAddress, bindChannel, ...)</code>	remote address/channel + <code>Local::setBtAddress(bindBtAddress)->setChannel(bindChannel)</code>

The remote side is the peer Bluetooth address plus RFCOMM channel. The optional local side can refine the local Bluetooth address, the local channel, or both.

L2CAP server-side listen(...)

A simple L2CAP server may look like this:

```
using L2Server = net::l2::stream::legacy::SocketServer<MyFactory>;

L2Server server("echo-l2");
server.listen(0x1001, 5, onStatus);
```

or with an explicit local Bluetooth address:

```
server.listen("10:3D:1C:AC:BA:9C", 0x1001, 5, onStatus);
```

The server-side convenience overloads are PSM-centered:

Surface call	Configuration effect
<code>listen(psm, ...)</code>	<code>Local::setPsm(psm)</code>
<code>listen(psm, backlog, ...)</code>	<code>Local::setPsm(psm) + backlog</code>
<code>listen(btAddress, psm, ...)</code>	<code>Local::setBtAddress(btAddress)->setPsm(psm)</code>
<code>listen(btAddress, psm, backlog, ...)</code>	<code>Local::setBtAddress(btAddress)->setPsm(psm) + backlog</code>

The structure mirrors RFCOMM. The service selector changes.

L2CAP client-side connect(...)

A simple L2CAP client may look like this:

```
using L2Client = net::l2::stream::legacy::SocketClient<MyFactory>;

L2Client client("echo-l2-client");
client.connect("10:3D:1C:AC:BA:9C", 0x1001, onStatus);
```

The client-side convenience overloads are PSM-centered:

Surface call	Configuration effect
<code>connect(btAddress, psm, ...)</code>	<code>Remote::setBtAddress(btAddress)->setPsm(psm)</code>
<code>connect(btAddress, psm, bindBtAddress, ...)</code>	remote address/PSM + <code>Local::setBtAddress(bindBtAddress)</code>
<code>connect(btAddress, psm, bindPsm, ...)</code>	remote address/PSM + <code>Local::setPsm(bindPsm)</code>
<code>connect(btAddress, psm, bindBtAddress, bindPsm, ...)</code>	remote address/PSM + <code>Local::setBtAddress(bindBtAddress)->setPsm(bindPsm)</code>

Again, the structure mirrors RFCOMM. The endpoint semantics differ.

12.5 Local and remote Bluetooth identity

Bluetooth endpoint identity may be less familiar than IP host-plus-port identity, so the local/remote distinction is worth making explicit.

Changing the endpoint family does not erase direction.

A server still has a local listening identity:

```
local Bluetooth address + channel
```

or:

```
local Bluetooth address + PSM
```

depending on the family.

A client still has a remote peer identity:

```
remote Bluetooth address + channel
```

or:

```
remote Bluetooth address + PSM
```

The client may also refine its local side:

```
local Bluetooth address
local channel
local PSM
```

depending on the overload and family. This continues Chapter 9's connection model. There is still a local side, a remote side, a bound identity, and a peer identity.

12.6 Pairing before SNode.C communication

Bluetooth endpoint identity is not the whole operational story.

Before two devices can communicate through SNode.C over RFCOMM or L2CAP, the devices must already be paired using the operating-system Bluetooth tools or user interface.

SNode.C uses the Bluetooth stack as a lower communication family. It does not replace Bluetooth discovery, pairing, trust management, or adapter setup.

That separation is important:

```
Bluetooth setup:
pairing, trust, adapter state, permissions

SNode.C communication:
configured endpoint identity, listen/connect registration, connection handling, context behavior
```

If pairing is missing, a SNode.C client may use the correct Bluetooth address and service selector and still fail at the platform Bluetooth layer.

This is not a different SNode.C architecture problem. It is an operational precondition of Bluetooth communication between devices.

12.7 What remains stable

Bluetooth changes endpoint semantics, but it does not require a different SNode.C application architecture.

Server/client/connection/context model

The stable core roles remain:

Role	Meaning with Bluetooth stream families
Application-side handle	the visible <code>SocketServer</code> or <code>SocketClient</code> object used to configure and register the role
Registered instance	the runtime-visible server-side or client-side communication role
<code>SocketConnection</code>	one concrete peer relationship
<code>SocketContextFactory</code>	creates a context for a connection
<code>SocketContext</code>	implements application protocol behavior

The server is still the listening role. The client is still the connecting role. The connection is still the concrete peer relationship.

The context is still the protocol endpoint attached to that connection.

Context and protocol logic

A `SocketContext` implementing a stream protocol does not need to become fundamentally RFCOMM-specific or L2CAP-specific just because it is carried over:

```
net::rc::stream::legacy
```

or:

```
net::l2::stream::legacy
```

What changes is the lower communication family.

What often remains stable is:

- how the protocol reacts to connection establishment,
- how it reads data,
- how it sends data,
- how it handles disconnection,
- how it thinks in terms of one connection and one context.

The context may inspect Bluetooth addresses for logging or diagnostics, but the application protocol logic can remain separated from the lower-family details.

Legacy and TLS

Chapter 7 introduced `legacy` and `tls` as connection-layer variants.

`legacy` is the non-TLS stream connection variant.

`tls` adds TLS connection handling.

RFCOMM and L2CAP can participate in that stream connection-layer pattern when the corresponding Bluetooth and TLS components are available.

The point here is architectural, not cryptographic.

Bluetooth support does not sit outside the connection-layer model. The same separation still applies:

```
network family
-> transport form
    -> connection handling
        -> application context
```

12.8 What changes operationally

Bluetooth is device-near, radio-based communication. That changes the operational setting.

Platform and build reality

Bluetooth support depends on platform Bluetooth support.

On Linux, that usually means BlueZ and the corresponding development files must be available when building the Bluetooth-enabled components.

The framework can model RFCOMM and L2CAP as lower communication families, but the build can only provide those components where the platform Bluetooth stack and development files are available.

That is normal for system-level Bluetooth support. `SNode.C` exposes RFCOMM and L2CAP as lower communication families, while still respecting the platform dependency.

Pairing, permissions, and adapter state

Bluetooth also has operational state outside the application process.

Before SNode.C can establish a Bluetooth connection between two devices, the devices must be paired by hand or by the surrounding system administration process. Depending on the platform setup, they may also need to be trusted, visible to the relevant adapter, and permitted by the local Bluetooth service policy.

That means Bluetooth failures should not be diagnosed only as SNode.C configuration failures.

The application may have selected the right family, address, and service selector while the platform still refuses the connection because pairing, adapter state, permissions, or service availability are not in the expected state.

For SNode.C code, the practical rule is:

First make Bluetooth communication possible at the operating-system level.
Then let SNode.C use that endpoint identity through its normal listen/connect model.

This keeps framework concerns and Bluetooth administration concerns separate.

Device-near and IoT systems

Bluetooth matters especially in device-near systems.

A system may combine several communication worlds:

- Bluetooth for local radio-based device communication,
- Unix domain sockets for local process-to-process communication,
- IPv4 or IPv6 for network communication,
- TLS for connection-layer security where appropriate,
- HTTP, WebSocket, MQTT, or other protocols above the lower layers.

SNode.C's value is that these worlds can be understood through one layered model instead of as unrelated programming domains.

This is especially useful in IoT and embedded systems, where local device communication and network communication often coexist.

What to remember

- Bluetooth support in this part is represented by two lower communication families: RFCOMM in `net::rc` and L2CAP in `net::l2`.
- RFCOMM endpoint identity is Bluetooth address plus channel; L2CAP endpoint identity is Bluetooth address plus PSM.
- Channel and PSM are not interchangeable service selectors.
- The application-side handle, registered instance, connection, factory, context, callbacks, and runtime model remain structurally familiar.
- Bluetooth convenience calls configure the application-side handle and then enter the usual registration path.
- The Bluetooth wildcard or deferred endpoint shape is the wildcard Bluetooth address plus service selector 0.

12.9 Public surface of Bluetooth stream roles

Bluetooth stream roles follow the same source/build rule, but only when the required Bluetooth development support is present. RFCOMM uses the `rc` family fragment:

```
#include <net/rc/stream/legacy/SocketServer.h>
#include <net/rc/stream/legacy/SocketClient.h>
```

with the corresponding component:

```
net-rc-stream-legacy
```

L2CAP uses the 12 fragment:

```
#include <net/l2/stream/legacy/SocketServer.h>
#include <net/l2/stream/legacy/SocketClient.h>
```

with the corresponding component:

```
net-l2-stream-legacy
```

The local lesson is the conditional family surface. Chapter 32 gives the broader include/component matrix.

12.10 Closing perspective

With host-plus-port, path-based, and Bluetooth endpoint identities in view, the lower-family tour has done its job. The next part moves upward to protocol behavior inside `SocketContext` objects and to the factories that create them.

Part IV

From Raw Connections to Application Protocols

This part moves from configured lower-family connections to protocol behavior. It focuses on writing `SocketContext` and `SocketContextFactory` classes well and then shows how the same protocol idea can remain recognizable over different lower layers.

The central lesson is that protocol meaning should live at the layer that can honestly own it.

Chapter 13

Writing SocketContext Classes Well

13.1 From lower-family choice to protocol behavior

Chapters 10, 11, and 12 completed the lower-family tour.

The book has now shown that SNode.C can express several different endpoint identities:

- host plus port,
- Unix-domain path,
- Bluetooth address plus RFCOMM channel,
- Bluetooth address plus L2CAP PSM.

Those endpoint identities differ strongly. They belong to different lower communication families, have different address classes, and lead to different operational questions. Yet the application-side shape remained recognizable across all of them.

The reason is that application protocol behavior does not live in the address class, the transport family, or the connection wrapper. It lives in the per-connection context.

For application code, the most important object in this part of the design is therefore:

SocketContext

Chapters 5 and 9 established the handle, instance, connection, factory, and context vocabulary. This chapter now uses that vocabulary from the context side: once a concrete connection exists, the context is where one endpoint's protocol behavior is written.

That is the starting point for this chapter.

13.2 What a SocketContext is

A compact definition is:

Context responsibility rule

A **SocketContext** is the per-connection application protocol endpoint attached to a **SocketConnection**.

Every part of that sentence matters.

Phrase	Meaning
per-connection	one context belongs to one concrete peer relationship
application protocol	this is where protocol behavior lives

Phrase	Meaning
endpoint	the context represents one side of a protocol conversation
attached to a <code>SocketConnection</code>	the context acts through a managed communication relationship

The context is therefore neither the whole application nor the transport object itself. It is the protocol endpoint for one connection.

This distinction is easy to underestimate. A context is close enough to the connection that it can send, read, close, set timeouts, and observe metrics. At the same time, it should not become the connection. It should express what the protocol does over that connection.

That makes the context the place where the book moves upward again. The previous part asked which lower family carries the connection. This part asks what the application protocol does once a connection exists.

Context, connection, and factory

The three objects should be kept separate:

Object	Main responsibility
<code>SocketConnection</code>	concrete peer relationship, data path, metrics, lifetime surface
<code>SocketContext</code>	per-connection protocol behavior
<code>SocketContextFactory</code>	creates contexts for connections

This distinction continues Chapter 9.

The connection represents communication with a peer. It owns the connection-facing runtime surface: data movement, shutdown, timeout, naming, counters, and endpoint views. The context implements what the application protocol does over that communication relationship. The factory creates the context when a new connection needs one.

If these responsibilities remain separate, SNode.C applications stay easier to read as they grow. The reader can ask three different questions and expect three different answers:

Which peer relationship exists?

-> connection

What protocol behavior runs over that relationship?

-> context

How is the right protocol endpoint created for a connection?

-> factory

A good SNode.C application keeps those questions visible.

Acting through the connection

A `SocketContext` acts through a `SocketConnection`.

That means it may send, stream, read, close, inspect metrics, set timeouts, or request shutdown through the surface provided to it. These are real operations, not descriptive helper functions.

But the context should not become the connection.

A good context uses connection-facing operations to express protocol behavior. It does not reinvent send queues, transport buffers, descriptor ownership, or connection lifecycle machinery.

The boundary is:

```

SocketConnection
    -> concrete communication relationship

SocketContext
    -> protocol behavior over that relationship

```

This boundary is one of the most important design habits in SNode.C application code.

13.3 The context surface seen by protocol code

The stream `SocketContext` surface is focused. It gives derived protocol classes enough operations to implement real stream protocols while still keeping transport machinery outside the protocol class.

There are two layers worth keeping apart.

The generic `core::socket::SocketContext` defines the protocol-context contract shared at the socket-context level: timeout, sending, reading, closing, metrics, received-data processing, signal handling, and read/write error handling.

The stream-specific `core::socket::stream::SocketContext` adds the stream-oriented surface: stream-to-peer support, stream EOF, read and write shutdown, access to the stream `SocketConnection`, and the connection lifecycle hooks used by stream contexts.

The chapter does not need to turn this into an inheritance lesson. The useful point is simpler: the context API is deliberately shaped around protocol behavior, while the concrete stream specialization provides the operations needed by stream protocols.

A useful overview is:

Interface area	Methods / hooks
sending	<code>sendToPeer(...)</code> , <code>streamToPeer(...)</code> , <code>streamEof()</code>
reading	<code>readFromPeer(...)</code> , <code>onReceivedFromPeer()</code>
lifecycle	<code>onConnected()</code> , <code>onDisconnected()</code>
shutdown	<code>shutdownRead()</code> , <code>shutdownWrite()</code> , <code>close()</code>
timeout	<code>setTimeout(...)</code>
metrics	<code>getTotalSent()</code> , <code>getTotalQueued()</code> , <code>getTotalRead()</code> , <code>getTotalProcessed()</code> , <code>getOnlineSince()</code> , <code>getOnlineDuration()</code>
connection access	<code>getSocketConnection()</code>
signals and errors	<code>onSignal(...)</code> , <code>onReadError(...)</code> , <code>onWriteError(...)</code>

This table is not meant to replace API documentation.

Its purpose is to show the design shape. The context sees enough of the connection to implement protocol behavior, but not so much that it becomes a second transport implementation.

What the surface tells us

The interface tells us several things about the intended role of a context. First, the context is allowed to send and read data. Second, it reacts to lifecycle events.

Third, it can close or shut down the connection when protocol semantics require it.

Fourth, it can observe useful connection-derived metrics. Some of those observations are measured relative to the moment where the context is attached to the connection, which makes them meaningful for the context's own protocol lifetime.

Fifth, it must handle signals and read/write errors explicitly enough that exceptional runtime situations are not invisible. That is a strong but focused contract.

13.4 Lifecycle, input, signals, and errors

A good way to understand a context is to group its hooks by responsibility.

Hook group	Examples	Meaning
lifecycle	<code>onConnected()</code> , <code>onDisconnected()</code>	connection lifecycle becomes visible to protocol code
input	<code>onReceivedFromPeer()</code>	available input is interpreted as protocol data
signals and errors	<code>onSignal(...)</code> , <code>onReadError(...)</code> , <code>onWriteError(...)</code>	exceptional or runtime-side events are handled explicitly

This grouping is more useful than memorizing method names in isolation.

It shows what kind of thinking a context needs. A context is not one sequential script. It is a set of reactions to events that occur during a connection's lifetime.

Lifecycle hooks

`onConnected()` should answer one clear question:

What protocol-relevant action, if any, should happen when this endpoint becomes ready?

Sometimes the correct answer is nothing. Sometimes the correct answer is to start the conversation.

For example, a client-side echo context may send the first message from `onConnected()` because that is the protocol-relevant beginning of the exchange.

`onDisconnected()` should answer a different question:

What protocol-local cleanup or final observation belongs to this endpoint when the connection ends?

It is often a good place for lightweight bookkeeping, understandable logging, or releasing protocol-specific transient state. It should not become a large recovery dump site for responsibilities that were unclear earlier.

For application authors, the important rule is not to manage the context as if it owned the connection. The framework attaches the context to the connection, calls the lifecycle hooks at the appropriate points, and detaches the context when the connection ends. The context should use those hooks to express protocol-relevant behavior, not to take over framework lifetime management.

Input processing hook

For many stream protocols, `onReceivedFromPeer()` is the central protocol method. This is where incoming bytes become protocol meaning.

A simple echo protocol may only read available bytes and send them back. A more complex protocol may need:

- framing,
- message accumulation,
- partial-message handling,
- command dispatch,
- validation,
- state transitions.

The important point is the same in both cases:

`onReceivedFromPeer()` should read and process data intentionally.

It should not blindly drain input and postpone all meaning indefinitely.

The return value matters. `onReceivedFromPeer()` returns the amount of data actually processed. That return value connects protocol processing to the framework's accounting of processed input. If a context reads more than it can understand, or reports progress that did not really happen, diagnostics become less trustworthy.

A good context keeps the relationship honest:

```
read data
  -> process data
    -> report what was processed
```

Signal and error hooks

The base context interface also requires explicit handling for signals and read/write errors.

This does not mean every example must build an elaborate signal or error policy. It does mean the context author should not pretend those events do not exist.

A small teaching example may keep these hooks minimal. A real protocol may use them to close a connection, record diagnostics, reject invalid state, or translate lower-level problems into protocol-level decisions.

The important habit is explicitness.

13.5 A compact worked context

The echo context in Chapter 3 showed the smallest useful pattern. A slightly more realistic teaching context can still remain compact: a line-command endpoint that remembers only a receive buffer, reacts to complete lines, writes protocol responses, and closes only for clear protocol reasons.

The protocol is deliberately small:

```
PING
  -> PONG

STATUS
  -> OK

QUIT
  -> close the connection

anything else
  -> ERR unknown command
```

The point is not the protocol. The point is where the responsibilities land. Input accumulation, command interpretation, response writing, and protocol-driven closure all stay inside the context. Listening, connecting, retrying, reconnecting, and choosing the lower communication family stay outside it.

An abridged context can look like this:

```
#include <core/socket/stream/SocketContext.h>

#include <cstddef>
#include <string>

namespace core::socket::stream {
    class SocketConnection;
}

class LineCommandContext : public core::socket::stream::SocketContext {
public:
```

```

explicit LineCommandContext(
    core::socket::stream::SocketConnection* socketConnection)
    : core::socket::stream::SocketContext(socketConnection) {
}

private:
    static constexpr std::size_t maxLineLength = 4096;

    void onConnected() override {
        sendToPeer("READY\n");
    }

    void onDisconnected() override {
        receiveBuffer.clear();
    }

    bool onSignal([[maybe_unused]] int signum) override {
        close();
        return true;
    }

    std::size_t onReceivedFromPeer() override {
        char chunk[1024];
        const std::size_t chunkLen = readFromPeer(chunk, sizeof(chunk));

        if (chunkLen > 0) {
            receiveBuffer.append(chunk, chunkLen);

            std::size_t lineEnd = receiveBuffer.find('\n');
            while (lineEnd != std::string::npos) {
                std::string line = receiveBuffer.substr(0, lineEnd);
                if (!line.empty() && line.back() == '\r') {
                    line.pop_back();
                }

                processLine(line);
                receiveBuffer.erase(0, lineEnd + 1);
                lineEnd = receiveBuffer.find('\n');
            }

            if (receiveBuffer.length() > maxLineLength) {
                close();
            }
        }

        return chunkLen;
    }

    void processLine(const std::string& line) {
        if (line == "PING") {
            sendToPeer("PONG\n");
        } else if (line == "STATUS") {
            sendToPeer("OK\n");
        } else if (line == "QUIT") {
            close();
        } else if (!line.empty()) {
            sendToPeer("ERR unknown command\n");
        }
    }

    std::string receiveBuffer;
};

```

The example has one piece of connection-local state: `receiveBuffer`. It exists because stream input is byte-oriented, while the protocol is line-oriented. The context accumulates bytes only until it can process a complete line.

The input path stays honest. `onReceivedFromPeer()` reads a chunk, appends it to the connection-local buffer, processes each complete line, and returns the number of bytes read from the peer. It does not claim to have interpreted bytes that were never read, and it does not hide incomplete input in unrelated global state.

The output path is equally narrow. The context sends protocol responses through `sendToPeer(...)`. It decides that `PING` means `PONG`, that `STATUS` means `OK`, and that an unknown command produces an error line. It does not build a second output queue or bypass the connection surface.

Closure also has protocol meaning. `QUIT` closes because the peer requested the end of the conversation. A signal closes because the runtime environment asks the endpoint to stop. An overlong pending line closes because the input no longer fits the protocol's safety rule. Those are different reasons, and good context code makes such reasons visible.

This is still not a full application. It is a compact worked context whose only job is to show how lifecycle handling, input handling, output behavior, state, and closure discipline fit inside one per-connection endpoint.

The complete runnable server and client version of this line protocol is included in `companion/examples/LineProtocol-Server` and `companion/examples/LineProtocol-Client`. Those source trees use the same public IPv4 legacy stream front-door roles as the Chapter 3 echo pair, but they replace byte reflection with command parsing, response writing, and protocol-driven closure.

13.6 Design habits for good context code

A good `SocketContext` is usually small, connection-local, and protocol-specific.

It answers the question:

What should this endpoint do on this connection?

That question is narrow. A context becomes harder to understand when it tries to become the whole application.

Keep responsibility connection-local

A context should represent one endpoint of one protocol conversation on one connection.

It should not become:

- a global session manager,
- a configuration database,
- a logger singleton,
- a factory for unrelated objects,
- a multiprotocol grab bag,
- or the outer server/client role.

This does not mean a context can never refer to application services. A protocol endpoint may need authentication state, a message registry, a database facade, a dispatcher, or a shared application service.

The design question is how that dependency becomes visible.

A context may receive references, pointers, or shared services through factory construction. That keeps the dependency visible at the creation boundary instead of hiding it behind unrelated global state. Chapter 14 will return to this point, because the factory is the natural bridge between application-level construction and per-connection protocol behavior.

Think in events, not in one linear script

`SNode.C` is event-driven.

Chapter 6 explained that the runtime advances registered work through events. A context is the protocol-level expression of that model. It is not a blocking script. It is a collection of event reactions attached to one connection.

A context should therefore not be designed as if the whole protocol were one blocking sequence such as:

```
connect
send
receive
process
send again
disconnect
```

A better model is:

```
onConnected()
    -> connection became usable

onReceivedFromPeer()
    -> input is available

onDisconnected()
    -> connection ended

onSignal(...) / onReadError(...) / onWriteError(...)
    -> exceptional event occurred
```

This event-oriented structure is what makes context code readable later.

Each hook should have a clear reason to exist. The reader should be able to see which part of the protocol is represented by each reaction.

Keep protocol state explicit

Many useful protocols need state. That is normal. The important question is whether the state is held well.

Good protocol state is:

- explicit,
- local to the connection when possible,
- named in protocol terms,
- updated at understandable points,
- and small enough that transitions remain readable.

Because a `SocketContext` is per-connection, it is often the right scope for connection-local protocol state. A frame accumulator, a parser state, a handshake state, a pending request, or a session-local flag often belongs here.

Hidden global state should be avoided unless the protocol genuinely needs shared application state and the dependency is made explicit.

Read only what can be processed

A context should read data with a clear idea of what it is prepared to process. For a simple echo protocol, a fixed buffer and immediate reflection may be enough.

For a framed or stateful protocol, the context should connect three questions:

- What data is available?
- What does the protocol currently expect?
- What can this invocation actually process?

This discipline prevents the receive path from becoming an unbounded bucket of bytes whose meaning is deferred indefinitely.

Send responses through the connection surface

A context sends through `sendToPeer(...)`, `streamToPeer(...)`, or related connection-facing operations. That keeps protocol behavior separate from transport machinery. The context decides *what* protocol response should be sent. The connection machinery handles the managed communication path.

This avoids the common mistake of reimplementing output buffering or transport state inside the protocol class.

Use timeouts with protocol intent

The context can set a timeout through `setTimeout(...)`. That does not mean every context should immediately set a custom timeout.

Timeouts should express protocol intent, such as:

- idle peer detection,
- protocol deadlines,
- liveness expectations,
- timeout-driven closure,
- or protection against stalled conversations.

Timeouts should not compensate for unclear state handling. A good context uses them because the protocol needs them.

Close or shut down with protocol intent

A context may call:

```
shutdownRead();
shutdownWrite();
close();
```

These operations should also express protocol meaning.

Examples:

- close when the protocol conversation is finished,
- close on invalid or unsafe peer behavior,
- half-shutdown when the protocol semantics justify it,
- avoid abrupt closure when the protocol has a graceful ending.

The context is allowed to close the connection. It should still know why it is doing so.

Use metrics as protocol information

Connection-derived metrics can be useful inside or around a context.

Examples include:

- total queued bytes,
- total sent bytes,
- total read bytes,
- total processed bytes,
- online duration.

A good context does not treat these only as raw counters.

It can interpret them in protocol terms:

- Has this peer been connected unusually long?
- Is the endpoint producing more than it can send?
- Is data being read without meaningful protocol progress?
- Did the session end after a complete or incomplete exchange?

The metrics are most useful when they help explain protocol behavior.

13.7 What not to put into a context

The context is an important object, but it should not absorb every responsibility nearby.

A compact contrast helps:

Good context	Problematic context
one protocol endpoint	whole application
explicit protocol state	hidden global state
reads what it can process	reads blindly
acts through the connection	reimplements connection machinery
closes with protocol intent	closes as a reflex
logs protocol-relevant events	logs repetitive noise
depends on services explicitly	reaches into unrelated global systems

This table folds several common mistakes into one design rule:

Context-scope warning

Keep the context focused on protocol behavior for one connection.

Do not move server/client role logic into the context

Listening, connecting, accepting peers, retrying, and reconnecting belong to the registered instance and its runtime or flow-controller machinery. They do not belong to the per-connection context.

The context begins where a concrete connection has protocol behavior to perform.

This is a useful boundary when reading code. If a context starts deciding when the server should listen again, when the client should reconnect, or how instances should be registered, it is probably reaching outside its role.

Do not turn the context into a second connection

The context acts through the connection. It should not become a competing connection implementation.

Using `sendToPeer(...)`, `readFromPeer(...)`, `close()`, shutdown operations, and metrics is appropriate. Recreating transport buffers, descriptor ownership, or connection lifecycle state inside the context is not.

Do not hide unrelated global orchestration inside the context

A context may need access to application services.

But if it starts managing all sessions, all endpoints, all routing decisions, and all global state, it has probably grown beyond its intended role.

A good context can be understood locally. It should be possible to read the class and understand what one protocol endpoint does on one connection.

13.8 Logging in a context

Logging in context code should illuminate protocol behavior.

Useful logging often includes:

- meaningful lifecycle transitions,
- important protocol state changes,
- invalid or unexpected input,
- concise summaries of received or sent messages,
- diagnostics around closure or timeout decisions.

Less useful logging repeats every low-level detail until the protocol shape disappears. The goal is not maximum output. The goal is useful visibility.

A good test is whether the log line helps answer a protocol question:

```
Why did this endpoint send that response?
Why did this context close the connection?
Why did this peer stop making progress?
What state was reached before disconnect?
```

Per-byte noise may be useful during a narrow diagnostic session. It should not be the default shape of a readable protocol implementation.

13.9 The echo context as a minimal pattern

The echo context is valuable because it is small.

The point of echo is not that echo is interesting. The point is that echo makes the boundary visible: input handling belongs in `onReceivedFromPeer()`, connection-ready behavior belongs in `onConnected()` when the protocol needs it, and the application-side server/client handle remains free of protocol details.

The minimal pattern is:

Method / area	Role in the example
<code>onConnected()</code>	starts or observes the protocol exchange
<code>onDisconnected()</code>	handles the end of the connection lightly
<code>onSignal(...)</code>	makes signal handling explicit
<code>onReceivedFromPeer()</code>	performs the actual protocol action

The example should not be treated as a full protocol architecture. It should be treated as the smallest complete pattern that shows where behavior belongs.

The useful habits are:

- lifecycle handling is explicit,
- receive handling is where protocol action occurs,
- responses are sent through the connection-facing surface,
- behavior remains local to the context,
- the application-side server/client handle stays clean.

A more complex protocol should grow from these habits, not abandon them.

13.10 Keeping context code mentally testable

A strong standard for a `SocketContext` is mental testability.

Another developer should be able to read the class and answer:

- What happens when the connection becomes ready?
- What input does the protocol expect?
- What state is remembered?
- What causes a response?
- What causes closure?
- What happens on invalid input?
- What happens on timeout?
- What happens on disconnect?

If those questions are hard to answer, the context may be too implicit, too stateful in hidden ways, or too overloaded.

A good context makes the protocol conversation visible.

That is a stronger standard than merely compiling. A context can compile and still be hard to reason about. A well-written context lets the reader reconstruct the conversation from the hooks, state variables, and connection-facing operations.

13.11 The factory as the next bridge

The factory should be mentioned here only as a bridge.

Its role is simple:

```
SocketContextFactory
-> creates SocketContext objects for SocketConnection objects
```

The factory is not where protocol behavior belongs. The protocol behavior belongs in the context.

The factory is important because it creates the right context for a connection. It is the natural place where construction-time dependencies become visible. If a context needs access to a shared service, configuration object, registry, or application state, the factory boundary is where that relationship can be made explicit rather than hidden.

If the context is the protocol endpoint, the factory is the construction boundary. Chapter 14 will look at that boundary more closely.

What to remember

- A `SocketContext` is the per-connection application protocol endpoint attached to a `SocketConnection`.
- The context implements protocol behavior; it does not own the server/client role or reimplement the connection.
- Context code should be event-oriented: lifecycle, input, signals, read errors, and write errors are separate responsibilities.
- `onReceivedFromPeer()` should process input intentionally and return the amount of data actually processed.
- Protocol state should be explicit, connection-local when possible, and named in protocol terms.
- Sending, streaming, timeout, shutdown, close, and metrics operations act through the connection-facing surface.

13.12 Closing perspective

This chapter moves the book upward again.

Chapter 14 turns to that bridge.

Chapter 14

Writing SocketContextFactory Classes Well

14.1 From protocol endpoint to construction boundary

Chapter 13 treated the `SocketContext` as the per-connection protocol endpoint. This chapter looks at the construction boundary that creates such endpoints: the `SocketContextFactory`.

The transition is small in API surface but important in design:

```
SocketConnection
-> SocketContextFactory::create(connection)
    -> SocketContext
        -> protocol behavior
```

If the context is the protocol endpoint, the factory is the construction boundary. It receives a concrete connection and creates the context that will contain the protocol behavior.

That distinction keeps the application architecture readable. The server or client role does not implement the protocol conversation. The connection does not decide which application endpoint class should exist. The context implements the behavior. The factory makes the construction decision.

14.2 What a SocketContextFactory is

A compact definition is:

A `SocketContextFactory` creates the right per-connection protocol endpoint for a concrete `SocketConnection`.

This definition is narrow. The factory does not implement the protocol conversation. It does not replace the registered server-side or client-side instance.

It does not own the runtime.

It answers one construction question:

Given this connection, which context object should be attached to it?

That question is small, but it protects an important architectural boundary.

The factory is where construction-time decisions become explicit. The context type can be chosen, stable dependencies can be passed, and a role value can be made visible. Once the context exists, protocol behavior belongs inside the context, not inside the factory that created it.

Factory, connection, and context

The three roles should remain distinct:

Object	Responsibility
<code>SocketConnection</code>	concrete peer relationship
<code>SocketContext</code>	protocol behavior for one connection
<code>SocketContextFactory</code>	context creation for a concrete connection

This table continues the distinction from Chapter 13. The connection is the managed peer relationship. The context is the protocol endpoint attached to that relationship.

The factory creates that endpoint.

When these responsibilities stay separate, application code remains easier to reason about. A reader can ask one question at a time: Which connection exists? Which context was created for it? What protocol behavior does that context implement?

The factory interface

The stream factory interface is narrow.

Its essential method is:

```
virtual core::socket::stream::SocketContext*
create(core::socket::stream::SocketConnection* socketConnection) = 0;
```

The interface receives a `SocketConnection*` and returns a `SocketContext*`.

That shape says exactly what the framework needs:

```
A concrete connection exists.
Create the protocol endpoint that belongs to it.
```

The factory is therefore not a general-purpose object creation service. It is the construction bridge between a connection and the protocol context that will run on that connection.

This narrowness is useful. It prevents the factory interface from becoming a second application framework. A concrete factory may still hold stable construction data, but the moment represented by `create(connection)` remains precise: a connection has appeared, and the correct context object must be created for that connection.

Source anchor: attaching the created context

The pinned stream-connection source in `src/core/socket/stream/SocketConnection.cpp` shows the same boundary in compact form. The connection asks the factory to create a context for `this` connection, attaches the result when creation succeeds, and closes the connection if no context can be created:

```
void SocketConnection::setSocketContext(
    const std::shared_ptr<SocketContextFactory>& socketContextFactory) {
    SocketContext* socketContext = socketContextFactory->create(this);

    if (socketContext != nullptr) {
        setSocketContext(socketContext);
    } else {
        close();
    }
}
```

The next step is equally important: the newly created context is attached to the connection. If a context already exists, the new one is staged as a replacement rather than treated as a shared global endpoint.

```

void SocketConnection::setSocketContext(SocketContext* socketContext) {
    if (this->socketContext == nullptr) {
        this->socketContext = socketContext;
        socketContext->attach();
    } else {
        newSocketContext = socketContext;
    }
}

```

The excerpt does not add a new responsibility to the factory. It confirms the narrow construction boundary: connection exists, factory creates the endpoint, connection attaches it, and protocol behavior then belongs to the context.

14.3 The construction path from connection to context

The factory sits in the connection creation path.

A compact view is:

```

application-side SocketServer / SocketClient handle
  -> registered server/client instance
    -> concrete SocketConnection
      -> SocketContextFactory::create(connection)
        -> new SocketContext(connection, dependencies...)
          -> protocol behavior

```

This diagram connects Chapters 9, 13, and 14. Chapter 9 explained the server/client/connection relationship. Chapter 13 explained the context as the per-connection protocol endpoint. This chapter explains the construction step between connection and context.

The accept or connect path makes the connection real. The factory creates the context that will handle protocol behavior once attached to the connection. The context then processes protocol messages.

That is why the factory is a boundary object. It is close to the connection, because it receives the connection. It is close to the protocol, because it creates the context. But it is not itself the connection and not itself the protocol endpoint.

14.4 One fresh context per creation

The most important rule is simple:

Factory responsibility rule

A factory should create a fresh context for the concrete connection it receives.

A `SocketContext` is per-connection.

Therefore, a factory should not reuse one context object across several connections. It should not treat the context as a singleton. It should not use one shared protocol shell to represent all peers.

The purpose of the factory is to create a protocol endpoint for one concrete peer relationship.

That means each ordinary call to `create(...)` should be understood as:

```

this connection
  -> this new protocol endpoint

```

This rule follows directly from Chapter 13. If the context is one per-connection protocol endpoint, the factory must preserve that relationship.

The framework also supports context replacement as a more advanced connection operation. That does not weaken the construction rule. If a later context switch is used, the new context is still a fresh endpoint object handed to the framework. It is not a reused singleton shared across peers and it is not a hidden global protocol object.

For the ordinary creation path, the rule remains simple: create the context that belongs to the connection you received, then hand it back.

14.5 Construction responsibilities of a factory

A factory contains construction policy.

That means it may reasonably decide:

- which concrete context type should be created,
- which stable constructor arguments should be passed,
- which role-specific value the context should receive,
- which explicit application services the context needs,
- and how to keep construction readable.

It should not perform the protocol itself.

A useful contrast is:

Belongs in the factory	Does not belong in the factory
choosing the context type	protocol message handling
passing stable dependencies	read/write behavior
setting role-specific constructor arguments	retry or reconnect policy
creating one fresh context	global orchestration
keeping construction readable	service-location dumping ground
making construction-time choices	mid-protocol state transitions

The boundary is:

```
Factory
-> context creation

Context
-> protocol behavior
```

The table is not meant to make factories trivial. Some factories need more than one constructor argument. Some factories choose between several context types. Some factories carry role information. The rule is not that every factory must be tiny. The rule is that factory complexity should remain construction complexity.

Choosing the context type

A factory can legitimately choose which concrete context type to create. For many simple applications, there is exactly one concrete context type.

In that case, the factory is almost mechanical. In schematic form:

```
SocketContext* create(SocketConnection* connection) override {
    return new MySocketContext(connection);
}
```

In other applications, the factory may choose among a small set of endpoint types. That can be reasonable if the choice is a construction-time decision.

It becomes problematic only when the factory starts performing protocol behavior that belongs inside the context. Choosing a context type is construction. Reading protocol frames, interpreting commands, and advancing protocol state are behavior.

Passing stable dependencies

Chapter 13 said that a context may depend on application services, but those dependencies should be visible. Chapter 14 shows the construction boundary where such dependencies can be passed deliberately.

A context may need stable dependencies at construction time. Examples include:

- a role indicator,
- immutable protocol configuration,
- a shared service interface,
- a parser helper,
- a statistics sink,
- a small dispatcher object.

This is a legitimate factory responsibility. The factory can receive, hold, or capture stable dependencies and pass them into each newly created context. The discipline is to keep those dependencies meaningful.

A factory should ask:

What does this context genuinely need in order to behave as the protocol endpoint for this connection?

That question prevents the factory from becoming a dumping ground for unrelated application objects.

Good dependency passing makes the construction path explicit. It says: this context needs this service to implement this protocol. That is very different from hiding a route to the whole application behind the factory.

Making role decisions explicit

A factory is often a good place to make role-specific construction decisions.

For example, a protocol may use the same context class on both sides but pass a role value:

```
enum class Role {
    Server,
    Client
};
```

Then the factory can make the role explicit. In schematic form:

```
SocketContext* create(SocketConnection* connection) override {
    return new EchoSocketContext(connection, Role::Server);
}
```

A separate client-side factory might use:

```
SocketContext* create(SocketConnection* connection) override {
    return new EchoSocketContext(connection, Role::Client);
}
```

This kind of factory is small, but useful. The role decision is visible. The context receives the information it needs.

The factory still remains construction-oriented.

A server-side factory can create a context configured for server-side protocol behavior. A client-side factory can create the corresponding client-side context. Both factories remain construction code. Neither one becomes the protocol conversation.

Keeping construction readable and unsurprising

A good factory is often simple. That is a strength.

A developer should be able to open the factory and quickly answer:

- Which context class is created?
- Which constructor arguments are passed?
- Which stable dependencies are encoded?
- Which role decision is made?

If answering those questions requires reading a large amount of unrelated logic, the factory has probably grown too far.

Readable construction is part of the design.

It is acceptable for a factory to express a real construction decision. It is not acceptable for construction code to hide a protocol conversation, a retry policy, a session manager, or a global service locator behind a method named `create(...)`.

14.6 Responsibilities that should stay out of the factory

Because the factory sits at an important boundary, it is tempting to put too much into it. That should be avoided.

Protocol behavior does not belong in the factory

The factory should not parse protocol messages. It should not decide mid-protocol transitions. It should not control conversation flow.

It should not perform authentication steps that belong to the protocol endpoint. Those are context responsibilities.

If protocol behavior starts moving into the factory, the application becomes harder to understand because the protocol is split across construction code and endpoint code.

The factory should create the endpoint. The context should behave as the endpoint.

Runtime flow control does not belong in the factory

The factory should not manage:

- retry logic,
- reconnect policy,
- event-loop sequencing,
- accept-loop tuning,
- or multiplexer details.

Those concerns belong to the registered instance, its configuration, the flow controller, the runtime, and lower-level framework machinery.

The factory should stay close to one moment:

```
A connection exists.  
Create the context for it.
```

This is the same separation Chapter 13 used for contexts. A context should not become the server/client role; a factory should not become the server/client role either. The registered instance remains the long-lived runtime-visible communication role. The factory remains the construction boundary for connection-local protocol endpoints.

Global service location does not belong in the factory

A factory may pass explicit stable services into a context. That is different from becoming a service locator.

A service-locator-style factory starts to behave as a hidden doorway to the entire application:

- global registries,
- database handles,
- unrelated service maps,
- orchestration callbacks,
- metrics systems,
- configuration trees,
- and session stores.

Sometimes a context genuinely needs access to one of these things. But the dependency should be explicit and justified. The factory should not quietly provide access to everything.

Explicit dependency passing keeps construction readable. Service location hides the dependency boundary.

That distinction matters because the factory is close to every new connection. If it becomes a hidden gateway to the whole application, every context becomes harder to understand.

Large mutable protocol state does not belong in the factory

Connection-local protocol state belongs in the context. Application-level shared state belongs in an application-level service or model.

The factory should not become the place where large mutable protocol state is accumulated simply because it is convenient.

If the factory holds too much state, it becomes harder to answer a basic question:

What does one context do on one connection?

That is a sign that responsibilities are drifting.

Factory state should usually be stable construction state, not evolving per-message protocol state. A role value, a configuration object, or a service reference can be construction state. A partially parsed message, peer conversation phase, or connection-local protocol buffer belongs in the context.

14.7 Ownership and lifecycle

The factory interface returns a raw `SocketContext*`.

This deserves a careful explanation. The raw pointer return should not be read as permission for arbitrary manual ownership throughout the application.

It should be read as part of this framework contract:

The factory creates a new endpoint object.
The object is handed into the framework-managed connection/context lifecycle.

The practical rule is:

Create the right concrete context object and hand it back to the framework.

The factory should not keep accidental ownership of the created context. It should not share the same context object across several connections. It should not create an object whose lifetime is unclear.

The raw pointer belongs to this construction boundary. It is not an invitation to unmanaged lifetime design elsewhere in the application.

The connection side of the framework decides what happens after creation. If the factory returns a context, the connection attaches it. If creation fails and returns no context, the connection cannot continue with protocol behavior and closes the relationship. The application author should therefore treat `create(...)` as a narrow handoff point: allocate the correct context, return it, and let the framework manage the connection/context lifecycle from there.

14.8 Passing shared application state

Real applications often need shared services. A small factory does not mean an isolated factory.

It may capture or store references to shared resources such as:

- immutable protocol configuration,
- a thread-safe statistics object,
- a reference-counted service interface,
- a dispatcher,
- a logger abstraction,
- or an application model needed by the protocol.

The important distinction is not whether a factory has state. The important distinction is what kind of state it has.

Stable construction state is appropriate. Hidden global orchestration is not.

Factory constructor arguments from server and client

The stream server and stream client templates make this dependency path explicit.

Both the `SocketServer` and the `SocketClient` template accept an argument pack:

```
typename... Args
```

Those arguments are forwarded to the concrete `SocketContextFactory` constructor when the application-side server or client handle is constructed.

In schematic form, the pattern is:

```
std::make_shared<SocketContextFactory>(
    std::forward<Args>(args)...
)
```

the argument pack allows the factory to be preconfigured at the point where the server or client handle is created. The resulting factory object is then part of the shared state used by the registered instance when concrete connections appear.

This means the server or client constructor can provide the stable information the factory needs. That information may then be stored in the factory and used later when `create(connection)` is called.

The flow is:

```
SocketServer / SocketClient constructor arguments
-> SocketContextFactory constructor
    -> stored factory state
        -> create(connection)
            -> SocketContext constructor arguments
```

This is an important distinction. The argument pack does not mean that arbitrary runtime behavior should be pushed into the factory.

It means that stable construction data can enter the factory at the point where the server or client handle is created.

Used well, this keeps dependency flow explicit:

```
handle construction
  -> factory construction
    -> context construction
```

The factory remains construction-oriented. It simply receives the data it needs to create the right context objects later. The important distinction is between explicit dependency passing and uncontrolled service location.

Good dependency passing looks like:

```
This context needs this service to implement this protocol.
```

Uncontrolled service location looks like:

```
The context can reach everything through the factory.
```

The first is a design choice. The second erodes the boundary.

14.9 Factory design shapes

There is more than one correct factory shape. The right design depends on what needs to vary at construction time. Different factory shapes are acceptable when they make construction-time variation explicit.

Pattern	Use when
separate server/client factories	the role distinction should be visible and simple
parameterized factory	the context type is the same and only stable constructor data differs
context-type-selecting factory	a small, explicit selection among endpoint types is needed
preconfigured role factory	higher-level endpoint roles should be fixed at handle construction

All four patterns can be valid.

The same rule applies to all of them:

Factory-scope warning

The selection should remain a construction-time decision, not protocol execution.

This section is therefore not a pattern catalogue. It is a set of examples showing how the construction-boundary idea can be expressed without moving protocol behavior into the factory.

Separate server/client factories

Separate server-side and client-side factories can be useful when the two roles should be visible. This is especially clear when both sides use the same context type but receive different role arguments. The benefit is readability.

The server construction path is explicit. The client construction path is explicit. No hidden role inference is required.

Small duplication is often acceptable when it makes the communication roles clearer.

Parameterized factories

A single reusable factory type may also be appropriate.

For example, the role or configuration object may be a constructor argument of the factory itself. In schematic form:

```
class EchoSocketContextFactory : public SocketContextFactory {
public:
    explicit EchoSocketContextFactory(Role role)
        : role(role) {

    }

    SocketContext* create(SocketConnection* connection) override {
        return new EchoSocketContext(connection, role);
    }

private:
    Role role;
};
```

This can be a clean design when the factory remains easy to read and the variation is stable. The factory is still only expressing a creation decision.

What preconfigured factories make possible

Preconfigured factories allow the same framework mechanism to create different communication roles without changing the surrounding server/client machinery.

An application-side server or client handle can pass stable role and dependency information into the factory constructor. The factory can then use that information whenever it creates a context for a new connection.

This can be used for simple role distinctions, such as server-side versus client-side contexts, but also for higher-level endpoint roles:

- model-side endpoints,
- view-side endpoints,
- controller-side endpoints,
- publisher endpoints,
- subscriber endpoints,
- command endpoints,
- event endpoints,
- gateway endpoints,
- adapter endpoints.

The important point in this chapter is still the mechanism, not the full pattern catalogue.

A factory does not implement MVC, publish/subscribe, or gateway behavior by itself. It creates the correctly preconfigured context objects that participate in such patterns.

Context-type-selecting factories

Some applications may need a factory that chooses among a small number of context classes. That can be acceptable when the selection is explicit and construction-time. For example, a configuration value might decide whether to create a diagnostic context or a normal protocol context.

But the selection should remain simple.

If the factory begins reading from the peer, interpreting protocol frames, or managing state transitions, it has crossed into protocol behavior.

That belongs in the context.

14.10 The strongest factory tests

A useful practical test is:

If the protocol behavior were removed from the context class, would the factory still appear to contain too much of the application?

If the answer is yes, the factory is probably overgrown. A good factory should feel incomplete without the context. That is exactly right.

Its job is to create the protocol endpoint, not to replace it.

Another useful test is:

Can a developer understand the factory in one glance?

A factory that is hard to skim is often mixing construction with other responsibilities.

One glance does not mean the factory must be trivial. It means the construction decision should be visible. A reader should be able to see which context is created, which stable dependencies are passed, and why that choice belongs at construction time.

14.11 The factory as the Chapter 15 bridge

Chapter 15 will show how the same protocol can be carried over different lower communication families.

The separation is:

```
SocketContext
    -> protocol behavior

SocketContextFactory
    -> context creation

SocketServer / SocketClient handle
    -> lower-family-specific communication role registration
```

Once protocol behavior is in the context and construction policy is in the factory, lower-family variation becomes easier to manage.

Different server/client types can choose different endpoint families while the factory/context pair preserves the application protocol boundary.

The factory supports that portability story because it keeps context creation separate from the lower-layer role. It does not perform the protocol itself.

What to remember

- A **SocketContextFactory** is the construction boundary between a concrete **SocketConnection** and the per-connection **SocketContext**.
- The factory creates context objects; the context implements protocol behavior.
- The ordinary creation path should produce a fresh context object for the connection it receives.
- The raw pointer returned by **create(...)** belongs to the framework construction contract, not to arbitrary manual lifetime management.
- Factory constructor arguments are a clean way to pass stable dependencies into future contexts.
- Construction-time selection is acceptable; mid-protocol behavior belongs in the context.

14.12 Closing perspective

Chapter 13 established the context as the place where protocol behavior lives. Chapter 14 established the factory as the construction boundary that creates those contexts for concrete connections.

Chapter 15 uses that separation to show how the same protocol can move across different lower layers without turning the application into a pile of special cases.

Chapter 15

Building the Same Protocol over Different Lower Layers

15.1 From context and factory separation to lower-family transfer

Chapter 13 placed application protocol behavior in the `SocketContext`. Chapter 14 placed context construction in the `SocketContextFactory`. This chapter uses both separations together.

The central question is:

What does the separation buy us when the lower communication family changes?

The short answer is that a protocol can often keep its core shape even when the carrier changes:

```
SocketContext
  -> protocol behavior

SocketContextFactory
  -> context creation, role preconfiguration, and stable dependency passing

application-side SocketServer / SocketClient handle
  -> selects the concrete lower-family server/client type

registered server/client instance
  -> long-lived runtime-visible communication role

SocketAddress / configuration
  -> lower-family-specific endpoint identity and deployment shape
```

This is where the design discipline from Chapters 13 and 14 becomes visible across lower families. The protocol does not become independent of all lower-layer reality.

Address identity, local/remote meaning, deployment, platform support, pairing, permissions, TLS, retry behavior, and operational configuration still matter. But when protocol behavior lives in a `SocketContext`, and when construction choices remain in a `SocketContextFactory`, much more can transfer than one might first expect.

The lower family matters. It does not always have to rewrite the protocol endpoint. That balance is the main idea of this chapter.

15.2 The lower-family transfer model

The transfer model keeps four questions apart:

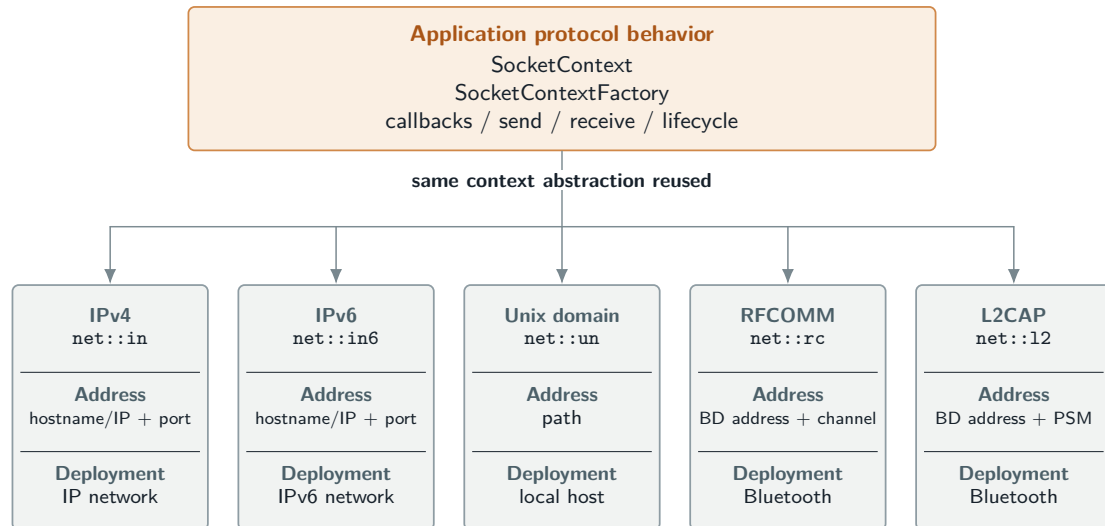


Figure 15.1: The lower-family transfer model. The protocol-side boundary remains stable while endpoint families change address types, concrete server/client types, configuration, and deployment assumptions.

Question	Best home
What does the protocol do on one connection?	SocketContext
Which context should be created for this connection?	SocketContextFactory
Which lower communication family is selected?	the application-side server/client handle type and its registration
Which endpoint identity and deployment values are used?	SocketAddress , configuration, and startup/deployment data

This separation is the reason the same protocol core can often move across lower families without rewriting the protocol itself.

Figure 15.1 shows the same idea as a transfer model: the protocol-side boundary remains stable, while the lower-family side changes endpoint identity, concrete server/client type, configuration, and deployment assumptions. It is a portability map for deciding what may stay stable and what must be reselected.

The visible server/client type changes because the application selects a different lower-family specialization. The registered instance then carries that configured lower-family role into the runtime. The concrete **SocketConnection** represents one peer relationship under that role. The factory creates the per-connection context for that connection. The context implements the protocol endpoint.

The naming reflects an architectural boundary.

It prevents one common design mistake: putting lower-family setup, protocol behavior, context construction, and deployment policy into the same class. **SNode.C** makes those boundaries visible. Application code becomes easier to move across families when it respects them.

15.3 What can remain stable

Not everything transfers. But several important parts often can.

Protocol shape

The protocol shape includes questions such as:

- Who sends first?
- What happens when data arrives?
- How are bytes interpreted?

- When is a response sent?
- What state is remembered per connection?
- What causes the connection to close?
- What happens when the peer disconnects?
- What errors or signals matter to the protocol?

These are context-level questions. They belong to the application protocol endpoint.

A well-written `SocketContext` can answer these questions without immediately depending on whether the connection came from IPv4, IPv6, Unix domain sockets, RFCOMM, or L2CAP.

That does not mean the context must be blind to its carrier.

It means the context should only depend on lower-family facts when those facts are part of the protocol's meaning.

Context behavior

The following parts often remain stable:

- lifecycle handling in `onConnected()` and `onDisconnected()`,
- input handling in `onReceivedFromPeer()`,
- per-connection protocol state,
- use of `sendToPeer(...)`,
- use of `readFromPeer(...)`,
- timeout and close decisions that are protocol-driven,
- metrics interpreted in protocol terms.

This is the main reason Chapter 13 emphasized context discipline.

A context that is truly a per-connection protocol endpoint is easier to carry to another lower family. A context that quietly performs address selection, deployment decisions, reconnect policy, or global orchestration is much harder to move.

The useful question is:

Protocol/carrier rule

Does this code describe the protocol conversation, or does it describe the carrier and deployment environment?

Protocol conversation belongs in the context. Carrier and deployment concerns usually belong outside it.

Factory construction policy

The factory may also transfer. If the same context type is suitable across several lower families, the factory can remain small and recognizable.

It may still create:

```
new EchoSocketContext(connection, Role::Server)
```

or:

```
new EchoSocketContext(connection, Role::Client)
```

The application-side server/client handle type changes. The endpoint identity changes. The deployment setting changes.

The context construction shape can still remain familiar.

This is why Chapters 13 and 14 came before this chapter. A well-factored context and a disciplined factory make lower-family transfer practical. The factory does not make the protocol portable by magic. It keeps the construction boundary clean enough that portability can be judged clearly.

15.4 What changes with the lower family

Lower-family transfer is not the same as pretending all families are identical. Several things usually change.

Server/client handle type and registered instance

The visible server/client handle type changes because the application chooses a different lower-family specialization.

Examples include:

```
net::in::stream::legacy::SocketServer<MyFactory>
net::in6::stream::legacy::SocketServer<MyFactory>
net::un::stream::legacy::SocketServer<MyFactory>
net::rc::stream::legacy::SocketServer<MyFactory>
net::l2::stream::legacy::SocketServer<MyFactory>
```

and the corresponding client types.

The role model remains recognizable:

```
application-side handle
-> registered instance
    -> connection
        -> factory
            -> context
```

But the selected namespace and lower-family component change.

That change is meaningful. `net::in`, `net::in6`, `net::un`, `net::rc`, and `net::l2` are not decorative names. They select different endpoint families, address semantics, platform requirements, and deployment assumptions.

Address and endpoint configuration

The endpoint identity changes with the lower family.

Lower family	Endpoint identity
IPv4	host + port
IPv6	host + port
Unix domain sockets	local path
RFCOMM	Bluetooth address + channel
L2CAP	Bluetooth address + PSM

This changes the values passed to convenience calls, configuration objects, command-line options, and deployment scripts. The local/remote distinction remains useful. The concrete address form changes.

That is the central transfer lesson from Chapters 8 through 12. The framework model can remain stable while the meaning of an endpoint changes. A good design does not hide that change. It places it where it belongs.

Deployment assumptions

Deployment also changes.

Lower family	Deployment consequence
IPv4	network reachability, interface exposure, firewall and routing questions

Lower family	Deployment consequence
IPv6	IPv6 addressing, dual-stack or IPv6-only behavior, address notation
Unix domain sockets	local IPC, path ownership, path cleanup, local permissions
RFCOMM	Bluetooth stack availability, pairing/trust setup, channel semantics
L2CAP	Bluetooth stack availability, pairing/trust setup, PSM semantics

These differences are real. The protocol may transfer. The deployment does not become identical.

For Bluetooth, for example, the protocol may still be a simple stream protocol. But the devices still need the platform Bluetooth stack, suitable permissions, adapter state, and pairing/trust setup before SNode.C can communicate between them. For Unix domain sockets, the same protocol may run locally, but path placement and cleanup become part of the service design. For IP families, routing, exposure, firewall behavior, and address selection remain operational concerns.

The correct design statement is therefore not:

The lower family does not matter.

The correct design statement is:

The lower family matters, but it does not always have to rewrite the protocol endpoint.

15.5 Echo as the smallest transfer microscope

The echo application gives the smallest useful example. Echo is not interesting because echo is sophisticated. It is interesting because it exposes the placement boundary.

The protocol behavior lives in one context type:

```
EchoSocketContext::EchoSocketContext(SocketConnection* socketConnection, Role role)
```

The role is stored in the context.

When the connection becomes ready, the client role starts the exchange:

```
if (role == Role::CLIENT) {
    sendToPeer("Hello peer! Nice to see you!!!");
}
```

When data arrives, the context reads a chunk, sends the same bytes back, and reports the processed amount:

```
const std::size_t chunklen = readFromPeer(chunk, 4096);

if (chunklen > 0) {
    sendToPeer(chunk, chunklen);
}

return chunklen;
```

The factories then create role-specific contexts:

```
return new EchoSocketContext(socketConnection, EchoSocketContext::Role::SERVER);
```

and:

```
return new EchoSocketContext(socketConnection, EchoSocketContext::Role::CLIENT);
```

This is the important shape:

```
same context type
  -> role-specific construction
      -> same receive behavior
          -> different possible outer carriers
```

The point is not that every real protocol is as simple as echo.

The point is that the echo example makes the boundary visible:

- receive behavior belongs in the context,
- role construction belongs in the factory,
- lower-family selection belongs in the server/client handle type and configuration,
- endpoint identity belongs in the address/configuration layer,
- deployment remains outside the protocol core unless the protocol genuinely depends on it.

A more complex protocol can grow from the same discipline.

15.6 One protocol, several carriers

A compact comparison makes the transfer visible.

Carrier	Endpoint identity	What changes	What may remain
IPv4	host + port	address, configuration, and deployment	context, factory shape, protocol behavior
IPv6	host + port	address form, IPv6 semantics, deployment	context, factory shape, protocol behavior
Unix domain sockets	local path	local IPC path and lifecycle	context, factory shape, protocol behavior
RFCOMM	Bluetooth address + channel	Bluetooth stack, pairing/trust setup, channel semantics	context, factory shape, protocol behavior
L2CAP	Bluetooth address + PSM	Bluetooth stack, pairing/trust setup, PSM semantics	context, factory shape, protocol behavior

This table is not a portability guarantee. It is a design test.

If the protocol's meaning is independent of the lower-family identity, the context can often remain stable. If the protocol's meaning depends on lower-family identity, the context may need to specialize.

The table is useful precisely because it does not hide the changing column. It shows how far reuse can go before honesty requires a different context, a different factory, or a different deployment shape.

15.7 Designing for lower-family transfer

A protocol that should travel well across lower families should be designed with clear boundaries. The following rules are not mechanical requirements. They are design habits that make transfer easier to reason about.

Keep endpoint identity out of core protocol behavior unless it matters

If the protocol does not conceptually care whether the peer is identified by host/port, path, channel, or PSM, then those details should not dominate the context.

For example, an echo protocol does not inherently care whether the peer arrived through:

- IPv4,
- IPv6,
- Unix domain sockets,
- RFCOMM,
- or L2CAP.

The context can think in terms of:

```
connection became ready
data arrived
data was processed
response was sent
connection ended
```

That is the correct level for many application protocols.

A protocol may still inspect lower-family data for logging, diagnostics, authorization, routing, or policy. That is not forbidden. The important question is whether the lower-family detail is part of the protocol meaning or merely part of the carrier.

Keep per-connection state in the context

Connection-local protocol state belongs in the context.

That makes the protocol endpoint portable because the state travels with the connection, not with the outer handle or lower-family setup.

Good context state is:

- explicit,
- protocol-named,
- connection-local where possible,
- updated at understandable lifecycle and input points,
- and independent of lower-family details unless those details are part of protocol meaning.

This follows directly from Chapter 13. A context is the per-connection protocol endpoint. If a value describes the state of this conversation with this peer, the context is often the right scope for it.

Keep lower-family setup in the handle, instance, and configuration

The lower-family-specific setup belongs outside the protocol core.

This includes:

- choosing `net::in`, `net::in6`, `net::un`, `net::rc`, or `net::l2`,
- choosing legacy or TLS connection handling,
- setting address, path, channel, or PSM values,
- configuring local and remote endpoints,
- selecting retry or reconnect behavior,
- selecting deployment-specific options.

These choices are real and important. They should not be hidden inside the protocol behavior unless the protocol truly needs them.

The visible server/client handle and its configuration define what will be registered. The registered instance then participates in the runtime as a lower-family-specific communication role. Keeping this distinction clear prevents protocol code from becoming the place where deployment decisions are quietly made.

Use factories for role and construction differences

Factories are the right place to keep construction differences visible.

A server-side context may receive:

```
Role::Server
```

A client-side context may receive:

```
Role::Client
```

A publisher-side endpoint may receive:

```
Role::Publisher
```

A command-sink endpoint may receive:

```
Role::CommandSink
```

The point is not the exact enum.

The point is the boundary:

```
factory
  -> creates the right endpoint

context
  -> behaves as that endpoint
```

Construction-time variation is healthy when it makes the protocol roles clear. It becomes unhealthy when the factory begins to execute the protocol conversation. That boundary was the main lesson of Chapter 14, and it becomes especially useful when the same protocol travels across carriers.

15.8 Preconfigured factories and endpoint roles

Chapter 14 explained that server and client constructors can forward an argument pack into the factory constructor. That makes it possible to preconfigure factories with stable role and dependency information. This matters for lower-family transfer because the same mechanism can create role-specific endpoints over different carriers.

Examples include:

- producer / consumer,
- command source / command sink,
- event source / event receiver,
- requester / request handler,
- model-side / view-side / controller-side endpoints.

This chapter uses those roles only lightly.

The mechanism matters here because it supports transfer:

```
same endpoint role
  -> preconfigured factory
      -> context created for each connection
          -> lower family selected by application-side handle and configuration
```

The broader discussion of higher-level communication patterns belongs later.

Later chapters return to patterns such as publisher/subscriber, request/response, command/event, gateways, adapters, bridges, and protocol translators. Here the narrower point is lower-family transfer: a factory can keep endpoint construction stable while the outer communication family changes.

15.9 Same protocol shape does not mean same deployment

This distinction is essential. The same protocol shape may run over several lower families, but the resulting systems are not operationally identical. An IPv4 service may be reachable over a LAN or wider network.

An IPv6 service raises IPv6 addressing and deployment questions. A Unix-domain service is local to one host and depends on path placement and local access. An RFCOMM service depends on Bluetooth stack support, pairing/trust setup, adapter state, and RFCOMM channel semantics.

An L2CAP service depends on Bluetooth stack support, pairing/trust setup, adapter state, and PSM semantics. TLS adds certificate, trust, and connection-layer deployment questions. So lower-family transfer must be understood as an architectural separation, not as a promise that deployment disappears.

A good design lets the stable protocol core remain visible while keeping the family-specific deployment surface explicit.

15.10 When reuse should stop

Lower-family transfer is useful only when it preserves clarity. There are cases where reuse should stop.

Family-specific semantics may be part of the protocol

A protocol may genuinely depend on a lower-family detail.

For example, a Bluetooth-oriented protocol may care about device identity in a way that is not equivalent to an IP address or a Unix-domain path.

A local IPC protocol may depend on path placement or local access assumptions.

A network-facing protocol may treat remote address information as part of authentication, rate limiting, routing, or trust decisions.

If those details are part of the protocol's meaning, hiding them would be dishonest. In that case, the context should be allowed to know what it needs to know.

Deployment-specific protocols may deserve separate contexts

Sometimes two deployments look similar at first but differ enough that one shared context becomes awkward. A local diagnostic Unix-domain service may have different assumptions than a network-facing service. A Bluetooth device-near endpoint may have different lifecycle expectations than an IP service.

A TLS-protected network service may have different trust assumptions than a local IPC service. If one context becomes full of conditionals trying to cover all deployments, separate contexts may be clearer. This is not a failure of abstraction.

It is a sign that the protocol meaning has diverged.

Small duplication can be healthier than over-abstraction

Experienced C++ developers often try to remove duplication aggressively. That is not always the right instinct here.

A small amount of explicit outer-layer difference can be healthier than a giant abstraction that erases meaningful distinctions.

A good design may have:

- one stable protocol context,
- small family-specific handle declarations,
- clear family-specific configuration,
- small role-specific factories,
- and explicit deployment choices.

This is not failure. It is clean factoring. The goal is not to make all lower families look identical. The goal is to keep the stable protocol core stable and the real family-specific differences visible.

15.11 What remains stable and what changes

The following table summarizes the chapter.

Concern	Often stable	Often changes
Protocol behavior	<code>SocketContext</code> logic	only when family semantics matter
Context creation	factory shape and role construction	factory constructor data or selected context type
Application-side handle	conceptual server/client shape	concrete lower-family type
Registered instance	long-lived runtime-visible communication role	configured family, endpoint identity, operational behavior
Endpoint identity	local/remote distinction	host/port, path, channel, PSM
Runtime model	event-driven lifecycle	operational
Deployment	broad application intent	startup/configuration details reachability, locality, platform, pairing, permissions, security assumptions

This table is the practical transfer model. Reuse is strongest when the stable column stays honest and the changing column is not hidden.

15.12 Configuration becomes visible here

Lower-family transfer naturally makes configuration more visible.

A family transfer may change:

- selected server/client handle type,
- registered instance name,
- local endpoint values,
- remote endpoint values,
- role-specific factory arguments,
- TLS or legacy selection,
- retry or reconnect behavior,
- startup arguments,
- deployment files,
- platform or permission requirements.

That is why the next part of the book turns to configuration. Configuration is one of the places where architectural variation becomes explicit.

Once the same protocol can be carried over different lower families, configuration becomes the visible home for endpoint identity, selected lower family, role naming, TLS/legacy choice, retry behavior, and deployment-specific variation.

This chapter shows why configuration matters. Chapter 16 begins to explain its philosophy.

What to remember

- Lower-family transfer is possible when protocol behavior is kept in the `SocketContext`.
- The factory keeps context creation, role preconfiguration, and stable dependency passing separate from protocol behavior.
- The application-side server/client type and the registered instance change with the lower family; the protocol endpoint may remain stable.

- Endpoint identity and configuration change with the lower family: host/port, path, channel, or PSM.
- Deployment assumptions still matter; protocol reuse does not make systems operationally identical.
- Reuse should stop when lower-family semantics become part of the protocol's meaning.
- Small family-specific outer code is often clearer than an over-generalized abstraction.
- Chapter 16 begins the configuration view because lower-family transfer makes configuration visible.

15.13 The public surface changes with the lower layer

Changing the lower carrier changes the public role header and the linked component when application code directly names that carrier. The protocol context can remain stable, but the source/build front door follows the selected lower family and connection mode.

Carrier role	Public include	Matching component
IPv4 legacy stream server	<code><net/in/stream/legacy/ SocketServer.h></code>	<code>net-in-stream-legacy</code>
IPv6 legacy stream server	<code><net/in6/stream/legacy/ SocketServer.h></code>	<code>net-in6-stream-legacy</code>
Unix-domain legacy stream server	<code><net/un/stream/legacy/ SocketServer.h></code>	<code>net-un-stream-legacy</code>
IPv4 TLS stream server	<code><net/in/stream/tls/ SocketServer.h></code>	<code>net-in-stream-tls</code>

Chapter 32 gives the complete matrix. Here the point is the transfer rule: keep protocol behavior stable where possible, and change the carrier-facing surface deliberately.

15.14 Closing perspective

Part IV has separated protocol behavior, context creation, carrier selection, registration, and configuration. That separation is what makes lower-family transfer a design technique rather than a copy-and-edit exercise.

The next part turns to configuration: how applications are named, shaped, and operated after those architectural roles are in place.

Part V

Configuration and Operational Behavior

This part explains configuration as part of the architecture rather than as a late convenience layer. It connects application-side handles, configured communication roles, registered runtime-visible instances, generated configuration, logging, diagnostics, and runtime introspection.

The goal is to make operational behavior visible before systems become large.

Chapter 16

Configuration Philosophy in SNode.C

16.1 From lower-family transfer to configured communication roles

Chapter 15 showed why configuration becomes visible as soon as a protocol is carried over different lower communication families.

The protocol behavior may remain in the same `SocketContext`. The context creation policy may remain in the same `SocketContextFactory`.

But the concrete application still has to decide which server-side or client-side role exists, which lower family it uses, which endpoint identity it receives, whether it uses legacy or TLS connection handling, whether it is enabled, and which values belong in source code, a configuration file, or a run-specific command line.

That is the subject of this chapter.

Part IV separated protocol behavior, context construction, lower-family selection, and endpoint identity. Part V asks how these choices become visible, adjustable, persistent, and operable.

The central idea is:

Configuration rule

In SNode.C, configuration is one of the main ways a communication role becomes concrete.

The visible `SocketServer` or `SocketClient` object is the application-side handle. Through that handle, the application configures a server-side or client-side communication role. When `listen(...)` or `connect(...)` registers that role, the configured instance enters the runtime.

Configuration is therefore not decoration around an otherwise complete object. It is one of the places where the role receives its operational shape.

16.2 Configuration as part of the architecture

Configuration in SNode.C is not a separate afterthought beside the framework architecture. It is part of the architecture.

A communication role has several aspects:

```
application-side handle
-> configuration surface
-> registered server/client instance
```

```
-> concrete connections
    -> factories and contexts
```

The context implements the protocol behavior. The factory creates the context. The lower-family server or client type selects the communication family.

Configuration gives the role its concrete endpoint values, operational switches, and persistent identity.

That is why this chapter belongs immediately after Chapter 15. Lower-family transfer is practical only when the changing parts have somewhere clear to live. In SNode.C, configuration is one of those places.

A configured communication role

A configured communication role combines several things:

Part	Meaning
instance identity	which role is being configured
role identity	server-side or client-side behavior
lower-family shape	IPv4, IPv6, Unix domain sockets, RFCOMM, L2CAP, and connection variant
endpoint values	host, port, path, channel, PSM, or related local/remote values
section structure	scoped areas such as <code>local</code> , <code>remote</code> , <code>connection</code> , <code>socket</code> , <code>server</code> , and <code>tls</code>
operational state	enabled or disabled, persistent or run-specific options
activation	<code>listen(...)</code> or <code>connect(...)</code> using supplied configuration

This is a different view from “an object with some options.”

A configured role is the architectural place where the application says:

```
this role exists
this is how it is named
this is how it is addressed
this is how it participates
this is how it can be operated
```

The previous chapters established the runtime and protocol roles. It explains how those roles become concrete enough to run in an application.

One model, several entry paths

SNode.C has three main configuration input paths:

Input path	Main use
C++ API	baseline defaults and programmatic shaping
configuration file	persistent operational configuration
command line	run-specific override, discovery, and control

These are not three unrelated configuration worlds. They are three ways to feed one underlying configuration model. The C++ API gives the application a baseline.

The configuration file gives deployment a durable expression. The command line gives a run-specific control surface.

That matters because the same conceptual role can be shaped in code, persisted in a file, and overridden for one invocation without changing its identity.

16.3 Three input paths, one configuration model

The three input paths have different strengths. They should be understood together.

Configuration through the C++ API

The C++ API is the most direct configuration path.

The application-side server or client handle exposes its configuration object, and the application can set values on it directly.

For an IPv4 server, a minimal example may look like:

```
EchoServer echoServer;

echoServer.getConfig()->Local::setPort(8001);
```

In this case, the explicit `Local::` qualification is valid, but it is not required. The same server-side port can also be configured as:

```
echoServer.getConfig()->setPort(8001);
```

`getConfig()` returns a pointer to the configuration object, so the examples use `->`.

The `Local::` qualification is optional here because this IPv4 server configuration does not expose two competing `setPort(...)` meanings at this call site. There is no local-versus-remote ambiguity for the server-side port in this example.

code can provide baseline configuration. It can also express values that are genuinely programmatic: values derived from application structure, construction-time decisions, tests, or small teaching examples.

Convenience overloads such as:

```
listen(8001, onStatus)
```

or:

```
connect("localhost", 8001, onStatus)
```

participate in the same idea.

They are readable API calls. Conceptually, they also fill configuration values on the application-side handle before the role enters the activation path.

In-code configuration is therefore useful for:

- small teaching examples,
- reasonable defaults,
- programmatically selected values,
- tests and experiments,
- application-level decisions known at construction time.

It is not a second-class configuration path. It is one native way to shape a communication role.

Configuration through the command line

Command-line configuration gives an already compiled application a way to be shaped at startup. That is especially important for named instances.

A named server instance such as:

```
EchoServer echoServer("echo");
```

can appear as an addressable communication role on the command line. Under that instance, sections expose the available configuration scopes.

A typical hierarchy is:

```
application
  -> instance
    -> section
      -> option
```

The command line is therefore not only a way to override values. It is a discovery surface.

A user can ask for help at different levels:

```
echoserver --help
echoserver echo --help
echoserver echo local --help
```

Each level reveals another part of the configuration model.

This is important for applications that expose more than one communication role. Instead of hiding all options behind a flat list, the configuration hierarchy shows how options belong to roles and sections.

Configuration through configuration files

Configuration files persist the same model. They do not introduce a second configuration language with a different worldview.

Their key structure follows the same hierarchy:

```
instancename.sectionname.optionname = value
```

For example:

```
echo.local.port = 8080
echo.remote.host = "localhost"
echo.remote.port = 8080
```

This is the file form of the same application/instance/section/option structure exposed by the command line.

The configuration file is therefore both:

- an operational artifact,
- and an inspectable map of the configured application.

It is operational because it can be used by a deployed application. It is inspectable because it shows the same structure that the command line and the C++ API feed.

16.4 Precedence and the startup boundary

The precedence rule is part of the configuration philosophy.

The order is:

```
command line
  > configuration file
    > C++ API
```

That means:

Source	Typical role
C++ API	baseline defaults or programmatic intent
configuration file	persistent operational configuration
command line	run-specific override

This is the practical operational model. The application can encode reasonable defaults. The configuration file can store deployment choices.

The command line can override them for one run.

The order is important because it lets an application be useful out of the box, adaptable in deployment, and still controllable at startup.

Instance creation timing

There is also an important timing boundary.

External configuration can only address roles that are present in the configuration hierarchy when the application parses its startup configuration. A role created later by application logic must be configured by application logic.

This separates two cases:

```
startup-known configured roles
-> C++ API, configuration file, and command line can participate

runtime-created roles
-> C++ API configuration is the available path
```

That is not a weakness. It is the natural boundary between startup configuration and runtime object creation.

The command line and configuration file are external startup inputs. They can shape what has been made addressable before startup parsing completes. They cannot retroactively reach into a role that did not yet exist in that hierarchy.

16.5 Named instances as configuration addresses

One of the most important ideas in the configuration model is the difference between anonymous and named instances. An anonymous instance exists as a configured role in application code. A named instance also becomes addressable by the external configuration hierarchy.

That is the key distinction.

Anonymous and named instances

A useful rule of thumb is:

Anonymous instance	Named instance
internal helper clients	externally operated communication roles
temporary or embedded helper servers	independently configurable server roles
no external configuration address needed	persistent or inspectable behavior
tiny demos	operational applications
one-off experiments	command-line and file control
simplest code	multi-instance applications

Anonymous instances are useful for communication roles that are intentionally internal to the program and should not become independently addressable through the external configuration model.

This is especially common for helper clients.

For example, an application may need to fetch data from a known remote resource as part of its own internal behavior. If that communication role should not be configured, disabled, persisted, or inspected independently, an anonymous client is often the clearer choice.

Anonymous servers can also make sense, but the case is narrower.

They are reasonable for temporary local test servers, embedded loopback-only services, or helper servers whose endpoint is fixed by the surrounding program.

Externally operated server roles are usually better expressed as named instances, because servers often need deployment-facing control over bind address, port, path, channel, enablement, and persistent configuration.

In other words:

```
anonymous
-> application-internal role

named
-> externally addressable configured role
```

A named instance is attached to the external configuration hierarchy. An anonymous instance remains application-internal and does not appear as an independently addressable command-line/config-file sub-command.

Why names matter

A name such as:

```
EchoServer echoServer("echo");
```

gives the configured role an address in the configuration hierarchy, with the readable C++ name as its source-level anchor.

That address can appear:

- in command-line help,
- in command-line overrides,
- in generated command lines,
- in shown configuration,
- and in configuration-file keys.

The name becomes part of the operational surface of the application. That is why named instances are central for serious applications: the name is a stable handle for operation.

Disablement as instance-level configuration

A configured communication role may exist but be inactive. SNode.C treats this as an instance-level configuration idea. A named instance can be disabled instead of being removed from the application.

That is useful for:

- multi-instance programs,
- optional carriers,
- staged deployment,
- development and testing,
- field diagnostics,
- temporarily disabling a communication role without rebuilding.

The important idea is:

```
configured role exists
-> configuration may decide whether it participates
```

Disablement belongs to the configured role. It lets the role remain part of the application shape while configuration decides whether it participates in a run.

This is cleaner than scattering ad hoc application flags across the program.

16.6 Sections as structural scopes

Sections are structural scopes in the configuration hierarchy. A section is the boundary between different kinds of responsibility inside a communication role.

The common section names make this visible:

- `local`,
- `remote`,
- `connection`,
- `socket`,
- `server`,
- `tls`.

A section groups options that belong to a particular part of the role.

Section hierarchy

The shape is:

```
application
  -> instance
    -> section
      -> option
```

For a named instance `echo`, examples include:

```
echo local --port 8080
echo remote --host localhost --port 8080
```

or in configuration-file form:

```
echo.local.port = 8080
echo.remote.host = "localhost"
echo.remote.port = 8080
```

The command-line hierarchy and the configuration-file hierarchy express the same structure.

That is the main point: the hierarchy is not an accidental CLI shape. It is the visible form of the configured communication role.

Local and remote sections

The `local` and `remote` sections are especially important because they connect directly to the address semantics from Chapters 8–12.

For a server, local configuration usually describes the endpoint to bind or listen on.

For a client, remote configuration usually describes the peer endpoint to connect to, while local configuration may describe an explicit bind side.

The exact fields depend on the lower family:

Family	Typical endpoint fields
IPv4 / IPv6	host and port
Unix domain sockets	path

Family	Typical endpoint fields
RFCOMM	Bluetooth address and channel
L2CAP	Bluetooth address and PSM

The section model lets these differences remain structured without changing the larger configuration idea. The family changes the endpoint fields. The hierarchy remains recognizable.

Connection, socket, server, and TLS sections

Other sections describe other parts of the communication role.

For example:

Section	Typical scope
<code>connection</code>	established connection behavior, timeouts, read/write behavior
<code>socket</code>	socket-level behavior such as retry or reuse options
<code>server</code>	server-specific behavior
<code>tls</code>	TLS-related connection-layer configuration

Chapter 17 can go into these details more concretely.

For Chapter 16, the important point is the structure:

```
sections are scopes
not loose option groups
```

This keeps configuration readable. A timeout, an address, a TLS certificate path, and a server backlog do not all live in one flat pile. They belong to different scopes of the same configured role.

16.7 Configuration as discovery, persistence, and inspection

A good configuration system should not only accept values. It should help users discover what can be configured, inspect the active shape, and persist durable choices. SNode.C does this through command-line help, configuration display, command-line generation, and generated configuration files.

Help output as discovery

Because named instances and sections participate in the command-line hierarchy, help output can reveal the configuration model step by step.

At application level, help shows application-wide options and available instances. At instance level, help shows instance-level options and sections. At section level, help shows the options for that section.

This makes the configuration system self-describing. The user does not have to guess the entire option universe at once. The hierarchy itself teaches the shape of the application.

Persistent and nonpersistent options

Not every option should be written into a configuration file. Some options describe lasting behavior. Others describe one run.

SNode.C makes that distinction visible by separating persistent and nonpersistent options.

The practical rule is:

```
persistent options
-> may be stored in configuration files

nonpersistent options
-> affect inspection, generation, help, or the current run only
```

Persistent options describe the desired shape of an application or instance. Nonpersistent options trigger inspection, generation, display, help, or one-run control actions.

This prevents operational commands such as help, display, command-line generation, or write-config from being confused with enduring instance configuration.

Generated and shown configuration

Generated configuration files show the configuration model in file form.

A generated file can contain:

- application-level options,
- instance-level options,
- section-level options,
- default values,
- values supplied in code,
- values overridden by file or command line,
- comments describing the available options.

Shown configuration and generated command lines serve a similar purpose from different angles. They make the active or possible configuration visible. That makes the file both editable and educational.

It is an inspectable artifact of the same hierarchy used by the command line and the C++ API.

16.8 Parameterless listen() and connect()

Parameterless activation is one of the clearest expressions of the configuration philosophy.

A call such as:

```
echoServer.listen(onStatus);
```

or:

```
echoClient.connect(onStatus);
```

means:

The role should already know enough about its configured shape to act.

The endpoint values may have been supplied by:

- C++ API calls,
- a configuration file,
- command-line arguments,
- or a combination of those according to the precedence rule.

The call itself does not need to repeat them.

Configured enough to act

For a server, being configured enough may mean that the local endpoint is known.

For a client, being configured enough may mean that the remote endpoint is known, and possibly also a local bind side.

For TLS instances, it may also mean that required TLS-related configuration is available. The exact requirements depend on the concrete server/client type.

The architectural idea is stable:

```
configuration shapes the role
activation uses the shaped role
```

This is the bridge between configuration and runtime behavior.

A configured role has already been prepared for activation. The parameterless call is readable precisely because the role has already been shaped.

Missing configuration is reported structurally

When required configuration is missing, the error path should point back into the same hierarchy. For example, a missing server port belongs to the configured server role that required it.

It belongs to a particular instance and section:

```
application
  -> instance
    -> local
      -> port
```

A missing client host or port likewise belongs to the remote section of a particular client instance. Therefore, parameterless activation is a strong proof point. It works only because the role has a structured configuration identity.

16.9 CLI11 as implementation foundation

SNode.C uses CLI11 as part of the implementation foundation for this unified model. That matters, but it should stay in the background of the chapter. The book does not need to become a CLI11 manual here.

The architectural point is:

```
SNode.C exposes one hierarchical configuration model
through code, command line, and files.
```

CLI11 helps implement that model.

The important thing for the SNode.C reader is the resulting structure:

```
application
  -> named instance
    -> section
      -> option
```

That is the model the application author and operator see.

The implementation foundation matters because it makes help output, configuration files, option grouping, command-line generation, and persistent/nonpersistent classification possible. But the conceptual model remains the SNode.C configuration hierarchy.

16.10 Configuration as architectural leverage

Configuration gives the architecture operational leverage.

Because configuration can shape a role without rewriting protocol code, the same application structure can often be reused across deployments.

A protocol may remain in the context. The factory may keep creating the same kind of endpoint. The lower-family handle type, registered instance, endpoint identity, TLS/legacy selection, and deployment values may change.

Configuration is one of the places where that variation becomes explicit. This connects directly back to Chapter 15. Lower-family transfer is practical only if the changing parts have somewhere clear to live.

In SNode.C, configuration is one of those places.

What to remember

- Configuration is part of the SNode.C architecture, not an afterthought beside it.
- The application-side handle configures a server-side or client-side communication role; the registered instance carries that configured role into the runtime.
- The C++ API, configuration files, and command line feed one hierarchical configuration model.
- Named instances become addressable in that hierarchy; anonymous instances remain internal to application code.
- Sections such as `local`, `remote`, `connection`, `socket`, `server`, and `tls` scope options by responsibility.
- Parameterless `listen(onStatus)` and `connect(onStatus)` are useful because the role can already be configured before activation.

16.11 Closing perspective

This chapter opened the configuration part of the book. The preceding part showed that protocol behavior, context creation, lower-family choice, and endpoint identity can be separated.

It looks more closely at application configuration, instance configuration, sections, and the concrete option structure that turns this model into something an application can expose and an operator can use.

Chapter 17

Application and Instance Configuration in Detail

17.1 From configuration philosophy to configuration anatomy

Chapter 16 introduced configuration as one of the main ways an application-side server/client handle shapes a communication role before that role is registered as a runtime-visible instance.

This chapter looks at the practical anatomy of that model.

The core hierarchy is:

```
application
  -> instance
    -> section
      -> option
```

The application is the executable-level operational shell. A named instance is the externally addressable configuration identity of one configured communication role inside that application. A section is one scoped part of that instance configuration.

An option is one concrete value inside such a scope.

This chapter follows that hierarchy from the outside inward. The goal is not to memorize every possible option. The goal is to understand where a value belongs, why it belongs there, and how the same structure appears in C++ code, on the command line, and in configuration files.

17.2 The anatomy of the configuration hierarchy

The configuration model is easiest to read from the outside inward. Figure 17.1 shows that hierarchy as one structural model. The named-instance level is where a configured server/client role receives an externally addressable identity and can be enabled or disabled without removing the role from the application shape.

The figure is a placement model. It shows where configuration meaning belongs: executable-wide concerns at application scope, externally addressable communication roles at instance scope, responsibility groups at section scope, and individual values at option scope. The concrete sections and options in real applications are narrower and more numerous than the diagram needs to show.

Scope	Addressed by	Typical concerns
application	executable-level options	config file, logging, daemonization, help, generated command line

Scope	Addressed by	Typical concerns
instance	named server/client communication role	disabled state, role identity, section collection
section	<code>local</code> , <code>remote</code> , <code>connection</code> , <code>socket</code> , <code>server</code> , <code>tls</code>	concrete endpoint or behavioral settings
option	individual setting	host, port, timeout, retry, backlog, certificate path

This hierarchy organizes runtime responsibility; documentation follows from that structure.

It is the practical shape used by the framework. It appears in code, on the command line, and in configuration files.

Application scope

Application scope belongs to the executable as a whole.

It contains concerns that are not specific to one network endpoint. Examples include:

- selecting a configuration file,
- writing a configuration file,
- showing configuration,
- printing a generated command line,
- showing help and version information,
- logging level and log-file handling,
- daemonization,
- user and group selection for daemonized runs.

These options form the operational envelope in which all configured communication roles live.

They answer questions about the process as a program, not about a particular server port, peer address, Unix-domain path, Bluetooth channel, or TLS certificate used by one instance.

Instance scope

Instance scope belongs to one named server-side or client-side communication role.

It contains concerns such as:

- instance identity,
- server/client role identity,
- disabled state,
- configurability,
- and the set of sections that shape the role.

The visible server/client object in application code is the application-side handle. In the configuration model, a named instance is the externally addressable configuration identity of the communication role that the handle configures and registers.

That distinction matters. The configuration system gives a communication role an operational address instead of merely decorating a local C++ variable.

Section scope

Section scope belongs to one aspect of one instance.

Common sections include:

- `local`,
- `remote`,
- `connection`,
- `socket`,
- `server`,

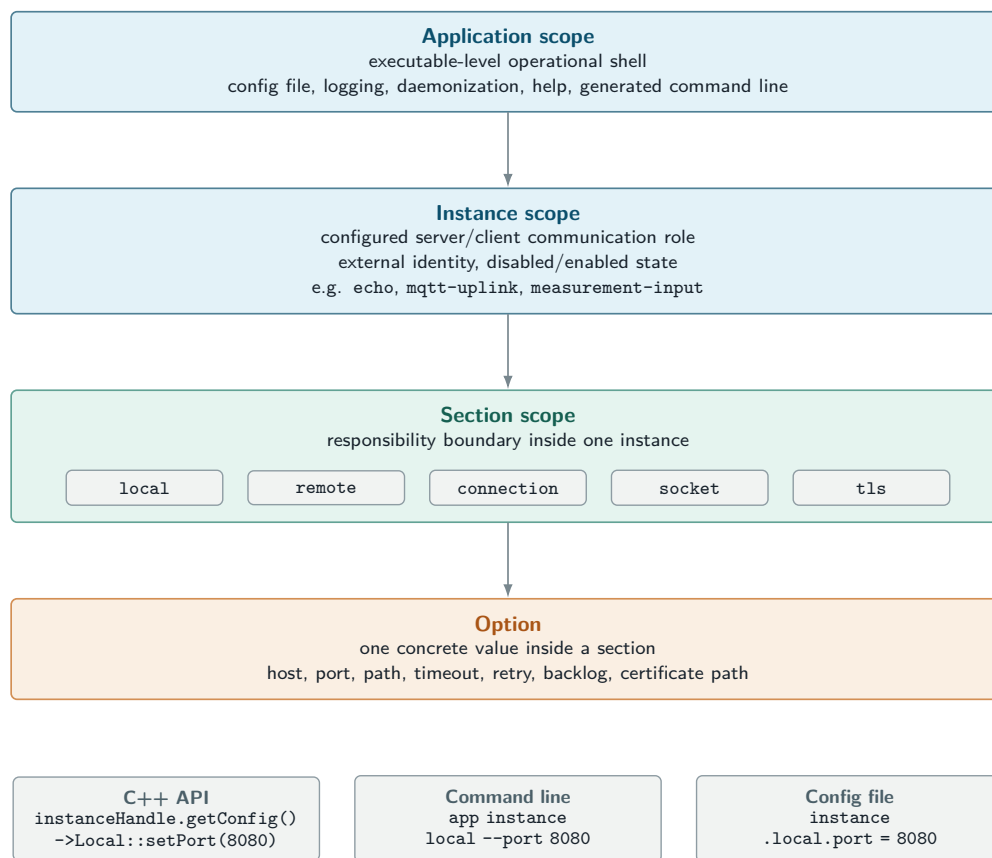


Figure 17.1: The SNode.C configuration hierarchy: application scope contains named communication-role instances; each instance contains responsibility sections; each section owns concrete options.

- `tls`.

A section groups options by responsibility.

The `local` section describes the local side.

The `remote` section describes the peer side.

The `connection` section describes established connection behavior.

The `socket` section describes socket-level behavior.

The `server` section describes server-role behavior.

The `tls` section describes TLS-related connection-layer behavior.

Sections are the boundaries between different kinds of responsibility inside one configured communication role. Help output reflects that boundary.

Same hierarchy in code, CLI, and files

The same hierarchy appears in three views:

View	Example shape
C++ API	<code>instance.getConfig()->Local::setPort(8080)</code>
command line	<code>app instance section --option value</code>
configuration file	<code>instance.section.option = value</code>

The syntax changes. The hierarchy remains the same.

That consistency is the main reason the detailed configuration model remains understandable. The C++ API gives the application direct programmatic control. The command line gives startup traversal and run-specific overrides. The configuration file gives a durable dotted-key representation of the same structure.

17.3 Application configuration: the operational envelope

Application configuration shapes the operational shell of the executable. It should not be confused with the configuration of one communication role.

Application-level options answer questions such as:

- Which configuration file should be read?
- Should a configuration file be written?
- Should the configuration be shown?
- Should a generated command line be printed?
- Should the application run as a daemon?
- Which log level should be used?
- Which log file should be used?
- Should the process run under a specific user or group?
- Should help or version information be displayed?

These are not address-family questions.

They do not say which port a server listens on or which peer a client connects to. They shape the executable as an operating program.

Operational shell of the executable

The application scope is above all instances. That boundary matters.

Application-level concern	Why it belongs at application scope
configuration-file path	it controls how the executable reads or writes configuration
<code>--show-config</code>	it displays the model as parsed and configured
generated command line	it describes how to reproduce the selected configuration
logging	it affects the executable's operational visibility
daemonization	it affects process behavior
user/group selection	it affects process permissions

These settings are shared by the application process. They are not owned by one server or client role.

Persistent and nonpersistent application options

Application options can be persistent or nonpersistent. Persistent options describe durable application shape and may be written into configuration files. Nonpersistent options inspect, generate, display, or control one run.

That distinction prevents operational commands from becoming durable configuration by accident. For example, a log level may be persistent. A request to show help, show configuration, print a command line, or write a configuration file is run-specific.

The distinction keeps the configuration file focused on lasting application behavior.

17.4 Instance configuration: the configured communication role

Instance configuration shapes one communication role.

A named instance is where the configuration model becomes concrete. It has an identity, a role, optional disabled state, and a collection of sections.

Named configurable communication roles

A named server handle such as:

```
EchoServer echoServer("echo");
```

creates a communication role that can become addressable by the configuration system.

The name can appear:

- in command-line help,
- in command-line overrides,
- in generated command lines,
- in shown configuration,
- and in configuration-file keys.

That is why named instances matter.

They connect the application-side handle to the operational surface of the application. The name is the address by which operators, scripts, configuration files, and diagnostic output can talk about that role.

Role identity: server or client

A configured role also has role identity. In the configuration model, that means an instance is constructed as a server role or a client role.

This text influences how the instance appears in help output and how its configuration is interpreted.

A server role is normally shaped around listening and accepting. A client role is normally shaped around connecting.

Both can share the same broad hierarchy, but their most important sections differ. For a server, `local` is usually central. For a client, `remote` is usually central.

Anonymous versus named instances

Chapter 16 distinguished anonymous and named instances.

Chapter 17 mostly concerns named instances, because named instances are addressable from the command line and from configuration files.

An anonymous instance may still be useful for an internal helper client or a temporary helper server. But once a communication role should be configured, disabled, persisted, inspected, or controlled from outside the source code, a named instance is usually the right model.

A useful distinction is:

Anonymous instance	Named instance
application-internal role	externally addressable role
configured by code	configurable through the external hierarchy
not independently visible in CLI/file keys	visible in help, shown configuration, generated command lines, and files
useful for helper clients or small experiments	useful for operational communication roles

This does not make anonymous instances inferior. It only means that they belong to a different design situation.

Disablement and requiredness

Disablement is a first-class instance state, not a loose Boolean label.

Disablement lets a configured role remain part of the application shape while being removed from the required startup path for a particular run. That matters for multi-instance programs.

An executable may contain several possible communication roles, while only some of them are active in a particular deployment. Configuration can then express:

```
this role exists
-> but it is disabled for this run
```

That is cleaner than removing the role from the application or inventing separate ad hoc flags.

It also keeps help output, configuration files, and generated command lines honest: the role still exists as part of the application design, but configuration decides whether it participates.

17.5 Section configuration: scoped responsibilities

Sections are the most important practical organizing device inside an instance. A section is a structural scope. It groups options that belong to one aspect of the communication role.

A compact overview is:

Section	Main meaning
<code>local</code>	local endpoint or bind side
<code>remote</code>	peer endpoint or connect side
<code>connection</code>	behavior of established connections
<code>socket</code>	socket-level behavior, retry, and reuse settings
<code>server</code>	listen and accept behavior
<code>tls</code>	TLS connection-layer configuration

Not every instance uses every section with the same importance.

Some sections appear only for particular role and layer combinations. The structure is shared, but the practical emphasis depends on the concrete instance.

The useful reading habit is:

```
section name
-> responsibility boundary
    -> representative options
```

This prevents the chapter from becoming a flat option catalogue.

The local section

The `local` section describes the local side of the communication role.

For a server, it is usually the most important endpoint section because it describes where the server binds or listens.

For a client, it may describe an explicit local bind side if the application does not want to leave that side wildcarded.

The concrete fields depend on the lower family.

Lower family	Typical local fields
IPv4 / IPv6	host and port
Unix domain sockets	path
RFCOMM	Bluetooth address and channel
L2CAP	Bluetooth address and PSM

The idea is stable:

```
local
-> this side of the connection
```

The concrete option names depend on the lower family.

The remote section

The `remote` section describes the peer side.

For a client, it is usually central because it answers:

```
Where should this client connect?
```

For example, an IPv4 client may need a remote host and port. A Unix-domain client may need a remote path. A Bluetooth client may need a Bluetooth address plus channel or PSM.

For a server, remote-side information may be less prominent in simple listen scenarios.

The durable distinction is:

```
local side
!= remote side
```

That distinction remains useful across address families. The concrete fields change, but the conceptual boundary remains.

The connection section

The `connection` section describes behavior of established peer relationships.

Representative concerns include:

- read timeout,
- write timeout,
- read block size,
- write block size,
- termination timeout.

These are not endpoint identity values. They apply after a connection exists.

That makes the section boundary clear:

```
connection
-> behavior of the established connection
```

The `connection` section therefore belongs between the lower communication machinery and the protocol context. It does not decide where to bind or where to connect. It shapes how established peer relationships behave.

The socket section

The `socket` section describes socket-level behavior around creation, retry, and reuse.

Representative concerns include:

- address reuse,
- retry behavior,
- retry on fatal errors,
- retry timeout,
- retry attempts,
- exponential backoff,
- jitter,
- retry limit,
- role- or family-specific socket options.

The exact set depends on the concrete instance.

The section groups behavior that is closer to the socket and flow-control machinery than to the application protocol. A retry timeout, for example, is not part of an echo protocol. It belongs to the machinery that tries to establish or maintain a communication role.

The server section

The `server` section belongs to server-role behavior.

Representative concerns include:

- backlog,
- accepting behavior,
- accept-per-tick style limits where supported.

These settings are not generic connection options. They are about the listening and accepting side of a server role. That is why they belong in a server-specific section.

The tls section

The `tls` section belongs to TLS connection-layer configuration.

Representative examples include:

- certificate chain,
- certificate key,

- CA certificate or directory,
- cipher and option settings,
- SNI-related settings,
- TLS initialization or shutdown timeouts.

TLS is not a minor socket flag.

It is a connection-layer specialization. Grouping TLS configuration under `tls` keeps that concern separate from endpoint identity, socket retry behavior, and application protocol behavior.

Chapter 19 discusses TLS in depth. Here the important point is the section boundary.

17.6 Three views of the same model

Chapter 16 established that `SNode.C` has three input paths into one model.

Chapter 17 makes the practical consequence visible: the same hierarchy can be read in code, on the command line, and in configuration files.

C++ API view

In the C++ API, the hierarchy is visible through the configuration object and its section-qualified operations.

Schematic examples are:

```
instance.getConfig()->Local::setPort(8080);
instance.getConfig()->Remote::setHost("localhost");
instance.getConfig()->Remote::setPort(8080);
```

`getConfig()` returns a pointer, so the examples use `->`. The section qualification is useful when local and remote meaning must remain visible.

In server-side cases, a shorter form may also be valid because there is no local/remote ambiguity at that call site:

```
instance.getConfig()->setPort(8080);
```

The important habit is to think in sections even when the C++ syntax permits a shorter call. The shorter call may be readable in a small example, but the section-qualified call documents the architectural boundary.

Command-line view

On the command line, the hierarchy becomes a path:

```
echoserver echo local --port 8080
```

This can be read as:

```
application: echoserver
instance:    echo
section:     local
option:      port
value:       8080
```

The command line is therefore not just a flat collection of flags. It is a textual traversal of the configuration hierarchy.

Command-line configuration as guided traversal

The command-line interface is also a guided configuration surface.

It can help the user discover the model step by step instead of requiring the full structure to be known in advance:

```
echoserver --help
echoserver echo --help
echoserver echo local --help
```

This mirrors the hierarchy:

```
application help
  -> instance help
    -> section help
      -> option details
```

The command line can also print command-line representations of the selected configuration. The `--command-line` option supports views such as `standard`, `required`, `active`, and `complete`.

Together with `--show-config` and `--write-config`, the command line becomes a way to inspect, reproduce, and persist the configuration of an application.

Guided errors for missing configuration

The same guided behavior appears when required configuration is missing.

A parameterless `listen()` or `connect()` can fail in a way that points the user back into the hierarchy:

```
application
  -> instance
    -> section
      -> required option
```

A compact server-side session shows the idea. Here the executable is `echoserver` and the named server instance is `echo`:

```
$ echoserver
[RequiresError] echoserver requires echo

$ echoserver echo
[RequiresError] echoserver:echo requires local

$ echoserver echo local
[RequiresError] echoserver:echo:local requires --port

$ echoserver echo local --port
[ArgumentMismatch] --port: 1 required port:UINT in [0 - 65535] missing

$ echoserver echo local --port 8080
2026-06-06 18:04:05 00000000000001 echo: listening on '0.0.0.0:8080 (0.0.0.0)'
```

The exact timestamp and tick counter are run-specific. The important point is the direction. The CLI leads from application, to instance, to section, to option.

A named client instance follows the same idea, but the required section is usually `remote` rather than `local`. The CLI therefore teaches the structure while it reports the missing values.

Startup guidance, not arbitrary live reconfiguration

This command-line guidance belongs to startup and run configuration. It should not be confused with arbitrary post-start interactive reconfiguration of instances created later at runtime.

As Chapter 16 explained, command-line and file configuration apply to startup-known instances. Runtime-created roles must be shaped through the C++ API.

That boundary keeps the model clear:

```
startup-known roles
  -> guided by code, command line, and configuration files

runtime-created roles
  -> configured by code
```

The command line traverses the startup configuration model. It does not become a live management protocol for roles created later by application logic.

Configuration-file view

In a configuration file, the same hierarchy becomes a dotted key:

```
echo.local.port = 8080
echo.remote.host = "localhost"
echo.remote.port = 8080
```

The dotted key is the file representation of:

```
instance
  -> section
    -> option
```

The syntax is different from the command line. The model is the same.

The file is therefore not a separate configuration universe. It is the persistent expression of the same hierarchy that the command line traverses and the C++ API configures directly.

17.7 Required values and progressive disclosure

Parameterless `listen()` and `connect()` rely on configuration that is already present. If required configuration is missing, the error path can reveal the missing part of the hierarchy.

For a server, a missing port can be understood as:

```
application
  -> instance
    -> local
      -> port
```

For a client, missing peer information can be understood as:

```
application
  -> instance
    -> remote
      -> host / port
```

This is scoped error reporting.

It reinforces the structure of the configuration model. The error belongs to a scope that the user can inspect, instead of appearing as a generic failure.

Parameterless activation

A parameterless activation call such as:

```
echoServer.listen(onStatus);
```

or:

```
echoClient.connect(onStatus);
```

means that the communication role should already be configured enough to act.

The required values may have come from:

- C++ API defaults,
- a configuration file,
- command-line arguments,
- or the precedence model combining all three.

Therefore, parameterless activation is such a strong proof point for the configuration architecture. The call does not repeat the endpoint identity because the configured role already owns that identity.

Progressive disclosure as a teaching tool

Progressive disclosure is useful for operators, but it is also useful for readers.

It teaches the model in the order in which the model is structured:

```
application
-> instance
    -> section
        -> option
```

This is especially valuable in multi-instance programs. Instead of forcing every option into one flat help page, the hierarchy lets the user ask increasingly specific questions.

17.8 Persistent and nonpersistent values

Persistent options describe durable configuration. Nonpersistent options perform run-specific inspection or control. This distinction appears throughout the configuration model.

Persistent examples include values such as:

- endpoint host or port,
- Unix-domain path,
- Bluetooth channel or PSM,
- connection timeout,
- retry settings,
- TLS certificate paths,
- logging level or log-file path where configured as durable behavior.

Nonpersistent examples include:

- help,
- show configuration,
- generated command-line output,
- write-configuration action,
- version display,
- one-run control actions.

The important distinction is not whether an option is “important.” Both kinds can be important. The distinction is whether the option describes lasting shape or performs an action for one run.

Configuration files should primarily describe durable shape. Command-line invocations can both supply durable values and trigger nonpersistent inspection actions.

17.9 Generated and shown configuration

Generated configuration and shown configuration make the hierarchy inspectable.

A shown configuration helps answer:

```
What did the application understand after code defaults, files, and command-line input were combined?
```

A generated configuration helps answer:

```
What file form would represent this hierarchy?
```

A generated command line helps answer:

```
What command-line form would reproduce the relevant option values?
```

These views are diagnostic tools. They make configuration visible instead of implicit.

That matters because configuration errors are often not bugs in protocol code. They are mismatches between intended deployment shape and actual configured values.

17.10 Configuration files as operational artifacts

Configuration files are operational artifacts.

They should be readable by humans, stable enough for deployment, and close enough to the command-line hierarchy that users can move between both views without learning a second model.

The dotted-key structure helps:

```
echo.local.port = 8080
```

This can be read as:

```
instance echo
  -> section local
    -> option port
```

For multi-instance applications, this becomes especially useful:

```
public.local.port = 8080
admin.local.port = 9090
backend.remote.host = "127.0.0.1"
backend.remote.port = 1883
```

Each key says which configured role it belongs to. That is the value of named instances. They make configuration files describe application structure, not just isolated values.

17.11 Designing configuration for real applications

The detailed model also suggests design habits.

Name externally operated roles

Externally operated server roles usually deserve names.

If a server binds a deployment-facing endpoint, accepts peers, uses TLS certificates, participates in logging, or may be disabled per deployment, it should normally be visible in the configuration hierarchy.

Helper clients may remain anonymous when they are intentionally internal to application logic.

The question is not:

Can this role be named?

The better question is:

Should this role be independently configured, inspected, persisted, or disabled?

If yes, naming it is usually the clearer design.

Keep endpoint options in endpoint sections

Endpoint values should live in `local` or `remote`, not in random application flags.

That makes address-family differences manageable. IPv4, IPv6, Unix-domain sockets, RFCOMM, and L2CAP all have different concrete endpoint fields, but the local/remote distinction remains stable.

This is the same design lesson as Chapters 08–12, now expressed through configuration.

Keep protocol behavior out of configuration

Configuration can select endpoints, timeouts, retry behavior, TLS settings, and activation shape. It should not become the protocol implementation.

The protocol still belongs in `SocketContext`.

The construction boundary still belongs in `SocketContextFactory`.

Configuration should expose variation; it should not replace application design.

Avoid hiding deployment shape in source-only defaults

C++ defaults are useful. They make examples small. They provide reasonable baselines. They help tests and embedded use cases.

But deployment-facing choices often deserve external visibility. Ports, paths, certificate files, log files, daemonization, and enablement are easier to operate when they are visible through the configuration hierarchy.

A good application can use both: source-level defaults for clarity and external configuration for deployment.

17.12 What remains stable

Across all these details, the stable model is:

```
application
  -> instance
    -> section
      -> option
```

The application gives the operational envelope. The instance gives the configured communication role an address. The section gives one responsibility scope.

The option gives one value. The same shape appears in C++ API calls, command-line traversal, and configuration-file keys. Therefore, the configuration model scales from a small echo example to applications with several communication roles.

What to remember

- The configuration hierarchy is **application** -> **instance** -> **section** -> **option**.
- Application configuration describes the operational envelope of the executable.
- A named instance is an externally addressable configured communication role; an anonymous instance remains internal to application code.
- Sections such as **local**, **remote**, **connection**, **socket**, **server**, and **tls** group options by responsibility.
- The same hierarchy appears through the C++ API, command line, and configuration files.
- Disablement lets a role remain present while no longer blocking startup as a required participant.

17.13 Closing perspective

Chapter 16 explained the philosophy of configuration. This chapter made the anatomy concrete.

Chapter 18 therefore turns to logging, diagnostics, generated command lines, shown configuration, and runtime introspection as ways to understand what an application actually did.

Chapter 18

Logging, Diagnostics, and Runtime Introspection

18.1 From configured roles to visible runtime behavior

Chapter 17 showed how the configuration hierarchy shapes the executable, named communication roles, sections, and option values.

The structure was:

```
application
  -> instance
    -> section
      -> option
```

Once those configured roles are activated and registered as runtime-visible instances, the next question is:

How can we see what the application is doing?

That is the subject of this chapter.

Chapter 16 introduced configuration as architectural structure. Chapter 17 showed the anatomy of that structure. This chapter treats configuration, generated command lines, logging, connection identity, counters, and protocol-level decisions as parts of one diagnostic surface.

SNode.C is event-driven. Connections are accepted or established later. Callbacks fire later.

Timers expire later. Retries may happen later. Contexts react to peer data later.

That makes runtime visibility essential.

The source code may explain what can happen, but diagnostics show what did happen in one concrete run. They show which configured roles entered the runtime, which endpoints were used, which connection episodes existed, which system boundaries failed, which retries were scheduled, and which protocol decisions were made by a context.

18.2 Runtime visibility as a diagnostic map

Runtime visibility in SNode.C is broader than printing log lines.

A useful diagnostic map has four parts:

Visibility source	What it answers
configuration visibility	What shape did the application start with?

Visibility source	What it answers
log visibility	Which lifecycle events, decisions, and failures occurred?
connection visibility	Which peer, address, counters, and durations are involved?
protocol visibility	What did the protocol endpoint decide?

These sources belong together. A surprising runtime behavior may come from protocol code.

It may also come from configuration, disabled instances, address selection, retry behavior, TLS setup, platform state, or peer lifecycle.

A good diagnostic style therefore does not start with random print statements. It asks where the relevant responsibility lives.

Configuration visibility

Configuration visibility answers questions such as:

- Which named instances exist?
- Which instances are enabled?
- Which local or remote endpoint values are active?
- Which retry or timeout values are in effect?
- Which TLS settings are configured?
- Which log destination and output mode are selected?
- Which command line would reproduce this run?

Chapter 17 explained the configuration hierarchy. In this chapter, the same mechanisms are treated as diagnostic artifacts. A runtime surprise is often a configuration surprise first.

If a server listens on an unexpected endpoint, if a client connects to the wrong peer, if retry behaves differently from what the operator expected, or if TLS setup fails because a path is missing, the first useful diagnostic question is not always about protocol code. It is often about the effective configured shape.

Log visibility

Log visibility answers questions such as:

- Which important lifecycle events occurred?
- Which warnings or errors happened?
- Which system call or system boundary failed?
- Which retry was scheduled?
- Which instance started listening or connecting?
- Which connection was created, became ready, or disconnected?
- Which protocol event was significant enough to record?

The logger gives those events a controlled output path.

The important point is not that messages are printed. It is that messages can be placed at the layer that understands their meaning.

Connection visibility

Connection visibility answers questions such as:

- Which instance owns this connection?
- What is the connection name?
- What are the bind, local, and remote addresses?
- How much data was queued?
- How much data was sent?
- How much data was read?

- How much data was processed?
- When did the connection become active?
- How long did the connection live?

This information belongs to the connection model, not to arbitrary application-side bookkeeping.

A connection episode should be diagnosable as a concrete peer relationship. Generic messages such as `connection closed` are rarely enough for that.

Protocol visibility

Protocol visibility answers questions such as:

- Which protocol state was entered?
- Was the first message sent?
- Was a frame accepted or rejected?
- Why did the protocol close the connection?
- Which application-level event was produced?
- Which input was intentionally left unprocessed?

This belongs in the `SocketContext`, because the context is where protocol behavior lives. The connection layer can say that bytes arrived. The context can say what those bytes meant.

18.3 The logging surface: ordinary logs, system-error logs, and verbose depth

`SNode.C` exposes three main logging forms:

Form	Purpose
<code>LOG(level)</code>	ordinary runtime reporting through the normal log-level ladder
<code>PLOG(level)</code>	ordinary runtime reporting plus captured system-error context
<code>VLOG(level)</code>	optional diagnostic depth controlled by the verbose level

These forms express different kinds of visibility and should not be used interchangeably. The distinction is architectural, not only syntactic.

`LOG(level)`

`LOG(level)` is for ordinary runtime reporting.

The level describes the operational importance or severity of the message.

A useful interpretation is:

Level	Typical use
<code>FATAL</code>	unrecoverable framework or application condition
<code>ERROR</code>	failed operation or serious runtime problem
<code>WARNING</code>	unexpected but recoverable condition
<code>INFO</code>	important normal lifecycle or operational fact
<code>DEBUG</code>	developer-facing state change or decision
<code>TRACE</code>	very low-level ordinary lifecycle or state reporting

`TRACE` is still part of the ordinary log-level ladder. It is not the same thing as `VLOG`.

A `TRACE` message is a low-level normal log event. A `VLOG` message is diagnostic detail selected by verbose depth.

That distinction keeps the ordinary log level focused on severity and lifecycle, while verbose depth can be raised only when a run needs deeper inspection.

`PLOG(level)`

`PLOG(level)` follows the ordinary log-level filtering path, but it also records captured system-error context. It is useful when `errno` or comparable system-boundary context is part of the diagnosis.

Typical cases include:

- bind failures,
- connect failures,
- accept failures,
- read or write errors,
- descriptor failures,
- filesystem or path problems,
- permission problems,
- other system-boundary failures where the platform error text explains the failure.

`PLOG` should not be used merely because a message is an error. It should be used when the system error context is part of the diagnosis.

A useful rule is:

Use `PLOG` when captured system-error context helps explain the failure.

This is narrower and more precise than “use `PLOG` for all errors.” Some errors are protocol errors, configuration errors, or application decisions. Those may deserve `LOG(ERROR)` or a protocol-specific log message, but not necessarily `PLOG`.

`VLOG(level)`

`VLOG(level)` is for optional diagnostic depth.

It is controlled by the verbose level, not by the ordinary log-level ladder.

A useful project convention is:

Verbose level	Typical use
<code>VLOG(1)</code>	common flow tracing during debugging
<code>VLOG(2)</code>	detailed lifecycle, address, and configuration diagnostics
<code>VLOG(3)</code>	counters, deltas, buffer sizes, parser details, repeated flow internals
<code>VLOG(4+)</code>	very high-volume or highly specialized internal traces

The framework enforces the numeric verbose-level threshold. The meaning assigned to each depth is a convention that makes the codebase readable.

The distinction is:

```
LOG level
-> how important or severe is this message?

VLOG level
-> how deep into the flow do we want to look?
```

This keeps ordinary logs readable while still allowing deep inspection when needed.

18.4 Severity and diagnostic depth

SNode.C uses two related visibility controls.

The ordinary log level controls operational class:

```
TRACE
DEBUG
INFO
WARNING
ERROR
FATAL
```

The verbose level controls diagnostic depth:

```
VLOG(1)
VLOG(2)
VLOG(3)
...
```

Internally, verbose messages are emitted through the logger's verbose path. For the reader and application author, the important distinction is simpler:

```
ordinary lifecycle and severity
-> LOG / PLOG

optional diagnostic depth
-> VLOG
```

This separation avoids a common logging failure mode: forcing all diagnostic detail into `DEBUG` or `TRACE` until ordinary logs become unreadable.

A low ordinary log level should be able to show the shape of a run. A higher verbose level should be able to explain the fine structure of that run. Those are different jobs.

18.5 Output modes in real deployments

Logging also has operational modes.

A network application may run in the foreground, as a daemon, with file logging, in quiet mode, with colored terminal output, or in monochrome output for files and pipes.

These are not cosmetic details. They decide whether diagnostics are usable in the environment where the program actually runs.

Mode	Purpose
foreground logging	immediate human feedback
file logging	later inspection and deployment diagnostics
quiet mode	suppress nonessential console output
color output	improve terminal readability
monochrome output	improve logs in files, pipes, and plain consoles

output mode belongs to the application shell. It is not the responsibility of every server, client, connection, or context to invent its own logging destination.

SNode.C uses a centralized logging surface so that application-level settings can control where and how messages appear. That is especially important for daemonized applications, OpenWrt deployments, services started from init systems, and long-running systems where the interesting event may happen long after startup.

The architecture matters here too:

```

application shell
  -> logging destination and output mode

communication role
  -> lifecycle facts

connection
  -> peer episode facts

context
  -> protocol meaning

```

18.6 Logging should follow responsibility boundaries

The central rule is:

Diagnostic responsibility rule

Log from the layer that has the right responsibility.

SNode.C already has clear architectural boundaries. Logging should follow them.

Boundary	Good log content
application	startup, shutdown, config file, log file, daemonization
instance	listen/connect activation, disabled state, retry scheduling
connection	addresses, lifecycle events, counters, duration
context	protocol-specific state and decisions

This avoids a common failure mode: placing all messages wherever they are convenient to print. A log line is most useful when its location matches the meaning of the message.

Figure 18.1 summarizes that responsibility map. The figure should not be read as a logging pipeline. It shows where different kinds of runtime evidence belong: the application shell explains operational setup, the instance scope explains role activation, the connection scope explains one peer episode, and the context scope explains protocol meaning.

The lower row of the figure separates logging form from architectural responsibility. `LOG(level)`, `PLOG(level)`, and `VLOG(n)` decide how a message is emitted and filtered. The application shell, instance scope, connection scope, and context scope decide what the message means.

Application-level logging

Application-level logs should describe the operational shell.

Examples include:

- configuration file selected,
- configuration display or command-line generation requested,
- logging initialized,
- daemonization started,
- process user or group applied,
- shutdown requested,
- application terminated.

These messages belong above the communication roles. They are about the executable as an operating program.

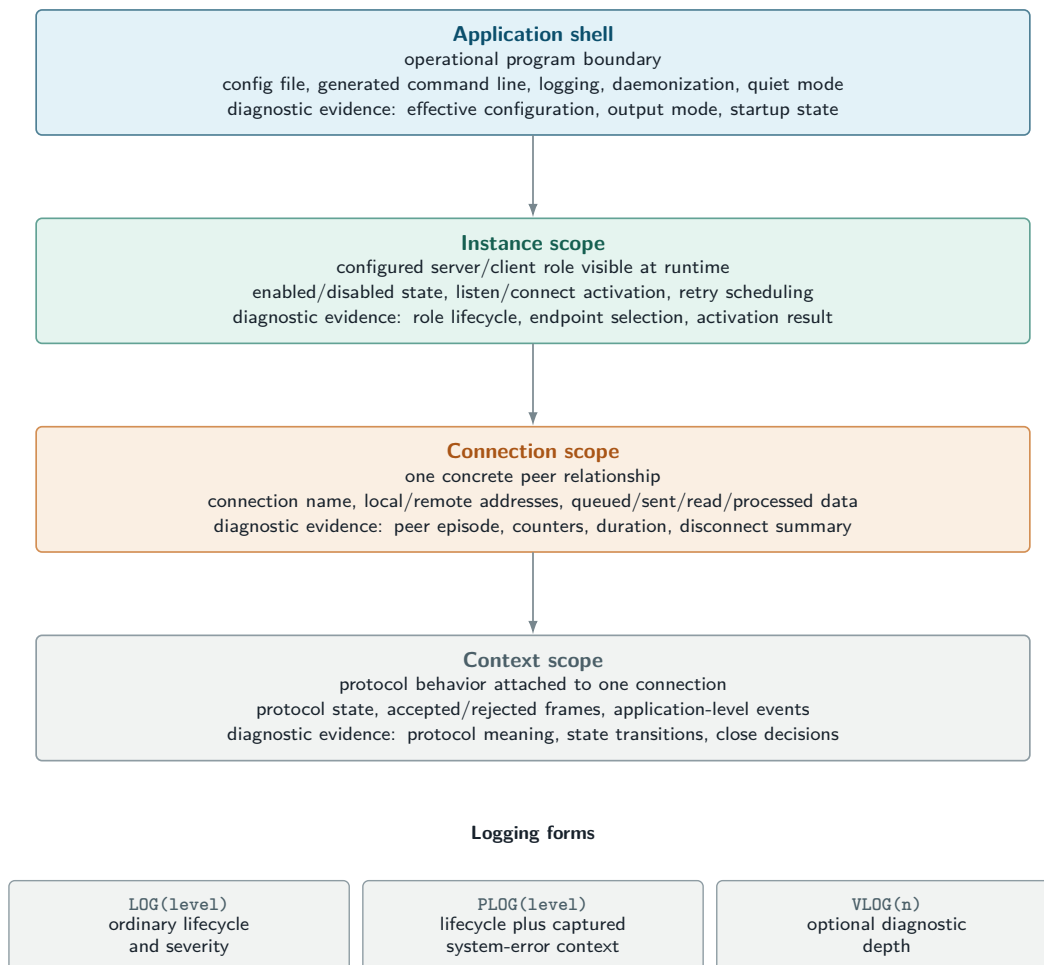


Figure 18.1: Logging and diagnostic visibility in SNode.C: application shell, instance, connection, and context each expose different runtime evidence and should log from the layer that owns the meaning.

Instance-level logging

Instance-level logs should describe server/client role activity.

Examples include:

- server starts listening,
- client starts connecting,
- retry is scheduled,
- reconnect is scheduled,
- instance is disabled,
- listen/connect fails or succeeds.

These messages belong to the configured communication role that has entered the runtime as an instance.

They should not be hidden inside protocol code, because protocol code should not have to explain why a role was activated, disabled, or retried.

Connection-level logging

Connection-level logs should describe one peer relationship.

Examples include:

- connection object created,
- connection became ready,
- peer disconnected,
- local and remote addresses,
- online duration,
- queued, sent, read, and processed totals.

These messages belong to the connection episode.

They are not application-wide facts, and they are not necessarily protocol facts. They describe the concrete relationship between this process and one peer.

Context-level logging

Context-level logs should describe protocol meaning.

Examples include:

- protocol conversation begins,
- first application message is sent,
- protocol frame accepted,
- protocol frame rejected,
- protocol state changes,
- protocol closes the session intentionally.

The context should not duplicate generic socket facts that the connection layer already knows. It should add protocol meaning.

That keeps the logs aligned with the architecture:

```
connection layer
-> connection facts

context layer
-> protocol meaning
```

18.7 Visibility at lifecycle boundaries

Lifecycle boundaries are natural diagnostic points. They correspond to meaningful transitions in the framework.

Listen and connect activation

The first useful visibility point is activation. For a server, this means listen activation. For a client, this means connect activation.

A good instance-level log can answer:

- Which instance is activating?
- Which endpoint is involved?
- Was activation successful?
- Will a retry be scheduled?
- Is the failure recoverable or fatal?

This belongs to instance-level visibility.

Activation is where configuration becomes runtime behavior. It is therefore a good place to report the configured endpoint and the resulting status, without forcing the reader to infer that from later connection events.

onConnect and onConnected

For stream connections, there is a useful distinction between early connection creation and full readiness.

A diagnostic style can reflect that distinction:

Boundary	Meaning
onConnect	connection object exists
onConnected	connection is fully ready
onDisconnect	connection episode can be summarized

onConnect and **onConnected** are good places for lifecycle visibility. They can show instance name, connection name, local address, and remote address. The amount of detail should depend on the output channel.

The lifecycle event itself can be an ordinary log message. Detailed address and state dumps can be verbose diagnostics.

This distinction is especially useful when a connection object can exist before the full protocol or connection-layer readiness is reached. A diagnostic message that preserves that difference is more valuable than a vague message that says only **connected**.

onDisconnect summaries

Disconnect is a natural summary boundary. At that point, the whole connection episode is known.

A good disconnect summary may include:

- connection name,
- local address,
- remote address,
- online since,
- online duration,
- total queued,
- total sent,
- total read,
- total processed.

The event itself is a lifecycle fact. The detailed counters and timing are diagnostic depth.

That makes **onDisconnect** a good example of the LOG / VLOG split:

```
LOG(DEBUG)
-> connection disconnected
```

```
VLOG(2)
-> addresses and duration

VLOG(3)
-> counters and deltas
```

This keeps normal debug output readable while preserving deeper information when needed.

The point is not the exact numeric verbose level chosen for every project. The important point is the separation of ordinary lifecycle facts from optional deeper evidence.

Context-level protocol events

The context is the right place for protocol-aware diagnostics. For example, an echo protocol may log when it sends the first client-side message. A WebSocket context may log when a frame is accepted or rejected.

An MQTT context may log when protocol state changes. These messages should use the vocabulary of the protocol, not the vocabulary of the socket layer.

A context log should help the reader answer:

What did the protocol endpoint decide?

not merely:

What did the socket do?

18.8 Configuration as a diagnostic source

Many runtime surprises are configuration surprises. Before assuming that protocol code is wrong, inspect the effective configuration.

Configuration diagnostic checklist

- Is the expected instance enabled?
- Is it listening on the expected local endpoint?
- Is the client connecting to the expected remote endpoint?
- Are retry, timeout, TLS, quiet-mode, and log-destination settings what the operator expects?
- Is the generated command line consistent with the intended deployment?
- Did a command-line override change a value supplied in the configuration file or in code?

Configuration display belongs in a diagnostics chapter because it shows the shape that the runtime is actually using.

The hierarchy from Chapter 17 becomes a diagnostic map:

```
application
-> instance
    -> section
        -> option
```

A wrong port, a missing TLS path, a disabled instance, a changed retry timeout, or an unexpected log file path can be understood as a value in that map.

Showing effective configuration

Showing the effective configuration answers:

What did this application actually start with?

It can reveal changed endpoint values, disabled instances, non-default retry settings, missing TLS values, or changed logging behavior.

This is diagnostic evidence, not only configuration output. It tells the reader whether the application that actually started is the application the operator thought they started.

Generated command lines

Generated command lines answer:

How can this configuration be reproduced?

They are useful for bug reports, deployment notes, comparing expected and effective configuration, sharing a minimal reproduction, or moving a working setup to another machine.

The generated command line is a textual representation of the effective configuration.

A generated command line can also expose accidental complexity. If reproducing a run requires many overrides, that may be a sign that the configuration file should be made more explicit or that defaults should be revisited.

18.9 Connection metrics and identity

The connection model carries important diagnostic information.

A useful table is:

Connection information	Diagnostic value
instance name	which configured role owns this connection
connection name	which concrete connection episode is being described
bind address	which bind-side endpoint was used
local address	which local endpoint is visible
remote address	which peer endpoint is visible
total queued	how much output was queued
total sent	how much output reached the writer
total read	how much input was read
total processed	how much input was consumed by the context
online since	when the connection became active
online duration	how long the connection lived

These values make diagnostics concrete. They also reduce guesswork.

A log line that includes the right identity and counters can explain a runtime episode much better than a generic message such as:

connection closed

A better diagnostic shape is:

```
connection closed
-> which instance?
-> which peer?
-> how long?
-> how much data?
-> was anything left queued or unprocessed?
```

The counters are not decorative. They are evidence.

If total read grows but total processed does not, the protocol endpoint may not be consuming input as expected. If data is queued but not sent, the problem may lie in write-side progress, backpressure, shutdown, or peer behavior. If a connection lives only briefly, the diagnostic question differs from a connection that stays online for hours and then fails.

The exact interpretation depends on the layer, but the values give the reader a starting point.

18.10 Runtime introspection is broader than logging

In this chapter, runtime introspection means practical runtime visibility.

It is the combined ability to inspect effective configuration, generated command lines, ordinary logs, verbose logs, system-error logs, connection identity, counters, durations, and protocol-level context decisions.

Different problems require different visibility sources. A wrong port is a configuration problem. A failed bind is a system-boundary problem.

A repeated retry is an instance-level flow problem. A peer closing unexpectedly is a connection-lifecycle problem. A rejected frame is a protocol problem.

A good diagnostic style uses the right visibility source for the problem.

Therefore, runtime introspection is broader than logging. Logs are one form of runtime evidence. Effective configuration, generated command lines, connection metrics, and context decisions are evidence too.

18.11 Too much logging is a failure mode

More logging is not automatically better diagnostics. Too much logging can make a system harder to understand. Too much logging is not visibility. It is another kind of opacity.

This happens when:

- high-frequency events flood the output,
- detailed flow internals appear at ordinary log levels,
- protocol meaning is hidden under repetitive counters,
- errors and normal flow are mixed without structure,
- every layer logs the same fact,
- verbose diagnostics are always enabled by default.

The solution is not to remove diagnostics. The solution is to place them at the right level. Use ordinary logs for important lifecycle and severity.

Use verbose logs for optional depth. Use system-error logs when captured system context matters. Use context logs for protocol meaning.

A quiet log is not necessarily a good log. A noisy log is not necessarily an informative log. The useful goal is a log stream whose messages answer real runtime questions at the layer that understands them.

18.12 A good log line answers a question

Before adding a log line, ask what runtime question it answers.

Examples:

- Which instance is active?
- Which endpoint is being used?
- Which peer connected?
- Did the connection become fully ready?
- Why is a retry scheduled?
- Which system boundary failed?
- How much data moved?
- Which protocol transition occurred?

- Which configuration state caused this behavior?
- Which layer is responsible for the decision?

If the message does not answer a useful question, it probably adds noise. This rule is simple, but it keeps logs readable.

It also helps decide where the log belongs. A message about a selected configuration file belongs at application level. A message about retry scheduling belongs at instance level. A message about queued bytes belongs at connection level. A message about an invalid MQTT packet belongs at context level.

18.13 A practical diagnostic workflow

A useful SNode.C diagnostic workflow is:

1. Inspect the effective configuration.
2. Confirm which instances exist and which are enabled.
3. Check ordinary lifecycle logs.
4. Check warnings and errors.
5. Use `PLOG` output when captured system-error context helps explain a boundary failure.
6. Increase verbose level only when more detail is needed.
7. Inspect connection identity, addresses, counters, and duration.
8. Use context-level logs for protocol meaning.

This is not a rigid law. It is a useful rhythm.

In compact form:

```
configured shape
-> ordinary lifecycle
    -> warnings, errors, and system-boundary context
        -> verbose depth
            -> connection evidence
                -> protocol meaning
```

It starts with the configured shape, then follows the runtime behavior, and only then increases diagnostic depth.

That order matters. If the endpoint is wrong, deeper protocol logs may only produce more noise. If the ordinary lifecycle already shows a bind failure, protocol tracing will not explain it. If the connection counters show unread or unprocessed data, the relevant question may be in the context. The diagnostic workflow should follow the architecture rather than fight it.

What to remember

- Runtime visibility is essential because SNode.C applications progress through events, callbacks, timers, retries, and context reactions.
- Diagnostics should start with the configured shape: application, instance, section, and option values.
- `LOG(level)` is for ordinary runtime reporting through the log-level ladder.
- `PLOG(level)` is for ordinary runtime reporting where captured system-error context helps explain the failure.
- `VLOG(n)` is for optional diagnostic depth controlled by the verbose level.
- Good log placement follows responsibility boundaries: application, instance, connection, and context.

18.14 Closing perspective

Part V began with configuration as part of the architecture. It then looked at the detailed hierarchy of application, instance, section, and option. This chapter has shown that the same structure is also diagnostic structure.

With that diagnostic foundation in place, the book can now look at secure communication without treating it as a black box.

Part VI

Secure and Robust Communication

This part adds secure connection handling and communication-over-time behavior to the same layered model. TLS, timeout policy, retry, reconnect, shutdown, termination, and failure visibility are treated as architectural concerns, not as scattered implementation details.

The reader should keep one principle in mind: robustness belongs to the boundary that owns the behavior.

Chapter 19

TLS Across the Framework

19.1 From runtime visibility to secure connection handling

Chapter 18 closed the operational visibility part of the book. It treated configuration, generated command lines, logging, connection identity, counters, durations, and protocol decisions as diagnostic evidence.

Chapter 19 begins the next part by adding secure connection handling to the same architecture.

Security and robustness do not remove the SNode.C model. They add stricter connection-layer responsibilities, more timing-sensitive transitions, and more failure states.

TLS does not introduce a second application model. It changes the connection layer.

That is the central idea of this chapter:

TLS boundary rule

TLS changes the connection layer; it does not replace the application architecture.

A TLS-enabled SNode.C application still has:

- application-side server/client handles,
- registered runtime-visible server/client instances,
- configuration sections,
- factories,
- contexts,
- connection lifecycle,
- runtime diagnostics.

What changes is the connection handling between the lower transport and the application protocol.

That distinction matters. TLS is serious: it brings identity material, trust material, handshake behavior, shutdown behavior, close-notify semantics, timeout handling, and TLS-specific diagnostics. But those concerns have a place in the architecture. They belong to secure connection handling and its configuration. They should not be spread randomly through the protocol context merely because encryption is involved.

19.2 TLS as a connection-layer specialization

The layered model is:

```
lower communication family
-> transport form
    -> connection handling
        -> application protocol
```

TLS belongs in the connection-handling position. It sits above the lower communication family and transport form. It sits below the application protocol.

That means:

```
IPv4 stream
  -> legacy connection handling
      -> protocol context

IPv4 stream
  -> TLS connection handling
      -> protocol context
```

The lower family still exists. The registered server/client instance still exists. The context still implements the protocol conversation.

TLS adds secure connection handling between those parts.

A more general view is:

```
lower communication family
  -> stream transport
      -> legacy or TLS connection layer
          -> SocketContext
```

Where a TLS wrapper exists for a lower family, this pattern applies. The chapter uses IPv4 examples because they are familiar, not because the architectural idea is IPv4-specific. The same connection-layer specialization can be expressed for other lower communication families where the corresponding TLS stream components are available.

The point is not that every lower family has identical deployment meaning. IPv4, IPv6, Unix domain sockets, RFCOMM, and L2CAP still have different endpoint identities and operating-system assumptions. The point is that TLS does not erase that lower-family identity. It specializes the stream connection handling above it.

Figure 19.1 shows the intended mental model. TLS is not a different application protocol and not a different lower communication family. It specializes the stream connection form. The surrounding role, address family, and socket-context structure remain recognizable. The diagram focuses on that specialization point rather than enumerating every TLS-enabled class variant.

19.3 Legacy and TLS streams as neighboring connection variants

A compact comparison makes the teaching point visible.

Concern	Legacy stream	TLS stream
application-side handle	selects a lower-family stream role	selects a lower-family TLS stream role
registered instance	same runtime-visible role model	same runtime-visible role model
lower family	IPv4, IPv6, Unix domain sockets, Bluetooth, etc.	still present beneath TLS
connection machinery setup work	legacy reader/writer socket/connect/listen	TLS reader/writer socket/connect/listen plus SSL object setup and TLS handshake
shutdown work	socket shutdown/close	socket shutdown/close plus TLS shutdown and close-notify handling
configuration	local / remote / socket / server / connection	ordinary sections plus tls

Concern	Legacy stream	TLS stream
context behavior	protocol endpoint	often unchanged after secure readiness
diagnostics	lifecycle, counters, errors	lifecycle, counters, errors plus TLS-specific handshake, trust, and shutdown diagnostics

This table is the chapter in miniature. TLS adds real work. It does not erase the surrounding framework structure.

That is why TLS is easiest to understand after the legacy stream shape is already clear. The legacy stream shows the connection model without secure transport. The TLS stream then shows which parts are added by secure connection handling.

19.4 The TLS wrapper shape in code

The code shape confirms the model. The TLS server is not a completely separate hand-built server type. It reuses the ordinary IPv4 stream server shell and changes the connection-layer pieces.

In simplified form, the shape is:

```
using SocketServer =
    net::in::stream::SocketServer<
        core::socket::stream::tls::SocketAcceptor,
        net::in::stream::tls::config::ConfigSocketServer,
        SocketContextFactoryT,
        Args...>;
```

The client has the same idea:

```
using SocketClient =
    net::in::stream::SocketClient<
        core::socket::stream::tls::SocketConnector,
        net::in::stream::tls::config::ConfigSocketClient,
        SocketContextFactoryT,
        Args...>;
```

The important parts are:

Part	Meaning
<code>net::in::stream::SocketServer</code> / <code>SocketClient</code>	existing lower-family stream server/client shell
<code>TLS SocketAcceptor</code> / <code>SocketConnector</code>	TLS-aware connection-layer creation
TLS config type	ordinary stream configuration extended with TLS settings
<code>SocketContextFactoryT</code>	the factory construction pattern from Chapter 14
context type	the per-connection protocol endpoint from Chapter 13

The application still selects a concrete server/client handle type. That type registers a server-side or client-side instance as before. The TLS specialization changes the acceptor or connector, the reader/writer behavior, and the configuration type used beneath that handle.

This is the architectural payoff. The handle/instance model remains recognizable. The connection machinery changes.

The protocol endpoint can often remain stable.

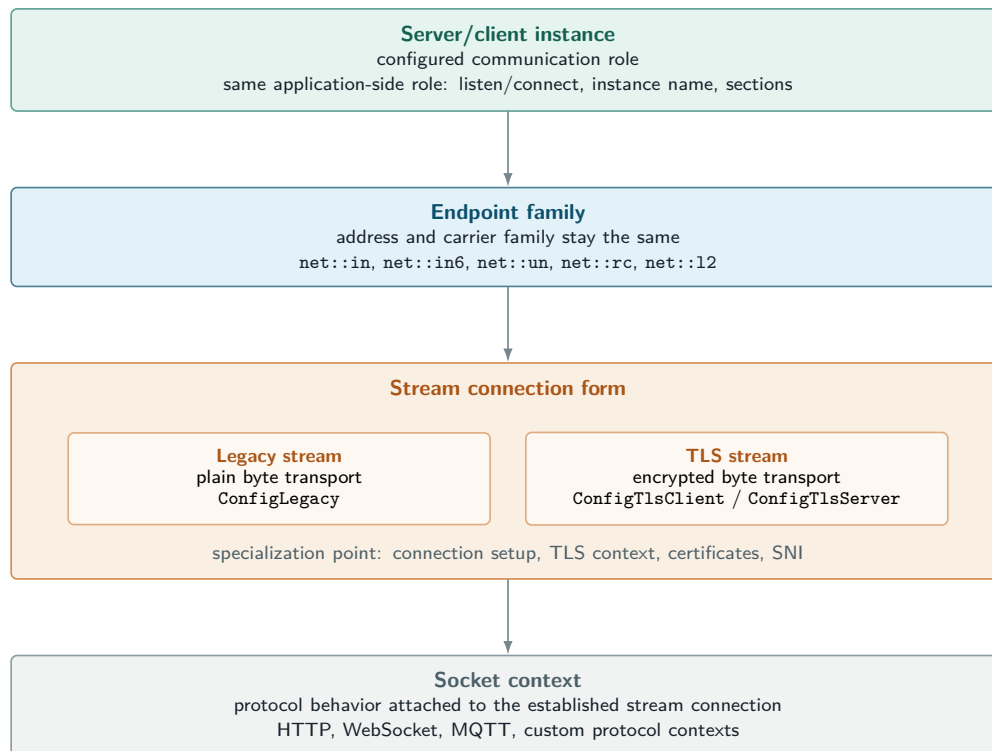


Figure 19.1: TLS as a connection-layer specialization: the lower communication family, application protocol, and socket context shape remain stable, while the stream connection is specialized from legacy byte transport to TLS-secured byte transport.

19.5 What TLS adds to the connection layer

TLS changes real parts of the communication path. It adds concerns that do not exist in a plain legacy stream.

The most important groups are:

TLS concern	Architectural meaning
identity material	certificate chain, private key, optional key password
trust material	CA certificate, CA directory, default CA directory use, accepting unknown certificates
policy	cipher list, SSL/TLS options, peer validation choices
SNI behavior	client-side SNI, server-side SNI certificate selection, optional forced SNI
timing	TLS initialization timeout and TLS shutdown timeout
shutdown semantics	close-notify handling and unexpected EOF behavior
diagnostics	handshake, trust, timeout, shutdown, and TLS-library error visibility

These are not superficial options. They are part of secure connection handling. TLS is therefore not just “turn encryption on.”

It adds identity, trust, handshake timing, shutdown semantics, and new diagnostic surfaces to the connection layer.

That is why the `tls` configuration section exists. The section is not decorative. It is the place where secure connection handling receives its configuration.

19.6 What can remain stable above TLS

TLS often leaves the higher-level application structure recognizable.

The following parts can often remain stable:

- application-side handle usage shape,
- registered instance naming,
- configuration hierarchy,
- factory pattern,
- context pattern,
- connection lifecycle vocabulary,
- protocol endpoint logic,
- use of `sendToPeer(...)`,
- use of `readFromPeer(...)`.

Therefore, TLS is teachable inside the same architecture. The reader does not need to learn a new framework model. The reader needs to understand where the secure connection layer fits.

The word *often* matters. TLS independence is not a universal law. It is a design result that holds when the protocol conversation after secure connection readiness is the same. If the protocol uses peer certificates, secure-transport properties, or SNI-derived policy as part of its own semantics, then TLS meaning may deliberately rise into the protocol layer.

19.7 The TLS connection object as the layer boundary

The TLS connection object is the clearest boundary.

It is still a stream `SocketConnection`.

But it is specialized with TLS-aware machinery:

- TLS reader,
- TLS writer,
- SSL object setup,
- SSL handshake,
- SSL shutdown,
- TLS initialization timeout,
- TLS shutdown timeout,
- close-notify handling,
- access to the underlying `SSL*`.

The connection remains the connection. The reader/writer behavior and internal lifecycle become TLS-aware. This is the right layer for encryption.

The context can still talk to the peer through the same conceptual operations. It sends data, reads data, reacts to connection readiness, and maintains protocol state. The connection layer handles the secure transport details that make that communication confidential, integrity-protected, and identity-aware.

`getSSL()` as TLS-specific access

The TLS connection exposes access to the underlying `SSL*`. That is useful, but it should be understood carefully.

`getSSL()` is not the normal protocol interface.

It is TLS-specific access at the connection boundary. It is appropriate when TLS properties genuinely matter.

Examples include:

- certificate inspection,
- auditing,
- peer identity checks,
- TLS-specific diagnostics,
- application behavior that depends on TLS details.

It should not make every ordinary protocol context TLS-dependent by default. A protocol context should care about TLS only when TLS has protocol meaning.

19.8 TLS configuration and the `tls` section

TLS configuration is added to the existing configuration model. It does not create a separate configuration universe.

The IPv4 TLS server configuration is built by layering TLS configuration on top of the normal IPv4 stream server configuration.

The IPv4 TLS client configuration follows the same pattern for clients.

Conceptually:

```
family-specific stream configuration
+ TLS configuration
  -> TLS-enabled stream configuration
```

This matches the section model from Chapters 16 and 17.

A TLS-enabled instance still has the ordinary sections that describe endpoint identity, socket behavior, connection behavior, and server/client role behavior.

It additionally has a `tls` section for secure connection handling.

```
instance
-> local
-> remote
-> socket
-> server/client-specific sections
-> connection
-> tls
```

The `tls` section groups TLS-specific responsibilities:

Responsibility	Representative settings
certificate identity	certificate chain, private key, key password
trust material	CA certificate, CA directory, default CA directory use
trust policy	accepting or rejecting unknown certificates
protocol/cipher policy	cipher list and SSL/TLS options
timing	initialization timeout and shutdown timeout
shutdown behavior	close-notify / EOF handling
SNI behavior	client-side SNI and server-side SNI certificate selection

This section boundary matters.

Endpoint identity stays in `local` and `remote`.

Socket and retry behavior stay in `socket`.

Server-specific listen behavior stays in server-side sections. TLS-specific security, handshake, trust, and shutdown policy belong in `tls`.

Client-side and server-side SNI

SNI appears on both sides of the TLS relationship, but with different meanings.

On the client side, SNI is a name sent during TLS setup so the peer can select an appropriate TLS identity.

On the server side, SNI is used to select certificate material for the requested name. A server-side configuration may also require SNI instead of silently falling back to a master certificate.

That distinction keeps the direction visible:

```
client side
  -> send SNI

server side
  -> select SNI certificate
  -> optionally require SNI
```

SNI is therefore not just another string option. It is part of TLS identity selection during handshake.

19.9 TLS adds work between connection creation and readiness

TLS changes the connection timeline.

A plain stream connection can often be described simply:

```
transport connection exists
  -> connection is ready
    -> protocol context works
```

TLS adds SSL setup and handshake work:

```
transport connection exists
  -> SSL object is created and configured
    -> TLS handshake starts
      -> TLS handshake succeeds
        -> secure connection is ready
          -> protocol context is attached
            -> protocol context works
```

Therefore, the lifecycle distinction between early connection creation and full readiness matters.

A useful reading is:

Boundary	Meaning
<code>onConnect</code>	connection object exists and TLS setup can begin
TLS handshake	secure connection setup is in progress
<code>onConnected</code>	connection is ready for protocol work
context <code>onConnected()</code>	protocol endpoint can begin its conversation

TLS does not make the lifecycle impossible to understand. It makes the middle part more meaningful.

The architecture remains legible because there is still a boundary between transport connection existence, TLS readiness, and protocol-context behavior.

19.10 TLS also affects shutdown

TLS also adds work at the end of a connection. A legacy stream shutdown may be mostly socket shutdown and close behavior. A TLS stream may also need TLS shutdown behavior, including close-notify handling.

A useful model is:

```
protocol wants to close
-> connection begins shutdown
    -> TLS shutdown / close-notify handling
        -> underlying socket shutdown / close
            -> disconnect summary
```

This matters because failures and timeouts can occur during shutdown as well as during setup.

A TLS-enabled connection therefore has two important TLS-sensitive phases:

```
startup
-> handshake

shutdown
-> TLS shutdown / close-notify
```

Both phases belong to the connection layer. Both phases are relevant for diagnostics.

Close-notify is not an application message. It is part of the TLS connection ending correctly. A missing or unexpected close-notify can therefore be a connection-layer diagnostic fact even when the application protocol itself has already decided to close.

This is one reason TLS diagnostics belong together with connection lifecycle visibility.

19.11 TLS does not erase the lower family

A TLS connection is still carried over a lower communication family. The lower family does not disappear just because the connection is encrypted.

The application may still need to know whether the carrier beneath TLS is:

- IPv4,
- IPv6,
- Unix domain sockets,
- Bluetooth RFCOMM,
- Bluetooth L2CAP,
- or another supported lower family with a TLS wrapper.

This affects endpoint configuration, address semantics, deployment shape, permissions, pairing/trust setup for Bluetooth carriers, local path behavior for Unix-domain sockets, and diagnostics.

TLS specializes connection handling above the lower transport. It does not replace the lower transport. That is why the earlier lower-family chapters still matter.

The chapter uses IPv4 code shapes because they are compact and familiar. The architectural idea is broader: lower family first, stream transport next, legacy or TLS connection handling above that, and protocol behavior in the context.

19.12 Protocol contexts should stay TLS-independent when possible

A `SocketContext` should usually describe the application protocol. It should not become TLS-heavy unless TLS has protocol meaning. For many protocols, the conversation is the same after the secure connection is ready.

The context still needs to:

- react to `onConnected()`,
- read from the peer,
- send to the peer,
- maintain protocol state,
- respond to disconnects.

That means the same protocol endpoint may often work over legacy and TLS streams.

The application-side handle type, registered instance, connection-layer wrapper, and TLS configuration decide whether the connection is secure.

The context implements the conversation. A context can stay TLS-independent when the protocol conversation after connection readiness is the same.

When protocol logic should care about TLS

There are also honest cases where higher-level logic should care about TLS.

Examples include:

- protocols whose identity model depends on peer certificates,
- mTLS-style authorization,
- applications that audit TLS properties,
- applications that adapt behavior depending on secure versus insecure transport,
- systems that expose certificate or handshake details to higher layers,
- protocols that bind authorization to TLS peer identity,
- SNI-derived behavior that is part of application policy.

In those cases, TLS-specific meaning may rise into protocol logic. But it should rise deliberately. Do not make every ordinary context TLS-aware just because the transport is TLS.

19.13 TLS diagnostics through the Chapter 18 lens

TLS makes diagnostics more important. A plain socket failure is already meaningful.

A TLS failure may involve several additional questions:

- Did the lower transport connection exist?
- Was the SSL object created?
- Did the TLS handshake start?
- Did the handshake succeed?
- Was certificate material present?
- Was the peer trusted?
- Did the client send the expected SNI?
- Did the server select the expected SNI certificate?
- Did initialization timeout?
- Did shutdown timeout?
- Was close-notify received or treated as EOF?

Chapter 18's diagnostic map applies directly.

Visibility source	TLS question
configuration visibility	Which certificate, key, CA, SNI, timeout, and trust settings were configured?
log visibility	Which handshake, shutdown, trust, or timeout events occurred?
connection visibility	Which concrete connection, peer, address, duration, and counters were involved?
protocol visibility	Did TLS meaning affect protocol authorization or behavior?

Use configuration display to inspect TLS settings. Use ordinary logs for lifecycle events. Use `PLOG` or TLS-specific error reporting where system, OpenSSL, or library context matters.

Use `VLOG` for handshake, shutdown, certificate, and trust diagnostics that are too detailed for normal output. Use connection identity and counters to connect TLS events to a concrete peer relationship. TLS does not require a new diagnostic philosophy.

It makes the existing one more important.

19.14 A useful teaching path: legacy first, TLS second

The clearest way to understand TLS in SNode.C is:

```
legacy stream first
-> then TLS stream
```

First understand the legacy application:

- application-side server/client handle,
- registered instance,
- instance configuration,
- factory,
- context,
- lower-family endpoint,
- connection lifecycle.

Then introduce TLS as a connection-layer upgrade. This lets the reader see what changed and what remained stable. It also avoids presenting TLS as a completely new application architecture.

TLS is easier to understand when the non-TLS shape is already clear because the architecture itself shows where the additional secure-connection work is inserted.

19.15 A rule of thumb for TLS-capable applications

A useful rule is:

Protocol continuity rule

Write the protocol endpoint as if secure and insecure transport are the same conversation whenever that is actually true.

Then let:

- the application-side handle type,
- the registered instance and its configuration,
- the connection-layer wrapper,
- the `tls` section,
- and the diagnostics

carry the secure-transport differences. Only promote TLS details into protocol logic when the protocol semantics require it. This keeps application code cleaner.

It also keeps the architecture readable.

19.16 TLS capability and deployment responsibility

TLS-capable components make secure connection handling possible; they do not by themselves complete a secure deployment. Certificate files, private-key permissions, trust anchors, SNI policy, and diagnostics still belong to the deployed role and its configuration.

That distinction will matter again in the deployment chapters. A binary may be linked with TLS support and still fail as a secure service if the surrounding filesystem, permissions, and trust material are wrong. TLS-capable linking is not the same as TLS deployment.

What to remember

- TLS is a connection-layer specialization, not a second application architecture.
- The lower communication family and transport form remain present beneath TLS.
- TLS changes reader/writer behavior, SSL object setup, handshake, shutdown, close-notify handling, certificate/trust configuration, and diagnostics.
- The application-side handle and registered instance model remain recognizable; the TLS wrapper changes the connection-layer machinery.
- A `SocketContext` can often remain TLS-independent when the protocol conversation is the same after secure connection setup.
- TLS-specific meaning should enter protocol logic only when certificate, trust, SNI, or secure-transport properties are part of the protocol semantics.

19.17 Closing perspective

Chapter 19 showed how secure connection handling fits into the same architecture. TLS is serious.

The next chapter generalizes this view into timeouts, retries, and failure modes across the framework.

Chapter 20

Timeouts, Retries, and Failure Modes

20.1 Communication over time

Chapter 19 showed that TLS adds meaningful connection-layer phases to the same SNode.C architecture. A secure stream may have to create and configure an SSL object, complete a TLS handshake, handle close-notify behavior, and shut the secure layer down before the underlying socket episode is fully over.

Chapter 20 widens that view from TLS to the whole framework.

Communication is not a single action. It unfolds over time. A configured communication role may be activated. The role may be registered as a runtime-visible server or client instance. A connection may be established below that instance. A peer may become ready. A protocol context may exchange data. A write may stall. A read may time out. A connection may close. A client instance may reconnect. A failed activation attempt may be retried. A role-level flow may be stopped.

These are not side cases. They are part of the normal shape of networked software.

The central model of this chapter is:

```
configured role
-> registered instance
    -> activation
        -> establishment
            -> connected operation
                -> interruption / timeout / failure
                    -> retry, reconnect, shutdown, or stop
```

Timeouts, retries, reconnects, shutdown behavior, failure states, and termination are different ways of describing communication over time. They belong together in one mental model, but they must not be collapsed into one concept.

20.2 The time-and-failure map

The title of the chapter names three ideas:

```
timeouts
retries
failure modes
```

A robust SNode.C application also needs three related ideas:

```
reconnect
```

**shutdown
termination**

A useful overview is:

Concern	Question it answers	Typical owner
timeout	How long may this phase wait?	connection, context, TLS, connect attempt, or protocol logic
retry	Should a failed activation attempt be tried again?	server/client flow controller
reconnect	Should a client instance restore a relationship after disconnect?	client flow controller
failure state	What kind of outcome happened?	status/state model
disablement	Is this role intentionally inactive?	configuration/instance
shutdown	How should an active connection end?	connection, TLS, or protocol
termination / stop	Should role-level flow end instead of scheduling more work?	flow controller / runtime lifecycle

These concerns are related, but each one answers a different question.

Timeout

A timeout bounds waiting.

It answers:

How long may this phase continue without progress?

The phase may be a read, a write, a client connection attempt, TLS initialization, TLS shutdown, connection termination, or a protocol-specific waiting phase. A timeout belongs to a phase that is already in progress.

Retry

A retry belongs to a failed activation attempt.

It answers:

Should this failed listen/connect attempt be tried again later?

A server instance may retry listening. A client instance may retry connecting. Retry policy belongs to the registered server/client instance and its role-level flow-control machinery, not to arbitrary protocol-context code.

Reconnect

Reconnect belongs to the lifecycle of an ongoing client instance.

It answers:

After an established client connection has ended, should the client instance restore the relationship?

That is different from retry. Retry belongs to failed attempts. Reconnect belongs to the longer-lived lifecycle of a client instance that is expected to remain present over time.

Failure state

A failure state describes what kind of outcome occurred.

It answers:

Did this succeed, fail, fail fatally, become disabled, or carry retry-control information?

The state model gives operational outcomes more vocabulary than a Boolean result.

Shutdown

Shutdown describes how an active connection ends.

It answers:

How should this connection close?

For a plain stream, shutdown may mostly be socket shutdown and close behavior. For TLS, shutdown may also involve TLS shutdown and close-notify handling. For a protocol, shutdown may have application meaning as well.

Termination

Termination or stop is different from failure.

It answers:

Should the role-level flow end instead of scheduling more work?

A stopped flow is not automatically a broken flow. It may be the intended result of application shutdown, disablement, runtime teardown, or an explicit decision not to continue retrying or reconnecting.

20.3 Where waiting is bounded

There is no single global timeout.

Timeouts and delays appear in several parts of the communication lifecycle. The distinction between them matters:

```
timeout
  -> bounds a phase that is already in progress

delay
  -> schedules a later attempt
```

A useful map is:

Area	Meaning
connect timeout	bound a client connection-establishment attempt
read timeout	bound waiting for input
write timeout	bound stalled outgoing progress
terminate timeout	bound connection termination
retry delay	schedule a later listen/connect activation attempt
reconnect delay	schedule a later client reconnection attempt
TLS init timeout	bound secure startup / handshake progress
TLS shutdown timeout	bound secure shutdown
protocol timeout	express protocol-specific waiting

This layered timing model is useful because different waits have different meanings.

A read timeout is not the same as a retry delay. A connect timeout is not the same as reconnect scheduling. A TLS shutdown timeout is not the same as a protocol response timeout. The names matter because the responsibility boundaries matter.

Connection, read, write, and termination timeouts

At the concrete connection level, a program may need to bound:

- waiting for input,
- blocked outgoing progress,
- termination after shutdown begins,
- overall inactivity as interpreted by the protocol.

These concerns belong to the peer relationship. A protocol context may use them because the protocol often knows what kind of waiting is meaningful.

For example:

```
no frame arrived in time
outgoing data could not progress
this session remained inactive too long
shutdown did not complete in the expected phase
```

Those are connection-level timing facts with protocol consequences.

Client connect timeout

A client connection attempt may also need a time bound.

This is not the same as retry. A connect timeout bounds the attempt that is already underway. Retry policy decides whether another attempt should be scheduled after the failed attempt has produced a status.

The distinction is:

```
connect timeout
-> this attempt did not complete in time

retry policy
-> should another attempt be scheduled?
```

That separation keeps the timing model readable.

Role-level retry timing

Retry timing belongs to the registered server or client instance.

It answers:

```
When should this instance try again after a failed activation attempt?
```

For a server, that may mean retrying listen activation. For a client, that may mean retrying connect activation. This is role-level behavior because it concerns the configured communication role as a whole. It should not be hidden inside one protocol context.

Client reconnect timing

Reconnect timing belongs to a client instance that is expected to remain present over time.

It answers:

After a previously established client connection ended,
when should the client try to restore it?

It is not another word for retry. It describes the long-term lifecycle of a client instance.

TLS timing

TLS adds timed phases to the connection lifecycle. Chapter 19 described those phases:

```
TLS startup
-> handshake

TLS shutdown
-> close-notify / shutdown behavior
```

Chapter 20 treats them as part of the broader time model. TLS initialization and TLS shutdown can both need time bounds. Both can fail. Both belong to the connection layer.

Protocol-level timing

Some timeouts have protocol meaning.

Examples include:

- waiting for a response,
- waiting for the next complete frame,
- bounding an upload phase,
- bounding a download phase,
- limiting an application-level handshake,
- closing idle sessions.

A protocol timeout should encode an expectation of the protocol conversation.

For example:

If no complete frame arrives within this time,
the peer is no longer following the expected conversation.

That is meaningful.

By contrast:

Add some timeout because the network is unreliable.

is not precise enough. Timeouts improve robustness when they match the lifecycle and semantics of the protocol.

20.4 Retry and reconnect are not the same thing

Retry and reconnect are the most important distinction in this chapter. They are close enough to be confused, but different enough that the distinction matters.

Mechanism	Situation	Meaning
retry	a listen/connect activation attempt failed	try that kind of activation again later
reconnect	an established client connection ended	restore the ongoing client role

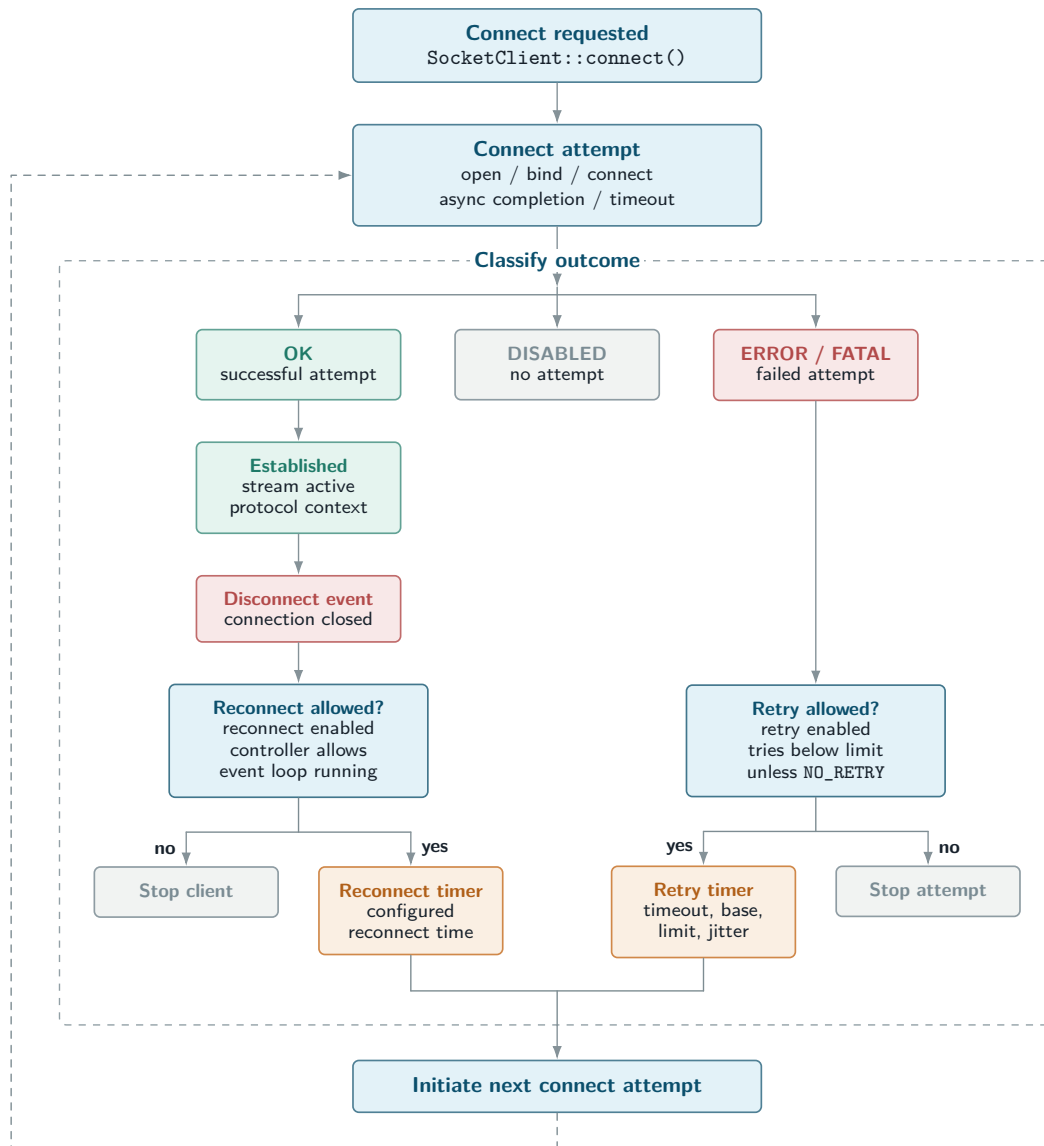


Figure 20.1: Retry and reconnect flow for a client-side stream role: failed connection attempts enter the retry path, established connections enter the reconnect path after disconnect, and both paths return through the same controlled connect initiation.

This distinction keeps the application model clear.

Figure 20.1 separates the two loops visually. Retry belongs to a failed connection attempt before a stable connection exists. Reconnect belongs to a previously established connection that later disconnects. Both paths eventually initiate another connection attempt, but they are triggered by different events and controlled by different configuration decisions. The diagram sketches the controlling decisions; it does not enumerate every socket or protocol error state.

The important point is that retry and reconnect are not synonyms. Retry reacts to classified connection-attempt failure. Reconnect reacts to connection loss after success. This distinction keeps failure handling predictable: an application can reason separately about failed startup attempts, address iteration, retry backoff, and later connection recovery.

Retry/reconnect rule

Retry belongs to failed connection attempts. Reconnect belongs to established connections that later disconnect.

Source anchor: two timer paths, one reconnecting client role

The client-side stream source in `src/core/socket/stream/SocketClient.h` keeps the two decisions in different branches of the same role-level flow. After a disconnect, reconnect policy can arm a reconnect timer and then re-enter the connect path for the ongoing client role:

```
if (config->getReconnect() &&
    sharedContext->flowController.isReconnectEnabled() &&
    core::eventLoopState() == core::State::RUNNING) {
    sharedContext->flowController.armReconnectTimer(
        relativeReconnectTimeout,
        [config, sharedContext, onStatus]() {
            sharedContext->flowController.reportFlowReconnect();
            SocketClient(config, sharedContext)
                .realConnect(onStatus, 0, config->getRetryBase());
        });
}
```

A failed connect attempt follows the retry branch instead. The status is classified, retry policy is checked, and a retry timer can schedule another activation attempt with updated retry state:

```
if (retryFlag && config->getRetry() &&
    sharedContext->flowController.isRetryEnabled() &&
    (state == core::socket::State::ERROR ||
     (state == core::socket::State::FATAL &&
      config->getRetryOnFatal())) {
    sharedContext->flowController.armRetryTimer(
        relativeRetryTimeout,
        [config, sharedContext, onStatus, tries, retryTimeoutScale]() {
            sharedContext->flowController.reportFlowRetry();
            SocketClient(config, sharedContext)
                .realConnect(onStatus, tries + 1,
                            retryTimeoutScale * config->getRetryBase());
        });
}
```

The shared call back into `realConnect(...)` is not evidence that retry and reconnect are the same concept. It shows the opposite: different lifecycle decisions can return to the same connect machinery after their own policy checks have been made.

A failed initial connect attempt is not the same situation as a client that was connected for an hour and then lost its peer. A server that cannot bind its listening endpoint is not in the same situation as a protocol context that decides to close a connection.

Server retry

A server instance may retry listen activation when configured to do so.

The retry decision belongs to the server-side instance and its flow controller. It depends on:

- whether retry is enabled,
- whether retry has not been stopped,
- whether the failure state allows retry,
- whether retry-on-fatal is enabled for fatal states,
- whether the retry count limit has been reached,
- what retry delay should be used,
- whether retry is still enabled when the timer fires.

The server does not need protocol code to reinvent this behavior. The listening instance carries retry policy.

Client retry

A client instance may retry connect activation when a connect attempt fails.

That retry path is similar in spirit to server retry:

```
connect attempt fails
-> state is reported
    -> retry policy is checked
        -> retry timer may be armed
            -> connect is attempted again
```

This still belongs to the client instance. The protocol context does not yet represent a stable peer conversation.

Client reconnect

Reconnect is different.

A client reconnect path starts after a connection existed and then ended. The client instance may still be intended to remain active. In that case, reconnect behavior can schedule a later attempt to restore the relationship:

```
connected client instance
-> disconnect
    -> reconnect policy checked
        -> reconnect timer armed
            -> connect is attempted again
```

Therefore, reconnect belongs to the lifecycle of the client instance, not to another failed connect attempt.

20.5 Server and client symmetry and difference

Servers and clients share a broad role model.

Both are configured communication roles. Both may be registered as runtime-visible instances. Both participate in the runtime. Both can use flow-control machinery. Both may retry activation.

But they are not identical.

A server instance is normally a listening role. A client instance is normally an initiating role. That difference matters for reconnect.

A server can retry listening when listen activation fails. A client can retry connecting when connect activation fails, and it can reconnect later after a previously established connection ends.

A useful summary is:

Instance kind	Retry focus	Reconnect focus
server	retry listen activation	normally not applicable
client	retry connect activation	restore client role after disconnect

This belongs to role behavior, not to application protocol behavior.

20.6 Retry timing policy

Retry timing is a policy with its own operational meaning.

A retry policy may need to answer several questions:

Setting	Question
retry	Is automatic retry enabled?
retry-on-fatal	May fatal states be retried?
retry-timeout	What is the base wait?
retry-base	How does the wait grow between attempts?
retry-limit	What is the maximum wait?
retry-jitter	How much random variation is added?
retry-tries	How many attempts are allowed?

Together, these settings prevent retry behavior from becoming an uncontrolled loop. They also let retry behavior adapt to deployment needs.

Retry-policy warning

Automatic retry can hide real failure if it is unbounded, invisible, or enabled in the wrong place. Retry is policy, not morality. It should be bounded, visible, and configurable.

Scaling and limits

Retry scaling lets repeated attempts be spaced out over time.

A retry limit can cap the maximum delay. That gives the role a bounded retry rhythm:

```
try
  -> wait
    -> try again
      -> wait longer
        -> try again
```

without allowing the delay to grow without bound.

Jitter

Jitter adds controlled variation to retry timing.

That matters when many roles or many processes may retry around the same time. Without jitter, repeated retry behavior can become synchronized. With jitter, the retry pattern becomes less rigid.

Jitter is part of retry timing policy, not a separate mechanism.

Retry tries

Retry tries bound the number of attempts.

This answers a different question from retry limit. A retry limit bounds delay growth. Retry tries bound attempt count. Both can matter.

Retry on fatal

Fatal failure does not automatically answer the retry question.

A fatal state describes severity. Retry-on-fatal describes policy. Some deployments may want a role to stop after a fatal failure. Others may want delayed reattempts even after fatal outcomes.

The framework separates the failure category from the retry policy. That separation keeps behavior configurable instead of hard-coded.

20.7 NO_RETRY as retry-control information

NO_RETRY is part of the failure-control vocabulary.

It should be understood carefully. It is not a separate ordinary outcome like OK or ERROR. It is retry-control information attached to a state.

It lets a status report remain an error or fatal condition while telling role-level retry logic not to continue automatically.

Conceptually:

```
ERROR | NO_RETRY
-> report an error
-> do not automatically retry
```

The useful distinction is:

```
what happened?
-> OK / DISABLED / ERROR / FATAL

what should retry logic do?
-> retry allowed or retry suppressed
```

NO_RETRY modifies retry policy. It does not replace the operational state that is reported to the application.

This avoids forcing every failure into one of two crude categories:

```
always retry
never retry
```

20.8 Failure states carry runtime meaning

Failure handling is easier to understand when the state vocabulary is explicit.

State	Meaning
OK	operation succeeded
DISABLED	role is intentionally inactive
ERROR	recoverable failure or ordinary error
FATAL	severe failure
NO_RETRY	retry-control flag attached to a state

This model is richer than a Boolean result.

A communication role can report that it succeeded. It can report that it is disabled. It can report a recoverable error. It can report a fatal error. It can also carry retry-control information. That matters because operational behavior depends on more than success or failure.

Disablement is not failure

DISABLED is an operational state, but not a failure state.

It represents intentional non-participation. That distinction matters in multi-instance applications. An executable may contain several possible roles, while only some are active in a particular deployment.

A disabled role should not be confused with a broken role.

The configuration says:

```
this role exists
-> but it is intentionally inactive
```

That is different from:

```
this role tried to participate
-> and failed
```

Failure can happen in different phases

Failure can occur at many points in the lifecycle.

Phase	Example failure
before activation	missing required configuration
activation	bind or connect fails
establishment	connect or TLS handshake fails
connected operation	read timeout, write timeout, peer close
shutdown	socket or TLS shutdown timeout
after disconnect	reconnect decision or reconnect failure
termination	role-level flow is stopped instead of continued

Therefore, a single error category is not enough.

Communication has phases. Failures belong to phases. The diagnostic question is broader than:

```
Did it fail?
```

The better question is:

```
Where in the lifecycle did it fail, and what policy applies there?
```

20.9 Flow controllers as role-level owners

Retry and reconnect policy belong to role-level flow control.

That is why server and client flow controllers matter. They keep outer lifecycle behavior out of ordinary protocol contexts.

A useful boundary is:

```
server/client flow controller
-> start role-level flow
-> retry timer
-> reconnect timer where supported
-> retry/reconnect enabled state
-> flow termination

connection/context
-> peer relationship
-> protocol behavior
-> protocol-meaningful timeout use
```

This prevents two design mistakes.

The first mistake is putting retry and reconnect policy into every protocol context. That makes each protocol endpoint responsible for operational behavior that belongs to the configured communication role.

The second mistake is forcing the outer role to understand protocol semantics it does not own. That makes the role-level machinery too clever and too protocol-specific.

The flow controller keeps the role coherent. The context keeps the protocol coherent.

20.10 Output pressure and bounded write-buffer policy

Timeouts, retries, and reconnects describe what happens when communication does not progress in time. Event-driven systems also need a policy for what happens when communication cannot progress in space.

The clearest example is the outgoing write buffer. A fast producer can generate data faster than a peer can receive it. A MQTT broker may fan one publication out to many subscribers. A WebSocket dashboard may have one slow browser among many fast ones. An SSE stream may remain open while the client reads slowly or stops reading altogether.

The issue is not performance alone; it is failure policy. An unbounded output queue is not robustness; it is delayed failure.

A useful SNode.C application should therefore decide which boundary owns output-pressure policy:

```
producer
  -> creates data

connection write buffer
  -> stores pending output for one peer

role/application policy
  -> decides what happens when the buffer limit is reached
```

Possible policies include:

- stop accepting more data for that peer,
- drop non-essential updates,
- disconnect the slow peer,
- report a degraded state,
- apply backpressure to the upstream producer where the application design allows it,
- or persist/defer work through a deliberate bounded queue.

The important word is *deliberate*. A bounded buffer with a visible failure policy is architecture. An unbounded buffer that grows until the process becomes unstable is not.

This topic connects Chapter 20 with later chapters on diagnostics, deployment, testing, benchmarking, and architectural judgment. A benchmark that ignores slow receivers may measure throughput but miss the real limiting boundary. A diagnostic system that cannot identify which peer is backpressured may hide the failure. A role that owns fan-out should also own the policy for what happens when fan-out cannot complete at the produced rate.

20.11 Protocol-level timeout use

Timeout controls are useful only when they express meaningful waiting.

A protocol endpoint should use timeouts for protocol reasons, such as:

- waiting for a response phase,
- guarding against peer silence,
- bounding an upload or download phase,
- limiting an application-level handshake,
- closing an idle session.

A timeout should answer a real protocol question.

For example:

```
If no complete frame arrives within this time,
the peer is no longer following the expected conversation.
```

That is meaningful.

By contrast:

Add some timeout because the network is unreliable.

is not precise enough.

The protocol timeout should encode an expectation of the protocol conversation. It should not become a substitute for understanding the protocol state machine.

20.12 Failure visibility

A retry or reconnect system must be observable.

The operator should be able to answer:

- which configured instance failed,
- which endpoint was involved,
- what state was reported,
- whether retry was scheduled,
- how long until the next retry,
- whether reconnect was scheduled,
- why a timeout occurred,
- whether the role was disabled rather than failing,
- whether role-level flow was stopped rather than continued.

This connects directly to Chapter 18.

The diagnostic map is:

```
configuration visibility
  -> intended retry/reconnect/timeout behavior

status callback visibility
  -> activation outcome and state

ordinary logs
  -> lifecycle events

PLOG / system / TLS diagnostics
  -> boundary failures with system or secure-transport context

VLOG
  -> deeper timing and retry decisions

connection counters and durations
  -> evidence from one peer episode

context-level protocol logs
  -> protocol meaning
```

Failure handling without visibility is difficult to operate. Retry without visibility is especially dangerous because it can turn a clear failure into a quiet loop.

20.13 The architectural boundary for failure behavior

A compact rule keeps the chapter together:

```
role-level configuration and flow controllers
  -> retry and reconnect policy

connection/context layer
  -> peer relationship and protocol-meaningful timeout use
```

```
status and logging surfaces
-> failure visibility
```

This boundary keeps the architecture balanced.

It prevents protocol code from swallowing outer operational policy. It prevents outer role logic from pretending to understand protocol semantics. It keeps failure behavior visible.

The important separation is explanatory and technical. A reader should be able to answer:

```
Is this an activation problem?
Is this a connection problem?
Is this a protocol problem?
Is this a retry/reconnect policy decision?
Is this intentional disablement or termination?
```

When the answer is clear, the system is easier to operate and easier to debug.

What to remember

- Robust communication is communication over time: activation, establishment, operation, interruption, shutdown, retry, reconnect, termination, or stop.
- Timeout, retry, reconnect, shutdown, disablement, and failure state are related but distinct concepts.
- Retry belongs to failed listen/connect activation attempts; reconnect belongs to client lifecycle after an established connection ended.
- Retry policy is role-level behavior controlled by retry, retry timeout, retry base, retry limit, retry jitter, retry tries, and retry-on-fatal settings.
- `NO_RETRY` is retry-control information attached to a state; it does not replace `ERROR`, `FATAL`, or another reported outcome.
- `DISABLED` means intentional non-participation, not failure.

20.14 Closing perspective

Chapter 20 completed the foundation for robust communication over time.

Once time, failure, retry, reconnect, and shutdown are understood at the stream layer, HTTP and WebSocket can be introduced as protocol-specific structures on top of the same event-driven timing and failure model. Chapter 21 begins that move with HTTP.

Part VII

Web Protocols and Web Applications

This part raises the communication model into HTTP, Express-like routing, Server-Sent Events, WebSocket upgrade, and WebSocket subprotocols. It shows how SNode.C supports web-facing protocol layers without erasing the lower connection model.

The web stack is presented as a sequence of boundaries, not as one monolithic abstraction.

Chapter 21

The HTTP Layer

21.1 From robust streams to HTTP messages

Chapter 20 completed the foundation for robust communication over time. It described activation, establishment, connected operation, interruption, timeout, retry, reconnect, shutdown, and failure state. That foundation remains in place as the book moves into Part VII.

Part VI established robust stream communication: TLS, timing, retry, reconnect, shutdown, and failure visibility. Part VII keeps that lower architecture in place and moves upward into web protocols. Chapter 21 begins that move with HTTP.

At the stream level, application code often works close to received data and protocol-specific byte interpretation. At the HTTP level, the application-facing unit changes:

```
request  
-> response
```

That is the central idea of this chapter:

HTTP layer note

HTTP raises the application-facing meaning from stream data to request and response objects without replacing the lower SNode.C architecture.

An HTTP server or client still depends on:

- the runtime,
- application-side server/client handles,
- registered runtime-visible server/client instances,
- lower communication families,
- stream transport,
- legacy or TLS connection handling,
- contexts,
- factories,
- configuration,
- diagnostics,
- timeouts and failure behavior.

HTTP does not erase those parts. It builds on them. The application sees richer message-level objects, but the lower framework model remains visible underneath.

21.2 HTTP in the layered SNode.C model

The layer model now looks like this:


```

lower communication family
  -> stream transport
      -> legacy or TLS connection handling
          -> HTTP protocol layer
              -> request / response application logic

```

The earlier chapters taught the lower part of this structure. They showed how a configured communication role becomes a registered instance, how that instance produces concrete connections, how factories create per-connection contexts, and how configuration and diagnostics make the runtime shape visible.

Chapter 21 introduces the first major web-facing protocol layer above that foundation. The transfer question therefore changes direction. Earlier chapters often asked:

Can the same application protocol move across different lower layers?

This chapter asks:

Can the same lower architecture support a richer protocol layer?

For HTTP, the answer is yes. The lower architecture stays visible. The protocol meaning becomes richer. HTTP is not a replacement for context, factory, connection, configuration, or runtime progress. It is a protocol layer implemented through HTTP-specific contexts and factories on top of the stream model.

21.3 Plain streams and HTTP messages side by side

A compact comparison shows the transition.

Concern	Plain stream layer	HTTP layer
lower family	IPv4, IPv6, Unix domain, RFCOMM where supported	still present
connection handling application-facing unit	legacy or TLS stream connection bytes or custom stream protocol data	still legacy or TLS underneath HTTP request and response
context role	custom protocol endpoint	HTTP-aware protocol endpoint
factory role	creates stream contexts	creates HTTP contexts
configuration	instance / local / remote / socket / TLS	same base plus HTTP-specific configuration where needed
diagnostics	connection lifecycle, counters, timing, and failure behavior	same plus HTTP parsing and request/response meaning
extension point	protocol code in the context	routing, SSE, WebSocket upgrade, higher web layers

This is the same architectural pattern at a higher semantic level. The pattern is the same because the framework still uses lower-family selection, connection handling, context creation, and runtime progress. The semantic level is higher because the application receives HTTP message objects instead of stream fragments.

The application is no longer forced to decide where an HTTP request begins and ends. More precisely, the HTTP layer takes responsibility for message boundary recognition, start-line and header parsing, content and trailer handling, request/response object construction, and HTTP-specific connection behavior.

21.4 Server-side HTTP: from bytes to ready requests

At the plain stream level, the protocol endpoint may react to incoming bytes. At the HTTP server level, the central application-facing event is different:

a complete HTTP request is ready

The HTTP layer consumes stream data, parses HTTP, creates request/response objects, and calls the application at the HTTP level. This changes what application code sees. It does not change the runtime model underneath.

The useful server-side picture is:

```
stream data
  -> parser state
      -> start line / headers / content / trailers
      -> Request
      -> application handler with Request and Response
```

That is the main semantic lift on the server side.

The HTTP server wrapper

The generic HTTP server wrapper has a simple shape:

```
web::http::server::Server<LowerSocketServerT>
```

The lower server template supplies the application-side handle shape and the registered server-instance machinery. The HTTP wrapper supplies the HTTP context factory and the request-ready callback shape.

In simplified form:

```
SocketServerT<
    web::http::server::SocketContextFactory,
    std::function<void(std::shared_ptr<Request>,
                      std::shared_ptr<Response>)>>>
```

This means:

Part	Meaning
SocketServerT	lower server shell used by the HTTP wrapper
web::http::server::SocketContextFactory	creates HTTP server contexts
Request	parsed HTTP request visible to the application
Response	HTTP response object visible to the application
application callback	runs when HTTP request handling can begin

A concrete IPv4 legacy HTTP server is then one specialization of this idea:

```
using Server =
    web::http::server::Server<
        net::in::stream::legacy::SocketServer>;
```

The lower layer is still present. The HTTP layer plugs into it.

Request and response as the application-facing unit

The server-side application callback receives HTTP objects, not raw transport data.

Conceptually:

```
stream data arrives
  -> HTTP parser consumes stream data
      -> HTTP request becomes available
      -> application receives Request and Response
```

The application no longer has to treat every connection as an uninterpreted stream. It can respond to HTTP meaning.

The server-side HTTP context still derives from the stream context model. It overrides stream-context lifecycle and receive behavior, but it uses that lower behavior to deliver HTTP requests. That is the right boundary:

```
stream context machinery
  -> receives data

HTTP server context
  -> interprets HTTP

application handler
  -> handles Request and Response
```

The same principle from Chapters 13 and 14 still applies: the context implements protocol behavior, and the factory creates the per-connection protocol endpoint. HTTP changes the protocol behavior implemented by the context; it does not remove the context/factory boundary.

21.5 Client-side HTTP: from connection to requests and responses

The HTTP client follows the same general pattern. It is a client-side HTTP protocol layer built on a lower client handle and a registered client instance, not a raw client that writes a manually assembled request line.

Conceptually:

```
lower client handle / registered client instance
  -> HTTP client SocketContextFactory
    -> HTTP client SocketContext
      -> MasterRequest / Request / Response
```

The lower client still establishes and maintains the connection. The HTTP layer gives the application a request/response-oriented client surface.

The HTTP client wrapper

The generic HTTP client wrapper has the shape:

```
web::http::client::Client<LowerSocketClientT>
```

It uses an HTTP client `SocketContextFactory` on top of the lower socket client template. The code shape includes:

- a lower `SocketClientT`,
- `web::http::client::SocketContextFactory`,
- `MasterRequest`,
- `Request`,
- `Response`,
- HTTP-connected and HTTP-disconnected callbacks,
- access to the instance configuration.

The important teaching point is the same as on the server side:

```
lower client handle / registered client instance
  -> remains responsible for connection behavior

HTTP client layer
  -> adds HTTP request/response meaning
```

The lower SNode.C architecture remains present. The application-facing unit changes.

MasterRequest, Request, and Response

The client side has to manage the relationship between a client connection and one or more HTTP requests and responses, rather than a single raw write.

That is why the HTTP client vocabulary contains:

Concept	Role
MasterRequest	client-side coordination object associated with the HTTP connection
Request	concrete HTTP request
Response	concrete HTTP response

For this chapter, **MasterRequest** can be read as the client-side coordination object associated with the HTTP connection. It owns the sending path for concrete requests and delivers responses or parse errors through callbacks. The exact internal mechanics belong to the implementation and reference documentation; the important point here is the semantic level. The client is now expressed in HTTP terms.

HTTP-specific configuration

The HTTP client adds an HTTP-specific configuration subcommand to the existing instance configuration hierarchy. That subcommand is named:

```
http
```

It contains HTTP behavior such as:

```
--host
--pipelined-requests
```

This continues the configuration model from Chapters 16 and 17. HTTP-specific configuration is not a random side channel. It is attached to the configured communication role.

The HTTP layer can also derive HTTP meaning from lower connection configuration where appropriate. For example, if the HTTP Host header is empty, the client setup can derive a default Host header from the remote socket address. That is a small but useful example of responsibility placement:

```
remote endpoint
  -> belongs to lower configuration

Host header
  -> belongs to HTTP configuration

HTTP client layer
  -> may derive the HTTP default from the lower endpoint
```

The application does not need to repeat that adaptation in every client.

21.6 What HTTP adds above the stream layer

The HTTP layer contains the machinery needed to turn stream communication into HTTP message semantics, in addition to server and client aliases.

Added concern	Meaning
parser	turns stream data into HTTP message structure
content decoder	handles transfer/content decoding
request/response types	expose HTTP meaning to application code
status and header utilities	model HTTP metadata
MIME handling	supports content-type decisions for served content
upgrade machinery	allows HTTP to negotiate another protocol
EventSource support	builds streaming-style behavior on HTTP

Therefore, HTTP is a real protocol layer in SNode.C, not a convenience function for writing a few text lines to a socket.

Parsing and decoding

HTTP arrives over a stream. A stream does not know HTTP message boundaries by itself. The HTTP layer therefore needs parsing and decoding machinery. That includes concerns such as:

- request parsing,
- start-line and header field processing,
- content and trailer reading,
- transfer-encoding handling,
- identity decoding,
- HTTP/1.0 response decoding,
- content decoding.

These are HTTP-layer responsibilities. They are what allow the application-facing server callback to receive a `Request` and `Response` instead of raw stream fragments.

MIME handling

MIME handling is practical HTTP support for serving content. A server often needs to associate a file or resource with a content type. SNode.C includes MIME support and can use libmagic when available for better type detection.

This belongs in the HTTP layer because content type is HTTP meaning. It should not be mixed into the lower socket layer. At the same time, MIME handling is not the central architectural transition of this chapter. The central transition is still:

```
stream data
-> HTTP request/response meaning
```

Status, headers, and utilities

HTTP also needs ordinary protocol utilities:

- status code handling,
- header and field helpers,
- content-type representation,
- case-insensitive HTTP maps,
- utility functions for HTTP syntax and behavior.

These pieces are part of the protocol layer. They keep HTTP concerns grouped with HTTP rather than spreading them through application code.

21.7 Lower families and connection handling still matter

Using HTTP does not make the lower carrier disappear. In SNode.C, HTTP remains above a selected communication family and connection mode:

```
lower communication family
  -> stream
      -> legacy or TLS
      -> HTTP
```

That is enough to prevent the main misconception. HTTP is the higher protocol layer; IPv4, IPv6, Unix-domain sockets, Bluetooth families where available, and legacy or TLS connection handling still define how the peer relationship is carried.

21.8 HTTP as a bridge to higher web protocols

HTTP is also a bridge. It can become the place where an application moves upward into more specialized web behavior.

Two examples are especially important for the next chapters:

- upgrade support,
- EventSource support.

Upgrade support

HTTP upgrade support belongs in the HTTP layer because HTTP is where the upgrade decision is negotiated. The upgraded protocol may later be WebSocket, but the boundary itself is not WebSocket-specific. An HTTP request names an upgrade target, the HTTP layer selects a socket-context upgrade factory for that name, and the selected upgraded context takes over the same connection episode after the HTTP response confirms the transition.

Conceptually:

```
HTTP request/response layer
  -> upgrade decision
      -> selected SocketContextUpgrade
          -> another protocol layer may take over
```

This is a clean architectural boundary. The lower connection remains the same peer episode. What changes is the protocol context attached to it.

A compact server-side upgrade route has this shape. The example uses `websocket` because that is the concrete upgrade protocol used in the following WebSocket chapter, but the call itself belongs to the HTTP/Express boundary:

```
#include <express/legacy/in/WebApp.h>

#include <memory>
#include <string>

using WebApp = express::legacy::in::WebApp;
using Request = WebApp::Request;
using Response = WebApp::Response;

WebApp app("legacy");

app.get("/ws", [](const std::shared_ptr<Request>& req,
                  const std::shared_ptr<Response>& res) {
    res->upgrade(req, [res](const std::string& selected) {
        if (!selected.empty()) {
            res->end();
        } else {
            res->sendStatus(404);
        }
    });
});
```

```
});
```

The HTTP route is still visible at the boundary. After a successful upgrade, the selected upgraded socket context owns the connection episode. The route does not become the message loop of the upgraded protocol.

On the client side, the HTTP client prepares an upgrade request and names the target upgrade protocol. For WebSocket that target name is `websocket`:

```
req->upgrade(
    "/ws",
    "websocket",
    [](bool success) {
        VLOG(1) << "upgrade request initiation: "
            << (success ? "accepted" : "rejected");
    },
    [](const std::shared_ptr<Request>&,
        const std::shared_ptr<Response>&,
        bool success) {
        VLOG(1) << "upgrade response: "
            << (success ? "accepted" : "rejected");
    },
    [](const std::shared_ptr<Request>&,
        const std::string& message) {
        LOG(ERROR) << "upgrade response parse error: " << message;
    });
```

The important point is not the particular upgraded protocol yet. The important point is that the HTTP layer supplies an explicit transition from HTTP request/response handling into a named socket-context upgrade.

From this chapter onward, some examples are printed as compact fragments so that the book can focus on the architectural idea being discussed. Complete buildable source versions are part of the book's electronic companion material; the published edition should make them available through its companion repository or download page. For this HTTP-upgrade example, the corresponding companion programs are `HttpUpgrade-Server` and `HttpUpgrade-Client`.

HTTP-upgrade deployment contract

Most `SNode.C` components follow the ordinary C++ library rule: source files include the public headers they use, the application links the corresponding component, and the installed libraries must be available to the platform loader at runtime. The book does not repeat that ordinary deployment rule for every component. HTTP upgrade is different because the upgrade name is also a runtime selection key.

`SNode.C` can resolve an HTTP upgrade factory through a linked registration path or by loading a role-specific shared object at runtime. The common dynamic deployment contract is compact:

```
HTTP upgrade directory:
    ${CMAKE_INSTALL_PREFIX}/${CMAKE_INSTALL_LIBDIR}/snode.c/web/http/upgrade

server-side module:
    libsnodec-<upgrade-name>-server.so.<SOVERSION>

client-side module:
    libsnodec-<upgrade-name>-client.so.<SOVERSION>

server-side factory symbol:
    <upgrade-name>ServerSocketContextUpgradeFactory

client-side factory symbol:
    <upgrade-name>ClientSocketContextUpgradeFactory
```

For the WebSocket upgrade, the HTTP upgrade name is `websocket`, so the dynamically loaded artifacts are named accordingly:

```
libsnodec-websocket-server.so.<SOVERSION>
libsnodec-websocket-client.so.<SOVERSION>

websocketServerSocketContextUpgradeFactory
websocketClientSocketContextUpgradeFactory
```

A linked deployment uses the same upgrade name but resolves it through the selector's linked-factory cache instead of opening the shared object later. For an application that uses the installed WebSocket upgrade components, the build-time shape is to link the HTTP role and the matching WebSocket upgrade role into the executable or into an application-loaded library:

```
target_link_libraries(my_ws_server PRIVATE
    snodec::http-server-express-legacy-in
    snodec::websocket-server
)

target_link_libraries(my_ws_client PRIVATE
    snodec::http-client
    snodec::net-in-stream-legacy
    snodec::websocket-client
)
```

The operational rule is simple: dynamic deployment needs the correctly named module in the HTTP upgrade directory; linked deployment needs the factory registration object to be linked and retained. Both paths must make the same upgrade name resolvable at the HTTP boundary.

EventSource and streaming-style HTTP

EventSource support shows that HTTP is not limited to short request/response exchanges. The response remains HTTP-based, but the application behavior becomes stream-like:

```
HTTP response stream
-> Server-Sent Events / EventSource behavior
```

Chapter 23 treats Server-Sent Events in detail. Here, the important point is placement: EventSource belongs naturally near HTTP because it uses HTTP semantics for streaming-style behavior.

21.9 What remains from the lower architecture

An HTTP server still has the same lower architectural skeleton as the stream examples: configured roles, registered instances, lower-family stream connections, context factories, runtime lifecycle, configuration, diagnostics, timing, and failure behavior. What changes is the semantic level of the application handler. The echo-style context interprets stream data directly; the HTTP handler receives request and response objects.

The same transfer applies to clients. An HTTP client still depends on endpoint configuration, connection establishment, lifecycle callbacks, and runtime integration, but the application now works in HTTP terms.

Diagnostics also become layered rather than replaced. Lower diagnostics explain connection lifecycle, counters, timeouts, retry, reconnect, and shutdown. HTTP adds parse errors, request boundaries, response completion, upgrade decisions, and streaming state.

21.10 From HTTP support to web application structure

Chapter 21 is a bridge between two parts of the book. Earlier chapters taught:


```
runtime
  -> lower families
    -> stream connections
      -> contexts and factories
        -> configuration and diagnostics
          -> TLS and robust failure behavior
```

This chapter raises that structure to HTTP. Later chapters move higher again:

```
HTTP
  -> Express-like routing and middleware
  -> Server-Sent Events
  -> WebSocket upgrade
```

That is why Chapter 21 should not become a complete HTTP reference. Its job is to show where HTTP sits in SNode.C. It teaches the architectural transition from streams to HTTP messages.

Chapter 21 explains how HTTP becomes request/response semantics. Chapter 22 asks how larger HTTP applications organize those request/response handlers into routing, middleware, and application structure.

What to remember

- HTTP is a protocol layer above the stream connection model.
- HTTP raises the application-facing unit from stream data to request and response objects.
- The lower SNode.C architecture remains present: runtime, configured roles, registered instances, connections, contexts, factories, configuration, diagnostics, timing, and failure behavior.
- HTTP server wrappers plug an HTTP context factory and request-ready callback into lower server shells.
- HTTP client wrappers plug an HTTP context factory, HTTP connection callbacks, and `MasterRequest` coordination into lower client shells.
- HTTP-specific client configuration lives in the `http` subcommand for behavior such as Host header and pipelining.

21.11 HTTP public surface: role headers and components

HTTP code includes the HTTP abstraction it directly names. An IPv4 legacy HTTP server uses:

```
#include <web/http/legacy/in/Server.h>
```

and the matching client role uses:

```
#include <web/http/legacy/in/Client.h>
```

It should not include a lower socket header merely because HTTP is carried by that socket stack. Chapter 32 consolidates the complete source/header and component mapping.

21.12 Closing perspective

HTTP raises stream communication to request/response semantics without removing the lower runtime, role, connection, context, configuration, and diagnostic surfaces. The next chapter moves one level higher again: the Express-like framework turns HTTP handling into routing, middleware, and application structure.

Chapter 22

The Express-Like Framework

22.1 From HTTP messages to application structure

Chapter 21 showed how `SNode.C` raises stream communication to HTTP request and response objects. Chapter 22 moves one level higher: HTTP messages are no longer handled only by an HTTP request-ready callback, but by a web-application structure made of routers, routes, middleware chains, request/response facades, and explicit continuation.

That is the central idea of this chapter:

Express-layer rule

The Express-like layer does not replace HTTP; it organizes HTTP handling into application structure.

The lower stack remains visible:

```
lower communication family
  -> stream transport
      -> legacy or TLS connection handling
          -> HTTP request / response layer
              -> Express-like application layer
```

The Express-like layer is therefore not a different universe. It is the application-organization layer above HTTP. HTTP gives message meaning; the Express-like layer gives application structure.

Part VII now moves in steps. Chapter 21 introduced HTTP messages. Chapter 22 shows how those messages become routed application flow. Chapter 23 will keep that web-application structure, but change the temporal shape of a handler: instead of answering once, a route may keep an HTTP response open and emit events over time.

22.2 The Express-like layer in the layered `SNode.C` model

The stack now reads:

```
runtime
  -> registered server instance
      -> lower communication family
          -> stream transport
              -> legacy or TLS connection handling
                  -> HTTP request / response
                      -> Express-like routing and middleware
```

The visible `WebAppT<ServerT>` object in application code is the application-side handle. Through that handle, the application configures a server-side communication role and registers a runtime-visible server

instance. The Express-like layer does not change that model; it changes what happens when a ready HTTP request reaches application code.

Earlier chapters established the lower parts. Chapter 21 introduced HTTP request and response semantics. Chapter 22 now asks what happens when HTTP handling becomes application structure. The answer is:

```
HTTP request ready
-> Controller(req, res)
    -> root route dispatch
        -> router / middleware / application handler
```

This is the transition from protocol handling to web-application composition.

22.3 HTTP messages and Express-like application flow side by side

A compact comparison makes the new layer visible.

Concern	HTTP layer	Express-like layer
application-facing unit	request / response	routed request / response flow
primary callback	request ready	route handler or middleware
structure	one HTTP protocol endpoint	application route tree
flow control	HTTP lifecycle	dispatcher chain and <code>next()</code>
path meaning	HTTP target/path	mount path, route path, params, base/original URL
response surface	HTTP response	application-oriented response facade
reuse mechanism	HTTP server/client wrappers	routers and middleware
lower layers	still present	still present underneath HTTP

The HTTP layer gives the application request and response objects. The Express-like layer gives the application a way to organize what should happen with them.

22.4 What “Express-like” means here

“Express-like” does not mean that SNode.C embeds Node.js or the JavaScript Express runtime. It means that the C++ web-application layer uses a familiar application model:

- `WebApp`,
- `Router`,
- `.use()`,
- `.get()`, `.post()`, and other HTTP verb methods,
- middleware callbacks,
- `Next`,
- mounted routers,
- request and response facades,
- built-in middleware.

The layer is Express-like in its programming model. It is still SNode.C in its runtime, connection, configuration, diagnostics, and lower-layer architecture.

This distinction matters because the familiar surface should not hide the underlying model. A SNode.C Express-like application is still carried by an application-side web-app/server handle, a registered server instance, an HTTP server layer, legacy or TLS connection handling, lower-family choices, configuration, diagnostics, timing, and failure behavior.

22.5 WebApp, WebAppT, and the HTTP server below

The core composition is visible in two steps.

First, `WebApp` is router-shaped:

```
class WebApp : public Router {
    // runtime-facing lifecycle surface
};
```

Second, `WebAppT` joins that application shape with a concrete HTTP server type:

```
template <typename ServerT>
class WebAppT
    : public WebApp
    , public ServerT {
    // bridge from HTTP request readiness to Express dispatch
};
```

This composition explains how HTTP request handling becomes Express-like application flow.

Type	Role
Router	route tree and middleware structure
WebApp	router-shaped application plus runtime-facing lifecycle
ServerT	concrete HTTP server type underneath
WebAppT<ServerT>	combined Express-like application surface and HTTP server handle

The result is not a separate web runtime. It is a web-application surface joined to a concrete HTTP server.

WebApp as router-shaped application

`WebApp` derives from `Router`. Because `WebApp` is router-shaped, the application root is not a separate routing concept. The application itself is the root router, and `WebAppT` makes that root router runnable by combining it with a concrete HTTP server.

`WebApp` also exposes runtime-facing lifecycle operations such as:

- `init(...)`,
- `start(...)`,
- `stop()`,
- `tick(...)`,
- `free()`,
- `state()`.

This keeps the web-application layer connected to the same event-driven runtime story introduced earlier in the book. A web application is a route structure that still runs inside the `SNode.C` runtime, not a loose set of route functions.

WebAppT as the bridge to a concrete HTTP server

`WebAppT<ServerT>` is the joining point. It inherits the router-shaped `WebApp` surface and the concrete HTTP server type. When the HTTP server reports a ready request, `WebAppT` wraps that request/response pair in a `Controller` and dispatches it into the root route.

The concrete alias is not the main point; it is an example of the general pattern. An IPv4 legacy Express-like web application can be shaped as:

```
using WebApp =
    WebAppT<web::http::legacy::in::Server>;
```

The SNode.C source also provides the corresponding convenience alias:

```
express::legacy::in::WebApp
```

The same idea applies to other lower-family and connection-handling variants where provided.

The pattern is:

```
Express-like WebAppT
+ concrete HTTP server
-> runnable web application
```

This matches the design pattern from Chapter 21. The HTTP layer remains underneath; the Express-like layer organizes application behavior above it.

From HTTP request readiness to Express dispatch

The key bridge is the moment when HTTP request readiness enters the Express-like routing structure.

Conceptually:

```
HTTP request ready
-> Controller(req, res)
    -> root route dispatch
        -> application callback / middleware / mounted router
```

The HTTP layer has parsed the request and prepared a response object. The Express-like layer now decides which application structure should handle that request. That decision belongs above HTTP because it depends on route paths, mounted routers, middleware, and routing policy.

The **Controller** is the dispatch-time object that carries the Express request/response facades through the route tree and tracks continuation state such as **next**, next route, and next router.

22.6 Router as the application composition unit

Router is the main composition unit of the Express-like layer. It lets a web application be assembled from parts instead of being written as one large request callback.

A router can contribute:

- route handlers,
- middleware,
- mounted routers,
- method-specific behavior,
- routing policy.

The routing surface includes familiar methods such as:

- `.use()`,
- `.all()`,
- `.get()`,
- `.put()`,
- `.post()`,
- `.del()`,
- `.connect()`,
- `.options()`,
- `.trace()`,

- `.patch()`,
- `.head()`.

The method list matters because each method attaches structure to the route tree. The point is not that there are many methods; it is that routing, middleware, and mounted routers become composable:

```
application
  -> router
    -> route
    -> middleware
    -> mounted router
```

Routes, mounted routers, and middleware

A route is a point where request properties and application behavior meet. A mounted router lets a larger application be built from smaller route structures. Middleware lets common behavior be placed into the request flow without duplicating it in every handler.

Together, these ideas move the application from:

```
one request callback
```

to:

```
structured request processing
```

That is the main application-level gain of the Express-like layer.

Routing policy

At this layer, route matching is part of application correctness. The router exposes policy controls such as:

Policy	Question
strict routing	Are <code>/x</code> and <code>/x/</code> distinct?
case-insensitive routing	Should route matching ignore case?
merge params	Should mounted routers receive parent params?

These policies belong to the router. They are not socket concerns and not generic HTTP parsing concerns. They are web-application routing concerns.

`merge params` is a good example. It decides how parameter information moves through nested router structures. That is meaningful only once the application has routers and mount points.

22.7 Application callbacks and middleware callbacks

The Express-like layer distinguishes two important callback shapes.

Callback shape	Meaning
<code>(req, res)</code>	application handler; may produce a response
<code>(req, res, next)</code>	middleware participant; may continue the chain

This distinction teaches the layer model clearly. Not every function attached to a route has the same responsibility. Some handlers answer. Some middleware participates in a chain. Some middleware may inspect, modify, authorize, log, or prepare state before passing control onward.

Application handlers

An application handler receives a request and a response. Its typical responsibility is to produce an answer for a matched request.

Conceptually:

```
matched route
  -> application handler
      -> response
```

This is the simplest Express-like callback shape.

Middleware handlers

A middleware handler receives a request, a response, and a `Next` object. Its responsibility is not necessarily to finish the response.

It may:

- inspect the request,
- add application state,
- enforce authentication,
- log or trace,
- parse or prepare input,
- serve content,
- pass control to the next component.

Conceptually:

```
middleware
  -> maybe handle
  -> maybe modify state
  -> maybe call next()
```

Therefore, middleware should not be read as another name for a route handler.

Next and middleware flow

`Next` makes middleware continuation explicit. A middleware can decide whether the request flow should continue.

The model is:

```
middleware(req, res, next)
  -> does its work
      -> next()
          -> following middleware / router / handler
```

`Next` is not a scheduler and not a thread handoff. It is the application-visible continuation object for the dispatcher chain.

This makes flow control part of the application model. The chain is visible to the user. That is one of the main differences between a simple HTTP callback and an Express-like application framework.

22.8 Dispatchers behind the API

The user-facing API is built on internal dispatcher roles.

A compact view is:

Dispatcher	Role
<code>ApplicationDispatcher</code>	invokes application-style callbacks
<code>MiddlewareDispatcher</code>	handles middleware and <code>next()</code> flow
<code>RouterDispatcher</code>	enters mounted routers

The dispatchers encode the fact that application callbacks, middleware callbacks, and mounted routers have different control-flow meanings.

This internal structure mirrors the application model. The framework does not pretend that all web actions are the same. The dispatcher structure reflects those differences.

22.9 Request and Response as web-application facades

Chapter 21 introduced HTTP request and response objects. Chapter 22 raises them into Express-like application facades.

The facade does two things:

```
keeps access to HTTP meaning
+ adds application-structure meaning
```

That is different from replacing the HTTP objects. The Express-like layer wraps and enriches them:

```
HTTP objects
-> protocol message meaning

Express facades
-> web-application meaning
```

Request facade

The Express-like `Request` adds routing and application context to the lower HTTP request.

Request aspect	Examples
routing context	<code>baseUrl</code> , <code>path</code> , <code>file</code> , <code>params</code> , <code>param(...)</code>
original target	<code>originalUrl</code> , <code>originalPath</code>
HTTP metadata	<code>method</code> , <code>URL</code> , <code>HTTP version</code> , <code>headers</code> , <code>trailer</code>
parsed input	<code>query values</code> , <code>cookies</code> , <code>body</code>

This is the request-side semantic lift. The application can ask route-level questions, not only HTTP parsing questions:

```
Which route parameter was matched?
What was the original URL?
Which mounted base path led here?
Which cookie or query value is present?
```

Those are web-application questions.

Response facade

The Express-like `Response` adds application-oriented response operations to the lower HTTP response.

Response aspect	Examples
status and headers	<code>status</code> , <code>set</code> , <code>append</code> , <code>type</code>
cookies	<code>cookie</code> , <code>clearCookie</code>
body output	<code>send</code> , <code>json</code> , <code>end</code>
files	<code>sendFile</code> , <code>download</code> , <code>attachment</code>
redirects	<code>redirect</code> , <code>location</code>
advanced HTTP	<code>upgrade</code> , <code>fragments</code> , <code>socket-context access</code>

This is the response-side semantic lift. The application can express common web responses directly:

```
send JSON
redirect
set a cookie
send a file
return a status
upgrade the connection
```

Those operations still produce HTTP behavior underneath. The facade makes the application intent clearer.

Controlled access to lower HTTP capabilities

The facade raises the application API, but it does not hide all lower HTTP capabilities. Advanced operations remain available when the application genuinely needs them.

Examples include:

- `upgrade`,
- fragment sending,
- explicit end control,
- `socket-context access`,
- lower header control.

The facade is not a wall. It is a raised application surface with deliberate access points back to lower HTTP capabilities when the application genuinely needs them. This keeps the application interface convenient without pretending that lower HTTP and connection realities no longer exist.

22.10 Built-in middleware as reusable application behavior

The Express-like module also provides reusable middleware. Built-in middleware packages common request-processing behavior so it can be mounted once and reused across routes.

Examples include:

Middleware	Application concern
<code>BasicAuthentication</code>	route-level authentication
<code>StaticMiddleware</code>	static file serving
<code>VHost</code>	virtual-host based routing
<code>VerboseRequest</code>	request visibility
<code>JsonMiddleware</code>	JSON request handling

These belong above HTTP and below application-specific route logic. They are reusable application behaviors. That is middleware territory.

Authentication, static serving, virtual hosts, request visibility, and JSON handling

Basic authentication, static serving, virtual-host routing, verbose request reporting, and JSON handling are examples of reusable application behavior. They can be attached through middleware instead of being baked into the lower socket layer, the HTTP parser, or every individual route handler.

Static serving may involve root directories, index handling, fall-through behavior, headers, cookies, and connection-state decisions after the response. Those are practical application concerns, but they do not belong in the socket layer or in every route handler.

`VHost` belongs here because host-based dispatch is web-application routing behavior. `VerboseRequest` belongs here because request visibility is useful across routes but should not be duplicated inside every handler.

The Express module includes JSON middleware when the required `nlohmann_json` dependency is present; the build treats that dependency as required for this module. Architecturally, JSON middleware belongs to the same group of reusable request-processing behavior.

The architectural point is:

```
common request-processing behavior
-> reusable middleware
```

22.11 Lower layers still matter

The Express-like layer changes application organization. It does not remove the lower stack. A route handler may look high-level, but `bind/listen` activation, TLS setup, HTTP parsing, response streaming, upgrade behavior, timeout boundaries, diagnostics, and shutdown still belong to the same layered runtime model.

That is the useful distinction: Express organizes web application logic; it does not hide the architecture beneath it.

22.12 From Express-like routing to Server-Sent Events

Chapter 22 is the second step in Part VII. Chapter 21 introduced the HTTP layer. Chapter 22 showed how HTTP handling becomes application structure.

The next chapter moves to a specific HTTP-based real-time mechanism:

```
Server-Sent Events
```

This is a natural next step. The reader now understands:

```
HTTP request / response
-> Express-like application structure
-> long-lived HTTP event streaming
```

Chapter 23 then asks what happens when a route is not a short request/response exchange, but a long-lived one-way event stream. Server-Sent Events will show how the web stack handles that shape while still using the same layered discipline.

What to remember

- The Express-like layer sits above HTTP and organizes HTTP handling into web-application structure.
- `WebApp` is a router-shaped application surface.
- `WebAppT<ServerT>` combines that application surface with a concrete HTTP server type.
- HTTP request readiness enters the Express layer through a controller and the root route.

- **Router** composes routes, middleware, mounted routers, method-specific handlers, and routing policy.
- Application callbacks and middleware callbacks have different responsibilities.

22.13 Express public surface: WebApp header and carrier component

A file that directly uses the IPv4 legacy Express WebApp includes:

```
#include <express/legacy/in/WebApp.h>
```

Code using the convenience server helpers includes:

```
#include <express/legacy/in/Server.h>
```

The corresponding carrier component is:

```
http-server-express-legacy-in
```

It selects the Express layer together with the IPv4 legacy stream carrier. The source-side and build-side names rhyme without being the same mechanism. Chapter 32 gives the source-derived matrix for the broader set of components and headers.

22.14 Closing perspective

Chapter 21 raised stream communication to HTTP messages. Chapter 22 raised HTTP messages into application structure.

Chapter 23 keeps that structure but changes the temporal shape of a handler: instead of answering once, a route may keep an HTTP response open and emit events over time. There, HTTP is used for long-lived one-way event streaming.

Chapter 23

Server-Sent Events and Real-Time HTTP

23.1 From request / response to event streams

Chapter 22 showed how HTTP handling becomes application structure through routing, middleware, and request/response facades. Chapter 23 keeps that structure inside the HTTP world, but changes the temporal shape of one response: a route may start a response stream and keep it open while the server sends event-stream records over time.

That is the central idea of this chapter:

SSE lifetime note

Server-Sent Events keep the application inside HTTP while changing the response from a short message into a long-lived event stream.

Here, “real-time” means live-update HTTP. It does not mean deterministic latency, hard real-time scheduling, or a replacement for the lower timing and failure model introduced earlier. SSE is a practical web mechanism for delivering server-side updates as they become available.

SSE is useful because it is still HTTP. It does not require a full bidirectional protocol when the application only needs server-to-client updates, and it fits naturally between ordinary request/response HTTP and WebSocket:

ordinary HTTP

-> one request, one response

Server-Sent Events

-> one request, one long-lived event response

WebSocket

-> HTTP upgrade, then bidirectional messaging

This makes SSE a good bridge inside Part VII:

Chapter 21: HTTP messages

Chapter 22: Express-like application structure

Chapter 23: long-lived one-way HTTP event streams

Chapter 24: HTTP upgrade to bidirectional WebSocket communication

23.2 SSE in the layered SNode.C web stack

The stack now looks like this:

```
lower communication family
  -> stream transport
      -> legacy or TLS connection handling
          -> HTTP request / response
              -> long-lived event-stream response
                  -> MessageEvent handling
```

SSE does not bypass the lower architecture. It still depends on the runtime, a lower communication family, stream transport, legacy or TLS connection handling, HTTP client/server behavior, connection lifecycle, retry and reconnect policy, and diagnostics.

The application-facing unit changes again. Earlier chapters showed a sequence of semantic lifts:

```
stream bytes
  -> HTTP Request / Response

HTTP request / response
  -> Express Request / Response facades

SSE event-stream fields
  -> MessageEvent
```

That is the main continuity. SSE remains HTTP, but the application no longer reacts to one completed response only. It reacts to event objects produced from a long-lived response stream.

23.3 Ordinary HTTP, SSE, and WebSocket side by side

A compact comparison helps place SSE without turning this chapter into a protocol reference.

Concern	Ordinary HTTP	Server-Sent Events	WebSocket
connection shape	request / response	request / long-lived response stream	upgraded bidirectional connection
direction	usually client request, server response	server-to-client events	both directions
protocol world	HTTP	HTTP	starts with HTTP upgrade, then WebSocket
application unit	response	event	message / frame
typical use	documents, APIs, files	notifications, dashboards, feeds	bidirectional interaction
complexity	lowest	moderate	higher

SSE is not WebSocket with fewer features. It is a different fit. It is useful when the server should push events and the client does not need to send messages back over the same long-lived channel. If both sides need to send independent messages over one long-lived channel, WebSocket becomes the more natural fit. Chapter 24 treats that case.

23.4 SSE as long-lived HTTP

SSE begins as HTTP. The client sends an HTTP request for an event-stream endpoint. If the server accepts the request and responds with an event-stream response, the response remains open.

A useful model is:

```
HTTP request
-> HTTP response starts
    -> response remains open
        -> server sends event-stream records over time
            -> client dispatches MessageEvent objects
```

The response is still an HTTP response. The difference is that it is not immediately completed. It becomes a long-lived stream of event records.

On the client side, the long-lived response is represented by an HTTP request that installs an SSE event receiver after the response has been validated as `text/event-stream`. On the server side, SSE is an HTTP or Express route that chooses to keep the response open and format outgoing data as event-stream records.

One request, one open response stream

Ordinary HTTP often looks like this:

```
request
-> response
-> done
```

SSE changes the response phase:

```
request
-> response starts
    -> event
    -> event
    -> event
    -> ...
```

This makes SSE suitable for live updates without repeatedly polling the server. The server can keep the response open and emit changes as they happen.

Server-to-client event direction

SSE is one-way by design. The long-lived stream carries events from server to client. That makes it useful for dashboards, notifications, monitoring views, activity streams, state updates, and incremental event feeds.

The client can still make ordinary HTTP requests elsewhere in the application. The SSE stream itself is simply the endpoint that carries a long-lived server-to-client update channel.

Why one-way is useful

One-way streaming keeps the model simpler than bidirectional messaging. The application can use SSE when the main requirement is:

```
server state changes
-> client view updates
```

That pattern is common in monitoring and dashboard applications. The application still has to think about reconnect, retry, timeouts, and diagnostics, but it does not need the full protocol shape of a bidirectional upgraded connection.

23.5 Client-side EventSource and server-side streaming endpoints

It is important to distinguish the two sides carefully.

Side	SNode.C view
client side	built-in <code>EventSource</code> abstraction on top of an HTTP client
server side	HTTP/Express endpoint that keeps a response open and writes event-stream records

The inspected source strongly exposes the client-side `EventSource` facility. Server-side SSE is not a different server instance type and not a symmetric server-side `EventSource` abstraction. It is a route or HTTP handler that keeps the response open and writes data in event-stream format.

Conceptually:

```
server route
  -> emits text/event-stream response

client EventSource
  -> requests text/event-stream
    -> parses events
    -> dispatches MessageEvent objects
```

The two sides meet at the HTTP/SSE boundary, but they are not the same API surface.

A server-side event-stream record may contain fields such as:

```
event: status
id: 42
data: {"state":"online"}
```

The blank line ends the accumulated event record. This is enough server-side format detail for this chapter. The central architectural point is that the server writes event-stream records over an HTTP response that remains open.

23.6 A compact server-side SSE endpoint

A server-side SSE endpoint is still an ordinary HTTP route. The route must first verify that the request actually asks for an event stream. In practice, that means checking the request's `Accept` header for `text/event-stream` before the route switches into long-lived streaming behavior.

The following sketch uses an application-owned measurement source. The important framework-facing points are the request validation, the response headers, the explicit header send, and the use of response fragments while the connection remains open:

```
app.get("/events", [&measurements] APPLICATION(req, res) {
  if (web::http::ciContains(req->get("Accept"), "text/event-stream")) {
    res->set("Content-Type", "text/event-stream")
    .set("Cache-Control", "no-cache")
    .set("Connection", "keep-alive")
    .sendHeader();

    if (const Measurement current = measurements.current(); current.sequence > 0) {
      res->sendFragment("event: measurement\n");
      res->sendFragment("id: " + std::to_string(current.sequence) + "\n");
      res->sendFragment("data: " + current.toJson().dump() + "\n\n");
    }

    measurements.subscribe([res](const Measurement& measurement) {
      const bool keepSubscriber = res->isConnected();
      if (keepSubscriber) {
        res->sendFragment("event: measurement\n");
        res->sendFragment("id: " + std::to_string(measurement.sequence) + "\n");
      }
    });
  }
})
```

```

        res->sendFragment("data: " + measurement.toJson().dump() + "\n\n");
    }

    return keepSubscriber;
});
} else {
    res->status(406).send("SSE requires Accept: text/event-stream");
}
});

app.get("/simulate", [&measurements] APPLICATION(req, res) {
    const Measurement measurement = measurements.publish("temperature", 24.0);

    res->set("Content-Type", "application/json")
        .send(measurement.toJson().dump());
});

```

The `Measurement` type and the `measurements` publisher are application code, not special SSE machinery. The `SNode.C`-specific shape is the HTTP route and response handling. The route rejects non-SSE requests with an ordinary HTTP response; only a request that accepts `text/event-stream` receives the streaming response.

After `sendHeader()`, the response body is written as SSE records. Each record is plain text. A blank line terminates the current event. The response is intentionally not ended after the first record. It remains open until the application decides to close it or until the peer disconnects.

23.7 A compact EventSource client

The corresponding client side enters through the concrete `EventSource` wrapper for the selected HTTP client stack. A compact IPv4 legacy client looks like this:

```

#include <core/SNodeC.h>
#include <net/in/SocketAddress.h>
#include <web/http/legacy/in/EventSource.h>
#include <log/Logger.h>

int main(int argc, char* argv[]) {
    core::SNodeC::init(argc, argv);

    const net::in::SocketAddress address{"127.0.0.1", 8080};

    auto events = web::http::legacy::in::EventSource(
        "http",
        address,
        "/events");

    events->onOpen([] {
        VLOG(1) << "SSE stream opened";
    });

    events->onMessage([](const web::http::client::tools::EventSource::MessageEvent& event) {
        VLOG(1) << "message: " << event.data;
    });

    events->addEventListener(
        "measurement",
        [](const web::http::client::tools::EventSource::MessageEvent& event) {
            VLOG(1) << "measurement: " << event.data;
        });

    events->onError([] {
        LOG(ERROR) << "SSE stream error";
    });
}

```



```
});

return core::SNodeC::start();
}
```

The server-side route produces event-stream syntax. The client-side `EventSource` parses that stream, applies retry and continuity rules, and dispatches ordinary `MessageEvent` callbacks. `readyState()`, `retry()`, `lastEventId()`, and `close()` remain the client-side controls for observing and steering that stream lifecycle. The matching companion programs are `SSE-Server` and `SSE-EventSource-Client`.

23.8 The EventSource client abstraction

The client-side `EventSource` abstraction gives the application an event-stream model while the lower stream and HTTP details remain underneath. The application sees a higher-level surface:

EventSource concept	Meaning
<code>ReadyState</code>	stream lifecycle: connecting, open, closed
<code>MessageEvent</code>	application-facing event object
<code>onMessage(...)</code>	default message listener
<code>addEventListener(...)</code>	custom event-type listener
<code>removeEventListeners(...)</code>	remove listeners for a type
<code>onOpen(...)</code>	stream open notification
<code>onError(...)</code>	stream error notification
<code>lastEventId()</code>	last event ID known to the client
<code>retry()</code> / <code>retry(...)</code>	current retry interval
<code>close()</code>	intentional shutdown

This follows the pattern prepared by the previous chapters. The lower client handle, configured client-side communication role, registered runtime-visible client instance, connection, HTTP request, and parser still exist. `EventSource` is the application-side abstraction that exposes the event-stream behavior.

ReadyState as client-side lifecycle

SSE is not a one-shot request. It has a lifecycle, and the client-side `EventSource` ready-state model expresses that lifecycle:

Ready state	Meaning
<code>CONNECTING</code>	the stream is not open yet or is reconnecting
<code>OPEN</code>	the event stream is active
<code>CLOSED</code>	the application has closed it or it will not reconnect

This is a runtime concept, not only a browser-style convenience. A long-lived stream needs a way to say whether it is connecting, open, or closed. That state also matters for diagnostics and application decisions.

MessageEvent as application-facing unit

The `MessageEvent` object is the semantic lift from SSE wire fields to application meaning. It contains exactly the kind of information the application normally needs:

Field	Meaning
<code>type</code>	"message" or a custom event type
<code>data</code>	event payload
<code>lastEventId</code>	event ID associated with this event

Field	Meaning
<code>origin</code>	stream origin

The application should not normally parse raw event-stream lines. It should react to events:

```
raw SSE fields
  -> MessageEvent
      -> type, data, lastEventId, origin
```

This is the SSE equivalent of the earlier request/response lifts.

Message, custom event, open, and error listeners

The EventSource client has several listener surfaces:

Listener	Meaning
<code>onMessage(...)</code>	receive default "message" events
<code>addEventListener(...)</code>	receive events with a named event type
<code>onOpen(...)</code>	observe successful stream opening
<code>onError(...)</code>	observe stream errors or interruptions

Custom event listeners receive events by type. The default `onMessage` listeners receive events whose type is "message".

This lets the application express event-stream behavior directly:

```
stream opened
  -> start showing live state

message event arrived
  -> update application state

custom event arrived
  -> dispatch by event type

stream error occurred
  -> show degraded or reconnecting state
```

The lower HTTP receive loop remains underneath, but the application-facing model is event-based.

Close as intentional shutdown

`close()` is important because not every disconnect is a failure. A user or application may intentionally stop the stream.

In SNode.C's EventSource client, `close()` marks the EventSource as **CLOSED**, disables reconnect and retry on the underlying client configuration, and shuts down the write side if a socket connection is present. The conceptual distinction is:

```
stream interrupted while not CLOSED
  -> may reconnect

stream closed intentionally
  -> should not reconnect
```

That matches the retry/reconnect vocabulary from Chapter 20.

23.9 SSE starts as an HTTP request

Even though the application eventually receives events, SSE starts as HTTP. The client sends a request for the event-stream endpoint.

A simplified request shape is:

```
GET /events HTTP/1.1
Accept: text/event-stream
Cache-Control: no-cache
Connection: keep-alive
```

The exact path depends on the application. the client does not start with a new non-HTTP protocol. It asks HTTP for an event stream.

Event-stream request setup

The client-side HTTP request for SSE uses HTTP/1.1 behavior and event-stream headers. The key header is:

```
Accept: text/event-stream
```

This tells the server that the client expects a server-sent event stream. The request also uses:

```
Cache-Control: no-cache
Connection: keep-alive
```

These settings match the long-lived streaming nature of the interaction.

Stream validation

The client should not treat every HTTP response as an event stream. It validates that the response is actually a server-sent event stream.

The important check is:

```
response Content-Type contains text/event-stream
```

The request still carries `Accept: text/event-stream`. If validation fails, the socket context is closed and the error callback is invoked. This prevents a normal HTML page, redirect, error page, or unrelated response from being interpreted as SSE data.

23.10 Parsing the event stream

Once the stream is open, the client receives bytes over time. The parser turns those bytes into event records:

```
incoming stream bytes
  -> pending buffer
      -> lines
          -> fields
              -> accumulated event
                  -> dispatch on blank line
```

SSE is therefore more than “read text forever.” The stream has field semantics. The parser accumulates, interprets, and dispatches.

SSE fields

A compact field table is enough for this chapter.

Field	Effect
data	append value plus newline to the accumulated payload
event	set custom event type
id	update the ID buffer if the value contains no NUL byte
retry	update retry timing if the value is numeric
blank line	dispatch accumulated event
comment line	ignored

Multiple **data:** lines belong to the same event until a blank line dispatches it. The default event type is "message" when no custom **event:** field is present. The **id:** field participates in continuity. The **retry:** field participates in reconnection timing.

Dispatch on blank line

SSE dispatch is line-oriented. The parser accumulates fields until it sees the event boundary. The event boundary is a blank line.

At that point:

```
accumulated data
  -> MessageEvent
    -> listeners
```

Before dispatch, the trailing newline added by **data:** accumulation is removed. If there is no accumulated data, no message event needs to be delivered. This keeps event dispatch tied to the SSE wire format.

Size guards and malformed streams

A long-lived stream must not allow unbounded pending data. The implementation applies finite limits to pending lines and accumulated data, so a malformed or hostile long-lived stream cannot grow parser state without bound. If parsing fails, the stream can be closed.

That is part of making SSE operationally usable. Long-lived inputs need limits.

23.11 Retry and continuity

SSE interacts naturally with retry and reconnect behavior. This should be read through the vocabulary of Chapter 20.

SSE has protocol-level retry information. SNode.C maps that information into the underlying client reconnect/retry configuration. That means the event-stream layer and the client role cooperate.

The **retry:** field

A server may send a **retry:** field. The value is interpreted as a retry interval. When accepted, it updates the EventSource retry value and the underlying client reconnect/retry timing.

Conceptually:

```
retry: 5000
  -> EventSource retry interval becomes 5000 ms
    -> client reconnect/retry timing is updated
```

retry: is not a generic transport timeout. It is event-stream protocol information that updates the client-side recovery interval. The SSE value is expressed in milliseconds; the underlying client configuration receives the corresponding seconds value.

Last-Event-ID

The `id:` field updates the client-side event ID buffer. When an event is dispatched, the last event ID can be remembered.

When a new request starts and a last event ID is known, the client sets:

```
Last-Event-ID: ...
```

on the request. This gives the server a continuity hint after reconnect. The exact server behavior depends on the application, but the client-side mechanism is important: it lets an event stream carry continuity information across interruptions.

Interruption versus intentional close

An interruption while the stream is not `CLOSED` is handled differently from an intentional close. Conceptually:

```
interrupted while not CLOSED
-> error notification
-> ready state returns to CONNECTING
-> parsing buffers are cleared
-> retry/reconnect timing is reapplied

closed intentionally
-> ready state becomes CLOSED
-> retry/reconnect is disabled
```

This is where Chapter 20's `retry/reconnect` distinction becomes visible at the SSE layer.

23.12 Express-like routes and SSE endpoints

Express-like applications are a natural place to expose SSE endpoints. They already organize routes, middleware, authentication, static assets, application APIs, and response behavior.

An SSE endpoint can be one route in that application. Server-side code can keep the HTTP response open and write event-stream records over time. The built-in `EventSource` facility discussed in this chapter is the client-side counterpart.

The two sides meet at the HTTP/SSE boundary:

```
server route
-> emits text/event-stream response

client EventSource
-> requests text/event-stream
    -> parses events
        -> dispatches MessageEvent objects
```

This pairing is useful, but it is not the same abstraction on both sides. The server route produces event-stream syntax; the client `EventSource` interprets it as events.

23.13 Lower layers and diagnostics still matter

SSE is high-level event-stream behavior, but it is still a long-lived HTTP response over a lower connection. Operationally, the relevant evidence is layered: connection establishment, response validation, transition to `OPEN`, parser errors, retry interval, last event ID, disconnects, and intentional close.

That connects directly to the diagnostic model from Chapter 18 and the configuration/timing model from Chapter 20. SSE does not need a new operational philosophy; it applies the existing one to a long-lived HTTP stream.

23.14 Real-time-style HTTP

The chapter title says “real-time HTTP.” The word “real-time” here means live-update behavior over HTTP. It does not mean deterministic latency or hard real-time scheduling.

SSE is a practical web mechanism for delivering events over a long-lived HTTP response. That makes it useful for user interfaces and monitoring systems where updates should arrive as the server produces them.

Use cases include:

- sensor dashboards,
- status pages,
- notification feeds,
- build or deployment logs,
- monitoring views,
- application activity streams.

The timing behavior still depends on the network, server, client, buffering, and reconnect policy.

What to remember

- Server-Sent Events keep the application inside HTTP while turning one response into a long-lived event stream.
- SSE is one-way: the long-lived stream carries events from server to client.
- SNode.C provides a client-side **EventSource** abstraction on top of an HTTP client.
- Server-side SSE is an HTTP or Express route that keeps a response open and writes event-stream records.
- **ReadyState** describes the client-side stream lifecycle: connecting, open, or closed.
- **MessageEvent** raises SSE fields into an application-facing event object.

23.15 EventSource public surface

Client-side EventSource code includes the EventSource abstraction it names. For the IPv4 legacy wrapper, that front door is:

```
#include <web/http/legacy/in/EventSource.h>
```

Server-side SSE remains an HTTP or Express route that validates the request and streams **text/event-stream** response fragments. Chapter 32 summarizes the broader component/header mapping.

23.16 Closing perspective

SSE keeps the HTTP/Express structure but stretches one response over time. WebSocket, the next step, uses HTTP differently: as an upgrade negotiation before bidirectional message communication begins.

Chapter 24

WebSocket and Protocol Upgrade

24.1 From event streams to upgraded bidirectional communication

Chapter 23 showed how HTTP can remain open as a one-way event stream. Chapter 24 takes the next step: HTTP can also negotiate an upgrade. After a successful upgrade, the same lower connection no longer behaves as ordinary request/response HTTP; that connection episode continues as a bidirectional, message-oriented WebSocket connection.

That is the central idea of this chapter:

Upgrade-boundary note

WebSocket starts in HTTP, but after the upgrade the connection is handled as a bidirectional message-oriented WebSocket connection.

This makes WebSocket different from both ordinary HTTP and Server-Sent Events. Figure 24.1 places these ideas in the relationship used in this part of the book: HTTP request/response is the common web-protocol foundation; Server-Sent Events and WebSocket are dependent protocol shapes inside the same web protocol layer; and the Express-like application structure uses that layer to organize application-facing endpoints. The paragraph stays at the protocol-relationship level before later sections discuss handler mechanics.

The key word for the WebSocket path in Figure 24.1 is:

upgrade

Upgrade does not mean that the lower connection disappears. The lower communication family, stream transport, TLS state if present, connection identity, counters, runtime lifecycle, and diagnostic surface remain part of the same peer episode. What changes is the protocol context that interprets that episode.

Figure 24.1 also prevents a wrong reading of the web chapters. Express-like routing is an application-structuring layer, not the protocol parent of SSE or WebSocket. Application code can use the Express-like structure to organize ordinary HTTP routes, SSE endpoints, and WebSocket upgrade entry points, or it can use the web protocol layer directly where that is the better fit.

Chapter 24 therefore closes the main web-protocol climb before the book moves to MQTT.

24.2 WebSocket in the layered SNode.C web stack

The stack now looks like this:

```
lower communication family
-> stream transport
```

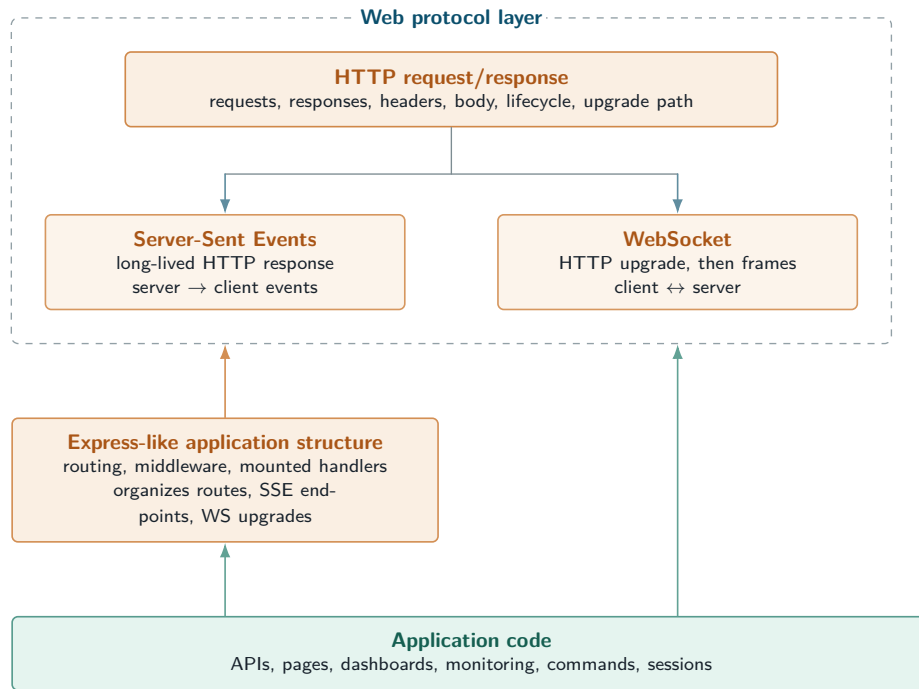


Figure 24.1: The web protocol layer with HTTP request/response as the common foundation for Server-Sent Events and WebSocket, and with the Express-like application structure using that layer.

```

-> legacy or TLS connection handling
  -> HTTP request / response
    -> upgrade negotiation
      -> WebSocket frames
        -> subprotocol semantics
  
```

This is still the same layered architecture. WebSocket does not erase the lower communication family. It does not erase TLS. It does not erase HTTP. It uses HTTP as the negotiation layer for moving the same connection episode into a different protocol context.

After a successful upgrade, ordinary request/response handling no longer owns that connection episode. The upgraded WebSocket context does.

A compact model is:

```

HTTP request
  -> upgrade negotiation
    -> upgraded connection context
      -> WebSocket frames
        -> subprotocol semantics
  
```

With TLS, the WebSocket upgrade happens above the secure stream connection. The secure connection remains the carrier underneath the HTTP negotiation and the WebSocket frames.

The guiding sentence for this chapter is:

```

same lower connection
  -> HTTP negotiation
    -> new protocol context
  
```

24.3 HTTP, SSE, and WebSocket side by side

The position of WebSocket becomes clearer when it is placed next to ordinary HTTP and SSE.

Concern	Ordinary HTTP	SSE	WebSocket
start	HTTP request	HTTP request	HTTP request with upgrade
after start	response completes	response remains open	protocol changes after upgrade
direction	request / response	server to client	bidirectional
unit	response	event	frame / message
connection shape	HTTP connection	long-lived HTTP response	upgraded connection
typical use	APIs, pages, files	feeds, dashboards, notifications	interactive bidirectional protocols
next semantic layer	application response	event handling	subprotocol

SSE remains inside HTTP. WebSocket begins in HTTP and then moves to WebSocket frame handling. That distinction is the main reason WebSocket deserves its own chapter.

This is not a question of one protocol being “better” than the other. The fit is different:

```
HTTP
-> short request / response

SSE
-> one-way long-lived HTTP response

WebSocket
-> upgraded bidirectional message channel
```

24.4 WebSocket after the HTTP upgrade boundary

Chapter 21 described HTTP upgrade as a generic HTTP-layer boundary. That is where the client names an upgrade target, the server accepts or rejects the transition, and the HTTP layer selects a socket-context upgrade implementation.

This chapter now specializes that generic boundary to WebSocket. In the HTTP upgrade layer the selected upgrade name is:

```
websocket
```

That name selects the upgraded connection type. It is not yet an application protocol name. After the WebSocket upgrade has succeeded, the WebSocket layer may perform another selection step: the WebSocket subprotocol.

This gives two separate questions:

```
HTTP upgrade name
-> which upgraded connection context should replace ordinary HTTP?

WebSocket subprotocol name
-> which application protocol should run inside the WebSocket connection?
```

Keeping those questions separate is important. The HTTP upgrade machinery belongs to the web/HTTP layer. The WebSocket subprotocol machinery belongs to the WebSocket layer.

24.5 WebSocket as the concrete upgraded protocol

WebSocket is the concrete upgrade target discussed here.

For WebSocket, the generic model becomes:

```

HTTP request
-> upgrade negotiation
    -> WebSocket SocketContextUpgrade
        -> WebSocket frames
            -> WebSocket subprotocol

```

After the WebSocket upgrade succeeds, ordinary HTTP request/response handling is no longer the application-facing model for that connection episode. The upgraded connection is handled as WebSocket.

That means the protocol layer now cares about:

- frames,
- message boundaries,
- fragmentation,
- ping,
- pong,
- close,
- payload counters,
- subprotocol behavior.

The connection is still the same peer relationship underneath. The protocol interpretation has changed.

24.6 SocketContextUpgrade as the boundary object

The central code-shaped concept in this chapter is `SocketContextUpgrade`.

In the generic upgrade model, it represents the boundary between HTTP negotiation and the selected upgraded protocol context. For WebSocket, it represents the boundary between HTTP and WebSocket.

In simplified form, the generic WebSocket upgrade context has this shape:

```

template <typename SubProtocolT, typename RequestT, typename ResponseT>
class SocketContextUpgrade
    : public web::http::SocketContextUpgrade<RequestT, ResponseT>
    , public SubProtocolContext {
    // WebSocket framing and subprotocol boundary
};

```

The class shape expresses the architecture in C++: one base keeps the object attached to HTTP upgrade, while the other exposes the WebSocket and subprotocol surface used after the transition.

The template parameters keep both sides visible at the type level:

```

Request / Response
-> HTTP negotiation side

SubProtocolT
-> upgraded WebSocket meaning

```

The object therefore lives exactly at the transition point. It is still able to explain where the connection came from, and it also provides the surface needed for the protocol that now takes over.

Base / role	Meaning
HTTP <code>SocketContextUpgrade</code>	keeps the transition connected to HTTP upgrade
<code>SubProtocolContext</code>	gives the upgraded connection its WebSocket/subprotocol surface
socket connection	continues carrying the same peer episode
<code>Request / Response</code>	represent the HTTP upgrade negotiation side

It is not an ordinary HTTP context. It is not a plain stream context. It is an upgraded context.

Same connection, new protocol identity

The upgrade does not create a completely unrelated connection. It changes what the existing connection means.

Before the upgrade:

```
connection
-> HTTP request / response context
```

After the upgrade:

```
same connection
-> WebSocket socket-context upgrade
    -> WebSocket frames
        -> subprotocol behavior
```

This is important for diagnostics and application reasoning. Connection identity, lower endpoint, TLS state, counters, timeout behavior, and runtime lifecycle still belong to the same peer episode. What changes is the protocol context interpreting that episode.

HTTP upgrade context plus WebSocket subprotocol context

The generic WebSocket upgrade context combines two responsibilities. The HTTP upgrade side describes where the transition comes from. The WebSocket/subprotocol side describes what the connection becomes after the transition.

That is why the class combines:

```
HTTP upgrade context
+ WebSocket subprotocol context
```

This keeps the layer boundary explicit. The framework does not pretend WebSocket is merely another HTTP callback. It also does not pretend WebSocket is unrelated to HTTP.

24.7 WebSocket messages and frames

After the upgrade, the application-facing model changes again.

Earlier chapters showed:

```
HTTP
-> request / response

SSE
-> event

WebSocket
-> message / frame
```

The WebSocket layer must understand message and frame structure. Frames are protocol units. Messages are the application-facing payload units assembled through frame handling.

A useful separation is:

```
application-facing shift
-> messages

protocol machinery
-> frames, continuation, control frames
```

```
callback / API surface
-> message start, data, end, error
```

The WebSocket layer provides operations for:

- sending a complete message,
- starting a message,
- sending continuation frames,
- ending a message,
- sending ping,
- sending pong,
- sending close,
- reading and writing frame chunks,
- reacting to message start,
- reacting to message data,
- reacting to message end,
- reacting to message errors.

This belongs in the WebSocket layer. Applications should not have to reimplement WebSocket framing in every protocol.

Data and continuation frames

A compact view of frame categories is:

WebSocket concern	Meaning
text frame	text message data
binary frame	binary message data
continuation	fragmented message continuation
ping	liveness / control probe
pong	ping response / control signal
close	protocol-level closing signal

Text and binary frames carry application data. Continuation frames allow fragmented messages. The WebSocket layer keeps those protocol details in one place.

Ping, pong, and close

Ping, pong, and close are WebSocket protocol concerns. They are not subprotocol business messages.

A ping is a control signal. A pong is a control response. A close frame is a protocol-level closing signal.

This distinction matters because it keeps application semantics separate from WebSocket protocol mechanics. An application protocol carried over WebSocket may define its own messages. It should not have to reinvent WebSocket liveness and closing behavior.

24.8 Server-side and client-side upgrade responsibilities

Server and client WebSocket support are related, but not identical.

Both sides use the same broad idea:

```
HTTP upgrade
-> WebSocket socket-context upgrade
    -> selected WebSocket subprotocol
```

But they enter the transition from different responsibilities.

Side	Upgrade responsibility
server	receive an HTTP upgrade request, parse requested subprotocol names, select a supported subprotocol, and confirm the upgrade
client	prepare WebSocket upgrade headers, request a subprotocol, validate the accept response, and load the selected subprotocol
both	continue over an upgraded WebSocket context

The server side receives a set of requested subprotocol names and selects what it can support. If the upgrade succeeds, the server confirms the change with an HTTP 101 **Switching Protocols** response and attaches the upgraded WebSocket context to the connection.

The client side initiates the upgrade. It prepares the WebSocket upgrade request, validates the response, and continues only when the server response proves that the upgrade was accepted.

This mirrors earlier server/client distinctions in the book. The symmetry remains. The responsibilities differ.

Upgrade calls belong to the HTTP boundary

Chapter 21 showed the compact server-side `res->upgrade(...)` and client-side `req->upgrade(...)` calls. This chapter deliberately does not repeat them. Here the concern is what the selected `websocket` upgrade means after the HTTP boundary has been crossed: WebSocket framing, control frames, subprotocol selection, and subprotocol deployment.

24.9 The WebSocket module structure

The build/module split mirrors the architectural split: shared WebSocket mechanics are separate from the server-side and client-side HTTP upgrade roles.

Module	Meaning
<code>websocket</code>	shared framing and subprotocol infrastructure
<code>websocket-server</code>	server-side HTTP upgrade integration and subprotocol selection
<code>websocket-client</code>	client-side HTTP upgrade integration and subprotocol selection

The shared layer contains WebSocket concerns such as:

- receiver,
- transmitter,
- socket-context upgrade,
- subprotocol,
- subprotocol context,
- subprotocol factory,
- subprotocol factory selector.

The server and client layers build on that shared base and connect it to the corresponding HTTP server or HTTP client side.

This module split is useful because the protocol mechanics are shared while the upgrade roles differ.

24.10 Subprotocols as the next semantic layer

WebSocket is not the final semantic layer in SNode.C. It is a bidirectional message channel with a subprotocol layer.

That subprotocol gives the WebSocket channel its application/message semantics. A raw WebSocket message is not yet a complete application idea. It may be a chat message, a telemetry update, a dashboard command, an MQTT packet, or something else.

A useful model is:

```
WebSocket
  -> upgraded bidirectional carrier

WebSocket subprotocol
  -> application/message semantics carried over WebSocket
```

This distinction is important. Without the subprotocol layer, all WebSocket application logic would collapse into one undifferentiated callback. SNode.C keeps the levels separate by making the subprotocol part of the WebSocket upgrade structure.

SubProtocol and SubProtocolContext

The WebSocket layer provides explicit subprotocol infrastructure.

Concept	Meaning
SubProtocolContext	WebSocket-facing context surface for send message/start/frame/end, ping, pong, close, counters, and connection access
SubProtocol	protocol-specific behavior riding over the WebSocket context, with lifecycle and message callbacks
SubProtocolFactory	creates subprotocol instances
SubProtocolFactorySelector	resolves requested subprotocol names in a server or client selection context

This keeps WebSocket mechanics and application-message semantics separated.

WebSocket provides the upgraded carrier. The subprotocol defines what the messages mean.

WebSocket subprotocol deployment contract

Chapter 21 distinguished ordinary linked-library deployment from runtime-selected upgrade modules. WebSocket subprotocols use the same idea one level later. Subprotocol selection is a second deployment boundary, below the HTTP-upgrade boundary: the HTTP upgrade module makes the connection a WebSocket connection, and the WebSocket subprotocol factory gives that connection its application semantics.

For dynamically loaded subprotocols, SNode.C looks below the WebSocket subprotocol directory. The selector first checks an application-specific subdirectory and then falls back to the shared directory:

```
application-specific directory:
  ${CMAKE_INSTALL_PREFIX}/${CMAKE_INSTALL_LIBDIR}/snode.c/web/http/upgrade/websocket/<
    application-name>

shared WebSocket subprotocol directory:
  ${CMAKE_INSTALL_PREFIX}/${CMAKE_INSTALL_LIBDIR}/snode.c/web/http/upgrade/websocket
```

The dynamic module contract is role-specific:

```
server-side module:
  libsnodec-websocket-<subprotocol-name>-server.so.<SOVERSION>

client-side module:
  libsnodec-websocket-<subprotocol-name>-client.so.<SOVERSION>
```

```
server-side factory symbol:
    <subprotocol-name>ServerSubProtocolFactory

client-side factory symbol:
    <subprotocol-name>ClientSubProtocolFactory
```

For an echo subprotocol this becomes:

```
libsndec-websocket-echo-server.so.<SOVERSION>
libsndec-websocket-echo-client.so.<SOVERSION>

echoServerSubProtocolFactory
echoClientSubProtocolFactory
```

A linked deployment uses the same subprotocol name but resolves it from the selector's linked-factory cache. For a project that builds its own subprotocol, the CMake shape is compact:

```
add_library(my-echo-server STATIC Echo.cpp EchoFactory.cpp)
target_link_libraries(my-echo-server PUBLIC sndec::websocket-server)

target_link_libraries(my_ws_server PRIVATE
    sndec::http-server-express-legacy-in
    sndec::websocket-server
    my-echo-server
)
```

The client side mirrors the role:

```
add_library(my-echo-client STATIC Echo.cpp EchoFactory.cpp)
target_link_libraries(my-echo-client PUBLIC sndec::websocket-client)

target_link_libraries(my_ws_client PRIVATE
    sndec::http-client
    sndec::net-in-stream-legacy
    sndec::websocket-client
    my-echo-client
)
```

The exact target names are application choices; the contract is the name-to-factory resolution. A WebSocket subprotocol name must resolve to a factory on the correct role side. Dynamic deployment provides that factory through a correctly named module. Linked deployment provides it through linked and retained registration code. A missing HTTP-upgrade module prevents the connection from becoming WebSocket; a missing subprotocol factory means that WebSocket exists, but the requested application protocol cannot be instantiated.

A compact WebSocket subprotocol

A WebSocket application is usually not written as a raw byte loop. The application supplies a subprotocol object. The WebSocket layer handles the upgraded carrier, frames, messages, and control behavior; the subprotocol object receives lifecycle and message callbacks.

A minimal echo pair needs one subprotocol on each side. The server-side object receives a complete WebSocket message and sends the same message back on the same upgraded connection:

```
#include <web/websocket/server/SubProtocol.h>
#include <log/Logger.h>

#include <cstdint>
#include <cstdint>
#include <string>
```

```

class EchoServer final : public web::websocket::server::SubProtocol {
public:
    EchoServer(web::websocket::SubProtocolContext* context, const std::string& name)
        : web::websocket::server::SubProtocol(context, name, 90, 3) {
    }

private:
    void onConnected() override {
        VLOG(1) << "WebSocket echo server connected";
    }

    void onMessageStart(int) override {
        currentMessage.clear();
    }

    void onMessageData(const char* chunk, std::size_t chunkLen) override {
        currentMessage.append(chunk, chunkLen);
    }

    void onMessageEnd() override {
        VLOG(1) << "WebSocket echo server received: " << currentMessage;
        sendMessage(currentMessage);
        currentMessage.clear();
    }

    void onMessageError(uint16_t errnum) override {
        LOG(WARNING) << "WebSocket echo server message error: " << errnum;
        currentMessage.clear();
    }

    void onDisconnected() override {
        VLOG(1) << "WebSocket echo server disconnected";
        currentMessage.clear();
    }

    bool onSignal(int) override {
        sendClose();
        return false;
    }

    std::string currentMessage;
};

```

The matching client-side object sends one message after the upgrade is complete, logs the echoed response, and then closes the WebSocket connection:

```

#include <web/websocket/client/SubProtocol.h>
#include <log/Logger.h>

#include <cstdint>
#include <cstdint>
#include <string>

class EchoClient final : public web::websocket::client::SubProtocol {
public:
    EchoClient(web::websocket::SubProtocolContext* context, const std::string& name)
        : web::websocket::client::SubProtocol(context, name, 90, 3) {
    }

private:
    void onConnected() override {
        VLOG(1) << "WebSocket echo client connected";
    }

```



```

        sendMessage("hello");
    }

    void onMessageStart(int) override {
        currentMessage.clear();
    }

    void onMessageData(const char* chunk, std::size_t chunkLen) override {
        currentMessage.append(chunk, chunkLen);
    }

    void onMessageEnd() override {
        VLOG(1) << "WebSocket echo client received: " << currentMessage;
        currentMessage.clear();
        sendClose();
    }

    void onMessageError(uint16_t errnum) override {
        LOG(WARNING) << "WebSocket echo client message error: " << errnum;
        currentMessage.clear();
    }

    void onDisconnected() override {
        VLOG(1) << "WebSocket echo client disconnected";
        currentMessage.clear();
    }

    bool onSignal(int) override {
        sendClose();
        return false;
    }

    std::string currentMessage;
};

```

The factories make the subprotocol selectable by name and let the WebSocket upgrade layer create protocol instances without knowing the concrete C++ type in advance. The exported symbol is role-specific:

```

extern "C" web::websocket::SubProtocolFactory<web::websocket::server::SubProtocol>*
echoServerSubProtocolFactory() {
    return new EchoServerFactory("echo");
}

extern "C" web::websocket::SubProtocolFactory<web::websocket::client::SubProtocol>*
echoClientSubProtocolFactory() {
    return new EchoClientFactory("echo");
}

```

A dynamically loaded deployment must install both role modules with the names expected by the selectors:

```

libsndec-websocket-echo-server.so.<SOVERSION>
libsndec-websocket-echo-client.so.<SOVERSION>

```

The base subprotocol and factory shape belongs to the shared WebSocket component, while each concrete role module links the corresponding role component:

```

target_link_libraries(echo-server PRIVATE sndec::websocket-server)
target_link_libraries(echo-client PRIVATE sndec::websocket-client)

```

The factory symbols in the example are not only code conveniences. They are the names that dynamically loaded subprotocol modules must export, and they are also the factory entry points that a linked deployment makes available to the selector. The companion source trees are `WebSocket-Echo-ServerSubprotocol` and `WebSocket-Echo-ClientSubprotocol`; together with `HttpUpgrade-Server` and `HttpUpgrade-Client`, they form the complete runnable WebSocket echo example.

24.11 Lower layers and diagnostics still matter

A WebSocket problem may belong to several layers:

```
lower connection
-> TLS
    -> HTTP upgrade
        -> upgrade factory / selector
            -> WebSocket framing
                -> subprotocol
                    -> application shutdown
```

Chapter 24 adds upgrade-boundary evidence to the earlier diagnostic vocabulary: negotiation, selected subprotocol, frame parsing, control frames, and close behavior. The operational question is not “which single layer failed?”, but where the first observable failure appears in this stack.

24.12 From WebSocket carriers to MQTT semantics

The next chapter moves to MQTT. MQTT can be used directly in `SNode.C`, and Chapter 26 will show how it can also be carried as a WebSocket subprotocol.

The bridge from this chapter is simple:

```
WebSocket
-> upgraded bidirectional message channel

MQTT over WebSocket
-> MQTT semantics carried through that channel
```

WebSocket provides the upgraded carrier. MQTT provides the protocol semantics.

What to remember

- WebSocket begins as HTTP upgrade and continues as a bidirectional message-oriented connection.
- Upgrade changes the protocol context while keeping the underlying connection.
- HTTP upgrade is the generic boundary described in Chapter 21; WebSocket is the concrete upgraded protocol used here.
- `SocketContextUpgrade` is the boundary object between HTTP negotiation and the upgraded protocol context.
- WebSocket uses frames, messages, and control frames such as ping, pong, and close.
- Server and client upgrade paths are related but not identical.
- The shared `websocket` module contains framing and subprotocol infrastructure; server and client modules connect it to the HTTP sides.
- WebSocket is an upgraded carrier; the subprotocol gives carried messages their application meaning.
- Subprotocol factories and selectors keep subprotocol selection structured and extensible.
- A dynamically loaded WebSocket subprotocol must be installed in the WebSocket subprotocol directory and follow the role-specific file-name and factory-symbol conventions.
- A linked subprotocol deployment resolves the same subprotocol name through the selector’s linked factory path instead of opening a shared object at runtime.
- Lower-family, TLS, runtime, configuration, diagnostics, timeout, and failure behavior remain

relevant.

- Chapter 25 moves from web protocols to MQTT.

24.13 WebSocket public surface

WebSocket crosses HTTP negotiation, upgraded socket context, framing, and optional subprotocol selection. The local rule is therefore to include the highest public header for the WebSocket abstraction directly named by the file and link the matching WebSocket component surface. Chapter 32 gives the consolidated component/header matrix, including the MQTT-over-WebSocket adapters.

24.14 Closing perspective

WebSocket completes the web-protocol climb by using HTTP as a negotiation boundary and then moving the connection episode into bidirectional message communication. Chapter 25 turns to MQTT, a message-oriented protocol family that can use SNode.C directly and later reappear over WebSocket.

Part VIII

IoT and Message-Oriented Systems

This part introduces MQTT and then places MQTT inside larger IoT communication structures. Native MQTT, MQTT over Web-Socket, and multi-protocol IoT system design are treated as related boundary choices.

The recurring question is which protocol surface best matches the conversation a system boundary needs to have.

Chapter 25

MQTT Support in SNode.C

25.1 From web protocols to message-oriented communication

Chapter 24 closed the web-protocol climb by showing how HTTP upgrade can move the same lower connection into bidirectional WebSocket communication. Chapter 25 opens the message-oriented part of the book. The focus now moves away from HTTP, routing, event streams, and upgrade negotiation toward MQTT as a packet-structured protocol family.

MQTT is not part of the web stack. It has its own packet vocabulary, session behavior, topic model, keep-alive timing, publish flow, and client/server roles. But the architectural discipline does not change.

That is the central idea of this chapter:

MQTT-layering note

MQTT is not part of the web stack, but it fits the same SNode.C layering discipline.

The shift is therefore twofold:

Part VII

-> web protocols and upgrade

next part

-> message-oriented protocol families

What changes is the application-layer protocol meaning. What remains is the SNode.C model the reader already knows: runtime, configured communication roles, registered runtime-visible instances, lower communication families, stream transport, legacy or TLS connection handling, per-connection contexts, configuration, diagnostics, timing, and failure behavior.

SNode.C supports MQTT in two important forms:

native MQTT

-> MQTT directly above the stream connection model

MQTT over WebSocket

-> MQTT carried as a WebSocket subprotocol

Chapter 25 introduces MQTT as a protocol family and shows how both forms belong to the same architecture. Chapter 26 then narrows the view to MQTT carried as a WebSocket subprotocol.

25.2 MQTT in the layered SNode.C model

The native MQTT layer can be placed into the model the reader already knows:

```

lower communication family
  -> stream transport
      -> legacy or TLS connection handling
      -> MQTT protocol layer
          -> sessions, topics, control packets, publish flow

```

This looks different from HTTP, but the architectural discipline is familiar. The lower layers still provide the communication relationship. MQTT changes the application-layer interpretation of that relationship. Incoming bytes are no longer raised into HTTP requests and responses. They are raised into MQTT control packets, sessions, topics, publish flows, acknowledgements, and keep-alive behavior.

The WebSocket-carried form adds the web-upgrade stack underneath MQTT:

```

HTTP upgrade
  -> WebSocket
      -> MQTT subprotocol
          -> MQTT protocol semantics

```

Therefore, Chapter 25 belongs after Chapter 24. Chapter 24 explained the carrier:

```

WebSocket
  -> upgraded bidirectional message carrier

```

Chapter 25 introduces a protocol family that can use that carrier:

```

MQTT
  -> packet-structured message-oriented protocol semantics

```

MQTT-over-WebSocket is not a new transport trick. It is MQTT semantics carried as a WebSocket subprotocol over an upgraded HTTP connection. The carrier changes. The MQTT protocol identity remains recognizable.

25.3 Native MQTT and MQTT over WebSocket side by side

A compact comparison is enough here; Chapter 26 treats the WebSocket-carried form in detail.

Concern	Native MQTT	MQTT over WebSocket
carrier	stream connection	WebSocket upgraded connection
SNode.C integration	stream <code>SocketContext</code> plus <code>MqttContext</code>	WebSocket subprotocol role plus <code>MqttContext</code>
protocol identity	MQTT directly above stream	MQTT above WebSocket
MQTT meaning	sessions, topics, control packets, publish flow	same MQTT semantics
book focus	introduced in this chapter	treated in detail in Chapter 26

The important point is not that there are two unrelated MQTT implementations. The MQTT semantics remain stable; the carrier path changes.

25.4 MQTT as a protocol family

MQTT support in SNode.C is broader than a broker application or a client helper. The shared MQTT module contains protocol structure. It provides the vocabulary and mechanics that server and client roles build on.

A useful way to read the module is:

```

protocol family
  -> shared vocabulary and mechanics

endpoint roles
  -> server/broker and client specialization

carriers
  -> native stream or WebSocket subprotocol

```

The shared MQTT layer includes concerns such as:

- control packets,
- fixed headers,
- deserialization,
- sessions,
- topics,
- MQTT context,
- native socket-context integration,
- WebSocket subprotocol integration.

At this level, MQTT is treated as a protocol family. Server and client roles then specialize that shared protocol foundation.

Core protocol module

The shared MQTT module provides the common pieces that both server and client roles need.

Shared concern	Meaning
<code>Mqtt</code>	MQTT protocol object and packet-facing behavior
<code>MqttContext</code>	bridge between the MQTT protocol object and the carrier underneath
<code>SocketContext</code>	native stream integration
<code>SubProtocol</code>	WebSocket subprotocol integration
<code>ControlPacket</code>	MQTT packet representation
<code>ControlPacketDeserializer</code>	packet deserialization
<code>FixedHeader</code>	MQTT fixed-header handling
<code>Session</code>	MQTT session state
<code>Topic</code>	topic representation

This shared vocabulary is not an application convenience. It is the protocol foundation from which server/broker-oriented behavior and client behavior are built.

Control packets, fixed headers, sessions, and topics

MQTT should not be described only as “messages.” MQTT is message-oriented in the broad sense, but its protocol unit is a control-packet vocabulary with session, topic, acknowledgement, and keep-alive semantics.

A compact model is:

```

packet shape
  -> control-packet type
      -> session state
          -> topic semantics
              -> acknowledgement flow

```

That is why packet classes, fixed-header handling, deserialization, sessions, and topics belong in the core MQTT module. They are part of the MQTT protocol layer, not incidental application helpers.

25.5 Mqtt as the protocol object

The central code-shaped MQTT object is `iot::mqtt::Mqtt`.

It is the protocol object, not a socket callback and not the complete endpoint by itself. It owns MQTT-level lifecycle, packet delivery, session setup, publish acknowledgement flow, packet identifiers, keep-alive state, and distribution hooks. To become a concrete endpoint, it is connected to a carrier through `MqttContext` and either a native stream `SocketContext` or a WebSocket `SubProtocol`.

A compact view of its responsibilities is:

Mqtt responsibility	Meaning
lifecycle hooks	connected, disconnected, signal handling
packet deserialization	create and deliver control-packet deserializers
session setup	initialize MQTT session state
publish flow helpers	send publish and acknowledgement packets
packet identifiers	allocate packet identifiers
keep-alive	MQTT protocol-level liveness timing
distribution hook	distribute received publish packets

The lower connection delivers bytes. The MQTT object gives those bytes protocol meaning.

Lifecycle and packet delivery

MQTT has lifecycle behavior. It reacts to connection and disconnection, and it handles packet creation and delivery.

Conceptually:

```

connection event
  -> MQTT lifecycle hook

incoming bytes
  -> fixed header
      -> control-packet deserializer
          -> MQTT packet delivery

```

The creation of a packet deserializer and the delivery of the completed packet are specialized by the server and client sides. That keeps shared MQTT framing and packet flow in the protocol core while still allowing broker-oriented and client-oriented behavior to differ.

Application code should not have to manually switch on raw incoming MQTT byte sequences inside an unrelated socket callback. Packet interpretation belongs in the MQTT layer.

Publish and acknowledgement flow

MQTT has explicit packet vocabulary. For example, the core object exposes helpers and hooks around:

Packet helper/hook	Meaning
<code>sendPublish</code> / <code>onPublish</code>	publish flow
<code>sendPuback</code> / <code>onPuback</code>	publish acknowledgement
<code>sendPubrec</code> / <code>onPubrec</code>	QoS 2 receive step
<code>sendPubrel</code> / <code>onPubrel</code>	QoS 2 release step
<code>sendPubcomp</code> / <code>onPubcomp</code>	QoS 2 completion step

It uses the packet names to show structure, not to teach MQTT QoS in detail. The architectural point is smaller and more important for this part of the book:

MQTT is packet-structured,
not just a generic byte stream.

The explicit packet vocabulary belongs in the MQTT layer.

Session and keep-alive timing

MQTT owns some of its own state and timing. A useful distinction is:

```
session
  -> MQTT relationship state

keep-alive
  -> MQTT protocol-level liveness timing
```

This connects directly to Chapter 20. Chapter 20 separated transport timeouts, retry/reconnect delay, and protocol-level timing. MQTT keep-alive is another example of protocol-level timing.

The lower connection may have read or write timeouts. A configured client or server instance may have retry or reconnect policy. TLS may have handshake and shutdown timing. MQTT adds its own liveness meaning at the protocol layer. Keep-alive is therefore not just a socket timeout with another name.

25.6 Native MQTT over stream connections

Native MQTT uses the lower stream architecture directly. Its code shape is:

```
core::socket::stream::SocketContext
+ iot::mqtt::MqttContext
  -> iot::mqtt::SocketContext
```

This tells the reader how MQTT fits into the earlier context model. The native MQTT endpoint is still a stream socket context, but its receive/send lifecycle is routed through the MQTT-facing context.

SocketContext plus MqttContext

The native MQTT `SocketContext` combines two sides.

Side	Meaning
stream <code>SocketContext</code>	connection lifecycle, receive, send, close, signal handling
<code>MqttContext</code>	bridge between the MQTT protocol object and the carrier underneath
MQTT <code>SocketContext</code>	MQTT-aware endpoint over a native stream connection

`MqttContext` is not a replacement for the lower socket context. It is the bridge that lets the protocol object read from, write to, end, or close whichever carrier is used underneath.

A useful way to read the composition is:

```
transport-facing context
+ MQTT-facing context
  -> MQTT-aware connection endpoint
```

That composition is important. Native MQTT does not replace the lower architecture. It plugs MQTT protocol behavior into the normal per-connection context model.

MQTT as protocol layer, not transport replacement

Native MQTT still sits above:

- a lower communication family,
- stream transport,
- legacy or TLS connection handling,
- runtime-driven lifecycle,
- configuration,
- diagnostics.

The difference is the application-layer interpretation. At the HTTP layer, incoming data becomes request/response meaning. At the MQTT layer, incoming data becomes MQTT packet, session, topic, acknowledgement, and keep-alive meaning.

The structure changes at the protocol level. The lower transport architecture remains.

25.7 Server and client specialization

The MQTT module separates shared protocol infrastructure from role-specific behavior.

A useful module view is:

Module/library	Meaning
<code>mqtt</code>	shared MQTT protocol core
<code>mqtt-server</code>	server/broker-oriented role layer
<code>mqtt-client</code>	client role layer
<code>mqtt-server-websocket</code>	server-side MQTT WebSocket subprotocol
<code>mqtt-client-websocket</code>	client-side MQTT WebSocket subprotocol

This mirrors earlier patterns in the book. There is a shared protocol core. Server and client roles specialize that core. WebSocket-carried forms then connect the same protocol family to the upgrade/subprotocol architecture from Chapter 24.

In the refined instance vocabulary, this means that an application-side server or client handle configures a communication role, which becomes a registered runtime-visible instance. The per-connection MQTT-aware context and the MQTT protocol object then carry protocol behavior inside that established framework shape.

Server as broker-oriented role

The server-side MQTT role derives from the shared MQTT protocol object and connects it to broker-oriented behavior, rather than acting as a listener that only parses MQTT bytes.

It handles concerns such as:

- connect handling,
- subscriptions,
- unsubscriptions,
- ping requests,
- disconnects,
- publish distribution,
- broker state,
- shared socket-context factories,
- server-side WebSocket subprotocol support.

This is the right place for broker-oriented behavior. Broker behavior does not belong in the shared protocol core as a hidden global assumption. The shared core provides packet, session, context, and topic vocabulary; the server role connects that vocabulary to broker-specific behavior.

Client as protocol participant

The client side is also a real protocol participant, not a transport wrapper that sends MQTT-looking bytes.

It handles client-side packet flow such as:

- CONNACK,
- SUBACK,
- UNSUBACK,
- PINGRESP,
- publish distribution,

and it exposes client-originated operations such as:

- CONNECT,
- SUBSCRIBE,
- UNSUBSCRIBE,
- PINGREQ,
- DISCONNECT.

This keeps the client/server model balanced. Both sides share the MQTT protocol family. Each side has its own role-specific behavior.

25.8 A compact MQTT client role

A compact MQTT client example should show the protocol role, not an entire transport setup. The following class expresses the MQTT side of the application: connect, subscribe, publish, receive publishes, and disconnect on shutdown.

```
#include <iot/mqtt/client/Mqtt.h>
#include <iot/mqtt/packets/Connack.h>
#include <iot/mqtt/packets/Publish.h>
#include <iot/mqtt/Topic.h>
#include <log/Logger.h>

#include <string>

class SensorClient final : public iot::mqtt::client::Mqtt {
public:
    SensorClient()
        : iot::mqtt::client::Mqtt(
            "sensor-mqtt",
            "sensor-client-1",
            60,
            "sensor-client.session") {
    }

private:
    void onConnected() override {
        sendConnect(
            true,           // clean session
            "",             // will topic
            "",             // will message
            0,              // will QoS
            false,          // will retain
            "",             // username
            "",             // password
            false);         // loop prevention
    }

    void onConnack(const iot::mqtt::packets::Connack&) override {
        sendSubscribe({
            iot::mqtt::Topic("sensors/+/command", 0)
        })
    }
}
```

```

    });

    sendPublish(
        "sensors/temperature/value",
        "23.5",
        0,
        false);
}

void onPublish(const iot::mqtt::packets::Publish& publish) override {
    const std::string topic = publish.getTopic();
    const std::string message = publish.getMessage();

    VLOG(1) << "MQTT command on " << topic << ": " << message;
}

bool onSignal(int) override {
    sendDisconnect();
    return false;
}
};

```

The example deliberately omits the concrete lower connection setup. That setup decides how the MQTT role is attached to a stream or to another carrier. The MQTT role itself shows the protocol behavior: it sends `CONNECT`, waits for `CONNACK`, subscribes, publishes, handles incoming publishes, and sends `DISCONNECT` during shutdown. The complete companion role example is named `MQTT-ClientRole`.

The corresponding native MQTT client component is:

```
target_link_libraries(my_mqtt_client PRIVATE snodec::mqtt-client)
```

The shared `mqtt` component is the protocol core. The `mqtt-client` component adds the client role on top of that core. The component exists only when the MQTT build prerequisites are available.

25.9 MQTT as a WebSocket subprotocol

MQTT-over-WebSocket is where Chapter 25 connects back to Chapter 24. The relevant composition is:

```

WebSocket subprotocol role
+ MqttContext
-> MQTT-over-WebSocket endpoint

```

Native MQTT uses a stream `SocketContext` plus `MqttContext`. MQTT-over-WebSocket changes the carrier-facing side to a WebSocket subprotocol role, while the MQTT-facing context remains recognizable. That is the symmetry Chapter 26 develops in detail.

Chapter 25 only establishes the model: MQTT remains MQTT, while the carrier can be native stream or WebSocket.

25.10 MQTT public surface: protocol headers and components

MQTT code includes the MQTT abstraction it directly names. A client-side MQTT protocol object is introduced through:

```
#include <iot/mqtt/client/Mqtt.h>
```

Shared support remains below `<iot/mqtt/...>` for topics, packets, socket-context bridging, and protocol support. The build-side components distinguish shared support, native roles, and WebSocket-carried compositions; Chapter 32 collects those mappings in one source-derived table.

25.11 Build/component note: JSON dependency is not MQTT identity

In the current SNode.C build, the MQTT component is enabled when `nlohmann_json >= 3.11` is found; otherwise CMake emits a warning and does not add the MQTT targets.

That is a build/component fact. It should not be confused with the conceptual definition of MQTT.

MQTT should first be understood through:

- sessions,
- topics,
- control packets,
- publish flow,
- client/server roles,
- native and WebSocket-carried forms.

Structured payloads and integration systems may use JSON. The current build uses `nlohmann_json` as a component dependency. But JSON is not the conceptual center of MQTT.

25.12 MQTT and IoT-facing systems

MQTT is the first major protocol family in the book that clearly points toward IoT and machine-to-machine messaging. That does not mean MQTT is only useful in small devices. It means MQTT is well suited to systems where many participants exchange state, telemetry, commands, or events through topics.

In SNode.C, this makes MQTT a natural bridge toward later system-level discussions:

```
devices
-> topics
    -> broker-oriented distribution
        -> protocol integration
            -> dashboards and applications
```

This is also where the diagnostic model remains important. MQTT problems may involve:

- lower endpoint selection,
- TLS configuration,
- stream lifecycle,
- retry or reconnect behavior,
- MQTT session state,
- keep-alive behavior,
- packet deserialization,
- publish and acknowledgement flow,
- broker distribution,
- WebSocket subprotocol selection when MQTT is carried over WebSocket.

The topic model and broker-oriented distribution make MQTT useful for IoT-facing systems. The SNode.C layering model keeps those systems explainable.

Chapter 27 will later combine these ideas with multiple protocols and IoT system design. For now, Chapter 25 keeps the focus on MQTT as a protocol family inside SNode.C.

What to remember

- MQTT opens the message-oriented part of the book.
- MQTT is not a web protocol, but it fits the same SNode.C layering discipline.
- MQTT is a protocol family with shared packet, session, topic, context, and keep-alive concerns.
- `Mqtt` is the MQTT protocol object, not just a socket callback.
- `MqttContext` bridges the MQTT protocol object to the carrier underneath.

- Native MQTT combines stream `SocketContext` with `MqttContext`.

25.13 Closing perspective

Chapter 25 introduced MQTT as a message-oriented protocol family in SNode.C.

Chapter 25 established MQTT as a protocol family and introduced its two carrier forms. Chapter 26 now narrows the view to one specific composition:

Chapter 26

MQTT over WebSocket

26.1 MQTT carried by the WebSocket upgrade stack

Chapter 25 introduced MQTT as a message-oriented protocol family in SNode.C. It showed two carrier forms:

```
native MQTT
  -> MQTT above a stream connection

MQTT over WebSocket
  -> MQTT above a WebSocket subprotocol
```

Chapter 26 narrows the view to the second form and follows the complete stack: HTTP upgrade, WebSocket, selected subprotocol role, `MqttContext`, and MQTT protocol semantics.

The central sentence is:

MQTT-over-WebSocket note

MQTT over WebSocket in SNode.C is MQTT expressed as a WebSocket subprotocol on top of the HTTP upgrade stack.

This sentence defines the composition precisely. MQTT-over-WebSocket is not ordinary native MQTT with a few additional HTTP headers. The HTTP headers belong to the upgrade negotiation. After a successful upgrade, MQTT packet data is carried as WebSocket message payload and interpreted by the MQTT layer.

It is also not ordinary HTTP, and it is not plain WebSocket without higher protocol meaning. It is a layered composition:

```
HTTP upgrade
  -> WebSocket
      -> WebSocket subprotocol
          -> MQTT semantics
```

This chapter depends on two earlier steps. Chapter 24 explained the WebSocket side:

```
HTTP upgrade
  -> WebSocket
      -> subprotocol
```

Chapter 25 explained the MQTT side:

```
MQTT
  -> MqttContext
```

-> packet, session, topic, keep-alive, and publish-flow semantics

Chapter 26 combines them:

```
WebSocket subprotocol role
+ MqttContext
  -> MQTT-over-WebSocket endpoint
```

Each layer still has a responsibility. HTTP negotiates the upgrade. WebSocket provides the upgraded bidirectional message carrier. The WebSocket subprotocol role gives that carrier a selected protocol surface. `MqttContext` bridges the carrier to the MQTT protocol object. MQTT provides packet, session, topic, keep-alive, and publish-flow semantics.

No layer disappears.

That is the main point of the chapter.

26.2 The composition in one model

The full stack can be read from bottom to top:

```
lower communication family
-> stream transport
  -> legacy or TLS connection handling
    -> HTTP request / response
      -> HTTP upgrade
        -> WebSocket
          -> WebSocket subprotocol role
            -> MQTT protocol semantics
```

This stack combines several layers that were introduced separately:

- lower communication families,
- stream transport,
- legacy and TLS connection handling,
- HTTP request/response,
- generic protocol upgrade,
- WebSocket frames and subprotocols,
- MQTT as a message-oriented protocol family.

The result is not a flattened protocol name. It is a composed stack whose layers remain architecturally and diagnostically visible.

A useful way to read the stack is:

```
same MQTT semantics
-> different carrier path
```

Native MQTT and MQTT-over-WebSocket preserve the same MQTT protocol identity. What changes is the carrier path that delivers bytes into the MQTT receive flow.

That distinction matters. It prevents the reader from treating MQTT-over-WebSocket as a special-case shortcut or as a second unrelated MQTT implementation. It is a composition of already introduced mechanisms.

26.3 Native MQTT and MQTT-over-WebSocket side by side

Chapter 25 introduced native MQTT first. Chapter 26 sharpens the carrier contrast.

Figure 26.1 shows the distinction. Native MQTT writes MQTT packets directly to a stream or TLS stream. MQTT over WebSocket reaches the same MQTT packet and session semantics through an

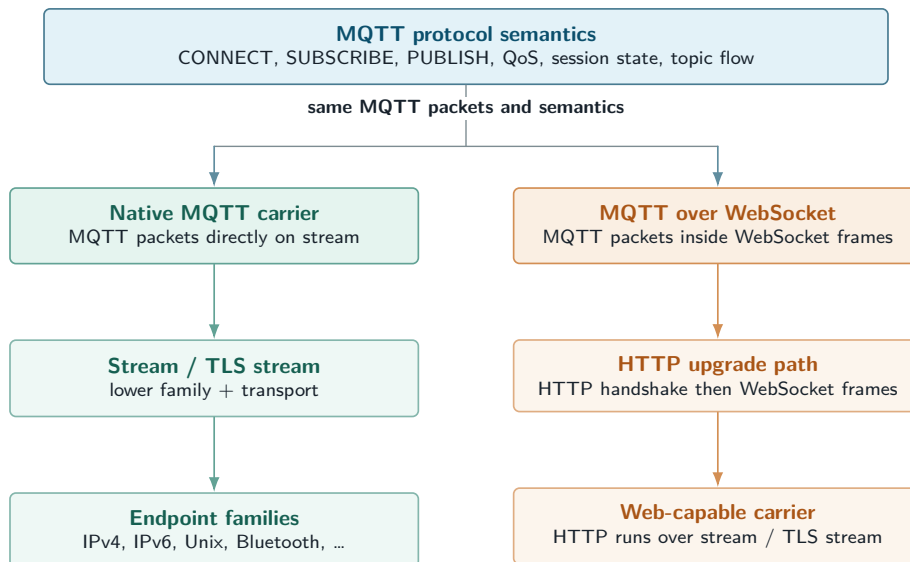


Figure 26.1: Native MQTT and MQTT over WebSocket share MQTT protocol semantics but use different carrier paths.

HTTP upgrade path and WebSocket frames. The point is the carrier contrast, not a split in MQTT semantics.

Concern	Native MQTT	MQTT over WebSocket
carrier	stream connection	WebSocket connection after HTTP upgrade
endpoint composition	stream <code>SocketContext</code> plus <code>MqttContext</code>	WebSocket subprotocol role plus <code>MqttContext</code>
HTTP layer	absent	used for upgrade negotiation
WebSocket layer	absent	present and still meaningful
MQTT semantics	sessions, packets, topics, keep-alive, publish flow	same MQTT semantics
failure surface	stream and MQTT layers	HTTP upgrade, WebSocket, subprotocol, and MQTT layers

The central message is simple: MQTT semantics stay MQTT; the carrier path changes.

26.4 Why this carrier composition matters

MQTT-over-WebSocket exists for systems that want MQTT semantics while using a WebSocket-capable communication path. That can matter when communication already lives inside web-facing infrastructure, when an HTTP upgrade path is available, or when MQTT participates in a system that otherwise uses HTTP and WebSocket boundaries.

For this book, the architectural point is more important than any single deployment reason: protocol meaning can remain recognizable while the carrier changes. Chapter 26 shows that idea above the web stack.

26.5 The MQTT-over-WebSocket subprotocol type

The code-shaped center of this chapter is the generic MQTT WebSocket subprotocol type.

In simplified form, its shape is:

```
template <typename WSSubProtocolRoleT>
```

```
class SubProtocol
: public WSSubProtocolRoleT
, private iot::mqtt::MqttContext {
// MQTT over WebSocket bridge
};
```

This is the chapter in code form.

The class shape is the architecture expressed in C++. The WebSocket role supplies the upgraded carrier surface. `MqttContext` supplies the MQTT-facing receive/send/end/close bridge. The combined type becomes an MQTT-over-WebSocket endpoint.

Part	Meaning
<code>WSSubProtocolRoleT</code>	server-side or client-side WebSocket subprotocol role
<code>MqttContext</code>	MQTT-facing receive/send/end/close bridge
<code>SubProtocol<WSSubProtocolRoleT></code>	MQTT protocol behavior carried over a WebSocket subprotocol
<code>OnReceivedFromPeerEvent</code>	scheduling hook that feeds buffered WebSocket payload into MQTT receive processing

This is not a second MQTT implementation. It is MQTT protocol behavior attached to a different carrier.

WebSocket role plus `MqttContext`

The template parameter keeps the WebSocket role open. The same generic MQTT-over-WebSocket adapter can be used with a server-side WebSocket subprotocol role or with a client-side WebSocket subprotocol role.

The composition can be read as:

```
WebSocket role
-> connection, frame, message, and subprotocol carrier behavior

MqttContext
-> MQTT-facing receive/send/end/close behavior

MQTT SubProtocol
-> protocol bridge between both sides
```

The WebSocket role does not become MQTT by itself. It remains the carrier-facing side of the adapter. The MQTT protocol object does not become WebSocket-specific by itself. It sees the carrier through `MqttContext`.

This separation is what keeps the composition clean.

WebSocket message flow into MQTT receive flow

WebSocket is a message-oriented carrier. MQTT is a byte-oriented packet protocol. The adapter has to bridge those two views.

A useful model is:

```
WebSocket message callbacks
-> MQTT subprotocol buffer
    -> MqttContext recv()
        -> MQTT packet deserialization
            -> MQTT packet handling
```

The WebSocket side reacts to message start, message data, message end, and message errors. MQTT processing then receives buffered bytes through the `MqttContext` receive path.

In source-shaped terms:

```
onMessageData(...)
  -> accumulate WebSocket payload data

onMessageEnd()
  -> move accumulated payload into the MQTT receive buffer
  -> schedule MQTT receive processing

recv(...)
  -> present buffered bytes to the MQTT protocol object
```

This is why the adapter needs both sides. WebSocket framing and MQTT packet semantics remain distinct. WebSocket decides how payload arrives. MQTT decides what the payload means.

There is also an important binary/text distinction. MQTT packet data is byte-oriented. In this adapter, text WebSocket messages are not the MQTT-over-WebSocket path. MQTT data is processed through the binary message path into the MQTT receive buffer. A text frame is therefore not simply “another way to carry MQTT”; it belongs to the wrong WebSocket message type for this adapter.

Runtime event integration

The MQTT-over-WebSocket adapter has to fit into the event-driven runtime and the static type composition.

Runtime integration is explicit. The adapter owns an `OnReceivedFromPeerEvent` that schedules MQTT receive processing after WebSocket message data has been buffered. This keeps MQTT packet processing inside the same event-driven runtime model as the rest of `SNode.C`.

The subprotocol therefore participates in several runtime-facing actions:

- connection handling,
- disconnection handling,
- signal handling,
- message-start handling,
- message-data handling,
- message-end handling,
- message-error handling,
- scheduled peer-data processing.

The important point is:

```
WebSocket delivers protocol data over time.
MQTT processing must be scheduled inside the same event-driven runtime.
```

The adapter keeps that relationship explicit.

Send, end, and close

The bridge also works in the other direction. MQTT-facing output has to become WebSocket output.

Conceptually:

```
MqttContext send(...)
  -> WebSocket sendMessage(...)

MqttContext end()
  -> WebSocket close handshake

MqttContext close()
  -> WebSocket protocol-error close
```

The exact protocol mechanics belong to the implementation. The architectural point is that `MqttContext` gives MQTT a stable carrier-facing surface, while the subprotocol adapter maps that surface to WebSocket behavior.

26.6 Server-side and client-side aliases

The generic MQTT WebSocket subprotocol type is specialized through simple role aliases.

On the server side:

```
using SubProtocol =
    iot::mqtt::SubProtocol<web::websocket::server::SubProtocol>;
```

On the client side:

```
using SubProtocol =
    iot::mqtt::SubProtocol<web::websocket::client::SubProtocol>;
```

The aliases do not duplicate the MQTT-over-WebSocket adapter. They bind the same generic adapter to either the server-side or client-side WebSocket subprotocol role.

Alias side	Meaning
server alias	MQTT over a server-side WebSocket subprotocol role
client alias	MQTT over a client-side WebSocket subprotocol role
shared template	common MQTT-over-WebSocket bridge logic

The MQTT bridge logic is shared. The WebSocket role type keeps side-specific behavior explicit.

This keeps reuse and role clarity together. The server/client distinction is not hidden, but the protocol bridge is not duplicated.

26.7 Each layer keeps its responsibility

MQTT-over-WebSocket is easy to misunderstand if the stack is flattened. Each layer has a different job.

Layer	Responsibility
lower family / stream	peer communication path
TLS, if used	secure connection handling
HTTP	upgrade negotiation
WebSocket	upgraded bidirectional message carrier, framing, binary payload delivery, and control behavior
WebSocket subprotocol role	selected protocol surface above WebSocket
MQTT	packet, session, topic, keep-alive, and publish-flow semantics

This table is the chapter's main operational reminder.

MQTT does not dissolve into WebSocket. WebSocket does not disappear under MQTT. HTTP still matters because it negotiated the upgrade. The lower connection still matters because it carries the whole stack.

This is exactly why the layered model is useful. It gives each concern a place.

26.8 Native MQTT and MQTT-over-WebSocket as sibling compositions

Native MQTT and MQTT-over-WebSocket are sibling compositions of the same MQTT protocol family.

```
native MQTT
  -> lower stream connection
    -> MQTT

MQTT over WebSocket
  -> HTTP upgrade
    -> WebSocket
      -> WebSocket subprotocol
        -> MQTT
```

The carrier-facing side changes. The MQTT-facing bridge remains recognizable.

26.9 Build artifacts mirror the composition

The build structure mirrors the architectural structure.

Artifact	Meaning
<code>mqtt-server</code>	native / broker-oriented server MQTT role
<code>mqtt-client</code>	native client MQTT role
<code>mqtt-server-websocket</code>	server-side MQTT WebSocket subprotocol
<code>mqtt-client-websocket</code>	client-side MQTT WebSocket subprotocol

MQTT-over-WebSocket is therefore not an application trick outside the framework. It has an explicit component surface for the server and client sides of the composition.

26.10 MQTT-over-WebSocket as component selection

MQTT-over-WebSocket should not repeat the MQTT client example from Chapter 25. The important point here is the carrier selection. The MQTT role remains an MQTT role; the WebSocket-carried variant adds the WebSocket subprotocol component that lets MQTT packets travel through an HTTP/WebSocket upgrade path.

A compact client-side build fragment therefore looks like this:

```
target_link_libraries(gateway
  PRIVATE
    snodec::mqtt-client
    snodec::mqtt-client-websocket
    snodec::websocket-client)
```

The native MQTT component expresses the MQTT client role. The WebSocket-carried component expresses the MQTT/WebSocket adapter. The WebSocket client component expresses the upgraded carrier side. This is a component-level statement of the same architecture described in prose:

```
MQTT client role
+ MQTT-over-WebSocket subprotocol
  + WebSocket client carrier
    -> MQTT semantics over an upgraded HTTP/WebSocket connection
```

The server side follows the same rule in the opposite role:

```
target_link_libraries(gateway
PRIVATE
    snodec::mqtt-server
    snodec::mqtt-server-websocket
    snodec::websocket-server)
```

The two fragments do not define a different MQTT API. They select a different carrier composition for the same MQTT protocol family.

26.11 Diagnostics across the composed stack

A failure in MQTT-over-WebSocket may belong to several layers.

It may be caused by:

- lower connection establishment,
- TLS configuration,
- HTTP upgrade negotiation,
- WebSocket framing or control behavior,
- binary/text message mismatch,
- subprotocol selection,
- WebSocket message data not reaching the MQTT receive buffer,
- MQTT packet parsing,
- MQTT session behavior,
- MQTT keep-alive behavior,
- application shutdown.

This is not a weakness. It is the reality of a composed stack.

The diagnostic question is:

At which layer did the failure occur?

A second question is just as important:

At which layer does this meaning belong?

Examples:

```
Could not connect
-> lower endpoint, transport, TLS, or configuration

Upgrade rejected
-> HTTP / WebSocket upgrade boundary

Subprotocol not selected
-> WebSocket subprotocol negotiation

Text frame appeared
-> wrong WebSocket message type for this MQTT adapter

Frame error
-> WebSocket layer

Payload not delivered to MQTT recv()
-> WebSocket-to-MQTT adapter boundary

Malformed MQTT packet
-> MQTT protocol layer

Session or keep-alive problem
```

-> MQTT lifecycle layer

There are also timing concerns at several layers:

```
transport or TLS timing
  -> lower connection behavior

HTTP upgrade timing
  -> upgrade boundary

WebSocket close / control behavior
  -> WebSocket lifecycle

MQTT keep-alive timing
  -> MQTT protocol liveness
```

This is where the earlier chapters on diagnostics, timeouts, and failure modes remain useful. MQTT-over-WebSocket does not need a new operational philosophy. It needs the existing layered diagnostic model applied to a deeper stack.

26.12 From one composed stack to multi-protocol systems

MQTT-over-WebSocket matters because it connects MQTT semantics with web-upgrade infrastructure. That makes it a useful example before moving to broader IoT system design.

Chapter 26 shows one concrete cross-stack composition:

```
MQTT
  -> carried as WebSocket subprotocol
      -> reached through HTTP upgrade
```

Chapter 27 widens the view from one composed stack to systems that combine several protocol families, carriers, gateways, dashboards, and integration points.

A single composed stack may look like this:

MQTT over WebSocket

A multi-protocol system may combine:

- MQTT,
- HTTP,
- WebSocket,
- Bluetooth or local links,
- lower IP or local communication choices,
- dashboards,
- gateways,
- sensors,
- brokers,
- integrations.

The next chapter is therefore not only about another protocol. It is about system composition.

What to remember

- MQTT-over-WebSocket carries MQTT as a WebSocket subprotocol.
- HTTP still matters because it negotiates the upgrade.
- WebSocket still matters because it provides the upgraded bidirectional message carrier.
- The WebSocket subprotocol role supplies the carrier-facing side of the adapter.
- `MqttContext` supplies the MQTT-facing receive/send/end/close bridge.

- The generic MQTT WebSocket subprotocol template is reused for server and client aliases.
- MQTT packet data is byte-oriented and is carried through the WebSocket message path into MQTT receive processing.
- Native MQTT and MQTT-over-WebSocket are sibling compositions of the same MQTT protocol family.
- The carrier path changes; MQTT packet, session, topic, keep-alive, and publish-flow semantics remain recognizable.
- Failures can belong to lower connection, TLS, HTTP upgrade, WebSocket framing, subprotocol selection, MQTT parsing, MQTT session, or keep-alive behavior.
- Chapter 27 moves from one protocol composition to multi-protocol IoT system design.

26.13 Native MQTT and MQTT-over-WebSocket public surfaces

Native MQTT files include the MQTT headers they directly use. MQTT-over-WebSocket files include the WebSocket-carried MQTT abstraction they directly name. On the build side, the distinction is explicit:

```
native MQTT:
  mqtt-client
  mqtt-server

MQTT carried by WebSocket:
  mqtt-client-websocket
  mqtt-server-websocket
```

Chapter 32 gives the consolidated source-derived component/header matrix.

26.14 Closing perspective

MQTT-over-WebSocket is a concrete cross-stack composition: HTTP negotiates the upgrade, WebSocket carries messages, the subprotocol selects MQTT, and `MqttContext` bridges the carrier to the MQTT protocol object. Chapter 27 widens the view from one composed stack to systems that combine several protocol families at once.

Chapter 27

Designing IoT Systems with Multiple Protocols

27.1 From protocol composition to system boundaries

Chapter 26 showed one carefully composed stack:

```
HTTP upgrade
-> WebSocket
    -> WebSocket subprotocol
        -> MQTT
```

Chapter 27 widens the lens from one composed stack to a whole IoT system. In such a system, several protocol stacks may coexist because they serve different communication boundaries. The central sentence is:

Multi-protocol boundary rule

A multi-protocol IoT system is clear when each protocol is assigned to an explicit boundary.

This chapter is not about using every protocol everywhere. It is about deciding where each protocol belongs. Chapter 25 introduced MQTT as a protocol family. Chapter 26 showed MQTT over WebSocket as one composed stack. Chapter 27 now treats multi-protocol IoT systems as boundary design.

The earlier chapters introduced the pieces one at a time: lower communication families, Unix domain sockets, Bluetooth RFCOMM and L2CAP, HTTP, the Express-like layer, Server-Sent Events, WebSocket, MQTT, MQTT over WebSocket, TLS, configuration, diagnostics, timeouts, and failure behavior. The question is no longer only:

How does this protocol work?

The question becomes:

Which system boundary does this protocol serve?

That question is the main design rule of this chapter: boundary first, protocol second.

27.2 IoT systems are boundary systems

IoT architecture is not defined by sensors alone. Sensors, boards, radio modules, and local devices matter, but they do not define the communication architecture by themselves. The architecture is defined by the points where information crosses from device-near code to local services, from local services to integration

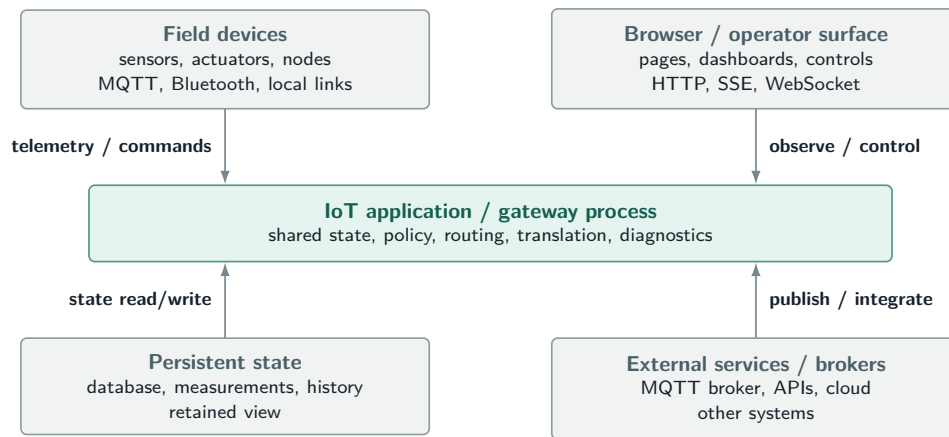


Figure 27.1: An IoT system as a constellation of protocol and state boundaries around an application or gateway process.

infrastructure, from integration infrastructure to user interfaces, and from live state into persistent or external systems.

Scientific and environmental data-collection systems are typical examples of this shape. Instruments or field nodes produce measurements, local services buffer or pre-process them, gateways forward them, MQTT may carry integration traffic, HTTP or SSE surfaces may expose observation and status, persistence stores long-running data, and a management HTTP instance may provide configuration or operational control. The domain may be biology, environmental monitoring, lab instrumentation, field sensing, or another measurement area; the architectural question remains how to keep the boundaries explicit.

Figure 27.1 shows an IoT system as a constellation of boundaries rather than as a single protocol chain. Field devices, browser/operator surfaces, persistent state, and external services all meet at the application or gateway process. The diagram frames boundary placement, not executable topology: one process may own several boundaries, and one boundary may be realized by more than one deployable part. The important design question is where each boundary belongs and which protocol surface is appropriate for that boundary.

Each connector in Figure 27.1 is a boundary. Each boundary may need a different protocol family. Multi-protocol design is therefore normal in IoT systems. It is not automatically a sign of complexity gone wrong. The design becomes unclear only when protocol choices are placed dishonestly: when a protocol is used because it is fashionable, convenient, or already present, rather than because it serves the boundary well.

A clear IoT design starts by naming the boundaries. Only then should it choose protocols.

27.3 Recurring boundary roles in IoT systems

A useful starting point is to name recurring boundary roles. In this chapter, a boundary role is a design position in the system. It is not automatically a fixed framework class. A concrete SNode.C application may realize such a role through one or more configured communication roles and registered runtime-visible instances.

Boundary role	Question
device-facing role	How does the system talk to device-near components?
local-control role	How do same-host processes coordinate?
integration role	How does the system exchange machine messages?
observation role	How do humans or monitoring consumers see state changes?
administration role	How is the system configured, controlled, and inspected?

These roles help discussion; they do not force implementation structure. A small application may combine several of them. A larger deployment may split them across several processes. The important point is that the roles are different conversations.

Device-facing role

The device-facing role talks toward hardware or device-near components. This may involve Bluetooth RFCOMM, Bluetooth L2CAP, a custom stream protocol, a serial or local helper process, or another device-near boundary.

This role is usually close to physical constraints. It may care about commissioning, local range, device identity, pairing, sampling, or hardware-specific timing. Device-facing does not automatically mean globally reachable, brokered, or web-facing. It means that the system is close to the physical edge and should not pretend that this boundary has the same shape as a dashboard, a broker, or an administration API.

Local-control role

The local-control role connects cooperating processes on the same machine. It may be used for command tools, helper processes, service supervision, local administration, process separation, or device-facing adapters.

Unix domain sockets are often a clean fit here. They express a local process boundary without pretending that the interaction is a remote network service. Not every local interaction needs HTTP, MQTT, or a TCP port. Sometimes the explicit boundary is local, same-host, and intentionally not exposed beyond that host.

Integration role

The integration role exchanges machine messages. MQTT is often a strong fit here because it provides publish/subscribe flow, topic-based routing, brokered distribution, decoupled producers and consumers, and machine-to-machine message exchange.

This role can become the integration spine of the system. That does not mean it should become every other boundary. The integration spine is powerful precisely because it has a clear job: machine-facing message exchange.

Observation role

The observation role exposes state and change to humans, dashboards, or monitoring consumers. It may involve HTTP status endpoints, SSE streams, WebSocket connections, dashboard views, metrics pages, or activity feeds.

Observation is not the same as control. It often needs a live or near-live view, but it does not necessarily need bidirectional command semantics. A dashboard that watches state changes has a different boundary shape from a control session that modifies state.

Administration role

The administration role configures, controls, and inspects the system. It may involve HTTP/Express endpoints, authentication middleware, command-line configuration, local Unix-domain control, diagnostic pages, or management APIs.

This role is often operator-facing. It should be explicit because administration errors can affect the whole system. A system that hides administration behind an accidental data path is harder to reason about and harder to secure operationally.

27.4 Choosing protocol families from boundary needs

A good protocol choice starts with the boundary's shape: local or remote, one-way or bidirectional, human-facing or machine-facing, transient or durable, brokered or point-to-point. A boundary-oriented

design asks what each boundary needs before choosing a protocol.

A compact mapping is:

System boundary	Typical protocol family	Reason
device-near edge	Bluetooth RFCOMM/L2CAP, custom stream service	close to hardware or local device exchange
local control	Unix domain sockets	same-host control, helper processes, local security
integration spine	MQTT	brokered machine-to-machine messaging
management surface	HTTP / Express-like layer	structured operator APIs and dashboards
live observation	SSE	server-to-client updates
interactive browser link	WebSocket	bidirectional live interaction
bridge boundary	MQTT over WebSocket	MQTT semantics through a WebSocket path
persistence boundary	database/client integration	durable state or external service state

This table is not a rulebook. It is a starting point. The decision should always come back to the boundary:

```

What crosses this boundary?
Who consumes it?
How long does it live?
Does it need bidirectional flow?
Does it need brokered distribution?
Does it need local-only access?
Does it need operator visibility?
Is the data transient or durable?
Does it need persistence?

```

Those questions usually lead to a clearer protocol choice than starting with a favorite protocol and forcing it everywhere.

27.5 A constellation of protocol stacks

An IoT system in SNode.C is often not one tall vertical stack. It is a constellation of smaller stacks that meet at domain state, integration points, dashboards, local control paths, or persistence boundaries.

For example:

```

device edge stack
  -> Bluetooth / custom stream

integration stack
  -> MQTT

admin stack
  -> HTTP / Express

observation stack
  -> SSE

interactive stack
  -> WebSocket

local-control stack
  -> Unix domain socket

```

These stacks are not redundant duplicates. They are different conversations at different system boundaries. That is the main reason SNode.C can be useful in larger IoT designs.

SNode.C's component structure reflects this breadth: lower-family stream components, HTTP/Express, WebSocket, MQTT, MQTT-over-WebSocket, database support, and IoT modules are separate but architecturally related pieces. The protocol surfaces can differ, but the framework keeps a shared architectural language: runtime lifecycle, configured roles, registered instances, contexts, factories, configuration, logging, diagnostics, timeouts, and failure handling.

The result is not protocol uniformity. The result is architectural coherence across protocol diversity.

27.6 Telemetry, control, observation, and administration are different conversations

Many weak IoT architectures become confusing because they collapse several conversations into one channel. A stronger design keeps the differences visible.

Conversation	Meaning
telemetry	reports measured or derived state
control	asks the system or a device to do something
observation	presents state or change to a human or monitoring consumer
device exchange	talks to hardware or device-near services
administration	changes how the system is operated

These conversations may share data, but they should not automatically share the same protocol boundary. The same domain fact may cross several boundaries because each consumer needs a different conversation.

For example:

```
sensor reading
  -> device-facing exchange
      -> internal state update
          -> MQTT telemetry publish
              -> SSE dashboard update
                  -> database persistence
```

This is one piece of information moving through several roles. Each role can have a different protocol surface. That is not necessarily duplication. It may be the clearest design.

A compact principle is:

```
same domain fact
  -> different conversations
      -> different protocol surfaces
```

This helps keep protocol-specific behavior out of the domain model. The domain meaning is shared; the surfaces are boundary-specific.

27.7 MQTT as an integration spine, not the whole system

MQTT is often a good integration spine. It is strong when the system needs asynchronous publish/subscribe flow, decoupled producers and consumers, brokered message routing, topic-based integration, and machine-to-machine message exchange.

That makes MQTT a natural fit for telemetry and integration boundaries. It does not make MQTT the whole system. A browser-facing dashboard may prefer HTTP, SSE, or WebSocket. A local management

plane may prefer Unix domain sockets. A device-near boundary may prefer Bluetooth or a custom stream service. A persistence boundary may involve a database or external service client.

The useful design rule is:

Use MQTT where brokered machine messaging is the real boundary.
Do not force MQTT to replace every other boundary.

MQTT is an integration spine, not a universal surface. It is most valuable when it is used for the conversation it expresses well.

27.8 HTTP and Express as management and operator-facing surfaces

HTTP and the Express-like layer often fit management and operator-facing boundaries. They are useful for dashboards, status pages, management APIs, REST-like control surfaces, authentication middleware, static assets, and structured routing.

A useful distinction is:

```
MQTT
  -> machine-facing integration

HTTP / Express
  -> operator-facing structure
```

This distinction is not a law. It is a design pattern. HTTP APIs may also be consumed by machines, and MQTT data may ultimately be visualized by humans. The point is that HTTP/Express naturally gives structure to operator-facing and application-facing surfaces, while MQTT naturally gives structure to brokered machine messaging.

A compact comparison:

Concern	Often better fit
publish telemetry	MQTT
subscribe to machine events	MQTT
serve dashboard page	HTTP / Express
expose admin API	HTTP / Express
authenticate an operator request	Express middleware
show current status	HTTP endpoint or dashboard
stream live state to a UI	SSE
interactive live browser session	WebSocket

This division of labor is often clearer than forcing MQTT to become the web interface or forcing HTTP to become the integration broker.

27.9 SSE and WebSocket as live observation and interaction surfaces

SSE and WebSocket become useful when the human-facing or monitoring-facing side needs live behavior. They are related to HTTP-facing design, but they serve different temporal shapes.

SSE is a good fit when updates flow mainly from server to client: live metrics, state changes, activity feeds, alerts, and dashboard updates. WebSocket is a good fit when the interaction is bidirectional: browser control sessions, live command channels, collaborative dashboards, interactive monitoring tools, or custom bidirectional protocols.

SSE and WebSocket are often presentation or interaction boundaries. They do not automatically replace the MQTT integration role. A clear design may use all of these:

```
device data
  -> MQTT integration

dashboard view
  -> HTTP / Express

live dashboard updates
  -> SSE

interactive browser control
  -> WebSocket
```

The protocols are not competing when the boundaries are different.

27.10 MQTT-over-WebSocket as a bridge boundary

MQTT-over-WebSocket is useful when MQTT semantics must travel through a WebSocket-oriented path. It is not a universal default, and it is not “more general MQTT.” It is MQTT semantics intentionally carried through a WebSocket-oriented boundary.

A good decision rule is:

```
use native MQTT
  -> when the integration boundary is naturally MQTT

use MQTT over WebSocket
  -> when the surrounding path is WebSocket-oriented
      and MQTT semantics must survive there
```

This follows directly from Chapter 26. MQTT-over-WebSocket connects:

```
MQTT semantics
  -> WebSocket subprotocol
  -> HTTP upgrade path
```

That is useful when the system genuinely needs that composition. It should not be used only because it sounds more general. Explicit boundary placement matters more than maximum reuse of one path.

27.11 Bluetooth at the device edge

Bluetooth belongs where the system is close to devices. RFCOMM and L2CAP are useful near boundaries such as local device exchange, commissioning, nearby peer communication, appliance or sensor interaction, and device-local service roles.

Bluetooth usually should not become the integration spine of the whole system. It normally belongs at one system edge:

```
Bluetooth device edge
  -> local adapter role
      -> MQTT integration
          -> dashboard / storage / external services
```

This keeps the local device-specific boundary separate from the wider integration boundary. Bluetooth can be exactly the right tool near the physical edge without being the right tool for brokered system-wide distribution.

27.12 Unix domain sockets for local control

Unix domain sockets are often a clean same-host boundary. They fit situations such as a command-line tool controlling a long-running service, a helper process talking to a broker or bridge process, a device-facing adapter communicating with a local integration service, a web-facing role coordinating with a local service, or local administration without remote exposure.

This is a good example of an explicit boundary. Not every local interaction needs to be a network service. Not every control path needs HTTP. A Unix-domain control plane can make a system simpler by keeping local coordination local.

27.13 One process or several cooperating applications

Multi-protocol design does not require one process that contains every role. Sometimes one process with several named roles is the right design. Often several focused cooperating applications are clearer.

For example:

```
device-facing process
-> talks to hardware or Bluetooth

integration process
-> publishes/subscribes via MQTT

web-facing process
-> serves dashboard and admin endpoints

bridge process
-> maps between protocol families
```

The right choice depends on deployment, failure isolation, configuration, resource limits, and operational clarity. The goal is not to maximize consolidation. The goal is to choose process boundaries that preserve system clarity.

SNode.C supports both styles because the same architectural ideas apply at both levels:

```
one process with several roles
or
several cooperating processes
```

A multi-protocol system may be a single executable when the roles are tightly related and share lifecycle. It may be clearer as several smaller processes when the boundaries, failure behavior, or operational ownership differ.

27.14 Configuration as a map of roles

In a multi-protocol system, configuration becomes a map of roles and boundaries. It can express which roles exist, which roles are enabled, which protocol family each role uses, which endpoints each role binds or connects to, which TLS material belongs to which role, which MQTT peers or brokers are used, which web interfaces are exposed, which retry or reconnect policies apply, and which local-control sockets exist.

This connects directly to Chapters 16 and 17:

```
Chapter 16
-> named instances and role visibility

Chapter 17
-> structured configuration

Chapter 27
-> configuration as a system boundary map
```


In SNode.C terms, configuration should make configured communication roles and registered runtime-visible instances legible. At the system-design level, it should also show which boundary each role serves. A configuration file should not be only a pile of values. In a multi-protocol IoT system, it should help answer:

```
Which boundary is this?
Which role owns it?
Which protocol family serves it?
How does it fail?
How is it observed?
```

The configuration becomes part of the architecture because it tells operators what system they are actually running.

27.15 Observability across role boundaries

More boundaries mean more possible failure points. That makes observability part of the architecture. Chapter 18’s diagnostic rule becomes more important here: visibility should preserve the boundary at which a fact belongs.

Useful questions include:

Question	Example
Is the device edge alive?	Bluetooth or local device service state
Is local control working?	Unix-domain socket and helper process state
Is the broker reachable?	MQTT connection and session state
Is the dashboard reachable?	HTTP application state
Is the live stream active?	SSE or WebSocket state
Which role is failing?	role-specific logs, metrics, counters, and configuration

In a multi-protocol system, diagnostics should be role-specific. It should be possible to see that:

```
HTTP admin is healthy
MQTT integration is reconnecting
Bluetooth edge is disabled
SSE dashboard stream is active
local control socket is unavailable
```

Those are different operational facts. They should not be hidden behind one generic “online/offline” flag.

27.16 Failure policy by role

Failure policy should match the role. A single global rule such as:

```
everything retries forever
```

or:

```
everything fails fast
```

is usually too crude. Different roles need different temporal behavior.

Role	Typical failure policy
MQTT integration client	reconnect for a long time

Role	Typical failure policy
local admin socket	fail fast and visibly
SSE dashboard	reconnect or show degraded state
WebSocket control session	close clearly and let the UI recover
Bluetooth commissioning	optional / disabled until needed
web admin interface	fail clearly, do not hide configuration errors

This connects to Chapter 20:

Chapter 20

-> timeout, retry, reconnect, disablement, shutdown, and failure state

Chapter 27

-> apply that separation per boundary

A retry policy that is sensible for an MQTT integration client may be actively misleading for a local administration endpoint. The role that owns the boundary should also own its failure policy. Timeouts, retries, reconnects, disablement, shutdown, and degraded state should be chosen for the boundary, not imposed globally.

A system becomes more understandable when each boundary has its own temporal behavior.

27.17 Reusing meaning across different surfaces

The same domain state may be exposed through different surfaces. That is not duplication if each surface belongs to a different boundary.

For example, the same sensor state may be published to MQTT, shown through an HTTP status endpoint, streamed through SSE, used in a WebSocket control session, written to a database, exposed through a local Unix-domain command, and reflected into a Bluetooth-facing commissioning role.

The meaning is shared. The surfaces are different.

A useful distinction is:

shared domain meaning

-> several honest protocol surfaces

This is better than forcing one protocol surface to serve every consumer. It also helps keep protocol-specific behavior out of the domain model. The domain model should not become an MQTT model, an HTTP model, or a WebSocket model merely because the same state is visible through those surfaces.

27.18 Practical design recipe

A practical recipe for multi-protocol IoT design is:

1. Identify the real system boundaries.
2. Name the boundary roles.
3. Choose the simplest appropriate protocol family for each role.
4. Keep telemetry, control, observation, administration, and device exchange distinct.
5. Configure, observe, and diagnose per role.
6. Match timeout, retry, reconnect, disablement, shutdown, and degraded-state policy to the role.
7. Split into several processes when that improves clarity.
8. Reuse domain meaning without forcing every surface to use the same protocol.

This is not the only valid method. It is a design discipline. It helps prevent two common mistakes:

one protocol everywhere

and:

```
many protocols without clear boundaries
```

The first mistake hides real differences. The second mistake creates accidental complexity. The better path is:

```
different protocols
-> clear roles
    -> explicit boundaries
        -> coherent system
```

The goal is not to minimize the number of protocols at all costs. The goal is to make every protocol choice explainable.

27.19 Protocol diversity is not architectural chaos

Protocol diversity becomes chaotic only when boundaries are unclear. A multi-protocol system is not chaotic because many protocols appear. It becomes chaotic when the reader cannot tell which boundary each protocol serves.

An IoT system may use Bluetooth near the device edge, Unix domain sockets for local control, MQTT for integration, HTTP/Express for administration, SSE for live observation, WebSocket for bidirectional interaction, MQTT-over-WebSocket for bridge boundaries, and database support for persistence. That is a diverse system. It is not automatically a messy system.

The difference is whether each protocol has a reason to be there.

SNode.C helps because many different protocol families still share the same architectural language: configured roles, registered instances, contexts, factories, configuration, diagnostics, runtime lifecycle, and failure behavior. Protocol diversity is manageable when those shared concepts remain visible.

What to remember

- IoT architecture is defined by communication boundaries, not only by devices.
- Telemetry, control, observation, administration, and device exchange are different conversations.
- MQTT often fits integration; HTTP/Express often fits management; SSE and WebSocket often fit live observation and interaction.
- Bluetooth belongs near device edges, while Unix-domain sockets often fit same-host control.
- MQTT-over-WebSocket is a bridge boundary, not a universal default.
- Multi-protocol design can live in one process or in several cooperating applications when roles, configuration, diagnostics, and failure policy stay boundary-specific.
- Protocol diversity is not chaos when boundaries are clear.

27.20 Closing perspective

Chapter 27 completed the protocol-facing IoT design view. The path through this part was:

```
MQTT as protocol family
-> MQTT over WebSocket as protocol composition
    -> multi-protocol IoT systems as boundary design
```

The important conclusion is simple:

```
clear boundary
-> honest protocol choice
    -> observable role
        -> maintainable system
```

Chapter 27 decided where communication belongs. Chapter 28 asks what happens after communication: which state is transient, which state is durable, and where persistence enters the system architecture.

Part IX

Persistence and Full Systems

This part moves from communication boundaries to application-state boundaries and complete executable systems. It introduces MariaDB-backed persistence, studies applications in `src/apps`, widens the view to systems, and then reads MQTTSuite as a concrete reference ecosystem.

The theme is that applications and systems become understandable when roles, persistence, configuration, and deployment surfaces remain visible.

Chapter 28

Database Support and Application State

28.1 From protocol boundaries to persistence boundaries

Chapter 27 described IoT systems as collections of communication boundaries. Chapter 28 adds a different kind of boundary: persistence. Communication boundaries decide how information moves between active participants; persistence boundaries decide what information should remain after the immediate conversation is over.

A protocol boundary asks:

How does information move between active participants?

A persistence boundary asks:

What information should remain after the immediate conversation is over?

That second question changes the architecture of an application. The application now has to decide what belongs in memory, what belongs in durable storage, when database work should be started, how database results flow back into the event-driven application, what happens when the database is unavailable, and how persistence interacts with protocol roles such as HTTP, MQTT, WebSocket, SSE, and local-control interfaces.

A useful model is:

```
protocol event
  -> application interpretation
      -> runtime state
          -> optional persistence boundary
              -> durable application state
```

Persistence is not automatically part of every request or every message. It is an application-state boundary that should be chosen deliberately.

Figure 28.1 shows the boundary that should stay visible when database support enters an SNode.C application. The upper lane stays on the transient communication side: peer connections, protocol contexts, and application interpretation live in the runtime and give incoming events their application meaning. The lower lane stays on the durable-state side: once the application has decided that something should survive process lifetime, the work crosses the persistence boundary and becomes explicit persistence work. The diagram keeps database integration behind that boundary instead of presenting persistence as part of transport or protocol mechanics.

Figure 28.1 therefore makes two decisions visible at the same time. First, protocol and transport activity do not automatically imply durable storage. Second, persistence begins with an application-state decision:

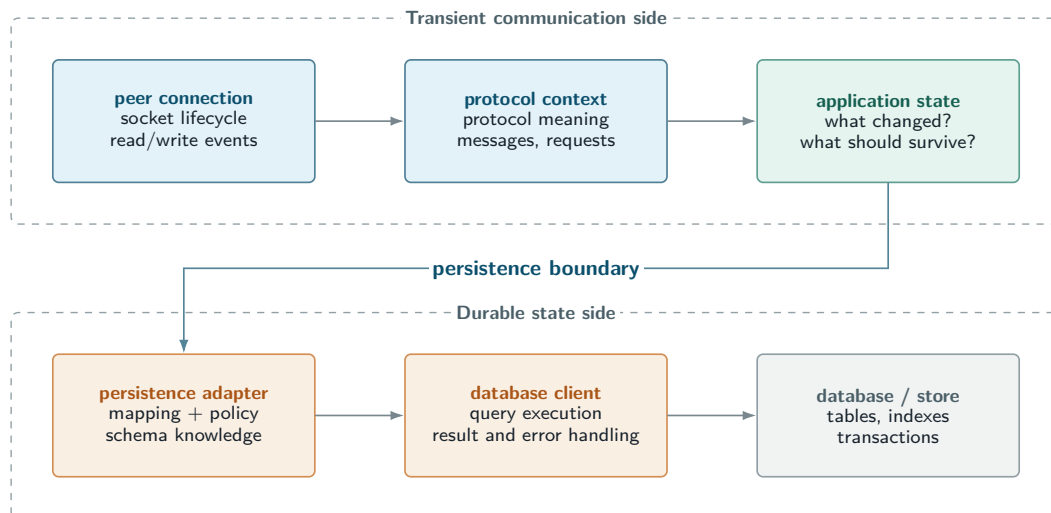


Figure 28.1: Persistence boundary in an SNode.C application: protocol contexts translate peer events into application meaning, application state decides what is worth keeping, and the database client belongs behind an explicit persistence boundary rather than inside transport or protocol mechanics.

what changed, what matters, and what is worth keeping. Only after that decision does the database client, its command API, and the durable store become part of the flow.

Persistence rule

Persist application facts, not raw transport accidents.

28.2 Persistence as an application-state boundary

A database connection should not be read as another transport protocol. An HTTP endpoint, MQTT session, WebSocket connection, Bluetooth link, or Unix-domain control socket is usually a communication boundary between active participants. A database connection is different. It is a persistence and query boundary: it stores state, retrieves state, changes state, and may become the durable memory of a larger system.

A compact comparison helps:

Concern	Protocol boundary	Persistence boundary
main purpose	exchange information	store/retrieve durable information
lifetime	often connection, session, request, or message scoped	survives beyond runtime episodes
examples	HTTP, MQTT, WebSocket, Bluetooth	MariaDB database
failure mode	peer unreachable, protocol error, timeout	connection failure, query failure, transaction failure
application question	what did the peer say?	what should be remembered?
SNode.C concern	event-driven communication	event-integrated persistence work

This distinction matters because database work should not be treated as random helper code hidden inside protocol callbacks. It belongs to the application-state architecture. A protocol callback may trigger a persistence decision, but the database boundary should still remain visible as its own architectural concern.

28.3 What SNode.C currently supports

The database support in SNode.C has a concrete scope. The current concrete database module is MariaDB-focused. It should not be presented as a generic multi-backend ORM or database-independent abstraction.

The accurate statement is:

MariaDB scope note

SNode.C currently provides a MariaDB integration layer shaped to work inside the same event-driven runtime discipline as the rest of the framework.

This narrower statement keeps the chapter aligned with the implementation. It lets the chapter focus on what the code actually provides:

```
MariaDB client object
-> connection details
    -> event-integrated MariaDB connection
        -> command objects
            -> command sequences
                -> callbacks for result and error handling
```

MariaDB-focused database module

The MariaDB module is added only when `libmariadb` is found by CMake; otherwise the `db-mariadb` target is not created. When it is available, the module builds the shared library:

```
db-mariadb
```

and installs MariaDB-specific headers under the MariaDB database include path.

For ordinary application use, the public surface has a clear source side and build side. The source-side front door is the MariaDB client header:

```
#include <database/mariadb/MariaDBClient.h>
```

The matching build-side component is:

```
db-mariadb
```

That header is the source-facing entry to the normal MariaDB client abstraction. Application code should not include individual command headers merely to use the usual client API; those lower headers belong behind the public client surface unless the application directly names a lower command type. The component supplies the binary/library surface for the same MariaDB module.

The module contains the concrete pieces needed for MariaDB integration:

- `MariaDBClient`,
- `MariaDBClientASyncAPI`,
- `MariaDBClientSyncAPI`,
- `MariaDBConnection`,
- `MariaDBConnectionDetails`,
- `MariaDBCommand`,
- asynchronous and sync-style command variants,
- `MariaDBCommandSequence`,
- `MariaDBLibrary`,
- asynchronous commands,
- sync-style metadata commands.

The module structure presents this part of the framework as concrete MariaDB support integrated with the runtime. It is not a broad ORM layer. It is not a generic database-independent abstraction. It is event-integrated MariaDB support.

Test application as demonstration, not production model

The `testmariadb` application should be read as a demonstration and test program for the API shape and runtime integration, not as a production persistence architecture. A test application can be compact, direct, and repetitive because its purpose is to exercise and show the pieces.

That is useful. It can demonstrate database configuration, runtime initialization, construction of connection details, use of client objects, state-change reporting, command execution, result handling, metadata access, command sequencing, transaction flow, and scheduling patterns.

But a real application must make its own architectural decisions.

Test app demonstrates	Production code should decide
API shape	ownership and lifetime of the database service
command chaining	domain-level persistence workflow
state callback	operational reporting policy
transactions	transaction policy
timers	scheduling policy
direct values	secure configuration and secret handling

The test application shows how the pieces work. A real application must decide how persistence belongs to its own roles, configuration, security model, resource limits, and failure policy.

28.4 Runtime state and database state

Runtime state and database state are not the same thing. Runtime state exists while the application is running. Database state survives beyond the current runtime episode.

A useful distinction is:

```
runtime state
-> live connections
-> sessions
-> timers
-> pending work
-> queues
-> caches
-> current protocol relationships

database state
-> persisted facts
-> measurement history
-> users
-> audit records
-> durable configuration
-> long-lived application data
```

The decision is not always obvious. Some information may exist in both places:

```
database
-> durable source of truth

runtime cache
-> current fast-access representation
```

The application must decide how those representations are synchronized. A cache without a synchronization policy is not architecture; it is an assumption.

What belongs where?

A practical table helps:

Information	Usually runtime state	Usually database state
active socket connection	yes	no
current WebSocket session	yes	no
active timer	yes	no
pending command	yes	no
sequence		
MQTT subscription	often yes	sometimes metadata
relationship		
last known device state	cache maybe	often yes
sensor measurement	no	yes
history		
audit log	no	yes
durable configuration	maybe loaded into memory	often source of truth
currently displayed	yes	maybe derived from stored data
dashboard state		

This table is not a rulebook. It is a way to force the application to say which state is transient, which state is durable, and which state is cached. The database should not become a dumping ground for every transient detail. Runtime memory should not pretend to be durable when it is not. The persistence boundary should be explicit.

28.5 MariaDBClient as the application-facing database object

The main application-facing object is `database::mariadb::MariaDBClient`. It combines the asynchronous command API and the sync-style metadata API, owns the internal connection object, stores the connection details, and reports connection state through a state-change callback.

It is constructed from:

```
MariaDBConnectionDetails
+ state-change callback
```

and exposes:

```
async command API
+ sync-style metadata API
```

A useful model is:

```
MariaDBConnectionDetails
+ onStateChanged callback
  -> MariaDBClient
    -> async commands
    -> sync-style metadata calls
    -> event-integrated connection
```

The state-change callback receives a state object containing:

- error number,
- error message,
- connected flag.

This fits SNode.C's runtime style. The application issues database operations and can also observe whether the database resource is connected, unavailable, or in an error state.

A database client is not the same kind of configured communication role as a socket server or client instance. It is an application-facing persistence object integrated with the runtime. It may belong to a role or service in the application, but it is not itself the same conceptual object as a registered runtime-visible socket instance.

MariaDBConnectionDetails

`MariaDBConnectionDetails` describes the database endpoint and credentials.

Field	Meaning
<code>connectionName</code>	diagnostic / operational name
<code>hostname</code>	database host
<code>username</code>	database user
<code>password</code>	database password
<code>database</code>	selected database
<code>port</code>	TCP port, if used
<code>socket</code>	local socket path, if used
<code>flags</code>	MariaDB client flags

The `connectionName` is especially useful for diagnostics. A named database connection is easier to understand in logs than an anonymous one.

The credential fields deserve care. A test program may place credentials close to the example to keep the demonstration compact. A real application needs an explicit policy for configuration, secret storage, deployment, and logging.

State-change callback

The state-change callback makes the database connection visible to the application. That matters because a database is a runtime participant. It may be connected, disconnected, unavailable, misconfigured, rejected by authentication, or failed during operation.

The callback gives the application a place to react:

```
database connected
  -> enable persistence-dependent work

database unavailable
  -> report degraded state
      -> pause periodic writes
          -> keep protocol roles alive if possible
```

The exact policy belongs to the application. The framework exposes the state. The application decides what the state means operationally.

The state callback is database observability at the application boundary. It makes the persistence role visible instead of hiding database availability inside failed commands.

28.6 A minimal MariaDB client example

The following listing is intentionally smaller than the repository's `testmariadb` application. It shows the shape of ordinary application use: configure connection details, construct the database client, observe connection state, submit SQL work, continue through callbacks, and then start the SNode.C runtime.

For the SQL statements below, assume a small table such as:

```
CREATE DATABASE snodec;

CREATE TABLE measurements (
    sensor VARCHAR(64) NOT NULL,
    value DOUBLE NOT NULL
);
```

This setup note is only there to make the example concrete. Chapter 28 is not a MariaDB administration chapter. A real deployment also needs a deliberate policy for database users, privileges, credentials, schema migration, backup, and secret handling.

```
#include <core/SNodeC.h>
#include <database/mariadb/MariaDBClient.h>
#include <log/Logger.h>

#include <mysql.h>
#include <string>

int main(int argc, char* argv[]) {
    core::SNodeC::init(argc, argv);

    const database::mariadb::MariaDBConnectionDetails details = {
        .connectionName = "measurements-db",
        .hostname = "localhost",
        .username = "snodec",
        .password = "<password>",
        .database = "snodec",
        .port = 3306,
        .socket = "",
        .flags = 0,
    };

    database::mariadb::MariaDBClient db(details, [](const database::mariadb::MariaDBState& state) {
        if (state.error != 0) {
            LOG(ERROR) << "MariaDB state error " << state.error << ": "
                << state.errorMessage;
        } else if (state.connected) {
            VLOG(1) << "MariaDB connected";
        } else {
            VLOG(1) << "MariaDB disconnected";
        }
    });

    db.exec(
        "INSERT INTO measurements(sensor, value) VALUES ('temperature', 23.5)",
        [&db]() {
            db.affectedRows(
                [](my_ulonglong rows) {
                    VLOG(1) << "insert affected rows: " << rows;
                },
                [](const std::string& error, unsigned int number) {
                    LOG(ERROR) << "affectedRows error " << number << ": "
                        << error;
                });
        },
        [](const std::string& error, unsigned int number) {
            LOG(ERROR) << "insert error " << number << ": " << error;
        })
        .query(
            "SELECT sensor, value FROM measurements",
            [](const MYSQL_ROW row) {
```

```

        if (row != nullptr) {
            VLOG(1) << "measurement: " << row[0] << " = " << row[1];
        } else {
            VLOG(1) << "measurement query complete";
        }
    },
    [](const std::string& error, unsigned int number) {
        LOG(ERROR) << "query error " << number << ": " << error;
    });

    return core::SNodeC::start();
}

```

The example deliberately keeps the database details local so that the API shape is visible. That is not a production credential policy. Production code should not hard-code database passwords in the source; it should read them from a controlled configuration or secret mechanism and should avoid logging secret material.

Nothing in this listing makes the database a transport peer. The database client is an application-owned persistence boundary. The SQL work is submitted before the runtime starts, but its progress and completion belong to the event-driven execution model. The row callback receives individual rows; a `nullptr` row marks the end of that result stream. For the database case, the companion source tree is `MariaDB-Minimal`.

The corresponding build-side dependency is the MariaDB component:

```
target_link_libraries(myapp PRIVATE snodec::db-mariadb)
```

That target exists only in `SNode.C` builds where `libmariadb` was found. The source-side public header and the build-side component therefore form the usual pair:

```

source side:
  <database/mariadb/MariaDBClient.h>

build side:
  snodec::db-mariadb

```

28.7 Database work inside the event-driven runtime

The architectural heart of the MariaDB module is event-loop integration. Database work participates in the `SNode.C` event-driven runtime instead of simply calling SQL and blocking.

A useful model is:

```

MariaDB socket descriptor
-> SNode.C event receivers
    -> command continuation
        -> result callback or error callback

```

Internally, `MariaDBConnection` participates as a read, write, and exceptional-condition event receiver. The database descriptor becomes part of the same event-driven discipline as socket descriptors. It handles command execution, command continuation, command completion, read events, write events, exceptional-condition events, related timeouts, and connection state.

Conceptually:

```

database command
-> MariaDB nonblocking operation
    -> wait for descriptor readiness
        -> continue command
            -> complete, fail, or wait again

```

The application-facing API remains callback-shaped. The internal connection handles the event-driven continuation.

Event-integrated database work avoids blocking the application around ordinary command progress, but it does not make database latency disappear. A slow database is still slow. A saturated database is still saturated. The difference is that database progress can be coordinated with the same event-driven runtime discipline as the network layers.

MariaDBConnection and event receivers

The internal connection owns the active database interaction. It tracks the active command, queues command sequences, observes connection state, and reacts to read, write, and exceptional-condition readiness.

A compact source-shaped model is:

```
start command
  -> wait for read/write readiness or completion
      -> commandContinue(...)
      -> commandCompleted()
```

This is the database equivalent of the earlier event-driven communication model. The specific resource is different. The runtime discipline remains familiar.

Command continuation

A database command may not complete immediately. It may need the underlying MariaDB socket to become readable or writable. The event loop tells the connection when progress is possible. Then the connection continues the current command.

A compact model is:

```
start command
  -> command needs read/write readiness
      -> event loop wakes connection
          -> command continues
          -> command completes or waits again
```

The point is not the exact MariaDB C API detail. Database work is expressed as explicit continuation, not as an invisible blocking detour inside a protocol callback.

28.8 Database work as commands and command sequences

The MariaDB layer represents database work explicitly. Database operations become command objects. Command objects can be grouped into command sequences. That gives database work structure inside the event-driven runtime.

A useful model is:

```
command
  -> command sequence
      -> connection queue
          -> event-driven continuation
          -> success/error callback
```

This pattern is central to the database API. The application describes ordered database work. The connection drives that work through the event loop.

Async API: query, exec, transactions, commit, rollback

The asynchronous API exposes the main database operations:

Method	Meaning
<code>query(...)</code>	execute SQL that produces rows
<code>exec(...)</code>	execute SQL action without row iteration as the main result
<code>startTransactions(...)</code>	enter transaction mode
<code>endTransactions(...)</code>	leave transaction mode
<code>commit(...)</code>	commit current transaction work
<code>rollback(...)</code>	roll back current transaction work

Each method uses success and error callbacks. The exact SQL is application-specific. The API shape is the important teaching point:

```
database operation
-> success callback
-> error callback
```

The source uses the plural method names `startTransactions(...)` and `endTransactions(...)`; prose may speak about a transaction, but code-form sections should keep the actual names.

Sync-style metadata calls

The sync-style API exposes metadata calls such as:

Method	Meaning
<code>affectedRows(...)</code>	report affected-row count
<code>fieldCount(...)</code>	report field count

“Sync-style” here identifies metadata-style command surfaces; the application-facing flow still uses callbacks. These calls read state that the MariaDB client library already has after a completed command or result step. They do not stand for arbitrary blocking SQL work inside an event-driven application.

query(...) versus exec(...)

The difference between `query(...)` and `exec(...)` is an application intention.

Method	Use when
<code>exec(...)</code>	the SQL statement performs an action and row iteration is not the main result
<code>query(...)</code>	the SQL statement produces rows the application wants to inspect

Examples:

```
exec
-> insert
-> update
-> delete
-> schema modification

query
-> select rows
-> inspect result records
```

The database receives SQL text in both cases. The application-facing API makes the intended use clearer.

Command sequences

A `MariaDBCommandSequence` allows database operations to be chained. The sequence itself is also an API surface: it inherits the asynchronous and sync-style API shapes so commands can be appended in order.

This matters because many database workflows are ordered. For example:

```
delete old rows
-> inspect affected rows
    -> insert new row
        -> inspect affected rows
            -> query current state
```

The command sequence expresses the order. The application does not need to write its own blocking loop. It describes the sequence and lets the database connection advance it through the event-driven runtime.

A command sequence is not the same thing as a transaction:

```
command sequence
-> ordered database work

transaction
-> database consistency boundary represented through ordered commands
```

Both can appear together, but they are different ideas.

Chaining, callbacks, and metadata reads

The call shape is important. A chained call appends to the command sequence returned by the previous operation:

```
db.exec(...).query(...);
```

Here the `query(...)` belongs to the same sequence as the preceding `exec(...)`. It is not a second immediate operation on the client object.

Starting async work from inside a callback has different meaning:

```
db.exec(..., [ &db ]() {
    db.exec(...);
}, ...);
```

The inner `exec(...)` creates a new command sequence and queues it on the same connection. It does not splice itself into the current sequence, and it does not jump ahead of commands already chained onto that sequence.

Metadata calls such as `affectedRows(...)` are the narrow exception to this “queue more SQL work” rule. They are synchronous metadata reads of the just-completed command state. Used inside an `exec(...)` success callback, `affectedRows(...)` observes the completed command before the connection advances to the next queued SQL command.

A compact mental model is:

```
chained call
-> append to the current returned sequence

async call created inside a callback
-> create a new queued sequence

metadata call inside a callback
-> read already available command/result state immediately
```


28.9 Transactions as sequenced database work

Transactions are not outside the event model. They are represented as ordered database commands in the same command-sequence mechanism.

A useful model is:

```
startTransactions
-> exec / query
    -> commit or rollback
    -> endTransactions
```

The important point is not simply that transactions exist. Transaction flow remains visible and ordered. A transaction can succeed. It can fail. A rollback may be needed. An application may need to report degraded state, retry, compensate, or stop a workflow.

A transaction is not outside the event model; it is a policy and ordering boundary expressed through database commands and callbacks.

A compact transaction sequence

A command sequence keeps ordered database work visible. Transaction flow is not hidden behind a helper that silently blocks. The sequence shows when transaction mode begins, which work belongs to the transaction, when the commit is requested, and when transaction mode is left again.

```
db.startTransactions(
    []() {
        // Transaction mode enabled.
    },
    [](const std::string& error, unsigned int number) {
        // Report transaction-start error.
    })
    .exec(
        "INSERT INTO measurements(sensor, value) VALUES ('humidity', 61.0)",
        []() {
            // Insert command accepted.
        },
        [](const std::string& error, unsigned int number) {
            // Report insert error.
        })
    .commit(
        []() {
            // Transaction committed.
        },
        [](const std::string& error, unsigned int number) {
            // Report commit error.
        })
    .endTransactions(
        []() {
            // Transaction mode disabled.
        },
        [](const std::string& error, unsigned int number) {
            // Report transaction-end error.
        });
```

The source uses the plural method names `startTransactions(...)` and `endTransactions(...)`; the listing keeps those exact names. A rollback path would use the same command-sequence shape with `rollback(...)` instead of, or before, a later `commit(...)`, depending on the application policy. The important architectural point is that transaction boundaries remain explicit and ordered.

28.10 Timers and database work

Timers can trigger database work. The database commands then continue through the MariaDB event integration.

A simple model is:

```
timer fires
-> start database command sequence
    -> MariaDB event integration continues the work
        -> callback reports result or error
```

This is especially relevant for IoT systems. A service may periodically aggregate sensor values, clean old rows, reconcile database state with runtime state, refresh derived status, publish stored values, or check for pending commands.

The timer starts the work. The database layer does not become a blocking loop. It remains part of the same runtime world.

28.11 Persistence service design

Database access should be designed as part of the application architecture. It should not automatically be scattered through every protocol callback. Sometimes direct access is acceptable. But in larger systems, a persistence service or application-level component is often clearer.

A useful shape is:

```
protocol callback
-> domain operation
    -> persistence service
        -> MariaDB command sequence
```

The protocol layer interprets protocol input. The domain layer decides what the input means. The persistence layer decides what must be stored, updated, read, or rolled back.

In the boundary language of Chapter 27, persistence may be its own role or service, especially when several protocol surfaces touch the same durable state.

Keep database logic out of low-level protocol callbacks

A protocol callback should usually interpret protocol input, not become the whole persistence layer. For example, an HTTP handler, MQTT callback, WebSocket message handler, or `SocketContext` method may receive an event. It should not automatically contain all SQL construction, transaction policy, retry policy, and error handling inline.

A cleaner design is often:

```
HTTP / MQTT / WebSocket / local-control input
-> validate and interpret
    -> call application service
        -> schedule persistence work
            -> report result/degraded state
```

This keeps protocol code readable. It also prevents database details from leaking into every communication boundary.

Database client ownership and lifetime

The lifetime of the database client should match the lifetime of the application role or service that owns persistence. The test application creates clients directly in `main()` because it is a compact demonstration. A structured application should decide ownership explicitly.

Ownership	Fit
per request / per connection	rarely, only if isolated short work is intended
application or service scope	often the clearest default for persistence ownership
dedicated persistence component	good for multi-protocol systems
shared global	risky unless ownership and shutdown are explicit

The design question is:

What lifetime should this database connection or database service have relative to the roles that use it?

That affects shutdown, error handling, resource limits, and system clarity.

28.12 Persistence, failure, and backpressure

Persistence changes failure thinking. A system may be partially healthy:

```

HTTP route healthy
+ database unavailable
  -> degraded application behavior

MQTT connected
+ insert fails
  -> buffering, dropping, retry, or error policy needed

timer fires
+ previous sequence still pending
  -> scheduling/backpressure policy needed

```

A database failure is not always an application failure. It may mean stop accepting certain operations, keep reading but stop writing, buffer with limits, drop non-critical values, show degraded status, retry later, or fail fast for administrative operations.

The MariaDB API exposes state and errors. The application must decide the policy.

Chapter 20's vocabulary still applies: timeout, retry, shutdown, failure state, and degraded behavior must be decided at the boundary that owns the failing work.

Persistence can introduce backpressure

Database work can accumulate. This happens when incoming protocol activity is faster than database completion.

A useful model is:

```

incoming protocol rate
> database completion rate
  -> pending command sequences grow

```

This is a system-design problem. The application needs a policy.

Policy	Meaning
reject	refuse new work when persistence is overloaded
buffer with limit	queue some work, then apply a limit
coalesce	merge repeated updates into one write
drop old values	keep only recent values
write latest state only	persist current state instead of every event
slow upstream producers	apply feedback if possible

Policy	Meaning
report degraded state	make overload visible to operators

Persistence is not free. A database boundary has throughput, latency, and failure behavior. A persistence queue without a limit is not a policy; it is delayed failure.

28.13 Database state in multi-protocol IoT systems

In a multi-protocol IoT system, persistence is one boundary among several. A possible state flow is:

```
sensor update
-> MQTT publish
    -> application state update
        -> database insert
            -> SSE dashboard update
```

Another system may use a different order:

```
HTTP command
-> validate operator request
    -> database transaction
        -> MQTT command publish
            -> WebSocket status update
```

The correct order depends on the domain. Sometimes persistence follows communication; sometimes persistence authorizes or precedes communication.

persistence participates in the system flow. It should not be treated as an isolated afterthought.

A multi-protocol system may need to coordinate:

- current runtime state,
- persisted state,
- MQTT messages,
- HTTP requests,
- WebSocket sessions,
- SSE streams,
- local-control commands,
- database transactions.

That coordination is application architecture. The database module provides the event-integrated persistence tool. The application decides how durable state fits into the system.

What to remember

- Persistence is another system boundary.
- Persistence is where selected application information survives beyond one connection, message, request, or runtime episode.
- SNode.C currently provides MariaDB-focused database support through `db-mariadb`.
- Database state and runtime state are different.
- `MariaDBClient` is the application-facing database object.
- `MariaDBConnectionDetails` describes the database endpoint and credentials.

28.14 Closing perspective

Chapter 28 introduced persistence as an application-state boundary.

The path now looks like this:

```
protocol event
-> application meaning
    -> runtime state
        -> persistence boundary
            -> durable state
```

That boundary changes application design. It introduces durable memory, query flow, transactions, database failure, and database backpressure into the same event-driven world as the protocol layers.

Chapter 28 introduced persistence as a boundary. Chapter 29 now turns to complete applications, where protocol roles, persistence choices, configuration, runtime lifecycle, and diagnostics meet in executable form.

Chapter 29

Learning from the Applications in `src/apps`

29.1 From framework pieces to application structure

Chapter 28 introduced persistence as an application-state boundary. Chapter 29 now turns to executable applications: the point where runtime setup, selected components, application-side objects, configured communication roles, callbacks, routes, persistence objects, and installable targets meet in concrete programs.

The question changes from:

What does this layer mean?

to:

How do the layers become executable programs?

That is an important shift. Up to this point, the book has introduced many framework pieces separately: runtime initialization, communication roles, contexts and factories, lower communication families, configuration, diagnostics, TLS, timeouts and failure behavior, HTTP, the Express-like layer, SSE, WebSocket, MQTT, MQTT over WebSocket, and database support. The applications in `src/apps` show how those pieces are assembled.

The central sentence for this chapter is:

Build-target reading note

In `SNode.C`, the build target often reveals the application architecture before the entry point is opened.

That is the main reason `src/apps` is useful as study material. It shows not only C++ entry points, but also public include choices, executable targets, linked libraries, optional dependencies, and installable application shapes. Chapter 28 showed persistence as a boundary. Chapter 29 shows applications as assembly points.

29.2 `src/apps` as study material

The applications in `src/apps` should not all be read in the same way. Some are application shells, some are focused examples, some are utility programs, and some are test or demonstration targets. Some demonstrate a protocol family. Some demonstrate a build pattern. They are not all production templates.

For this chapter, we will not discuss every target in the directory. Instead, we select a representative set:

Example	Why it is useful here
<code>snode.c</code>	web application shell with Express-style structure, routes, SSE, timers, and listen-state reporting
<code>express-compat-server</code>	behavior and compatibility comparison using the same broad web stack
echo family	generated client/server matrix over several carriers and stream modes
<code>jsonserver</code> / <code>jsonclient</code>	HTTP server/client pair with JSON-oriented request and response behavior
<code>testpost</code>	focused HTTP POST example with legacy and TLS web application composition
<code>testpipe</code>	core runtime utility showing pipe-based event handling
<code>database/testmariadb</code>	MariaDB persistence demonstration after the persistence chapter

The selection is broad enough to show different application shapes and narrow enough to keep the chapter readable. The goal is not to catalogue a directory. The goal is to learn how SNode.C applications are assembled.

The top-level app build also contains other targets, such as `configtest`, `warema-jalousien`, and a conditional `testregex`. Those are useful in their own contexts, but they are not needed for the main teaching path here.

29.3 Build targets as the first architectural reading layer

Before opening the C++ entry point, the build target already tells part of the application story.

A SNode.C application target tells the reader three things before C++ is opened:

```
which executable is produced
which public headers and framework components form its public face
which optional dependencies decide whether the target exists
```

Selected application targets in the repository

A first reading pass should start with the build targets. The top-level `src/apps/CMakeLists.txt` creates executables and links them against selected SNode.C targets. This file is an in-source-tree build file, and that distinction matters.

Application reading checklist

- Read the CMake target.
- Identify the lower layer.
- Find the configured role.
- Find the context factory.
- Find the context.
- Find the runtime start point.

Inside the SNode.C source tree, application targets can link local CMake target names such as:

```
core
http-server-express
net-in-stream-legacy
http-client
db-mariadb
```

An application outside the SNode.C source tree should use the installed package interface instead:

```

find_package(snodec REQUIRED
  COMPONENTS
    http-server-express
    net-in-stream-legacy
)

target_link_libraries(my-ipv4-legacy-webapp
  PRIVATE
    snodec::http-server-express
    snodec::net-in-stream-legacy
)

```

The namespace is not cosmetic. It tells CMake that the application is using exported SNode.C component targets, not internal targets from the SNode.C build tree. The direct component choices are the same as in the in-tree example; the external form uses the installed `snodec::...` namespace.

A simplified view of the selected application targets is:

Application target	In-tree target shape visible in <code>src/apps</code>	What it teaches
<code>snode.c</code>	<code>http-server-express</code> + <code>net-in-stream-legacy</code>	web application shell over IPv4 legacy stream
<code>express-compat-server</code>	<code>http-server-express</code> + <code>net-in-stream-legacy</code>	Express-style compatibility behavior
<code>testpost</code>	<code>http-server-express</code> + <code>net-in-stream-legacy</code> + <code>net-in-stream-tls</code>	HTTP POST handling and legacy/TLS application composition
<code>jsonserver</code>	<code>http-server-express</code> + <code>net-in-stream-legacy</code> , built when JSON support is available	JSON-capable HTTP server example
<code>jsonclient</code>	<code>http-client</code> + <code>net-in-stream-legacy</code>	outgoing HTTP request/response example
<code>testpipe</code>	<code>core</code>	pipe event behavior inside the runtime
<code>database/testmariadb</code>	<code>db-mariadb</code> , built when MariaDB support is available	MariaDB API and persistence demonstration
echo family	<code>echosocketcontext</code> + generated <code>net-...-stream-...</code> combinations	one protocol model across carriers and stream modes

The target name tells the reader what executable is produced. The link line shows which application-facing component or local target is selected. Conditional build rules show which optional components must be available. Install rules show which executables become part of the application installation set.

The link line should not be misread as a manual list of every lower layer. It shows the direct application-facing components selected by the executable. The include block should be read in the same disciplined way: it should show the public headers for the abstractions directly named by the source file, not a manual list of every lower header behind them. The next section reads this link line in more detail.

Linked components reveal application shape

A SNode.C application can often be understood by reading the components it links. For many high-level protocol applications, the normal direct link shape is:

```

protocol/application component
+ selected transport component

```

This rule applies both inside the SNode.C source tree and outside it. Only the target names change:


```

in-tree target names
-> http-server-express
-> net-in-stream-legacy

installed/exported target names
-> snodec::http-server-express
-> snodec::net-in-stream-legacy

```

Consider this in-tree build fragment:

```

add_executable(snode.c main.cpp)

target_link_libraries(
    snode.c
    PUBLIC
        http-server-express
        net-in-stream-legacy
)

```

It tells us that the application directly selects two visible building blocks: the Express-like HTTP server layer and the IPv4 legacy stream carrier. The equivalent external form uses exported `snodec::...` targets:

```

find_package(snodec REQUIRED
    COMPONENTS
        http-server-express
        net-in-stream-legacy
)

add_executable(my-ipv4-legacy-webapp
    main.cpp
)

target_link_libraries(my-ipv4-legacy-webapp
    PRIVATE
        snodec::http-server-express
        snodec::net-in-stream-legacy
)

```

The direct application face is the same in both cases, but the target names belong to different build contexts:

```

HTTP/Express application layer
  in-tree target: http-server-express
  external target: snodec::http-server-express

IPv4 legacy stream carrier
  in-tree target: net-in-stream-legacy
  external target: snodec::net-in-stream-legacy

```

The `snodec::...` form is not another dependency step. It is the installed/exported target name used outside the SNode.C source tree. The application does not need to list the lower dependency chain behind those components, because the selected component targets already declare the further SNode.C components they need.

The direct link line is short, but the component-owned dependency graph is deeper. For this example, the public component-dependency graph expands into the following teaching view. System libraries and non-SNode.C implementation details are intentionally not expanded; some are only shown as named leaf dependencies.

```
my-ipv4-legacy-webapp
```

```

|-- snodec::http-server-express
|   |-- snodec::http-server
|   |   |-- snodec::http
|   |   |   |-- snodec::core-socket-stream
|   |   |   |   |-- snodec::core-socket
|   |   |   |   |   |-- snodec::core
|   |   |   |   |   |   |-- snodec::utils
|   |   |   |   |   |   |-- snodec::logger
|   |   |   |   |-- libmagic, if available
|   |-- nlohmann-json support
|-- snodec::net-in-stream-legacy
|   |-- snodec::net-in-stream
|   |   |-- snodec::net-in-phy-stream
|   |   |   |-- snodec::net-in-phy
|   |   |   |   |-- snodec::net-in
|   |   |   |   |   |-- snodec::net
|   |   |   |   |   |   |-- snodec::core-socket
|   |   |   |   |   |   |-- snodec::core
|   |   |   |   |   |   |   |-- snodec::utils
|   |   |   |   |   |   |   |-- snodec::logger
|   |-- snodec::core-socket-stream-legacy
|   |   |-- snodec::core-socket-stream
|   |   |   |-- snodec::core-socket
|   |   |   |   |-- snodec::core
|   |   |   |   |   |-- snodec::utils
|   |   |   |   |   |-- snodec::logger

```

The two top-level branches are still the important part. They show the two direct decisions of the application:

```

http-server-express
  -> the Express-style HTTP server/application layer

net-in-stream-legacy
  -> the selected IPv4 legacy stream carrier

```

The HTTP branch remains the protocol/application branch. It carries lower implementation dependencies, such as HTTP server support, generic HTTP support, stream context machinery, core socket support, core runtime support, utilities, and logging. Optional MIME support can add `libmagic`, and the Express layer requires JSON support for its JSON middleware.

The transport branch remains the selected carrier branch. Lower dependencies may overlap internally, but that does not change the application-facing rule. The same way of reading applies inside `net-in-stream-legacy` itself. It is a composed SNode.C component:

```

net-in-stream-legacy
  -> net-in-stream
  -> core-socket-stream-legacy

```

The component name therefore stands for a layered composition:

```

IPv4 network family
+ stream transport shape
+ legacy stream operation
+ physical network side
+ generic/core stream machinery

```

A higher-level component name abstracts the internal link list, but it still exposes the design structure. The diagram is not a linker command; it is a reader-facing view of the public component graph.

The important build model is:

```

application
-> link the direct protocol/application component
-> link the selected transport component
    -> CMake propagates those components' public dependencies

```

This keeps the boundary explicit:

```

Which high-level protocol/application component does this application use?
Which transport/carrier component does this application use?
    -> link those direct components

What do those components need internally?
    -> their targets declare it

```

The point is not that the application knows the whole graph. It selects the components that define its public shape: the HTTP application layer and the concrete transport carrier. The remaining dependencies belong to those selected components.

When reading another SNode.C application, start with the executable target, then read the linked protocol/application component, the selected transport component, and any optional feature components. Only then open the C++ entry point. A direct application link line should describe the application face, not repeat every implementation layer below it.

The build system records application choices

The build system is part of the application story. In SNode.C, CMake targets often reveal the application structure before the C++ entry point is opened.

The build file records:

Build element	Architectural meaning
executable target	the runnable program
linked libraries	selected protocol/application, transport, and feature components
conditional target creation	optional feature dependency
install rule	deployment-facing executable
subdirectory discovery	application families and grouped examples

For example, the JSON server target is only built when JSON support is available:

```

if(NLOHMANN_JSON_FOUND)
    add_executable(jsonserver jsonserver.cpp)
    target_link_libraries(
        jsonserver
        PRIVATE
            http-server-express
            net-in-stream-legacy
    )
endif()

```

This is an application-level fact, not only a build-system detail:

```

JSON support available
    -> JSON server example becomes part of the build

JSON support unavailable
    -> target is not produced

```

The same idea appears with the database example:

```

MariaDB support available
  -> database demonstration target exists

MariaDB support unavailable
  -> database demonstration target is absent

```

Optional dependencies therefore influence the set of applications that exist in the build.

29.4 Read include blocks beside link lines

When studying an application, read the include block beside the CMake link line. The two views should agree, but they do different jobs:

```

include block
  -> C++ source-facing abstractions directly named by the file

link line
  -> binary/link-facing components directly selected by the target

```

For an external Express IPv4 legacy application, the source might include an Express front-door header while the CMake target links the concrete Express carrier component. If the same source file also directly creates an MQTT carrier client, it includes the matching socket-client or MQTT headers and links the matching components. This is not duplication; it is the same architecture expressed in C++ source and in the build system.

29.5 Entry points as assembly points

After the build target has established the application face, the entry point shows how the selected pieces are assembled.

The executable as an assembly point

In many SNode.C applications, the executable entry point is not the place where the whole framework is reimplemented. It is the assembly point. It wires selected framework pieces, application callbacks, configuration, listen/connect actions, and runtime start together.

A typical entry point answers questions such as:

- How is the runtime initialized?
- Which application-side object or communication role is created?
- Which configured communication roles and registered runtime-visible instances become relevant?
- Which middleware, routes, contexts, factories, or command objects are registered?
- Which listen or connect action is started?
- Where do diagnostics and state callbacks enter?
- How is the runtime started?

The `snode.c` application is a good example. Here, `snode.c` refers to the application target in `src/apps`, not to the entire framework.

A compact teaching shape of its structure is:

```

int main(int argc, char* argv[]) {
    core::SNodeC::init(argc, argv);

    const express::legacy::in::WebApp app;

    app.use(express::middleware::VerboseRequest());

    app.get("/health", [] APPLICATION(req, res) {
        res->json({"ok", true});
    });
}

```

```
});

app.listen(8080, /* state callback */);

return core::SNodeC::start();
}
```

This small shape already shows application assembly:

```
runtime initialization
-> web application object
    -> middleware
        -> route
            -> listen action
                -> runtime start
```

The real file contains more routes, nested routers, SSE behavior, timer-driven output, and listen-state handling. But the assembly principle is already visible in the compact form. The executable is not a giant custom abstraction. It wires framework pieces and application behavior together.

Chapter 16 and Chapter 17 become practical here: application entry points and build targets decide which roles and configuration surfaces become visible. Chapter 18 becomes practical as well: application examples show where diagnostics enter through state callbacks, log output, visible target names, and conditional build choices.

snode.c as application-shell example

The `snode.c` target is the best main example for application-shell structure. It combines:

```
snode.c
-> Express-like HTTP application layer
-> IPv4 legacy stream carrier
```

Its entry point demonstrates several useful patterns:

- an Express-style `WebApp`,
- middleware installation,
- route registration,
- nested routers,
- JSON responses,
- SSE output,
- timer-driven repeated sending,
- listen-state callback handling,
- runtime start.

A useful reading model is:

```
build target
-> linked layers
    -> application object
        -> middleware and routes
            -> listen/connect/state handling
                -> runtime start
```

This is the path from CMake to application behavior. For a reader, that path is more useful than memorizing individual API calls. It shows how to approach a `SNode.C` application file.

express-compat-server as compatibility-oriented example

`express-compat-server` links the same broad layer family as `snode.c`:

```
http-server-express
net-in-stream-legacy
```

Its role in the manuscript should be secondary. It is useful because it shows that the Express-like layer can also be used for behavior comparison, compatibility checking, and focused route/middleware experiments.

A good way to present it is:

```
snode.c
-> application-shell example

express-compat-server
-> compatibility / behavior-comparison example
```

The goal is not to study both applications deeply, but to show that the same layer composition can serve different application intentions.

29.6 Application families and focused examples

The selected examples show different application shapes without turning the chapter into a directory catalogue.

Echo as an application family

Chapter 3 introduced a deliberately simplified echo pair:

```
EchoSocketContext
-> echoserver
-> echoclient
-> IPv4 / stream / legacy
```

The repository echo family generalizes the same idea. The full echo application structure uses a shared echo protocol model, generated server executables, generated client executables, several network families, legacy and TLS stream modes, and compile definitions for the selected combination.

The key idea is:

```
stable protocol model
-> generated client/server targets
    -> selected network family
        -> selected legacy/TLS stream mode
```

This is one of the most useful application patterns in the repository. It shows how SNode.C can keep the protocol core stable while changing the outer communication boundary.

The build includes IPv4, IPv6, and Unix-domain variants by default. Bluetooth L2CAP and RFCOMM echo variants are added only when BlueZ support is available. That conditionality matters because it is part of the application shape: not every generated executable exists in every build.

JSON server and JSON client

The JSON examples are useful because they show a clean server/client split.

The server side combines the Express-like HTTP server layer with the selected stream carrier and JSON middleware:

```
jsonserver
-> Express legacy WebApp
    -> JsonMiddleware
        -> POST /index.html
```

```
-> JSON attribute access
-> response
```

The client side combines the HTTP client layer with the selected stream carrier:

```
jsonclient
-> HTTP legacy client
  -> MasterRequest
    -> POST /index.html
      -> application/json body
        -> response / parse-error callbacks
```

This pair is good manuscript material because it shows both sides of an HTTP interaction. It also shows optional feature availability: the server target depends on JSON support being present, while the client demonstrates an outgoing HTTP request shape.

testpost as focused HTTP POST example

`testpost` should be presented as a focused example, not as a polished application. It demonstrates an HTTP POST-oriented surface and links both legacy and TLS stream support.

A useful model is:

```
legacy web app
-> GET form
  -> POST body handling

TLS web app
-> reuses legacy app behavior
  -> listens on TLS-capable endpoint
```

The source structure is useful because it shows two related application roles in one file:

```
express::legacy::in::WebApp
-> legacy HTTP endpoint

express::tls::in::WebApp
-> TLS HTTP endpoint
```

The TLS app reuses the legacy app behavior. That makes `testpost` useful for Chapter 29 because it shows application composition rather than only route syntax. It also connects back to Chapter 19:

```
same application behavior
-> legacy carrier
-> TLS carrier
```

This example should remain compact in the manuscript. The point is the application shape, not the details of the HTML upload form or the example certificate configuration.

testpipe as a small core utility

`testpipe` is useful because it does not depend on HTTP, MQTT, WebSocket, or database support. It links only against the core layer. That makes it a small example of runtime-managed utility behavior.

A compact model is:

```
core runtime
-> Pipe
  -> PipeSink callbacks
    -> PipeSource send
      -> runtime start
```

A simplified excerpt captures the idea:

```
core::SNodeC::init(argc, argv);

const core::pipe::Pipe pipe(
    [](core::pipe::PipeSource& source,
       core::pipe::PipeSink& sink) {
        sink.setOnData([&source](const char* chunk, std::size_t len) {
            source.send(chunk, len);
        });

        sink.setOnEof([]() {
            VLOG(1) << "Pipe EOF";
        });

        source.send("Hello World!");
    },
    [](int errnum) {
        PLOG(ERROR) << "Pipe not created";
    });

return core::SNodeC::start();
```

This example is useful because it reminds the reader that applications are not limited to servers. SNode.C also contains core utilities and runtime-managed helpers. The same pattern appears again:

```
initialize runtime
-> create runtime object
    -> register callbacks
    -> start runtime
```

Not every application target is a server/client communication-role example. Some targets demonstrate core runtime objects or database integration.

database/testmariadb after the persistence chapter

Chapter 28 introduced the MariaDB integration layer. Chapter 29 can now show how a database demonstration becomes an application target.

The database example is located under a dedicated application subdirectory:

```
src/apps/database
```

It is built only when MariaDB support is available. Its build shape is compact:

```
if(LIBMARIADB_FOUND)
    add_executable(testmariadb testmariadb.cpp)
    target_link_libraries(testmariadb PRIVATE db-mariadb)
endif()
```

This is a good continuation from Chapter 28. The target does not need HTTP, MQTT, or WebSocket to teach persistence. It directly demonstrates:

- database-related configuration,
- MariaDBConnectionDetails,
- MariaDBClient,
- state-change callbacks,
- exec(...),
- query(...),
- affected-row and field-count metadata,
- command chaining,

- timers that trigger database work,
- transaction sequences with rollback and commit.

A compact application model is:

```
configuration
  -> MariaDBConnectionDetails
    -> MariaDBClient
      -> command sequence
        -> callback result/error handling
          -> timer-driven repeated queries
```

`testmariadb` is a focused demonstration target. It is not a recommended production persistence architecture. Its value is that it makes the Chapter 28 API concrete.

29.7 Reading applications in src/apps

The examples lead to a practical reading method for unfamiliar SNode.C applications.

Application categories in src/apps

The selected examples can be grouped by what they teach:

Category	Examples	What to learn
web application shell	<code>snode.c</code>	runtime + Express-style application assembly
compatibility / behavior comparison	<code>express-compat-server</code>	another application using the same broad web stack
generated application family	echo family	one protocol model across executable variants
HTTP client/server examples	<code>jsonserver</code> , <code>jsonclient</code>	server/client split and optional JSON support
HTTP POST / TLS-capable example	<code>testpost</code>	legacy/TLS application composition
core runtime utility	<code>testpipe</code>	core-only runtime object and callbacks
database demonstration	<code>database/testmariadb</code>	MariaDB API and event-integrated persistence

This grouping is more useful than a flat directory list. It tells the reader what to study and why.

Small applications and larger applications differ by composition depth

Small applications and larger applications often use the same pattern. They differ in composition depth.

Application depth	Typical composition
minimal utility	runtime + one core object + callbacks
small server	runtime + server-side communication role + context/factory or route
web app	runtime + HTTP/Express + middleware/routes
client/server pair	server target + client target + shared protocol expectation
generated family	stable model + build matrix of roles/carriers
persistence demo	runtime + database client + command callbacks
system-facing app	several roles + configuration + persistence + diagnostics

This keeps the mental model stable. A larger application is not necessarily a different kind of thing. It may simply compose more roles, more layers, and more operational behavior.

The recurring structure is:

```
select layers
  -> create application object or role
      -> register behavior
          -> expose configuration
              -> start runtime
```

Optional dependencies and application availability

Some applications only exist when optional dependencies are available. That is part of the application model, not a minor CMake technicality.

Examples:

```
JSON support available
  -> JSON server target can be built

MariaDB support available
  -> database demonstration target can be built
```

This prepares the reader for later build and deployment chapters. A deployed system may not contain every possible SNode.C component. It contains the components enabled by selected modules, available dependencies, build configuration, install rules, and deployment purpose.

The application tree therefore teaches a practical lesson:

```
available framework capability
  -> build target
      -> installable executable
          -> deployed application
```

Reading a SNode.C application: a practical recipe

When reading a SNode.C application, use a repeatable method:

1. Start with its CMake target.
2. Read the linked components.
3. Check conditional build guards.
4. Identify whether this is a main application, focused example, utility, or demonstration.
5. Identify the executable entry point.
6. Find runtime initialization.
7. Identify application-side objects, configured communication roles, and registered runtime-visible instances.
8. Find contexts/factories, routes, middleware, command objects, or persistence objects.
9. Look for configuration hooks.
10. Look for diagnostics, state callbacks, and operational reporting.
11. Ask whether the build target and the entry point agree.

This recipe works well because SNode.C applications are often explicit about their layers. The build target and the entry point usually agree.

For example:

```
in-tree: target_link_libraries(... http-server-express net-in-stream-legacy ...)
external: target_link_libraries(... snodec::http-server-express snodec::net-in-stream-legacy ...)
-> expect an HTTP / Express-style application over an IPv4 legacy stream carrier
```

```

in-tree: target_link_libraries(... http-client net-in-stream-legacy ...)
external: target_link_libraries(... snodec::http-client snodec::net-in-stream-legacy ...)
    -> expect outgoing HTTP client behavior over an IPv4 legacy stream carrier

in-tree: target_link_libraries(... db-mariadb ...)
external: target_link_libraries(... snodec::db-mariadb ...)
    -> expect MariaDB client and command flow

in-tree: target_link_libraries(... core ...)
external: target_link_libraries(... snodec::core ...)
    -> expect runtime/core utility behavior

```

The reader should learn to move between CMake and C++. Both files are part of the application:

```

CMake tells what the application selects.
main.cpp shows how it assembles and runs it.

```

The reader should also distinguish local in-tree target names from installed `snodec::...` imported targets.

Build structure and operational clarity

A build target produces a binary and makes a choice visible.

For example:

```
one executable per carrier/role combination
```

may be clearer than:

```
one universal executable with every carrier hidden behind runtime branching
```

The echo family demonstrates this. A separate generated target can make each variant easier to test, install, run, and explain. That does not mean one executable per variant is always the right design. It means the build structure can express operational clarity when the variants are meaningful.

The JSON examples demonstrate a different form of clarity:

```

server target
    -> server-side behavior

client target
    -> client-side behavior

```

The database example demonstrates feature-dependent clarity:

```

MariaDB available
    -> database target exists

MariaDB unavailable
    -> database target is absent

```

These are application-design decisions. The build system records them.

What to remember

- `src/apps` shows how framework layers become executable targets.
- The chapter studies representative applications, not every target in the directory.
- In `SNode.C`, the build target often reveals the application architecture before the entry point is opened.
- In-tree applications use local target names; external applications use installed `snodec::...`

targets.

- A direct link line shows the application-facing component choices; component-owned dependency graphs may be deeper.
- The executable entry point is usually an assembly point for runtime initialization, application objects, routes, callbacks, configuration, diagnostics, and runtime start.

29.8 Closing perspective

Repository applications turn framework layers into executable programs. Chapter 30 widens that application view into system structure: several roles, deployment boundaries, configuration files, and operational relationships. Chapter 31 then studies MQTTSuite as a concrete reference ecosystem.

Chapter 30

From Applications to Systems

30.1 Why this chapter comes after applications

Chapter 29 showed how SNode.C framework pieces become executable applications: build targets, entry points, linked components, configured roles, callbacks, and runtime start. Chapter 30 widens the view again. The question is no longer only how one executable is assembled, but how several roles, boundaries, state responsibilities, and operational policies form a system.

An executable remains an important architectural object. It has a build target, an entry point, linked components, application-side objects, configured communication roles, instance names, configuration, and runtime behavior. But a communication framework becomes most useful when applications are no longer seen only as isolated endpoints. They become parts of larger arrangements:

- several communication roles,
- several protocol families,
- several deployment boundaries,
- several state boundaries,
- and one coherent operational architecture.

The question changes from:

How is one executable assembled?

to:

How do several roles become a system?

This is the progression from the previous chapters:

```
Chapter 29
-> executable applications as assembly points

Chapter 30
-> systems as role constellations

Chapter 31
-> MQTTSuite as a concrete reference ecosystem
```

A SNode.C system may be one executable with several named roles. It may also be several cooperating executables. It may run on one host, across several processes, or across several machines. The word *system* does not automatically mean a distributed cloud of services. It means that the design is now understood as a constellation of roles and boundaries rather than as one application in isolation.

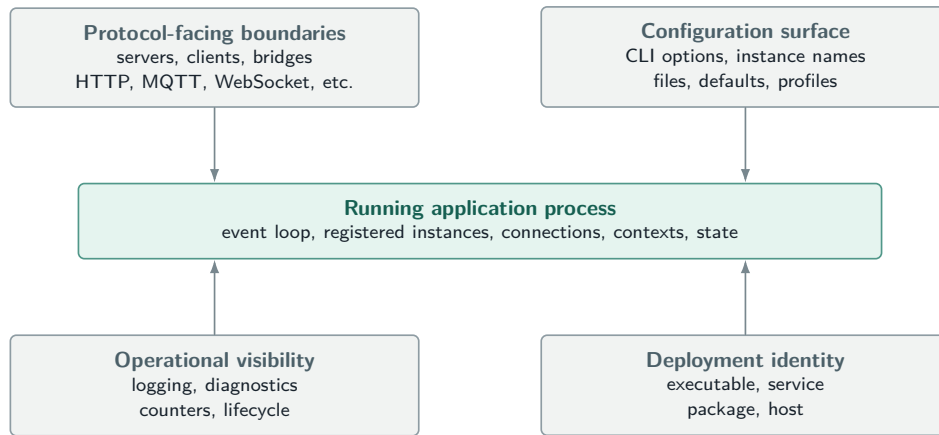


Figure 30.1: An application as a system role, with protocol-facing boundaries, configuration, operational visibility, and deployment identity around the running process.

30.2 From applications to role constellations

A system is not simply a larger single application. That distinction matters.

A larger application may still have one main role, one deployment boundary, and one dominant operational shape. A system introduces a constellation of concerns around the running process. Figure 30.1 shows the application as a system role rather than as an isolated program: protocol-facing boundaries, configuration, operational visibility, and deployment identity all meet at the running application process.

Figure 30.1 is intentionally not a build pipeline. The important point is that the running process is where configured instances, protocol boundaries, connection state, diagnostics, and deployment assumptions meet.

Those are architectural concerns, not only size concerns. A useful first question is therefore:

Which roles exist?

not only:

Which classes are instantiated?

In this chapter, a role is a system-design responsibility. A concrete SNode.C program may realize such a role through an application-side server or client handle, a configured communication role, and a registered runtime-visible instance. These terms should not be collapsed into one another. The role belongs to the system design; the configured communication role belongs to the SNode.C configuration surface; the registered runtime-visible instance is what the runtime can observe and operate.

A concrete system sketch

A small SNode.C-based monitoring and integration system might contain these roles:

```

admin-http
live-events
mqtt-ingest
bridge-client
local-control
database-state
  
```

A simple shape could look like this:

```

browser
-> admin-http
-> live-events
  
```

```

devices
  -> mqtt-ingest
      -> database-state

external platform
  <- bridge-client

operator tool
  -> local-control

```

This sketch is deliberately generic. Chapter 31 will make the idea concrete with MQTTSuite. Here, the sketch only introduces the way of thinking:

```

admin-http
  -> browser-facing administration boundary

live-events
  -> event stream for observation

mqtt-ingest
  -> machine-to-machine input boundary

bridge-client
  -> outgoing integration boundary

local-control
  -> host-local operational boundary

database-state
  -> persistence boundary

```

These are role names, not necessarily executable names. Some roles may be routes inside one application. Some may be configured communication roles. Some may be service-level responsibilities. Some roles are nested inside others: an SSE route may belong to a web role, while still being useful as a named observation boundary.

The role `database-state` is intentionally different from `admin-http` or `mqtt-ingest`. It is not the same kind of communication role as a socket server or client. It names the persistence boundary that owns durable application state.

The system is not defined by one large binary name. It is defined by the roles that cooperate. Some roles may live inside one executable. Some may be separate services. That is an architectural decision, not a limitation imposed by the framework.

Named roles as system vocabulary

One of the most useful mental models for SNode.C systems is the **named role constellation**.

A named role constellation is a set of system responsibilities whose names are stable enough to appear in code, configuration, logs, diagnostics, deployment discussion, and operator language. For example:

```

admin-http
  -> accepts browser-facing administrative requests

live-events
  -> streams live status to connected clients

mqtt-ingest
  -> receives or brokers device-oriented messages

bridge-client
  -> forwards selected information to another system

```

```
local-control
-> exposes host-local control or diagnostics
```

This is where SNode.C's named-instance model enters the system vocabulary. Where a role is realized by a SNode.C communication role, the configured instance name can identify:

- the configuration section,
- the log context,
- the role in an operator discussion,
- the failure location,
- the deployment responsibility,
- and the runtime behavior that should be observed.

Named roles are not noise. They help code, configuration, logs, diagnostics, deployment, and operator discussion use the same vocabulary.

A web-facing administration role should remain visibly different from a broker-facing MQTT role. A local control interface should remain visibly different from a public network interface. A streaming observation role should remain visibly different from a transactional control role. When the roles are explicit, the system becomes easier to reason about.

One executable or several services

A SNode.C system does not have to be packaged in exactly one way.

A serious system is not defined by being one binary, and it is not automatically improved by being split into many processes. Sometimes one executable with several roles is the best design. Sometimes several cooperating executables are better. Packaging is an operational boundary decision, not a measure of architectural maturity.

The choice should be guided by operational clarity.

Choose one executable when...	Choose several executables when...
the roles share one lifecycle	roles need independent restart
deployment should remain simple	privileges differ between roles
the roles are tightly coupled	scaling needs differ
failure of one role reasonably means failure of the whole application	failure domains should be separated
configuration should be managed as one unit	deployment boundaries should be explicit
local communication inside one process is sufficient	process or host boundaries are part of the architecture

This is not a moral distinction. A single executable is not automatically less serious. Several services are not automatically more architectural. The right boundary is the one that makes the system easier to deploy, operate, diagnose, and evolve.

30.3 Boundaries define the system

A system becomes understandable when its boundaries are visible before its features are listed.

Useful boundary questions include:

- Which roles are internal only?
- Which roles are public or externally reachable?
- Which roles are browser-facing?
- Which roles are machine-to-machine?
- Which roles are local to one host?
- Which roles require TLS?
- Which roles require retry or reconnect behavior?

- Which roles own durable state?
- Which roles should be observable independently?

The result is a set of explicit boundaries rather than a feature list.

Protocol boundaries

Chapter 27 discussed multi-protocol IoT thinking. Chapter 30 uses the same idea at the system-architecture level without repeating the IoT discussion.

The system-level question is not:

Which one protocol should the entire system use?

The better question is:

Which protocol belongs at which boundary?

A system may legitimately use:

- HTTP or an Express-style layer for administration,
- SSE for server-to-browser live updates,
- WebSocket for bidirectional browser interaction,
- MQTT for brokered machine communication,
- Unix-domain sockets for local control,
- IPv4 or IPv6 stream carriers for network-facing roles,
- TLS where the boundary requires encryption and authentication support.

The point is not to use many protocols for decoration. The point is to choose the protocol family that matches each boundary. SNode.C's component structure supports that kind of breadth, but system design still has to decide where each protocol belongs.

Local, network-facing, and upgraded boundaries

A single system can mix different boundary types coherently.

For example:

```
local-control
  -> Unix-domain socket on the same host

admin-http
  -> IPv4 or IPv6 HTTP interface

live-events
  -> SSE endpoint mounted into the web role

interactive-view
  -> WebSocket upgrade path

mqtt-ingest
  -> MQTT role for machine communication

device-near-link
  -> Bluetooth-facing lower communication in device-near contexts
```

The previous chapters introduced these communication families one by one. The system-level lesson is that they can coexist without forcing a different programming model for each one.

```
local boundary
  -> same-host control or diagnostics

network boundary
```

```
-> externally reachable service or integration path

upgraded boundary
-> HTTP-negotiated transition to WebSocket behavior
```

Different boundaries can remain different while the architecture remains consistent. Bluetooth-facing communication, for example, belongs near device edges or nearby peer interaction. It should not be treated as a general integration bus merely because it is another available lower family.

Stateful and stateless roles

System design also requires a clear distinction between stateful and stateless roles.

```
stateless role
-> can often be restarted or replaced with little coordination

stateful role
-> owns information whose correctness outlives a single request,
    connection, or message
```

A stateless role may serve requests, forward messages, expose an interface, or adapt one protocol to another. A stateful role owns information that must survive beyond one request or one connection. That may include:

- configuration-derived runtime state,
- retained or cached observations,
- device state,
- session-related state,
- database-backed application state,
- integration progress,
- or persistent domain data.

Chapter 28 introduced persistence as an application-state boundary. In a system, persistence becomes an ownership question:

```
Which role owns the state?
Which roles may read it?
Which roles may change it?
Which roles only observe it?
Which failure mode can leave it inconsistent?
```

Without state ownership, a system sketch is only a communication sketch.

30.4 System operation

Once roles and boundaries are visible, operation becomes more concrete. Configuration, diagnostics, failure behavior, and build structure no longer describe isolated applications only. They describe the system surface.

Configuration

At the application level, configuration shapes one communication role. At the system level, configuration can describe a constellation of roles.

Chapters 16 and 17 introduced named instances and structured configuration. At system scale, those ideas become a role map. The purpose of the following pseudo-configuration is not syntax. It is to show how names can make role boundaries visible.

```
[admin-http]
role = http
```

```
listen = 0.0.0.0:8080
tls = false

[live-events]
role = sse
mount = /events
parent = admin-http

[mqtt-ingest]
role = mqtt-client
broker = mqtt.example.internal
client-id = monitoring-ingest
reconnect = true

[bridge-client]
role = http-client
target = https://integration.example/api

[local-control]
role = unix-domain
path = /run/example/control.sock

[database-state]
role = mariadb
database = monitoring
```

This is not meant as exact SNode.C syntax. It shows the architectural idea: named roles make the system visible.

Once names like `admin-http`, `mqtt-ingest`, and `local-control` exist, the rest of the operational story becomes easier:

- command-line options can be grouped by role,
- configuration files can be read as system descriptions,
- logs can identify which role produced an event,
- diagnostics can be localized,
- and deployment discussions can use the same vocabulary as the code.

Configuration is no longer only setup. It can become one of the clearest descriptions of the system's role constellation and boundaries.

Diagnostics and observability

At application scale, diagnostics help a developer understand one program. In a system, diagnostics become observability across role boundaries.

The important questions change:

```
Which role produced this log entry?
Which connection belongs to which boundary?
Which retry loop belongs to which integration?
Which named instance failed?
Which part of the system is still healthy?
```

A useful diagnostic fact should keep the boundary visible:

```
which role
which configured instance
which connection
which retry loop
which state owner
```

SNode.C already contains ingredients for this kind of visibility:

- named instances,
- role-oriented configuration,
- logging,
- diagnostics,
- connection identity,
- generated command-line structure,
- runtime status information.

A system may still need external monitoring, dashboards, metrics storage, or log aggregation. `SNode.C` does not replace those tools. It gives the system a clear internal vocabulary that those tools can build on.

Failure behavior

At application scale, retry and reconnect behavior may look like local communication policy. In a system, it becomes topology policy.

The architect now asks:

- Which role may reconnect aggressively?
- Which role should fail fast?
- Which role is optional?
- Which role should wait because another service may come up later?
- Which role must avoid retry storms?
- Which role should degrade gracefully?
- Which role owns state that must be protected during failure?

Chapter 20 separated timeout, retry, reconnect, disablement, shutdown, and failure state. Chapter 30 applies that vocabulary to a constellation of roles.

A reconnect policy for `mqtt-ingest` may be reasonable. The same policy for `database-state` may be dangerous if it hides a persistent state problem. A database role may require visible degraded-state behavior rather than silent retry, because persistence failures can corrupt the system's understanding of durable state. A local control interface may not need the same failure behavior as an external integration client.

Failure behavior belongs to the role that owns the boundary, not merely to the socket that reports the error.

Build structure as the first system map

Chapter 29 showed that application targets reveal application shape. In a system, the build structure also reveals the system surface.

Separate libraries and executables tell the reader something about:

- what can be deployed independently,
- what is reusable as a protocol or support component,
- what is a higher-level composition,
- which optional features depend on external libraries,
- and which application targets represent runnable roles.

The build system does not describe the whole runtime topology. Configuration, deployment, and operation complete that picture. But the build system gives the first map.

30.5 Stable protocol cores and domain code

Chapter 15 introduced an important idea: protocol logic can often remain stable while lower carriers change. In a system, that idea becomes a design strategy.

A system is easier to evolve when:

`protocol core`

```
-> remains stable

carrier / deployment boundary
-> can change around it
```

The earlier carrier-change idea now becomes a system-evolution strategy: keep protocol or domain logic stable where possible, and let carrier or deployment boundaries change around it deliberately.

For example, a message-oriented domain protocol may begin as a native internal service. Later it may also be exposed through a WebSocket path, an HTTP-facing endpoint, or an MQTT integration boundary. The system remains easier to evolve if the protocol logic is not fused unnecessarily to one carrier or deployment shape.

This is where the earlier layer discipline pays off. SNode.C gives the architect communication roles, protocol layers, configuration structure, diagnostics, runtime behavior, and reusable web and messaging facilities.

It does not replace domain-specific code. Real systems still need:

- business rules,
- device models,
- integration mapping,
- authorization policy,
- database semantics,
- scheduling policy,
- orchestration,
- user-facing behavior,
- and deployment-specific decisions.

The responsibility split is useful:

```
SNode.C
-> communication architecture, runtime, roles, configuration,
    diagnostics, reusable protocol layers

domain code
-> business rules, device model, authorization, persistence semantics,
    orchestration, user-facing behavior
```

The framework's job is to give domain logic a clear communication architecture to live in. That separates communication structure from domain decisions without pretending that either one can replace the other.

30.6 Reading a SNode.C system

Chapter 29 gave a way to read an application. A SNode.C system can be read with a wider checklist:

1. Which roles exist?
2. Which roles share an executable?
3. Which roles cross process, host, or network boundaries?
4. Which protocol family belongs at each boundary?
5. Which role owns which configuration surface?
6. Which role owns durable state, and which roles only read, update, or observe it?
7. Which roles are stateless adapters?
8. Which roles require TLS, retry, reconnect, disablement, or degraded-state behavior?
9. Which roles must be observable independently?
10. Which parts may fail or restart independently?
11. Which protocol cores should remain reusable if carriers or deployment boundaries change?

This checklist is not a rigid method. It is a way to prevent system design from collapsing into a pile of features.

A framework can tolerate some inconsistency in tiny examples. In a system, inconsistency becomes expensive. This is why SNode.C's consistency matters:

- roles remain visible,
- lower families remain explicit,
- protocol layers remain composable,
- configuration remains structured,
- diagnostics use the same vocabulary,
- failure policy remains role-oriented,
- and build targets still expose important architectural choices.

That consistency allows a developer, maintainer, or operator to keep thinking clearly even when several communication styles are present at once.

This is the real transition from applications to systems.

What to remember

- A system is not simply a bigger application; it is a constellation of named roles and boundaries.
- Role names should align code, configuration, logs, diagnostics, deployment, and operator discussion.
- Protocol choice, packaging, persistence, diagnostics, and failure behavior are boundary decisions.
- Stable protocol cores and domain logic are easier to evolve when they are not fused unnecessarily to one carrier or deployment shape.
- SNode.C provides communication architecture; domain code still owns domain semantics.

30.7 Closing perspective

This chapter moved from executable applications to communication systems: roles, boundaries, protocol families, deployment shape, configuration, observability, persistence, and failure topology. Chapter 31 applies that vocabulary to MQTTSuite as an ecosystem, not merely as a list of programs.

Chapter 31

MQTTSuite as a Reference Ecosystem

31.1 From system vocabulary to a concrete ecosystem

Chapter 30 introduced the vocabulary of systems: named roles, boundaries, protocol families, configuration, observability, persistence, and failure behavior. Chapter 31 applies that vocabulary to MQTTSuite, not as a second manual, but as a concrete SNode.C-based ecosystem whose applications make those ideas visible.

MQTTSuite is the concrete reference ecosystem for the system vocabulary developed in Chapter 30. A reference ecosystem is not the only possible way to build systems with SNode.C. It is a source-backed example that shows how the design vocabulary can be used in a real family of cooperating applications.

It shows how SNode.C concepts become a family of MQTT-centered tools: broker, integrator, bridge, command-line client, persistence service, and shared support libraries. A compact model is:

```
SNode.C framework ideas
-> MQTT and web infrastructure
    -> focused applications
        -> operational ecosystem
```

MQTTSuite is therefore architectural evidence: SNode.C concepts can be combined into a coherent communication ecosystem without turning every concern into one monolithic executable.

Figure 31.1 shows the suite as a role ecosystem around the MQTT broker role and topic space. MQTT clients and operational tools, MQTTStore, MQTTBridge, and MQTTIntegrator all touch the same MQTT-oriented center from different system boundaries.

The figure is intentionally not a pipeline. A deployment may use only the broker, one bridge, one store, one integrator, or several cooperating processes. The important point is that each tool occupies a distinct system role around topic flow, client state, persistence, bridging, and integration.

31.2 Ecosystem shape

The first architectural fact is that MQTTSuite is not one large executable with every feature hidden behind switches. It is structured as focused applications plus shared support where useful. That is the first system lesson.

The suite exposes the following visible application roles:

```
MQTTBroker
MQTTIntegrator
MQTTBridge
MQTTCli
```

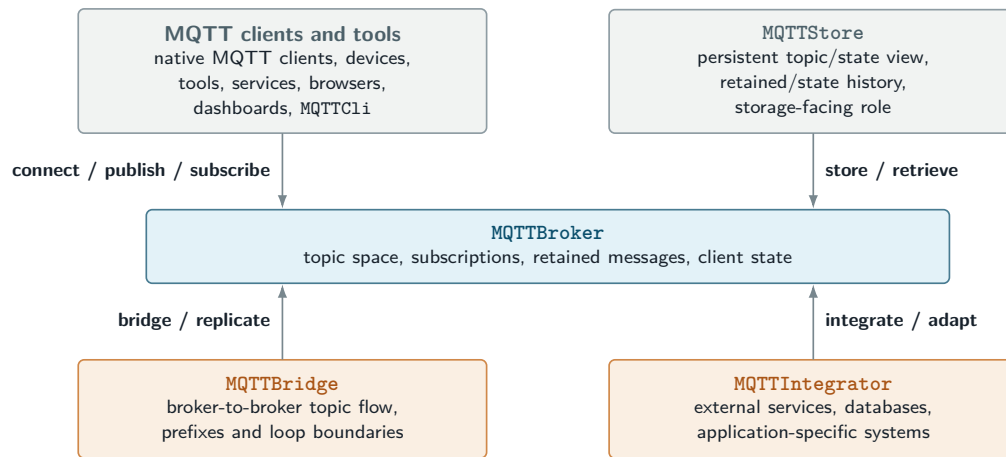


Figure 31.1: MQTTSuite as an ecosystem around the MQTTBroker role and topic space.

MQTTStore

The repository and build layout use the corresponding lower-case directory and target vocabulary:

```

lib
mqttbroker
mqttintegrator
mqttbridge
mqttcli
mqttstore

```

That layout is already a system lesson: shared infrastructure belongs in libraries, operational roles become executables, and application boundaries remain visible. A system does not always mean one executable. Sometimes several focused tools sharing the same architectural vocabulary make the system clearer.

MQTTSuite demonstrates that clearly.

A first role map

A compact way to read the suite is:

```

MQTTBroker
-> central MQTT broker role
-> web/admin surface
-> live observation surface
-> MQTT-over-WebSocket entry point

MQTTIntegrator
-> subscribes
-> maps and transforms
-> republishes

MQTTBridge
-> connects MQTT worlds
-> moves selected traffic between broker connections

MQTTcli
-> command-line access for operation and testing

MQTTStore
-> MQTT-to-MariaDB persistence boundary
-> raw envelope storage
-> optional projection into typed tables

```


The sketch is an architectural map, not a deployment prescription. A concrete installation may use all of these applications, or only one or two of them. The point is that each application has a recognizable ecosystem role.

MQTTSuite roles at a glance

The applications can be summarized like this:

Component	Ecosystem role	Architectural lesson
<code>mqttbroker</code>	broker plus web/admin and observation surfaces	one executable can host several related broker boundaries
<code>mqttintegrator</code>	mapping-driven integration service	application semantics can live above MQTT core
<code>mqttbridge</code>	broker-to-broker traffic bridge	topology and selected traffic movement are separate from mapping
<code>mqttcli</code>	command-line MQTT utility	the same protocol stack can become an operational tool
<code>mqttstore</code>	MQTT-to-MariaDB persistence service	MQTT publishes can become durable and queryable state
<code>mqtt-mapping</code>	shared mapping/admin support where needed	selected applications can share integration semantics without becoming the same application

The table shows the suite as a set of roles rather than as a flat feature list. It also keeps shared support separate from executable applications. `mqtt-mapping` is important, but it is not the whole ecosystem and it is not an application executable.

Reading the repository

A useful reading path is:

```
top-level CMakeLists.txt
  -> suite structure

lib/CMakeLists.txt
  -> shared mapping support

mqttbroker/CMakeLists.txt
  -> broker and web/admin surface

mqttintegrator/CMakeLists.txt
  -> mapping integration and administration

mqttbridge/CMakeLists.txt
  -> bridge topology and optional carriers

mqttcli/CMakeLists.txt
  -> operational client tool

mqttstore/CMakeLists.txt
  -> persistence-facing MQTT client role
```

This is the same build-first reading method from Chapter 29, now applied to a multi-application repository. Only after the build shape is clear should the corresponding entry points, factories, configuration classes, and route handlers be opened.

31.3 Shared infrastructure where it belongs

The `lib` directory is important because it contains shared support code. One visible piece is the `mqtt-mapping` library.

It contains mapping-related infrastructure such as mapping reading, mapping execution, schema validation and schema generation, template support, mapping administration routing, and configuration helpers. This shared layer is used where mapping and mapping administration belong, especially around broker and integrator behavior.

`mqtt-mapping` is not the center of the whole ecosystem. It is not used by every application in the same way. It is shared infrastructure for the applications that need mapping and mapping administration behavior.

MQTTSuite should therefore not be reduced to:

```
mqtt-mapping plus applications
```

A better reading is:

```
focused MQTT applications
+ shared support where useful
+ common SNode.C architectural vocabulary
```

Mapping as application semantics

Mapping is an important architectural lesson because it is clearly above MQTT core. MQTT itself provides topics, subscriptions, publishes, QoS, retain flags, and session behavior. Mapping is a higher-level integration concern. It decides how selected topics and payloads should be transformed and republished.

A mapping may say:

```
subscribe to this topic pattern
-> extract this payload information
    -> transform it
        -> publish the result to another topic
```

That is not broker behavior. It is integration behavior. The lower MQTT layer should not become a template engine, a mapping editor, or a domain-rule interpreter. The mapping layer belongs above it, and the administration layer belongs above the mapping layer.

A useful shape is:

```
MQTT protocol layer
-> message transport and broker/client semantics

mapping layer
-> integration semantics

administration layer
-> controlled runtime management of mapping behavior
```

The administration surface can then be discussed in terms such as inspection, deployment or reload, persistence, history, and rollback where supported by the concrete application. The important architectural point is not the exact route vocabulary. The important point is that changing integration rules is an application-management concern above MQTT core.

This is the same layered thinking used throughout the framework.

31.4 MQTTBroker: broker role plus web administration

MQTTBroker is the central broker application in the suite. From a SNode.C perspective, it is especially useful because it combines MQTT brokerage with additional application roles.

MQTTBroker is one executable, but not one boundary. It combines related roles inside one operational broker application:

```
native MQTT server roles
TLS MQTT server roles
Unix-domain MQTT server roles
HTTP/Express web roles
static web interface
JSON administration routes
SSE live event endpoints
MQTT-over-WebSocket upgrade paths
```

This is Chapter 30's role-constellation idea in a concrete executable. One executable can host several related communication roles when those roles belong to one operational boundary. The broker owns the MQTT broker function. The web surface supports administration and observation. The WebSocket upgrade path allows MQTT to cross a web-compatible boundary.

Those are different boundaries, but they form one coherent broker application.

MQTT server role

The broker is first of all an MQTT broker. It exposes direct MQTT listener roles such as:

```
plain MQTT/TCP listeners
MQTT/TCP listeners with TLS
Unix-domain MQTT listeners, where enabled
```

These listeners handle brokered machine messaging directly. They are not a side effect of the web surface. They are the broker's primary machine-facing boundary.

Web-facing administration and observation

MQTTBroker should also be read as a web-facing application. That connects several earlier chapters:

```
MQTT server layer
-> plain MQTT/TCP listeners
-> MQTT/TCP listeners with TLS
-> Unix-domain MQTT listeners, where enabled

HTTP layer
-> Express-like composition
    -> static assets
    -> JSON administration routes
    -> SSE live events
    -> WebSocket upgrade
        -> MQTT subprotocol
```

Direct MQTT listeners handle brokered machine messaging directly. HTTP/Express handles web-facing and administrative interaction. SSE supports live observation. WebSocket provides the upgraded HTTP path for MQTT-over-WebSocket, while MQTT remains the selected WebSocket subprotocol.

These are not competing mechanisms. They are different conversations at different boundaries.

Administration is itself a boundary. It is not UI decoration. It is the operational surface through which broker state, client behavior, retained data, subscriptions, mapping behavior, or live observation can become visible and manageable.

Named instances

The broker uses names such as:

```
in-mqtt
in-mqtts
in6-mqtt
in6-mqtts
un-mqtt
un-mqtts
in-http
in-https
in6-http
in6-https
un-http
un-https
```

The exact available instances depend on the build configuration and enabled roles. The naming pattern is the lesson.

The names encode:

```
address family
-> IPv4, IPv6, Unix-domain

protocol role
-> MQTT, HTTP

security mode
-> legacy/plain or TLS
```

Where such a name belongs to a configured communication role, it can also become a registered runtime-visible instance. That makes the broker easier to configure, log, operate, and discuss. Good instance names are part of the architecture. They are not cosmetic labels.

31.5 Integration and topology: MQTTIntegrator and MQTTBridge

MQTTIntegrator and MQTTBridge both connect MQTT worlds, but they do not solve the same problem. One is mainly about transformation. The other is mainly about topology.

Keeping this distinction visible prevents the ecosystem from becoming a vague collection of MQTT clients:

```
MQTTIntegrator
-> transformation / mapping semantics

MQTTBridge
-> topology / selected traffic movement
```

MQTTIntegrator

MQTTIntegrator does not primarily own the MQTT universe as a broker. It connects to MQTT brokers, subscribes according to mapping rules, transforms selected traffic, and republishes mapped results.

A useful shape is:

```
MQTT client role
-> receives selected publishes
    -> mapping layer interprets rules
        -> publishes transformed output
```

This makes MQTTIntegrator the clearest example of application semantics above MQTT core. Broker and integrator both live in the MQTT world, but they do different work. MQTTBroker accepts and distributes client traffic. MQTTIntegrator interprets and transforms traffic.

The distinction is the same kind of role clarity developed throughout this part.

Administration for integration

MQTTIntegrator combines its transformation loop with an HTTP/Express administration role for mapping management.

The application combines:

```
MQTT client role
  -> machine-message integration

HTTP/Express administration role
  -> mapping inspection and management

shared mapping support
  -> domain-specific integration rules
```

This is a common reference pattern. A long-running integration service often needs both a machine-facing protocol role and an operator-facing administration role. SNode.C makes that composition natural without collapsing the roles into one opaque block.

Mapping lifecycle and controlled change

Mappings can become managed application state instead of remaining static startup data.

A compact conceptual model is:

```
mapping data
  -> inspection
    -> deployment or reload
    -> persistence
    -> history or rollback where supported
```

The MQTT protocol layer still transports messages. The mapping layer expresses integration rules. The administration layer controls changes to those rules.

That separation prevents the lower protocol layer from becoming responsible for application policy.

MQTTBridge

MQTTBridge solves a different integration problem. It is not the mapping story repeated. It is the topology story.

A bridge connects MQTT worlds to each other. At the architectural level, it can manage logical bridge definitions, broker connections, selected topic movement, and loop-prevention policy.

Its responsibility can be read as:

```
broker connection A
  -> selected topic traffic
    -> bridge policy
    -> broker connection B
```

MQTTBridge does not primarily own the MQTT universe like a broker. It does not primarily transform payloads like an integrator. It moves selected traffic among broker connections. That makes it a separate ecosystem role.

Systems often need both transformation and topology management. Those are not the same concern.

Optional extended carriers

Most deployments can be understood through ordinary IP-based broker connections first. Optional lower SNode.C components simply show that the bridge role is not conceptually tied to one transport worldview.

For example, optional Bluetooth L2CAP and RFCOMM stream components may be available where the build and platform support them. They should be read as carrier flexibility for specific deployment situations, not as the normal starting point for every bridge design.

A SNode.C-based bridge can remain an MQTT bridge while the lower connectivity options vary. This is the kind of variation a layered framework can support.

31.6 Operational and persistence roles

Not every important ecosystem member is a large service. Some roles exist because practical systems need operational access and durable state. MQTTCli and MQTTStore represent those edges.

MQTTCli

MQTTCli is the operator and developer surface of the ecosystem. It can publish, subscribe, test paths, inspect connectivity, and exercise the same MQTT client infrastructure that larger applications use.

The architectural lesson is:

```
same SNode.C MQTT/client transport stack
-> packaged as a command-line tool
```

Not every SNode.C application has to be a daemon, broker, web application, or long-running integration service. Some applications are focused operational tools. They still benefit from the same configuration, transport, TLS, WebSocket, and runtime model.

MQTTStore

MQTTStore brings the persistence boundary into the ecosystem. It is the persistence-facing member of the suite.

At the MQTT edge, it behaves as an MQTT client. At the persistence edge, it uses MariaDB-oriented storage. That makes it a direct continuation of Chapter 28's persistence-boundary model.

Its safe default is to store received MQTT publishes as raw MQTT envelopes. That means the database can preserve important MQTT-level information such as:

```
connection name
topic
payload
QoS
retain flag
duplicate flag
packet identifier
```

The current storage implementation can also classify payloads and preserve different views of them, such as raw payload data, optional text representation, optional parsed JSON, and a payload-format classification such as JSON, text, or binary.

This preserves the original message shape.

But MQTTStore can also go further. It can project selected JSON payload fields into typed database tables. That makes it not only a subscriber with a database connection, but a persistence boundary where MQTT traffic can become queryable application state.

A useful model is:

```
MQTT publish
  -> raw envelope storage
      -> optional JSON interpretation
          -> optional typed table projection
```

The first level preserves what arrived. The projection level gives the application a more queryable database shape. Both belong to the persistence boundary.

Projection as state design

The projection capability should not be treated as a small convenience feature. It changes the system question.

Without projection, MQTTStore mainly answers:

```
What MQTT publishes arrived?
```

With typed table projection, it can also help answer:

```
What application state can be derived from those publishes?
```

That requires design choices:

- which topic filters should be stored,
- which payloads are expected to be JSON,
- which fields matter,
- which database table owns the projection,
- which SQL types should represent the projected values,
- and how raw storage and projected state relate to each other.

The projection model makes persistence a system design concern. Persistence is a system boundary with its own design rules.

31.7 System surfaces

The applications differ, but the recurring communication surfaces remain familiar. This is why MQTTSuite works well as a reference ecosystem.

Transport vocabulary

Across MQTTSuite, the same transport vocabulary appears repeatedly. A simplified view is:

Pattern	Examples
direct MQTT	MQTT over IPv4, IPv6, or Unix-domain streams
secured MQTT	TLS variants where enabled
MQTT over WebSocket	MQTT crossing a web-compatible upgraded boundary
web administration	HTTP/Express over selected carriers
command-line operation	MQTT client behavior packaged as a CLI
persistence	MQTT client behavior combined with database storage

The exact combinations vary by application and build configuration. That is fine. The architectural point is that the roles change while the lower vocabulary remains familiar.

A developer can move from broker to integrator to bridge to CLI to store without learning a completely unrelated communication model each time.

Build options

MQTTSuite's CMake files are part of the architecture story. They expose feature surfaces such as:

```
TCP IPv4
TCP IPv6
Unix-domain sockets
TLS variants
WebSocket / WSS variants
HTTP/Express administration roles
MQTT client or server components
optional lower carriers where available
```

The same build-system lesson appears here again. The build system tells the reader which roles and communication boundaries the application can expose.

Configuration and repeatable role constellations

MQTTSuite also demonstrates a useful operational style:

```
start with explicit options
  -> verify the instance constellation
      -> persist selected configuration where supported
          -> restart later from repeatable configuration
```

The architectural point is that command-line and configuration surfaces can describe repeatable role constellations. Mapping configuration also has explicit persistence support.

The command line can shape a role constellation. Persisted configuration can then become operational state. That style is especially useful for routers, SBCs, embedded Linux systems, lab setups, and other environments where repeatable startup matters.

OpenWrt and small-footprint deployment

MQTTSuite points toward resource-constrained systems such as embedded Linux, routers, and SBCs. That fits the broader SNode.C story.

The suite is built from focused applications rather than one giant server platform. It uses an event-driven framework. It exposes explicit roles. It can persist selected configuration. It can be deployed as a set of compact tools.

This does not mean that every deployment must use OpenWrt, and it does not turn this chapter into package documentation. It means the architecture is compatible with constrained operational environments. That is an important part of the deployment story.

31.8 Reading MQTTSuite architecturally

Read MQTTSuite by roles and boundaries first; read command-line options, mapping schema fields, bridge JSON properties, database columns, and web UI routes second.

An architectural reading of MQTTSuite focuses on roles and boundaries rather than on every detail of every tool. The architectural view is narrower:

```
real SNode.C applications
  -> several focused executables
      -> shared support where useful
          -> web/admin surfaces
              -> bridging, CLI, and persistence roles
```

A possible deployment flow might look like this:


```
device publishes MQTT
  -> MQTTBroker

MQTTIntegrator
  <- subscribes to selected broker traffic
  -> republishes mapped results

MQTTBridge
  <- forwards selected broker traffic
  -> another broker world

MQTTStore
  <- subscribes to selected broker traffic
  -> raw storage and typed projections

MQTTCLI
  -> observes, tests, or injects traffic
```

This is only a teaching view. A real deployment may use a different topology or only a subset of the tools.

The point is cooperation. MQTTSuite shows how SNode.C-based applications can remain focused while still forming a larger system.

What to remember

- MQTTSuite is a reference ecosystem, not a second framework manual.
- `mqttbroker` demonstrates MQTT listener roles, web/admin roles, live observation, and MQTT-over-WebSocket.
- `mqttintegrator` demonstrates mapping-driven integration semantics above MQTT.
- `mqttbridge` demonstrates topology management and selected traffic movement between MQTT worlds.
- `mqttcli` demonstrates the MQTT stack as an operational tool.
- `mqttstore` demonstrates a persistence boundary: raw MQTT envelope storage plus optional typed projections.
- Shared support exists where it serves reusable ecosystem semantics, such as mapping and mapping administration.
- The build system and configuration surfaces expose much of the system surface.

31.9 Closing perspective

MQTTSuite is best understood here as evidence that SNode.C can support a coherent ecosystem of focused tools: a broker, an integrator, a bridge, a command-line client, a persistence service, and shared support libraries.

That makes MQTTSuite the right closing example for this part of the book.

Part IX moved from persistence, to applications, to systems, and finally to a concrete SNode.C-based reference ecosystem.

Part X turns from systems to building, porting, and maintaining. Chapter 32 returns to CMake components and linking strategy with this larger system view now in place.

Part X

Building, Porting, and Maintaining

This part treats build, deployment, testing, debugging, and benchmarking as architectural surfaces. CMake targets, install components, package dependencies, runtime paths, deployed services, and confidence strategies are all part of the framework story.

The goal is to show how a layered design remains trustworthy outside the source tree.

Chapter 32

CMake Components, Public Headers, and Linking Strategy

32.1 Why CMake starts Part X

Chapter 31 closed Part IX by reading MQTTSuite as a concrete SNode.C-based ecosystem. Chapter 32 opens Part X by asking how such ecosystems are built, linked, installed, exported, and consumed. The subject is CMake, but the real topic is architectural visibility in the build system.

Part IX looked at applications, systems, and a reference ecosystem. Part X turns toward build structure, component selection, packaging, porting, optional dependencies, and long-term maintenance. That makes CMake the right first topic.

For SNode.C, CMake is one of the places where the framework declares its own architecture, while also turning source files into libraries and executables. The public include tree is the source-side counterpart of that architecture: headers are not only files to make declarations visible, but also front doors into selected C++ abstraction stacks. The build structure expresses many of the same ideas that appeared earlier:

- layers,
- components,
- protocol families,
- lower carriers,
- legacy and TLS variants,
- optional dependencies,
- installable package components,
- exported targets,
- consumer-facing component selection,
- installed public headers,
- and source-facing include selection.

This chapter is therefore not a generic CMake tutorial. It is about how SNode.C uses CMake to preserve architectural clarity.

The central idea is:

CMake is not only build mechanics.
Public headers are not only textual declarations.
Together they declare how SNode.C is consumed.

32.2 The build structure as architecture

A beginner may look at a CMake build structure and see only commands:

- `add_library`,
- `target_link_libraries`,
- `install`,
- `add_subdirectory`,
- `configure_package_config_file`.

Those commands matter, but in a framework like SNode.C the more important question is:

What does the build structure reveal about the architecture?

The answer is: a great deal. The build structure shows which parts are lower runtime infrastructure, which parts are protocol layers, which parts are transport/family compositions, which parts are optional features, which parts are exported components, and which parts are applications.

Top-level project shell

The top-level `CMakeLists.txt` stays small. It declares the project metadata, sets the version, extends the module path, includes helper modules such as formatting, Doxygen, uninstall, and graph visualization support, descends into `src`, and then includes packaging.

That is the right division of responsibility:

```
top level
-> project shell

src
-> framework shape

packaging
-> distribution
```

A top-level file that tried to describe every target directly would hide the architecture in one overloaded script. SNode.C does the opposite. It delegates target construction to the module tree.

src as the structural center

The real structural center of the build is `src/CMakeLists.txt`. That file does several important things:

- checks the supported compiler baseline,
- sets the C++ standard,
- applies warning and linker policy,
- defines the in-tree build context,
- applies logging-related compile definitions,
- descends into the major framework modules,
- computes target dependencies,
- declares supported installable components,
- and generates the exported package configuration.

This is where the build becomes an inventory of what the framework believes its component surface is. The install rules in the module directories also define the matching public include surface: selected headers are installed below the SNode.C include root so external applications can include the same public front doors that the examples use in-tree.

The supported component list is especially important. It includes core runtime pieces, stream legacy/TLS pieces, network-family variants, HTTP, Express, WebSocket, MQTT, MQTT-over-WebSocket, database support, and more. the CMake component list is also an architectural table of contents.

Compiler policy

SNode.C requires a modern compiler baseline. The build checks for sufficiently recent GNU or Clang versions and then sets:

```
set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)
```

This matters. SNode.C is not presented as old-style portable C++ that happens to compile with anything. It is a modern C++ framework, and the build makes that identity explicit.

The compiler baseline is part of the framework contract. A port must provide a compiler sufficiently modern for the framework's language and diagnostic expectations.

Warning policy

The build enables a strict diagnostic posture, including warnings such as:

- -Wall,
- -Wextra,
- -Wconversion,
- -Wuninitialized,
- -Wunreachable-code,
- -Wfloat-equal,
- and -Werror.

This is a maintenance decision. Warnings are not treated as harmless background noise. They are treated as build-breaking signals.

That is demanding, especially in a cross-platform framework, but it has real benefits. Questionable conversions are caught early, unreachable code becomes visible, compiler upgrades reveal problems immediately, and warning regressions cannot accumulate silently.

The important lesson is:

Build-policy rule

build policy is code-quality policy.

A strict build is one of the ways a systems framework protects itself over time.

Strict does not mean naïve

The build is strict, but it is not naïve. It also includes carefully chosen suppressions for warnings that would be unhelpful, compiler-specific, or platform-specific.

Examples include suppressions for shadowing, ABI notes on Raspberry Pi, Android/Termux deprecations, and several Clang-specific diagnostics. That is the right style:

```
strict default
+ documented practical suppressions
  -> maintainable diagnostic policy
```

A serious build does not blindly enable every warning forever and pretend that context does not matter. It chooses a strict default, then documents exceptions through build flags. This is especially important for a framework that targets Linux desktops, embedded Linux, routers, and other constrained systems.

The build must be disciplined and practical at the same time.

Linker policy

The build treats compiler warnings strictly and applies a strict linker posture:

```
add_link_options(LINKER:--as-needed LINKER:--no-undefined)
```

This belongs to the same dependency-hygiene story.

`--as-needed` discourages unnecessary linkage. `--no-undefined` requires shared libraries to declare the dependencies they need instead of relying on a final application link step to accidentally complete missing symbols.

For SNode.C's component model, a target should not just compile; it should have a truthful link face. Dependencies belong to the component that needs them.

In-tree and installed build contexts

SNode.C distinguishes between building the framework itself and consuming the installed framework. The in-tree build defines `SNODEC_INTREE_BUILD`. That is useful, but it is not the consumer-facing package interface.

Inside the SNode.C source tree, targets can refer to local target names such as:

```
http-server-express
net-in-stream-legacy
core-socket-stream-legacy
```

Outside the source tree, an application should use the installed package interface and the exported namespaced targets:

```
snodec::http-server-express
snodec::net-in-stream-legacy
snodec::core-socket-stream-legacy
```

Those are different build contexts. The architecture is the same, but the target names belong to different views of the build.

32.3 Component targets, public headers, and dependency surfaces

The word *component* appears in two related senses.

At the CMake package level, a component is a selectable install/package component. At the architecture level, a component is a named framework piece represented by a target. SNode.C tries to keep these meanings aligned, but they are not literally the same concept in CMake semantics.

That is why the target names matter. They are not arbitrary labels. They are part of the public shape of the framework.

Libraries mirror layers

The library target names in SNode.C are architectural statements. Names such as `net-in-stream-legacy`, `http-server-express-legacy-in`, `websocket-client`, and `mqtt-client-websocket` tell the reader which family, role, protocol layer, or carrier composition is being selected.

For example:

```
net-in-stream-tls
```

says:

```
network family: IPv4
transport form: stream
connection handling: TLS
```

Likewise:

```
mqtt-client-websocket
```

says:

```
protocol family: MQTT
role: client
carrier composition: WebSocket
```

This is why target names are not cosmetic in SNode.C. They are part of the public build surface.

A public component graph read from logger upward

The public component graph can also be read from lower shared targets upward. The following view starts at `logger`, because `logger` is a useful starting point: many public dependency paths eventually reach it through `utils`. From there, the view expands through `utils`, `core`, socket layers, network families, HTTP/Express, WebSocket, MQTT, and database support.

This is a selected public component dependency view, read bottom-up from lower shared targets toward targets that publicly depend on them. It is not a complete list of source files and not a literal source-directory tree. System libraries, generated helper targets, private implementation details, and some optional platform-specific components are either omitted or shown only as short leaf notes.

Bluetooth RFCOMM (`net-rc`) and L2CAP (`net-l2`) branches, if available, are shown as optional branches. Some targets have more than one relevant public path. The ASCII drawing is tree-shaped for readability, but the real dependency structure is a graph, not a tree. Shared nodes are repeated or summarized because the purpose here is to show selected public paths.

```
logger
|-- utils
|   |-- websocket
|   |   |-- websocket-server
|   |   |   |-- http-server, shared path
|   |   |   |-- mqtt-server-websocket
|   |   |       |-- mqtt-server, shared path
|   |   |-- websocket-client
|   |       |-- http-client, shared path
|   |       |-- mqtt-client-websocket
|   |           |-- mqtt-client, shared path
|   |-- core
|   |   |-- db-mariadb
|   |   |   |-- libmariadb, if available
|   |   |-- core-socket
|   |   |   |-- core-socket-stream
|   |   |   |   |-- core-socket-stream-legacy
|   |   |   |   |   |-- net-in-stream-legacy
|   |   |   |   |   |-- net-in6-stream-legacy
|   |   |   |   |   |-- net-un-stream-legacy
|   |   |   |   |   |-- net-rc-stream-legacy, if available
|   |   |   |   |   |-- net-l2-stream-legacy, if available
|   |   |   |-- core-socket-stream-tls
|   |   |   |   |-- net-in-stream-tls
|   |   |   |   |-- net-in6-stream-tls
|   |   |   |   |-- net-un-stream-tls
|   |   |   |   |-- net-rc-stream-tls, if available
|   |   |   |   |-- net-l2-stream-tls, if available
|   |   |-- http
|   |       |-- libmagic, if available
```

```

|-- http-server
|   |-- http-server-express
|       |-- nlohmann-json support
|       |-- http-server-express-legacy-in
|           |-- net-in-stream-legacy
|       |-- http-server-express-legacy-in6
|           |-- net-in6-stream-legacy
|       |-- http-server-express-legacy-un
|           |-- net-un-stream-legacy
|       |-- http-server-express-legacy-rc, if available
|           |-- net-rc-stream-legacy
|       |-- http-server-express-tls-in
|           |-- net-in-stream-tls
|       |-- http-server-express-tls-in6
|           |-- net-in6-stream-tls
|       |-- http-server-express-tls-un
|           |-- net-un-stream-tls
|       |-- http-server-express-tls-rc, if available
|           |-- net-rc-stream-tls
|
|-- http-client
|
|-- net
|   |-- net-in
|       |-- net-in-phy
|           |-- net-in-phy-stream
|               |-- net-in-stream
|                   |-- net-in-stream-legacy
|                   |-- net-in-stream-tls
|
|   |-- net-in6
|       |-- net-in6-phy
|           |-- net-in6-phy-stream
|               |-- net-in6-stream
|                   |-- net-in6-stream-legacy
|                   |-- net-in6-stream-tls
|
|   |-- net-un
|       |-- net-un-phy-stream
|       |-- net-un-stream
|           |-- net-un-stream-legacy
|           |-- net-un-stream-tls
|       |-- net-un-dgram
|
|   |-- net-rc, if available
|       |-- net-rc-phy
|           |-- net-rc-phy-stream
|               |-- net-rc-stream
|                   |-- net-rc-stream-legacy
|                   |-- net-rc-stream-tls
|
|   |-- net-l2, if available
|       |-- net-l2-phy
|           |-- net-l2-phy-stream
|               |-- net-l2-stream
|                   |-- net-l2-stream-legacy
|                   |-- net-l2-stream-tls
|
|-- mqtt
|   |-- mqtt-server
|       |-- mqtt-server-websocket, shared path
|   |-- mqtt-client

```



```
-- mqtt-client-websocket, shared path
```

The graph view is useful because it makes several things visible at once.

First, the lower shared base path is small:

```
logger
  -> utils
    -> core
      -> core-socket
```

Second, the graph has shared public paths. For example, `websocket` links through `utils`, while `websocket-server` also belongs to the HTTP upgrade side. MQTT-over-WebSocket targets similarly connect MQTT roles with WebSocket roles.

Third, higher-level components do not all grow from one single branch. HTTP, MQTT, database support, network families, and concrete transport compositions attach to the lower framework surface in different ways.

That is the component architecture the build exposes.

Public header hierarchy mirrors the component hierarchy

The component graph is the build-side view. The include hierarchy is the source-side view. A `SNode.C` source file should normally include the highest public header that owns the abstraction it directly names; it should not assemble the whole lower stack by including implementation-support headers manually.

For example, a concrete IPv4 legacy socket server is selected in source by:

```
#include <net/in/stream/legacy/SocketServer.h>
```

The matching component is `net-in-stream-legacy`. At the Express level, an IPv4 legacy WebApp is selected by:

```
#include <express/legacy/in/WebApp.h>
```

The corresponding component-side stack is not textually identical, but it rhymes with the same architecture:

```
snodec::http-server-express-legacy-in
  -> snodec::http-server-express
    -> snodec::http-server
      -> snodec::net-in-stream-legacy
```

Headers expose declarations, aliases, templates, inline helpers, and source-facing public roles. Components expose compiled libraries, exported targets, usage requirements, and transitive link dependencies. They are two public contracts for the same stack, not one mechanism repeated twice.

Source-derived component/header matrix

The following matrix reflects the `SNode.Cv1.0.2` source snapshot used to prepare this edition of the book. It is intentionally selective. It is not a generated ABI manifest and not a complete list of every installed header. It lists the public header front an application would normally include when it directly names a role, and the component target or targets it would normally link when it needs the corresponding compiled surface.

In the source tree, examples and framework code include headers relative to the `SNode.C` source include root, for example `<express/legacy/in/WebApp.h>`. Installed consumers use the same public header shape below the installed `include/snode.c` prefix.

Area	Public header front	Link component(s)	What the selection means
Core runtime	<code><core/SNodeC.h></code>	<code>snodec::core</code>	Event-loop/runtime entry and framework initialization surface.
IPv4 legacy stream socket	<code><net/in/stream/legacy/SocketServer.h></code> / <code><net/in/stream/legacy/SocketClient.h></code>	<code>snodec::net-in-stream-legacy</code>	IPv4 stream sockets over the unencrypted legacy stream layer.
IPv4 TLS stream socket	<code><net/in/stream/tls/SocketServer.h></code> / <code><net/in/stream/tls/SocketClient.h></code>	<code>snodec::net-in-stream-tls</code>	IPv4 stream sockets over the TLS stream layer.
IPv6 legacy stream socket	<code><net/in6/stream/legacy/SocketServer.h></code> / <code><net/in6/stream/legacy/SocketClient.h></code>	<code>snodec::net-in6-stream-legacy</code>	IPv6 stream sockets over the unencrypted legacy stream layer.
Unix-domain legacy stream socket	<code><net/un/stream/legacy/SocketServer.h></code> / <code><net/un/stream/legacy/SocketClient.h></code>	<code>snodec::net-un-stream-legacy</code>	Local stream sockets over the legacy stream layer.
Unix-domain datagram socket	<code><net/un/dgram/Socket.h></code>	<code>snodec::net-un-dgram</code>	Local datagram socket surface.
HTTP server/client role	<code><web/http/legacy/in/Server.h></code> / <code><web/http/legacy/in/Client.h></code>	<code>snodec::http-server</code> or <code>snodec::http-client</code> , plus the selected carrier component when the source directly names one	Raw HTTP role over a concrete carrier. Unlike Express, raw HTTP does not use a separate concrete Express carrier target.
EventSource client	<code><web/http/legacy/in/EventSource.h></code>	<code>snodec::http-client</code> , plus the selected carrier component when needed	SSE client role over the selected HTTP client carrier. Server-side SSE is ordinary HTTP response streaming, not a separate component.
Express base layer	<code><express/WebApp.h></code> and other base Express headers	<code>snodec::http-server-express</code>	Express-like routing and middleware layer above the HTTP server surface.
Express IPv4 legacy WebApp	<code><express/legacy/in/WebApp.h></code>	<code>snodec::http-server-express-legacy-in</code>	Express-like server over IPv4 legacy stream. The concrete target owns both the carrier choice and the Express base layer.
Express IPv4 TLS WebApp WebSocket base	<code><express/tls/in/WebApp.h></code> <code><web/websocket/SubProtocolFactory.h></code> and related base WebSocket headers	<code>snodec::http-server-express-tls-in</code> <code>snodec::websocket</code>	Express-like server over IPv4 TLS stream. Shared WebSocket/subprotocol machinery. This alone is not a concrete server or client role.
WebSocket server/client role	<code><web/websocket/server/SubProtocol.h></code> / <code><web/websocket/client/SubProtocol.h></code>	<code>snodec::websocket-server</code> or <code>snodec::websocket-client</code>	Role-specific WebSocket upgrade/subprotocol surface.

Area	Public header front	Link component(s)	What the selection means
MQTT shared layer	<code><iot/mqtt/Mqtt.h> / <iot/mqtt/Topic.h></code>	<code>snodec::mqtt</code>	Shared MQTT packet/session/protocol machinery. This component is available only when the MQTT dependency gate is satisfied.
MQTT native client	<code><iot/mqtt/client/Mqtt.h></code>	<code>snodec::mqtt-client</code>	Native MQTT client role.
MQTT native server	<code><iot/mqtt/server/Mqtt.h></code>	<code>snodec::mqtt-server</code>	Native MQTT server role.
MQTT-over-WebSocket client	<code><iot/mqtt/client/SubProtocol.h></code> plus the MQTT client role header directly named by the application	<code>snodec::mqtt-client-websocket</code>	MQTT client role carried through a WebSocket subprotocol.
MQTT-over-WebSocket server	<code><iot/mqtt/server/SubProtocol.h></code> plus the MQTT server role header directly named by the application	<code>snodec::mqtt-server-websocket</code>	MQTT server role carried through a WebSocket subprotocol.
MariaDB application state	<code><database/mariadb/MariaDBClient.h></code>	<code>snodec::db-mariadb</code>	MariaDB persistence boundary for application state. This component is available only when the MariaDB client library is found.

The table should be read with two cautions.

First, headers and components are related public surfaces, not the same mechanism. A public header can expose inline/template surfaces, aliases, or a role front; a component target exposes compiled code, usage requirements, transitive link dependencies, and packaging identity.

Second, a consumer should not include lower-layer headers merely because a selected component depends on them. Include the public header for the abstraction the source file directly names, and link the component that owns the binary surface selected by the application. The component target should carry its declared lower dependencies.

Namespaced targets are the consumer-facing interface

Most library targets also receive namespaced aliases such as:

```
add_library(snodec::core ALIAS core)
add_library(snodec::net-in-stream-legacy ALIAS net-in-stream-legacy)
add_library(snodec::http-server-express ALIAS http-server-express)
```

The namespace is not cosmetic. It tells an external application that the target is an exported SNode.C component, not a local helper target from the application's own build tree.

An external consumer can write:

```
target_link_libraries(myapp
PRIVATE
    snodec::http-server-express
    snodec::net-in-stream-legacy
)
```

and read the application face directly:

```
Express-like HTTP application layer
+ IPv4 legacy stream carrier
```

The matching source file should show the same idea through public headers, for example an Express public header for the WebApp surface and a lower socket public header only if the source directly names that lower socket role.

That is clearer than linking every application against one vague monolithic target.

PUBLIC, PRIVATE, and INTERFACE

CMake's visibility keywords are architectural words in a framework. They decide which dependencies become part of a component's public surface and which remain implementation details.

A dependency linked as **PUBLIC** becomes part of what consumers of the target also need. A dependency linked as **PRIVATE** remains internal to the target. An **INTERFACE** usage requirement shapes consumers without necessarily being a compiled object in the same way.

This is not CMake trivia. It is dependency hygiene. A framework that gets this wrong can make downstream applications difficult to build, difficult to package, or accidentally dependent on internals.

Component-owned dependencies

The central rule is simple:

Dependency and include ownership rule

The target that needs a dependency should declare it. The source file that names an abstraction should include the public header that owns that abstraction.

That is why a consumer-facing link line does not manually repeat the whole lower dependency chain. A direct link line should describe the application face:

```
application
-> protocol/application component
-> selected transport component
-> optional feature components directly used by the application
```

The selected component targets then propagate their own public dependencies. This keeps the public link line small without hiding the architecture.

Package-boundary checklist

- Does the component declare its own dependencies?
- Does the installed target expose the right usage requirements?
- Can an external consumer link it?
- Does the package export match the architecture?

It also prevents two opposite mistakes:

too little:

```
link only the high-level protocol component
and forget the selected carrier
```

too much:

```
link every lower layer manually
even though selected components already own those dependencies
or include every lower header manually
even though a public front-door header owns the selected C++ abstraction
```

The correct model is:

```
choose the direct building blocks
-> include the public headers for the abstractions named in source
-> let the component targets carry their declared lower dependencies
```

A concrete Express carrier target shows this well. `http-server-express-legacy-in` owns both sides of its composition: the selected IPv4 legacy carrier and the base Express component.

32.4 Core, network, and transport composition

The lower build layers show how SNode.C separates runtime machinery, socket abstractions, network families, stream behavior, and connection variants.

That separation is one of the reasons the component model works.

Core and multiplexer choice

One of the most interesting build decisions appears in the core module. The build offers an I/O multiplexer selection among:

- `epoll`,
- `poll`,
- `select`.

If no explicit choice is given, the build chooses the first entry from that list. This is a lower runtime concern made explicit at build time.

The event loop and the multiplexer are runtime concepts, but the default implementation choice is also a build surface. Ordinary application code should not normally be written around that choice. An HTTP server, MQTT client, or custom stream protocol context should not need to care whether the runtime is built with `epoll`, `poll`, or `select`.

The normal build selects the default low-level waiting backend. The multiplexer implementations are also built as separate shared libraries. Because of that, process-local override techniques such as `LD_PRELOAD` can be useful in deployment experiments, diagnostics, or platform-specific operation. That does not change the application-facing event-driven model.

The two levels should be kept distinct:

```
build/default choice
-> selected by CMake

process-local override technique
-> useful for diagnostics or deployment experiments
```

Both mechanisms affect the low-level waiting backend, not the application-facing event-driven model.

Core socket stream layers

The core socket build descends through clear stages:

- `core`,
- `core-socket`,
- `core-socket-stream`,
- `core-socket-stream-legacy`,
- `core-socket-stream-tls`.

This progression mirrors the architecture. First there is runtime and core infrastructure. Then there are socket abstractions. Then there is stream-oriented socket machinery. Then there are connection-layer variants: legacy and TLS.

The important current dependency shape is:

```
core-socket-stream
-> core-socket
```

That distinction matters. The core stream machinery belongs to the core/socket side. The network-family side is selected separately through targets such as `net-in-stream-legacy`.

Network-family targets

The network-family targets combine family identity with stream and connection behavior.

For example, an IPv4 legacy stream component is built as a composition of:

```
net-in-stream
core-socket-stream-legacy
```

The IPv4 TLS stream component follows the same shape, but with the TLS core stream variant:

```
net-in-stream
core-socket-stream-tls
```

This is the build-system equivalent of the architectural layering used throughout the framework. The family-specific side provides IPv4, IPv6, Unix-domain sockets, Bluetooth RFCOMM, Bluetooth L2CAP, or another lower communication family. The core stream side provides generic stream operation and the legacy or TLS connection mode.

The combined target becomes the usable carrier component.

Legacy and TLS variants

The separation between legacy and TLS stream targets is central to the build model. TLS is not hidden behind one global Boolean that silently changes the meaning of every target.

Instead, TLS variants have their own targets. A consumer can choose:

- unencrypted legacy stream components,
- TLS stream components,
- or both.

This matches the architectural model:

```
application protocol
-> remains mostly stable

connection layer
-> selected as legacy or TLS
```

TLS is a connection-layer specialization, not a rewrite of the application model. The build system reinforces that directly.

32.5 Protocol and application-layer components

Above core and transport composition, SNode.C builds protocol and application-layer targets. The rule remains the same: read each target as the layer, role, and composition it owns; include the public header for the abstraction the source file directly names; link the component that owns the corresponding binary surface.

HTTP and upgrade layout

Most SNode.C components follow ordinary include/link/deploy rules: include the public header for the abstraction the source names, link the component that owns the binary surface, and install the resulting shared libraries where the platform loader can find them. The book does not repeat those normal library-deployment rules for every component.

HTTP upgrade support is different because an upgrade protocol can be selected later by name. The build therefore records HTTP and upgrade-library directories as target properties. Chapter 21 gives the HTTP-upgrade name-to-factory contract, and Chapter 24 applies the same pattern to WebSocket subprotocols. This chapter only identifies the build-side boundary; Chapter 33 follows that boundary into the installed runtime system.

RPATH and runtime composition

HTTP, Express, and WebSocket build files set library output directories and install RPATH-related properties. In this chapter, those settings matter as build-side evidence that runtime composition exists; they are not a complete deployment policy.

The deployment question is answered later: the installed system must contain the ordinary shared libraries and the runtime-selected modules in locations the loader and SNode.C can actually find. Chapter 33 treats that installed filesystem shape explicitly.

Express base and concrete carrier targets

The Express targets are a good example of separating base application-layer behavior from concrete carrier selection.

The base target is:

```
http-server-express
```

It represents the Express-like application layer above the HTTP server layer. Its direct dependencies are:

```
http-server
JSON support
```

A concrete carrier target, for example:

```
http-server-express-legacy-in
```

has a different role. It represents the Express-like server over a concrete carrier shape.

The source-side counterpart is the concrete Express public header, for example `<express/legacy/in/WebApp.h>` or `<express/legacy/in/Server.h>`. That header is the C++ entry into the Express/HTTP/carrier stack. The graph below shows the corresponding build-side ownership.

The intended dependency model is:

```
http-server-express-legacy-in
-> net-in-stream-legacy
-> http-server-express
    -> http-server
```

The concrete target selects the IPv4 legacy stream carrier and the base Express component. The same pattern applies to the other concrete Express family targets:

```
http-server-express-legacy-in6
http-server-express-legacy-un
http-server-express-tls-in
http-server-express-tls-in6
http-server-express-tls-un
```

and to optional Bluetooth RFCOMM targets where available.

This is the same dependency-hygiene rule in another form:

```
concrete Express carrier target
-> selected carrier component
-> base Express component
    -> HTTP server dependency
```

The lower HTTP server layer is reached through the base Express component. The concrete target stays responsible for the concrete carrier choice. That keeps the direct dependency face meaningful.

WebSocket upgrade components

WebSocket belongs to the HTTP upgrade family. The WebSocket build obtains HTTP upgrade directories from the HTTP target and places WebSocket-related artifacts beneath that layout. This mirrors the protocol model: WebSocket is an HTTP upgrade, not a separate protocol island. Chapter 24 gives the compact deployment contract for WebSocket subprotocol modules beneath the WebSocket upgrade directory.

MQTT native and WebSocket-carried components

Native MQTT and MQTT-over-WebSocket are separate component selections. The base `mqtt` target supplies shared MQTT support, while `mqtt-client` and `mqtt-server` express native endpoint roles. `mqtt-client-websocket` and `mqtt-server-websocket` express the WebSocket-carried composition.

The distinction is intentional:

```
mqtt-client
mqtt-server
mqtt-client-websocket
mqtt-server-websocket
```

A consumer can therefore select the MQTT role and carrier composition deliberately.

32.6 Optional features and generated configuration

Not every feature is always available. Some depend on external libraries. Some affect compile definitions. Some determine whether a component is built at all.

SNode.C keeps these boundaries visible in the build.

Optional dependencies

Different external dependencies have different meanings:

```
optional enhancement
-> improves a component when available

required dependency for a component
-> the component needs it to build

availability gate for a component family
-> the component family is absent when the dependency is absent
```

For example:

- HTTP can use `libmagic` for better MIME detection.
- Express requires `nlohmann_json`.
- The current MQTT component is gated by `nlohmann_json >= 3.11`.
- MariaDB support requires the MariaDB client library.

- Bluetooth-related components depend on platform Bluetooth support.

That distinction is important for packagers and application developers. Optional functionality should be visible in the build, not hidden as mysterious runtime absence.

Build-time defaults as compiled policy

The build contains a helper that appends compile definitions to specific source files. It is used to inject default values such as:

- READ_BLOCKSIZE,
- WRITE_BLOCKSIZE,
- READ_TIMEOUT,
- WRITE_TIMEOUT,
- TERMINATE_TIMEOUT,
- ACCEPTS_PER_TICK,
- BACKLOG,
- RETRY,
- RETRY_ON_FATAL,
- RETRY_TIMEOUT,
- RETRY_TRIES,
- RETRY_BASE,
- RETRY_JITTER,
- RETRY_LIMIT,
- ACCEPT_TIMEOUT,
- RECONNECT,
- RECONNECT_TIME,
- CONNECT_TIMEOUT,
- TLS_INIT_TIMEOUT.

This is a subtle but important build feature. Some framework defaults can be shaped at CMake time and compiled into the relevant implementation files instead of being fixed in ordinary source text.

For a systems framework, that can be valuable. It gives builders and packagers another controlled way to shape default behavior.

Build-time defaults and runtime configuration

Build-time defaults and runtime configuration should not be confused.

A build-time default defines the library's compiled baseline. Runtime configuration describes what a particular configured instance chooses when the application starts.

Both are useful, but they operate at different times:

```
CMake-time default
-> compiled into the library or component

runtime configuration
-> selected by application options, configuration files, or instance settings
```

Keeping those two levels separate avoids confusion.

A distributor may want different compiled defaults for an embedded package. An operator may still want runtime configuration for a particular deployment. Those are not the same decision.

32.7 Exported package targets and external consumers

SNode.C is also consumed by external applications. That is where package configuration, exported targets, component selection, and namespaced target names become essential. This is still a build-interface topic: it describes how an installed SNode.C package is consumed by another CMake project, not how a running service is deployed.

From build targets to exported package targets

The internal build tree produces targets. The install/export machinery turns selected targets into an external CMake package interface.

The shape is:

```
internal target
-> install/export rule
    -> exported snodec:: target
        -> find_package(snodec COMPONENTS ...)
```

This is the bridge between framework build architecture and consumer build architecture.

A component that is not exported properly cannot be selected cleanly by an external application. A component that exports too much can force consumers to see internal details. A component that exports too little can make consumers link missing dependencies manually.

The package interface is therefore part of the framework design.

The installed package configuration verifies requested components, recursively loads component dependencies, and includes the exported target files. That keeps the consumer-facing build line short while preserving the component model underneath.

The installed public headers do the corresponding source-side job. A consumer includes the public front-door header for the abstraction it names; the header hierarchy exposes the required lower declarations through the public include surface. The consumer should not depend on an accidental transitive include from an unrelated header.

External project example

A minimal external application that uses the Express-like HTTP layer over an IPv4 legacy stream could look like this:

```
cmake_minimum_required(VERSION 3.18)

project(my-ipv4-legacy-webapp LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)

find_package(snodec REQUIRED
  COMPONENTS
    http-server-express
    net-in-stream-legacy
)

add_executable(my-ipv4-legacy-webapp
  main.cpp
)

target_link_libraries(my-ipv4-legacy-webapp
  PRIVATE
    snodec::http-server-express
    snodec::net-in-stream-legacy
)
```

A matching `main.cpp` would include the public header for the source abstraction it directly uses. For example, a file that directly uses the Express WebApp surface would include:

```
#include <express/legacy/in/WebApp.h>
```

This is a minimal consumer shape, not the only possible application structure. The important part is the application face:

```
http-server-express
  -> Express-like HTTP application layer

net-in-stream-legacy
  -> selected IPv4 legacy stream carrier
```

The application does not list `core`, `core-socket`, `http`, `http-server`, `utils`, or `logger` manually. Those lower dependencies belong to the selected SNode.C component targets. Likewise, the source file does not include every lower HTTP and socket header manually when a public Express front-door header owns the abstraction it names.

Component selection is not “select everything”

The list of supported components should not be treated as a menu from which every application selects everything.

A consumer-facing link line selects the components that describe the application face. A source file selects public headers that describe the abstractions it names:

```
protocol/application component
selected transport component
optional feature components directly used by the application
```

For example, a web application over IPv4 legacy stream does not need to link every HTTP, WebSocket, MQTT, database, IPv6, Unix-domain, Bluetooth, TLS, and utility component.

That would make the build line noisy and misleading. The same is true of source files that include every lower header behind a selected public abstraction. The direct link line and the include block should both remain short architectural statements.

In-tree names and external names

The same architectural selection can appear with different target names depending on build context.

Inside the SNode.C source tree:

```
http-server-express
net-in-stream-legacy
```

Outside the source tree:

```
snodec::http-server-express
snodec::net-in-stream-legacy
```

The `snodec::...` prefix is the installed/exported namespace. It marks the target as a framework component provided by the package.

Therefore, examples for external applications should use namespaced targets.

Base components and concrete composition components

Not every target sits at the same abstraction level.

Base components name reusable protocol or application layers, for example:

```
http
http-server
http-server-express
mqtt
websocket
```

Concrete composition components bind those layers to a family, role, or carrier:

```
net-in-stream-legacy
net-in-stream-tls
http-server-express-legacy-in
http-server-express-tls-in
mqtt-server-websocket
mqtt-client-websocket
```

Lower operational components provide reusable machinery below both, while application targets assemble selected pieces into executables. Reading target names with this distinction in mind prevents many linking misunderstandings.

32.8 Reading a SNode.C CMake target

A SNode.C CMake target can be read systematically. A practical recipe is:

1. Identify the target name.
2. Decide whether it is a framework component, concrete composition component, lower operational component, or application target.
3. Identify the public headers used by the source files and the SNode.C abstractions directly named there.
4. Read `target_link_libraries`.
5. Separate `PUBLIC`, `PRIVATE`, and `INTERFACE` dependencies before opening C++ source files.
6. Check whether the target has an installed/exported alias.
7. Check whether it is listed as a supported component.
8. Look for optional dependency gates around the target.
9. Check header install destinations, library install destinations, and `RPATH`-related properties.
10. For external use, translate local target names to `snodec::...`
11. Only then inspect implementation files if the public header and build shape are not enough.

This method follows the same pattern used in Chapter 29. The build target often reveals the architecture before the implementation file is opened.

What to remember

- CMake is an architectural surface in SNode.C, not only a build-script language.
- The top-level build creates the project shell; `src/CMakeLists.txt` exposes the framework surface.
- Compiler, warning, and linker policies are part of the maintenance strategy.
- `SNODEC_INTREE_BUILD` separates the in-tree build context from the installed consumer view.
- Component targets should own their dependencies, and public headers should own their source-facing abstraction boundary.
- `PUBLIC`, `PRIVATE`, and `INTERFACE` describe dependency visibility.
- Include paths and component names are parallel public contracts: one for C++ source, one for linking and installation.

32.9 Closing perspective

The build system is now visible as architecture: components, public include surfaces, exported targets, optional dependencies, and installable boundaries. The remaining chapters can discuss deployment, testing, and extension without treating the build as a black box.

Chapter 33

Deployment on Linux and OpenWrt

33.1 Why deployment belongs after CMake

Chapter 32 showed how SNode.C expresses architecture through CMake targets, install components, exported package targets, dependency visibility, and the build-side boundaries of runtime-selected extensions. Chapter 33 follows those decisions into the installed system: packages, filesystem layout, service ownership, configuration, runtime state, deployment-specific resources, and constrained Linux targets.

Chapter 32:
component and linking strategy

Chapter 33:
installed runtime environment

Chapter 34:
testing, debugging, and benchmarking

The subject is deployment, but the real topic is still architectural visibility. Deployment is where SNode.C's build choices enter the filesystem. Shared libraries, component packages, executable applications, exported CMake package files, runtime-loaded protocol modules, configuration directories, log directories, pid directories, service definitions, and package-manager metadata are not unrelated operational leftovers. They are the installed form of the architecture.

The distinction is simple:

linking:
the binary can be produced

deployment:
the complete runtime environment exists

That distinction matters for SNode.C because the framework is used to build long-running communication roles: HTTP applications, MQTT tools, WebSocket upgrade handlers, database-connected services, IoT-facing components, and full systems such as MQTTSuite. Once those applications leave the build tree, the important questions are no longer only compiler and linker questions:

Where are the libraries installed?
Where are runtime-loaded modules installed?
Which packages are required?
Which user or group owns runtime state?
Where do configuration, logs, and pid files live?
How is the service supervised?
Which TLS, database, or web-asset resources belong to the deployment?
What changes on embedded Linux or OpenWrt?

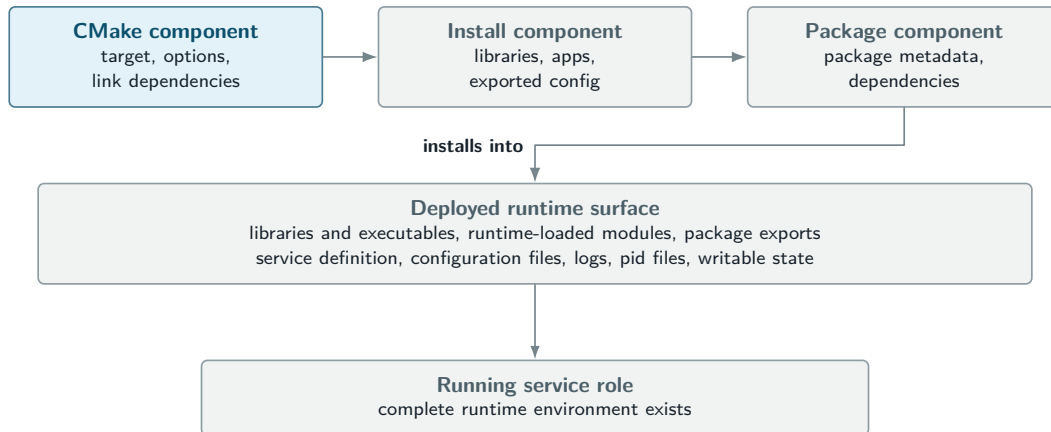


Figure 33.1: The build-to-deployment path in which component choices become installed packages, service definition, configuration, and runtime state.

These questions are not afterthoughts. They decide whether the built architecture can actually run.

33.2 Deployment as installed architecture

Architecture entering the filesystem

Deployment is architecture entering the filesystem. When SNode.C is installed, the framework’s internal boundaries become visible as libraries, executables, exported package files, runtime-loaded modules, configuration directories, service definitions, package metadata, and managed runtime state.

A small experimental program may run from the build tree with a manually adjusted environment. A serious deployment needs a reproducible filesystem shape. Figure 33.1 shows the path from a build component to an installed runtime surface.

Install-tree warning

Do not confuse build-tree success with install-tree success. A running role also needs its installed libraries, modules, configuration, service shape, and writable runtime state.

Figure 33.1 is intentionally not only a build pipeline. The important point is that each step preserves architectural information. A CMake target expresses a component boundary. An install component gives that boundary a filesystem location. A package component adds dependency metadata. The deployed runtime surface then has to contain everything the running role needs: libraries, executables, runtime-loaded modules, configuration, service definition, writable state, and deployment-specific resources.

The installed system should still reveal what was built. If the build selected HTTP, Express, MQTT, WebSocket, TLS, MariaDB support, or a particular network family, deployment should not hide those choices behind one opaque pile of files.

General-purpose Linux as the first deployment target

The first deployment target is general-purpose Linux: development workstations, servers, virtual machines, single-board computers running Debian-like distributions, and Linux-based lab systems. This environment is easiest to understand because it is close to the development system:

```

CMake build
-> CMake install
    -> CPack component package
        -> package manager installation
            -> filesystem layout
                -> service definition
                    -> runtime configuration and state
  
```

Even there, SNode.C has a richer deployment story than a single executable. The framework installs component libraries, exported package configuration, runtime-sensitive protocol-extension layouts, and applications that may need logs, pid files, TLS material, database access, web assets, and service supervision.

Component meaning should survive installation

Chapter 32 treated SNode.C components as architectural surfaces. Deployment should preserve that meaning. A Linux package set does not have to collapse everything into one enormous install unit.

For example, a small IPv4 stream tool does not need HTTP, WebSocket, MQTT, and MariaDB libraries. A web administration application may need HTTP and Express but not MQTT. An MQTT bridge may need MQTT client support and selected transport components. A database-backed store may need MariaDB support while a pure broker may not.

This is where component packaging becomes practical. A deployment is easier to reason about when installed packages still reflect the same component boundaries that existed in the build.

For runtime packages, this mostly means the right libraries, configuration, service files, and runtime-loadable modules. For development packages, it also means that public headers and exported CMake targets preserve the same component boundaries for external consumers.

Deployment as runtime shape

Copying one executable is rarely the whole deployment. A SNode.C application may need:

- shared libraries,
- runtime-loaded upgrade or subprotocol modules,
- external library dependencies,
- configuration files,
- a writable log directory,
- a writable pid directory,
- TLS certificates and private keys,
- database connectivity,
- application-specific files such as web assets,
- and a service definition.

The binary is only one part of the deployed system. Deployment proves that all the other pieces are present, placed correctly, owned correctly, and visible to the runtime.

33.3 Packages and installed component surfaces

CPack as the ordinary Linux packaging bridge

SNode.C's top-level build enters `src` and then includes the project packaging configuration. Packaging is therefore part of the project build story, not an unrelated external script.

The packaging configuration enables Debian package generation, shared-library dependency generation, shared-library metadata generation, component dependency handling, component installation, and one package per component group. That matters because it lets the component model survive into deployable packages.

A useful deployment path is:

```
SNode.C component target
-> install component
    -> CPack component package
        -> package dependency metadata
            -> target installation
```

The component model is therefore not only useful while compiling from source. It can become visible to the target system's package manager.

What CPack solves, and what it does not solve

CPack can package installed components and express package dependencies. It can help generate binary packages from install rules and component metadata.

It does not create a complete deployment policy. It does not decide service supervision, certificate material, database schema, site-specific configuration, OpenWrt feed layout, repository signing policy, or operational update policy.

CPack provides the packaging bridge. It does not replace system design.

Package dependencies follow component dependencies

Package dependencies should follow component dependencies, and development packages should install the public headers that correspond to the components they expose. That is exactly what the SNode.C packaging configuration expresses.

Examples include:

```
utils
-> logger

core
-> mux- $\{IO\_Multiplexer\}$ 
-> utils

websocket-server
-> websocket
-> http-server

websocket-client
-> websocket
-> http-client

net-in-stream-tls
-> net-in-stream
-> core-socket-stream-tls
```

This mirrors the component graph from Chapter 32. A package manager should not have to rediscover the architecture from filenames. The package metadata should already carry the dependency story.

Multiplexer packages and runtime overrides

Chapter 32 introduced the multiplexer choice. Deployment makes that choice visible.

SNode.C has package components for the multiplexer libraries:

```
mux-epoll
mux-poll
mux-select
```

The `core` package depends on the selected default multiplexer component:

```
core
-> mux- $\{IO\_Multiplexer\}$ 
-> utils
```

That is the ordinary deployment path. A deployment may still install additional multiplexer libraries deliberately if process-local override techniques such as `LD_PRELOAD` are part of the operational or diagnostic strategy. That gives two deployment modes:

```
normal package dependency
-> selected default multiplexer installed with core
```



```
explicit operational override
-> additional multiplexer library available for process-local override
```

both mechanisms remain deployment decisions. They do not change the application-facing event-driven model.

Installed paths and runtime-loaded modules

Some libraries are ordinary link-time dependencies. Others participate in runtime protocol composition. HTTP upgrade support, WebSocket upgrade handling, WebSocket subprotocol support, and MQTT-over-WebSocket composition make installation paths operationally important.

For ordinary linked libraries, the platform loader must find the dependency. For upgrade and subprotocol composition, SNode.C must also find the libraries that participate in runtime protocol composition. A dynamically loaded or path-sensitive module must therefore be installed where the runtime expects it, or where configuration tells the runtime to find it.

Chapter 32 identified the build-side properties that make such runtime composition visible. In deployment, those choices stop being abstract build details. They become the difference between:

```
application starts
-> runtime protocol extension is found
    -> WebSocket or MQTT-over-WebSocket works
```

and:

```
application starts
-> runtime protocol extension is missing
    -> upgrade path fails
```

A missing WebSocket upgrade or subprotocol module may look like an application failure, but it can be a deployment-path failure.

RPATH as deployment policy

RPATH is often treated as a linker detail. For SNode.C it can also be part of deployment policy.

In a build tree, library locations are usually convenient and known. In an installed system, the runtime loader must still find the libraries. For ordinary system libraries, the platform linker configuration may be enough. For nested SNode.C protocol-extension layouts, install RPATH and well-defined library directories can be part of the runtime contract.

A deployment should answer:

- will the platform loader find ordinary shared libraries?
- will SNode.C find runtime-loaded upgrade or subprotocol modules?
- does the installed path match what the build encoded?
- does cross-compilation staging avoid leaking build-host paths into target binaries?

RPATH can preserve intended lookup paths for installed components, but deployment still has to ensure that the files, permissions, and package dependencies are actually present on the target. The last question becomes especially important for OpenWrt and other cross-compiled environments.

33.4 Runtime state and service operation

Configuration directories

SNode.C's configuration model creates real deployment expectations. When run with effective root privileges, the current configuration startup code uses system-level directories:

```
/etc/snode.c
/var/log/snode.c
/var/run/snode.c
```

In non-root mode, the current code chooses a user base from `XDG_CONFIG_HOME`, `HOME`, or the `passwd` home directory and constructs SNode.C-specific configuration, log, and pid paths below that base:

```
<user-base>/etc/snode.c
<user-base>/var/log/snode.c
<user-base>/var/run/snode.c
```

This distinction is useful. The same application can run in development as a normal user and in deployment as a system service, without pretending that those are the same mode. The directories are part of the operational shell of the application.

The snodec group and ownership model

When root-mode directory creation is needed, SNode.C expects a management group. The default group name is compiled from `GROUP_NAME`, whose current build default is `snodec`. During root-mode directory creation, the code looks up that group and uses it for group ownership. If the group is missing, directory setup is treated as a configuration/runtime setup error.

A deployment package or installation procedure therefore has to prepare the runtime ownership model. That usually means:

- create the configured group if it does not exist,
- set directory ownership and permissions correctly,
- add service users where appropriate,
- avoid world-writable runtime directories,
- and make sure the application can create or read its configuration, log, and pid files.

This supports operational reliability and security hygiene. A service that cannot create its pid file or log file because deployment forgot directory ownership will fail in a way that looks unrelated to HTTP, MQTT, WebSocket, or database code.

Daemonization and service managers

SNode.C applications expose daemonization-related options through the configuration shell. The root configuration includes options such as:

```
--daemonize
--user-name
--group-name
--kill
--log-file
--enforce-log-file
```

An application can run in foreground mode or daemon mode, write pid files, and switch user or group privileges. That is useful, but it does not remove the need for a host service manager.

On general-purpose Linux, a long-running service is often better supervised by `systemd` or the local service manager. That service manager can own restart policy, logging integration, dependencies, user identity, and lifecycle. On OpenWrt, a continuously running packaged SNode.C application should normally be integrated with the platform service supervision model, commonly `procd`.

The deployment question is therefore not only:

```
Can the program daemonize itself?
```

It is also:

Which operating-system service model should supervise this role?

SNode.C gives applications an operational shell. Deployment integrates that shell with the host service model.

Foreground mode during development

During development and debugging, foreground execution is usually the clearest mode. It keeps process output, configuration experiments, and failure behavior visible before the role is hidden behind a service manager.

A typical development rhythm is:

```
run in the foreground
-> inspect help and effective configuration
    -> increase verbose level if needed
        -> write a configuration file
            -> move the role into a managed service
```

Deployment begins with understanding the effective configuration, not with hiding the process in the background.

Generated configuration as deployment artifact

SNode.C's ability to show and write configuration is especially important in deployment. A generated configuration file is a reproducible description of the deployed role shape.

It can record:

- which configured communication roles exist,
- which registered runtime-visible instances are enabled,
- which local and remote addresses are used,
- which TLS settings are active,
- which retry and timeout policies matter,
- which application-level options shape logging or daemon behavior,
- and which runtime-state directories are used.

This is particularly valuable for multi-role systems. A SNode.C configuration file can become a readable map of the deployed role constellation. That is much clearer than a shell script full of opaque command-line fragments.

Logs and pid files as managed state

Logs and pid files are runtime state. They are not source files, and they are not static configuration. A deployment answers:

- where are logs written?
- who can read them?
- are they rotated by the operating system?
- where is the pid file written?
- does the service user have permission to write it?
- does the init system rely on it, or is it only a SNode.C convenience?

A pid file and a log file are not architectural features, but unmanaged pid and log locations can break an otherwise correct deployment. A network framework can be technically correct and still fail operationally if runtime state is unmanaged.

33.5 Deployment-specific resources

TLS certificate material

TLS deployment is not finished when an application links against TLS-capable components. A TLS deployment also installs and protects certificate material:

- certificate chains,
- private keys,
- CA files or CA directories,
- renewal policy,
- file ownership,
- file permissions,
- and SNI-related configuration where needed.

A TLS-capable binary is not yet a secure deployment. The certificate and trust material must be deployed correctly too.

Database dependencies and persistent state

Chapter 28 treated persistence as an application-state boundary. Deployment turns that boundary into library dependencies, credentials, schema assumptions, database reachability, and failure policy.

An application may need the MariaDB client library on the target. It may also need access to a database server, socket path or network endpoint, database name, user, password source, schema, and permissions. Database deployment is therefore both:

```
package dependency deployment
+ persistent-state deployment
```

A good deployment makes the database contract visible:

- which package provides the client library,
- which configuration values identify the database endpoint,
- which schema is expected,
- which credentials are required,
- and what happens when the database is unavailable.

This becomes especially important for MQTTStore, IoT history, monitoring, or other persistence-facing applications.

Web assets and application-specific files

Some applications deploy more than binaries and shared libraries. For example, the MQTTSuite broker build installs browser assets for its web interface below a MQTTSuite web directory.

That is a concrete reminder that application deployment may include:

- executables,
- SNode.C libraries,
- web assets,
- configuration files,
- service definitions,
- and runtime directories.

Those files belong to different parts of the filesystem and have different update and ownership expectations. A deployment that treats all of them as one anonymous copy step loses useful structure.

33.6 Embedded Linux and OpenWrt

Embedded Linux changes the deployment priorities

General-purpose Linux teaches the basic model. Embedded Linux does not change the architecture, but it changes the cost of every dependency.

On smaller targets, deployment becomes more sensitive to:

- storage size,
- memory use,
- library count,
- package granularity,
- startup time,
- log volume,
- writable flash use,
- cross-compilation correctness,
- and external dependency selection.

Therefore, SNode.C's componentized design matters. A constrained target does not need to carry protocol families, database support, example applications, or dynamic modules it never uses. On embedded systems, linking and packaging discipline become resource discipline.

OpenWrt is Linux under embedded constraints

OpenWrt is Linux, but it is not ordinary server deployment. It targets routers and embedded network devices, and it commonly appears with BusyBox, musl, `procd`, overlay filesystems, cross-compilation workflows, and package-manager-based system assembly rather than a full desktop/server distribution model.

A compact mental model is:

```
OpenWrt deployment
-> SNode.C Linux deployment
    -> under embedded, cross-compiled, package-managed constraints
```

That is the heart of the OpenWrt part of the chapter.

The OpenWrt SDK as packaging environment

For application packages, the OpenWrt SDK is usually the right starting point. A binary copied from a desktop build tree is usually the wrong deployment artifact for OpenWrt.

The target may differ in:

- CPU architecture,
- C library,
- ABI,
- compiler version,
- linker behavior,
- available libraries,
- filesystem layout,
- package manager,
- and default security expectations.

The SDK provides a target-specific cross-compilation environment. That is the right environment for C++ applications with shared libraries, optional dependencies, and package metadata.

OpenWrt deployment flow

A compact OpenWrt deployment flow looks like this:

```
OpenWrt SDK
-> package recipe
    -> cross-compiled package
        -> feed or image build
            -> package manager installation
                -> procd service definition
                    -> SNode.C configuration and runtime state
```

The exact commands depend on the OpenWrt release and build setup. The architectural flow remains stable.

Package recipes should preserve component meaning

An OpenWrt package recipe should not flatten SNode.C blindly. It should preserve the smallest useful deployable surface.

A feed may choose to split packages along component lines such as:

- core runtime libraries,
- selected multiplexer libraries,
- network-family components,
- TLS stream support,
- HTTP support,
- Express support,
- WebSocket support,
- MQTT support,
- MariaDB support,
- selected applications,
- selected MQTTSuite executables.

The exact split is a packaging policy decision, not a statement that the SNode.C repository already ships this particular OpenWrt feed layout. The principle is stable:

```
small target
-> small package surface
    -> explicit dependencies
        -> reproducible image
```

This is how an embedded target stays understandable.

OpenWrt dependencies must be explicit

OpenWrt packages declare dependencies explicitly. A package that needs TLS depends on the selected TLS library packages. An MQTT or HTTP component that needs JSON support depends on the JSON package if it is dynamically provided, or ensures it is built appropriately. A MariaDB-backed tool depends on the MariaDB client library. A Bluetooth component depends on Bluetooth stack support where applicable.

This is image reproducibility and build correctness. An OpenWrt image is assembled from many small packages. Hidden dependencies make the final image fragile and difficult to reproduce.

Package-manager format is release-specific

OpenWrt package-manager tooling and package formats are release-specific. The durable lesson is not a filename extension.

A package recipe should express the software correctly and let the OpenWrt build system produce the package format used by the selected release. That keeps the deployment lesson stable even when the package-manager workflow changes.

Feeds are distribution boundaries

An OpenWrt feed is a distribution boundary expressed through package recipes.

A feed tells downstream systems:

- which packages exist,
- which versions are available,
- which dependencies they require,
- which target architectures are supported,
- and which packages can be installed or built into an image.

A compact view is:

```
feed
-> package recipes
    -> package dependencies
        -> target architecture builds
            -> repository metadata
```

A clear feed layout makes the system easier to build, install, update, and reproduce.

Repository signing and update trust

Once packages are distributed through an OpenWrt package repository, signing becomes part of deployment trust. The target device must be able to trust repository metadata and package source according to the package manager used by that OpenWrt release.

This is especially important for routers and network infrastructure devices. A technically correct package is still not a complete deployment story if the repository cannot be trusted or updated cleanly.

procd service integration

On OpenWrt, `procd` is the normal service supervision layer. A continuously running SNode.C application packaged for OpenWrt should therefore be described as a `procd`-managed service.

That service definition is where deployment expresses:

- how the process starts,
- which configuration file is used,
- which user or group is used,
- whether the service restarts,
- and which dependencies must be available before startup.

The application may still support daemonization, but on OpenWrt the deployed role should fit the platform service model.

33.7 Reading a deployment

Practical deployment checklist

A SNode.C deployment can be read with a checklist. It is not a command sequence; it is a way to keep deployment architectural.

1. Which executable or service role is being deployed?
2. Which SNode.C components does it require?
3. Which package components provide those libraries?
4. Which runtime-loaded modules must be installed?
5. Which external libraries are required?
6. Which configuration file describes the role?
7. Which paths are read-only, and which must be writable at runtime?
8. Which user and group run the process?

9. Which directories hold configuration, logs, and pid files?
10. Which TLS or database resources are required?
11. Which service manager owns lifecycle and restart policy?
12. Which package manager installs or updates the service?
13. Which repository or feed provides the packages?
14. Which trust model protects package updates?

A shorter mental grouping is:

```
artifact
  -> dependencies
    -> configuration
      -> runtime state
        -> service manager
          -> package source
            -> trust/update model
```

What deployment should not hide

A deployment should not hide:

- missing runtime-loaded modules,
- implicit package dependencies,
- writable directories with unclear ownership,
- certificate placement,
- private-key permissions,
- database schema assumptions,
- service supervision policy,
- package-manager format assumptions,
- repository trust assumptions,
- or platform-specific differences between general-purpose Linux and OpenWrt.

When these details are hidden, failures appear later as unrelated runtime problems. When they are explicit, deployment becomes another readable part of the system.

What to remember

- Deployment is architecture entering the filesystem.
- Linking proves that the binary can be built; deployment proves that the runtime environment exists.
- SNode.C's component model and public include hierarchy should survive installation and packaging.
- CPack connects install components to ordinary Linux packages, but it does not replace service, certificate, database, repository, or update policy.
- Package dependencies should follow component dependencies.
- Runtime-loaded upgrade and subprotocol modules make installation paths and RPATH part of deployment.

33.8 Closing perspective

Deployment is installed architecture: libraries, runtime module paths, configuration directories, service definitions, package repositories, and trust relationships. Chapter 34 asks how to verify that this installed architecture behaves correctly.

Chapter 34

Testing, Debugging, and Benchmarking

34.1 Why testing follows deployment

Chapter 33 made deployment visible as installed architecture. Chapter 34 asks how that architecture becomes trustworthy: a built and deployed system must still be tested, inspected, debugged, and measured at the boundaries where it claims to be clear.

SNode.C is delivered as libraries, component packages, exported CMake targets, runtime-loaded modules, configuration directories, service definitions, TLS and database dependencies, and, on OpenWrt, cross-compiled packages. Testing, debugging, and benchmarking check whether the boundaries taught throughout the book still hold under pressure:

```
build structure
  -> installed structure
    -> runtime behavior
      -> diagnosed behavior
        -> measured behavior
```

A network framework is not finished when it compiles or installs. Compilation accepts the build graph; installation produces the filesystem shape. Trustworthiness requires reproducible, diagnosable, measurable behavior.

In a layered framework, failures can occur at many boundaries: undeclared component dependencies, accidental transitive includes, exported CMake packages that differ from the in-tree graph, invalid protocol parsing, wrong route dispatch, missing runtime modules, incorrect MQTT session behavior, partial database updates, leaked connection state, or benchmarks that miss the real bottleneck.

The useful question is therefore not only:

Do we have tests?

The better question is the chapter's central rule:

Testing-boundary rule

A test is useful when it says which SNode.C boundary it protects.

34.2 Testing follows framework boundaries

Testing strategy should follow the framework's boundaries:

```
component boundary
  -> what a library target promises to provide

package boundary
  -> what an installed component brings to a target system

protocol boundary
  -> where bytes become HTTP, WebSocket, MQTT, or another meaning

runtime boundary
  -> where configured communication roles become active connections and contexts

configuration boundary
  -> where defaults, command-line arguments, files, and named instances combine

deployment boundary
  -> where the installed filesystem layout must match runtime expectations

operational boundary
  -> where logs, diagnostics, memory behavior, and service supervision matter

performance boundary
  -> where load reveals the limiting part of the system
```

A good test does not have to cover all of these at once. A broad end-to-end test is useful, but it is not a substitute for a focused boundary test when the boundary itself has a contract.

End-to-end warning

End-to-end success does not prove that every boundary is well protected. Use it together with focused tests that make the failing boundary visible.

A focused failure should point toward a focused cause: parser syntax handling, exported targets, public headers, package configuration, runtime module paths, RPATH, upgrade selector configuration, buffering, backpressure, fan-out, or event-loop pressure.

Testing is a map of confidence surfaces, not a flat checklist. Figure 34.1 turns that idea into a boundary map. SNode.C is not protected by one abstract test category. It is protected by tests that make specific architectural promises observable: components should be honest, installed packages should be consumable, protocols should preserve semantics, runtime behavior should remain stable, deployment should match the installed shape, diagnostics should localize failures, and benchmarks should measure the relevant pressure point.

Testing confidence surfaces

- component surface
- package surface
- protocol surface
- runtime surface
- deployment surface
- diagnostic surface
- performance surface

The figure is intentionally different from a testing pyramid. A pyramid answers how many tests of each kind a project might want. Here, the question is architectural: when something fails, which boundary did the failure make visible?

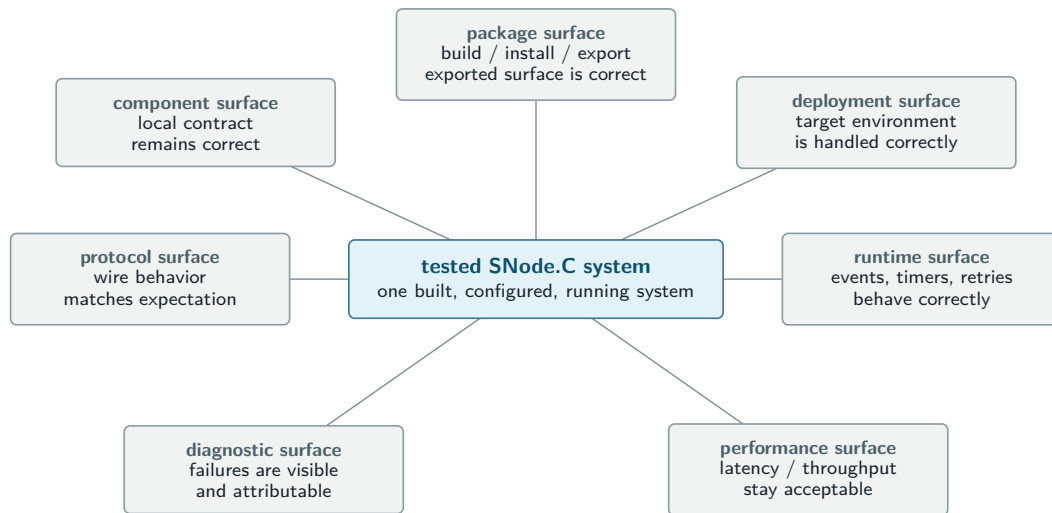


Figure 34.1: Testing confidence surfaces in SNode.C: different tests protect different architectural boundaries, from component truth and installed-package consumption to protocol semantics, runtime behavior, deployment shape, diagnostics, and performance.

SNode.C surface	What may be wrong	Useful confidence method
Component target	hidden include or undeclared library dependency	warning-clean builds, include checks, minimal builds
Exported package	in-tree build works but installed use fails	external consumer project using <code>find_package(snodec ...)</code>
Protocol parser	malformed input accepted or valid input rejected	parser tests with valid, invalid, incomplete, and boundary input
Protocol serializer	generated bytes are non-canonical or wrong	exact-byte tests
Express-style dispatcher	wrong route, wrong params, wrong middleware order	behavior tests and Express compatibility tests where intended
Runtime	socket readiness, timers, retries, shutdown, lifetime	real-socket integration tests
Runtime-loaded module	installed module not found or wrong path encoded	installed deployment tests
Configuration	effective role constellation differs from expectation	generated/effective configuration tests
Database-backed state	hidden local state or partial updates	controlled database fixtures and cleanup
Long-running service	leaks, retained cycles, stale handles	Valgrind, sanitizers, long-running diagnostics
Performance	benchmark measures the wrong boundary	workload-specific benchmarking and bottleneck analysis

The rest of this chapter expands these surfaces: build, protocol, runtime, deployment, diagnostic, and performance confidence.

34.3 Build-time confidence

Build policy is the first test surface

Chapter 32 treated CMake policy as architecture. The same is true for testing. SNode.C's strict build policy is already a form of automated pressure on the component graph.

Warnings are not decorative. They catch suspicious conversions, hidden overloads, unreachable code,

inconsistent initialization, lifetime hazards, and small mistakes that become large when framework code is reused. A bug in a socket abstraction can affect HTTP, WebSocket, MQTT, database-connected applications, and MQTTSuite tools; a hidden dependency in a public header can make a component appear usable only because another component happened to be built first.

Build-time confidence checks whether the component graph is truthful before runtime behavior is observed.

Strict does not mean careless

Strict warnings are valuable; blind strictness is not. SNode.C is built in ordinary Linux environments, optional-feature configurations, and cross-compilation contexts such as OpenWrt, with system headers and third-party libraries.

A useful warning policy distinguishes project-code warnings, platform/compiler differences, third-party header noise, real dependency or lifetime problems, and warnings that would make a target platform fragile without improving quality.

The principle is:

```
be strict for project code
-> suppress narrowly where platform reality requires it
    -> avoid global suppressions that hide real regressions
```

This is how strictness remains maintainable.

Include discipline protects component truth

Include discipline is a component test and a formatting preference. Every public header should include what it uses. Every source file should avoid relying on accidental transitive includes.

This matters because SNode.C is consumed through components, not only as one monolithic source tree. A header that compiles only because another unrelated header came first is not a stable public interface.

The repository already treats include checking as a build concern. When `include-what-you-use` is available, the build can configure it, while cross-compilation contexts are handled more carefully. This supports the architectural promise that a public header expresses its own requirements.

```
public header
-> expresses its own requirements
    -> component can be consumed independently
        -> package boundary remains truthful
```

The component graph drawn by CMake must also be true at the level of headers.

For SNode.C this includes the front-door headers. A minimal consumer that includes `<net/in/stream/legacy/SocketServer.h>` and links `snodec::net-in-stream-legacy` should not need unrelated HTTP, MQTT, WebSocket, or database headers just to compile. Similarly, an Express consumer that includes `<express/legacy/in/WebApp.h>` should receive the source-facing declarations that belong to that abstraction without manually assembling the lower HTTP and socket headers.

Minimal builds are boundary tests

Full builds prove that the broad source tree still compiles, but they can hide mistakes: missing includes, missing link dependencies, optional dependencies that appear mandatory, or components that accidentally rely on nearby higher layers.

Minimal builds expose these problems. A build that requests only an IPv4 legacy stream component should not silently require HTTP, WebSocket, MQTT, or MariaDB support. A small external application that links only against:

```
snodec::net-in-stream-legacy
```

should not need unrelated protocol components. If it does, the component boundary is lying. Minimal builds are architectural tests.

```
minimal build:
    tests component honesty

full build:
    tests broad integration
```

Full builds still matter

Minimal builds protect boundaries; full builds protect integration. SNode.C also needs confidence that the complete system still compiles when optional surfaces are enabled together: legacy and TLS streams, IPv4 and IPv6, Unix domain sockets, Bluetooth where available, HTTP/Express, WebSocket, MQTT and MQTT-over-WebSocket, MariaDB, examples, and applications. Both are needed because they answer different questions.

Installed-consumer builds are especially important

An in-tree build is not enough for a framework. A downstream application does not usually link against local build-tree target names. It uses the installed package interface:

```
find_package(snodec REQUIRED COMPONENTS net-in-stream-legacy)

target_link_libraries(myapp
    PRIVATE
    snodec::net-in-stream-legacy
)
```

This checks that the package configuration was installed, the requested component is supported, dependencies load recursively, the exported target and namespace are correct, include directories and public dependencies are present, public front-door headers can be included, link dependencies are truthful, and the installed library layout can be consumed outside the source tree.

This kind of test is easy to underestimate. For a framework with exported CMake targets, it is one of the most important tests. The question is broader than:

Can SNode.C build itself?

The question is also:

Can a separate project consume the installed SNode.C package exactly as the book teaches?

For a framework, the installed consumer is not a secondary scenario. It is the public contract.

Package builds test deployable truth

Chapter 33 explained that package dependencies should follow component dependencies. Chapter 34 adds the testing consequence: a package build tests whether that deployment story is still true.

A CPack package build can reveal mistakes that a source build misses: missing install rules, missing package dependencies, wrong component grouping, libraries installed without their runtime dependencies, headers installed without the package metadata that makes them usable, or applications installed without their required data files. It checks whether CPack component metadata still follows the component graph that Chapter 32 described.

OpenWrt-targeted package builds add another confidence surface. They can check whether SNode.C and applications built on top of it survive cross-compilation, target ABI assumptions, package recipes, feed layout, and embedded filesystem constraints. This is deployment/testing guidance, not a claim that every such OpenWrt package test already exists in the repository.

A package build is therefore not just a release step. It is a test of the deployable shape of the architecture.

34.4 Protocol-boundary confidence

Protocol tests protect meaning boundaries

A protocol layer is where bytes receive meaning. At a lower stream level, incoming data is just a sequence of bytes. At a higher layer, those bytes may become an HTTP request, an HTTP response, a WebSocket frame, a server-sent event stream, an MQTT control packet, a route match, a middleware transition, or a database command response.

A protocol test protects the place where bytes stop being anonymous and start carrying protocol meaning. It should make clear which boundary it protects:

```
bytes
-> parser
    -> protocol object
        -> application-facing behavior
```

or, in the other direction:

```
protocol object
-> serializer
    -> exact bytes on the wire
```

The closer a layer is to external input, the more important invalid and boundary input becomes. Network code receives incomplete input, split input, malformed input, hostile input, and input that arrives at inconvenient times. A parser that only accepts beautiful examples has not been tested. It has only been demonstrated.

Parser tests should include bad input

HTTP parser tests should not only test a common valid request. They should also cover malformed request lines, invalid methods, malformed header fields, missing line terminators, duplicate or unusual headers, invalid percent encodings, oversized values, chunked transfer edge cases, and incomplete input that arrives in several pieces.

WebSocket parsing and frame handling require their own invalid cases: invalid opcodes, incorrect control-frame lengths, fragmentation errors, masking errors, unexpected close behavior, and reserved-bit misuse where unsupported.

MQTT parsing requires another set: malformed remaining-length encodings, invalid control-packet combinations, invalid topic names, incomplete packets, unexpected packets for the current session state, and illegal QoS-related packet sequences where those paths are supported.

The goal is not to list every possible protocol conformance case in this book. The goal is to name the testing responsibility correctly. When SNode.C raises stream bytes into protocol meaning, tests must check both the accepted and rejected forms of that meaning.

Serializer tests should check exact bytes

Serializers deserve exact-byte tests. This is especially true for binary protocols. A loose test that only checks whether another implementation happened to accept the output may hide mistakes.

A serializer test should ask whether fixed header bits are correct, whether the length is encoded correctly, whether reserved bits are clear where they must be clear, whether flags are correct, whether strings are length-prefixed correctly, whether byte order is correct, and whether canonical output is preserved where the protocol expects it.

Exact-byte tests are sometimes criticized as brittle. For protocol serializers, that brittleness is useful. The protocol is defined in bytes. The test should be precise enough to catch byte-level drift.

HTTP tests should check the whole response surface

At the HTTP layer, a useful test checks both message correctness and connection correctness:

```
message correctness:
  status, reason phrase where relevant, headers, body, body length

connection correctness:
  keep-alive, close behavior, chunking, error responses, upgrade interaction
```

A test that checks only the body may pass while the server handles lifetime or headers incorrectly. That matters because HTTP is both a message protocol and a connection behavior. The body is only one part of the contract.

Express-style routing needs semantic tests

The Express-like layer is one of the strongest cases for semantic testing in SNode.C. It is not enough to check that a handler was called once. The dispatcher has a detailed semantic contract: path matching, prefix matching, end anchoring, middleware order, nested router behavior, application mounting, `next()` behavior, error flow where supported, wildcard captures, named parameters, parameter decoding, query-string treatment, route metadata, and parameter scoping.

Small differences here can create large application differences. A dispatcher can look correct in simple examples and still fail in real applications.

Therefore, behavior names are useful. A test named:

```
query_string_does_not_affect_route_match
```

already documents a semantic contract. A test named:

```
nested_router_restores_params
```

points directly to a boundary between one router context and another. Names like these turn a test suite into executable documentation.

Express compatibility is a special confidence source

SNode.C does not need to imitate Node.js internally. But where SNode.C intentionally offers Express-like behavior, compatibility tests against Node.js/Express are a particularly strong confidence source.

An independent reference implementation gives stronger confidence than self-testing alone. A useful compatibility test can run the same route tree against:

```
Node.js / Express reference
SNode.C Express-like application
```

and compare structured observations: final response body, status and headers, which route matched, which middleware ran, what parameters were visible, what mounted path remained, and what happened after leaving a nested router.

This is a very SNode.C-specific testing story. It protects the public semantic promise of the Express-like layer. The goal is not imitation for its own sake. The goal is to avoid surprising users at the boundary where the book deliberately says “Express-like”.

WebSocket tests have two phases

WebSocket testing has two different confidence surfaces. First, the HTTP upgrade must be correct. Second, the WebSocket frame behavior must be correct.

The upgrade phase checks HTTP-level negotiation: **Upgrade** handling, **Connection** handling, **Sec-WebSocket-Key**, **Sec-WebSocket-Accept**, subprotocol negotiation, rejection paths, and interaction with ordinary HTTP routes.

The frame phase checks WebSocket-level behavior: text frames, binary frames, ping and pong, close handshake, fragmentation, invalid frames, and lifetime after abnormal peer behavior.

A WebSocket echo test is a beginning. It is not enough. Echo proves that one happy-path data loop works. It does not prove that the upgrade boundary, frame boundary, control-frame boundary, close boundary, and subprotocol boundary are correct.

EventSource tests are lifetime tests

Server-sent events look simple at the protocol surface. Operationally, they are mostly about time and lifetime.

An EventSource test should check **Content-Type**, connection persistence, event formatting, event names, data framing, retry hints where used, flushing behavior, disconnect behavior, cleanup when the peer disappears, and behavior when producers pause and resume.

One event is not enough. One event proves formatting; a stream test proves lifetime. A useful test asks whether events continue to flow and whether resources are released when they stop flowing.

```
protocol surface
  -> event stream syntax

runtime surface
  -> long-lived connection and cleanup
```

Both matter.

MQTT tests must model sessions

MQTT is a session protocol rather than a packet format alone. Packet tests are necessary, but not sufficient.

Useful MQTT tests include connect and disconnect, keep-alive behavior, subscribe and unsubscribe, publish to matching subscribers, retained messages where supported, QoS behavior where supported, duplicate client identifiers, malformed packet rejection, broker restart behavior, bridge reconnect behavior, topic matching, and mapping behavior in integrator-style applications.

The important distinction is:

```
packet correctness
  -> are MQTT bytes interpreted correctly?

session correctness
  -> does the MQTT relationship behave correctly over time?
```

A test strategy that checks only packets may miss MQTT behavior. Broker-oriented and client-oriented roles must be tested as roles, not just as byte writers.

Database-backed protocol tests must control state

Database-backed components introduce persistent state. Persistent state makes tests powerful. It also makes them dangerous if uncontrolled.

A test that depends on a developer's local database contents is not reproducible. A database-backed test should create or use controlled state: a temporary test database, a generated schema, known fixtures, transaction rollback, explicit cleanup, isolated credentials, and isolated database names for test runs.

This matters for MQTT storage, IoT history, monitoring, or any persistence-facing application. A test should be repeatable. Running it twice should not depend on hidden leftovers from the first run.

State is useful. Uncontrolled state is a source of false confidence.

34.5 Runtime confidence

Real sockets are not optional

Mocks are useful for local logic. They cannot replace real-socket tests for a socket framework.

A real-socket test checks behavior that mocks often hide: bind behavior, listen behavior, accept behavior, connect behavior, local port conflicts, IPv4 and IPv6 differences, Unix domain socket path behavior, readiness notification, half-close behavior, buffering, backpressure, operating-system error paths, and TLS handshake behavior.

These tests are more expensive than pure unit tests. They may be more sensitive to the environment. They are still essential. A socket framework that is never tested with sockets is under-tested.

For SNode.C, this is also a model check. The framework teaches:

```
configured communication role
  -> registered runtime-visible instance
    -> connection
      -> context
```

Real-socket tests prove that this path works against the operating system, outside the conceptual diagram.

Timers, retries, reconnects, and shutdown need runtime tests

Chapter 20 described communication-over-time concerns: timeouts, retries, reconnects, shutdown paths, and failure states. Chapter 34 treats them as runtime confidence surfaces. These behaviors cannot be fully proven by local unit tests because they depend on runtime sequencing.

Useful tests should cover questions such as: what happens when a client connects before the server is available; what happens when a server disappears after a connection was established; what happens when a peer stops reading; what happens when a peer closes cleanly; what happens when a peer disappears abruptly; what happens when retry limits are reached; and what happens during shutdown while data is queued.

Not every case must be automated immediately, but these are runtime questions. They belong where configured roles meet real time.

Backpressure and buffer pressure deserve explicit attention

Event-driven frameworks are often easiest to demonstrate with fast peers. Real systems often contain slow peers. A slow client, a full write buffer, or a burst of MQTT fan-out can reveal problems that normal examples never show.

SNode.C applications should therefore be tested under buffer pressure: a peer that reads slowly, a peer that stops reading, many subscribers receiving the same MQTT publication, WebSocket clients with different receive speeds, SSE clients held open for long periods, and database writes slower than incoming message rate.

Backpressure tests ask a different question from throughput tests:

```
What happens when output cannot progress as fast as input?
```

The answer should be a policy, not an accident. Depending on the role, the policy may involve bounded buffering, a drop policy, a disconnect policy, a retry/defer policy, or an operator-visible degraded state. A robust application should not grow memory without limit, block the event loop accidentally, or hide the failure until the process becomes unstable.

This topic is especially important for brokers, bridges, dashboards, live-update streams, and IoT ingestion systems.

Connection and context lifetime must be checked over time

SNode.C's context model gives each connection a protocol endpoint. That is clean architecture. It also creates a lifetime responsibility.

A connection ending should release its associated protocol state. A reconnecting client should not retain stale connection objects. A WebSocket upgrade should not leave the HTTP-side object graph alive unintentionally. A callback should not keep a connection alive after the runtime believes it is gone.

Memory tools help here, but the architectural question comes first:

Does the lifetime of each object match the lifetime of the role it represents?

This question applies to connections, socket contexts, protocol contexts, factories, callbacks, WebSocket subprotocol objects, MQTT sessions, EventSource clients, database handles, and dynamically loaded modules.

Long-running tests are useful because some lifetime bugs do not appear in a short request/response example. They appear after many connect/disconnect cycles, many reconnect attempts, many upgrades, or many subscriptions.

Configuration is runtime behavior

SNode.C configuration shapes the effective runtime system.

A configuration test should therefore check the effective configuration after defaults, command-line options, configuration files, named instances, option groups, generated configuration, and application-specific settings have been combined.

Useful checks include whether a configured communication role is enabled or disabled as expected, whether a server binds to the expected address, whether a client connects to the expected remote endpoint, whether TLS settings are applied to the correct role, whether retry and timeout settings have the expected effective values, whether generated configuration can be reused, and whether invalid combinations fail clearly.

This matters because configuration is one of the main ways SNode.C systems become multi-role systems. A generated configuration file can be a map of the deployed role constellation. Testing that map protects the operational shape of the application.

34.6 Deployment confidence

Installed behavior is not the same as build-tree behavior

A program that works from the build tree may fail after installation. That is not unusual. The installed environment is different: library search paths change, RPATH behavior matters, runtime-loaded modules move to installed locations, configuration directories change, service users may differ from development users, TLS material may be read from protected paths, database access may depend on system configuration, and package dependencies may be incomplete.

Chapter 33 explained these deployment surfaces. Chapter 34 adds the test implication:

installed behavior must be tested as installed behavior

It is not enough to run only from the source tree.

Runtime-loaded modules are deployment test surfaces

HTTP upgrade support, WebSocket handling, WebSocket subprotocols, and MQTT-over-WebSocket composition can depend on runtime-loaded or path-sensitive libraries. This creates a special deployment test surface.

A build-tree test may pass because every library is nearby. An installed-system test may fail because a module is not installed, not found, not in the expected directory, or not reachable through the encoded search path. A useful installed test should therefore exercise the real protocol composition:

```

start installed application
  -> perform HTTP request
    -> negotiate WebSocket upgrade
      -> select subprotocol where needed
        -> exchange WebSocket or MQTT-over-WebSocket traffic

```

This checks the installed runtime shape and protocol correctness.

RPATH and library search should be verified by behavior

RPATH problems often do not look like architecture problems at first. They look like runtime loader errors, missing symbols, or modules that cannot be opened. For SNode.C they are architecture problems because runtime-loaded protocol composition depends on correct installed paths.

A deployment test should answer whether the application starts without ad-hoc environment variables, whether the platform loader finds the required shared libraries, whether the framework finds upgrade and subprotocol modules, whether the installed layout works after package installation, and whether the same package works on the intended target system.

The last point is especially important for OpenWrt. Cross-compilation can accidentally encode build-host assumptions into target artifacts if deployment is not tested carefully.

Service-level tests protect operational reality

A service is a process under supervision. General-purpose Linux may use systemd or another service manager. OpenWrt normally uses `procd`.

A protocol test asks whether HTTP, WebSocket, MQTT, or another protocol behavior works. A service-level test asks whether the deployed role can be operated. It checks whether the service starts as the intended user, has access to configuration files, can create log and pid files, handles restart behavior, shuts down cleanly, exposes useful logs, and fails clearly when required resources are missing.

This is not the same as a protocol test. A protocol test may show that HTTP or MQTT works. A service-level test shows that the deployed role can be operated.

OpenWrt tests should respect embedded constraints

OpenWrt deployment should be tested as OpenWrt deployment. A desktop Linux binary copied onto a router is not the intended confidence path.

For OpenWrt-targeted packages, useful checks include whether the package recipe builds in the SDK, whether dependencies are declared explicitly, whether package size is acceptable for the target, whether installed files land in expected paths, whether service scripts integrate with `procd`, whether configuration files match OpenWrt filesystem expectations, whether optional components are not pulled in unnecessarily, and whether logs and writable state are appropriate for embedded storage.

This connects testing to the main lesson of Chapter 33:

```

OpenWrt deployment
  -> Linux deployment
    -> under embedded, cross-compiled, package-managed constraints

```

The test strategy should respect those constraints instead of pretending that all Linux targets are the same.

34.7 Debugging and diagnostics

Debugging begins by finding the broken boundary

When a SNode.C application fails, the first debugging question should not be:

Which line of code is wrong?

The better first question is:

Which boundary is failing?

Examples:

```
Does the component fail to build?
-> check includes, link dependencies, exported targets

Does the application work in-tree but fail installed?
-> check exported package, installed paths, RPATH, runtime modules, service user

Does the connection never establish?
-> check endpoint configuration, bind/connect status, firewall, TLS handshake, retry state

Does HTTP parse incorrectly?
-> check parser boundary and input bytes

Does routing choose the wrong handler?
-> check Express-style dispatcher semantics

Does MQTT publish flow fail?
-> check session state, topic matching, QoS path, subscriber fan-out

Does behavior degrade over time?
-> check lifetime, buffering, backpressure, leaks, slow peers
```

This boundary-first approach is one of the practical benefits of a layered framework. SNode.C gives the developer names for the places where failure can occur. Debugging should use those names.

Configured instance names are diagnostic handles

Configured instance names become diagnostic handles when they connect configuration, logs, callbacks, generated files, and operator language.

For example, a system may have roles such as:

```
admin-http
mqtt-in
mqtt-out
local-control
event-stream
store-db
```

When logs use names like these, the system becomes easier to reason about. A vague message such as:

```
connection failed
```

is much less useful than a message that identifies the role, endpoint, state, protocol phase, and reason.

Good diagnostics preserve architecture in the log output. The log should help the reader see which role, connection, context, or protocol boundary produced the event.

Reproducibility is part of debugging

A bug report is most useful when it can be reproduced at the right boundary. For SNode.C, a good reproduction should identify the component set, build type, installed or build-tree context, enabled lower families, legacy or TLS mode, configured instance names, exact protocol input where relevant, runtime sequence, and target platform.

A routing bug should ideally be reproducible with a small route tree. A parser bug should ideally be reproducible with a small byte sequence. A deployment bug should ideally be reproducible from package

installation and service startup. A reconnect bug should ideally describe the timing of peer availability, failure, retry, and recovery.

The goal is not bureaucracy. The goal is to put the bug at the smallest explicit boundary.

Memory tools verify lifetime assumptions

Valgrind, sanitizers, and similar tools verify architectural lifetime assumptions alongside generic C++ memory behavior.

They help answer questions such as: does a context disappear when its connection disappears; does a callback retain something longer than intended; does a WebSocket upgrade leave old HTTP state reachable; does a reconnect loop accumulate stale objects; does a database command path leak handles; does a dynamically loaded module leave unexpected reachable memory; and does a long-running MQTT broker accumulate session or subscription state incorrectly?

The tool reports are low-level. The interpretation should be architectural. In a framework, a memory leak often means an ownership boundary was not expressed correctly.

Runtime diagnostics should be designed, not improvised

Useful diagnostics are designed around the same boundaries as the framework.

A good diagnostic event should identify the configured instance, endpoint, state transition, active protocol boundary, severity or retryability, and relevant configuration value.

Long-running systems need logs that remain useful after the developer has left the terminal.

34.8 Benchmarking

Benchmarking should find the limiting boundary

Benchmarking is not about producing large numbers. A useful benchmark asks:

Which boundary becomes limiting under this workload?

The answer may be different for different applications. One benchmark may be parser-bound. Another may be TLS-bound. Another may be dispatcher-bound. Another may be socket-buffer-bound. Another may be database-bound. Another may be dominated by MQTT fan-out, slow subscribers, or backpressure behavior.

A number without a boundary is hard to interpret.

Workload shape matters

A benchmark should describe its workload clearly. Useful dimensions include:

```
connection shape:
  number of connections, connection duration, slow peers

message shape:
  message size, request rate, publish rate, frame size, frame frequency

protocol shape:
  HTTP route depth, middleware count, TLS or legacy mode, WebSocket, SSE, MQTT fan-out

state shape:
  number of subscribers, retained/session behavior where supported, database write rate

platform shape:
  build type, target platform, installed or build-tree context

operational shape:
  logging level, diagnostics enabled, service supervision
```

Changing one dimension may change the bottleneck. One fast client, thousands of slow clients, one MQTT subscriber, many fan-out subscribers, HTTP without TLS, TLS, database-backed state, and in-memory protocol handling all tell different stories. The result should never be separated from workload shape.

Avoid misleading comparisons

Benchmarks mislead when they measure the client tool instead of the server, change logging between runs, compare build-tree and installed runs with different paths, generalize plain HTTP to TLS, generalize loopback to deployment, ignore slow subscribers, or test database-backed behavior only against an empty local database.

The refined rule is simple:

A benchmark is only meaningful together with the boundary and workload it measures.

The benchmarking tone should be measured behavior tied to architecture, not marketing numbers.

Latency and throughput are different questions

Throughput asks how much work the system completes per time interval. Latency asks how long one operation takes. They are not interchangeable.

A system may have high throughput but poor tail latency under load. A system may respond quickly to one client but degrade when many connections stay open. A system may process MQTT packets quickly until database persistence becomes active. A system may handle many SSE clients until one slow group creates buffer pressure.

Benchmarking should separate average latency, tail latency, throughput, connection count, memory growth, CPU load, backpressure behavior, and recovery after overload.

In an event-driven model, one slow boundary can affect other connections if the application does not handle pressure correctly.

Benchmarking should not replace profiling

A benchmark says that something became slow. Profiling helps explain where.

For SNode.C, profiling may show pressure in parsing, routing, serialization, TLS operations, logging, memory allocation, topic matching, database commands, dynamic dispatch, event-loop activity, or user application callbacks.

```
benchmark:
  what became slow?

profile:
  where is the cost?
```

The benchmark identifies the workload. The profile identifies the cost. The architecture helps interpret both.

34.9 Current practice and future strategy

A chapter like this must be honest: not every confidence surface is already automated. Some checks are daily development practice, some manual debugging, some desirable CI coverage, and some future hardening work.

A useful distinction is:

```
existing repository practice
-> what is already present and used
```

```
manual development practice
-> what a developer can run while diagnosing or refining behavior

recommended strategy
-> what should be protected over time as the framework and applications grow
```

This distinction keeps the chapter precise and useful. A reader does not need every test category immediately; they need to learn how to think about confidence in a layered framework.

The practical habit is to add tests at the boundary where a bug mattered. URL decoding may need a parser or utility test; router parameter restoration may need a dispatcher test; installation-only bugs need installed tests; slow-client bugs need pressure tests.

Regression tests should protect the semantic boundary that failed.

What to remember

- Testing in SNode.C should follow framework boundaries and make clear which boundary each test protects.
- Build-time checks, include discipline, minimal builds, and installed-consumer builds protect component truth, public headers, dependency surfaces, and exported package behavior.
- Runtime tests, diagnostics, and benchmarks should preserve the same layer and boundary vocabulary used in the design.

34.10 Closing perspective

Testing asks whether the chosen boundaries hold under build, runtime, deployment, diagnostic, and measurement pressure. The chapter's practical standard is deliberately narrow: every confidence activity should name the boundary it protects. Chapter 35 turns to the prior question: how to choose those boundaries wisely in the first place.

Part XI

Designing with SNode.C

This part turns the accumulated technical vocabulary into design judgment. After the reader has seen layers, roles, protocols, applications, systems, build structure, deployment, and testing, the remaining question is how to choose and evolve boundaries wisely.

Chapter 35 focuses on architectural judgment: choosing the right layer and boundary for a concern. Chapter 36 then asks how an application or framework codebase can be extended safely once those boundaries are understood.

Chapter 35

Architectural Judgment: Choosing the Right Layer and Boundary

35.1 Why this chapter matters now

Chapter 34 asked how to test, debug, and measure whether SNode.C boundaries hold. Chapter 35 asks the prior architectural question: how should those boundaries be chosen in the first place?

By this point, the reader has seen the runtime, lower communication families, connection model, contexts and factories, configuration, diagnostics, TLS, HTTP, Express-like routing, SSE, WebSocket, MQTT, MQTT over WebSocket, applications, systems, build components, deployment, and testing strategy. That is enough knowledge to build many things; judgment turns that familiarity into architectural maturity.

The hardest questions in real systems are often not questions of syntax or API usage. They are questions like these:

- Which layer should own this concern?
- Should this be one role or two?
- Should this be native MQTT or MQTT over WebSocket?
- Should this be a local Unix-domain service or a network-facing HTTP endpoint?
- Should this behavior belong in a context, middleware, subprotocol, persistence service, or separate application?
- Should these roles live in one executable or in several?

This is not another API chapter. It is a synthesis chapter about the architect's task in SNode.C:

```
right concern
-> right layer
    -> right boundary
```

The following questions are not a mechanical checklist. They are recurring pressure points where SNode.C forces the architect to place meaning deliberately.

Boundary judgment checklist

- What kind of concern is this?
- Which boundary owns it?
- Which role should see it?
- Which failure policy belongs here?

Here, *role* is often used in the system-design sense. When the text refers to SNode.C runtime entities, it uses the more precise language of configured communication roles and registered runtime-visible instances.

35.2 Good judgment begins by refusing category mistakes

One of the simplest SNode.C habits is to avoid putting the right concern in the wrong layer. A category mistake happens when a concern is implemented in a layer that cannot properly own it.

Common examples are:

- putting protocol behavior into the application-side server/client handle or configuration shell,
- putting connection policy into an arbitrary application handler,
- putting role orchestration into a factory,
- putting transport concerns into the web layer,
- putting database ownership policy into a protocol endpoint,
- putting service-supervisor policy into per-connection code,
- putting system-boundary decisions into request callbacks.

SNode.C teaches this judgment because it makes layers, roles, and boundaries explicit. It asks:

Category rule

Choose the layer by the kind of concern, not by where the code first feels convenient.

That is often the most important design question of all.

A warning sign is one class, callback, configuration section, or executable that knows too many unrelated reasons for change. Often the code is locally reasonable but placed at the wrong boundary.

35.3 Choosing the communication family

At the lowest practical architectural level, the communication family shapes endpoint identity, deployment assumptions, permissions, diagnostics, and operational boundaries.

IPv4, IPv6, Unix domain sockets, Bluetooth RFCOMM, and Bluetooth L2CAP imply different communication situations, not just different address syntax.

A good first rule is:

- use **IPv4/IPv6** when the role is genuinely network-facing,
- use **Unix domain sockets** when the role is local to one machine and path-based IPC is the natural boundary,
- use **Bluetooth RFCOMM/L2CAP** when the role is genuinely nearby, paired, device-near, or commissioning-oriented,
- and do not choose a family merely because it is familiar.

The right family is not the one the developer already knows best. It is the one that matches the system boundary directly.

Bluetooth is a useful example. It should not be treated as a general-purpose integration bus merely because it can carry byte streams. It belongs where nearby peer exchange, device-local setup, or device-near communication is the real boundary.

35.4 Choosing the protocol surface

Once the lower family is chosen, the next decision is the protocol surface. The useful question is not simply which protocol is available, but:

What kind of conversation does this boundary actually want to have?

A compact decision view is:

Boundary wants...	Candidate surface
compact, domain-specific byte conversation	plain stream-oriented protocol endpoint
request/response web compatibility	HTTP

Boundary wants...	Candidate surface
structured web application composition	Express-like layer
one-way live HTTP observation	SSE
upgraded bidirectional messaging	WebSocket
brokered machine messaging	MQTT
MQTT semantics through a web-compatible boundary	MQTT over WebSocket

This is not a checklist to memorize; it forces the architectural question into the open.

A plain stream endpoint is not automatically more appropriate because it is lower. HTTP, Express-like routing, SSE, WebSocket, or MQTT may be simpler when the boundary already has those semantics. The simplest appropriate stack matches the conversation without pretending it is something else.

35.5 Choosing native or composed protocol form

Protocol composition raises a second question: not only *which* protocol family to use, but whether to use it directly or as part of a larger stack.

For example:

```

native MQTT
  or
MQTT over WebSocket

plain HTTP response
  or
SSE over HTTP

ordinary HTTP request/response
  or
HTTP upgrade to WebSocket

```

A native protocol form means the protocol itself is the boundary. A composed protocol form means the protocol semantics ride through another boundary.

A good decision rule is:

```

choose the simplest appropriate stack
-> that matches the actual system boundary

```

That means:

- do not force WebSocket where SSE is enough,
- do not force MQTT over WebSocket where native MQTT is the clearer boundary,
- do not force plain HTTP where middleware and route composition are already required,
- do not force raw stream handling when the communication is naturally HTTP- or MQTT-shaped.

SSE, WebSocket, and MQTT over WebSocket are a useful comparison:

```

SSE
-> long-lived one-way observation

WebSocket
-> bidirectional interaction

MQTT over WebSocket
-> MQTT semantics through a WebSocket-compatible boundary

```

The question is not which stack is richer, but which stack fits the boundary.

35.6 Choosing role boundaries

At application and system scale, the architect must decide how many roles should exist. A common mistake is to begin with one all-purpose communication role and keep adding responsibilities until it becomes impossible to reason about.

SNode.C encourages a healthier question:

Is this really one role, or are several distinct roles being forced together?

For example, these often deserve to remain explicit:

- administration,
- observation,
- integration,
- local control,
- interactive browser communication,
- persistence,
- bridge or gateway behavior.

These roles may be implemented in one process or in several. Separating roles does not automatically mean separating executables. A role can remain explicit inside one process if the boundary remains visible in code, configuration, diagnostics, and failure policy.

Visibility matters. When roles differ in audience, protocol, state ownership, or deployment boundary, hiding them behind one vague communication surface makes the system harder to operate and change.

35.7 Choosing process and deployment boundaries

SNode.C supports both multi-role single applications and systems built from cooperating executables. The architect must decide which shape fits the system.

Use **one executable** when:

- the roles belong naturally together,
- they share operational lifecycle closely,
- their coupling is strong and intentional,
- a single configuration shell improves clarity,
- failure policy is naturally shared.

Use **several executables or services** when:

- deployment boundaries are genuinely different,
- operational lifecycles differ,
- failure isolation matters,
- protocol responsibilities are cleanly separable,
- one role constellation would otherwise become muddy.

Packaging is an operational boundary decision, not a moral ranking. Good architecture has an operational shape that matches the real system boundary.

35.8 Choosing the implementation layer

At the code-organization level, a very practical question appears again and again:

Where should this logic live?

A useful map is:

Concern	Usually belongs near
endpoint identity	lower communication family / configured role
byte interpretation	parser, context, or protocol object
connection-local protocol endpoint behavior	<code>SocketContext</code> or protocol context

Concern	Usually belongs near
endpoint construction policy	factory
web flow and route composition	middleware / router
semantics above WebSocket	subprotocol
MQTT session behavior	MQTT layer
deployment variability	configuration
cross-role orchestration	application or system layer
durable state	persistence service
failure visibility	role/application diagnostics

A `SocketContext` is not a dumping ground for every connection-adjacent concern. It owns behavior that belongs to the protocol endpoint of a concrete connection. Construction policy belongs in factories, web request flow in middleware or routers, semantics above WebSocket in subprotocols, and cross-role orchestration above the connection-local endpoint when it truly spans roles.

35.9 Choosing code or configuration

Earlier chapters showed that `SNode.C` treats configuration as a first-class architectural surface. That means the reader frequently has to decide whether a choice should live in code, in configuration, or in both.

A useful distinction is:

```
code
  -> invariants and protocol truth

configuration
  -> deployment, role selection, operator-facing variability

generated configuration
  -> repeatable deployed role shape
```

Put choices in **configuration** when they are deployment-specific, operator-facing, role-selecting, environment-dependent, or likely to vary across runs. Keep choices in **code** when they are part of essential protocol logic, invariants of the application design, not reasonably changeable by operators, or fundamental to internal correctness.

Making a choice configurable is not automatically better. A configurable invariant is often just an unprotected design mistake. `SNode.C`'s configuration model is useful because it separates stable design from deployment variability.

35.10 Asking what the endpoint should know

Whenever adding behavior to a `SocketContext`, an HTTP handler, middleware, a WebSocket subprotocol, or an MQTT-facing context, ask:

Does this endpoint genuinely need to know this?

Endpoint-local state belongs there when it describes the current protocol relationship. Protocol parsing, protocol response logic, and connection-local protocol state usually belong there; durable state, global orchestration, cross-role coordination, deployment topology, log-file policy, database ownership, and service-supervisor policy usually do not.

Strong boundaries make this question easier to ask.

35.11 Separating protocol meaning from carrier mechanics

Another frequent category mistake is to confuse protocol semantics with carrier mechanics.

A useful discipline is to ask:

Is this about the meaning of the conversation?
 or
 Is this about how the conversation is carried?

For example:

- TLS configuration is usually a connection-layer concern,
- connection retry/reconnect policy is usually role/configuration-level,
- protocol keep-alive timing may belong to the protocol layer,
- MQTT keep-alive is a protocol concern,
- route matching in Express is an application-layer concern,
- bind host / port / path / channel / PSM are carrier or lower-family concerns.

A richer protocol layer does not erase the carrier beneath it; it gives meaning to communication carried by that lower layer.

35.12 Choosing the extensibility point

SNode.C offers several extensibility points. Choose the one that matches the actual variation.

A useful map is:

```
factory
  -> construction variation

middleware
  -> web flow variation

router
  -> path and application structure variation

HTTP upgrade / WebSocket subprotocol selector
  -> protocol-above-HTTP/WebSocket variation

configured communication role or separate executable
  -> operational variation

package/build component
  -> deployment and availability variation
```

Use a factory when the variation is endpoint construction. Use middleware when the variation is cross-cutting web behavior. Use routers when the variation is mounted path structure. Use subprotocol selection when the variation is truly above WebSocket. Use separate executables, configured communication instances, or package components when the variation is operational or deployment-level rather than only code-level.

The wrong extensibility point can feel flexible at first and chaotic later.

35.13 Placing failure policy

The framework's timeout, retry, reconnect, shutdown, disablement, and state model gives the architect strong tools. But the key question remains:

Where should this failure policy be expressed?

A useful rule is:

- put outer retry/reconnect policy in configured role and flow-control settings,
- put protocol-specific timing expectations in the context or protocol layer,
- put operator-visible failure shaping in the application shell and diagnostics layer,

- put degraded-state behavior at the role or system boundary that owns the consequence,
- do not force every failure concern into one place.

Failure policy belongs to the role that owns the boundary, not just to the lowest socket that notices the error.

35.14 Choosing what must be operator-visible

System architecture is not limited to internal correctness. It is also about operability.

A good SNode.C architect should routinely ask:

- Which roles should be named explicitly?
- Which configured communication roles need visible instance names?
- Which settings should be externally configurable?
- Which boundaries need clear logs?
- Which connection or protocol counters matter?
- Which role failures should be immediately visible?
- Which protocol stacks are important enough to be obvious in packaging and configuration?

A concern that must be operated must be named, configured, logged, or measured somewhere visible. SNode.C's configuration and diagnostics shell make that concrete.

A useful log message should preserve the boundary: role, configured instance name where applicable, endpoint, state, protocol phase, and reason. Without that information, a system may still be technically correct but operationally opaque.

35.15 Simplicity usually beats cleverness

In SNode.C, the best architecture is often not the cleverest one. It is often the simplest appropriate one.

That means:

- one protocol layer when one is enough,
- one role when one is enough,
- one executable when one is enough,
- explicit separate roles when the boundaries are real,
- clear factories instead of hidden construction magic,
- explicit subprotocols instead of hidden branching,
- configuration where the operator truly needs control,
- domain rules kept in the application domain rather than smuggled into transport or protocol plumbing.

SNode.C provides communication architecture. Domain code still owns domain semantics. The framework helps decide how domain rules meet communication boundaries; it does not make those domain decisions automatically.

The framework rewards clarity more than cleverness.

35.16 The deeper rule: preserve meaning as long as possible

A more philosophical way to summarize architectural judgment in SNode.C is this:

`preserve meaning as long as possible`

That means:

- keep lower-family meaning visible until it is properly abstracted,
- keep role meaning visible until roles are properly composable,
- keep protocol meaning visible until a higher protocol layer can properly own it,
- keep persistence meaning visible until a durable-state boundary can properly own it,

- keep system-boundary meaning visible until deployment packaging properly hides it.

Preserving meaning does not mean refusing abstraction. It means not erasing a distinction before another layer can properly own it.

A system stays maintainable when its important meanings are not erased too early.

35.17 Common bad instincts this chapter tries to cure

Some common bad instincts are:

- choosing a protocol because it is fashionable rather than because the boundary wants it,
- collapsing too many roles into one binary too early,
- over-generalizing before the real variation is understood,
- choosing the most general abstraction before the real reasons for variation are known,
- hiding important system structure from configuration and logs,
- pulling cross-role logic into per-connection contexts,
- turning every cross-cutting concern into middleware even when it belongs to configuration, protocol state, or system orchestration,
- treating every richer layer as if it replaces the layers beneath it.

SNode.C helps cure these instincts by keeping the actual boundaries visible.

35.18 Common misunderstandings

It helps to clear away a few misunderstandings explicitly.

Misunderstanding 1: “Architectural judgment means memorizing a preferred stack.”

Corrected view: architectural judgment means learning how to choose the right layer and boundary for the actual system concern.

Misunderstanding 2: “The framework should tell me the one correct packaging style.”

Corrected view: SNode.C supports several valid packaging and deployment shapes; the architect must choose the one that matches the operational boundary.

Misunderstanding 3: “Good abstraction means hiding as many layers as possible.”

Corrected view: in SNode.C, good abstraction usually means preserving the right meanings until they can be properly composed.

Misunderstanding 4: “Once I know the APIs, architectural judgment is automatic.”

Corrected view: API knowledge is necessary, but judgment comes from learning where concerns belong and where they do not.

Misunderstanding 5: “This chapter is less technical because it is about judgment.”

Corrected view: this chapter is technical because boundary mistakes become technical debt. Bad boundary decisions are among the most expensive technical mistakes in real systems.

35.19 The architectural principle

Architectural judgment in SNode.C means choosing the right layer, role boundary, protocol family, packaging style, configuration surface, and diagnostic surface for a real system concern. Good judgment uses those boundaries explicitly, so that protocol meaning, operational policy, durable state, and system structure remain clear rather than collapsing into one giant custom abstraction.

What to remember

- Architectural judgment means choosing the right layer and boundary for the actual concern.
- A category mistake happens when a concern is placed in a layer that cannot properly own it.
- Choose the lower communication family that matches the system boundary.
- Choose the protocol surface by asking what kind of conversation the boundary wants to have.
- The simplest appropriate stack is not always the lowest stack.
- Native protocols and composed protocols solve different boundary problems.

35.20 Closing perspective

Architectural judgment uses the existing protocol, role, configuration, deployment, diagnostic, and testing vocabulary to place each concern at the boundary that can own it. Chapter 36 turns from choosing boundaries to evolving them safely.

Chapter 36

Extending the Framework Safely

36.1 Why this chapter follows architectural judgment

Chapter 35 asked where behavior should live: lower family, transport layer, connection context, middleware, WebSocket subprotocol, MQTT, configuration, deployment role, or application/domain code.

This chapter asks the next question:

After the right boundary has been chosen,
how can the framework or application be extended without damaging that boundary?

That question matters because SNode.C is not static in practice. New applications need new protocol behavior, middleware, configuration sections, carriers, build targets, diagnostics, and sometimes reusable framework components. The danger is not extension itself, but extension that makes the architecture less clear.

```
unsafe extension:
  add code where it is easiest to reach today
  hide the real boundary
  make the next change harder to place cleanly

safe extension:
  identify the boundary first
  extend the smallest appropriate layer
  keep configuration, diagnostics, build targets, and tests aligned
```

This chapter does not introduce a new API. It teaches how to grow a SNode.C codebase without turning it into an accidental framework fork.

36.2 What safe extension means

An extension is safe when it preserves four kinds of clarity.

First, it preserves **layer clarity**. Lower communication families, stream transport, connection handling, protocol parsing, application semantics, configuration, and deployment should not collapse into one large callback.

Second, it preserves **ownership clarity**. A connection-local concern should not become global state. A domain rule should not be hidden in a socket handler. A persistence decision should not be smuggled into a parser. A retry policy should not accidentally become part of a protocol grammar.

Third, it preserves **operational clarity**. A new behavior should remain visible in configuration, logs, diagnostics, build targets, package dependencies, or tests if operators and maintainers need to reason about it.

Fourth, it preserves **evolution clarity**. The next reasonable change should still have a place to go.

```
safe extension:
  the new behavior has a clear owner
  the owner is at the right layer
  the layer remains testable
  the build target still describes the architecture
  diagnostics still name the relevant role or boundary
```

This is stricter than compilation.

36.3 Start application-local unless the boundary is reusable

A common mistake is moving behavior into the framework too early. A concern appears in one application, and the developer immediately creates a generic abstraction. SNode.C encourages reusable layers, but not every application concern belongs in SNode.C itself.

A useful rule is:

```
first occurrence:
  keep the concern application-local unless the boundary is already clear

second or third similar occurrence:
  compare the shape of the variation

stable repeated boundary:
  consider a reusable component
```

Application-local code is often the right place for domain behavior, project-specific policy, business rules, device assumptions, and deployment orchestration.

Framework code should represent behavior reusable at the same architectural level: a new network family, reusable protocol layer, generic middleware, or shared configuration mechanism. A greenhouse threshold rule, warehouse device policy, or project-specific MQTT topic convention usually does not.

```
application-local:
  domain rule
  project-specific policy
  one deployment's orchestration
  device model
  customer-specific state transition

framework-level:
  reusable carrier
  reusable protocol layer
  reusable middleware concept
  reusable configuration surface
  reusable diagnostic boundary
```

The framework should make application architecture possible, not absorb every application architecture.

36.4 Three extension levels

Most SNode.C extensions fit one of three levels.

```
application extension:
  new code in one application or product

reusable library extension:
  shared code used by several applications

framework extension:
```

new `SNode.C` component, layer, or public surface

The safest path is to start application-local, extract to a reusable application library when repetition becomes real, and move into the framework only when the abstraction is stable and general enough.

That path avoids two opposite mistakes.

The first mistake is **framework pollution**: domain-specific code becomes part of the framework and forces future users to carry assumptions they did not choose.

The second mistake is **application sprawl**: the same layer-shaped behavior is copied into several applications without a shared boundary.

Reuse is good when the reused boundary fits the concern.

36.5 Extending with a new `SocketContext`

A new `SocketContext` is appropriate when the extension is connection-local protocol endpoint behavior.

That means the concern belongs to one concrete connection: reading and writing protocol data, owning connection-local parsing or state, and reacting to lifecycle.

A `SocketContext` is not the right place for every concern that happens after a connection exists.

belongs near `SocketContext`:

- connection-local protocol parsing
- connection-local protocol state
- per-peer handshake behavior
- per-peer message dispatch
- connection-local cleanup

usually does not belong there:

- global orchestration
- durable application state
- database ownership policy
- service supervision
- cross-role retry policy
- user-interface routing unrelated to the protocol endpoint

A connection context may know the connection. It should not become the whole application.

A safe `SocketContext` extension should therefore have a small statement of responsibility:

This context owns the protocol endpoint behavior for one connection.
It does not own the whole system role.

If that sentence feels false, the extension probably belongs somewhere else.

36.6 Extending with a new `SocketContextFactory`

A `SocketContextFactory` is appropriate when the extension is about constructing the correct context.

The factory associates a connection with the correct protocol endpoint object; it is not an allocation hook alone.

In `MQTTSuite`, the MQTT CLI factory retrieves configuration sections and creates an `iot::mqtt::SocketContext` with a client-side MQTT protocol object. That keeps construction policy separate from both the MQTT protocol object and the socket client role.

A safe factory extension answers questions such as:

Which context type belongs to this connection?
 Which configuration sections are needed to construct it?
 Which application/protocol object should be attached?
 Which ownership relationship begins here?

A factory should not become a global service locator or hide unrelated application dependencies.

The warning sign is a factory that has to know too much:

```
bad factory smell:
  creates the protocol context
  opens a database
  registers web routes
  starts timers
  publishes unrelated messages
  applies global deployment policy
```

At that point, the factory has become an accidental orchestrator.

36.7 Extending with middleware or routers

Middleware and routers are the right extension point for web-application flow: request logging, preprocessing, authentication or authorization, content negotiation, route grouping, static assets, structured REST-like endpoints, and application-specific HTTP behavior.

Middleware is not a general-purpose place for every cross-cutting concern. A concern is suitable for middleware when it is naturally expressed as part of request/response flow.

```
middleware/router concern:
  inspect request
  decide whether to continue
  modify response behavior
  group route logic
  apply web-facing policy

not automatically middleware:
  MQTT session state
  database schema ownership
  low-level socket retry policy
  device-specific domain rules
  cross-process supervision
```

Middleware is attractive, especially when the application already has a web surface. SNode.C's Express-like layer is powerful because it keeps web flow explicit, not because it should swallow the system.

36.8 Extending with a WebSocket subprotocol

A WebSocket subprotocol is appropriate for bidirectional WebSocket message exchange with semantics above the carrier.

The HTTP layer negotiates the upgrade, WebSocket provides the carrier, and the subprotocol gives the stream protocol-specific meaning. That is the architectural position used by MQTT-over-WebSocket.

```
HTTP:
  upgrade negotiation

WebSocket:
  upgraded bidirectional message carrier

subprotocol:
```

meaning of the WebSocket messages

A safe subprotocol extension should not reimplement HTTP, pretend WebSocket frames are raw TCP bytes, or hide application semantics so deeply that diagnostics cannot identify the subprotocol.

A good subprotocol extension keeps these questions visible:

```
Which WebSocket messages are valid?
Which state belongs to the subprotocol?
Which close/error behavior belongs to the subprotocol?
Which diagnostics identify the selected subprotocol?
```

If the extension is merely an HTTP route, use an HTTP route. If it is bidirectional live interaction with its own message rules, a WebSocket subprotocol may be right.

36.9 Extending MQTT behavior

MQTT extensions need discipline because MQTT already owns connection setup, sessions, topics, subscriptions, QoS flow, keep-alive, acknowledgements, and disconnect behavior. Application semantics can sit above MQTT; they should not casually change MQTT itself.

A safe MQTT application extension usually looks like this:

```
MQTT core:
  packet/session/topic semantics

application MQTT object:
  what to publish
  what to subscribe to
  how to react to publishes
  how to expose application state
```

MQTTSuite's `mqttcli` follows this pattern: a client-side MQTT class derives from the `SNode.C` MQTT client class, overrides semantic callbacks, and uses `SNode.C`'s send operations rather than rewriting packet transport.

That is the right direction.

```
safe MQTT extension:
  use MQTT callbacks for application reaction
  use sendConnect / sendSubscribe / sendPublish / sendDisconnect
  keep topic policy explicit
  keep session behavior configurable

unsafe MQTT extension:
  bypass packet/session logic randomly
  hide topic mapping inside transport code
  treat MQTT as a generic string pipe
  merge broker/client/application roles without naming the boundary
```

The boundary remains: MQTT owns MQTT semantics; the application owns application meaning.

36.10 Extending configuration

Configuration is often where an extension becomes operationally real. A feature that cannot be configured, shown, disabled, logged, or reproduced may work locally but fail as a maintainable system feature.

`SNode.C` uses named configured instances and subcommands to make role-specific options visible. An extension should follow that discipline instead of inventing a parallel configuration universe.

A safe configuration extension asks:

```

Is this option a build-time choice?
Is this option a runtime role choice?
Is this option a protocol/session choice?
Is this option an application/domain choice?
Is this option safe to make configurable at all?

```

Not every value should be configurable. Protocol invariants should remain code; deployment variation usually belongs in configuration; operator-adjustable domain policy may be configuration; security-sensitive choices need care around secrets, logging, and generated files.

```

code:
  invariant behavior
  protocol truth
  type-level composition

configuration:
  deployment address
  role enablement
  retry policy
  topic names
  operator-adjustable behavior

```

A configurable invariant is not flexibility. It is often an unprotected design error.

36.11 Extending the build and component surface

A reusable extension should eventually be visible in the build. As Chapter 32 showed, target names describe component meaning.

If an extension becomes reusable, it needs a component surface and a public include surface that tell the truth.

```

bad target name:
  misc-network-stuff
  common-utils2
  app-helper

better target name:
  mqtt-client-websocket
  http-server-express-legacy-in
  net-in-stream-tls

```

The target should say what architectural role the component plays and own its dependencies directly. The public header should say what source-facing abstraction the extension exposes. A component that needs HTTP or a selected carrier should declare that fact; a public front-door header should include or export the lower public declarations needed by the abstraction. A consumer should not have to know private implementation dependencies or private header order.

For framework-level extensions, the build and package consequences matter:

```

new framework component
-> public front-door header
-> target name
-> dependency visibility
-> exported target
-> package component
-> optional dependency behavior
-> installed runtime layout if needed

```

If that chain is not considered, the extension may compile in-tree but fail as part of the installed framework.

36.12 Extending diagnostics

A new feature should be diagnosable at the boundary it introduces. That means preserving the right vocabulary, not flooding logs.

A useful diagnostic message should usually make at least some of these visible:

```
role name
configured instance name
connection name
endpoint address
protocol phase
state transition
reason for failure
retry/degraded/shutdown decision
```

A poor diagnostic message says only:

```
error
```

or:

```
failed
```

A better diagnostic message says which boundary failed.

```
mqtt-uplink: broker connection failed: reconnect scheduled
admin-http: listening on 0.0.0.0:8080
live-events: observer disconnected: output no longer connected
measurement-store: database unavailable: entering degraded mode
```

Diagnostics are part of extension safety because they let maintainers reason about the system after deployment.

36.13 Extending failure policy

Failure policy should be deliberate. A low-level socket may detect an error, but the role that owns the boundary usually owns the policy response: retry, reconnect, disablement, shutdown, or degraded behavior.

```
socket detects:
  connection closed
  write failed
  timeout occurred

role decides:
  retry
  reconnect
  disable
  report degraded state
  shut down
```

A safe extension should describe failure policy together with the happy path.

If the extension introduces an output buffer, it needs a bounded policy. If it introduces a retry loop, it needs a limit, backoff, or operator-visible state. If it introduces persistence, it needs a degraded mode when durable state is unavailable. If it introduces live observers, it needs a policy for slow observers.

```
new behavior without failure policy:
  demonstration code
```



```
new behavior with visible failure policy:
    maintainable system feature
```

36.14 Extending tests with the same boundary vocabulary

Chapter 34 used one question:

Which SNode.C boundary does this test protect?

That question should follow every extension.

A new `SocketContext` needs tests for protocol endpoint behavior; middleware needs request/response tests; a WebSocket subprotocol needs upgrade, frame/message, close, and invalid-input tests; MQTT application behavior needs session, topic, publish, and reconnect tests; a build component needs an installed-consumer test, including a test that includes its public front-door header. The extension is complete only when its boundary can be tested, debugged, and explained.

Extension checklist

- What boundary was added?
- What behavior belongs to it?
- What behavior must not belong to it?
- How is it configured?
- How is it diagnosed?
- How is it built and installed?
- What test protects it?

36.15 Avoiding framework pollution

Framework pollution happens when project-specific behavior enters the framework merely because the framework was the easiest place to modify.

Framework-pollution warning

Do not extend the framework by smuggling application policy into lower layers. Keep project-specific behavior at the application boundary unless the reusable boundary is real.

Typical symptoms include:

```
framework class knows one product's topic names
transport layer knows a domain rule
socket context opens a project-specific database table
generic middleware knows one deployment's user model
core build target depends on an application library
```

Framework pollution makes the next application inherit assumptions from the previous one. The cure is not to avoid reuse, but to reuse only the right boundary.

A project-specific rule may be shared by several executables in that project. That suggests a project library, not a framework component. A generic MQTT mapping mechanism may be shared by many applications. That may justify reusable support. A new lower communication family belongs in the framework only if it really fits the lower-family abstraction and can carry several upper layers without becoming application-specific.

36.16 Avoiding abstraction too early

The opposite danger is abstraction too early. Two similar fragments become a generic layer; later the uses diverge, and the layer fills with flags, callbacks, special cases, and conditional behavior.

A useful warning sign is this:

```
one abstraction
-> too many unrelated reasons for change
```

In SNode.C terms, this may appear as:

```
one context that supports several unrelated protocols
one middleware that performs routing, authentication, persistence, and MQTT publication
one build target that hides several incompatible carrier assumptions
one configuration section that mixes deployment, protocol, and domain policy
```

A safe extension preserves meaning first. Generality can emerge when the repeated boundary is real.

Extension rule

Extend at the boundary whose responsibility actually changes.

36.17 Review questions for a proposed extension

Before adding a reusable extension to a SNode.C application or to the framework itself, ask these questions.

1. What concern is being extended?
2. Which boundary should own that concern?
3. Is the concern connection-local, role-local, application-local, or framework-wide?
4. Does it belong in a context, factory, middleware, router, subprotocol, MQTT object, configuration section, or separate service?
5. What existing layer must not be polluted by it?
6. What name will make the boundary visible in code and build targets?
7. What configuration makes it reproducible?
8. What diagnostics make it observable?
9. What failure policy makes it operable?
10. What test protects it?
11. What deployment or package consequence follows from it?
12. What future change would this design make easier or harder?

The questions prevent an extension from becoming accidental architecture.

What to remember

- Safe extension starts with the boundary, not with the easiest file to edit.
- Application-local code is not inferior; it is often the right home for domain behavior.
- Framework-level extensions should represent reusable framework boundaries, not one project's policy.
- A new `SocketContext` is appropriate for connection-local protocol endpoint behavior.
- A new `SocketContextFactory` should construct the right context, not become a service locator.
- Middleware and routers are for web-application flow, not every cross-cutting concern.

36.18 From safe extension to a guided project

The next part applies these rules to MiniGateway, a small application with HTTP administration, SSE observation, MQTT integration, configuration, failure policy, optional persistence, and deployment shape. It shows how to extend an application without losing the framework's layered vocabulary.

Part XII

Building a MiniGateway Application

This final technical part turns the book's architectural vocabulary into a compact guided project. MiniGateway is deliberately small, but it combines HTTP administration, SSE observation, MQTT integration, application-owned state, configuration, diagnostics, failure policy, and deployment thinking in one coherent example.

The goal is not to create a second MQTTSuite. The goal is to show how a reader can assemble SNode.C concepts into a small but real application without losing the layer and boundary discipline developed throughout the book.

Chapter 37

Building MiniGateway

37.1 Why this chapter exists

The previous chapters taught SNode.C in layers: lower communication families, stream connections, protocol contexts, web protocols, MQTT, configuration, deployment, testing, and architectural judgment. MiniGateway is the point where those ideas stop being separate topics and become one small application.

MiniGateway is modest. It is not a second MQTTSuite. It is not a broker, a dashboard product, or a hardware driver. It is a compact SNode.C application that owns one piece of domain state and exposes that state through several communication boundaries.

The application keeps the latest environmental measurement in memory. A measurement contains temperature, humidity, voltage, a sequence number, and a timestamp. Whenever a new measurement enters the application, MiniGateway performs one internal state transition and then fans the result out to the outward-facing roles.

```
new measurement
  -> update current MeasurementState
  -> publish through MeasurementBus
      -> notify SSE observers
      -> ask MQTT integration to publish, if connected
```

This is the core of the project. Everything else in the chapter exists to keep this path honest.

37.2 What the application does

The base version built in this chapter has two outward-facing communication roles.

The first role is an HTTP/Express role. It is the browser-friendly and command-line-friendly observation surface. It exposes four paths:

```
GET /health
  -> report that the process is alive

GET /status
  -> return the latest measurement as JSON

GET /events
  -> keep an SSE connection open and push measurements as they arrive

GET /simulate
  -> create one synthetic measurement and inject it into the normal application path
```

The second role is a native MQTT client role named `mqtt-uplink`. It connects to an MQTT broker, starts an MQTT session, subscribes to a command topic, and publishes measurements to a measurement topic.

In the base version, incoming MQTT commands are logged only. That is deliberate. The chapter shows an incoming MQTT boundary without turning the guided project into a control system.

The base version does not poll measurements. SNode.C runs the application in its event-driven runtime model, and MiniGateway keeps the project consistent with that model. A new measurement is handled when a new measurement event appears.

MiniGateway role checklist

- HTTP administration role
- SSE observation path
- MQTT-uplink role
- shared application state
- configuration
- runtime startup

The base version creates such events through `/simulate`. That route is not the final device interface. It is a controlled teaching input. Chapter 38 replaces this with a small Unix-domain socket input to demonstrate how the application can grow without changing the HTTP, SSE, MQTT, or domain-state structure.

37.3 How to use the base version

The first useful command for an SNode.C application is still the generated help output:

```
./minigateway --help
```

The concrete configuration tree depends on the selected SNode.C components, but the source below sets the HTTP listener to port 8080 and the MQTT remote port to 1883 by default. Start the application, then inspect the HTTP surface:

```
curl http://localhost:8080/health
curl http://localhost:8080/status
```

At startup, `/status` returns the default measurement because no measurement has entered the application yet. Create one measurement:

```
curl http://localhost:8080/simulate
curl http://localhost:8080/status
```

To watch live events, keep one terminal connected to the SSE endpoint:

```
curl -N -H 'Accept: text/event-stream' http://localhost:8080/events
```

Then trigger another measurement from a second terminal:

```
curl http://localhost:8080/simulate
```

The SSE terminal should receive an event. If an MQTT broker is reachable and the MQTT role is connected, the same measurement is also published through MQTT.

Port ownership warning

Before testing `/status`, verify that MiniGateway really owns the configured HTTP port. If another SNode.C application, for example MQTTBroker, is already listening on the same port, `curl` may reach the wrong process.

A simple check is:

```
ss -ltnp 'sport = :8080'
```

This is not a MiniGateway-specific trick. It is part of testing deployed communication roles honestly.

37.4 The shape of the base application

MiniGateway has a small source tree, but the files are separated by responsibility:

```
MiniGateway/
  CMakeLists.txt
  main.cpp
  Measurement.h
  MeasurementState.h/.cpp
  MeasurementBus.h/.cpp
  ConfigSections.h/.cpp
  MiniGatewayMqtt.h/.cpp
  MiniGatewaySocketContextFactory.h/.cpp
  README-BUILD.md
```

That split is part of the lesson. The domain object is not an HTTP route. The state holder is not an MQTT protocol object. The bus is not SSE. The MQTT object does not own the HTTP responses. The application function in `main.cpp` wires these boundaries together, but it does not erase them.

```
domain fact
  -> Measurement

current state
  -> MeasurementState

state-change distribution
  -> MeasurementBus

MQTT session behavior
  -> MiniGatewayMqtt

MQTT context construction
  -> MiniGatewaySocketContextFactory

role assembly
  -> main.cpp
```

A one-file example would be shorter, but it would teach the wrong reflex. This chapter wants the reader to see how a small SNode.C application is assembled from visible boundaries. Chapter 38 keeps the same split visible when it adds the Unix-domain measurement input. That extension is introduced as another SNode.C communication role, not as behavior hidden inside the HTTP routes, the SSE response path, or the MQTT client object.

37.5 Stage 1: the build target

MiniGateway is an external SNode.C consumer. It therefore uses installed package targets and installed public headers, not private in-tree targets or private implementation headers. The selected components already say a lot about the application:

```
http-server-express-legacy-in
  -> HTTP/Express role over IPv4 legacy stream

net-in-stream-legacy
  -> native IPv4 stream carrier for the MQTT client
```

```
mqtt-client
-> MQTT client-side protocol support
```

The build file does not manually list every lower library that those components need. SNode.C's exported targets carry that dependency information. That is the same component discipline discussed in Chapter 32, now used by a small application.

The source files follow the matching include discipline. `main.cpp` includes the public Express header because it directly creates the HTTP/Express role, the public IPv4 legacy socket-client header because it directly creates the MQTT carrier client, and MQTT headers where the code directly names MQTT abstractions. It does not include every lower `http`, `core/socket`, or `net` header behind those front doors.

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.14)

project(MiniGateway LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)

find_package(nlohmann_json 3.7.0 REQUIRED)

find_package(
    snodec 1.0.0 REQUIRED COMPONENTS http-server-express-legacy-in
                                     net-in-stream-legacy mqtt-client
)

add_executable(
    minigateway
    main.cpp
    MeasurementState.cpp
    MeasurementBus.cpp
    MiniGatewayMqtt.cpp
    MiniGatewaySocketContextFactory.cpp
    ConfigSections.cpp
    MeasurementState.h
    MeasurementBus.h
    MiniGatewayMqtt.h
    MiniGatewaySocketContextFactory.h
    ConfigSections.h
)

target_link_libraries(
    minigateway
    PRIVATE snodec::http-server-express-legacy-in snodec::net-in-stream-legacy
            snodec::mqtt-client nlohmann_json::nlohmann_json
)
```

37.6 Stage 2: the domain fact

The first application type is `Measurement`. It is deliberately plain. The domain value has no knowledge of HTTP, SSE, MQTT, sockets, configuration, or deployment. It is the fact that the application owns and exposes.

The JSON conversion lives with the measurement because this guided project needs a compact representation for `/status`, SSE payloads, and MQTT publications. In a larger application, the representation layer might be separated further. For MiniGateway, keeping the conversion next to the small value type is clear enough and keeps the example readable.

Measurement.h

```
#pragma once

#include <chrono>
#include <cstdint>
#include <string>
#include <nlohmann/json.hpp>

namespace minigateway {

    struct Measurement {
        double temperature = 0.0;
        double humidity = 0.0;
        double voltage = 0.0;
        std::uint64_t sequence = 0;
        std::chrono::system_clock::time_point timestamp = std::chrono::system_clock::now();
    };

    inline nlohmann::json toJson(const Measurement& measurement) {
        return {
            {"temperature", measurement.temperature},
            {"humidity", measurement.humidity},
            {"voltage", measurement.voltage},
            {"sequence", measurement.sequence}
        };
    }

    inline std::string toPayload(const Measurement& measurement) {
        return toJson(measurement).dump();
    }

} // namespace minigateway
```

37.7 Stage 3: the current state

MeasurementState holds the latest accepted measurement. It is not a cache for HTTP and it is not an MQTT buffer. It is the application-local state owner.

The state object is deliberately plain. MiniGateway is written for the SNode.C single-event-loop execution model used throughout the book, so the base application keeps state ownership simple and explicit.

MeasurementState.h

```
#pragma once

#include "Measurement.h"

namespace minigateway {

    class MeasurementState {
    public:
        Measurement current() const;
        void update(Measurement measurement);

    private:
        Measurement measurement;
    };

} // namespace minigateway
```


MeasurementState.cpp

```

#include "MeasurementState.h"

#include <utility>

namespace minigateway {

    Measurement MeasurementState::current() const {
        return measurement;
    }

    void MeasurementState::update(Measurement newMeasurement) {
        measurement = std::move(newMeasurement);
    }

} // namespace minigateway

```

37.8 Stage 4: the internal measurement bus

The bus is the smallest useful separation between accepting a measurement and deciding which outward-facing role should react to it. It keeps the measurement path event-driven.

```

accept measurement
  -> update state
  -> publish to bus
      -> MQTT subscriber
      -> SSE subscribers

```

The listener return value is a small but important design choice. A listener that returns `false` is removed. The SSE route uses this to remove observers whose HTTP response is no longer connected. This is lifetime management inside the event-driven application.

MeasurementBus.h

```

#pragma once

#include "Measurement.h"

#include <functional>
#include <list>

namespace minigateway {

    class MeasurementBus {
    public:
        using Listener = std::function<bool(const Measurement&)>;

        void subscribe(Listener listener);
        void publish(const Measurement& measurement);

    private:
        std::list<Listener> listeners;
    };

} // namespace minigateway

```

MeasurementBus.cpp

```

#include "MeasurementBus.h"

#include <utility>

namespace minigateway {

    void MeasurementBus::subscribe(Listener listener) {
        listeners.push_back(std::move(listener));
    }

    void MeasurementBus::publish(const Measurement& measurement) {
        for (auto listenerIt = listeners.begin(); listenerIt != listeners.end(); ) {
            if ((*listenerIt)(measurement)) {
                ++listenerIt;
            } else {
                listenerIt = listeners.erase(listenerIt);
            }
        }
    }

} // namespace minigateway

```

37.9 Stage 5: MiniGateway-specific MQTT configuration

The MQTT client role needs application-specific MQTT settings: client id, keep-alive, command topic, measurement topic, QoS, and retain behavior. These are not hard-coded inside `MiniGatewayMqtt`. They belong to the configured communication role.

The shape follows the same general pattern as `MQTTSuite`'s command-line/configuration sections: the application adds a subcommand to the configured instance, exposes configurable options, and the protocol object reads the effective values when the connection context is constructed.

ConfigSections.h

```

#pragma once

#include <cstdint>
#include <string>
#include <string_view>
#include <utils/SubCommand.h>

namespace minigateway {

    class ConfigMqtt : public utils::SubCommand {
    public:
        constexpr static std::string_view NAME{"mqtt"};
        constexpr static std::string_view DESCRIPTION{"MiniGateway MQTT integration"};

        explicit ConfigMqtt(utils::SubCommand* parent);
        ~ConfigMqtt() override;

        std::string getClientId() const;
        std::uint16_t getKeepAlive() const;
        std::string getCommandTopic() const;
        std::string getMeasurementTopic() const;
        std::uint8_t getQoS() const;
        bool getRetain() const;

    private:
        CLI::Option* clientIdOpt;
        CLI::Option* keepAliveOpt;
    };
}

```

```

        CLI::Option* commandTopicOpt;
        CLI::Option* measurementTopicOpt;
        CLI::Option* qosOpt;
        CLI::Option* retainOpt;
    };

} // namespace minigateway

```

ConfigSections.cpp

```

#include "ConfigSections.h"

namespace minigateway {

    ConfigMqtt::ConfigMqtt(utils::SubCommand* parent)
        : utils::SubCommand(parent, this, "MiniGateway")
        , clientIdOpt(
            setConfigurable(addOption("--client-id", "MQTT Client-ID", "string", "minigateway"
, CLI::TypeValidator<std::string>(), true))
            , keepAliveOpt(setConfigurable(
                addOption("--keep-alive", "MQTT keep-alive in seconds", "uint16_t", "30", CLI::
TypeValidator<std::uint16_t>(), true))
            , commandTopicOpt(setConfigurable(
                addOption("--command-topic", "MQTT command topic", "string", "minigateway/command"
, CLI::TypeValidator<std::string>(), true))
            , measurementTopicOpt(setConfigurable(
                addOption(
                    "--measurement-topic", "MQTT measurement topic", "string", "minigateway/
measurement", CLI::TypeValidator<std::string>(),
                    true))
            , qosOpt(setConfigurable(
                addOption("--qos", "MQTT QoS for MiniGateway publications and subscriptions", "
uint8_t", "0", CLI::Range(0, 2)), true))
            , retainOpt(setConfigurable(
                addFlag("--retain{true}", "Retain measurement publications", "bool", "false", CLI
::IsMember({"true", "false"})), true)) {
    }

    ConfigMqtt::~ConfigMqtt() = default;

    std::string ConfigMqtt::getClientId() const {
        return clientIdOpt->as<std::string>();
    }

    std::uint16_t ConfigMqtt::getKeepAlive() const {
        return keepAliveOpt->as<std::uint16_t>();
    }

    std::string ConfigMqtt::getCommandTopic() const {
        return commandTopicOpt->as<std::string>();
    }

    std::string ConfigMqtt::getMeasurementTopic() const {
        return measurementTopicOpt->as<std::string>();
    }

    std::uint8_t ConfigMqtt::getQoS() const {
        return qosOpt->as<std::uint8_t>();
    }

    bool ConfigMqtt::getRetain() const {
        return retainOpt->as<bool>();
    }
}

```

```
} // namespace minigateway
```

37.10 Stage 6: the MQTT protocol object

`MiniGatewayMqtt` is the MQTT client-side protocol object for this application. It owns MQTT session behavior, not domain state. On connection, it sends `CONNECT`. After a successful `CONNACK`, it subscribes to the command topic and announces a small status message. When a measurement is available, it publishes the measurement payload if the MQTT session is connected.

The static client list is a small guided-project convenience. It gives the application a simple way to publish a measurement to the currently connected MQTT protocol objects without making `MeasurementBus` know anything about MQTT. A production system might wrap this in a more explicit integration service. In this chapter, the static list keeps the source small while preserving the important boundary: the bus still distributes measurements, and the MQTT object still owns MQTT publication.

Incoming command messages are logged only. This keeps the incoming MQTT boundary visible while avoiding a new command/domain policy in the base project.

MiniGatewayMqtt.h

```
#pragma once

#include "Measurement.h"

#include <stdint>
#include <iot/mqtt/client/Mqtt.h>
#include <iot/mqtt/packets/Connack.h>
#include <iot/mqtt/packets/Publish.h>
#include <string>
#include <vector>

namespace minigateway {

    class MiniGatewayMqtt : public iot::mqtt::client::Mqtt {
    public:
        MiniGatewayMqtt(const std::string& connectionName,
                        const std::string& clientId,
                        std::uint16_t keepAlive,
                        std::string commandTopic,
                        std::string measurementTopic,
                        std::uint8_t qos,
                        bool retain);

        ~MiniGatewayMqtt() override;

        static void publishMeasurementToConnected(const Measurement& measurement);

    private:
        using Super = iot::mqtt::client::Mqtt;

        void onConnected() final;
        void onDisconnected() final;
        [[nodiscard]] bool onSignal(int signum) final;

        void onConnack(const iot::mqtt::packets::Connack& connack) final;
        void onPublish(const iot::mqtt::packets::Publish& publish) final;

        void publishMeasurement(const Measurement& measurement) const;

        static std::vector<MiniGatewayMqtt*> clients;
```

```

        bool connected = false;
        const std::string commandTopic;
        const std::string measurementTopic;
        const std::uint8_t qoS;
        const bool retain;
    };

} // namespace minigateway

```

MiniGatewayMqtt.cpp

```

#include "MiniGatewayMqtt.h"

#include <algorithm>
#include <iot/mqtt/Topic.h>
#include <list>
#include <log/Logger.h>
#include <utility>
#include <utils/system/signal.h>

namespace minigateway {

    std::vector<MiniGatewayMqtt*> MiniGatewayMqtt::clients;

    MiniGatewayMqtt::MiniGatewayMqtt(const std::string& connectionName,
                                     const std::string& clientId,
                                     std::uint16_t keepAlive,
                                     std::string commandTopic,
                                     std::string measurementTopic,
                                     std::uint8_t qoS,
                                     bool retain)
        : iot::mqtt::client::Mqtt(connectionName, clientId, keepAlive, "")
        , commandTopic(std::move(commandTopic))
        , measurementTopic(std::move(measurementTopic))
        , qoS(qoS)
        , retain(retain) {
        clients.push_back(this);
    }

    MiniGatewayMqtt::~MiniGatewayMqtt() {
        clients.erase(std::remove(clients.begin(), clients.end(), this), clients.end());
    }

    void MiniGatewayMqtt::onConnected() {
        VLOG(1) << "MQTT: initiating session";

        sendConnect(true, "", "", 0, false, "", "");
    }

    void MiniGatewayMqtt::onDisconnected() {
        connected = false;

        Super::onDisconnected();
    }

    bool MiniGatewayMqtt::onSignal(int signum) {
        VLOG(1) << "MQTT: exit due to signal " << signum << " (SIG" << utils::system::
sigabbrev_np(signum) << ")";

        sendDisconnect();

        return Super::onSignal(signum);
    }

```

```

}

void MiniGatewayMqtt::onConnack(const iot::mqtt::packets::Connack& connack) {
    if (connack.getReturnCode() == 0) {
        connected = true;

        sendSubscribe(std::list<iot::mqtt::Topic>{iot::mqtt::Topic(commandTopic, qoS)});
        sendPublish(measurementTopic, R"json({"status":"connected"})json", qoS, retain);
    } else {
        connected = false;
        sendDisconnect();
    }
}

void MiniGatewayMqtt::onPublish(const iot::mqtt::packets::Publish& publish) {
    VLOG(1) << "MQTT command on " << publish.getTopic() << ": " << publish.getMessage();
}

void MiniGatewayMqtt::publishMeasurement(const Measurement& measurement) const {
    if (connected) {
        sendPublish(measurementTopic, toPayload(measurement), qoS, retain);
    }
}

void MiniGatewayMqtt::publishMeasurementToConnected(const Measurement& measurement) {
    const std::vector<MiniGatewayMqtt*> currentClients = clients;

    for (MiniGatewayMqtt* client : currentClients) {
        if (client != nullptr) {
            client->publishMeasurement(measurement);
        }
    }
}

} // namespace minigateway

```

37.11 Stage 7: constructing MQTT contexts

SNode.C's MQTT layer is used through a socket context. The factory creates an `iot::mqtt::SocketContext` and passes it a `MiniGatewayMqtt` protocol object. This is the same shape used by MQTTSuite-style MQTT applications:

```

stream SocketConnection
-> SocketContextFactory
  -> iot::mqtt::SocketContext
    -> MiniGatewayMqtt

```

The factory reads the MiniGateway MQTT configuration from the configured connection instance. That keeps configuration on the role side and protocol behavior on the protocol-object side.

MiniGatewaySocketContextFactory.h

```

#pragma once

#include <core/socket/stream/SocketContextFactory.h>

namespace minigateway {

    class MiniGatewaySocketContextFactory : public core::socket::stream::SocketContextFactory {
    private:

```

```

        core::socket::stream::SocketContext* create(core::socket::stream::SocketConnection*
        socketConnection) final;
    };

} // namespace minigateway

```

MiniGatewaySocketContextFactory.cpp

```

#include "MiniGatewaySocketContextFactory.h"

#include "ConfigSections.h"
#include "MiniGatewayMqtt.h"

#include <core/socket/stream/SocketConnection.h>
#include <iot/mqtt/SocketContext.h>
#include <net/config/ConfigInstance.h>

namespace minigateway {

    core::socket::stream::SocketContext* MiniGatewaySocketContextFactory::create(core::socket::
    stream::SocketConnection* socketConnection) {
        const ConfigMqtt* configMqtt = socketConnection->getConfigInstance()->getSubCommand<
        ConfigMqtt>();

        return new iot::mqtt::SocketContext(socketConnection,
                                            new MiniGatewayMqtt(socketConnection->
        getConnectionName(),
                                                                configMqtt->getClientId(),
                                                                configMqtt->getKeepAlive(),
                                                                configMqtt->getCommandTopic(),
                                                                configMqtt->getMeasurementTopic
        ()),
                                            configMqtt->getQoS(),
                                            configMqtt->getRetain()));
    }

} // namespace minigateway

```

37.12 Stage 8: assembling the runtime roles

The main file is where the small application becomes a running SNode.C program. It performs five tasks:

1. initialize SNode.C,
2. create the domain state and bus,
3. define the measurement acceptance path,
4. register HTTP/SSE routes,
5. start the HTTP and MQTT roles before entering the SNode.C event loop.

The central function is not named as a separate function in the source, but it is the lambda `acceptMeasurement`:

```

acceptMeasurement(measurement)
-> state.update(measurement)
-> bus.publish(state.current())

```

Both `/simulate` and later real inputs should enter the application through that same path. This is the reason Chapter 38 can add a Unix-domain measurement input without rewriting `/status`, `/events`, or the MQTT publication logic.

The HTTP role is intentionally straightforward. `/health` reports process health. `/status` serializes the current measurement. `/events` establishes an SSE stream and subscribes that response to the bus. `/simulate` creates one new measurement and injects it.

The MQTT role is started as a native IPv4 stream client. The role name `mqtt-uplink` appears in configuration, diagnostics, and state reporting.

In the base source, `startMqttClient()` returns the application-side client handle, and `main()` does not keep that handle because the example does not later reconfigure, stop, or inspect the `mqtt-uplink` role. The call to `connect()` registers the configured runtime role; concrete `MiniGatewayMqtt` protocol objects are created later through `MiniGatewaySocketContextFactory` when actual connections exist. In an application that needs later control over the role, keeping the returned handle would be the appropriate design.

The `SNode.C` include block in `main.cpp` is an architectural map of the source-facing roles:

```
<express/legacy/in/WebApp.h>
-> HTTP/Express server role over IPv4 legacy stream

<net/in/stream/legacy/SocketClient.h>
-> native IPv4 legacy stream client used as the MQTT carrier

<iot/mqtt/client/Mqtt.h> and <iot/mqtt/SocketContext.h> in the supporting files
-> MQTT client behavior and MQTT socket-context bridge
```

These headers correspond to, but are not identical with, the CMake components selected by the target. Headers provide the C++ source-facing front doors; components provide the link-facing binary surface.

main.cpp

```
#include "ConfigSections.h"
#include "MeasurementBus.h"
#include "MeasurementState.h"
#include "MiniGatewayMqtt.h"
#include "MiniGatewaySocketContextFactory.h"

#include <chrono>
#include <core/SNodeC.h>
#include <cstdlib>
#include <express/legacy/in/WebApp.h>
#include <express/middleware/VerboseRequest.h>
#include <log/Logger.h>
#include <net/config/ConfigInstance.h>
#include <net/in/stream/legacy/SocketClient.h>
#include <string>
#include <web/http/http_utils.h>

namespace {

void reportState(const std::string& instanceName, const core::socket::SocketAddress&
socketAddress, const core::socket::State& state) {
    switch (state) {
        case core::socket::State::OK:
            VLOG(1) << instanceName << ": connected/listening on '" << socketAddress.
toString() << "'";
            break;
        case core::socket::State::DISABLED:
            VLOG(1) << instanceName << ": disabled";
            break;
        case core::socket::State::ERROR:
            LOG(ERROR) << instanceName << " " << socketAddress.toString() << ": " << state.
what();
            break;
        case core::socket::State::FATAL:
```



```

        LOG(FATAL) << instanceName << " " << socketAddress.toString() << ": " << state.
    what();
        break;
    }
}

minigateway::Measurement makeSimulatedMeasurement(std::uint64_t& sequence) {
    minigateway::Measurement measurement;
    measurement.temperature = 20.0 + static_cast<double>(sequence % 50) / 10.0;
    measurement.humidity = 40.0 + static_cast<double>(sequence % 20);
    measurement.voltage = 3.7 + static_cast<double>(sequence % 10) / 100.0;
    measurement.sequence = ++sequence;
    measurement.timestamp = std::chrono::system_clock::now();

    return measurement;
}

void createMqttConfig(net::in::stream::legacy::config::ConfigSocketClient& config) {
    config.Instance::newSubCommand<minigateway::ConfigMqtt>();
}

auto startMqttClient() {
    using Client = net::in::stream::legacy::SocketClient<minigateway::
MiniGatewaySocketContextFactory>;

    Client socketClient("mqtt-uplink");

    auto* config = socketClient.getConfig();
    config->Remote::setPort(1883);
    config->setDisableNagleAlgorithm();
    createMqttConfig(*config);

    socketClient.getConfig()->setRetry();
    socketClient.getConfig()->setRetryBase(1);
    socketClient.getConfig()->setReconnect();

    socketClient.connect([](const Client::SocketAddress& socketAddress, const core::socket::
State& state) {
        reportState("mqtt-uplink", socketAddress, state);
    });

    return socketClient;
}

template <typename ResponseT>
void sendMeasurementEvent(const minigateway::Measurement& measurement, const ResponseT& res)
{
    res->sendFragment("event: measurement");
    res->sendFragment("id:" + std::to_string(measurement.sequence));
    res->sendFragment("retry: 1000");
    res->sendFragment("data: " + minigateway::toPayload(measurement) + "\r\n");
}

} // namespace

int main(int argc, char* argv[]) {
    core::SNodeC::init(argc, argv);

    minigateway::MeasurementState state;
    minigateway::MeasurementBus bus;

    auto acceptMeasurement = [&state, &bus](minigateway::Measurement measurement) {
        state.update(measurement);
    };

```

```

        bus.publish(state.current());
    };

    bus.subscribe([] (const minigateway::Measurement& measurement) {
        minigateway::MiniGatewayMqtt::publishMeasurementToConnected(measurement);
        return true;
    });

    const express::legacy::in::WebApp app;

    app.use(express::middleware::VerboseRequest());

    app.get("/health", [] APPLICATION(req, res) {
        res->json({{"ok", true}});
    });

    app.get("/status", [&state] APPLICATION(req, res) {
        res->json(minigateway::toJson(state.current()));
    });

    app.get("/events", [&state, &bus] APPLICATION(req, res) {
        if (web::http::ciContains(req->get("Accept"), "text/event-stream")) {
            res->set("Content-Type", "text/event-stream").set("Cache-Control", "no-cache").set("
Connection", "keep-alive");
            res->sendHeader();

            const minigateway::Measurement current = state.current();
            if (current.sequence > 0) {
                sendMeasurementEvent(current, res);
            }

            bus.subscribe([res] (const minigateway::Measurement& measurement) {
                const bool keepSubscriber = res->isConnected();
                if (keepSubscriber) {
                    sendMeasurementEvent(measurement, res);
                }

                return keepSubscriber;
            });
        } else {
            res->status(406).send("SSE requires Accept: text/event-stream");
        }
    });

    std::uint64_t simulatedSequence = 0;

    app.get("/simulate", [&acceptMeasurement, &simulatedSequence] APPLICATION(req, res) {
        minigateway::Measurement measurement = makeSimulatedMeasurement(simulatedSequence);
        acceptMeasurement(measurement);

        res->json(minigateway::toJson(measurement));
    });

    app.listen(8080,
        [instanceName = app.getConfig()->getInstanceName()] (const express::legacy::in::
WebApp::SocketAddress& socketAddress,
                                                                    const core::socket::State&
listenState) {
        reportState(instanceName, socketAddress, listenState);
    });

    startMqttClient();

```

```
    return core::SNodeC::start();
}
```

37.13 Building the base source package

The source package that accompanies this chapter contains a short build note. It is simple because the chapter assumes `SNode.C` is already installed as a CMake package.

README-BUILD.md

```
# MiniGateway
```

This is the current event-driven MiniGateway source package extracted from the book project.

It contains the complete source tree used by Chapter 37:

```
- `CMakeLists.txt`
- `main.cpp`
- `Measurement.h`
- `MeasurementState.h/.cpp`
- `MeasurementBus.h/.cpp`
- `ConfigSections.h/.cpp`
- `MiniGatewayMqtt.h/.cpp`
- `MiniGatewaySocketContextFactory.h/.cpp`
```

Expected build shape, assuming `SNode.C` is installed as a CMake package:

```
```sh
cmake -S . -B build
cmake --build build
```
```

This package intentionally contains no TLS, no persistence, no MQTT-over-WebSocket, no frontend, and no real sensor input.

What to remember

- MiniGateway is a guided application, not a framework subsystem.
- The base version owns one current measurement and exposes it through HTTP, SSE, and MQTT.
- `/simulate` is a teaching input boundary; it injects a measurement into the same path that a real input source can use later.
- Measurement arrival is event-driven. The application does not poll for measurements.
- `MeasurementState` owns the current value.
- `MeasurementBus` distributes state changes without knowing about HTTP, SSE, or MQTT.

Chapter 38

Extending, Testing, and Deploying MiniGateway

38.1 Why this chapter follows the base build

Chapter 37 built the base MiniGateway: an HTTP/Express surface, an SSE observation path, a native MQTT client role, an in-memory current measurement, and `/simulate` as a controlled input boundary.

Chapter 38 shows how the application can grow without losing its shape. The concrete source extension is `MiniGateway-Extended`, which adds a Unix-domain socket input for external measurement injection. The larger topic is how SNode.C applications can be extended, tested, debugged, and deployed while keeping boundaries visible.

```
base MiniGateway
-> explicit new input boundary
    -> same domain state
        -> same MeasurementBus
            -> same HTTP / SSE / MQTT outputs
```

The extension demonstrates this part's main design rule:

MiniGateway extension rule

Add the boundary that owns the new concern.

Convenience-callback warning

Do not hide new behavior inside an arbitrary callback merely because that callback is convenient.

38.2 What changes in the extended version

The extended version adds one responsibility: measurements can now arrive through a Unix-domain stream socket. The input format is simple: one comma-separated measurement per line.

Figure 38.1 shows the extended application as an instance architecture. The HTTP/Express `WebApp`, the new `measurement-input` Unix-domain server, and the `mqtt-uplink` client are visible communication roles around one shared application core. The figure is not meant as a complete call graph. It shows which instance owns which boundary, and where the new input role joins the same `acceptMeasurement(...)`, `MeasurementState`, and `MeasurementBus` path.

```
printf '21.5,43.0,3.72
' | nc -U /tmp/minigateway-measurements.sock
```

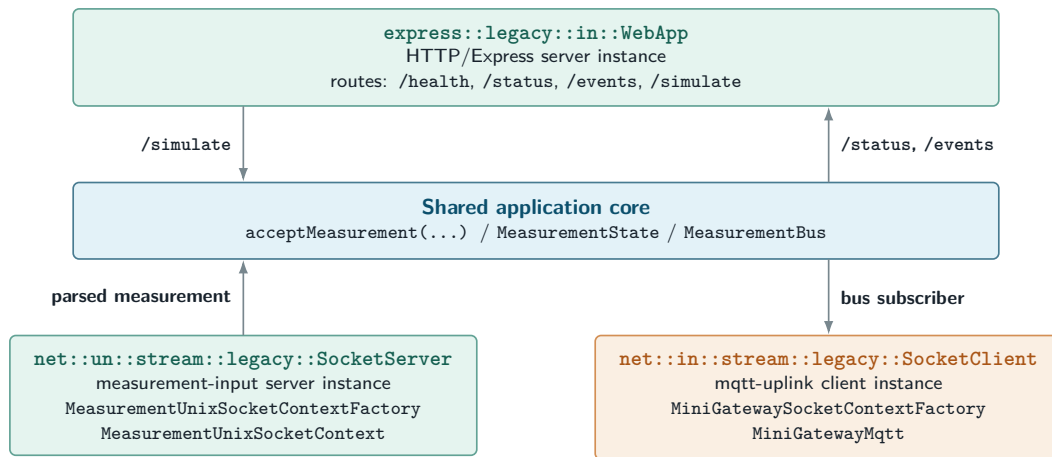


Figure 38.1: MiniGateway-Extended as SNode.C communication roles around one shared application core. The Unix-domain measurement input is a separate server instance, while HTTP routes, state, bus distribution, and MQTT publication remain separated.

The accepted fields are:

```
temperature, humidity, voltage
temperature, humidity, voltage, sequence
```

If the sender does not provide a sequence number, MiniGateway assigns the next local sequence number. After that, the measurement follows the same path as a simulated measurement:

```
Unix-domain input line
-> parse Measurement
-> acceptMeasurement(...)
    -> update MeasurementState
    -> publish through MeasurementBus
        -> SSE observers
        -> MQTT publication
```

What matters is what does not change: `/status` does not learn Unix-domain sockets, the SSE route does not parse CSV, `MiniGatewayMqtt` does not read the local socket, and the new input boundary ends at the same `acceptMeasurement` function from Chapter 37.

The ownership vocabulary also stays the same. The extended version keeps the measurement-input server handle and the MQTT client handle as named local objects in `main()` because this chapter compares the registered runtime roles. This is a source-layout choice, not a different connection model.

38.3 Extension overview

Compared with the base source tree, the extended version adds two new classes and changes three existing files:

```
new:
  MeasurementUnixSocketContext.h/.cpp
  MeasurementUnixSocketContextFactory.h/.cpp

changed:
  CMakeLists.txt
  main.cpp
  README-BUILD.md
```

The rest of the source tree remains the base application. A new input carrier should not force a rewrite of the output roles.

38.4 The build target after extension

The extended CMake file adds the Unix-domain stream component and the two new source files. The source side adds the matching Unix-domain public server header where the new concrete role is directly named.

```
base components:
  http-server-express-legacy-in
  net-in-stream-legacy
  mqtt-client

extended component:
  net-un-stream-legacy
```

That is the component-level expression of the new boundary: a Unix-domain local input role in addition to the HTTP and MQTT roles.

The include-level expression of the same boundary is:

```
#include <net/un/stream/legacy/SocketServer.h>
```

The extension therefore changes the application consistently in three places: a new public include for the source-facing role, a new C++ server instance using that role, and a new CMake component for the link-facing surface.

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.14)

project(MiniGateway LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)

find_package(nlohmann_json 3.7.0 REQUIRED)

find_package(
  snodec 1.0.0 REQUIRED COMPONENTS http-server-express-legacy-in
                                   net-in-stream-legacy net-un-stream-legacy mqtt-client
)

add_executable(
  minigateway
  main.cpp
  MeasurementState.cpp
  MeasurementBus.cpp
  MeasurementUnixSocketContext.cpp
  MeasurementUnixSocketContextFactory.cpp
  MiniGatewayMqtt.cpp
  MiniGatewaySocketContextFactory.cpp
  ConfigSections.cpp
  MeasurementState.h
  MeasurementBus.h
  MeasurementUnixSocketContext.h
  MeasurementUnixSocketContextFactory.h
  MiniGatewayMqtt.h
  MiniGatewaySocketContextFactory.h
  ConfigSections.h
)

target_link_libraries(
```

```

minigateway
PRIVATE snodec::http-server-express-legacy-in snodec::net-in-stream-legacy
        snodec::net-un-stream-legacy snodec::mqtt-client
        nlohmann_json::nlohmann_json
)

```

38.5 The Unix-domain measurement context

The new `MeasurementUnixSocketContext` owns the local input protocol. Its protocol is small: read bytes from a stream connection, collect complete lines, parse each line as a measurement, and pass accepted measurements to the application handler.

This class is not a general CSV library and not a sensor framework. It is the protocol endpoint for one local `MiniGateway` input boundary.

```

Unix stream bytes
-> line buffer
    -> CSV measurement parser
        -> MeasurementHandler

```

The context does not update `MeasurementState` directly. It does not publish MQTT messages. It does not know about SSE clients. It hands a parsed `Measurement` to the handler it received from the application assembly. That is the boundary discipline.

MeasurementUnixSocketContext.h

```

#pragma once

#include "Measurement.h"

#include <core/socket/stream/SocketContext.h>
#include <cstdint>
#include <functional>
#include <string>

namespace minigateway {

    class MeasurementUnixSocketContext : public core::socket::stream::SocketContext {
    public:
        using MeasurementHandler = std::function<void(Measurement)>;

        MeasurementUnixSocketContext(core::socket::stream::SocketConnection* socketConnection,
            MeasurementHandler measurementHandler);

    private:
        void onConnected() final;
        void onDisconnected() final;
        [[nodiscard]] bool onSignal(int signum) final;
        std::size_t onReceivedFromPeer() final;

        void processLine(const std::string& line) const;

        MeasurementHandler measurementHandler;
        std::string receiveBuffer;
    };

} // namespace minigateway

```

MeasurementUnixSocketContext.cpp

```

#include "MeasurementUnixSocketContext.h"

#include <algorithm>
#include <cctype>
#include <chrono>
#include <cmath>
#include <core/socket/SocketAddress.h>
#include <core/socket/stream/SocketConnection.h>
#include <exception>
#include <log/Logger.h>
#include <stdexcept>
#include <utility>
#include <vector>

namespace minigateway {

    namespace {

        std::string trim(std::string value) {
            value.erase(value.begin(), std::find_if(value.begin(), value.end(), [](unsigned char
ch) {
                return !std::isspace(ch);
            }));
            value.erase(std::find_if(value.rbegin(),
                value.rend(),
                [](unsigned char ch) {
                    return !std::isspace(ch);
                })
                .base(),
                value.end());

            return value;
        }

        std::vector<std::string> splitCsvLine(const std::string& line) {
            std::vector<std::string> values;
            std::size_t valueStart = 0;

            while (valueStart <= line.length()) {
                const std::size_t valueEnd = line.find(',', valueStart);
                values.push_back(trim(line.substr(valueStart, valueEnd - valueStart)));

                if (valueEnd == std::string::npos) {
                    break;
                }
                valueStart = valueEnd + 1;
            }

            return values;
        }

        double parseDouble(const std::string& value, const std::string& fieldName) {
            std::size_t parsedLength = 0;
            const double parsedValue = std::stod(value, &parsedLength);
            if (parsedLength != value.length() || !std::isfinite(parsedValue)) {
                throw std::invalid_argument("invalid " + fieldName + " value '" + value + "'");
            }

            return parsedValue;
        }

        std::uint64_t parseSequence(const std::string& value) {

```



```

        std::size_t parsedLength = 0;
        const auto parsedValue = std::stoull(value, &parsedLength);
        if (parsedLength != value.length()) {
            throw std::invalid_argument("invalid sequence value '" + value + "'");
        }

        return parsedValue;
    }

    Measurement parseMeasurementLine(const std::string& line) {
        const std::vector<std::string> values = splitCsvLine(line);
        if (values.size() != 3 && values.size() != 4) {
            throw std::invalid_argument("expected temperature, humidity, voltage[, sequence]");
        }

        Measurement measurement;
        measurement.temperature = parseDouble(values[0], "temperature");
        measurement.humidity = parseDouble(values[1], "humidity");
        measurement.voltage = parseDouble(values[2], "voltage");
        measurement.sequence = values.size() == 4 ? parseSequence(values[3]) : 0;
        measurement.timestamp = std::chrono::system_clock::now();

        return measurement;
    }
} // namespace

MeasurementUnixSocketContext::MeasurementUnixSocketContext(core::socket::stream::
SocketConnection* socketConnection,
                                                             MeasurementHandler
measurementHandler)
    : core::socket::stream::SocketContext(socketConnection)
    , measurementHandler(std::move(measurementHandler)) {
}

void MeasurementUnixSocketContext::onConnected() {
    VLOG(1) << "Measurement socket connected from " << getSocketConnection()->
getRemoteAddress().toString();
}

void MeasurementUnixSocketContext::onDisconnected() {
    VLOG(1) << "Measurement socket disconnected from " << getSocketConnection()->
getRemoteAddress().toString();
}

bool MeasurementUnixSocketContext::onSignal(int signum) {
    VLOG(1) << "Measurement socket disconnected due to signal " << signum;

    return true;
}

std::size_t MeasurementUnixSocketContext::onReceivedFromPeer() {
    char chunk[4096];
    const std::size_t chunkLen = readFromPeer(chunk, sizeof(chunk));

    if (chunkLen > 0) {
        receiveBuffer.append(chunk, chunkLen);

        std::size_t lineEnd = receiveBuffer.find('\n');
        while (lineEnd != std::string::npos) {
            std::string line = receiveBuffer.substr(0, lineEnd);
            if (!line.empty() && line.back() == '\r') {
                line.pop_back();
            }
        }
    }
}

```

```

    }

    processLine(line);
    receiveBuffer.erase(0, lineEnd + 1);
    lineEnd = receiveBuffer.find('\n');
}

if (receiveBuffer.length() > 4096) {
    LOG(WARNING) << "Measurement socket line exceeds 4096 bytes; dropping buffered
input";
    receiveBuffer.clear();
}

return chunkLen;
}

void MeasurementUnixSocketContext::processLine(const std::string& line) const {
    if (!line.empty()) {
        try {
            Measurement measurement = parseMeasurementLine(line);
            measurementHandler(std::move(measurement));
        } catch (const std::exception& ex) {
            LOG(WARNING) << "Ignoring invalid measurement line '" << line << "': " << ex.
what();
        }
    }
}

} // namespace minigateway

```

38.6 The Unix-domain measurement factory

The factory is small. Its only job is to construct a `MeasurementUnixSocketContext` for each accepted Unix-domain stream connection and pass the configured measurement handler into it.

This mirrors the factory pattern used earlier for MQTT, but with a different protocol object:

```

Unix SocketConnection
-> MeasurementUnixSocketContextFactory
    -> MeasurementUnixSocketContext
        -> MeasurementHandler

```

MeasurementUnixSocketContextFactory.h

```

#pragma once

#include "MeasurementUnixSocketContext.h"

#include <core/socket/stream/SocketContextFactory.h>

namespace minigateway {

    class MeasurementUnixSocketContextFactory : public core::socket::stream::
        SocketContextFactory {
    public:
        explicit MeasurementUnixSocketContextFactory(MeasurementUnixSocketContext::
            MeasurementHandler measurementHandler);

    private:

```

```

        core::socket::stream::SocketContext* create(core::socket::stream::SocketConnection*
        socketConnection) final;

        MeasurementUnixSocketContext::MeasurementHandler measurementHandler;
    };
} // namespace minigateway

```

MeasurementUnixSocketContextFactory.cpp

```

#include "MeasurementUnixSocketContextFactory.h"

#include <utility>

namespace minigateway {

    MeasurementUnixSocketContextFactory::MeasurementUnixSocketContextFactory(
        MeasurementUnixSocketContext::MeasurementHandler measurementHandler)
        : measurementHandler(std::move(measurementHandler)) {
    }

    core::socket::stream::SocketContext*
    MeasurementUnixSocketContextFactory::create(core::socket::stream::SocketConnection*
    socketConnection) {
        return new MeasurementUnixSocketContext(socketConnection, measurementHandler);
    }
} // namespace minigateway

```

38.7 Runtime assembly after extension

The extended `main.cpp` keeps the base application path and adds one new server role named `measurement-input`. The new role listens on `/tmp/minigateway-measurements.sock` and uses the same `acceptMeasurement` function as `/simulate`.

This is the key design point already visible in Figure 38.1. The input changes, but the application state path does not. The HTTP route and the Unix-domain server are different communication roles; after they have produced a `Measurement`, both end at the same acceptance boundary.

Once a measurement reaches `acceptMeasurement`, the rest of the application is identical: current state is updated, the bus publishes, SSE observers are notified, and MQTT publication is attempted if the MQTT role is connected.

Compared with the base version, the new `SNode.C` include is the Unix-domain stream server front door:

```
#include <net/un/stream/legacy/SocketServer.h>
```

It mirrors the new component `net-un-stream-legacy`. The rest of the `SNode.C` include block remains at the abstraction level used by the source: Express for the HTTP server role, IPv4 legacy socket client for the MQTT carrier role, and Unix-domain legacy socket server for the local measurement input role.

main.cpp

```

#include "ConfigSections.h"
#include "MeasurementBus.h"
#include "MeasurementState.h"
#include "MeasurementUnixSocketContextFactory.h"
#include "MiniGatewayMqtt.h"
#include "MiniGatewaySocketContextFactory.h"

```

```

#include <chrono>
#include <core/SNodeC.h>
#include <cstdlib>
#include <express/legacy/in/WebApp.h>
#include <express/middleware/VerboseRequest.h>
#include <log/Logger.h>
#include <net/config/ConfigInstance.h>
#include <net/in/stream/legacy/SocketClient.h>
#include <net/un/stream/legacy/SocketServer.h>
#include <string>
#include <type_traits>
#include <utility>
#include <web/http/http_utils.h>

namespace {

    void reportState(const std::string& instanceName, const core::socket::SocketAddress&
        socketAddress, const core::socket::State& state) {
        switch (state) {
            case core::socket::State::OK:
                VLOG(1) << instanceName << ": connected/listening on " << socketAddress.
toString() << " ";
                break;
            case core::socket::State::DISABLED:
                VLOG(1) << instanceName << ": disabled";
                break;
            case core::socket::State::ERROR:
                LOG(ERROR) << instanceName << " " << socketAddress.toString() << ": " << state.
what();
                break;
            case core::socket::State::FATAL:
                LOG(FATAL) << instanceName << " " << socketAddress.toString() << ": " << state.
what();
                break;
        }
    }

    minigateway::Measurement makeSimulatedMeasurement(std::uint64_t& sequence) {
        minigateway::Measurement measurement;
        measurement.temperature = 20.0 + static_cast<double>(sequence % 50) / 10.0;
        measurement.humidity = 40.0 + static_cast<double>(sequence % 20);
        measurement.voltage = 3.7 + static_cast<double>(sequence % 10) / 100.0;
        measurement.sequence = ++sequence;
        measurement.timestamp = std::chrono::system_clock::now();

        return measurement;
    }

    void createMqttConfig(net::in::stream::legacy::config::ConfigSocketClient& config) {
        config.Instance::newSubCommand<minigateway::ConfigMqtt>();
    }

    auto startMqttClient() {
        using Client = net::in::stream::legacy::SocketClient<minigateway::
MiniGatewaySocketContextFactory>;

        Client socketClient("mqtt-uplink");

        auto* config = socketClient.getConfig();
        config->Remote::setPort(1883);
        config->setDisableNagleAlgorithm();
        createMqttConfig(*config);
    }
}

```

```

        socketClient.getConfig()->setRetry();
        socketClient.getConfig()->setRetryBase(1);
        socketClient.getConfig()->setReconnect();

        socketClient.connect([](const Client::SocketAddress& socketAddress, const core::socket::
State& state) {
            reportState("mqtt-uplink", socketAddress, state);
        });

        return socketClient;
    }

template <typename MeasurementHandler>
auto startMeasurementSocketServer(MeasurementHandler measurementHandler) {
    using Handler = std::decay_t<MeasurementHandler>;
    using Server = net::un::stream::legacy::SocketServer<minigateway::
MeasurementUnixSocketContextFactory, Handler>;

    Server socketServer("measurement-input", std::move(measurementHandler));
    socketServer.listen("/tmp/minigateway-measurements.sock",
        [](const Server::SocketAddress& socketAddress, const core::socket::
State& state) {
            reportState("measurement-input", socketAddress, state);
        });

    return socketServer;
}

template <typename ResponseT>
void sendMeasurementEvent(const minigateway::Measurement& measurement, const ResponseT& res)
{
    res->sendFragment("event: measurement");
    res->sendFragment("id:" + std::to_string(measurement.sequence));
    res->sendFragment("retry: 1000");
    res->sendFragment("data: " + minigateway::toPayload(measurement) + "\r\n");
}

} // namespace

int main(int argc, char* argv[]) {
    core::SNodeC::init(argc, argv);

    minigateway::MeasurementState state;
    minigateway::MeasurementBus bus;

    auto acceptMeasurement = [&state, &bus](minigateway::Measurement measurement) {
        if (measurement.sequence == 0) {
            measurement.sequence = state.current().sequence + 1;
        }

        state.update(measurement);
        bus.publish(state.current());
    };

    bus.subscribe([](const minigateway::Measurement& measurement) {
        minigateway::MiniGatewayMqtt::publishMeasurementToConnected(measurement);
        return true;
    });

    const express::legacy::in::WebApp app;

    app.use(express::middleware::VerboseRequest());

```

```

app.get("/health", [] APPLICATION(req, res) {
    res->json({{"ok", true}});
});

app.get("/status", [&state] APPLICATION(req, res) {
    res->json(minigateway::toJson(state.current()));
});

app.get("/events", [&state, &bus] APPLICATION(req, res) {
    if (web::http::ciContains(req->get("Accept"), "text/event-stream")) {
        res->set("Content-Type", "text/event-stream").set("Cache-Control", "no-cache").set("
Connection", "keep-alive");
        res->sendHeader();

        const minigateway::Measurement current = state.current();
        if (current.sequence > 0) {
            sendMeasurementEvent(current, res);
        }

        bus.subscribe([res](const minigateway::Measurement& measurement) {
            const bool keepSubscriber = res->isConnected();
            if (keepSubscriber) {
                sendMeasurementEvent(measurement, res);
            }

            return keepSubscriber;
        });
    } else {
        res->status(406).send("SSE requires Accept: text/event-stream");
    }
});

std::uint64_t simulatedSequence = 0;

app.get("/simulate", [&acceptMeasurement, &simulatedSequence] APPLICATION(req, res) {
    minigateway::Measurement measurement = makeSimulatedMeasurement(simulatedSequence);
    acceptMeasurement(measurement);

    res->json(minigateway::toJson(measurement));
});

app.listen(8080,
    [instanceName = app.getConfig()->getInstanceName()](const express::legacy::in::
WebApp::SocketAddress& socketAddress,
    const core::socket::State&
listenState) {
    reportState(instanceName, socketAddress, listenState);
});

const auto measurementSocketServer = startMeasurementSocketServer(acceptMeasurement);
const auto mqttClient = startMqttClient();

return core::SNodeC::start();
}

```

38.8 Build and usage note for the extended version

The extended README documents the new local measurement input. The example uses `nc -U`, but the same socket could be written by a helper process, a data-acquisition script, a serial-to-local-socket adapter, or another local service.

README-BUILD.md

```
# MiniGateway
```

This is the current event-driven MiniGateway source package extracted from the book project.

It contains the complete source tree used by Chapter 38:

```
- `CMakeLists.txt`
- `main.cpp`
- `Measurement.h`
- `MeasurementState.h/.cpp`
- `MeasurementBus.h/.cpp`
- `MeasurementUnixSocketContext.h/.cpp`
- `MeasurementUnixSocketContextFactory.h/.cpp`
- `ConfigSections.h/.cpp`
- `MiniGatewayMqtt.h/.cpp`
- `MiniGatewaySocketContextFactory.h/.cpp`
```

Expected build shape, assuming SNode.C is installed as a CMake package:

```
```sh
cmake -S . -B build
cmake --build build
```
```

Measurements can be injected through the Unix domain socket `/tmp/minigateway-measurements.sock`. Send one comma-separated line per measurement:

```
```sh
printf '21.5,43.0,3.72\n' | nc -U /tmp/minigateway-measurements.sock
```
```

The accepted fields are `temperature,humidity,voltage` or `temperature,humidity,voltage,sequence`. Measurements without a sequence receive the next local sequence number and are forwarded to the `MeasurementState`, `MeasurementBus`, MQTT publisher, SSE subscribers, and `/status` endpoint.

This package intentionally contains no TLS, no persistence, no MQTT-over-WebSocket, no frontend.

38.9 What the extension teaches

The extension is small, but it teaches a general SNode.C lesson. A new carrier or local input path should be added where that communication concern belongs. It should not be smeared across the existing output protocols.

bad extension:

```
parse device input inside /status
publish MQTT directly from the parser
let SSE own the latest measurement
```

better extension:

```
parse input in a device-facing context
hand a Measurement to the application
reuse the existing state and bus path
```

The extended MiniGateway source is therefore not interesting because it adds many features. It is interesting because it adds one feature without disturbing the old boundaries.

38.10 Extending beyond this gateway input

The Unix-domain socket is only one example of an extension boundary. The same reasoning applies to other changes.

A serial input adapter would still be a device-facing input boundary. A Bluetooth RFCOMM or L2CAP input would also be a device-near input boundary, with the additional operational condition that the devices must already be paired at the operating-system level before `SNode.C` can use the Bluetooth connection. A database history feature would be a persistence boundary. MQTT-over-WebSocket would be a carrier variation for MQTT semantics. A browser dashboard would be an application/UI surface, not a reason to move domain ownership into HTTP route handlers.

| New requirement | Preferred owner |
|--|--|
| local line-oriented measurement input | input <code>SocketContext</code> |
| serial or Bluetooth measurement source | device-facing input boundary |
| durable measurement history | persistence service |
| MQTT over WebSocket | MQTT carrier boundary |
| authentication | HTTP/MQTT boundary and deployment policy |
| browser dashboard | web application/static asset boundary |
| command execution | explicit command/domain service |
| retained measurement backlog | persistence or bounded queue, not SSE response state |

The pattern is always the same: identify the concern first, then choose the owner.

38.11 Testing MiniGateway by boundaries

MiniGateway testing warning

Do not treat testing and deployment as an afterthought after the architecture is already fixed. End-to-end checks are useful, but they often hide the boundary that failed.

| Test surface | What it protects |
|--|--|
| CMake build | external <code>SNode.C</code> consumer shape |
| <code>/health</code> | basic HTTP request/response path |
| <code>/status</code> | current measurement representation |
| <code>/simulate</code> | controlled base input boundary |
| <code>/events</code> | SSE lifetime and event formatting |
| <code>measurement-input</code> Unix socket | line parsing and local input role |
| <code>MeasurementState</code> | current-value ownership |
| <code>MeasurementBus</code> | event fan-out without protocol coupling |
| <code>MiniGatewayMqtt</code> | MQTT session and publish behavior |
| generated configuration | effective role shape |
| deployment | installed binary, libraries, paths, and service user |

A useful test name should often answer the question from Chapter 34:

MiniGateway test-surface checklist

- extension point
- failure behavior
- diagnostics
- test surface
- deployment surface

- runtime configuration

For example:

```
status_returns_current_measurement
simulate_publishes_to_bus
unix_socket_accepts_measurement_line
sse_disconnect_removes_listener
mqtt_publish_only_when_connected
```

These names preserve architectural intent.

38.12 Manual checks while developing

A small manual sequence can already exercise most of the guided application:

```
./minigateway
```

In another terminal:

```
curl http://localhost:8080/health
curl http://localhost:8080/status
curl http://localhost:8080/simulate
curl http://localhost:8080/status
```

For SSE:

```
curl -N -H 'Accept: text/event-stream' http://localhost:8080/events
```

Then trigger a measurement:

```
curl http://localhost:8080/simulate
```

For the extended Unix-domain input:

```
printf '21.5,43.0,3.72
' | nc -U /tmp/minigateway-measurements.sock
curl http://localhost:8080/status
```

And before diagnosing an HTTP route, verify endpoint ownership:

```
ss -ltnp 'sport = :8080'
```

This last command matters. A wrong response from `localhost:8080` may mean the wrong process owns the port, not that MiniGateway's route is wrong.

38.13 Debugging by role

When MiniGateway fails, start with the role or boundary, not with the whole application.

```
/status does not change:
  inspect input path, acceptMeasurement, MeasurementState

/events does not receive updates:
  inspect SSE request headers, response lifetime, MeasurementBus listeners

MQTT does not publish:
```

```
inspect mqtt-uplink connection state, broker reachability, CONNACK result

Unix socket input fails:
inspect measurement-input state, socket path, line format, permissions

wrong HTTP response appears:
inspect which process owns the port
```

Configured role names are diagnostic handles. `mqtt-uplink` and `measurement-input` are not decorative names. They connect configuration, state callbacks, logs, and operator language.

38.14 Deployment shape

A deployed MiniGateway includes linked SNode.C components, installed public headers for development builds, configuration, runtime state, log output, and possibly service-manager integration.

A general-purpose Linux deployment should make these questions answerable:

```
Which binary is installed?
Which SNode.C components are required?
Which configured roles exist?
Which ports or socket paths are owned?
Where are configuration, logs, and pid files written?
Which user/group owns the service?
How is restart handled?
```

For the extended version, the Unix-domain socket path is a deployment surface. `/tmp/minigateway-measurements.sock` is convenient for a guided project, but a packaged service may want a path below a runtime directory owned by the service user. That is a deployment decision, not a parser decision.

38.15 OpenWrt and constrained systems

On OpenWrt or another constrained Linux target, MiniGateway's architecture does not change, but the cost of each dependency becomes more visible. The package should include only the required SNode.C components. The service definition should expose the configured roles clearly. Runtime paths must fit the platform's filesystem and permission model.

For an OpenWrt-style deployment, ask:

```
Is HTTP really needed on the device?
Is native MQTT enough, or is MQTT-over-WebSocket required by the network?
Where should local measurement input live?
Should measurement input be enabled on this target?
Which logs are useful under constrained storage?
What happens if the broker is temporarily unavailable?
```

These are not questions for CMake alone. They are application-deployment questions.

38.16 When to split the application

MiniGateway is intentionally one process. That is appropriate for the guided project because it keeps the role constellation visible. A larger system may eventually need process separation.

Split only when the operational boundary is real:

```
same process is fine when:
failures are shared
deployment is shared
configuration is shared
```

```

the roles form one small application

separate processes may be better when:
  input collection must survive HTTP restarts
  persistence must be isolated
  security policies differ
  deployment cadence differs
  one role must be disabled independently

```

Process splitting is not automatically more mature. It is useful when it makes an existing operational boundary honest.

38.17 Extending safely in general

The MiniGateway extension is a concrete example, but the rule applies throughout SNode.C applications.

```

new protocol semantics
  -> protocol context or subprotocol

new carrier
  -> lower-family or upgrade boundary

new domain behavior
  -> application/domain service

new durability requirement
  -> persistence boundary

new operator choice
  -> configuration surface

new failure policy
  -> role that owns the failure

```

If an extension requires unrelated parts of the application to know too much about each other, the boundary is probably wrong. That does not always mean a new framework abstraction is needed. Sometimes the correct answer is simply a small application-local class with a clear name and a narrow responsibility.

What to remember

- The extended version adds a Unix-domain measurement input, not a new application architecture.
- The new input path ends at the same `acceptMeasurement` function used by `/simulate`.
- HTTP, SSE, and MQTT behavior remain unchanged because the extension is placed at the input boundary.
- A `SocketContext` is a good owner for local line-oriented stream input.
- A factory should construct the context and inject the application handler, not own the domain state.
- New requirements should be placed at the boundary that owns their variation.

Epilogue — What to Take Away from SNode.C

SNode.C is not primarily a shortcut for opening sockets.

It is a way of keeping network-system architecture visible while the application grows from simple connections to protocols, protocols to roles, roles to applications, and applications to deployable systems.

That visibility is the real subject of this book.

Layers are not decoration

The word layered can be used lazily. In SNode.C, it is meant operationally.

Lower communication families, stream transport, TLS or legacy connection handling, protocol layers, application roles, configuration, diagnostics, build targets, packages, and deployment shape all have their own responsibilities.

The important lesson is not that every layer must be visible to every programmer at every moment. The lesson is that the architecture remains understandable because the layers have not been erased prematurely.

```
lower family
-> transport and connection handling
    -> protocol meaning
        -> application role
            -> system boundary
```

A reader who understands this stack can move between IPv4, IPv6, Unix domain sockets, Bluetooth, HTTP, WebSocket, MQTT, persistence, and deployment without treating each subject as a separate world.

Boundaries are design decisions

Many mistakes in network applications are not caused by the wrong syntax. They are caused by the wrong boundary.

A protocol concern is placed in a transport layer. A deployment concern is hidden in a parser. A domain rule is buried in a socket callback. A persistence policy is smuggled into an HTTP route. A retry policy becomes invisible because it is treated as a low-level accident.

SNode.C gives the programmer many possible places where behavior can live. That is powerful, but it also requires judgment.

The central question returns again and again:

Which boundary honestly owns this concern?

That question is useful before writing code, while debugging code, while designing configuration, while packaging the application, and while deciding whether a system should remain one executable or become several cooperating services.

Roles and instances need precise language

The book deliberately distinguishes system-design roles, application-side server or client handles, configured communication roles, registered runtime-visible instances, connections, contexts, factories, middleware, subprotocols, and application services.

That precision may feel demanding at first. It is worth the effort.

When the vocabulary is clear, a system can be discussed by developers, operators, teachers, and students using the same names.

```
configured role name
-> appears in configuration
-> appears in logs
-> appears in diagnostics
-> appears in operational discussion
```

This terminology is how a running system becomes understandable.

Protocols carry meaning, not only bytes

The book started with simple communication and gradually moved upward: custom stream protocols, HTTP, Express-like routing, SSE, WebSocket, MQTT, MQTT-over-WebSocket, and multi-protocol IoT systems.

The recurring lesson is that bytes become useful only when a layer gives them meaning.

```
raw data
  -> parsed protocol
    -> session or request meaning
      -> application state
        -> system behavior
```

Therefore, protocol code should not be treated as accidental glue. It is where communication becomes part of an application.

Build and deployment are part of architecture

A network framework does not end at the source tree.

The public include path says which C++ abstraction the source file selects. The build target says what the application links. The exported package says what external consumers can depend on. The install component says what files need to exist. The package dependency says what the target system must provide. The runtime path says where loaded modules and services can be found. The configuration file says what role shape is deployed.

That is why this book treated CMake, packaging, deployment, testing, and debugging as architectural surfaces rather than as afterthoughts.

```
public include path
-> source component
    -> build target
        -> installed component
            -> package dependency
                -> deployed role
                    -> observable runtime behavior
```

A system that is clear only in the source tree is not yet fully clear.

The guided project is not the end of the idea

MiniGateway was small. It did not try to become a product. It showed how the pieces can be assembled while preserving the vocabulary of the book.

That is the transferable lesson.

A real application may use different lower families, different protocols, more roles, stronger persistence, stronger authentication, different deployment targets, or a larger ecosystem such as MQTTSuite. The architectural questions remain the same.

What is the domain state?
Which roles expose it?
Which protocols serve which boundaries?
Which configuration makes the deployment reproducible?
Which diagnostics make failure understandable?
Which tests protect the real boundaries?

When those questions remain answerable, the application can grow without becoming opaque.

A philosophical review of the book

This book has used SNode.C as a concrete framework, but its deeper subject is the question of how network software can remain intelligible when it grows. That question is older than any particular library. Every serious network application eventually meets the same forces: different carriers, different protocols, changing deployment targets, operational failures, partial knowledge, and the temptation to hide complexity behind one convenient abstraction.

The position taken by this book is deliberately not that complexity can be made to disappear. A framework that pretends to remove all complexity usually only moves it somewhere less visible. SNode.C takes a different path. It gives names and places to concerns that are often blurred: lower communication family, transport form, connection handling, protocol meaning, context, configured role, runtime-visible instance, application state, package component, deployed service, diagnostic boundary. The philosophical value of such a framework is not that it makes design automatic. It makes design discussable.

That is why the book has returned so often to boundaries. A boundary is not a wall erected for its own sake. It is a claim that a certain concern has a natural owner. When the claim is right, the program becomes easier to reason about. When the claim is wrong, the program may still compile, but it becomes conceptually misleading. The HTTP route begins to own domain state. The MQTT callback begins to decide deployment policy. A socket context becomes a dumping ground for orchestration. A configuration option becomes a disguised invariant. These failures are technical, but they are also failures of thought.

The recurring discipline of the book can therefore be read as a kind of engineering humility. Do not pretend that a byte stream already is a protocol. Do not pretend that a protocol already is an application. Do not pretend that an application already is a deployed system. Do not pretend that a running process is understood merely because it is running. Each step adds meaning, and each step deserves a place where that meaning can be named, tested, logged, configured, and maintained.

There is also a positive side to this discipline. Visible boundaries make change less frightening. MiniGateway could be extended because the base application did not collapse measurement input, state ownership, observation, and MQTT publication into one accidental block of code. MQTTSuite can be understood as an ecosystem because its roles are architectural positions expressed as executables. Build targets, package components, and runtime configuration are not administrative noise; they are part of how the architecture survives contact with real machines.

Seen this way, SNode.C is less a library of shortcuts than a vocabulary for layered network systems in C++. It rewards programmers who are willing to be explicit. It asks for more care at the boundary between concepts, but it pays that care back when systems need to be extended, diagnosed, ported, packaged, or taught. That is the philosophical center of the book: clarity is not achieved by hiding all structure. Clarity is achieved by preserving the right structure long enough that the system can still explain itself.

Where to go next

A reader who finishes this book should know how SNode.C public headers, servers, clients, contexts, HTTP, WebSocket, MQTT, configuration, and CMake components fit together.

The reader should know how to think with them.

Useful next steps are:

```
build a small application with one clear role
add one second protocol surface deliberately
make configuration reproduce the role shape
add diagnostics before the system becomes hard to debug
test one boundary at a time
only then generalize or split the system
```

SNode.C rewards this discipline because its architecture makes the boundaries explicit. It does not force good design automatically, but it gives good design places to live.

Closing perspective

The promise of SNode.C is not that networking becomes trivial.

Networking is not trivial. Protocols, timing, failures, deployment, security, diagnostics, and system evolution remain real engineering problems.

The promise is different:

```
network systems can remain understandable  
when their layers and boundaries remain visible
```

That is the book's central lesson.

If the reader carries one idea forward, it should be this:

```
Do not hide meaning too early.  
Place behavior where the right boundary can own it.  
Keep that boundary visible in code, configuration, diagnostics, build, deployment, and tests.
```

That is how SNode.C should be used. And that is how layered network applications in modern C++ can stay clear as they grow.

Further Reading

The book is intentionally focused on SNode.C. The references below give stable background for readers who want to go deeper into the standards, protocols, tools, and systems context that appear throughout the chapters.

C++ and build systems

- ISO/IEC 14882, *Programming Language — C++*. The language standard is the authoritative reference for C++ itself.
- Bjarne Stroustrup, *The C++ Programming Language*. A broad reference for the language and its design vocabulary.
- Scott Meyers, *Effective Modern C++*. A compact guide to many C++11/14 idioms that remain relevant in framework and application code.
- CMake documentation, especially the manuals for `cmake-buildsystem(7)`, `cmake-packages(7)`, and `cmake(1)`. These are useful when reading Chapter 32 and the example build files.

POSIX, Linux, and network programming

- POSIX.1, *The Open Group Base Specifications*. This is the standards background for many Unix and POSIX APIs used by Linux-oriented network software.
- W. Richard Stevens, Bill Fenner, and Andrew M. Rudoff, *UNIX Network Programming, Volume 1: The Sockets Networking API*. Still valuable for understanding the socket model beneath higher-level abstractions.
- Michael Kerrisk, *The Linux Programming Interface*. Useful background for Linux system calls, file descriptors, processes, signals, and deployment-oriented behavior.

TLS and security protocols

- RFC 8446, *The Transport Layer Security (TLS) Protocol Version 1.3*.
- OpenSSL documentation. Useful when connecting framework-level TLS configuration to the underlying TLS library and certificate material.

HTTP, Server-Sent Events, and WebSocket

- RFC 9110, *HTTP Semantics*.
- RFC 9112, *HTTP/1.1*.
- WHATWG HTML Living Standard, section on server-sent events and the `EventSource` interface.
- RFC 6455, *The WebSocket Protocol*.

MQTT and messaging systems

- OASIS, *MQTT Version 3.1.1*.
- OASIS, *MQTT Version 5.0*.

- *MQTT V3.1 Protocol Specification* (International Business Machines Corporation and Eurotech, 2010). Useful historical context for the Stanford-Clark/Nipper design lineage before MQTT moved into OASIS standardization.

Deployment and embedded Linux

- OpenWrt documentation. Useful background for package layout, init scripts, configuration, and deployment on router-class Linux systems.
- systemd documentation. Useful when adapting SNode.C applications to service supervision on general-purpose Linux distributions.

Databases and persistence

- MariaDB documentation, especially the client library and connector documentation. Useful background for applications that move from transient protocol state to durable application state.

SNode.C project material

- The SNode.C repository is the authoritative source for framework development beyond the version discussed in this book. This edition covers the SNode.C_{v1.0.2} baseline.
- The companion source trees under `companion/examples/` are the authoritative sources for the book's compact examples and MiniGateway variants.

Index

- adapter state, 128
- address classes, 80
- address family, 107
- address resolution, 86
- address semantics, 80
- administration role, 302
- alias types, 32
- API surface, 414
- application callbacks, 250
- application configuration, 180, 183
- application families, 338
- application layer, 67, 73, 236
- application protocol, 47, 281
- application reading workflow, 341
- application scope, 180
- application state, 314
- application structure, 330
- application targets, 331
- application-first reading, 30
- application-level logging, 200
- application-local extension, 423
- application-state boundary, 315
- architectural judgment, 413
- architectural principle, 421
- architecture
 - build structure, 367
- assembly point, 336
- authentication middleware, 253

- backpressure, 232, 405
- benchmarking, 397
- blank-line dispatch, 263
- Bluetooth, 122
 - deployment, 129
 - device edge, 307
 - L2CAP, 84, 123
 - pairing, 128
 - RFCOMM, 83, 123
 - server/client use, 125
 - service selectors, 122
- Bluetooth address, 124
- boundary choice, 413
- boundary discipline, 4
- boundary extension, 459
- boundary needs, 303
- boundary preservation, 422
- boundary roles, 301
- boundary testing, 397
- boundary tests, 460
- bounded waiting, 223

- bridge topology, 360
- broker role, 286, 359
- build architecture, 469
- build artifacts, 297
- build environment, 9
 - dependencies, 11
 - optional dependencies, 11
 - packages, 11
- build extension, 427
- build options, 13, 363
- build structure, 29, 367
- build system, 75
- build targets, 331
- build tree, 12, 15
- build-time confidence, 399
- build-time defaults, 380
- bytes versus meaning, 468

- C++20, 10
- callbacks, 97
- carrier choice, 157
- carrier mechanics, 417
- channel, 83
- CLI11, 178
- client application, 23
- client program, 13
- client role, 90
- client type, 37
- close frame, 271
- CMake, 10, 24, 367
 - as navigation, 29, 34
 - link interfaces, 376
 - options, 13
- CMakeLists.txt, 434
- code versus configuration, 417
- codebase orientation, 29
- command sequences, 322
- command-line parsing, 178
- commit, 325
- communication family, 414
- communication stack, 67
- compiler requirements, 10
- component architecture, 75
- component names, 68
- component selection, 297, 381
- component surface
 - extension, 427
- component targets, 370
- component/header matrix, 373
- components, 14, 367

- libraries, 370
- composed protocol, 292, 415
- composition depth, 341
- ConfigSections**, 438
- configuration, 49, 169, 350, 363
 - architecture, 169
 - C++ API, 171, 188
 - command line, 171, 188
 - diagnostics, 204
 - discovery, 176
 - extension, 426
 - file, 171, 188
 - lower-family transfer, 166
 - precedence, 172
 - role map, 308
- configuration addresses, 173
- configuration design, 192, 417
- configuration files, 192
- configuration hierarchy, 180
- configuration philosophy, 169
- configuration sections, 175
- configured communication role, 43, 169, 184
- configured instance names, 407
- Connack**, 287
- connect()**, 23, 92, 116, 125
 - parameterless, 177
- connection, 7, 33, 43, 46, 88, 134
- connection handling, 47
- connection identity, 205
- connection layer, 67, 72, 210
- connection metrics, 94, 205
- connection section, 185
- connection timeout, 223
- connection variants, 211
- connection-layer specialization, 210
- connection-level logging, 200
- connection-local behavior, 133
- connection-local state, 139
- connection-to-context path, 147
- constrained systems, 462
- construction tests, 155
- context, 7, 18, 19, 33, 44, 46, 134
- context anti-patterns, 142
- context behavior, 158
- context class, 37
- context construction, 44, 99, 145, 147
- context design, 133, 139
- context surface, 135
- context-level logging, 200
- controlled stepping, 56
- core**, 33
- core components, 377
- core runtime, 53
- core::EventLoop**, 56
- core::SNodeC**, 43, 55
- core::socket::State**, 98
- CPack**, 387
- cross-layer responsibility, 76
- daemonization, 389
- data-collection systems, 302
- database client, 318
- database client ownership, 326
- database commands, 322
- database dependencies, 392
- database failure, 327
- database module, 316
- database state, 317
- database support, 314
- database-backed tests, 402
- datagram sockets, 120
- debugging, 207, 397, 407
- debugging by role, 461
- default construction, 84
- dependency injection, 148
- dependency passing, 152
- dependency surface, 370
- deployment, 385
 - packages, 387
- deployment architecture, 469
- deployment assumptions, 160
- deployment boundaries, 416
- deployment logging, 199
- deployment policy, 389
- deployment reading workflow, 395
- deployment shape, 192
- deployment tests, 406
- descriptor events, 61
- descriptor publishers, 61
- descriptor receivers, 62
- design decisions, 466
- development environment, 9
- device-facing role, 302
- diagnostic depth, 199
- diagnostic map, 195
- diagnostic workflow, 207
- diagnostics, 195, 233, 350, 407
 - extension, 428
- disablement, 230
- dispatcher, 251
- DNS, 86
- domain code, 352
- dual-stack deployment, 109
- echo examples, 338
- echo pair, 17
 - lower-family transfer, 161
- EchoSocketContext**, 19, 143
- effective configuration, 204
- embedded Linux, 393
- endpoint configuration, 160
- endpoint identity, 70, 80, 81, 113, 162
- endpoint roles, 164
- entry point, 336
- error handling, 136
- Event**, 59
- event delivery, 58
- event loop, 56

- cleanup, 63
- node.js comparison, 6
- tick cycle, 57, 63
- timers, 62
- event processing, 53
- event queue, 60
- event receivers, 62
- event stream fields, 263
- event streams, 257
- event-driven database access, 321
- event-driven design, 139
- event-driven network applications, 2
- event-driven runtime, 42, 53
- event-stream validation, 263
- EventMultiplexer**, 60
- EventReceiver**, 59
- EventSource**, 242, 256, 258
 - client abstraction, 261
 - public surface, 266
- EventSource tests, 402
- example applications, 330
- example build, 24
- example code, 31
- exec()**, 322
- exported package targets, 381
- exported targets, 375
- Express components, 255, 378
- Express routing tests, 402
- Express-like framework, 246
 - layered model, 246
 - operator surface, 306
 - semantics, 247
- extensibility point, 418
- extension, 422
- extension design, 418
- extension levels, 423
- external consumer project, 14
- external consumers, 381
- facade, 252
- factory, 7, 18, 33, 44, 134
- factory anti-patterns, 150
- factory class, 37
- factory construction policy, 158
- factory design, 145
- factory design shapes, 153
- factory extension, 424
- factory interface, 145
- factory responsibilities, 148
- factory tests, 155
- failure behavior, 350
- failure handling, 221
 - reconnect, 225
 - retry, 225
 - termination, 221
- failure modes, 221
- failure policy, 418
 - by role, 309
 - extension, 428
- failure state, 230
- failure visibility, 233
- feeds, 393
- filesystem layout, 386
- first program, 17
- flow control, 33, 49
- flow controllers, 231
- framework boundaries, 27, 397
- framework code, 31
- framework extension, 422
- framework pollution, 429
- fresh context per connection, 147
- gateway application, 432
- generated configuration, 176, 192, 380
- getSSL()**, 214
- git checkout**, 11
- guided project, 432, 470
- handle, 7, 46
- handle lifetime, 443
- handle type, 160
- host and port, 82
- HTTP, 236
 - layered model, 236
 - management surface, 306
- HTTP administration role, 434
- HTTP client, 239
- HTTP components, 245, 378
- HTTP headers, 240
- HTTP layer, 236
- HTTP parsing, 240
- HTTP server, 237, 248
- HTTP status, 240
- HTTP streaming, 257
- HTTP tests, 402
- HTTP upgrade, 242, 267, 269
- implementation layer, 416
- include blocks, 336
- init()**, 56
- input processing, 136
- install tree, 12, 15
- installable components, 35
- installation, 12
- installed architecture, 14, 386
- installed-consumer builds, 399
- instance configuration, 180, 184
- instance names, 173
- instance scope, 180
- instances, 18, 43, 467
- integration role, 302
- INTERFACE**, 376
- internal event bus, 437
- IoT, 2, 281
- IoT systems, 301
 - administration, 305
 - boundary systems, 301
 - control, 305

- observation, 305
 - telemetry, 305
- `iot::mqtt`, 288
- IPv4, 82, 103
 - shared model, 103
- IPv6, 82, 103
 - IPv4-mapped addresses, 109
 - IPv6-only, 109
 - shared model, 103
- jitter, 228
- JSON dependency, 289
- JSON examples, 338
- JSON middleware, 253
- keep-alive, 284
- L2CAP, 84, 122, 123
 - PSM, 122
- Last-Event-ID, 264
- layer boundaries, 76
- layer choice, 413
- layer model, 47
- layer responsibility, 296, 416
- layered architecture, 2, 41, 67, 465
 - libraries, 370
 - motivation, 4
- legacy streams, 211
- legacy variants, 378
- lifecycle, 151
- lifecycle logging, 202
- lifetime, 46
- lifetime checks, 405
- line protocol, 137
- link lines, 336
- linked components, 34, 331
- linking strategy, 297, 367
- Linux deployment, 385
- `listen()`, 21, 91, 116, 125
 - parameterless, 177
- local boundary, 348
- local endpoint, 81
- local IPC, 113
- local section, 175, 185
- local-control role, 302
- `LOG()`, 197
- logging, 195
 - context level, 142
 - output modes, 199
 - responsibility boundaries, 200
- logging surface, 197
- lower carriers, 76, 157, 162
- lower-family transfer, 157
 - design rules, 162
- M2M communication, 2
- management surface
 - HTTP, 302
- mapping, 358
- MariaDB, 316
 - minimal client example, 319
- `MariaDBClient`, 318
- `MariaDBConnection`, 321
- `MariaDBConnectionDetails`, 318
- `MasterRequest`, 239
- meaning preservation, 421
- `Measurement`, 435
- measurement gateway, 432
- `MeasurementBus`, 437
- `MeasurementState`, 436
- `MeasurementUnixSocketContext`, 451
- `MeasurementUnixSocketContextFactory`, 454
- memory tools, 407
- mental model, 41
- message-oriented communication, 281
- `MessageEvent`, 261
- metrics, 49, 206
 - protocol information, 141
- middleware, 249, 253
- middleware callbacks, 250
- middleware extension, 425
- MIME, 240
- MiniGateway, 432, 470
 - application purpose, 432
 - base architecture, 434
 - base source package, 447
 - boundary testing, 460
 - build target, 434
 - current state, 436
 - debugging, 461
 - deployment, 448
 - deployment shape, 462
 - extended build and usage, 458
 - extended build target, 450
 - extended version, 448
 - extension, 448
 - extension lessons, 459
 - measurement model, 435
 - MQTT configuration, 438
 - MQTT context construction, 442
 - OpenWrt deployment, 462
 - process split, 462
 - runtime assembly, 443
 - runtime assembly after extension, 455
 - safe extension, 463
 - testing, 448
 - Unix-domain factory, 454
- `MiniGatewayMqtt`, 440
- `MiniGatewaySocketContextFactory`, 442
- minimal application, 17
- minimal builds, 399
- minimal context, 143
- mounted routers, 249
- MQTT, 281
 - client example, 287
 - client role, 286
 - component identity, 289

- control packets, 282
- extension, 426
- integration spine, 305
- layered model, 281
- MiniGateway protocol object, 440
- native, 282, 292
- native over streams, 285
- over WebSocket, 282, 291, 292
- protocol family, 282
- protocol object, 284
- public surface, 288, 300
- server role, 286
- sessions, 282
- topics, 282
- WebSocket subprotocol, 288
- Mqtt, 284
- MQTT applications, 355
- MQTT client role, 434
- MQTT components, 378
- MQTT over WebSocket, 278, 291
 - bridge boundary, 307
 - component selection, 297
 - diagnostics, 298
 - public surface, 300
 - upgrade stack, 291
- MQTT tests, 402
- MQTTBridge, 360
- MQTTBroker, 359
- MQTTCli, 362
- MqttContext, 285, 293
- MQTTIntegrator, 360
- MQTTStore, 362
- MQTTSuite, 355
 - architectural reading, 364
 - ecosystem shape, 355
 - shared infrastructure, 358
- multi-protocol applications, 2
- multi-protocol systems, 301
- multiplexer, 60
- multiplexer packages, 388
- named instances, 173
- named roles, 346
- names as architecture, 31, 48, 68
- namespace structure, 31
- namespaced targets, 375
- naming convention, 48
- native protocol, 415
- net, 34
- net::in, 104
- net::in6, 104
- net::in6::SocketAddress, 82, 105
- net::in6::stream::legacy, 111
- net::in::SocketAddress, 82, 105
- net::in::stream::legacy, 111
- net::l2::SocketAddress, 123, 124
- net::l2::stream::legacy, 130
- net::rc::SocketAddress, 123, 124
- net::rc::stream::legacy, 130
- net::un::SocketAddress, 83, 114
- net::un::stream::legacy, 120
- network families, 34, 47, 103, 157
- network family, 81
- network family selection, 414
- network frameworks, 2
- network layer, 67, 70
- network-facing boundary, 348
- network-family targets, 377
- new protocol behavior, 424
- Next, 250
- NO_RETRY, 230
- node.js, 6
- nonpersistent options, 191
- observability, 195, 419
- observation role, 302
- onConnect, 97, 202
- onConnected, 97, 202
- onDisconnect, 97, 202
- OpenWrt, 385
 - deployment, 393
 - deployment flow, 393
 - MQTTSuite deployment, 363
 - service integration, 395
- OpenWrt SDK, 393
- OpenWrt tests, 406
- operational artifacts, 192
- operational envelope, 183
- operational model, 49
- operator visibility, 419
- optional features, 380
- ordinary socket programming, 27
- out-of-tree build, 12
- output pressure, 232
- over-abstraction, 165, 429
- ownership, 151
- package builds, 399
- package dependencies, 387
- package recipes, 393
- parameterized factories, 153
- parameterless activation, 177, 190
- parser tests, 402
- path cleanup, 119
- path identity, 83, 113, 114
- path ownership, 119
- peer relationship, 43, 46, 72, 94
- per-connection state, 147
- permissions, 129
- persistence, 314
 - backpressure, 327
- persistence boundary, 315
- persistence role, 362
- persistence service design, 326
- persistent options, 176, 191
- pid files, 389
- ping frame, 271
- playground project, 14

- PLOG(), 197
- pong frame, 271
- precise language, 467
- preconfigured factories, 153, 164
- PRIVATE, 376
- procd, 395
- process boundaries, 416
- progressive disclosure, 190
- protocol behavior, 44, 46, 73, 133, 150
- protocol boundaries, 4, 348
- protocol code, 135
- protocol composition, 292, 296, 415
- protocol family selection, 303
- protocol flow, 44
- protocol meaning, 417, 468
- protocol reuse, 76, 157, 162
- protocol shape, 158
- protocol specialization, 165
- protocol stacks, 304
- protocol state, 139
- protocol surface, 414
- protocol tests, 402
- protocol timeout, 232
- protocol upgrade, 267
- PSM, 84
- PUBLIC, 376
- public headers, 14, 111, 120, 245, 367, 370
- public include paths, 68
- public includes, 35
- public surface, 373
- Publish, 287
- query(), 322
- read timeout, 223
- README-BUILD.md, 447
- ReadyState, 261
- real sockets, 405
- real-time HTTP, 256
- reconnect, 93, 221, 225, 405
- reconnect timer, 227
- reference ecosystem, 355
- registered instance, 88
- remote endpoint, 81
- remote section, 175, 185
- reproducibility, 407
- Request, 237, 239, 252
- required options, 184
- required values, 190
- Response, 237, 239, 252
- responsibility boundary, 139
- retry, 93, 221, 225, 405
- retry control, 230
- retry field, 264
- retry on fatal, 228
- retry policy, 228, 418
- retry timer, 227
- retry tries, 228
- reusable boundary, 423
- RFCOMM, 83, 122, 123
 - channel, 122
- role assembly, 455
- role boundaries, 416
- role constellations, 304, 345, 346
- role map, 355
- role-level ownership, 231
- roles, 7, 33, 46, 467
- rollback, 325
- Router, 249
- router extension, 425
- routes, 249
- routing, 246
- routing dispatch, 251
- RPATH, 379, 389, 406
- runtime, 43
- runtime composition, 379
- runtime control, 33
- runtime facade, 55
- runtime intent, 58
- runtime introspection, 195, 206
- runtime model, 42
- runtime output, 13
- runtime overrides, 388
- runtime phases, 56
- runtime roles, 443
- runtime state, 57, 317, 389
- runtime tests, 405
- runtime visibility, 195
- runtime-loaded modules, 387, 406
- runtime-visible instance, 90
- safe extension, 422, 463
- scientific data collection, 302
- section configuration, 185
- section hierarchy, 175
- section scope, 180
- secure communication, 210
- serializer tests, 402
- server application, 21
- server program, 13
- server role, 90
- server section, 185
- server type, 37
- Server-Sent Events, 256
- server/client role logic, 142
- server/client symmetry, 93
- server/client/context model, 103
- service boundaries, 416
- service managers, 389
- service operation, 389
- service split, 462
- service-level tests, 406
- severity, 199
- shared application state, 152
- shown configuration, 192
- shutdown, 221, 405
- SNI, 215
- SNode.C, 2

- cloning, 11
- mental model, 41
- source baseline, 4
- source reading, 55
- system reading, 353
- teaching context, 2
- snode.c**, 336
- socket abstractions, 33
- socket section, 185
- socket state, 98
- SocketAddress**, 80
- SocketClient**, 18, 23, 88, 92
 - retry and reconnect, 227
- SocketConnection**, 88, 94
 - context attachment, 146
- SocketContext**, 19, 37, 117, 133, 214, 285
 - extension, 424
 - lifecycle, 136
 - worked example, 137
- SocketContextFactory**, 18, 37, 99, 145
 - create, 146
 - extension, 424
- SocketContextUpgrade**, 270
- SocketServer**, 18, 21, 88, 91
- source baseline, 4
- source checkout, 9
- source paths, 35
- source tree, 15
- source-level reading, 55
- source-tree reading, 29
- src/apps**, 30, 330
 - study material, 330
- SSE, 256
 - Express routes, 265
 - HTTP request setup, 263
 - layered model, 257
 - long-lived HTTP, 257
 - observation surface, 306
 - parsing, 263
 - retry, 264
- SSE observation role, 434
- stable protocol core, 352
- stack diagnostics, 298
- start()**, 56
- startup boundary, 172
- startup flow, 44
- stateful roles, 350
- stateless roles, 350
- static serving, 253
- stop()**, 56
- stream sockets, 120
- stream transport, 71
- streaming endpoints, 258, 265
- SubProtocol**, 273, 425
- SubProtocolContext**, 273
- system boundaries, 301, 348, 466
- system design, 345
 - services, 346
- system operation, 350
- system reading workflow, 353
- systems thinking, 2
- target linking, 24
- termination timeout, 223
- testing, 397
 - extension, 429
- testmariadb**, 338
- testpipe**, 338
- testpost**, 338
- tick()**, 56
- TickStatus**, 57
- timeouts, 63, 221
 - protocol intent, 141
 - protocol level, 232
- timers, 62, 405
- TLS, 210
 - certificate handling, 213
 - configuration, 215
 - connection layer, 72, 210
 - deployment responsibility, 219
 - diagnostics, 218
 - handshake, 213
 - layer boundary, 214
 - protocol independence, 214
 - shutdown, 213
- TLS certificates, 392
- TLS connection, 212
- tls section, 175, 185, 215
- TLS streams, 211
- TLS variants, 378
- TLS wrapper, 212
- topic mapping, 360
- transactions, 325
- transport form, 47
- transport layer, 67, 71
- transport variants, 377
- transport vocabulary, 363
- type aliases, 32
- type naming, 104
- Unix domain socket input, 448
- Unix domain sockets, 83, 113
 - deployment, 119
 - local control, 308
 - MiniGateway input, 451
 - server/client use, 116
 - SNode.C model, 113
 - stable model, 117
- upgrade boundary, 270
- upgraded boundary, 348
- v1.0.2, 4
- virtual hosts, 253
- VLOG()**, 197
- web administration, 359
- web application framework, 246

- web assets, 392
- web protocols, 236
- `web::http`, 245
- `web::websocket`, 279
- WebApp, 248
 - public surface, 255
- WebAppT, 248
- WebSocket, 267
 - as carrier, 278
 - client-side upgrade, 272
 - interaction surface, 306
 - layered model, 267
 - module structure, 273
 - MQTT carrier, 291
 - public surface, 279
 - server-side upgrade, 272
 - subprotocol extension, 425
 - subprotocols, 273, 288, 293
 - upgrade boundary, 269
- WebSocket components, 378
- WebSocket frames, 271
- WebSocket messages, 271
- WebSocket tests, 402
- WebSocket upgrade, 242, 291
- wildcard address, 84
- wildcard form, 107
- write buffer, 232
- write timeout, 223